

Writing an Agent Harness in Rust

A build-along from the first token to a shipped agent



```
> cargo run  
  first token...  
  thinking...  
  action...  
  agent shipped ✓
```



Enzo Lombardi

Writing an Agent Harness in Rust

A build-along from the first token to a shipped agent

Enzo Lombardi

Copyright (c) 2026 - Enzo Lombardi
Last edited September the 24th 2026

TABLE OF CONTENTS

INTRODUCTION

A language model running on the machine under the desk behaves differently from the same kind of model behind a hosted API, and most of the difference is invisible until a coding agent starts using it. On a provider, a long conversation costs money and some latency, and the provider's cache quietly absorbs the rest. On a laptop, the same conversation costs minutes. Every time the agent sends the model its transcript, the local engine has to read that transcript again unless someone has arranged for it not to, and the someone is not the model. It is the program around it.

This book was born out of the need to understand that program. Plank, the Rust coding agent whose history supplies every example in these pages, started as a port of the C reference agent that ships with DwarfStar, a small native inference engine written in C and built first for the DeepSeek V4 Flash model, and it runs that model locally on Apple hardware through DwarfStar. The port was supposed to be a translation exercise. It became a long education in what a harness has to do when the model is local: how to keep the engine's memory of the conversation alive across turns, forks and rewrites, how to decide what to forget when the context fills, how to remember things across sessions without paying to re-read everything each time the memory changes, how to notice that the model is going around in circles, how to keep the prompt responsive while the model works, and how to let other people's tools and plugins into the loop without letting them take it over. None of those questions is answered by the model, and very few of them are answered by the documentation of any hosted API, because a hosted API hides the part of the problem where the answers live.

The chapters that follow are the answers plank arrived at, rebuilt from scratch in a small teaching codebase so that the reader can write each subsystem, test it and watch it fail in the ways plank's failed.

Harness, engine, turn, pass

Four words carry the whole book, and they are worth fixing before anything else.

The *engine* is whatever turns a sequence of tokens into more tokens. It might be a process on the same machine holding the weights in memory, or an HTTP endpoint on the other side of the world. From the outside it is a function from a prefix to a continuation, and the book treats it that way on purpose: everything the agent appears to do is done by code that calls that function.

The *harness* is that code. It owns the transcript, the tools, the display, the persistence and the loop that ties them together. It decides what text the model sees, reads the model's output for requests, carries those requests out, frames the results as text the model will understand, and decides when to call the engine again. When a coding agent is said to have read a file, fixed a test and explained the fix, the harness did the reading, the fixing and the running. The model produced text.

Introduction

A *turn* is one user prompt driven to a settled answer. A *pass* is one generation inside a turn. A turn that uses tools is several passes: the model asks for a file, the harness reads it and feeds it back, the model asks for an edit, and so on until a pass ends without a request. The distinction matters because most of what goes wrong in a harness goes wrong between passes, where the harness is in charge, rather than inside one, where the model is.

What a local model moves into the harness

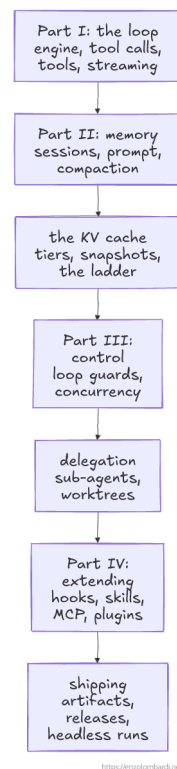
With a hosted model, many of the hard parts of running an agent belong to the provider. The provider decides how to cache the prompt, when a cached prefix can be reused, how many concurrent requests share one loaded model, and what happens when a machine runs out of memory. The harness sees a price and a latency and optimises against both.

With a local model, every one of those decisions comes home. The engine keeps a key-value cache of the conversation it has processed, and whether the next pass reads ten tokens or a hundred thousand depends entirely on whether the harness sends a prompt that extends what the engine already holds. Rewriting a single old tool result in place, the most natural thing a compaction scheme can do, invalidates everything after it. Changing one character of the system prompt invalidates everything. Snapshotting the cache to disk, restoring it, deciding which snapshots to keep and which to sweep, and proving that a snapshot on disk belongs to the prompt it claims to belong to, all become the harness's job.

The same shift reaches features that look unrelated to caching. Persistent memory is the clearest example. A harness that remembers facts about the user and the project across sessions has to put those facts somewhere in the prompt, and on a local engine where it puts them decides whether a memory update costs nothing or forces a full re-read of the conversation. Plank's answer, which chapter 6 and chapter 7 walk through, is that memory lives in a cached tier of the prompt beside the project's instruction file, and that a write lands on disk immediately but joins the model's context only at the next session start. That rule is not a product decision. It falls out of the cache.

The chapters on the loop, the tools and the display would read much the same for a harness built on a hosted API. The chapters on sessions, compaction, caching, loop guards and concurrency would not, and they are the reason the book exists. The following chart shows how the parts build on one another:

Introduction



The four parts of the book, each resting on the one before it

What you build

The code in this book is not plank. Plank is more than a hundred thousand lines of Rust shaped by a specific model, a specific engine and a specific user, and reading it cold teaches mostly how plank happens to be organised. The reader builds `blanka` instead: a small coding-agent harness in a Cargo workspace, one subsystem per chapter, with tests at every step and a tag at the end of each chapter so that the state of the code always matches the text.

Blanka talks to its model through an `Engine` trait from the first chapter. Two implementations are built along the way. An echo stub answers every call deterministically, so that every test in the book runs without a network, a GPU or a model download. An OpenAI-compatible HTTP client talks to anything that speaks that protocol, which includes Ollama, vLLM, OpenRouter and most hosted providers. The local-engine material, where the cache becomes the harness's problem, is taught against a mock engine that counts which tokens it reused and which it had to re-read, so that a cache miss is something a test can assert on. An optional second crate, `rllama`, wraps `llama.cpp` behind the same trait and runs Qwen3 8B in sixteen gigabytes of memory, for readers who want to see the numbers the mock predicts on real hardware.

What you need is Rust 1.85 or later, an OpenAI-compatible endpoint of any kind for the chapters that talk to a real model, and no GPU at all for the rest. Traits, ownership and error handling are assumed. Nothing about language model APIs is.

Introduction

Plank appears alongside blanka as a case study, in the third person. Each chapter opens on the problem plank actually hit, the session that went wrong, the measurement that surprised, the finding that had to be written down so nobody would rediscover it, and then builds the part of blanka that answers it. Where plank's own code is the clearest statement of a pattern, a short excerpt appears, labelled as plank and pinned to the commit that introduced it. Near the end of each chapter a section lists what plank does that blanka does not, which is both an honest account of what the book leaves out and a map of what a production harness accumulates around an idea that fits in a page.

Why the wire format is sacred

One lesson from the port shaped everything else, and it is worth stating before the first chapter. When plank was ported from the C reference, most of the code could be rewritten freely: the data structures, the concurrency, the error handling and the file layout all became idiomatic Rust. Three things could not change by a single byte. The syntax in which the model writes a tool call, the framing in which the harness hands a tool's output back, and the text of the system prompt are what the model was trained on. A harness that reformats them, however reasonably, is talking to the model in a dialect it half understands, and the model's behaviour degrades in ways that are hard to attribute. Plank enforces the rule with a test that compares its model-facing text against committed fixtures and, when the C source is present, against the reference itself.

The consequence for the reader is a distinction that runs through the book. Some text is for the model, and it is treated as data with a contract: pinned by fixtures, versioned, and changed only on purpose, because every change also moves the fingerprint of every cached prompt built on it. The rest of the harness is engineering, and it is as free as any other code.

A book that is kept alive

This book is published online first, and it will stay online only for a while, because the subject has not settled.

The clearest example arrived while the chapters were being drafted. A new class of model appeared that drops text generation for pure decision making: instead of asking a model for an answer and parsing whatever comes back, the harness arranges a question so that the only sensible next tokens are a handful of lettered options, runs one forward pass, and reads the probabilities of those letters directly from the output layer. No tokens are generated, and the answer carries an honest measure of how sure the model was. The vocabulary for this is borrowed from psychology: *System 1* is the fast, intuitive answer, *System 2* the deliberate one worked out step by step.

For a harness running a local model the idea is unusually attractive, because the expensive half of such a decision is prefilling the state the question is about, and a local harness has usually prefilled that state already for other reasons. The cache that the second part of the book works so hard to keep warm turns out to be exactly the substrate a decision layer wants. Plank adopted the technique within days. Its first consumer is persistent memory. At the end of a turn, plank can hand the model the new part of the conversation and ask for a set of memory changes, which costs a snapshot of the session, a prefill and a full generation. Most turns contain nothing worth remembering, so most

Introduction

of those passes return nothing. A System-1 question in front of the pass, one bit asking whether the turn produced anything durable at all, lets the expensive reading run only when it is likely to earn its keep. The same warm-cache trick, applied to ordinary short generation rather than to decisions, is what lets plank guess the next thing the user will type and show it as ghost text in the prompt at almost no cost, by generating off the session that is already hot instead of from scratch.

Both features are younger than most of this book. The memory gate works, and a single wrong token, scoring the letter A when the model wanted to write a space followed by A, was enough to make it decide nothing while every test stayed green. Its thresholds have not been calibrated against real outcomes, so it ships switched off. The suggestion pass went through half a dozen fixes in its first day, one of which saved a little token budget by turning the model's reasoning off and turned out to force a full re-read of the conversation on DeepSeek V4.1, the newer of the two DeepSeek models plank serves. Chapter 7 and chapter 11 tell those stories as they stand, and they will not stand still. Other candidates for the same primitive are already visible: a judge that decides whether a shell command reaches the network, and a compaction pass that asks of each old tool result only whether it is still needed.

Keeping the book online while this settles is the honest way to handle it. Chapters will be revised when plank's answers change, the case studies will be extended when a new finding replaces an old one, and the code will keep its chapter tags so that a revision of the text never silently breaks the build-along. The plank excerpts are pinned to commit hashes for the same reason: a reader can always see exactly which version of an idea a paragraph describes. When the subject stops moving under the text, the book will be frozen and printed.

How to read it

The chapters are meant to be read in order and coded along. The order is the argument: each part of the harness is introduced at the moment the previous parts make its absence painful, which is the order in which plank's problems appeared. A reader who only wants the ideas can skip the code and lose surprisingly little, because every code listing is preceded by the reasoning that produced it. A reader who only wants the code will find it in the workspace, tagged by chapter, with a test for every behaviour the text claims.

Some chapters are more about plank than others. The chapters on caching and on loop guards lean heavily on plank's record, because that is where the lessons were most expensive and least obvious, and because the failure modes they describe are the kind a reader's own harness will reproduce unless someone has warned them. The measurements in those chapters come with their biases stated: a corpus of recorded failures contains the failures a harness could see, and a detector tuned on two evenings of sessions is tuned on two evenings of sessions.

About the author

Enzo Lombardi has spent more than two decades building software systems at scale, and has the gray hair to show for it. He began at Microsoft in Redmond, where he spent twelve years as a Senior Software Developer and contributed to the first two releases of

Introduction

the .NET Framework, an experience that taught him how platforms are designed and that an API decision, like a tattoo, is far easier to add than to remove.

He then moved to Google in Kirkland, again as a Senior Software Developer, working on large distributed systems including the messaging backend for Google Hangouts and the directory services underneath Google Workspace. That work came with the concurrency and reliability problems Rust solves at the language level, and with enough on-call rotations to make him grateful for any tool that catches a bug before production does.

He is now a Software Architect at Cleafy, building security solutions for financial institutions, where performance requirements, security constraints and regulation together make a strong daily case for Rust's guarantees, and where "it works on my machine" does not count as a deployment strategy.

His open source work includes turbo-vision-4-rust, a Rust port of the classic Turbo Vision framework for those who consider the 1990s the high point of user interface design, and blink, a security-focused project released through Cleafy. Plank is the most recent entry in the same pattern: a piece of code that worked, and that he could not leave alone once there was a way to rewrite it in Rust.

He writes about software development at <https://enzolombardi.net>, on subjects that range from language design to distributed systems, drawing on decades of watching patterns succeed and fail at scale and on enough debugging to make anyone question reality. His code is at <https://github.com/aovestdipaperino>. Pull requests are welcome, and criticism is received with the calm of someone who has seen worse in code review.

Before the first token

The book starts from a plain observation: a model produces text, and everything a coding agent seems to do is done by the program around it. That program, the harness, becomes most interesting when the model is local, because the costs a provider hides come home, and features as different as compaction, persistent memory and prompt suggestions turn out to be questions about a cache. The reader builds blanka, a small harness with a test for every claim, while plank supplies the problems that forced each design. The model-facing text is treated as a contract and everything else as engineering. The book is kept alive online while the newest ideas in it, System-1 decisions above all, find their final shape, and it will be revised as plank's answers change. The first chapter begins where plank did: with a stub in place of the model, and a surprisingly complete agent around it.

CHAPTER 1: THE MODEL IS A FUNCTION

Watch a coding agent work for ten minutes and it is hard not to attribute the whole performance to the model. It reads files, forms a plan, edits code, runs the tests, notices a failure, fixes it, and reports back in complete sentences. The model gets the credit, and the model gets the blame when the session goes wrong. Yet the model did none of the reading, none of the editing and none of the running. It produced text. Every other verb in that description belongs to a program wrapped around it, and that program is what this book is about.

There is a quick way to see this for yourself, and it is how plank, the harness this book draws its examples from, came into being. Plank is a Rust port of `ds4_agent`, the C reference agent shipped with DwarfStar, the native inference engine that runs DeepSeek V4 Flash on consumer hardware. On the first day of the port the model was not available: the weights were an eighty-seven gigabyte download and the native engine had not been built. In its place sat a stub that answered every question with a note about how many bytes the conversation had grown to. The agent could nonetheless read a file, edit it, run a shell command and show the result, with a system prompt copied byte for byte from the original. Typing into it was a surprisingly complete experience, and it made something visible that a working model hides: everything a user would have called agent behaviour was being done by ordinary code. The model, when it arrived, produced text. The program around it decided what text to show the model, when to call it again, which fragments of its output were requests, how to carry those requests out, and what to remember.

The C reference calls into the model in one place. Plank, two months and a hundred and twenty thousand lines later, still does. The program around the model is the harness, and the harness is the agent. This chapter builds the smallest one that can honestly be called by that name. It is also where the book fixes its vocabulary, states the handful of rules that every later chapter obeys, and shows the first two of the many implementations of one small trait.

A function with a slot in the wall

The phrase "the model produces text" does more work than it appears to, so it is worth being precise about what it means.

A language model, observed from outside its own weights, is a function. Give it a sequence of tokens and it returns a probability distribution over what the next token might be. Pick one, append it, call the function again. Every paragraph a model has ever written was produced by that loop, one token per call, with nothing carried from one call to the next except the growing sequence itself. Inside the function there is a great deal of machinery, tokenization and embeddings and attention and the final projection onto the vocabulary, all of it deterministic given the weights and the input, none of it retaining anything between invocations. Outside the function there is the decision of what to feed in, the decision of when to stop, the interpretation of what came out, and the memory of anything at all.

Chapter 1: The model is a function

The caveat about caches belongs here, because it will matter enormously later. An inference engine does keep state between calls: the attention keys and values it computed for the tokens it has already seen, so that extending a sequence by one token costs one token's worth of work rather than the whole sequence's. That state is an optimization, not memory in any sense the model can use. It changes how fast the function runs, never what it returns. A harness that forgets this distinction ends up either trusting the cache as if it were the conversation, or throwing it away as if it were free, and plank's history contains examples of both.

Searle's Chinese room is a useful picture here, more useful as engineering than as the philosophical argument it was meant to be. Clerks in a warehouse follow rulebooks that turn incoming symbols into outgoing symbols, and no clerk understands the message. Whether the room as a whole understands is a question this book leaves open. What the picture offers is the slot in the wall. Symbols go in, symbols come out, and the person standing outside the slot decides what the symbols mean and what happens next. A coding agent is that person, equipped with a filing cabinet for the conversation so far, a terminal for carrying out instructions, and a list of rules about which strips of paper coming out of the slot count as instructions in the first place.

The picture also explains why so much of an agent's apparent intelligence is negotiable. The model was trained to produce text of a certain shape in response to text of a certain shape. Which shapes count as a tool request, how a tool's result is phrased when it goes back through the slot, what the model is told about the tools it has, whether it sees a summary of the last hour or a verbatim record: all of this is decided by the person at the slot, and all of it changes what comes out. Two harnesses around the same weights can behave like two different agents, and the difference between a session that finishes in fifteen minutes and one that runs for two hours is more often in the harness than in the model. Later chapters make that claim with numbers. For now it is the reason the book exists.

Four words follow from the picture, and the rest of the book uses them without variation. The **harness** is everything outside the slot. The **engine** is whatever is behind it, turning a token prefix into more tokens, whether that is a model in local memory, an HTTP endpoint, or a stub that plays back scripted answers. A **turn** is one user prompt driven to a settled answer, and a **pass** is one call to the engine inside a turn. In the harness this chapter builds, a turn is a single pass, because there is nothing yet for the harness to do between passes. That changes the moment tools arrive.

What the C reference had, and what the port kept

The C reference, `ds4_agent`, is a single file of thirteen thousand lines that calls directly into **DwarfStar**, the inference engine of the same project. It has no abstraction between the agent logic and the model, and it does not need one: there is one engine, it is linked in, and the author of both sides is the same team. It is also, and this matters for everything that follows, the program whose output the model was trained to expect. The system prompt in that file, the syntax it teaches the model for requesting a tool, the way it frames a tool's result when it hands it back: these are not arbitrary choices that a port is free to improve. They are the shape of the model's training data, and a port that changes them by a byte is talking to the model in a dialect it has never heard.

Chapter 1: The model is a function

That constraint shaped the porting method. Plank was not translated line by line, which would have produced C written in Rust and kept none of the benefits of the move. It was ported section by section: each functional block of the C, the tool table, the transcript, the compaction, the renderer, became a Rust module with its own tests, designed the way a Rust module should be designed, and then checked against the C where the C's behaviour was load-bearing. Where the behaviour was model-facing, the check is a byte-for-byte fixture. Where it was not, the port was free to do better. The line between the two is the most important line in the port, and drawing it correctly is a theme chapter 2 returns to at length.

The engine boundary was the first decision that this chapter turns on. Should the Rust call the engine directly as the C did, or put something between them.

Plank put something between them, and the reason was not foresight about future backends. It was the stub. To run the agent end to end before the model existed, the agent has to talk to the engine through a boundary a stub can also implement. The first commit's engine module opens with exactly that reasoning, and it is quoted here because it is the founding argument for the shape of everything that follows:

```
// plank src/engine.rs, commit 510af95
//! Inference engine abstraction.
//!
//! The C agent calls directly into the ds4 engine. Plank keeps that surface
//! behind a narrow trait so the UX layer works against any backend; a stub
//! echo engine makes the agent runnable end-to-end without a model.
```

The word "narrow" is doing deliberate work. A trait with thirty methods is a description of one implementation wearing a costume. A trait with three is a statement of what the harness actually needs from an engine, and it forces the harness to get by on that. Blanka's version has three methods, and one of them exists only so the harness can tell how full the context is.

There is a second, quieter benefit to the boundary, and it took plank weeks to notice it. Everything the harness sends to the engine passes through one function, and everything the engine sends back comes out of one function. When a session goes wrong, and sessions go wrong in ways that take hours to reproduce, the question "what exactly did the model see" has a single place to be answered. Plank's architecture notes eventually promoted this to a rule they call the log-everything invariant: anything that reaches a model request must be reconstructible from the session log. The rule is what makes replaying a session, forking it, rolling it back, or filing a reproducible bug report possible at all, and it is only enforceable because there is one door.

Before the trait, though, there is the thing the trait is fed.

Why Rust, and why not a framework

If you arrive at this book already convinced of Rust you can skip this section. It is here because the question comes up every time a harness in Rust is described, and because the answer says something about what a harness is.

Chapter 1: The model is a function

The obvious argument is the engine. The inference engine is C with a Metal backend, and it exposes a C API: load a model, create a session, feed tokens, sample a token, save and restore the cache. A harness written in a language whose foreign function interface costs nothing at the call site can drive that API directly, one call per token, and keep the whole loop in one process. A harness written in a language that reaches C through a marshalling layer pays for that layer on every one of the tens of thousands of calls a session makes, and usually ends up running the engine as a separate process and talking to it over a socket instead, which is a fine architecture with a different set of problems. Rust's foreign function interface is C's calling convention with a type signature on top, and the entire native engine binding in plank is a few hundred lines. The choice of language was, at the start, the choice to link the engine rather than to talk to it.

The less obvious argument is everything the rest of this book is about. A harness is, to a first approximation, a program that parses untrusted text under time pressure and then does something irreversible with the result. The model's output is untrusted in the precise sense that it is produced by a process the harness does not control and cannot fully predict; a tool request that looks almost right is the normal case, not the exception. The something irreversible is an edit to the user's files or a command on the user's shell. Between the two sits a parser, a dispatcher and a set of invariants about what state the transcript and the cache must be in for the next pass to make sense. Rust's enumerations with data, its exhaustive matching, and its insistence that every error be handled or explicitly propagated are exactly the tools for that middle layer. Porting the C showed this concretely: its most subtle bugs, the ones that had needed comments explaining which pointer was valid when, became either impossible to express or loud compile errors. That is not a claim that the Rust is bug-free. Plank's findings file is two and a half thousand lines of things that went wrong. It is a claim that the bugs moved from the category of silent state corruption to the category of visible wrong behaviour, and a visible wrong behaviour is one a user can report.

The third argument is threads. A harness that wants to stay responsive while the model thinks needs concurrency of a very simple shape: one thread blocks on the engine, another handles the keyboard, and they exchange messages. The model call takes seconds to minutes, the keyboard needs milliseconds, and nothing in between wants to be interleaved with anything else. This is the case for which operating system threads and channels were designed, and Rust's ownership rules make the message-passing version of it hard to get wrong: a value sent down a channel is gone from the sender, so the two threads cannot share the transcript by accident. Chapter 11 builds exactly this, and the whole of it is a few hundred lines with no runtime.

Why not a framework, then, since several exist for building agents in several languages. Partly because the port had a reference to match and a framework would have put its own opinions between the port and the reference. Mostly because the point of this book is the harness itself. A framework hides the loop, the tool dispatch, the context management and the retry behaviour behind an interface, and hiding them is a reasonable thing to do for someone who wants an agent and does not want to think about them. This book wants you to think about them, because every serious problem that surfaced in two months of running plank against real work turned out to live in one of those hidden places, and the fix required understanding what the harness was doing

Chapter 1: The model is a function

rather than what a framework said it was doing. If you finish this book and then pick up a framework, you will at least know what is inside the box. Readers who start with the framework often do not find out until something breaks.

None of this says a harness cannot be written well in another language. Several have been. It says that the problems a harness has are the problems Rust was built to make tractable, and that for a book about those problems, the language gets out of the way.

The filing cabinet

The harness's only memory is the transcript, and the transcript is a list of messages, each with a role and a body. That is the whole data structure, and the point of saying so plainly is what it implies about cost: since the engine remembers nothing between calls, the harness must hand it the entire conversation every pass. Not the latest line. All of it, from the system prompt onward, re-read from the beginning every time the model is asked for one more reply.

With the stub this is free. With a real model it is the single largest expense the harness incurs, and the second half of this book is largely about paying it less often. For now the cabinet is a vector:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum Role {
    System,
    User,
    Assistant,
    Tool,
}

#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct Message {
    pub role: Role,
    pub content: String,
}

#[derive(Debug, Clone, Default, PartialEq, Eq, Serialize, Deserialize)]
pub struct Transcript {
    messages: Vec<Message>,
}
```

The vector is private on purpose, and the purpose is not encapsulation for its own sake. Later chapters need to rewrite messages in place, truncate the list, and replace its head with a summary, and every one of those operations has consequences for a cache you have not met yet. A struct that hands out a mutable vector cannot enforce anything about them; a struct that hands out named operations can insist that each one leaves the cache consistent with the text. The type starts small so that the operations can be added one at a time, each with its rule attached.

Two decisions about the type are early on purpose, and they deserve a defence because they look like clutter. There are four roles, not three: the fourth, `Tool`, has no use until

Chapter 1: The model is a function

chapter 2, but a role enum is the kind of thing that gets matched exhaustively all over a crate, and adding a variant later means visiting every match. And the type can be serialized from the start, although nothing writes it to disk until chapter 5, because the derives cost nothing now and their absence would be an irritation then. Such things are best added while a type is still small enough to hold in your head.

The one method with a story attached is the flat rendering, which turns the message list into a single string with role markers:

```
pub fn render_flat(&self) -> String {
    let mut out = String::new();
    for m in &self.messages {
        out.push_str("<|");
        out.push_str(m.role.label());
        out.push_str(">\n");
        out.push_str(&m.content);
        out.push('\n');
    }
    out.push_str("<|assistant>\n");
    out
}
```

The closing assistant marker is the part people forget. A model completes text; it does not know it is in a conversation unless the text says so, and the way the text says "your line now" is an opening marker with nothing after it. Leave it off and the model is as likely to continue the user's sentence as to answer it.

The markers themselves are invented, and for the moment that is fine; a real local model has a template it was trained on, and the appendix that adds one uses it. What matters is the comment above this function in the source, which says the format is fixed on purpose. A local engine caches its work keyed on these exact bytes. Change a newline here and every cached conversation on disk stops matching, silently, and the engine re-reads everything from scratch while reporting nothing wrong. Plank's rule that model-facing text stays byte for byte what the reference produces grew out of that class of failure, and the habit of treating such text as frozen starts here, in the one function in this chapter where it costs nothing to form.

One trait, two shapes of prompt

The trait plank wrote on its first day took a flat string. That was the natural shape: the C took a flat string, the stub took a flat string, and the local engine about to be linked took a flat string. The trait was a day old when it turned out to be wrong.

The work on remote engines that began the next day describes two different things under one word. The first was hosting plank's own engine on another machine and talking to it over HTTP, which changes nothing about the prompt: a flat string goes over the wire instead of over a function call, and the machine on the other end tokenizes it exactly as the local engine would have. The second was driving somebody else's model through the OpenAI-compatible chat protocol that Ollama, vLLM, OpenRouter and most commercial providers speak, and that protocol does not take a flat string at all. It takes a

Chapter 1: The model is a function

JSON array of messages, each with a role and content, and applies the model's conversation template on the server side, out of the client's sight.

Flattening anyway, sending the rendered transcript as one user message, was considered, and the design notes record the rejection: the server would wrap the harness's role markers in its own role markers and the model would see a conversation quoted inside a conversation. Everything would still work, in the sense that text would come back, and it would work noticeably worse, in the sense that the model would be reasoning about a transcript someone had pasted to it rather than participating in one. The notes call this option acceptable as a stopgap and reject it as the long-term answer.

So the trait was widened, and the widening is worth describing because it is a pattern you will find yourself using whenever a boundary turns out to need a second shape. The design added an enum with two arms and left every existing implementation reading the arm it already understood. The local engines went on receiving flat text, and a test asserted, byte for byte, that they received exactly the text they had received before the change. Only the new provider read the structured arm. The change was additive, the old path was provably untouched, and the trait told the truth about what engines exist. Blanka adopts the two-armed shape from the start, which is the kind of thing a book can do and a port cannot:

```
pub enum Prompt<'a> {  
    Flat(&'a str),  
    Structured(&'a [Message]),  
}
```

Both arms borrow. The transcript owns the messages, the prompt is a view of them for the duration of a single pass, and the lifetime parameter is the compiler enforcing what is true anyway: a prompt is a momentary thing that must not outlive the conversation it looks into. Nothing is copied to call the engine, which for a flat prompt of a hundred thousand characters is not a trivial saving when it happens on every pass.

Three methods

With the prompt in hand, the trait itself is short enough to show whole.

```
pub trait Engine {  
    fn generate(  
        &mut self,  
        prompt: Prompt<'_>,  
        sink: &mut dyn Sink,  
    ) -> Result<(String, GenStats), EngineError>;  
  
    fn count_tokens(&self, text: &str) -> usize;  
  
    fn ctx_size(&self) -> usize;  
}
```

`generate` takes the prompt and a sink, streams text into the sink as it arrives, and returns the complete text together with the pass's accounting. Returning the text as well

Chapter 1: The model is a function

as streaming it looks redundant until the caller is written: the code that appends the reply to the transcript should not have to reassemble it from fragments, and the sink should not have to know it is being used for that. Plank's trait hands out a stream of typed events through a callback rather than a sink object, which is the same idea with more machinery, and the machinery arrives in chapter 4 when there is more than one kind of event to deliver.

The sink is the smallest trait in the crate, and its second implementation is the one the tests lean on:

```
pub trait Sink {
    fn text(&mut self, chunk: &str);
}

#[derive(Debug, Default)]
pub struct Collect(pub String);

impl Sink for Collect {
    fn text(&mut self, chunk: &str) {
        self.0.push_str(chunk);
    }
}
```

The sink is separate from the transcript for a reason that becomes a whole chapter later: the transcript receives the finished reply once, while the sink receives fragments as they come and, in time, has to recognize structure inside them. A tool request that the model emits mid-reply has to be noticed while it is still arriving, so the user can see that a command is about to run, and that noticing belongs to the sink side, not the transcript side. For now the only structure is text, and the only sink that matters prints it.

The other two methods exist because the harness has to know how full the context is. Compaction, the reminder that fires under pressure, and the loop guards all ask that question in later chapters, and they ask it before every pass, so the answer has to be cheap. A local engine answers exactly, from its tokenizer. A hosted one has no tokenizer to consult, and the honest options are to call a remote counting endpoint, which most providers do not offer and which would add a network round trip per pass, or to estimate. Blanka estimates at four bytes per token. The figure is a rule of thumb for English prose and code with a modern tokenizer, it is wrong by a factor of two or more for some scripts, and it is good enough for every decision that depends on it, because those decisions are about whether the context is roughly half full or roughly nine-tenths full, and an estimate that is off by twenty percent does not change the answer. The context size itself the harness has to be told; a hosted model's window is a fact about the deployment, not something the protocol reports.

What a pass costs

The accounting struct is where the chapter earns its place in the story, so it deserves a moment on why it has four fields rather than two.

Chapter 1: The model is a function

```
pub struct GenStats {  
    pub prompt_tokens: usize,  
    pub completion_tokens: usize,  
    pub prefill: Duration,  
    pub decode: Duration,  
}
```

Prefill is the time the engine spends reading the prompt before it produces its first token. Decode is the time it spends producing the rest. From the user's chair they are both waiting, which is why almost everyone measuring an agent for the first time collapses them into one number, and the collapse hides the only thing worth knowing about them: they grow differently. Decode is proportional to the length of the reply, which stays roughly constant over a conversation. Prefill is proportional to the length of the transcript, which only grows. Report them together and a slow session looks like a slow model. Report them apart and it usually turns out to be a harness re-reading forty thousand tokens to produce a hundred.

The two phases are also different kinds of work for the hardware, which is why the split is not merely bookkeeping. Prefill processes many tokens at once and is limited by arithmetic; it runs at hundreds or thousands of tokens per second on a machine that can decode at twenty-five. Decode processes one token at a time and is limited by how fast the weights can be read from memory; on a machine with an eighty-gigabyte model and a memory bus of a few hundred gigabytes a second, the arithmetic is simple and the ceiling is low. A harness that wants to be faster has two entirely different problems to solve depending on which number dominates, and it cannot know which without keeping them apart.

The C reference did not separate them. Plank does, at the end of every session, per model, and the exit report is where several of the book's later chapters came from. The measurement that motivated the most work was a single request against a real project, adding a three-second auto-dismiss option to a dialog box, which the original build turned into two hours of the model thinking in circles and editing nothing. Successive fixes brought the same request down to twenty-three minutes and then sixteen and a half, and at each stage the exit report said where the remaining time went. That story is chapter 10's. What matters here is that none of it could have been told without the two numbers being kept apart from the very first pass.

Errors round out the trait's vocabulary, and there are three kinds:

```
pub enum EngineError {  
    Transport(String),  
    Protocol(String),  
    Interrupted,  
}
```

Three, not one, because the harness does different things with each. A transport error is the network, the process or the file system failing, and the right response is to tell the user and let them retry; nothing about the conversation is wrong. A protocol error is the engine answering with something the harness could not read, which hosted providers

Chapter 1: The model is a function

manage more often than one would hope, since each extends the format a little differently; the right response is to show the offending payload, because the fix is almost always in the harness's parser and the payload is the bug report. An interruption is the caller asking the pass to stop, and it is an error only in the type-system sense; by chapter 4 it is the ordinary way a user cuts off a reply that has gone wrong, and the harness's job on seeing it is to make sure the transcript records exactly what the engine had produced up to that moment and nothing more.

A general-purpose error type, the kind that wraps anything and prints a chain of causes, would be the wrong tool here, because the harness genuinely branches on the variant. An error type the caller only ever prints can be opaque. An error type the caller matches on has to name its cases.

The stub, dignified

The scripted engine is, by frequency of execution, the most important code in the book. Every unit test from here to the last chapter drives the harness through it, and it deserves better than the afterthought it usually gets.

```
pub struct EchoEngine {
  script: VecDeque<String>,
  ctx: usize,
}

impl EchoEngine {
  pub fn scripted<I, S>(replies: I) -> Self
  where
    I: IntoIterator<Item = S>,
    S: Into<String>,
  {
    Self {
      script: replies.into_iter().map(Into::into).collect(),
      ctx: 32_768,
    }
  }
}
```

Given a script, it plays the replies back in order. When the script is exhausted it echoes the last user message with a prefix, so the output is recognizably synthetic and a test that only cares about the shape of the loop need not script anything at all. The context size is configurable because several later tests need a small window to force compaction without building a transcript of a hundred thousand words.

The reason a scripted stub is the right test double, rather than a mock that records calls or a fake that runs a tiny real model, is what the harness is being tested for. The harness's correctness is about what it does with the model's output: whether it recognizes a tool request, whether it feeds the result back in the right frame, whether it stops when it should. None of that depends on the output being intelligent. It depends on the output having a particular shape, and a script is the most direct way to produce an output of a

Chapter 1: The model is a function

particular shape. When chapter 2 needs to test that the loop runs exactly twice for a reply containing one tool call, the script is a tool call followed by a plain answer, and the test asserts on the pass count. A real model could not be made to do that reliably, and a mock that records calls would test the wrong thing.

There is one detail in how the stub delivers a reply that deserves attention, because it is where the stub stops being a convenience and starts being a test of the harness:

```
for piece in reply.as_bytes().chunks(7) {  
    sink.text(std::str::from_utf8(piece).unwrap_or(""));  
}
```

The reply goes to the sink in seven-byte pieces rather than all at once. A stub that delivered whole replies would let the harness get away with assuming it always receives whole words, or whole lines, or whole tags, and a real engine makes no such promise. Tokens are not words, and a streaming HTTP response is chunked by the network, not by meaning. Plank's stub is nastier still: the note it prints contains a crab, and the crab is positioned to straddle an eight-byte chunk boundary, so any code that treats a chunk as valid UTF-8 on its own breaks in the test suite rather than in front of a user. Blanka's version drops a piece that would split a character, which is gentler and slightly dishonest, and chapter 4 replaces it with a renderer that carries partial bytes across chunks properly. The principle survives the simplification: the stub should be at least as awkward as the thing it stands in for.

A test against it reads the way a conversation does:

```
#[test]  
fn plays_the_script_in_order_then_echoes() {  
    let mut e = EchoEngine::scripted(["one", "two"]);  
    let mut t = Transcript::new();  
    t.push(Role::User, "hi");  
    let mut sink = Collect::default();  
    let (a, _) = e.generate(Prompt::Structured(t.messages()), &mut sink).unwrap();  
    let (b, _) = e.generate(Prompt::Structured(t.messages()), &mut sink).unwrap();  
    let (c, _) = e.generate(Prompt::Structured(t.messages()), &mut sink).unwrap();  
    assert_eq!((a.as_str(), b.as_str(), c.as_str()), ("one", "two", "echo: hi"));  
    assert_eq!(sink.0, "onetwoecho: hi");  
}
```

The last assertion checks what was streamed, separately from what was returned. Later chapters lean on that distinction hard: a renderer that hides the model's reasoning from the user must still let the harness see it, and the only way to test that is to look at both sides.

Talking to a real one

The OpenAI-compatible engine is the first code in the book that leaves the process, and it is structured so that almost none of it needs to.

Chapter 1: The model is a function

The protocol is a POST to a completions endpoint with a JSON body naming the model and carrying the message array, plus a flag asking for a streamed response. The response is then a server-sent event stream: lines beginning with `data:`, each carrying a JSON chunk with a fragment of the reply, blank lines between events, and a closing `data: [DONE]`. If the client asks, one more chunk near the end reports token usage. It is a protocol designed for browsers to consume incrementally, and it has the virtues and the faults of that origin: trivially easy to read line by line, and with no framing beyond the blank line, so a fragment can end anywhere, including in the middle of a multi-byte character, and the client is expected to cope.

The request body is built from the prompt:

```
pub fn request_body(&self, prompt: Prompt<'_>) -> Value {
  let messages: Vec<Value> = match prompt {
    Prompt::Structured(msgs) => msgs.iter().map(to_wire).collect(),
    Prompt::Flat(text) => vec![json!({"role": "user", "content": text})],
  };
  json!({
    "model": self.model,
    "messages": messages,
    "stream": true,
    "stream_options": {"include_usage": true},
  })
}
```

The `Flat` arm is the rejected stopgap from the previous section, made visible and made harmless. A flat prompt sent to a chat API can only be wrapped as a single user message, which is the quoted-conversation problem, and the harness never does it deliberately: the agent hands this engine the structured arm. Blanka wraps it anyway rather than panicking, because the trait promises that every engine accepts both shapes, and an engine that refused one of them would be lying about that promise. A reasonable answer to an unreasonable input keeps the trait honest, and it also keeps the engine usable from a test or a script that has a string and nothing else.

The usage option deserves a sentence, because it is the kind of thing that varies by provider and that the harness should ask for anyway. Without it, most servers report nothing about tokens, and the harness falls back to its estimate. With it, a server that supports the option sends real counts and one that does not ignores the field. Asking costs nothing and sometimes gets an exact answer.

The reader of the stream is a free function over anything that implements `BufRead`, and this is the decision that keeps the tests offline:

```
pub fn read_sse<R: BufRead>(
  reader: R,
  sink: &mut dyn Sink,
) -> Result<(String, Option<(usize, usize)>), EngineError> {
  let mut text = String::new();
  let mut usage = None;
```

Chapter 1: The model is a function

```
for line in reader.lines() {
  let line = line.map_err(|e| EngineError::Transport(e.to_string()))?;
  let Some(payload) = line.strip_prefix("data:") else {
    continue;
  };
  let payload = payload.trim();
  if payload == "[DONE]" {
    break;
  }
  let chunk: Chunk = serde_json::from_str(payload)
    .map_err(|e| EngineError::Protocol(format!("{e}: {payload}")))?;
  for choice in &chunk.choices {
    if let Some(c) = &choice.delta.content {
      if !c.is_empty() {
        sink.text(c);
        text.push_str(c);
      }
    }
  }
  if let Some(u) = chunk.usage {
    usage = Some((u.prompt_tokens, u.completion_tokens));
  }
}
Ok((text, usage))
}
```

Every branch in that loop corresponds to something a real server does. Lines that do not begin with the data prefix are comments, blank separators, or event-type lines that some servers emit and some do not, and they are skipped rather than rejected because a client that breaks on an unfamiliar but harmless line is a client that breaks every time a provider updates. The done marker is checked before parsing, because it is not JSON. An unparseable payload is a protocol error carrying the payload itself, since when a provider changes its format the first thing anyone needs is to see what it actually sent. Empty content fragments are dropped before they reach the sink, because several servers send an empty first delta to establish the role, and a sink that flushes on every call would otherwise flush nothing.

The chunk types the payload is parsed into are deliberately permissive. Every field has a default, so a chunk missing the choices array, or a delta missing its content, deserializes to an empty value rather than failing. Strictness here would be a mistake: the harness only needs the content fragments and the usage numbers, and everything else a provider chooses to include is none of its business.

The point of the signature is that a recorded stream in a text file satisfies `BufRead` as well as a live socket does. The fixture in the repository is seven events written by hand in the shape a server sends, and the test checks the reassembled text, what reached the sink, and the usage numbers. A second test feeds the reader a line of broken JSON:

Chapter 1: The model is a function

```
#[test]
fn bad_json_is_a_protocol_error() {
    let bad = "data: {not json}\n";
    let err = read_sse(bad.as_bytes(), &mut Collect::default()).unwrap_err();
    assert!(matches!(err, EngineError::Protocol(_)));
}
```

Neither test opens a connection. This split, wire format on one side and transport on the other, is the single most useful habit for code that talks to model providers, because the wire format is where they all differ and the transport is where they are all the same. When a provider changes its format, the fix is a new fixture and a change to the reader, and the test suite proves the change without anyone needing credentials or a network. When a provider is unreachable, the transport error says so and no parser is involved.

The transport is a dozen lines of `ureq`, a synchronous HTTP client, and the choice of synchronous is deliberate. Plank's core has no async runtime. A turn runs on one thread and blocks on the engine, and when the interface later needs to stay live during a long reply, the turn gets a worker thread rather than the whole crate getting an executor. A harness makes one request at a time and waits for it; an async runtime would buy nothing and would cost every signature in the crate, because a single asynchronous function at the bottom of a call chain colours every function above it. When a hosted provider is slow, the right response is to show the user that it is slow, not to find something else to do. There are places in a harness where concurrency genuinely pays, and chapter 11 is about them, but they are about the user interface staying responsive, not about the engine call, and threads serve them better than tasks.

The only part of `generate` that is not plumbing is the clock:

```
let started = Instant::now();
// ... send the request ...
let first_byte = started.elapsed();
let (text, usage) = read_sse(reader, sink)?;
let total = started.elapsed();
```

Time to first byte stands in for prefill and the remainder for decode. It is an approximation, since the provider's own queueing and network latency are folded into the first number, but it preserves the property that matters: the first number grows with the conversation and the second does not. On a local engine the split is exact, because the engine reports it, and the appendix that adds one shows the two numbers side by side for the same conversation.

When the stream says it failed

The reader above has one weakness that plank found the expensive way, and it is worth stating before any reader of yours inherits it: it trusts that a stream which ended ended well.

The occasion was a second wire format. The newest reasoning models from one large provider refuse function tools on the chat completions endpoint at any reasoning effort. They ask for the effort to be set to none, and then reject none as an unsupported value,

Chapter 1: The model is a function

which leaves no request body that works. Their tools live on a different endpoint, the Responses API, whose stream is typed events rather than anonymous deltas: the system prompt travels as instructions, the transcript as input items, tool calls and their results as top-level items paired by a call identifier. Plank added it as a third provider family, a third translator that re-emits the native calls as the same canonical markup the rest of the harness already parses, so that nothing downstream of the translator learned a new shape. It also learned to recognise the refusal, which arrives before any output byte, switch the engine to the new endpoint and retry once, keeping the switch for the rest of the session. The trait boundary from earlier in this chapter is what made that a contained change.

A few days later a saved session on that endpoint showed four passes that ran between one and thirteen seconds, generated zero tokens at zero tokens per second, and ended with the stop reason "answer". The last of them was the last pass of the session: the user typed "finish", an empty reply came back, and the turn ended. No guard tripped and the error log had nothing in it for nearly two hours around them. The cause was three lines in the translator. The stream's failure event and its error event set the done flag and returned, with no message kept. The engine then took its ordinary success path, found no usage because the stream never reached its completion event, fell back to reporting zero output tokens, and returned a successful pass. A turn loop that receives a successful pass with no tool calls ends the turn. From the user's seat a failed request and a model that chose to say nothing were the same event.

The tell was in the dump's header, and it is a useful one to know. The last context size reported and the transcript's token count differed by three tokens. A server's count of input tokens would not track the harness's own tokenizer that closely; the fallback does, because the fallback is the harness's own count. A figure that agrees too well with a local estimate is a figure the server never sent.

Two rules come out of it, and both apply to the chat completions reader in this chapter as much as to the one plank fixed. An HTTP status cannot diagnose a streamed provider: the 200 is committed before the model runs, so a failure in mid-generation can only be reported inside the body, and a reader that drops the terminal frame is the last place it could have been caught. And a terminal frame that is not a success must not share a branch with one that is. Plank's translator handled the incomplete event, which means the output budget ran out or a filter stopped the reply, exactly as it handled the completed one, and so presented a truncation as a finished answer. The fix, in plank commit 0e4a972, carries the provider's message out as an engine error with a line in the error log, and lets a truncation keep its text and usage while announcing its reason as a notice. Neither event had a test before; both have one now, from a hand-written fixture of the kind described above.

Blanka's reader stops at the done marker and returns whatever arrived. If you point it at a provider whose stream can carry an error object, the error branch belongs in the loop, beside the done check, returning `EngineError::Protocol` with the provider's text, and the fixture that proves it is a few lines long.

Chapter 1: The model is a function

Typing into it

The complete example is the first day's agent minus the tools: a program that reads a line, runs a turn, prints the reply, and does it again. Its sink writes to standard output and flushes after every chunk, since a buffered stream defeats the purpose of streaming:

```
pub struct Stdout;

impl Sink for Stdout {
    fn text(&mut self, chunk: &str) {
        let mut out = io::stdout().lock();
        let _ = out.write_all(chunk.as_bytes());
        let _ = out.flush();
    }
}
```

The flush is the whole point of the type. Rust's standard output is line-buffered, and a model's fragments do not arrive on line boundaries, so without an explicit flush the user would see nothing until the first newline and then a burst. The write errors are ignored, which is a considered choice rather than laziness: if standard output is closed, the user has gone, and there is nobody to report the failure to.

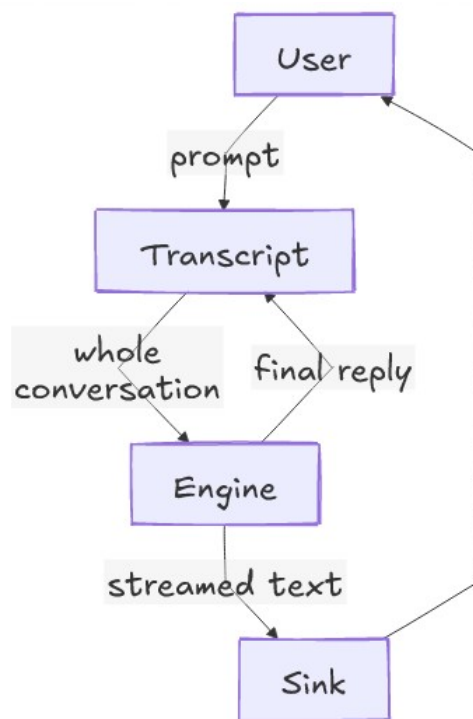
The turn is one call and two pushes:

```
transcript.push(Role::User, line);
let mut sink = Stdout;
match engine.generate(Prompt::Structured(transcript.messages()), &mut sink) {
    Ok((reply, stats)) => {
        transcript.push(Role::Assistant, reply);
        writeln!(
            out,
            "\n[{} prompt, {} completion tokens]",
            stats.prompt_tokens, stats.completion_tokens
        )?;
    }
    Err(e) => writeln!(out, "\nerror: {e}")?,
}
```

A failed pass prints the error and pushes nothing, so the transcript never records a reply that did not happen. That is a small rule with a long reach. Once transcripts are saved and replayed, a recorded reply that the engine never produced would make the replay diverge from the original, and once the engine's cache is keyed on the transcript, a phantom reply would poison every prefix after it. The rule is easy to keep here and expensive to retrofit.

The shape of the smallest harness, with nothing yet between the engine and the transcript, is this:

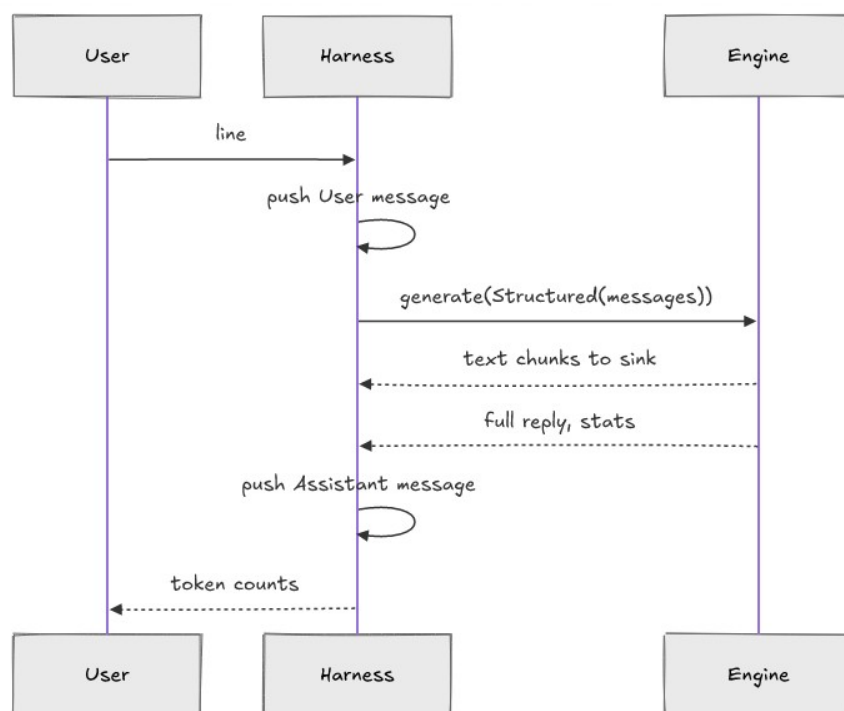
Chapter 1: The model is a function



<https://enzolombardi.net>

The minimum harness: a transcript, an engine, and a sink for the text as it streams.

And one turn through it, as a sequence, is this:



<https://enzolombardi.net>

One turn, one pass: the harness appends, calls, appends again.

Chapter 1: The model is a function

The binary picks its engine from the command line:

```
let mut engine: Box<dyn Engine> = match engine_name.as_str() {
    "echo" => Box::new(EchoEngine::new()),
    "openai" => Box::new(
        OpenAiEngine::new(url,
model).with_api_key(std::env::var("BLANKA_API_KEY").ok()),
    ),
    other => {
        eprintln!("unknown engine: {other}\n{USAGE}");
        return ExitCode::FAILURE;
    }
};
```

The API key comes from the environment and never from an argument, since arguments end up in shell history and process listings, and a coding agent is exactly the kind of program that will one day run a shell command that lists processes and show the output to a model. The default endpoint is a local Ollama, because that is the setup you are most likely to have and the one that needs no key at all.

This is the one place the engine is a trait object. Everywhere else in the crate it is a generic parameter, so the compiler can see through it and the call into `generate` is a direct call rather than a jump through a table. The choice is made from a string at startup, once, and a single dynamic dispatch there is the right price for not having every type in the crate carry an engine parameter it does not care about. Later chapters keep the same arrangement: the agent is generic over its engine, and the binary boxes one at the edge.

Run it with the echo engine and type a few lines, and watch the bracketed numbers after each reply. The second number stays put. The first climbs with every turn. That is the whole conversation going back through the slot each time. With the stub it costs nothing, which is exactly why the stub is the right place to first see it; with a real engine the same climbing number is the reason a long session's replies take longer and longer to begin, and the reason the second half of this book exists.

Pointing it at something real

The echo engine shows the shape. To see the cost, point the same binary at a model, and the cheapest way to do that is a local Ollama with a small instruction-tuned model pulled, which takes a few minutes and no account. The command is the same with the engine flag changed, and the first thing to notice is how little else changes: the transcript type, the turn, the sink and the accounting are identical, and the only code that knows a network exists is the dozen lines of transport. That is what the trait bought.

The second thing to notice is the bracketed numbers, and now they mean something. With a model behind the slot, the prompt count reported after each reply is the server's own count when the server reports one, and the first number in a long session grows in a way that can be felt: the pause before the first character of each reply lengthens, turn by turn, while the rate at which characters arrive once they start stays the same. That pause is prefill, and it is proportional to everything that has been said. A user who has not been told to look for it experiences it as the model getting slower, or getting tired,

Chapter 1: The model is a function

and both descriptions come from people who know better. Neither is true. The model is doing exactly the same work per token. The harness is handing it more tokens.

The third thing to notice is what a hosted model does with a transcript that has no system prompt. It behaves as a general assistant, because that is what the server's template tells it to be when the client says nothing, and it will happily discuss files it cannot see and propose commands it cannot run. Chapter 6 gives it instructions. Until then, the point of typing into it is not to get work done but to watch the numbers, and to notice that the program which does nothing but relay text is already the thing that decides how expensive each reply is.

There is also something to notice about what you cannot see. The server's template, the role markers it wraps the messages in, the exact tokenization of the transcript, whether the server cached anything from the previous request: all of this happens behind the slot, and the harness has no access to it. A local engine exposes all of it, and the appendix that adds one is where the two halves of this chapter's accounting become exact rather than estimated. Working against a hosted model is working with a partial view, and a good deal of what makes plank's local path faster than its hosted path is that the local path can see what it is doing.

What can be tested, and what cannot

The two engines in this chapter draw a line that the rest of the book keeps drawing, so it is worth being explicit about where it falls.

Everything on the harness side of the slot is deterministic, and everything deterministic can be tested. Given a transcript and a scripted reply, the loop does the same thing every time; given a recorded stream, the reader produces the same text; given a message list, the request body has the same shape. Blanka's test suite is built entirely on that side of the line. It runs in a fraction of a second, needs no network, no credentials and no model, and it grows with every chapter. When the book says that a piece of behaviour is guaranteed, it means there is a test on that side of the line that would fail if the guarantee broke.

Everything on the engine side is not deterministic in any useful sense, and the harness's tests do not pretend otherwise. Whether a particular model, given a particular prompt, emits a well-formed tool request, or emits one at all, or emits three when one was wanted, is a fact about the weights and the sampling, and it changes with the temperature, the quantization, the model version and sometimes the phase of the moon. A test that asserts on model output is a test that fails when the model is updated and passes when it happens to behave, and neither outcome tells the harness's author anything they can act on. The C parity fixtures that chapter 2 introduces sit on the harness side: they assert that the text the harness sends is byte-identical to the reference, not that the model does anything in particular with it.

That leaves a gap, and it is the gap that plank's repro dumps and the measurements in chapter 10 exist to fill. The question "does this harness make this model behave well" cannot be answered by a unit test. It is answered by running real requests against real projects, recording everything, and reading the recordings. The two-hour session mentioned earlier was diagnosed that way, and the fixes that brought it to sixteen

Chapter 1: The model is a function

minutes were each validated by running the same request again and reading the new recording. That is empirical work, it is slow, and it is the only kind of evidence that speaks to the model side of the slot. The harness's unit tests make sure that when a recording shows the model doing something odd, the oddness is the model's and not a bug in the loop, which is the precondition for the empirical work meaning anything.

The scripted engine is where the two kinds of evidence meet. When a recording shows the model emitting a tool request in a shape the parser rejects, the shape goes into a script and becomes a unit test, and the parser is fixed until the test passes. The recording is the source of the case, the script is what makes it repeatable, and the test is what keeps it fixed. Several of the parser's edge cases in chapter 2 arrived that way, and every one of the loop guards in chapter 10 did.

One more thing about the tests is worth saying now, because it is a choice you will face when porting this design to a harness of your own. The fixtures are hand-written, not recorded from a live server. A recorded fixture captures a particular provider on a particular day, and the seven events in the file are meant to document the protocol's shape rather than one server's behaviour. When a provider does something the hand-written fixture did not anticipate, the right move is to add a second fixture with the observed payload and a comment saying where it came from, so the suite accumulates real cases without losing the canonical one.

Five rules that fell out of the first weeks

Plank's architecture notes open with a short list of design principles, and they belong here because every one of them is already visible in this chapter's code, and every later chapter is an application of at least one.

The first is the narrow engine surface: all inference sits behind the trait, and the agent, the interface and the tools never touch model internals. The stub is what enforces this in practice. Any code that reaches past the trait to something only the real engine has cannot run in a test, and a codebase where most of the tests cannot run is a codebase where the rule erodes.

The second is sink-agnostic rendering: model output flows through a pipeline whose final destination is swappable. Here the destination is standard output and the pipeline is a single method, but the separation is already there, and it is what lets a full-screen terminal interface arrive later as a second sink rather than a rewrite.

The third is the preservation of model-facing formats. The tool output framing, the tool-call syntax and the system prompt are reproduced verbatim from the reference, because that is what the model was trained on. The flat rendering in this chapter is the first piece of model-facing text in blanka, and its comment about being fixed on purpose is this principle in miniature.

The fourth is correctness before cleverness, which in the notes is stated about the cache: reuse only a genuinely matching token prefix, and rebuild a stale checkpoint rather than trust it. It appears in this chapter as the refusal to record a reply that did not happen, and it becomes the organizing principle of chapters 8 and 9.

Chapter 1: The model is a function

The fifth is the log-everything invariant already mentioned: anything that reaches a model request must be reconstructible from the session log. The narrow trait is what makes this enforceable, and the transcript being the only memory is what makes it true by construction, for now. Later chapters add text at request-assembly time, and each of them has to say how that text gets logged.

None of these was written down on plank's first day. They were extracted, weeks later, from the decisions that had turned out well, and the extraction is itself a lesson: the rules a codebase actually follows are the ones its early structure makes easy to follow. The trait, the stub and the private transcript vector are that structure.

What the first version did not have

Anyone who has used a coding agent will see what is missing. There is no system prompt, so the model has no instructions. There is no tool, so the model can describe an edit but not make one. Closing the program loses the conversation. A long reply cannot be interrupted, raw text is all that is rendered, and nothing stops the model from repeating itself until the context fills.

Each of those is a chapter, and each attaches to something defined here. Tools attach to the loop around `generate`, when a reply turns out to contain a request rather than an answer. The system prompt attaches to the transcript, as text that sits in front of the messages and changes at a different rate from them, and so does persistence. Interruption and rendering attach to the sink. The loop guards attach to the pass. The engine trait grows methods for cache reuse in chapter 8 and nothing else about it changes.

The order in which those chapters arrive is not arbitrary, and it is not the order a textbook would choose. A textbook would build the layers bottom up: engine, then protocol, then tools, then state. This book follows the order in which the problems appeared in plank, because each problem is the motivation for the next piece of machinery, and machinery introduced before its problem is machinery you have no reason to want. Tools come first because the first version of the agent was useless without them. Streaming comes next because a tool request has to be recognized while it arrives. Sessions come after that because a conversation that vanished on exit was the first thing users of the port complained about. The context and cache chapters come in the middle because the cost they address only became the dominant one after the earlier problems were solved, and the loop guards come late because the loops they catch only became visible once sessions were long enough to loop in. The order is the argument, and whenever you want to know why a piece of the harness exists, the answer is in the chapter before it.

This chapter's harness is deliberately small because it is the baseline every later addition must justify itself against. The first version worked, in its way. Everything after it is an argument about what "worked" ought to mean.

The parts that were always there

A language model is a function from a token prefix to a distribution over the next token, and everything an agent appears to do is done by the code around that function. That

Chapter 1: The model is a function

code is the harness; the engine is whatever produces tokens; a turn is one prompt driven to a settled answer; a pass is one call to the engine.

The smallest harness is a transcript, an engine behind a narrow trait, and a sink for streamed text. The transcript is the only memory, and the engine re-reads all of it every pass. A prompt has two shapes, flat text for local engines and a message list for hosted ones, and plank learned within a day of its first commit that the trait must accept both.

The scripted `EchoEngine` streams in awkward pieces on purpose and is what makes the rest of the book testable. The HTTP engine keeps its wire format, tested from a fixture, apart from its transport, a dozen synchronous lines. Prefill and decode are accounted separately because they grow differently, and the first of them is the story of the book's second half. Five principles that plank wrote down weeks later are already present in the shape of this chapter's code.

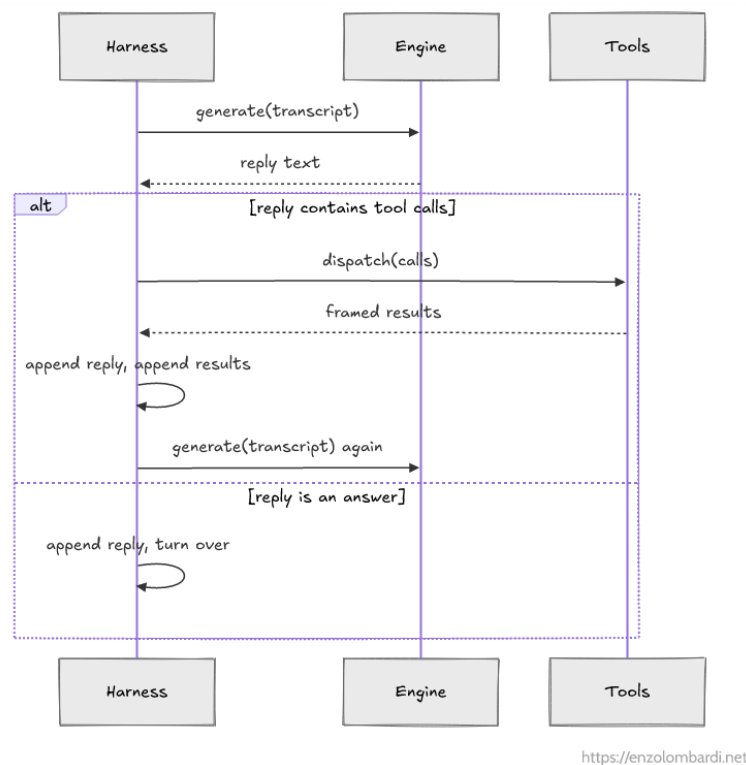
CHAPTER 2: TOOL CALLS ARE TEXT

The harness as it stands can hold a conversation and nothing else. Ask it to fix a bug and the model will describe, often quite well, the edit it would make, and then stop, because there is no way for its description to become an action. The gap between an assistant that talks about files and an agent that changes them is the subject of this chapter, and the gap is narrower than it looks. The model still produces text. What changes is that the harness agrees, in advance, to read certain shapes of text as requests, to carry those requests out, and to tell the model what happened.

That agreement is a protocol, and like every protocol it has two sides that must match exactly. The model learned its side during training: given a description of the tools available and a task, it emits a request in a particular syntax, then expects a result in a particular frame. The harness's side is written by hand, and if it differs from what the model learned by so much as a bracket, the model is talking to a stranger. Plank spent a significant fraction of its first weeks discovering where that boundary runs, and the discoveries are what this chapter is built around.

The turn grows a loop

Once a reply can contain a request, a turn is no longer a single pass. The harness generates, reads the reply, and if the reply asks for something, the harness does it, appends the result to the transcript, and generates again. The model now sees its own request and the result underneath it, and either asks for something else or answers. The turn ends when a pass produces a reply with no request in it. The loop is this:



A turn with tools: generate, dispatch, feed back, until a pass contains no request.

Chapter 2: Tool calls are text

Everything interesting about the loop is in the two places where text crosses the boundary. Reading the reply for requests is a parsing problem, and how strict the parser is decides what the harness will and will not do on the model's behalf. Feeding the results back is a formatting problem, and the format is model-facing: it is part of what the model was trained to expect after a request, and it cannot be changed casually. The loop itself, once those two are right, is a dozen lines.

The vocabulary from chapter 1 needs one more word. A **tool round** is one dispatch of a batch of calls and the feeding back of their results. A turn with three tool rounds has four passes: three that asked for something and one that answered.

Two families of syntax

There are two ways a model can tell a harness it wants a tool, and every harness that talks to more than one provider ends up handling both.

The first is in-band. The model emits, as part of its ordinary text output, a stanza in a syntax it was trained to produce, and the harness scans the text for it. The DeepSeek models plank was built for use a markup the reference calls DSML, with tags such as `< | DSML | tool_calls>` and `< | DSML | invoke name="read">` around parameter blocks. The Qwen family uses a `<tool_call>` tag around a body, and the body's shape has changed across Qwen generations: Qwen3 writes a JSON object with a `name` and an `arguments` member, while the Qwen3.8 model that plank also supports writes nested `<function=...>` and `<parameter=...>` tags instead. In every case the request is text, mixed with whatever prose the model wrote around it, and the harness has to find it.

The second is native. The hosted chat protocol that Ollama, vLLM and the commercial providers speak has a field for tool calls: the client sends a list of tool definitions with the request, and the server returns any calls the model made as structured JSON, separate from the text content, with the name in one field and the arguments as a JSON string in another. The model behind the server is still emitting an in-band stanza in whatever dialect it was trained on. The server parses it and hands the client the result, so the client never sees the dialect.

The two families look like they need two code paths, and a first design usually gives them two. Plank did not, and the reasoning is worth following because it recurs throughout the book. Plank's provider engine takes the native calls a server returns and synthesizes from them the DSML stanza a local model would have written, byte for byte, then hands the whole thing to the same parser the local path uses. The design document calls this making the provider "produce the same executable calls a local model would emit". The consequence is that there is one parser, one set of parser tests, one dispatcher, and one place where a malformed request is turned into an error message, and none of them knows or cares where the request came from.

Blanka does the same, with one substitution. The teaching dialect is the Qwen3 shape, a `<tool_call>` tag wrapping a JSON object, because it is what the book's reference local

Chapter 2: Tool calls are text

model emits and because JSON is a body the reader already knows how to reason about. Where plank's excerpts show DSML, the shape is different and the argument is the same.

How the model learns what it can ask for

Before a model can request a tool it has to know the tool exists, what it is called, what arguments it takes and, less evidently, when to prefer it over the alternatives. That knowledge reaches the model as text, and the two families of syntax deliver it in two ways.

For an in-band model the tool descriptions are part of the system prompt. The reference calls this the tools prompt: a block of text that names each tool, explains its arguments, shows the exact stanza syntax with an example, and lays down rules about usage. In plank the block runs to several kilobytes inside a system prompt of about five thousand tokens, and it is the largest single piece of parity surface in the harness, reproduced from the C byte for byte because the model was trained with exactly that text in front of every conversation. Chapter 6 assembles the system prompt; what matters here is that the descriptions are model-facing text in the same sense the result frame is, and that they sit at the very front of every pass, so their cost is paid on every prefill until chapter 8 arranges for the engine to remember them.

For a native provider the descriptions travel as a JSON registry in the request, one entry per tool with a name, a description and a parameter schema, and the server renders them into whatever text its model expects. The harness never sees that rendering, which is convenient and also means it cannot check it. Blanka's tool trait exposes a description in the registry shape for this reason, and the hosted engine forwards the list unchanged.

The less obvious part is that descriptions steer behaviour, not only syntax. A model given a shell tool described as "run a shell command" and a search tool described in a sentence will use the shell for searching, because it has seen far more shell commands in training than it has seen any harness's search tool. Plank counted this across thirty saved sessions: the model reached for grep through the shell a hundred and twenty-five times and for the purpose-built search tool forty-five, and prefixed most of its shell commands with a change of directory that the shell did not need. The fix was not code. It was two short sections of text, naming the wrong shell command beside the right native tool, added to the descriptions the model reads. Chapter 3 returns to that finding, because it is the tool chapter's clearest lesson. Here it establishes the point: everything in this chapter's protocol, the syntax, the frame, and what the model chooses to request, is downstream of text the harness wrote.

What the model actually writes

A reply from a model that has decided to read a file, in the Qwen3 dialect, looks like this:

I will start by reading the entry point.

```
<tool_call>
{"name": "read", "arguments": {"path": "src/main.rs"}}
</tool_call>
```


Chapter 2: Tool calls are text

Three properties of that text drive the parser's design. The stanza is embedded in prose, so the parser has to find it rather than expect the whole reply to be a request. The body is a complete JSON object, so a stanza is either well-formed or it is not; there is no partial credit. And the closing tag may never arrive, because the model can run out of tokens, be interrupted, or lose its thread mid-stanza, and an open stanza with no close is a different situation from a closed stanza with a bad body.

Those three properties become the parser's output type:

```
pub enum ParseOutcome {
    None,
    Calls(Vec<ToolCall>),
    Incomplete { raw: String },
    Malformed { reason: String, raw: String },
}
```

`None` means the reply is an answer and the turn is over. `Calls` carries every well-formed stanza in the order the model wrote them, because a model will often batch several reads into one reply and the harness should run them as a batch. `Incomplete` and `Malformed` are the two ways a stanza can be wrong, kept apart because the harness answers them differently and because the model needs to be told different things.

The parsed request itself keeps three things:

```
pub struct ToolCall {
    pub name: String,
    pub args: Value,
    pub raw: String,
}
```

The raw stanza, exactly as the model wrote it, is kept alongside the parsed name and arguments. It is not needed to execute the call. It is needed the first time a call executes in a way nobody expected and the question is what the model actually said, as opposed to what the parser understood it to say. Plank's parser keeps the raw bytes for the same reason, and its repro dumps, which chapter 5 introduces, print them. Diagnostics that reconstruct the model's output from the parsed form are diagnostics that hide parser bugs.

Strict on purpose

The parser is deliberately unforgiving, and the reason deserves more than a sentence, because the temptation to be forgiving is strong and the arguments for it sound reasonable.

A model's output is noisy. It will sometimes write a stanza with a trailing comma in the JSON, or quote a number, or put the arguments at the top level instead of under an `arguments` key, or misspell the tag. A forgiving parser that accepts these near-misses gets the tool call executed and the turn moving, where a strict parser stops to report an error. Forgiving looks like better behaviour, and in the short run it is.

Chapter 2: Tool calls are text

In the long run it is a mistake, for two reasons. The first is that a forgiving parser executes things the model did not mean. A parser that accepts arguments at the top level will, on the day the model writes a JSON object that happens to contain a `path` key for some other reason, read a file the model never asked to read, and the harness will then feed that file's contents back as the result of a request that was never made. The harness has now put text in front of the model that the model has no explanation for, and the model will do its best to make sense of it, which is how sessions go strange in ways that take hours to reconstruct. A strict parser cannot do this. It either executes exactly what was written or it executes nothing.

The second reason is that leniency in the executable parser hides the information the harness needs to fix the actual problem. If the model is writing malformed stanzas, the fix is in the prompt, or the tool descriptions, or the model choice, and the way to find out is to see the malformed stanzas and count them. A parser that quietly repairs them makes the problem invisible until it produces a repair that is wrong. Plank's parser documentation states the rule in one line: the parser stays strict, so the actual tool parser stays small and predictable, and tolerance for the model's near-misses lives elsewhere.

Where it lives is instructive. Plank does tolerate one class of near-miss: a model that drops a leading bar from a DSML tag, or that spells the marker as SSML instead of DSML, gets its stanza recognized. But the tolerance is in the streaming detector, the component from chapter 4 that watches the text as it arrives and decides whether a stanza has started, and not in the parser that produces executable calls. The reason the misspelling exists at all is a lesson in how the model sees text: the `| DSML |` marker is a dedicated token in the model's vocabulary, but plank composes its tools prompt as an ordinary system message, so the marker reaches the model as ordinary byte-pair pieces and the model spells it back out, and the far more common pretraining string "SSML" occasionally wins the first letter. Without the alias the stanza parses as nothing, prints as prose, and the turn ends with no error for the model to recover from. With it, the stanza is recognized, and the prompt tells the model that SSML is unsupported so the alias stays a recovery path rather than a second syntax.

Blanka's parser has no such aliases yet, because the book has not reached the point where a real model's near-misses have been observed and counted. When it does, the tolerance goes in the same place plank's does, and the executable parser stays as it is.

A parameter that vanished

Strictness has a second edge, and plank found it in a parser that was lenient by accident rather than by design. The model occasionally writes the next parameter's opening tag with a leading slash, so that one tag closes a parameter and opens another at the same time. In DSML, where each parameter sits between an opening tag carrying a name attribute and a bare closing tag, the fused form looks like this:

```
< | DSML | parameter name="query" string="true">impl Rng</ | DSML | parameter  
name="path" string="true">src/words.rs</ | DSML | parameter | >
```

Chapter 2: Tool calls are text

Plank's parser accepted only the bare closing tag as a terminator, which is correct, and so it read the attribute-bearing one as ordinary value text, which was not. The first parameter's value ran on to the final close. The dispatched search had a query equal to the whole run of markup and no path argument at all. The parameter did not arrive wrong; it disappeared.

What made this expensive is that nothing looked broken from the model's side. The search answered "No matches", which was the right answer for that query, and the model concluded that the search tool ignored the directory it had been given. It spent two turns investigating a bug that did not exist before falling back to grep through the shell. A parse error costs one re-emit. A plausible wrong answer costs the turn, and can end up recorded as a defect in the wrong component. Since plank commit 99b5c45 the fused tag is a parse error whose message names both tags the model has to write instead.

Two details of the fix are the kind that only a streaming parser has to get right. An attribute means an equals sign before the closing angle bracket, and not merely any byte after the tag name, because a parameter value can legitimately contain a close tag followed by a bare word, and a test built from adversarial payloads already required that shape to stay value text. And the candidate has to be held, undecided, until the deciding byte arrives, because the scan never revisits a position it has passed. Holding needs a bound, or a value that opens a close tag and never finishes it would pin the scan cursor and make every later byte rescan from there. Blanka's whole-reply parser sees the finished text and needs neither detail, which is one more argument for keeping the executable parser away from partial input.

The alternative that removes the problem, and why it is not taken

A reader with local inference experience will know that there is a way to make malformed stanzas impossible rather than merely detected. Local engines can constrain sampling with a grammar: at each step the sampler is allowed to choose only among tokens that keep the output inside a specified formal language, so a model told to emit a tool call as JSON of a given schema cannot emit anything else. The technique is mature, it is cheap at inference time, and it is the right answer for a class of problems.

It is not the answer for a coding agent's tool calls, for three reasons that are worth stating because they are about what a harness is rather than about the technique.

The first is that the grammar has to know when to apply. A model's reply is prose with a stanza somewhere in it, and the stanza may come after a paragraph of reasoning and before a sentence of summary. A grammar constraining the whole reply would forbid the prose; a grammar that switches on when a stanza opens is the streaming detector from chapter 4 with an extra step, and the detector is needed anyway. The constraint would save the parser's error path and nothing else.

The second is that it is unavailable where it would matter most. Hosted providers expose no grammar hook, and the native tool-call field is their answer to the same problem: the server constrains or validates on the client's behalf. A harness that leaned on grammars for its local path would have a different set of failure modes on its hosted path, and the whole point of the adaptation described below is that the two paths converge before the loop sees them.

Chapter 2: Tool calls are text

The third is the one that decides it. A malformed stanza is information. It says the model did not fully understand the tool description, or was confused by something in the transcript, or is being run at a temperature where it should not be. A grammar that forces well-formed output discards that information and replaces it with a call that is syntactically valid and semantically whatever the sampler could reach, which is a worse outcome than an error the model can read and correct. The strict parser turns a model's confusion into a message to the model, and the model usually resolves it in one pass. Constrained sampling turns it into a plausible-looking request that may execute.

Plank does not constrain sampling for tool calls, and its parsers are the strictness described above. The engine's sampler is left to sample, and the harness reads what came out.

The parser

With the outcome type fixed, the parser is a loop over the text looking for opening tags:

```
pub fn parse_qwen(text: &str) -> ParseOutcome {
    let mut calls = Vec::new();
    let mut rest = text;
    while let Some(start) = rest.find(OPEN) {
        let after_open = &rest[start + OPEN.len()..];
        let Some(end) = after_open.find(CLOSE) else {
            return ParseOutcome::Incomplete {
                raw: rest[start..].to_string(),
            };
        };
        let body = &after_open[..end];
        let raw = rest[start..start + OPEN.len() + end + CLOSE.len()].to_string();
        match parse_body(body) {
            Ok((name, args)) => calls.push(ToolCall { name, args, raw }),
            Err(reason) => return ParseOutcome::Malformed { reason, raw },
        }
        rest = &after_open[end + CLOSE.len()..];
    }
    if calls.is_empty() {
        ParseOutcome::None
    } else {
        ParseOutcome::Calls(calls)
    }
}
```

Two decisions in that loop are not obvious from reading it. An open tag with no close returns `Incomplete` immediately, discarding any well-formed calls found before it. This is deliberate: a reply that was cut off mid-stanza is a reply the model did not finish, and executing the first half of what it wanted while ignoring the second half is a good way to leave the project in a state the model does not expect. Better to tell the model the reply was incomplete and let it try again from a known position. And a malformed body

Chapter 2: Tool calls are text

returns `Malformed` for the whole reply, for the same reason: a batch is executed entirely or not at all.

The body parser is where the strictness is enforced:

```
fn parse_body(body: &str) -> Result<(String, Value), String> {
  let value: Value =
    serde_json::from_str(body.trim()).map_err(|e| format!("body is not JSON: {e}"))?;
  let Value::Object(map) = value else {
    return Err("body is not a JSON object".to_string());
  };
  let name = match map.get("name") {
    Some(Value::String(s)) if !s.is_empty() => s.clone(),
    Some(_) => return Err("name is not a string".to_string()),
    None => return Err("missing name".to_string()),
  };
  let args = match map.get("arguments") {
    Some(v @ Value::Object(_)) => v.clone(),
    Some(_) => return Err("arguments is not an object".to_string()),
    None => Value::Object(Default::default()),
  };
  Ok((name, args))
}
```

Every rejection carries a reason, and the reasons are written to be read by the model, not by a programmer. "Missing name" and "arguments is not an object" are the kind of phrase a model can act on. A serde error message alone is not, which is why the JSON failure is wrapped in a sentence before the serde detail is appended.

The one leniency in the body parser is that a missing `arguments` member is treated as an empty object rather than an error. Tools with no parameters exist, and a model that omits an empty object for one of them has not made a mistake worth stopping for. This is the shape of a defensible leniency: it accepts an input whose meaning is unambiguous, and it never causes a call to execute with arguments the model did not write.

The tests for the parser follow the outcome type: a single call from a fixture, two calls with prose around them, prose alone yielding `None`, an unclosed stanza yielding `Incomplete` rather than `Malformed`, and broken JSON and a missing name each yielding `Malformed` with the expected reason. One more test renders a call back into the syntax and parses it again, asserting that the round trip is the identity, because the rendering is about to be used for something important.

Whole replies now, streams later

The parser above reads a finished reply. The model has stopped, the text is complete, and the parser scans it from the top. That is the simplest place to parse and it is where blanka parses for the rest of this chapter, but it is not where plank parses, and the difference is worth understanding before chapter 4 changes it.

Chapter 2: Tool calls are text

Plank's parsers are incremental. They are fed bytes as the engine produces them, they keep a small state machine that knows whether a stanza is open, which tag it is inside, and how many bytes of a tag prefix have matched so far, and they can report at any moment whether a call is complete, pending or broken. The C reference does the same with one parser structure shared across its three dialects, and plank's port keeps that shape: the DSML and Qwen parsers share a state enum and a call type and differ only in the tags they look for.

The reason for incremental parsing is not speed. It is the user. A model that has decided to run a shell command emits the stanza over a second or two, and during that time the user is watching text arrive. If the harness knows the stanza has started, it can stop printing the model's markup as if it were prose, show a banner saying which tool is about to run, and hide the raw tags. If it does not know until the reply ends, the user sees a screenful of angle brackets and JSON, and then a result. Plank's users saw the second version for exactly as long as it took to write the first, which was not long.

Incremental parsing also changes what "incomplete" means. In a whole-reply parser, an open stanza with no close is a finished reply that was cut off, and the harness reports it. In a streaming parser, an open stanza is the normal state for the second or two while the model is writing it, and it is only incomplete if the stream ends first. Plank's finding about the parser that started in the wrong state is a finding about exactly this distinction: a parser that began in the "stanza is open" state reported every plain-prose reply as an unfinished call, because from its point of view the stanza had been open since the first byte and had never closed. The fix was a one-line change to the initial state. The lesson is that a streaming parser's states have meanings to the components that read them, and an unread state is a state with a surprising meaning waiting to be discovered.

Blanka separates the two concerns rather than merging them as plank does. The executable parser in this chapter reads whole replies and stays as simple as the code above. Chapter 4 adds a streaming detector whose only job is to notice that a stanza has opened, so the renderer can react, and which hands nothing to the dispatcher. The two agree on the tag strings and nothing else. This is more code than plank's single incremental parser and it is easier to reason about, because the component that decides what executes never has a partial view of anything. Whether the trade is worth it in a production harness is a fair question; in a book, it lets each chapter carry one idea.

Adapting native calls to the same shape

The hosted engine from chapter 1 needs to learn about tools, and it needs to learn in two places: the request and the response.

On the request side, the provider protocol accepts a `tools` array, each entry a function description with a name, a description and a JSON schema for the parameters. When it is present, the server instructs the model about the tools in whatever way the model expects, and returns the model's calls in the structured field instead of as text. Blanka's `OpenAiEngine` gains a `builder` method that takes the tool descriptions and, when the list is not empty, adds them to the body wrapped in the provider's `{"type": "function", "function": {...}}` envelope. The tool table, described below, produces the descriptions; the engine only carries them.

Chapter 2: Tool calls are text

On the response side, native calls arrive as fragments. The stream delivers the call's name once and its arguments as a JSON string spread across many deltas, each tagged with an index so that several simultaneous calls can be reassembled. The stream reader from chapter 1 grows a second accumulator alongside the text:

```
for tc in &choice.delta.tool_calls {
  if calls.len() <= tc.index {
    calls.resize(tc.index + 1, NativeToolCall::default());
  }
  if let Some(n) = &tc.function.name {
    calls[tc.index].name.push_str(n);
  }
  if let Some(a) = &tc.function.arguments {
    calls[tc.index].arguments.push_str(a);
  }
}
```

When the stream ends, the reassembled calls are rendered into the in-band syntax and appended to the reply text, so that what the agent receives from a hosted engine is indistinguishable from what it would have received from a local model that wrote the stanza itself:

```
pub fn synthesize_stanzas(calls: &[NativeToolCall]) -> String {
  let mut out = String::new();
  for c in calls {
    let args: Value = serde_json::from_str(c.arguments.trim())
      .ok()
      .filter(Value::is_object)
      .unwrap_or_else(|| json!({}));
    if !out.is_empty() {
      out.push('\n');
    }
    out.push_str(&render_qwen(&c.name, &args));
  }
  out
}
```

An argument string the provider streamed but that does not parse as a JSON object becomes an empty object rather than dropping the call. This looks like a leniency of the kind the previous section argued against, and the distinction is worth drawing. The parser's strictness is about not executing what the model did not write. Here the model did write a call, the server has already parsed and validated its shape, and what arrived is the provider's serialization of the arguments, which is occasionally cut short by the stream. Passing an empty object through means the tool runs, finds its argument missing, and reports that in words the model can act on. Dropping the call would mean the model's request silently vanished, which is the outcome the whole design exists to prevent.

Chapter 2: Tool calls are text

The result is that the provider engine's test for tool calls is a fixture of four stream events, reassembled into one call, rendered to a stanza, and parsed by the same function the local path uses, with the test asserting on the parsed arguments. The adaptation is tested end to end without a network, and the agent's loop has exactly one input shape to handle.

There is one piece of the native protocol the adaptation deliberately drops, and it will come back. Each native call carries an id, and the provider protocol expects the tool's result to be sent back in a message that names that id, so the server can pair results with calls. Blanka's hosted engine discards the ids and sends results as ordinary tool-role messages with the frame text inside, which every server the book was tested against accepts but which is not what the protocol specifies. Plank's design document records the same shortcut and the same reason: pairing by id requires the harness to keep a side map from its own numbered frames to the provider's ids across the pass boundary, and until the harness needs to talk to a server that enforces the pairing, the map is complexity without a customer. When such a server appears, the id rides alongside the raw stanza in the parsed call, and the frame gains a field. The design leaves room for it and does not build it.

Names and schemas are prompt too

The tool trait asks each tool for a name and a description, and both deserve more thought than they usually get, because both are read by the model and both change what it does.

Names first. The reference calls its tools `read`, `write`, `edit`, `bash`, `glob` and `search`, and the shortness is not laziness. A tool name appears in every request the model writes and in every result frame it reads back, and a name of one token costs less than a name of three on every occurrence. More important, a name the model has seen in training as a verb for the action is a name it will reach for without being told. A model that has read a million shell sessions knows what `read` and `edit` mean before it has seen the harness's description; a tool called `file_content_retriever` has to be learned from the description alone, and the description competes with everything else in the prompt for the model's attention. Blanka keeps the reference's names for that reason, and any tool you add should be named the way its action would be named in a sentence.

Schemas next. The description a tool exposes includes a JSON schema for its arguments, and for a hosted provider the schema is what the server uses to teach the model the argument shape and, on some servers, to validate what the model produced. For a local model the schema is rendered into the tools prompt as text. In both cases the schema is a contract about what the parser will receive, and the chapter's strictness about the body's shape stops at the schema's edge: the parser checks that `arguments` is an object and hands the object to the tool, and the tool checks that the object has what it needs. That division is deliberate. The parser knows nothing about individual tools and should not; a tool knows its own arguments and is the right place to report that one is missing or the wrong type. The test-only tool in this chapter does exactly that, returning an error that says which argument is required, and the error travels back in the frame like any other.

Chapter 2: Tool calls are text

The pattern that follows from this is that adding a tool touches nothing in the loop. A new capability is a type with three methods, a name the model will recognise, a schema that says what it takes, and a call that reports its own failures in words. Register it, and the model can ask for it on the next pass. Chapter 3 adds six such types and spends its length on what each of them gets wrong.

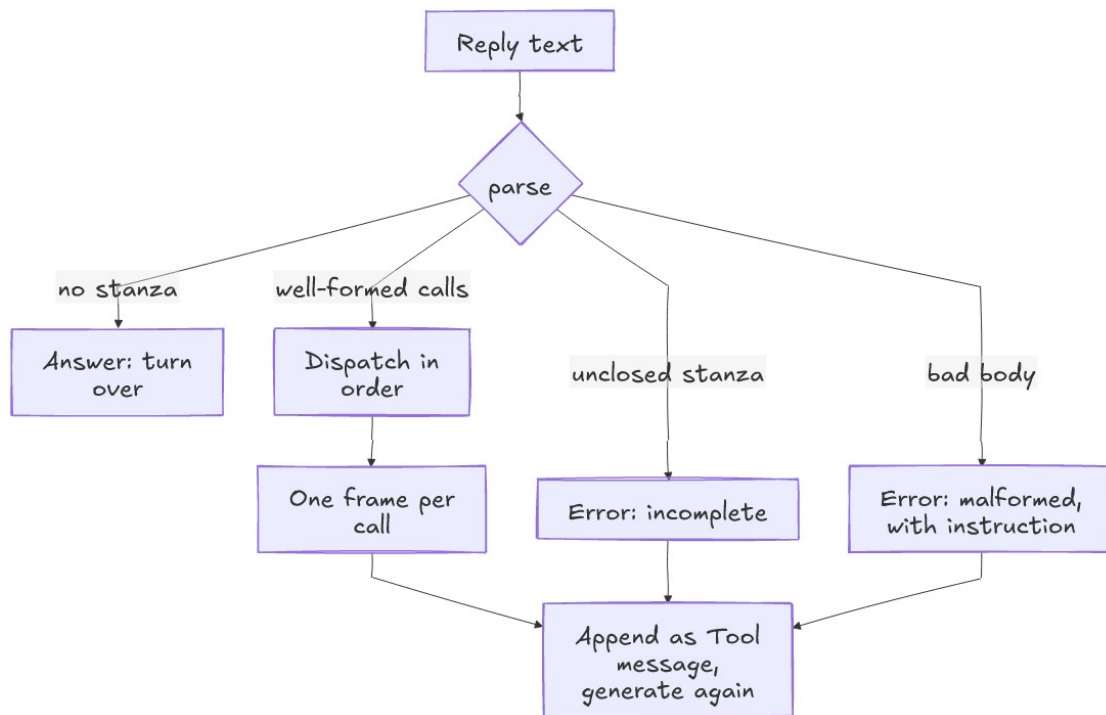
Batches, order and side effects

A model that has decided it needs three files will often ask for all three in one reply, and the dispatcher above runs them in the order written and returns three numbered results. Two design questions hide in that sentence: whether the calls in a batch should run concurrently, and what happens when one of them fails.

Concurrency looks attractive and is not worth it. The calls in a batch are almost always reads, which are fast, or a read followed by an edit that depends on it, which cannot be reordered. Plank's measurements on real sessions found that batching paid off only during exploration, where a model issued three or four reads per reply and the reads were independent; once the model moved to editing, it issued one edit per reply, because each edit's success determines the next. Running the reads of an exploration batch in parallel would save milliseconds on a pass that costs seconds of prefill. Running edits in parallel would risk two edits to the same file racing. Sequential dispatch in written order is the simple choice and it is also the correct one.

Failure inside a batch is handled by the frame, not the loop. Call two of three fails; its frame carries the error; calls one and three carry their results; the model sees all three and decides. This matches what a person would want from a colleague who was asked to do three things: report on each, not stop after the first problem. The alternative, aborting the batch at the first failure, would hide the results of calls that already ran and cannot be un-run, which is worse than reporting them.

The decision tree for a completed reply, then, is this:



<https://enzolombardi.net>

What happens to a completed reply: one of four outcomes, three of which continue the turn.

Three of the four outcomes feed text back and generate again. Only the first ends the turn. That asymmetry is the loop's whole character: the harness is biased toward giving the model another chance, and the guards that bound how many chances arrive in chapter 10 with the evidence to size them.

The tool table

Before the loop can dispatch a call, it needs something to dispatch to. A tool is one capability the model may request, and blanka describes it with a small trait:

```
pub trait Tool {  
    fn name(&self) -> &str;  
    fn describe(&self) -> Value;  
    fn call(&self, args: &Value, ctx: &mut ToolContext) -> Result<String, ToolError>;  
}
```

The name is what the model writes in its request. The description is a JSON fragment in the provider registry shape, a name, a description and a parameter schema, which the hosted engine forwards and the system prompt of chapter 6 renders into text for local models. The call takes the parsed arguments and a context, and returns either output text or an error.

The context is the harness's way of giving tools what they need without giving them the whole agent. For now it holds a working directory and nothing else. In chapter 3 it grows the write-containment roots and the spill directory, and later chapters add hooks and a

Chapter 2: Tool calls are text

sandbox profile. The principle is that a tool sees a struct designed for tools, not a reference to the agent, so a tool cannot reach into the transcript or call the engine, and the set of things a tool can do is exactly the set of fields on the context.

The table is a map from name to boxed tool, and its one interesting method is the dispatcher:

```
pub fn dispatch_all(&self, calls: &[ToolCall], ctx: &mut ToolContext) -> String {
    if calls.is_empty() {
        return frame_error("empty tool call block");
    }
    let mut out = String::new();
    for (i, call) in calls.iter().enumerate() {
        let output = match self.get(&call.name) {
            Some(tool) => match tool.call(&call.args, ctx) {
                Ok(s) => s,
                Err(e) => frame_error(&e.O),
            },
            None => frame_error(&format!("unknown tool: {}", call.name)),
        };
        out.push_str(&frame_result(i + 1, &call.name, &output));
    }
    out
}
```

Three behaviours here are copied from the reference and each has a reason. An empty batch produces a single error line rather than nothing, because a reply that opened and closed a tool-call block with nothing inside is a reply the model needs to be told about. An unknown tool name is an error inside a numbered result frame, not a rejection of the batch, because the model asked for three things and should learn that the second one does not exist while still receiving the first and third. And a tool's own failure is rendered into the same error form and placed in the same frame, so that from the model's side there is exactly one way a request can go wrong: it gets a result whose first line begins with the error prefix.

The whole batch produces a single string, which the loop appends as one message. The reference stores tool results as user-role turns and recognizes them on replay by their prefix; blanka uses the dedicated `Tool` role instead, which is what the role was reserved for in chapter 1, and the hosted engine maps it onto the provider's tool role on the wire. The difference is representational. What both agree on, and what the model was trained on, is the text inside.

The reference's choice of the user role for tool results is not an accident of C, and it is worth a paragraph because it shows how much of a wire protocol is convention layered on a model's training. A model trained on conversations with exactly two speakers has no native notion of a third speaker called "tool". What it has is training data in which, after it wrote a request, the next user turn began with a recognizable header and contained a result. The role is user because there was no other role; the header is what tells the model this user turn is a result and not a question. Providers that expose a

Chapter 2: Tool calls are text

distinct tool role are doing the same thing behind their template: the role is rendered into whatever text the model was trained to see, which for many models is a user turn with a header. Blanka's use of the dedicated role is therefore a representation choice inside the harness, with the frame text carrying the meaning, and the hosted engine's mapping of the role onto the provider's tool role is a courtesy to the server, not to the model.

The frame is model-facing

The text that goes back to the model after a tool runs is the most easily overlooked piece of model-facing text in a harness, and the one that plank's parity tests guard most carefully. It is produced by two small functions:

```
pub fn frame_result(n: usize, name: &str, output: &str) -> String {
    let name = if name.is_empty() { "unknown" } else { name };
    let mut s = format!("Tool result {n} ({name}):\n");
    s.push_str(output);
    if !output.is_empty() && !output.ends_with('\n') {
        s.push('\n');
    }
    s
}

pub fn frame_error(reason: &str) -> String {
    format!("Tool error: {reason}\n")
}
```

Every detail of the first function is a rule from the reference. Results are numbered from one. The tool's name is in parentheses, and a call with no name is labelled `unknown` rather than left blank. A trailing newline is appended only when the output is not empty and does not already end with one, so an empty result produces a header and nothing else, and a result that ends in a newline does not gain a blank line. None of these rules matters individually. Together they are the exact bytes the model saw after every tool call in its training data, and a harness that produces different bytes is showing the model a frame it has never seen.

Plank enforces this with a committed fixture and a test that compares the framing output against it byte for byte. Blanka does the same: a fixture holds the framed output for three calls, one with content, one empty and nameless, one whose output already ends in a newline, and the test asserts equality with the fixture. The point of a fixture rather than inline assertions is that it can be regenerated and reviewed as a diff when the framing is meant to change, and that any accidental change shows up as a failure that names the exact byte.

The error function is different in kind, and the difference is the subject of one of plank's more expensive findings. The result frame is parity surface: it reproduces training data and must not change. The error text is not. The reference has its own error strings, but the model was not trained on the specific wording of any of them, only on the convention that a failure begins with the error prefix. Plank spent a session discovering this the hard way. A tool that could not run because a vision encoder was not loaded produced the

Chapter 2: Tool calls are text

reference's exact refusal text, which mentioned a command-line flag that existed in the C program and not in plank. The model read the refusal as an instruction, tried to run the C program's command from the shell, got an unknown-option error, searched plank's help for the flag, and concluded the feature was broken. The fix was to write plank's own refusal, saying what was actually true about plank, and to record the principle: parity covers what the model was trained on, and error strings are the harness's own words, to be written for the harness that exists.

The principle has a corollary that shapes every error message in this book. Error text is not read by a programmer looking at a log. It is read by a model deciding what to do next, and the model will do whatever the text most plausibly suggests. An error that says "invalid tool call" tells the model something went wrong and nothing about how to proceed. An error that says "invalid tool call: missing name. Emit one JSON object with name and arguments inside the tags" tells it exactly what to do, and a model will usually do it. Blanka's malformed-call error is written that way for that reason.

The loop itself

With parsing, dispatch and framing in place, the turn loop is short:

```
pub fn run_turn(&mut self, prompt: &str, sink: &mut dyn Sink) -> Result<TurnReport,
AgentError> {
    self.transcript.push(Role::User, prompt);
    let mut report = TurnReport::default();
    loop {
        let (reply, stats) = self
            .engine
            .generate(Prompt::Structured(self.transcript.messages()), sink)?;
        report.passes += 1;
        report.stats = accumulate(report.stats, stats);
        self.transcript.push(Role::Assistant, reply.clone());

        let feedback = match parse_qwen(&reply) {
            ParseOutcome::None => break,
            ParseOutcome::Calls(calls) => self.tools.dispatch_all(&calls, &mut self.ctx),
            ParseOutcome::Incomplete { .. } => {
                frame_error("incomplete tool call: the block was opened but never closed")
            }
            ParseOutcome::Malformed { reason, .. } => frame_error(&format!(
                "invalid tool call: {reason}. Emit one JSON object with \"name\" and
                \"arguments\" inside <tool_call> and </tool_call>"
            )),
        };
        report.tool_rounds += 1;
        self.transcript.push(Role::Tool, feedback);
    }
    Ok(report)
}
```

Chapter 2: Tool calls are text

The reply is pushed onto the transcript before it is parsed, whatever it contains. This is the log-everything invariant from chapter 1 at work: the model's output is recorded as the model produced it, including a malformed stanza, so that a replay of the transcript reproduces exactly what the model saw next, which was its own broken request followed by the error about it. A harness that recorded only successful requests would be replaying a conversation that never happened.

The two error outcomes each produce feedback and continue the loop rather than ending the turn. This is the right default and it has a limit. At temperature zero a model that produced a malformed stanza, given the same transcript plus an error line, will often produce the same malformed stanza again, and the loop will run until the context fills. Plank met this failure when it added the Qwen dialect: a parser that started in the wrong state reported a phantom incomplete call on every plain-prose reply, fed an error back to a model that had done nothing wrong, and the model answered, was told again, and looped. The immediate fix was the parser's starting state. The general fix, a cap on how many times the harness will feed back the same error before ending the turn, is one of the loop guards in chapter 10, and it is deferred there because it needs the evidence from real sessions to be sized correctly. For this chapter the loop trusts the model to recover, and the tests confirm that when it does, the turn proceeds.

The accounting from chapter 1 is summed across passes, so a turn's report says how many tokens the whole turn cost rather than only its last pass. A turn with four passes over a growing transcript re-reads the transcript four times, and the summed prompt count is where that shows.

The test that pins the loop's shape is the one chapter 1 promised:

```
#[test]
fn a_call_then_an_answer_is_two_passes_and_one_tool_round() {
    let mut a = agent(vec![
        render_qwen("shout", &json!({"text": "hi"})),
        "It said HI.".to_string(),
    ]);
    let report = a.run_turn("try the tool", &mut Collect::default()).unwrap();
    assert_eq!((report.passes, report.tool_rounds), (2, 1));
    let msgs = a.transcript.messages();
    assert_eq!(msgs[2].role, Role::Tool);
    assert_eq!(msgs[2].content, "Tool result 1 (shout):\nHI\n");
    assert_eq!(msgs[3].content, "It said HI.");
}
```

The scripted engine plays a tool call and then an answer; a test-only tool upper-cases its argument; the assertions check the pass count, the tool round count, the exact framed text that went back, and the final answer. Three more tests cover an unknown tool producing an error inside its frame, a malformed stanza producing an instruction and the turn continuing, and a plain answer ending the turn in one pass. All four run against the stub in a fraction of a millisecond, and between them they specify the loop completely.

What a tool round costs

Chapter 1 ended on a climbing number, and the loop makes it climb faster. Every tool round appends the model's request and the tool's result to the transcript, and the next pass re-reads both, along with everything before them. A turn with four passes over a transcript of thirty thousand tokens reads a hundred and twenty thousand tokens of prompt to produce perhaps five hundred tokens of output. The summed accounting in the turn report makes this visible, and it is the number a user should look at when a turn feels slow, because the model's per-token speed has not changed. The harness is asking it to read more.

Tool results are the part of that growth the harness controls. A file read that returns ten thousand lines puts ten thousand lines into every subsequent pass of the turn, and into every pass of every later turn until something removes them. Two later chapters are responses to exactly this. Chapter 3 caps the size of any single result and spills the rest to disk with a locator, so one large file cannot fill the context by itself. Chapter 7 rewrites old tool results in place once the model has moved past them, so a read from twenty rounds ago does not keep costing its full length. Neither is needed to make the loop correct. Both are needed to make it affordable, and the loop's accounting is how the need shows up.

The other cost of a tool round is time that is not the model's. A shell command that runs a test suite can take a minute, during which the model is idle and the user is watching a cursor. Plank's exit report separates this too: it reports the session's time in tools apart from its time in prefill and decode, and in a session dominated by builds the tool time is often the largest of the three. A harness that shows the user which tool is running, and for how long, is answering a question that the model's speed has nothing to do with. Chapter 4's renderer takes that up, and chapter 11's background jobs let a long-running command finish after the model has stopped waiting for it.

Scripts as specifications

The tests in this chapter deserve a closer look than the one they got, because their shape is the shape every later test in the book takes, and the shape is unusual enough to be worth defending.

Each test is a script: a list of replies for the stub engine to play, in order, followed by assertions about what the harness did with them. The test that checks the loop scripts a tool call and then an answer. The test that checks malformed handling scripts a broken stanza and then a clean reply, and asserts that the harness fed an instruction back and then accepted the recovery. The test for an unknown tool scripts a call to a name that is not registered. In every case the script is the specification of a model behaviour, and the assertions are the specification of the harness's response to it.

What makes this work is that the stub is indifferent to the transcript. It does not read what the harness fed back; it plays the next line of its script regardless. That sounds like a weakness and is the point. The harness's job is to respond correctly to whatever the model produces, and the script lets the test author produce any sequence of model outputs at all, including sequences a real model would rarely produce, and check the response to each. A test that scripted a malformed stanza followed by the same

Chapter 2: Tool calls are text

malformed stanza ten times would specify the runaway loop that chapter 10 guards against, and the guard's tests are written exactly that way.

The scripts are also the bridge from field evidence to unit test that chapter 1 described. When a session recording shows the model writing a stanza the parser rejected, the stanza is copied into a script, the expected handling is written as assertions, and the parser or the error text is adjusted until the test passes. The recording becomes reproducible, the fix becomes permanent, and the script stays as documentation of a shape the model has actually produced. Several of plank's parser tests carry a comment naming the session they came from, and the pattern is worth copying: a test whose input is a real model output is worth more than one whose input was imagined, because it encodes a fact about the model rather than a guess.

One caution about scripted tests is that they cannot tell the author when the script is unrealistic. A test that scripts a perfectly formed tool call in a shape the model never emits passes and proves nothing about the model. The defence is the one already described: script what has been observed, and when in doubt, go and observe. The stub makes the harness testable. It does not make the model predictable, and a test suite that forgets the difference is a test suite that passes while the agent loops.

Where parity begins and ends

The chapter has used the word parity several times, and plank's approach to it deserves a section of its own, because the approach is the single most transferable idea in the port and because getting it wrong in either direction is expensive.

The claim is this: the model was trained on the C agent's exact bytes, so the text the harness sends that the model was trained to expect must be byte-identical to what the C sends. That text is the system prompt, the tools prompt that describes the tools and their syntax, the tool-call syntax itself, and the tool-result framing. Nothing else. Plank's parity test file opens with that sentence and then enforces it in two layers.

The first layer is fixtures: committed snapshots of the reference bytes, compared on every test run, including on machines that do not have the C source checked out. The second layer, when the C source is present, decodes the named string constants straight out of the C file and compares them to the fixtures, so the fixtures cannot drift from the reference without a test noticing. Non-deterministic spans such as timestamps are masked, and everything around the masks is compared exactly.

The first thing those tests caught is a lesson about Rust as much as about parity. The tools prompt had been written as one long string literal with backslash continuations at the ends of lines, and a backslash at the end of a line inside a Rust string literal skips the newline and all leading whitespace on the next line. The literal silently deleted the four-space indentation in the anchored-edit example and every indent inside the JSON tool schemas, thousands of bytes that no longer matched what the model was trained on, invisible in review because the source showed the indentation. The prompt now lives in a separate text file included at compile time, and the finding closes with a rule: any string that must survive byte-exact should either avoid continued literals or keep its indentation on the same physical line as the newline escape.

Chapter 2: Tool calls are text

The second thing to know about the DSML markers is that the vertical bar in them is not the ASCII bar. It is U+FF5C, the fullwidth vertical bar, and a harness that types the ASCII character is producing a tag the model has never seen. This is the kind of detail that no amount of reading the reference source will reliably catch and that a byte-diff fixture catches on the first run.

The boundary matters in the other direction too, and that is the vision-encoder story from earlier in the chapter. A harness that copies the reference's error strings because they are the reference's is copying text the model was never trained on, and inheriting whatever that text says about a program the harness is not. Parity is a discipline about a specific set of bytes, and knowing which bytes are in the set is most of the discipline.

Blanka has no C reference to diff against, so its version of the discipline is smaller: the framing fixture, and a rule that the constants for the tool-call tags and the frame text live in one module and are used everywhere by name. The rule is enough for a book. For a harness that talks to a model trained on some other harness's output, the two-layer approach is the one to copy.

A dialect is a table, and the opener names it

The rule about constants living in one module sounds like tidiness until a model family changes its spelling. DeepSeek V4.1 did not invent a new tool syntax. It respelled the existing one: where V4 writes `< | DSML | tool_calls>`, `< | DSML | invoke and </ | DSML | parameter>`, V4.1 writes `< | DSML | calls>`, `< | DSML | invoke and </ | DSML | parameter>`. The word `tool_calls` becomes `calls`, and the space after the second bar is part of the tag, not formatting. It survives no trimming and no normalisation.

A difference that small invites hand-typed constants at each of the half-dozen places that name a tag, and plank's findings record why that fails. The tolerance plank already grants the V4 spelling, the dropped leading bar and the SSML alias, has to compose with the respelling. The streaming detector builds its accepted markers by substituting the marker word into the table's opening tag, replacing only `DSML` and never the space after it. Type the V4.1 opener by hand at one site and the SSML or dropped-bar variant of it silently stops being recognised, on V4.1 only, which surfaces as a generic "markup outside a valid tool calls block" error on every turn. So every tag comes from one table, a tag set per dialect in plank's streaming crate, and the dialect is keyed on the shape name DwarfStar's engine reports rather than on the path of the weights file, so a renamed or relocated checkpoint still resolves. Anything that is not V4.1 falls through to V4.

The next fix was about who gets to choose the table. Plank first fixed the dialect when the renderer was constructed, from the loaded model's name. That works for the live session and fails for every consumer that has no model to ask: a debugging console that renders bytes arriving on a socket, and transcript replay, where a saved session may hold either dialect. The opener never matched, the markup streamed to the screen as prose, and the calls were dropped. Plank commit `f26c645` matches every known opener at once and adopts the dialect the opener was spelled in for the whole stanza, so the inner

Chapter 2: Tool calls are text

tags stay exactly as strict as before. The same change repaired a quieter defect: a renderer that outlived its first V4.1 stanza used to reset to V4 afterwards and go deaf to the second.

The Qwen dialect then made the round trip that justified the design. When the upstream engine moved to V4.1 it deleted the Qwen support that had lived only in a fork, and plank removed the family, its flag, its dialect and its cache namespace with it. Two days later upstream merged Qwen3.8 properly and began publishing the weights itself, and the removal was reverted. The dialect came back wired the new way, in commit 9dcd8c3: `<tool_call>` joins the openers the detector matches, and a match hands the stanza to the Qwen parser, so a renderer told nothing about the model still reads one. The commit is candid about the one behaviour that reversed. A DeepSeek model that mistakenly writes Qwen markup used to be told to write DSML instead; now its call runs. That cost is paid for the callers who cannot know which model produced the bytes. The Qwen opener also carries no dedicated marker token, so unlike the DSML one it is a string a model can write in prose, and a bare mention is reported rather than executed.

The lesson for a harness of your own is narrower than it looks. You will probably speak one dialect. If you ever speak two, make the tag set a value rather than a scattering of literals, and let the text decide which value applies whenever something downstream of the engine has to read it without the engine's help.

What the tool table does not yet contain

The binary at the end of this chapter runs turns through the agent loop, and its tool table is empty. A model that asks for a tool gets back an error saying the tool is unknown, and a model that does not ask for one gets its answer as before. The loop works. It has nothing to dispatch to.

That is deliberate, because the tools are a chapter of their own, and the reason they are a chapter of their own is that each one turns out to have a way of going wrong that costs a session. An edit tool that matches text exactly fails on every file with Windows line endings. A search tool that does not skip the build directory returns Cargo's copies of dependency sources before the project's own. A read tool with no limit on output size fills the context with one file. The six tools that cover most of what a coding agent does are small, and the traps are where the chapter's length goes.

What this chapter has fixed is everything around them: the shape of a request, the strictness with which it is read, the single dispatcher, the exact text that goes back, and the loop that ties them together. A tool added in chapter 3 implements three methods and is registered by name, and nothing in this chapter changes.

Where blanka stops short of plank

A reader comparing blanka's parser to plank's will find plank's several times longer, and the difference is a useful inventory of what a production harness accumulates around an idea that fits in a page.

Plank's parser is incremental, as described above, and carries the state and the partial-match bookkeeping that implies. It speaks three dialects, the two DSML spellings from one tag table and Qwen's through a parser of its own, and it lets the stanza opener choose

Chapter 2: Tool calls are text

among them rather than the loaded model's name, because a harness that supports several model families has to read all of them, a user should not have to say which, and a replayed transcript has no model to ask. It tolerates the two observed near-misses in the streaming layer, the dropped bar and the misspelled marker, each with a comment recording the session that produced it. It distinguishes a closing tag from a `</` sequence inside a parameter value, so that a model writing HTML through the edit tool does not terminate its own stanza, and it recognizes a newer weight's habit of ending the opening tag at the bar without a closing angle bracket. Each of those is a paragraph in the source and a session in the findings file.

Blanka has none of them, and will acquire the ones the book needs as the chapters call for them. The whole-reply parser stays as it is. Chapter 4 adds the streaming detector. Chapter 6 adds the tools prompt that teaches the model the syntax. Nothing in the book adds a second dialect, because the reference local model speaks one, and a harness for one model family does not need a dialect switch.

The comparison is offered not as a confession but as a map. The core of a tool-call protocol is what this chapter has shown: a strict parser with three failure modes, a numbered frame, an error convention, and a loop. Everything plank adds around that core is a response to a specific model doing a specific thing in a specific session, and a harness of your own will accumulate a different set, in response to your model's habits. The core is what to get right first, because every later addition is a patch on it.

Requests, read strictly, answered exactly

The model still produces only text. A tool call is a stanza in that text that the harness has agreed to read as a request, and the agreement has two sides that must match: the syntax the model was trained to write and the frame it was trained to expect back. In-band stanzas and native provider calls are two routes to the same request, and adapting the native form into the in-band syntax gives the loop one shape to handle and one parser to test.

The parser is strict on purpose. It executes exactly what the model wrote or nothing, distinguishes an unfinished stanza from a malformed one, and reports reasons a model can act on. Tolerance for a model's near-misses belongs in the streaming detector or the prompt, never in the executable parser.

The result frame is model-facing and pinned by a fixture; the error text is the harness's own and is written as an instruction the model will follow. The loop records every reply before parsing it, feeds errors back rather than ending the turn, and sums its accounting across passes. Parity is a discipline about a specific set of bytes, and knowing which bytes are in the set is most of the discipline.

CHAPTER 3: A TOOL TABLE WORTH TRUSTING

The loop can dispatch a request to any tool that implements three methods, and the table it dispatches to is empty. Filling it is the work of this chapter, and the work turns out to be less about what each tool does than about the ways each one goes wrong. Six tools cover most of what a coding agent does: read a file, write a file, edit a span of a file, run a shell command, find files by name, and find text in files. None of them is more than a page of code in its straightforward form. Each of them, in that form, has cost a plank session, and the cost is what the chapter is about.

A tool's failure is expensive in a way that ordinary program failures are not. When a tool returns the wrong thing, the wrong thing goes in front of a model that will act on it. When a tool refuses to do something the model asked for, the model will read the refusal and try another way, and the other way is often a shell command that does the same thing without the protection the refusal was providing. When a tool returns too much, the excess sits in the transcript for the rest of the session and is re-read on every pass. Every one of those has a concrete instance in plank's findings, and the fixes are what make the difference between a table of tools and a table worth trusting.

The context grows

Before the tools, the struct they see. Chapter 2 gave every tool a `ToolContext` holding a working directory and nothing else, with the promise that later chapters would add to it. This chapter adds three things:

```
pub struct ToolContext {
    pub cwd: PathBuf,
    pub write_roots: Vec<PathBuf>,
    pub spill_dir: PathBuf,
    pub spill: SpillPolicy,
}
```

The write roots are the directories a tool may write under, and they are the subject of the next section. The spill directory and policy govern what happens to a result too large to hand to the model whole, and they are the subject of the section after. The working directory gains a method, `resolve`, that turns a model-supplied path into an absolute one by joining it to the working directory unless it is already absolute. Every tool that takes a path uses it, so there is one definition of what a relative path means.

The reason to keep adding fields to one struct, rather than giving each tool its own configuration, is the reason given in chapter 2: a tool sees exactly the context and nothing else, so the context is the complete list of what a tool can do. When a later chapter adds hooks that run before a tool, or a sandbox profile for the shell, they are fields here, and a reader who wants to know what a tool can reach has one struct to read.

Arguments and their defaults

Every tool begins by fetching its arguments from the JSON object the parser handed over, and the way it does so is a small policy decision repeated six times, so it is made once in two shared helpers.

A required string argument either exists, is a string and is non-empty, or the tool fails with an error naming the tool and the argument: "edit requires old", "read requires path". The three failure cases, missing, wrong type and empty, are distinguished in the message, because a model that wrote an empty path and one that forgot the path have made different mistakes and correct them differently. An optional integer argument is fetched with a default and a range, and the helper is deliberately lenient about type: a number written as a string, "50" rather than 50, is parsed rather than rejected, because a model that quoted a count has not made a mistake worth a round trip. The reference's integer parsing does the same, tolerating trailing whitespace and falling back to the default on anything else, and the finding-driven rule from chapter 2 applies: a leniency is defensible when the input's meaning is unambiguous and the tool does nothing the model did not ask for.

The range clamp is the other half of the integer helper. A read of a million lines, a search capped at ten thousand results, a shell timeout of a week: each is a value a model might write and none is one the harness should honour as written. Clamping to a sensible range, and saying nothing about it, is the right response for a value that is merely too large. The alternatives are to refuse, which costs a round trip for a request whose intent was clear, or to honour it, which puts a million lines in the transcript. The clamp bounds the damage without a conversation.

Paths are the one argument kind with a helper of their own, and it is the `resolve` method on the context. A model supplies paths in whatever form it last saw them: relative to the project root, because that is how a search result or a glob listing reported them; absolute, because that is how a shell command printed them; or relative to some directory it changed to in a shell command three rounds ago, which the tools know nothing about. The method joins a relative path to the working directory and leaves an absolute one alone, and every tool uses it, so the meaning of a relative path is fixed in one place. Plank's finding about the model prefixing shell commands with a change to the working directory is the same confusion seen from the shell side: the model was not sure where it was, so it said so on every call, at the cost of tokens and of hiding the command from the containment classifier. A tool table whose paths are unambiguous, and whose descriptions say that paths are relative to the project root, removes the reason for the habit.

Reads are free, writes are contained

The first design decision that shapes every tool in the table is about where a tool may write, and it is a decision about trust in two directions at once.

A model is not malicious, but it is also not careful in the way a person is careful. It will resolve a path by string manipulation and produce something with a `...` in it. It will take a path from a file it read and use it as a destination without noticing that the path is absolute and points at the user's home directory. It will, if the working directory contains

Chapter 3: A tool table worth trusting

a symbolic link, follow the link without knowing it is one. None of these is an attack. All of them can put a file where the user did not expect a file, and a coding agent that writes outside the project it was pointed at has done something the user cannot easily undo and may not notice for days.

So writes are contained. Every tool that writes a path resolves it through one method on the context, and that method refuses anything that does not land under one of the write roots:

```
pub fn resolve_for_write(&self, tool: &str, path: impl AsRef<Path>) -> Result<PathBuf,
ToolError> {
    let path = path.as_ref();
    let full = self.resolve(path);
    let real = realpath_for_write(&full);
    let roots: Vec<PathBuf> = self
        .write_roots
        .iter()
        .map(|r| r.canonicalize().unwrap_or_else(|_| r.clone()))
        .collect();
    if roots.iter().any(|r| real.starts_with(r)) {
        Ok(full)
    } else {
        Err(ToolError(format!(
            "{tool} path escapes workspace: {}",
            path.display()
        )))
    }
}
```

The interesting line is the call to `realpath_for_write`, and it exists because a prefix check on the path as written is not a containment check at all. A path can contain `..` components that a string comparison will not see through. A directory inside the workspace can be a symbolic link to a directory outside it, and a path through the link is inside by name and outside in fact. The standard library's canonicalization resolves both, but it requires the path to exist, and the whole point of a write is that the target often does not exist yet. So the function walks up from the target to the longest existing ancestor, canonicalizes that, and appends the remaining components:

```
pub fn realpath_for_write(target: &Path) -> PathBuf {
    let mut existing = target.to_path_buf();
    let mut tail: Vec<std::ffi::OsString> = Vec::new();
    while !existing.exists() {
        match (existing.file_name(), existing.parent()) {
            (Some(name), Some(parent)) => {
                tail.push(name.to_os_string());
                existing = parent.to_path_buf();
            }
            _ => break,
        }
    }
}
```

Chapter 3: A tool table worth trusting

```
    }  
    let mut real = existing.canonicalize().unwrap_or(existing);  
    for part in tail.iter().rev() {  
        real.push(part);  
    }  
    real  
}
```

A test creates a symbolic link inside a scratch workspace pointing outside it and asserts that a write through the link is refused. Another asserts that a relative path to a file in a directory that does not exist yet is accepted, because the write tool is about to create the directory and the check must not require it first. A third asserts the exact error text, because the error is model-facing and the model will read it.

Plank's version of this method is the same shape with a larger set of roots: the working directory, the system temporary directories, any extra paths the user has configured, and the git metadata directory when the working directory is a linked worktree, since a `git commit` inside a worktree writes to a directory outside it and refusing that would not be a containment win. Plank also wires the same roots into a macOS sandbox profile for shell commands, so that a model-chosen command running under the operating system's sandbox is bound by the same list the tools check against. The single method is what makes that possible: there is one list of roots, and everything that writes consults it.

The other half of the decision is that reads are not contained, and this is deliberate rather than an omission. A model working on a project legitimately reads files outside it: system headers, installed library sources, configuration in the home directory, the output of a command that wrote to a temporary file. A read has no side effect on the user's files, and a read that is refused will be retried through the shell, where it will succeed, so the refusal buys nothing and costs a round trip. Plank's documentation puts it in one sentence: reads are deliberately not contained, and the bash sandbox itself allows reads everywhere. The asymmetry is the point. Containment is about what the model can change, not what it can see.

One plank finding shows how carefully the boundary between the two has to be drawn even when the intent is clear. Plank's home directory holds the session store, the cache and the hooks, so a model-chosen command that can write there can rewrite plank's own behaviour, and writes to it are refused unless the user grants them. The check that asked the user was based on the command text: if a shell command mentioned the plank home directory, the tool asked whether to allow writing there. That meant a command that only read a settings file from that directory, `cat` on a path under it, asked for write permission, and denying it changed nothing because the read was allowed anyway. The fix was a classifier that recognizes a read-only command, one whose every simple command begins with a known reader and which has no output redirection, and only asks for the rest. The first version of the classifier still asked on a command that redirected standard error to the null device, and a second finding a day later stripped the two `stderr-only` redirection spellings before checking for output redirection. The lesson generalizes: a containment check that over-asks trains the user to click through,

Chapter 3: A tool table worth trusting

and a check that over-grants is not a check, and getting the classifier to sit exactly between the two took two findings and three sessions.

What comes back can be too big

The second decision is about size. A tool's output goes into the transcript, and everything in the transcript is re-read on every pass until something removes it. A read of a ten-thousand-line file, or a search that matches every line of a generated file, or a build that prints its entire dependency tree, puts that text in front of the model for the rest of the turn and, without compaction, for the rest of the session. Plank measured one instance: two search results in a single session came to fifty-four and seventy-one thousand characters of duplicated matches, a third of that session's total prefill, and the model then spent a turn complaining about polluted search output rather than working.

The response has two layers. The first is per tool: the read tool returns a window of lines rather than the whole file, the shell tool keeps the head and tail of long output, the search and glob tools stop at a maximum number of results. Each tool knows what a sensible bound is for its own kind of output, and each says in its result how to get more. The second layer is a policy applied after every dispatch, to every tool, including tools added later and tools the harness did not write:

```
pub fn apply(policy: &SpillPolicy, dir: &Path, result: String) -> String {
    if result.len() <= policy.max_bytes {
        return result;
    }
    // ... write the full result to `dir/<n>.txt` ...
    let mut out = result[..cut].to_string();
    out.push_str(&format!(
        "[Output truncated at {cut} bytes of {total}. Full output saved to {}. Read it with the
read tool, starting at line 1.]\n",
        path.display()
    ));
    out
}
```

Above the cap, the full result is written to disk and the model receives a preview followed by a locator. The full payload is kept because it is what the tool actually returned, and a session record that lost it would be a session record that could not explain what the model saw a summary of. The locator tells the model where the rest is and how to read it, in a sentence shaped like the read tool's own truncation notice, because that is a sentence the model has already learned to act on.

Plank's finding about this feature is short and worth quoting in spirit: the locator line is new model-facing text. The reference agent has no spill concept, so there is no reference wording to match, and the shape deliberately reuses an existing fixture-blessed sentence to minimize the new surface. It is recorded as a deliberate deviation from parity, and it is gated behind a setting whose default is high enough that ordinary sessions never spill. That last point is a design judgement of its own: the spill exists for the pathological case,

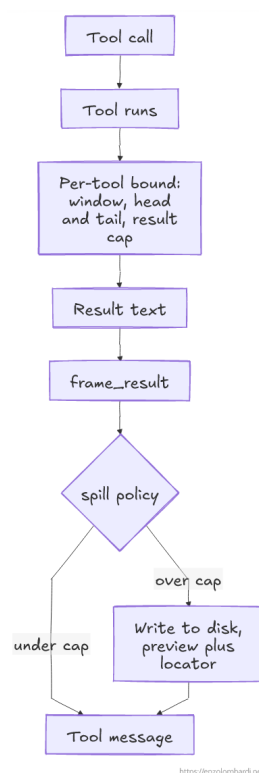
Chapter 3: A tool table worth trusting

and a policy that fired often would be training the model to expect summaries where it should expect content.

Blanka applies spill in the dispatcher, between the tool's return and the result frame, so a tool's implementation never sees it. The dispatcher from chapter 2 gains one line, and the policy's tests exercise the three cases that matter: a small result passes through unchanged, a large one lands on disk and comes back as a preview with a locator, and the preview cut never splits a multi-byte character.

Two layers of bounding

The size problem has two answers that look alike and are not, and the chart shows where each sits:



Two layers of bounding: each tool limits what it returns in its own terms, and the spill policy catches whatever is still too large.

The per-tool bound is the one that knows what the output means. The read tool bounds in lines, because lines are how a model addresses a file. The shell tool bounds in bytes at the head and the tail, because that is where a command's useful output lives. The search tool bounds in matches, because a match is the unit the model asked for. Each says in its own terms how to get more, and each is tuned to what a model doing that kind of work needs. A single generic bound could not do any of that; it would cut a file mid-line and a search result mid-path.

The spill policy is the one that knows nothing. It sees a string and a byte count, and it exists for the outputs the per-tool bounds did not anticipate: a tool added later without bounds of its own, a tool from another process, a per-tool bound that was set too

Chapter 3: A tool table worth trusting

generously for one pathological input. It is a safety net, and a safety net that fires often is a sign that the layer above it is wrong. Plank's default for the spill cap is a megabyte, high enough that ordinary sessions never spill, and the finding recording the feature says so explicitly. Blanka's default is lower, sixty-four kilobytes, because the book's reader is more likely to be watching the transcript than running long sessions, and a visible spill is a useful thing to see once. Either way the principle holds: the tools bound, and the policy catches.

There is a third bound that neither layer provides, and it is the reason chapter 7 exists. Both layers limit how much a single result can put into the transcript. Neither limits how long it stays there. A well-bounded read of five hundred lines is re-read on every pass for the rest of the session, and after forty rounds the transcript is mostly old tool results the model has already acted on. Removing them, once the model has moved past them, is compaction, and it is a different problem from bounding because it requires deciding what the model no longer needs, which no tool can know at the moment it returns.

Reading

The read tool is the most used tool in any coding session, by a wide margin. Plank counted four hundred and seventy-two reads against forty-five searches and twelve globs across thirty sessions. It is also the tool most likely to fill a context, and its design is a set of defaults chosen to prevent that.

It reads a window: a starting line, a number of lines, and a default of five hundred when the model does not say. It numbers every line, because the edit tool that follows needs the model to know exactly where things are, and because a model asked to summarize a file will otherwise invent line numbers. And when the window ends before the file does, it says so in a sentence that names the next starting line:

```
if to < total {
  out.push_str(&format!(
    "[Read truncated at line {to} of {total}. Call read again with start_line={ } to
continue.]\n",
    to + 1
  ));
}
```

That sentence is doing more than reporting. A model that reads the first five hundred lines of a file and receives no indication that there are more will assume it has read the file. A model that receives a note saying how to continue will continue if it needs to and stop if it does not, which is exactly the judgement the harness wants to leave with it.

A file over a size limit is refused before it is read, with the size in the message. This is the read tool's one hard refusal, and it exists because a model asked to read a multi-megabyte log will, and the read would either fill the context in one call or take long enough that the spill policy has to save it, and neither is what the model wanted. A refusal that names the size lets the model choose a different approach, usually a search or a windowed read of the tail.

Chapter 3: A tool table worth trusting

The reference's read tool reads bytes and splits them into line spans with an explicit handling of carriage returns, and blanka's does the same in a simpler form, splitting on newlines and keeping each line's own terminator so that a file's line endings are preserved through a read and can be seen by the model. That detail becomes important two sections from now.

Writing

The write tool creates or overwrites a whole file, and it is the first tool to pass through write containment. Its body is short and one decision in it deserves explanation:

```
let full = ctx.resolve_for_write("write", path)?;
if let Some(parent) = full.parent() {
  std::fs::create_dir_all(parent)
    .map_err(|e| ToolError(format!("write {path}: create parent: {e}")))?;
}
std::fs::write(&full, content).map_err(|e| ToolError(format!("write {path}: {e}")))?;
```

A missing parent directory is created rather than refused. The reference agent refuses, and the model then issues a shell command to create the directory and retries the write, which costs a full round trip carrying the entire file content twice. Plank's source records the reasoning for changing this: the target has already passed containment, so the hierarchy being created lands inside the workspace, and making the model re-issue the write behind a `mkdir` costs a whole round trip of the file it has only now generated. The user is told that a directory appeared, since that is a side effect they did not ask for by name, and the model is not made to ask for it.

The pattern, a tool doing the evidently intended thing rather than refusing and letting the model discover the workaround, recurs. The test of whether it is the right call is whether the workaround is one the model would find anyway. If it is, the refusal has only added a round trip and a chance for the model to try something worse. If the workaround would be something the harness does not want, the refusal is doing its job. Creating a parent directory inside the workspace is the first kind. Writing outside the workspace is the second.

Overwrites, and why there is no confirmation prompt

The write tool overwrites an existing file without asking, and a reader used to interactive tools may find that alarming. The decision follows from what the user did when they started the agent.

A coding agent is pointed at a directory and asked to change it. That is the authorization: the user has said, by running the agent here, that files under this directory may be changed. A confirmation prompt on every write would ask the user to re-grant, dozens of times a session, a permission they granted once at the start, and users asked the same question dozens of times answer it without reading. Plank's sandbox documentation makes the parallel point about shell commands the user types: the user typing the command is the authorization, and it is not sandboxed. The write roots are the boundary, and inside the boundary the agent acts.

Chapter 3: A tool table worth trusting

What the user needs instead of a prompt is visibility, and this is the second-audience point again. Plank shows an overwrite as a diff card, with the lines that changed, and shows a newly created file's content as it streams, dimmed, so the user can see what is being written without it being presented as the model's answer. The distinction between the two displays is deliberate: an overwrite changed something the user had, and the user wants to see what; a new file is something the user asked for, and the user wants to see it once. Blanka's write returns a one-line confirmation to the model and, until chapter 4, nothing to the user. The confirmation names the path and the byte count, which is what the model needs to move on.

There is one place where a write does deserve a pause, and plank has it: version control. A coding agent that commits, pushes or rewrites history is acting beyond the directory in a way a file write is not, and plank's prompt rules for git say what not to do without being asked. That is a rule in text, not a prompt in code, because a rule in text is read once per pass and a prompt in code is a round trip; but it is also a boundary the harness draws around the shell tool specifically, because the shell is the only tool that can reach version control. Chapter 12 returns to this when worktrees give the agent a place to act where nothing it does can reach the user's checkout.

Bytes that are not text

Every tool in the table assumes it is working with text, and files are not always text. The assumption is handled differently by the tools that read and the tool that writes, and the difference is deliberate.

The read tool converts a file's bytes to text lossily: a byte sequence that is not valid UTF-8 becomes the replacement character and the read proceeds. A model that asks to read a file with a stray invalid byte, or a file in a legacy encoding, gets the readable parts and a marker where the unreadable ones were, which is more useful than a refusal. The search tool goes further and skips a file entirely if its first kilobyte contains a null byte, on the reasoning that a match inside a compiled object or an image is never what the model wanted and the file's size would dominate the result. Neither of these is a precise test for "binary", and neither needs to be; both are judgements about what a model reading or searching a codebase is looking for, and both say what they did.

The edit tool refuses. A file whose bytes are not valid UTF-8 cannot be edited by the tool, and the error says so. The reasoning is the reasoning behind the tool's whole design: an edit is a byte-exact replacement of one span with another, and a lossy conversion on the way in would mean the bytes written back are not the bytes that were there, in places the model never touched. A tool that silently rewrote every invalid byte in a file as a replacement character, as a side effect of editing one line, would be corrupting files while reporting success. Refusing costs the model a round trip and the chance to use the shell; corrupting costs the user data. The reference's edit tool works on bytes throughout and avoids the question; blanka's works on strings and answers it by refusing, and a reader whose projects hold non-UTF-8 sources will want the byte version.

Editing, and the line-ending trap

The edit tool replaces one span of a file with another, and it is where a coding agent does most of its actual work. It is also where plank's most expensive tool finding lives.

Chapter 3: A tool table worth trusting

The tool's contract is conservative on purpose. The model supplies the exact text to replace and the exact text to replace it with, and the old text must occur exactly once in the file. An anchor that does not occur is an error. An anchor that occurs twice is an error, with a message telling the model to include more surrounding lines. There is no fuzzy matching, no whitespace normalization, no "closest match", and the reason is the same as the parser's strictness in chapter 2: an edit that guesses which occurrence the model meant will, eventually, edit the wrong function, and the model will not know, because the result will say the edit succeeded.

```
pub fn apply_edit(text: &str, old: &str, new: &str) -> Result<String, EditError> {
    if old.is_empty() {
        return Err(EditError::EmptyOld);
    }
    let (old, new) = adapt_line_endings(text, old, new);
    let Some(first) = text.find(&old) else {
        return Err(EditError::NotFound);
    };
    if text[first + 1..].contains(&old) {
        return Err(EditError::NotUnique);
    }
    let mut out = String::with_capacity(text.len() + new.len());
    out.push_str(&text[..first]);
    out.push_str(&new);
    out.push_str(&text[first + old.len()..]);
    Ok(out)
}
```

The function is pure, taking the file's text and returning the edited text, so it can be tested without a file system, and the tool wraps it with the read, the containment check and the write. After a successful edit the tool echoes the lines around the change with their new numbers, because an edit shifts everything below it and a model that goes on to make a second edit needs to know where things are now without paying for another read. The reference does this too, and calls it echoing post-edit context.

The line-ending adaptation in the middle of that function is the finding. Plank was run against a project whose source files used Windows line endings, carriage return then line feed, on most lines. The model writes line feeds, because that is what it was trained on and what it sees when a file is read back to it in most displays. A byte-exact match of a multi-line anchor therefore never succeeded: the model's `old` text had a line feed where the file had two bytes, and the tool reported the anchor not found. Each failure cost the model a full re-read of the file and a retry, and there were five such failures in one forty-minute run. Worse, a single-line anchor with no line ending in it matched as a substring, and the model's `new` text, which ended in a line feed, replaced a line whose ending had been two bytes, so the diff showed the anchored line changed by its line ending alone and the file now had mixed endings.

The fix is the adaptation:

Chapter 3: A tool table worth trusting

```
pub fn adapt_line_endings(data: &str, old: &str, new: &str) -> (String, String) {
    if !uses_crlf(data) || old.contains('\r') || new.contains('\r') {
        return (old.to_string(), new.to_string());
    }
    (old.replace('\n', "\r\n"), new.replace('\n', "\r\n"))
}
```

When the whole file uses the two-byte ending and the model's text carries no carriage return of its own, every line feed in both the anchor and the replacement becomes the two-byte form, so the match succeeds and the replacement keeps the file's convention. Mixed files are left alone. Plank's finding gives the reason in one clause: guessing there would corrupt the other kind of line. A file that is already inconsistent has no convention to preserve, and the tool should do exactly what it was told rather than impose one.

The test for this is the one that would have caught the finding before it cost a session: a file with the two-byte ending, a multi-line anchor written with line feeds, and an assertion that the edit succeeds and the output keeps the two-byte form throughout. A second test covers the single-line case that corrupted the file, and a third asserts that a mixed file is edited byte for byte as written.

This is also the finding whose fix produced the largest measurable improvement in plank's controlled reruns. Chapter 1 mentioned a request that went from two hours to sixteen minutes over several fixes; the line-ending adaptation alone removed five failed edits from one of those runs, each of which had been a re-read and a retry turn.

The shell, and the head and tail of it

The shell tool runs a command and returns what it printed, and its two design problems are time and volume.

Time first. A command may not finish. A test suite may hang, a server may start and wait for connections, a build may run for longer than anyone intended. A shell tool that waits forever has hung the agent, and a user who has been watching a cursor for ten minutes has no way to know whether the model is thinking or the command is stuck. So the tool has a deadline, a default of two minutes that the model may raise, and a command that reaches it is killed and reported as such:

```
let status = loop {
    if let Some(s) = child.try_wait().map_err(|e| ToolError(format!("bash: wait: {e}")))? {
        break Some(s);
    }
    if started.elapsed() >= timeout {
        let _ = child.kill();
        let _ = child.wait();
        timed_out = true;
        break None;
    }
    std::thread::sleep(Duration::from_millis(20));
};
```

Chapter 3: A tool table worth trusting

The loop polls rather than blocking, because the standard library's blocking wait has no timeout, and the poll interval is short enough that a fast command is not noticeably delayed. Standard output and standard error are drained on their own threads, because a command that prints more than the pipe buffer holds will block on its own output if the parent is waiting for it to exit, and the two will wait for each other until the deadline. This is the kind of deadlock that appears only with chatty commands and only sometimes, and draining on threads removes it.

The reference agent goes further than blanka here, and the difference is worth knowing about. Its shell tool does not block at all: every command is a tracked job with its own process, reader threads and an output file, and the tool returns an observation of the job's state, running or finished, with the output so far. A model that starts a long build gets control back immediately and can poll the job or start another. Chapter 11 builds this asynchronous version, and the reason it is deferred is that it needs the event machinery of that chapter to notify the model when a job it stopped watching finishes. For now the tool is synchronous, bounded by its deadline, and that is enough for a command that runs the tests.

The job tools carry a porting lesson worth taking before chapter 11 arrives. In the C, the function that observes a job takes three separate flags, whether to wait, how long to refresh for, and whether to stop, and its status and stop callers derive the first from the other two: wait when stopping or when a refresh interval was given, with the interval defaulting to zero. Plank's port copied the function's signature faithfully and collapsed the derivation at the call site, passing the stop flag where the C passed wait, and gave the interval the shell tool's default of sixty seconds. The status tool therefore made one non-blocking poll whatever interval the model asked for, and the model, told by the reference prompt that the status tool returns immediately unless an interval is given, asked for three hundred seconds once per generation while a long test ran, in a tight loop. The prompt was accurate to the C and false for plank. When a ported function takes flags that are derived from one another, the derivation is part of the contract, and it has to be ported at the call site as well as the callee.

Volume second. A command's output can be enormous, and the reference's observation of it is shaped by an observation of how people read command output. The first time a job's output is shown, the head is what matters: the first lines carry the headers, the command that was run, the first error. Later observations of a running job show the tail, because the tail is where progress is. Plank's job observation is head-biased on first sight, showing the first hundred lines up to eight kilobytes, and tail-biased after that. Blanka's synchronous tool keeps the head and the tail of any output that exceeds both limits, with a note between them saying how many bytes were omitted:

```
pub fn head_tail(output: &str, head: usize, tail: usize) -> String {
    if output.len() <= head + tail {
        return output.to_string();
    }
    // ... find character boundaries ...
    format!(
        "{}\n[... {} bytes omitted ...]\n{}",

```

Chapter 3: A tool table worth trusting

```
    &output[..h],  
    t - h,  
    &output[t..]  
  )  
}
```

The result carries the exit status on its first line, before any output, because the exit status is the one thing a model needs to know about a command and it should not have to scroll for it. A build that fails prints its errors near the end; a test suite that fails prints them in the middle; both print a non-zero status, and a model that reads the status first knows whether to read the rest carefully.

The description the model sees for this tool deserves a separate look, and the next section is about why.

Why the shell stays

A reader who has followed the containment argument may wonder why the table includes a shell tool at all. Every other tool is bounded, contained and interpretable, and the shell is none of those things: it runs whatever the model writes, its output is whatever the command printed, and the harness cannot tell a `grep` from a `rm` without parsing shell syntax, which the findings above have already shown to be a losing game.

The shell stays because a coding agent that cannot run the build, the tests, the formatter and the version-control commands is not a coding agent. Those are the actions that tell the model whether its edits worked, and no table of purpose-built tools can anticipate every project's build system. The shell is the escape hatch, and the design question is not whether to have it but how to keep it from becoming the path of least resistance for things the other tools do better.

The answer has three parts, two of which this chapter has already given. The description steers the model toward the native tools for reading, editing, listing and searching, so the shell is used for what only the shell can do. The output is bounded at the head and the tail and carries the exit status first, so a command's result is as usable as a native tool's. The third part is what plank does and blanka does not yet: run the command under the operating system's sandbox with the same write roots the tools check against, so that a model-chosen command is bound by the same containment as a model-chosen write. Plank generates a profile for macOS's sandbox facility from the write roots, and runs every model-initiated command under it, with reads allowed everywhere and writes allowed only under the roots. Commands the user types at the prompt are not sandboxed, because the user typing the command is the authorization. The profile is what makes the shell an acceptable escape hatch: it can do anything the model asks, within the same boundary as everything else.

Two details of plank's version are worth carrying forward as design notes even where the sandbox itself is not available. A project's own configuration can tighten the sandbox but never loosen it, because a cloned checkout must not be able to relax the containment for the user who opens it; a repository is untrusted input in exactly the way a model's output is. And the escape list, commands that skip the sandbox by pattern, is documented as a convenience and not a boundary, because a pattern match against a

Chapter 3: A tool table worth trusting

command line is not a security check and pretending otherwise would be worse than not having it.

A sandbox that forgot the toolchain

Plank's sandbox began with the write roots the tools use, the working directory and the temporary directory, and that turned out to describe the project but not the work. A `cargo install` of the project inside a sandboxed shell failed with "Operation not permitted" on a file under `~/.cargo`, a file the user owned, with ordinary permissions, on a writable volume. The message reads like a file system problem, and the tell that it is not is the error number. The sandbox denies with `EPERM`, error 1, "Operation not permitted"; an ordinary permission problem is `EACCES`, error 13, "Permission denied". A writable directory the user owns that refuses both an existing file and a new one with `EPERM` is a sandbox profile every time, and the distinction is worth teaching the model's error text as much as the developer.

The narrow reading, that installing was blocked, was the wrong lesson. The same denial covered Cargo's registry cache, so a build had worked only as long as every dependency happened to be cached already. The first build that needed an uncached crate failed the same way, and so would a package install for any other language. Build tools write a machine-wide cache outside the project by design, and a sandbox whose roots are the project and the temporary directory does not describe how they work. The split plank settled on, in commit 4b11629, is by what a write costs rather than by where it lands. Toolchain caches are writable by default, because a write there costs disk space and nothing else. Directories on the user's executable search path are withheld and granted on request, because a binary installed there is one the user will later run without thinking about who wrote it. That is why the grant names `~/.cargo/registry` and `~/.cargo/git` rather than `~/.cargo`: the parent carries `bin` with it, and granting it would hand the model the user's search path while looking like a build fix. Cargo's configuration file stays denied for a related reason, since it can redirect the registry or inject a linker flag. One detection wrinkle follows from the profile being built before the command runs: an install with no explicit root writes to Cargo's `bin` directory without ever spelling the path, so the request is recognised by the shape of the command rather than by the paths it mentions.

The fix also corrected a sentence. The sandbox's "blocked" hint had been telling the model to add the path to the project's sandbox file, which by the rule above may only tighten the policy, so a model that followed the hint changed nothing at all. Only the user's own file can widen it. A hint is model-facing text, and a hint that describes a remedy the harness will refuse is worse than no hint, because the model spends a round obeying it.

Descriptions are where behaviour is decided

Chapter 2 mentioned a plank finding in passing, and this is where it belongs in full. Across thirty saved sessions, plank counted how often the model used each native tool against how often it used a shell command that did the same thing. Searches: forty-five through the search tool, a hundred and twenty-five through `grep` in the shell. File listings: twelve through the glob tool, eighteen through `find` and `ls`. Reads: four

Chapter 3: A tool table worth trusting

hundred and seventy-two through the read tool and thirty-seven through `cat`, `head` and `sed`. And two hundred and twenty-eight of the three hundred and seven shell commands began by changing to the working directory, although the shell already ran there, a prefix that cost tokens on every call and hid the real command from the containment classifier until after the first operator.

The model was not being perverse. It had seen a million `grep` invocations in training and one harness's search tool in its prompt, and the tool table it had been trained on described the shell as "run a shell command" and never said what not to run through it. The search tool's description said what it did. Nothing said it was preferable to `grep`.

The fix was text. Plank added two short sections to the prompt, one for the shell and one for `git`, each line naming the wrong shell command beside the right native tool, in a shape borrowed from another harness's tool description: the native tool, then in parentheses the shell command not to use. The two sections came to about nineteen hundred bytes, roughly four hundred and eighty tokens, added to a system prompt of about five thousand, and the finding closes with a method for measuring the effect, which is to rerun the count on sessions recorded after the change.

The lesson for a tool's description is that it is not documentation. It is the only lever the harness has over which tool the model picks, and the model picks by familiarity unless told otherwise. Blanka's descriptions are written with this in mind. The shell tool's description says to use the file tools rather than `cat`, `sed`, `find` and `grep`. The glob tool's description says to use it instead of `find` and `ls`. The search tool's description says to use it instead of `grep`. None of that is information about what the tools do. All of it is information about when to use them, and that is what the model was missing.

It matters which route the model takes even when both produce the file's contents, for three reasons in increasing order of weight. The native tools are bounded and the shell is not: a `cat` of a large file puts all of it in the transcript, where the read tool would have windowed it. The native tools skip what should be skipped: `grep` over a Cargo project searches the build directory, which is the next section's finding. And the native tools are where the harness can see and shape what the model does: a read through the read tool can be recorded, compacted later, and shown to the user as a read, where a read through the shell is a string the harness cannot interpret. Every capability the later chapters add to the file tools is a capability the shell path lacks, and the description is what routes the model to the path that has it.

Two audiences for every result

A tool's output has two readers, and they want different things. The model wants the result: the file's lines, the command's output, the confirmation that the edit landed. The user wants to know what happened: which file was read, that an edit changed these lines and not those, that a directory was created they did not ask for. A tool that serves only the first audience produces a session the user cannot follow, and a tool that serves only the second produces a transcript the model cannot use.

Blanka's tools return text for the model and, so far, nothing for the user beyond what the renderer shows of the transcript. Plank's tools return both, and the shape is instructive.

Chapter 3: A tool table worth trusting

A write that creates a missing directory publishes a status line saying so, because directories appearing is a side effect the user did not name. An edit produces a diff card for the display, with the added and removed lines coloured, while the model receives the post-edit context with line numbers; the two are built from the same edit but rendered for their readers. A read records what was read so that the display can show a compact "reading such-and-such" line rather than the file's contents, which the user has on disk and does not need repeated. And a shell command's progress is shown live, because a build that takes a minute is a minute the user is otherwise staring at nothing.

The mechanism for the second audience is the streaming display of chapter 4, and the reason it is deferred is that it needs a sink richer than standard output. What this chapter fixes is the division: the text a tool returns is for the model, and anything the user should see that the model should not is a separate channel. Mixing them, by putting user-facing chatter in the tool result, costs tokens on every pass and teaches the model to expect prose where it expects data. Plank's context has a method for publishing a status line to the display, separate from the result, and blanka's context gains one in chapter 4.

Searching, and what to skip

The search tool finds text in files, and its design is a list of what not to look in.

Version-control metadata is skipped, as it is by every tool that walks a tree. So is `node_modules`, for the same reason: it holds copies of other people's code, and a search that returns matches from it is returning the wrong project. The finding that shaped the tool is about a third kind of directory. When Cargo packages a crate, it leaves a full copy of the crate under the build directory, and a search that skipped only the version-control directory returned that copy of every file first. The two search results mentioned earlier, fifty-four and seventy-one thousand characters, were this: every match duplicated from the packaged copy, a third of the session's prefill spent on them, and the model then spending a turn on the pollution rather than the task.

The fix has a detail that is easy to get wrong. The obvious fix is to skip any directory named `target`. Plank did not do that. It checks for a file the build tool writes into the directory:

```
pub fn should_skip_dir(dir: &Path) -> bool {
    let name = dir.file_name().and_then(|n| n.to_str()).unwrap_or("");
    SKIP_DIRS.contains(&name) || dir.join("CACHEDIR.TAG").exists()
}
```

Cargo writes a `CACHEDIR.TAG` file into its build directory, a convention shared by several tools that produce regenerable caches, and the tag is checked rather than the name so that a source directory that happens to be called `target` is still searched. The finding says so in its last sentence, and the test for the tool builds exactly that situation: a build directory with the tag and a match inside it, a source directory named `target` without the tag and a match inside it, and an assertion that only the second match is returned.

The tool also stops at a maximum number of matches, fifty by default, and says so with a suggestion to narrow the pattern. It skips binary files by looking for a null byte in the

Chapter 3: A tool table worth trusting

first kilobyte, and it skips files over a few megabytes, because a match in a generated data file is almost never what the model wanted and the file's size would dominate the result. Every one of these limits is a judgement about what a model searching a codebase is looking for, and every one says in the result what it did, so the model can override it if the judgement was wrong.

Those limits count matches and file sizes, and plank learned that neither counts the thing that hurts. A literal search for a three-digit number hit two minified SVG files of 238 kilobytes each, every byte of each on a single line. Neither file was large by the size rule and each held one match, so the tool printed both lines in full: a 480-kilobyte result, roughly three hundred thousand tokens, which prefilled for seven minutes at local speed and is part of that session's transcript for good. The reference never gets there. Its search skips any file with a line reaching the tool byte limit of 128 kibibytes and reports that it did, and every C tool writes through a buffer that clips the whole result at the same size with a note telling the model to narrow the request. Plank had neither for search, and since commit 8db0863 has both, scoped to search alone, because plank's read tool bounds by the context window on purpose and adopting the C's global limit wholesale would undo that. The detail that matters most is in the reporting: a skipped file appears in the C's "Search incomplete" footer even when nothing else matched, so the model learns there is a hole in the coverage instead of reading "No matches" and believing it. The same distinction runs through this whole chapter. A bound that says what it cut is a tool the model can reason about; a bound that cuts silently is a tool that lies by omission. Blanka's search has the match cap and the size rule, and a reader whose projects contain generated assets should add the line rule beside them.

Blanka's search matches a literal substring. The reference and plank support more, and a reader extending the tool will want a regular expression option; the shape of the tool does not change, and the description gains a sentence.

Finding files

The glob tool lists files matching a pattern, and it exists mostly so that the model does not run `find`. Its matcher understands two wildcards: a star that matches within one path component and a double star that crosses components, so `src/*.rs` lists the Rust files directly under `src` and `**/*.rs` lists every Rust file in the tree. The distinction matters because a model asked to look at a project's structure will ask for one or the other, and a matcher that treats them the same either floods the result or misses most of it.

The walker never enters version-control directories, stops at a hundred results with a note, and returns paths relative to the working directory, sorted, because a sorted list is one the model can scan and a list of absolute paths wastes tokens on a prefix that is the same for every entry. The tests cover the wildcard semantics and the version-control skip, and the matcher is a pair of small recursive functions with no dependencies.

There is not much more to say about glob, and that is itself worth noticing. Of the six tools, it is the one with the fewest ways to go wrong, and it is the one the model uses least. The tools that cost sessions were the ones that touch content: reading it, changing it, searching it, and running commands over it.

Testing tools against a real file system

The tools in this chapter touch the disk, and testing them raises a question the earlier chapters did not: how to test code whose effect is a file, without the tests interfering with each other or with the developer's machine.

The first answer is to keep as much as possible pure. The edit tool's replacement is a function from a string to a string, and its tests never open a file; they pass in a file's text, including text with two-byte line endings, and assert on the result. The shell tool's head-and-tail cut is a function from a string to a string. The glob matcher is a pair of functions over path components. The spill policy's boundary handling is tested on a string of multi-byte characters. Every one of those tests runs in microseconds and cannot collide with anything, and every one of them tests the logic that a session would have exercised through a file.

The second answer is scratch directories, and they are less simple than they look. Each test that needs a file system creates a directory under the system temporary root, named with the test's own tag and the process id, removes anything already there, builds the files it needs, and runs. Two traps live here. One is that the temporary root is a symbolic link on macOS, where `/var` resolves to `/private/var`, so a test that compares a path a tool returned against the path it constructed fails unless both are canonical. Blanka's containment test hit this on its first run and now canonicalizes before comparing; the tool was right and the test was wrong, which is the better order to discover it in. The other trap is naming. Plank's model-manifest tests built scratch directories named with the process id and a nanosecond timestamp, on the assumption that the pair was unique, and under the test harness's parallelism two tests in the same process landed on the same name in the same nanosecond window, so one test's marker file leaked into the other and failed it for a reason unrelated to what it tested. The fix was an atomic counter folded into the name, so two tests in one process cannot collide even when the clock does not move between them. Blanka's tests use a tag per test, which is a manual version of the same guarantee and is enough while the tests are few; a reader whose table grows should reach for a temporary-directory crate or the counter.

A third trap is the one that reaches past the scratch directory into the developer's machine, and plank's version of it is a warning about the obvious fix. Two tests scoped themselves by pointing `HOME` at a scratch directory and restoring it at the end, and both wrote to the user's real plank directory anyway, taking four unrelated sandbox tests down with them. Half of the cause is the race: the environment is process-global and the test harness runs tests on parallel threads, so while one test held the scratch value every other test resolving a path from `HOME` got it too, which is why the failures moved around and each test passed alone. The other half inverted the tests' intent. Plank resolves its home directory with a fallback to a shared machine-wide directory, and gates that fallback on the root being the process's own `HOME`, precisely so an injected root can never reach machine state. Setting `HOME` to the scratch directory made the gate open again; the scratch directory's plank folder did not exist yet, the shared one did, and the fallback won. One test appended to the real error log. The other deleted the real marker that recorded the user's standing consent to web access. The rule plank now applies everywhere is that a function resolving a path from `HOME` gets a variant taking an explicit

Chapter 3: A tool table worth trusting

root, the tests call that variant, and no test touches the environment. Where the write itself needs testing, the body is split so the resolved path is a parameter and the public function stays a one-line wrapper. Blanka already follows the rule from the other end: its binary reads `HOME` once at startup and passes the resulting paths down, so no tool resolves anything from the environment, and that is worth keeping true as the table grows.

The third answer is to test the model-facing text, not only the behaviour. The containment test asserts the exact error string. The read test asserts the exact truncation notice. The framing test from chapter 2 asserts against a fixture. These look like brittle tests, and they are brittle in the way a parity check is brittle: a change to model-facing text should break a test, because a change to model-facing text is a change to what the model sees, and it should be made deliberately.

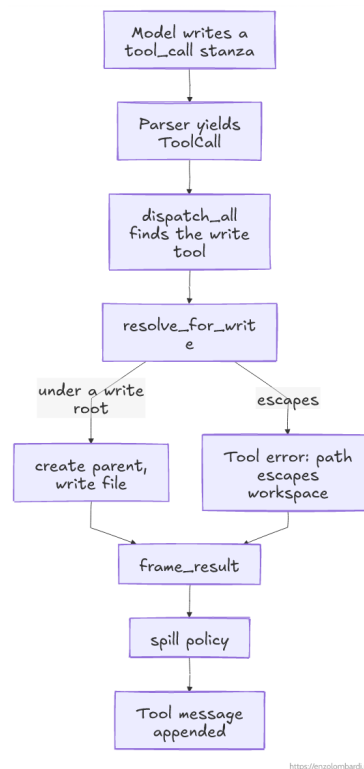
The table, assembled

With the six tools written, the table is one function:

```
pub fn default_set() -> Self {
  Self::new()
    .with(Box::new(read::Read))
    .with(Box::new(write::Write))
    .with(Box::new(edit::Edit))
    .with(Box::new(bash::Bash::default()))
    .with(Box::new(glob::Glob))
    .with(Box::new(search::Search))
}
```

The binary registers this set and hands it to the agent, and the hosted engine receives its descriptions as a registry. The flow of a single write request, from the model's stanza through the loop to the disk, is this:

Chapter 3: A tool table worth trusting



A write request from stanza to disk: containment decides, the frame and spill policy apply to whichever branch was taken.

Every path through that diagram produces a result frame, and every frame passes through the spill policy. The tools themselves know about neither. That separation is what lets the table grow: chapter 12 adds a tool that runs a sub-agent and chapter 13 adds tools that live in other processes, and both get containment, framing and spill without implementing any of them.

What each tool says when it fails

The chapter has mentioned error text repeatedly, and it is worth collecting the rules, because the six tools between them exercise all of them.

An error names the tool and the path or argument that caused it. "read src/main.rs: No such file or directory" tells the model which call failed and why; "file not found" tells it neither. Every tool in the table builds its errors this way, and the shared argument helpers do too: a missing required argument produces "edit requires old", not "missing argument".

An error that can be acted on says how. The edit tool's non-unique anchor error ends with "include more surrounding lines", because that is the fix and the model will apply it. The read tool's truncation notice names the next starting line. The search tool's cap notice suggests narrowing the pattern or raising the limit. The containment error says the path escapes the workspace, which tells the model the constraint it violated rather than leaving it to guess.

Chapter 3: A tool table worth trusting

An error never suggests something the harness does not want. This is the vision-encoder lesson from chapter 2 seen from the tool side. A refusal that mentions a shell command teaches the model to run it. A refusal that says a file is too large and stops there leaves the model to choose between a windowed read and a search, both of which the harness is happy to run.

And a failure inside a tool is still a result, framed and numbered, so the model sees it in the position it expected a result. The dispatcher from chapter 2 ensures this. A tool implementation returns an error, the dispatcher wraps it in the error prefix and the result frame, and the model reads a numbered result whose first line says what went wrong.

These rules cost nothing to follow at the point a tool is written and are hard to retrofit across a table of twenty tools. They are the reason the shared argument helpers exist: a tool that fetches its arguments through them gets consistent error text for free.

Where blanka stops short of plank

Plank's tool module is several thousand lines, and the difference from blanka's few hundred is an inventory worth having.

Plank's read tool handles PDF documents by converting them to text, pages through results with a continuation tool, and records what was read so that compaction can later replace a stale read with a note. Its edit tool supports an anchored form where the old text is a head and a tail with a marker between, so a large span can be replaced by naming only its edges; the edit is echoed to the user as a diff card with the added and removed lines. Its shell tool is the asynchronous job system described above, with a macOS sandbox profile generated from the write roots, a consent prompt for writes to plank's own directory, a read-only command classifier to avoid asking when the command only reads, and a per-dispatch deadline that is off by default. Its search tool takes a regular expression. Its web tools fetch pages through an embedded headless browser. And the whole table is preceded by hooks that can block or observe any call, gated by a plan mode that makes the table read-only until the user approves a plan.

Blanka has the six tools in their direct form, containment, spill, and the line-ending and cache-directory fixes that the findings made non-negotiable. Chapters 11, 12 and 13 add the asynchronous shell, plan mode and hooks, each when the machinery it depends on exists. The rest stays in plank as a reference for what a tool table accumulates when it is used every day against real projects.

The pattern across every addition in that inventory is the one this chapter has followed: a session went wrong in a specific way, the recording showed which tool and how, and the fix was a bounded change to that tool with a test that reproduces the session. A tool table worth trusting is not one designed to be complete. It is one where every trap that has been found has been closed, and the closing is recorded well enough that the next person does not have to find it again.

Six verbs and their traps

Six tools cover most of what a coding agent does, and each has a way of failing that costs more than an ordinary bug, because the failure goes in front of a model that acts on it.

Chapter 3: A tool table worth trusting

Writes are contained through one method that resolves symlinks and missing ancestors before checking the roots, and reads are deliberately not, because a refused read is retried through the shell and a read has no side effect. Results too large for the context are spilled to disk after every dispatch, with a preview and a locator, so no tool has to know.

The read tool windows and numbers, and says how to continue. The write tool creates parent directories rather than refusing, because the target already passed containment. The edit tool requires a unique anchor and adapts line endings when the whole file uses the two-byte form, a fix that removed five failed edits from one forty-minute run. The shell tool has a deadline, drains its pipes on threads, and keeps the head and tail of long output with the exit status first. The search tool skips the build directory by its cache tag rather than its name. The glob tool distinguishes one star from two.

Tool descriptions are not documentation; they are the only lever over which tool the model picks, and a model picks by familiarity unless told otherwise. Error text names the tool and the argument, says what to do, and never suggests what the harness does not want.

CHAPTER 4: STREAMING WITHOUT LYING

Run the harness as it stands against a real model and ask it to fix something, and the screen fills with angle brackets. The model's reasoning, if the model reasons out loud, arrives as a wall of text between two tags. Its tool request arrives as raw JSON between two more. Then, after a pause, the tool's result appears in the transcript and the next reply begins, and the user has watched all of it happen without any indication of what was reasoning, what was a request, and what was the answer. Everything the harness has built so far is correct and none of it is watchable.

The fix is a layer between the engine and the screen that recognizes structure in the text as it arrives, and the word "as it arrives" is where the difficulty lives. A tag can be split across two chunks. A tool call's name is not known until its JSON body is complete. The model's reasoning may or may not be something the user wants to see. And at any moment the user may decide they have seen enough and press a key, at which point the harness has to stop the engine and leave the transcript in a state that truthfully records what happened. Plank's port of this layer is a thousand lines and the source of several findings. Blanka's is smaller, and it carries the same obligations, which this chapter names in its title: the display may hide things and summarize things, but it must never show the user something the model did not produce, or record in the transcript something the user did not see.

Two readers of the same stream

Chapters 2 and 3 established that a tool's output has two audiences, the model and the user, and that the text for each should be kept apart. The model's output has the same two audiences, and the same rule applies from the other direction.

The transcript needs the model's output verbatim. Every byte, including the reasoning and the raw stanza, in the order produced, because the transcript is what the model sees on the next pass and what a replay reproduces. A transcript that recorded a cleaned-up version of the reply would be showing the model a conversation that did not happen, and chapter 1's log-everything rule forbids it.

The user needs something else. The user needs to see the answer as prose, the reasoning as visibly distinct from the answer or not at all, and a tool call as a one-line announcement of what is about to run rather than as JSON. The user also needs all of this while it is happening, not after, because a model's reply takes seconds to minutes to arrive and a blank screen for that long reads as a hang.

Plank's stream renderer describes its own role in a sentence that draws the line exactly: it rewrites the terminal projection, never the transcript. The transcript is the record; the display is a projection of it, computed on the fly, that may hide and may summarize but that is derived from the same bytes. Blanka adopts the same division, and the structure of this chapter follows from it. One component receives raw text from the engine, keeps a verbatim copy for the transcript, and forwards classified pieces to a display. The display is a trait with several channels. The classification is a state machine. And the pieces fit together such that the agent loop from chapter 2 changes in one place.

What the engine emits

Chapter 1 gave the engine a sink with a single method that receives text, and said the richer version would arrive when there was more than one kind of thing to deliver. There is now, and the design question is where the richness goes.

Plank puts it in the engine's output. Its engine emits an enum of events: a text piece, a prefill progress update, a human-facing notice about something the engine is doing such as rebuilding a cache, and speculative-decoding counters. The engine knows about all of these because the engine is where they happen, and a front end that does not care about a kind of event ignores it. The text event's documentation carries a warning that matters for everything below: a text piece may split UTF-8 across pieces.

Blanka keeps the engine's sink as it was, a single text channel, and puts the richness on the display side instead. The reasoning is about what each component knows. The engine knows that bytes came out; it does not know whether they are reasoning or a request, because that is a property of the text's shape and the engine did not read the tool descriptions. The display knows what the user wants to see. Between them sits a component that reads the shape, and that component is what this chapter builds. Plank's prefill progress and cache notices are real and useful and belong to the local engine of the appendix, where the engine can report them; for a hosted engine there is no prefill progress to report, only the wait for the first byte.

The display trait, then, is a set of channels for classified text:

```
pub trait RenderSink {
    fn visible(&mut self, text: &str);
    fn thinking(&mut self, text: &str);
    fn tool_banner(&mut self, text: &str) {
        self.visible(text);
    }
    fn error(&mut self, text: &str) {
        self.visible(text);
    }
    fn status(&mut self, text: &str) {
        self.visible(text);
    }
}
```

Two channels are required and three have defaults, and the split is deliberate. Every display has to say what it does with visible text and with reasoning; a display that showed reasoning as if it were the answer would be lying to the user about which words the model stands behind. Banners, errors and status lines are refinements: a display that cannot style them routes them through the visible channel and loses nothing but emphasis. Plank's equivalent trait has the same shape, with visible and thinking text required and tool text and error text defaulting, plus a channel for a single-line status that replaces itself in place, used for a live counter while a large write streams. The defaults are what let a minimal sink exist in a few lines, and a minimal sink is what the tests use.

Chapter 4: Streaming without lying

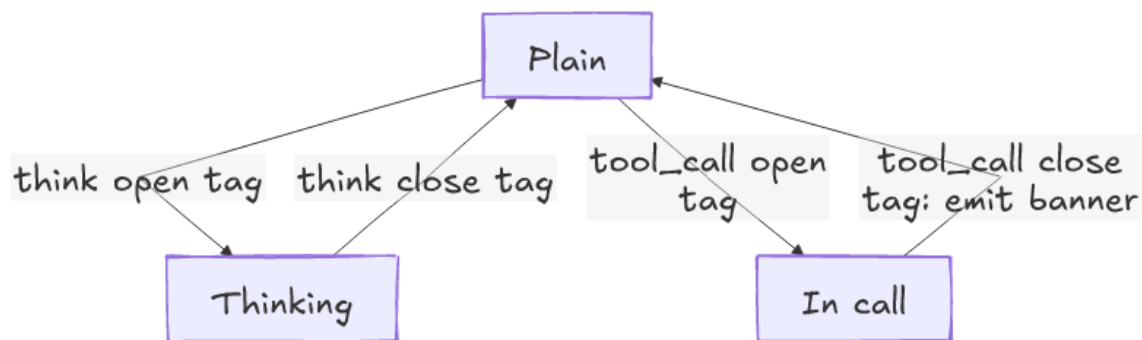
The test sink collects each channel separately:

```
pub struct Channels {  
  pub visible: String,  
  pub thinking: String,  
  pub banners: Vec<String>,  
  pub errors: Vec<String>,  
  pub status: Vec<String>,  
}
```

It is the display-side counterpart of chapter 1's collecting sink, and it makes the chapter's central assertion writable: after a pass, the visible string contains no tag, the banners contain exactly the calls the model made, and the transcript contains the tags anyway.

The state machine

The renderer sits between the engine's sink and the display, implementing the first and holding the second. It is a state machine with three states, and the states are what the text is currently inside of: plain text, a reasoning block, or a tool-call stanza. The transitions are the tags:



<https://enzolombardi.net>

The renderer's three states. Text that is not a tag stays in the current state and goes to that state's channel; a closing stanza becomes a banner.

In the plain state, text goes to the visible channel until an opening tag appears. In the thinking state, text goes to the thinking channel until the closing tag. In the call state, text is accumulated and nothing is shown, until the closing tag arrives and the accumulated body is turned into a banner. The reply's raw bytes are appended to a buffer in every state, because the transcript wants all of them.

The structure of the type reflects the three things it has to keep:

```
pub struct StreamRenderer<S: RenderSink> {  
  sink: S,  
  state: State,  
  carry: String,  
  call_buf: String,  
  raw: String,  
}
```

Chapter 4: Streaming without lying

```
    show_thinking: bool,  
}
```

The raw buffer and the call buffer have been explained. The carry buffer is the one that makes streaming honest, and it deserves its own section.

The transitions are written as a loop that consumes text until none is left, and it is short enough to show whole:

```
fn consume(&mut self, mut text: &str) {  
    while !text.is_empty() {  
        match self.state {  
            State::Plain => {  
                let think_at = text.find(THINK_OPEN);  
                let call_at = text.find(OPEN);  
                let next = match (think_at, call_at) {  
                    (None, None) => None,  
                    (Some(t), None) => Some((t, State::Thinking, THINK_OPEN.len())),  
                    (None, Some(c)) => Some((c, State::InCall, OPEN.len())),  
                    (Some(t), Some(c)) if t <= c => Some((t, State::Thinking, THINK_OPEN.len())),  
                    (Some(_), Some(c)) => Some((c, State::InCall, OPEN.len())),  
                };  
                let Some((at, state, tag_len)) = next else {  
                    self.sink.visible(text);  
                    return;  
                };  
                self.sink.visible(&text[..at]);  
                if state == State::InCall {  
                    self.call_buf.clear();  
                }  
                self.state = state;  
                text = &text[at + tag_len..];  
            }  
            State::Thinking => match text.find(THINK_CLOSE) {  
                None => {  
                    if self.show_thinking {  
                        self.sink.thinking(text);  
                    }  
                    return;  
                }  
                Some(i) => {  
                    if self.show_thinking {  
                        self.sink.thinking(&text[..i]);  
                    }  
                    self.state = State::Plain;  
                    text = &text[i + THINK_CLOSE.len()..];  
                }  
            },  
            State::InCall => match text.find(CLOSE) {
```

Chapter 4: Streaming without lying

```
None => {
    self.call_buf.push_str(text);
    return;
}
Some(i) => {
    self.call_buf.push_str(&text[..i]);
    let banner = describe_call(&self.call_buf);
    self.sink.tool_banner(&banner);
    self.state = State::Plain;
    text = &text[i + CLOSE.len()..];
}
},
}
}
```

In the plain state both opening tags are searched for and the earlier one wins, so a reply that opens a tool call and later reasons, or the reverse, is classified in the order the model wrote it. Each arm either consumes all remaining text and returns, or consumes up to and including a tag, changes state, and goes round again with the rest. A chunk containing a complete reasoning block followed by a complete stanza followed by prose passes through the loop four times and leaves the machine in the plain state with three channels written.

Two things the loop does not do are as important as what it does. It does not look inside a reasoning block for a tool-call tag; a stanza that opens inside reasoning is shown as reasoning, and the executable parser from chapter 2 will find it in the raw text and execute it, which is a behaviour the next section revisits. And it does not attempt to recover from a closing tag with no opening tag; a stray close in the plain state is ordinary text and is shown, because the model wrote it and the display has no grounds to hide it.

The tag that has not finished arriving

A model emits tokens, and a tag is several tokens. The opening tag of a tool call might arrive as `<tool_` in one chunk and `call>` in the next, or as `<` alone followed by everything else. A renderer that looked for whole tags in each chunk would show the first half as prose and then, on seeing the second half, have no way to recall it. The user would see a stray `<tool_` in the answer and the state machine would never enter the call state, so the JSON that followed would print as prose too. Chapter 1's stub streams in seven-byte pieces for precisely this reason: it is a test that the harness does not assume chunks align with anything.

The solution is to hold back any text at the end of a chunk that could be the beginning of a tag, and classify it only when the next chunk arrives and either completes the tag or shows that it was ordinary text. The held-back text is the carry:

```
pub fn feed(&mut self, chunk: &str) {
    self.raw.push_str(chunk);
    let mut text = std::mem::take(&mut self.carry);
```

Chapter 4: Streaming without lying

```
text.push_str(chunk);
let keep = dangling_prefix(&text);
let (now, later) = text.split_at(text.len() - keep);
self.consume(now);
self.carry = later.to_string();
}
```

The function that decides how much to hold back checks, for each tag the renderer knows, whether the text ends with a proper prefix of that tag, and keeps the longest such prefix. A chunk ending in `<tool_` holds back six characters. A chunk ending in `<` holds back one, because a single angle bracket might begin any of the tags. A chunk ending in `< b` holds back one as well, and when the next chunk shows that the bracket was followed by a space rather than a tag name, the bracket is released as visible text. A test asserts exactly that case, because a renderer that swallowed comparison operators in code would be a renderer nobody could use for programming.

The raw buffer is updated before any of this, with the chunk as received. The carry is a display concern: it delays what the user sees by at most the length of the longest tag, and the transcript is never delayed at all.

When the stream ends, the carry is flushed according to the state the renderer is in. Plain text is shown. Reasoning is shown or not according to the setting. A carry inside an unfinished stanza is appended to the stanza buffer and an error is reported to the display, because a reply that ended inside a tool call is a reply that was cut off, and the user should know that what they did not see was a request rather than an answer. The executable parser from chapter 2 will make the same determination from the finished text and feed the model an error; the renderer's job is only to tell the user.

Plank's renderer keeps two carry buffers rather than one, and for a different reason: they hold bytes, not characters, so that a multi-byte character split across two engine pieces is never emitted partially. Blanka's engine sink delivers string slices, which Rust guarantees are valid UTF-8, so the byte-level carry is unnecessary at this layer and the stub's gentle behaviour of dropping a piece that would split a character, noted in chapter 1, is the cost of that simplification. A local engine that delivers raw bytes, as the appendix's does, needs the byte carry in front of the renderer, and it is a dozen lines.

Four tests that specify the renderer

The renderer's tests are short, and they are the specification of the chapter's promise, so each deserves a sentence about what it rules out.

The first feeds a reply in three chunks with both tags of a tool call split across chunk boundaries, and asserts that the visible text contains neither tag nor JSON, that exactly one banner was produced with the tool's name and argument, and that the raw reply returned for the transcript still contains the stanza verbatim. It rules out a renderer that looks for whole tags per chunk, and it rules out a renderer that cleans the transcript.

The second feeds a reply with a reasoning block twice, once with the flag on and once with it off, and asserts that the thinking channel received the reasoning in the first case and nothing in the second, that the visible channel received only the answer in both, and

Chapter 4: Streaming without lying

that the raw reply is identical in both. It rules out a renderer whose setting changes the record.

The third feeds `a < b` and `c <` followed by `d`, and asserts the visible text is the concatenation with nothing missing. It rules out a carry that swallows a bracket that turned out not to begin a tag, which is the failure that would make the renderer unusable for anyone whose model writes code.

The fourth feeds an opening tag and half a body and then ends the stream, and asserts that the display received one error and no banner. It rules out a renderer that shows a half-written stanza as a call, or that says nothing when a reply was cut off inside one.

None of the four takes longer than a microsecond, and together they pin the state machine, the carry, the flag and the end-of-stream handling. A fifth test lives in the agent module and drives the whole loop: a scripted reply containing prose and a stanza, through the agent, into a collecting display, with assertions that the visible channel has the prose and no tag and the banners have the call. That one rules out a wiring mistake between the renderer and the loop, which is the mistake plank's sub-agent finding describes.

What the user sees of a tool call

When a stanza closes, the accumulated body is a JSON object, and the renderer turns it into a line:

```
pub fn describe_call(body: &str) -> String {
  let Ok(v) = serde_json::from_str::<serde_json::Value>(body.trim()) else {
    return "tool call (unparseable)".to_string();
  };
  let name = v["name"].as_str().unwrap_or("?");
  let mut out = name.to_string();
  if let Some(args) = v["arguments"].as_object() {
    for (k, val) in args {
      let s = match val {
        serde_json::Value::String(s) => s.clone(),
        other => other.to_string(),
      };
      let s: String = s.chars().take(60).collect();
      let s = s.replace('\n', " ");
      out.push_str(&format!("{}", {k}={s}"));
    }
  }
  out
}
```

The name, then each argument's key and a truncated value, on one line. A read of a file becomes `read path=src/main.rs`. A write of a two-hundred-line file becomes `write path=src/new.rs content=use std:... with the content cut at sixty characters`, because the user wants to know that a file is being written and where, not to read it in the banner. An unparseable body becomes a banner that says so, because the display's

Chapter 4: Streaming without lying

job is to report what the model did, and what it did was write something that is not a request.

Two details of plank's banners are worth carrying as design notes. The first is that banner text goes to the display through a channel that is never treated as formatted text. Plank's terminal sink routes visible text through a renderer that interprets it as markdown, so that the model's answer shows with headings and code blocks styled, and the sink's comment on the tool-text channel explains why banners bypass it: a parameter value such as a glob pattern contains asterisks and underscores, and a markdown renderer would eat them. Blanka's plain sink does no markdown yet, so the distinction is latent, but the channel exists so that the appendix's richer display can make it.

The second is that plank's renderer does more than announce. For a read, it shows a compact line naming the file and the line range. For an edit, it streams the old and new text as diff lines with plus and minus prefixes, coloured, as the model writes them, so the user watches the change take shape. For a write, it shows the first few lines of the content dimmed and then a live counter of lines written, collapsing the rest. And for an edit, it does something that is only possible because the renderer is incremental: the moment the old-text parameter closes, before the model has begun writing the new text, the renderer hands the old text to a preflight check that looks for it in the file, and if the anchor is missing or ambiguous, generation stops right there. The model is spared writing a replacement for an edit that was going to fail, the user is spared watching it, and the error goes back after a fraction of the tokens. The reference agent has the same mechanism, and it is a good example of what an incremental view of the stream buys beyond display.

Blanka's banner is a summary line, and the per-tool visualizations belong to the appendix on the full-screen display. What the chapter fixes is the channel and the principle: a tool call is shown as what it is, not as what it looks like.

Formatted text and the channels that bypass it

Blanka's plain display writes visible text as it arrives, and a model's answer is usually markdown: headings, lists, fenced code blocks. A display that interprets the markdown shows a better answer, and every serious harness does it, so the shape of the problem is worth describing even though the chapter's display does not solve it.

The first difficulty is that markdown is a document format and the display receives a stream. A heading is recognizable only when its line ends. A fenced code block is open from its first fence to its second, and the second may be thousands of tokens away. A renderer that interprets markdown incrementally has to hold state about what is open, and it has to decide what to show while a construct is incomplete. Plank's terminal path does this with a token renderer that tracks fence state and applies syntax highlighting inside fences as lines complete, and its full-screen path renders markdown into a widget after each chunk. Both are real work and both are downstream of the stream renderer described here, which hands them classified text and nothing else.

The second difficulty is the one that gives the display trait its channels. Not everything that reaches the display is markdown. A tool banner is a line of harness text with a parameter value in it, and parameter values contain characters that markdown treats as

Chapter 4: Streaming without lying

formatting: a glob pattern is full of asterisks, a path is full of underscores, a shell command may contain backticks. Plank's terminal sink routes banner text around the markdown renderer for exactly this reason, and its comment says so: going through the markdown path would eat the asterisks and backticks out of parameter values. An error banner is routed the same way and coloured red. The channels exist so that a display can make this distinction, and a display that does not interpret markdown loses nothing by ignoring it.

The third difficulty is fences that span the reasoning boundary. A model may open a code block in its reasoning, close the reasoning, and continue the code block in its answer, or the reverse. A display that resets its fence state at the reasoning boundary shows the second half unformatted or, worse, treats the rest of the answer as code. Plank's renderer feeds every byte of the stream to its fence tracker, including reasoning, so that a fence's open-and-close parity survives the boundary, while it skips tag matching inside reasoning. It is the kind of detail that appears only in a session with a real model, and it is recorded in the source rather than in the findings because it was fixed the same day.

Blanka's display defers all three to the appendix on the full-screen display, and the trait is what makes deferring possible. The renderer in this chapter would not change; the sink would.

A stanza inside the reasoning

The state machine above treats a tool-call tag inside a reasoning block as reasoning. Whether that is right is a genuine design question, and plank and the reference answer it one way while blanka, for the moment, answers it another.

A model that reasons out loud will sometimes draft a tool call in its reasoning, as a way of thinking through what it wants to do, and then either emit the real call after the reasoning closes or decide against it. If the harness executes the draft, it has executed something the model was considering rather than something the model decided, and the model then sees a result for a request it did not make. The reference agent's rule is that tool calls inside reasoning are not allowed: a stanza that opens there is discarded with a notice in the display, and on the next pass the model is told, in the reference's exact words and as a fixture-pinned string, that tool calls are not allowed inside the reasoning tags and it should finish thinking before emitting one. Plank keeps that rule and its text, and adds a setting that instead dispatches such a call as ordinary, off by default, for users whose models reliably mean what they draft.

The interesting part of plank's version is what it tells the model. The parser, on discarding the stanza, has a verdict about it, malformed or incomplete or well-formed. The model is not told the verdict. It is told the placement rule, because the placement is what it got wrong, and a model told "your stanza was malformed" when the stanza was fine would try to fix the syntax and emit the same stanza in the same place. Chapter 2's principle that error text is an instruction the model will follow applies with particular force here: the instruction has to name the actual mistake.

Blanka's renderer does not look inside reasoning for stanzas, and blanka's executable parser does not know about reasoning tags at all, so a stanza drafted inside reasoning is shown to the user as reasoning and then executed by the parser. This is the simpler

Chapter 4: Streaming without lying

behaviour and it is the wrong one for a model that drafts, and the book leaves it wrong until chapter 10, where the loop guards need to watch the reasoning stream anyway and the reasoning-aware parsing arrives with them. The reason to defer is evidence: whether the reference model drafts calls in its reasoning often enough to matter is something to measure against sessions before building the rule, and chapter 10 is where the sessions are.

When the banner appears

Blanka's banner appears when the stanza closes, because the banner is built from the parsed JSON body and the body is not parseable until it is complete. For a read, that is a fraction of a second after the model began the stanza. For a write of a large file, it is however long the model takes to emit the file's content, which can be tens of seconds, during which the display shows nothing for the call.

Plank's renderer does better because it parses the stanza incrementally. The reference's markup puts the tool name in the opening tag of each invocation, so the name is known the moment that tag closes, and plank shows the banner then, before any parameter has arrived. For a write, it then streams the content dimmed under the banner, and for an edit it streams the old and new text as diff lines. The user sees what the model is doing from the first token of the call rather than from the last.

The JSON body that blanka's dialect uses puts the name first too, as a matter of convention, and a renderer that wanted the banner early could parse the body incrementally, watching for the name member to close. It is more machinery than the book wants in a chapter that is about the separation of display from execution, and the delay is invisible for every tool except a large write. The appendix on the full-screen display takes the incremental route when it adds the write preview, and this is noted here so that a reader who watches a large write go by with a silent display knows the silence is a known cost and where the fix lives.

Reasoning, shown or hidden

Some models reason before answering, and emit the reasoning between tags so that a harness can tell it apart from the answer. Whether the user wants to see it depends on the user and on the moment. Reasoning is often the most interesting part of a session to watch and the least useful part to read afterwards; it can run to thousands of words for a short answer; and it is where a looping model loops, which chapter 10 will make much of.

The renderer separates it unconditionally and displays it conditionally. The separation happens in the state machine: text inside the tags goes to the thinking channel and text outside goes to the visible channel, whatever the setting. The display decision is a flag consulted at the moment of forwarding:

```
State::Thinking => match text.find(THINK_CLOSE) {
  None => {
    if self.show_thinking {
      self.sink.thinking(text);
    }
  }
}
```

Chapter 4: Streaming without lying

```
    return;  
  }  
  // ...  
}
```

The transcript gets the reasoning regardless, because the model produced it and the model will see it on the next pass. The test that pins this feeds a reply with a reasoning block through a renderer with the flag off and asserts three things: the thinking channel is empty, the visible channel holds only the answer, and the raw reply returned for the transcript holds the tags and the reasoning in full.

Plank has a finding about exactly this flag, and it is a finding about defaults rather than about logic. A new renderer showed reasoning unless told not to, and the main display paths told it. Then a sub-agent, running its passes on a separate thread and building its own renderer, did not, so the sub-agent's pane showed the model's reasoning while the main log hid it. The same gap appeared again for the banner flag and the status-line flag: the sub-agent's pane showed raw banners and uncoloured diff lines where the main log showed a formatted diff card, because the second renderer had the defaults and the first had the settings. The finding closes with a rule that generalizes well beyond plank: whatever the main path configures on its renderer, every other path that constructs one must mirror, or the two views silently drift. Plank now carries the display flags together in one struct that travels with the pass, and records the switches in effect in the header of every saved session, so that a session dump says what the user was actually looking at.

Blanka has one renderer construction site, in the agent loop, and the flag comes from a field on the agent that the binary sets from the command line. That is the arrangement the finding recommends, and it is worth keeping as the number of sites grows: chapter 12's sub-agents will construct renderers of their own, and they will take the flags from the same place.

There is one more thing plank does with hidden reasoning that is worth knowing. When reasoning is hidden, the display shows nothing at all while the model thinks, which for a long think is indistinguishable from a hang. Plank's renderer takes the first sentence of the hidden reasoning, capped at a hundred and sixty characters and cut at a word boundary, and shows it as a status line, so the user sees "Looking at how the parser handles the closing tag" rather than a blank line for forty seconds. It is a small thing and it changes how a long pass feels.

The cost of showing everything

A display that flushes after every chunk does a system call per chunk, and a model produces tens of chunks per second. That sounds like a cost and it is not one. A terminal write of a few bytes takes microseconds, the model produces a chunk every few tens of milliseconds at most, and a display that batched writes to save system calls would be saving a thousandth of the time it has available while adding latency the user can see. The plain sink flushes every time, and chapter 1 explained why: standard output is line-buffered, and a model's fragments do not end on line boundaries.

Chapter 4: Streaming without lying

The cost that does matter is on the other side, and it is the one plank's frame finding describes later in this chapter: a display that redraws proportionally to the transcript's length on every chunk gets slower for the whole session. A plain sink cannot have this problem, because it never redraws. A full-screen display can, and the finding is about a full-screen display that did.

There is a third cost that is easy to overlook, and it is what the reasoning flag is for. Reasoning can be several times the length of the answer, and a display that shows it is scrolling several screens of text past the user for every reply. Plank's default is to hide reasoning, and to show a one-line status taken from it instead, so that the user sees the model is working and what it is working on without reading its working. When reasoning is hidden and the user wants it anyway, plank can mirror the whole raw stream to a separate debugging console rather than into the main display, so the reasoning is available in a window of its own and the main log stays readable. The design point is that the reasoning's cost to the user is attention, not bytes, and a display should spend the user's attention on the answer.

Stopping

A user watching a reply go wrong wants to stop it, and the harness has to make stopping both possible and truthful. Possible means the engine notices promptly. Truthful means the transcript afterwards records exactly what the engine produced, no more and no less, and the display and the transcript agree.

The mechanism is a flag the engine polls between chunks:

```
pub struct InterruptFlag(Arc<AtomicBool>);

impl InterruptFlag {
    pub fn install_ctrlc(&self);
    pub fn request(&self);
    pub fn is_set(&self) -> bool;
    pub fn clear(&self) -> bool;
}
```

Ctrl-C sets it from a signal handler, and the handler does nothing but an atomic store, because an atomic store is the only thing that is safe to do in a signal handler and it is all that is needed. A front end can set it from a key press through the same method a signal uses. Plank's interrupt module has exactly this shape and says why the second entry point exists: the full-screen display reads keys itself, so an Escape pressed during a long command has to reach the generation loop by the same route Ctrl-C takes, and the loop polls one flag and should not care which of the two set it.

The engine trait gains the flag as a parameter, and both engines poll it. The stub checks between its seven-byte pieces. The hosted engine checks between lines of the event stream, and stopping is a matter of dropping the reader, which closes the connection, at which point the server stops generating on its own because its client has gone. A local engine checks between tokens, and stopping is a matter of not asking for the next one.

Chapter 4: Streaming without lying

The truthfulness requirement is about what happens next. An interrupted pass returns the text it produced up to the interruption, with a flag in the accounting saying so:

```
pub struct GenStats {  
    // ...  
    pub interrupted: bool,  
}
```

This is a deliberate choice over returning an error. The `Interrupted` variant from chapter 1 still exists, for a pass that is stopped before any text is produced, but a pass that is stopped mid-stream has produced text, the user has seen that text, and the transcript must contain it. Returning it as a result rather than an error is what makes that natural: the agent appends the partial reply exactly as it appends a complete one, then appends a note that the reply was interrupted, and ends the turn.

```
self.transcript.push(Role::Assistant, reply.clone());  
if stats.interrupted {  
    self.transcript.push(Role::User, "[The previous reply was interrupted by the user.]");  
    sink.status("[interrupted]");  
    report.interrupted = true;  
    self.interrupt.clear();  
    break;  
}
```

The note is a user-role message, and it exists so that the model, on the next turn, knows why its last reply stops mid-sentence. Without it the model would see its own truncated output and try to make sense of it, sometimes by continuing where it left off, sometimes by apologizing for an error it did not make. With it the model has an explanation, and the explanation is true.

The test for this drives the agent with a display that requests the interrupt as soon as it receives its first visible text, and then checks that the assistant message in the transcript is byte-identical to what the display received, that it is shorter than the scripted reply, that the note follows it, and that the display got a status line. The stub's chunking makes this test deterministic: the first chunk is seven bytes, the interrupt lands after it, and the transcript holds seven bytes.

The flag is cleared at the start of every turn and again after an interrupt is handled. The first clear is the one that matters and is easy to forget: a Ctrl-C pressed at the prompt, between turns, would otherwise be waiting when the next turn began and cut off its first chunk. Plank's interrupt handling distinguishes further between a pass the user stopped and a pass that stopped itself at a tool call, because the two need different messages and the flag alone cannot tell them apart; that distinction arrives in chapter 10 with the guards that stop passes.

Stopping a tool, not only a pass

The interrupt flag stops the engine between chunks, and the agent checks the result after each pass. There is a gap in that: a tool that is running when the user presses the key. A

Chapter 4: Streaming without lying

shell command that has been running for a minute is not producing engine chunks, so nothing polls the flag until it finishes, and a user who pressed Ctrl-C during a slow build sees the build continue.

Blanka's shell tool from chapter 3 has a deadline and nothing else, so an interrupt during a command waits for the command or its deadline, and then the agent sees the flag and ends the turn. The flag is checked at the top of the next pass, so the interrupt is not lost; it is late. Plank closes the gap: a pending interrupt kills the running command's whole process group and reports an exit status of one hundred and forty-three, the conventional code for a process terminated by the termination signal, and the command's result says it was interrupted. The user guide states the guarantee plainly: interrupting a running command kills its group, and a pipeline or a sleep cannot outlive the harness.

Two design points from plank's version are worth carrying forward. The first is process groups. A shell command that starts children, a build that spawns compilers, a test runner that spawns test binaries, is not stopped by killing the shell; the children keep running with no parent and no one watching them. Every job in plank runs in its own process group so that killing the group kills the tree. The second is that the interruption is reported to the model as a tool result, with the exit status and a note, rather than silently. The model asked for a command and the command was stopped; the model should know, because its next step depends on whether the tests ran.

Blanka's shell tool gains process-group handling and interrupt polling in chapter 11, when commands become background jobs and the agent has an event loop that can observe them. Until then the gap is documented here so that a reader who presses Ctrl-C during a build knows what to expect.

What an interrupted transcript looks like later

A transcript that ends a turn with a partial assistant message and a note is a slightly odd object, and it is worth thinking through what happens to it afterwards, because two later chapters depend on it behaving well.

On the next turn the model sees its own cut-off reply, the note explaining the cut, and the user's new prompt. This is the truthful state and it is also a workable one: models handle "you were interrupted" gracefully when told, and badly when not. The alternative of deleting the partial reply would leave the model with no record of having said anything, which is a lie about the conversation and also loses whatever useful work the partial reply contained, since a user often interrupts a reply that was going right but going long.

When the transcript is saved and replayed in chapter 5, the partial reply replays as a partial reply, and the display shows it as it was shown live, followed by the note. Nothing in the replay path needs to know about interrupts, because the transcript contains the whole story.

When the engine's cache enters in chapter 8, the partial reply is a prefix of what the engine produced, and a prefix the engine has already computed the cache for. Ending the pass at an interrupt and recording exactly the produced text means the cache and the

Chapter 4: Streaming without lying

transcript agree at the point of the cut. A harness that recorded something other than what the engine produced, a cleaned-up version, or the full reply the engine would have gone on to produce, would put the cache and the transcript out of step at that point and pay a full re-read on the next pass. Truthfulness here is not only a courtesy to the user; it is what keeps the expensive machinery of Part II consistent.

The note itself is a user-role message and not a system one, and the choice is deliberate. Plank's log-everything rule says that anything injected at request time is logged as a transcript entry so that a replay sees it, and a user-role entry is the form the model was trained to read as coming from outside itself. Chapter 6 introduces a second kind of injected text, the reminder that fires under context pressure, and it uses the same form for the same reason.

The loop, changed in one place

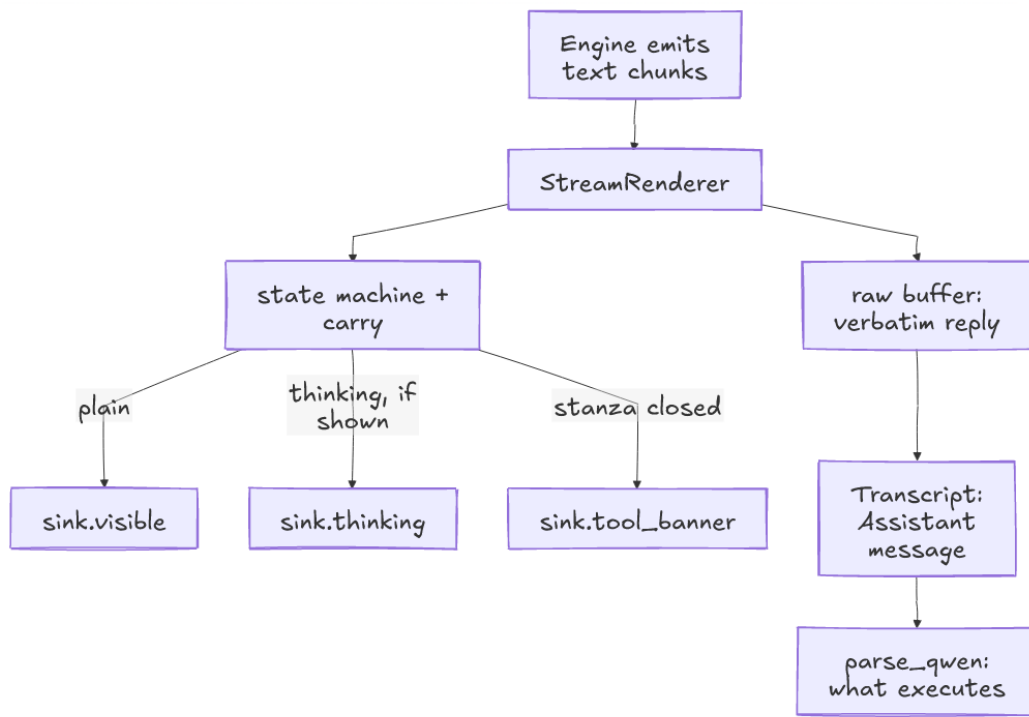
With the renderer, the display trait and the flag in hand, the agent loop from chapter 2 gains a renderer per pass and loses nothing else:

```
let mut renderer = StreamRenderer::new(&mut
    *sink).show_thinking(self.show_thinking);
let (_, stats) = self.engine.generate(
    Prompt::Structured(self.transcript.messages()),
    &mut renderer,
    &self.interrupt,
)?;
let (reply, _) = renderer.finish();
```

The renderer borrows the display for the duration of the pass, receives the engine's text through the sink trait, and returns the verbatim reply when the pass ends. The text the engine returned directly is discarded in favour of the renderer's copy, and the two are the same bytes; the renderer's is used because the renderer is the component that promises to have seen everything. One renderer per pass rather than one per turn means the state machine starts clean for every reply, so a stanza left open by an interrupted pass cannot leak its state into the next.

The loop also gains two uses of the display's other channels. After a dispatch, the agent drains the notices that tools left in the context, the created-directory and overwrote-file messages that chapter 3 said belonged in a separate channel, and sends them to the status channel. And on a malformed stanza, the agent sends the display an error, so the user sees that something went wrong at the moment it went wrong, alongside the error text that goes back to the model. The two texts differ: the user's says what happened, the model's says what to do.

The complete flow for one pass, from engine to screen and transcript, is this:



<https://enzolombardi.net>

One pass through the renderer: the verbatim reply reaches the transcript and the parser, while the display receives a projection.

The parser at the bottom of the diagram is the one from chapter 2, and it reads the raw reply, not the projection. This is the separation the chapter promised: the component that decides what executes never has a partial view, and the component that decides what to show never decides what executes.

The renderer borrows the display rather than owning it, and the borrow is what makes one renderer per pass cheap. A blanket implementation lets a mutable reference to any display act as a display:

```

impl<S: RenderSink + ?Sized> RenderSink for &mut S {
    fn visible(&mut self, text: &str) {
        (**self).visible(text);
    }
    // ... the other four channels forward the same way
}
  
```

With it, the agent constructs a renderer over a reborrow of its display, drives the pass, calls `finish` to recover the raw reply, and the borrow ends. The display outlives every renderer and keeps whatever state it has, scrollbar or cursor position or a count of lines written, across passes and turns. Without the blanket implementation the agent would have to give the display to the renderer and take it back, which is possible and clumsy, or wrap it in shared ownership, which is heavier than the problem deserves. This is a Rust idiom worth knowing in general: a trait implemented for a mutable reference to

Chapter 4: Streaming without lying

itself lets a short-lived component borrow a long-lived one without either side knowing about the arrangement.

The display, plain

The plain terminal display is the second sink in the crate and the one the binary uses:

```
impl RenderSink for AnsiSink {
    fn visible(&mut self, text: &str) {
        self.put("", text);
    }
    fn thinking(&mut self, text: &str) {
        self.put(DIM, text);
    }
    fn tool_banner(&mut self, text: &str) {
        self.put(BOLD, &format!("{}", "\n[tool] {text}\n"));
    }
    fn error(&mut self, text: &str) {
        self.put(RED, &format!("{}", "\n{text}\n"));
    }
    fn status(&mut self, text: &str) {
        self.put(DIM, &format!("{}", "\n{text}\n"));
    }
}
```

Visible text as it is, reasoning dimmed, banners bold on their own line, errors red, status dimmed. Every write flushes, for the reason given in chapter 1. The escape sequences are the four that every terminal has understood for decades, and the sink takes a flag to omit them when the output is not a terminal, so that a session piped to a file does not fill with control characters.

It is a small sink and it is the right size for the book's binary. Plank's plain-terminal path is a markdown renderer with syntax highlighting, gray reasoning, and a status footer, and its full-screen path is a Ratatui application with scrollbar, mouse selection and a two-row status bar. Both implement the same trait. That is the point of the trait, and the appendix that builds the full-screen display adds a sink and changes nothing above it.

A finding about frames

Plank's full-screen display produced a finding that is not about streaming at all, and belongs here because it is about what happens when a display forgets that the transcript is large.

Mouse selection in long sessions became sticky. The cause was that every frame redrew from the whole transcript: the display converted its entire output log to text, cloned it to count lines, and then re-wrapped every line above the visible window one at a time, on every redraw. A mouse drag emits an event per pixel of movement and each event redraws, so the cost showed on the interaction that redraws fastest, and scrolling had the

Chapter 4: Streaming without lying

same problem less visibly. Measured in a debug build, a frame went from twenty-six milliseconds at a thousand lines to over a second at forty thousand.

The fix was a cache of per-line wrapped heights, keyed by the display width, extended lazily and invalidated wherever the log is truncated, so that rendering clones only the lines at or below the viewport. Frame cost went flat: a hundred and forty-four microseconds at a thousand lines, a hundred and ninety-one at forty thousand. The finding records the rule that keeps it that way, every truncation of the log must invalidate the cache from that point, and notes that a stale height does not crash but silently scrolls to the wrong row, which is why a test pins the cache against the naive full re-wrap.

The lesson for a display of any kind is that the transcript grows without bound over a session and the display must not do work proportional to it per event. Blanka's plain sink writes each chunk once and never looks back, so it has the property trivially. A display with scrollbar does not have it trivially, and the finding is the warning.

Sinks that are not screens

The display trait has one more use that the chapter should name, because it shapes later chapters. A sink does not have to be a screen.

Chapter 12's sub-agents run passes whose output the user should not see inline, because a sub-agent's job is to return a conclusion and keep its working out of the main transcript. Plank routes a sub-agent's renderer output through a channel to the main display, which shows it in a separate pane or discards it, and the sub-agent's renderer is unchanged: it forwards to a sink, and the sink happens to be a channel sender. Chapter 11's side questions do the same with a split pane. And a headless run, where the harness is driven by a script and standard output is a protocol rather than a screen, uses a sink that discards everything, so that the protocol stream stays clean.

None of those is written yet, and all of them are a type implementing five methods. The renderer and the agent loop do not change for any of them. That is what the chapter's separation buys beyond correctness: the projection is a trait, and the harness has as many projections as it has sinks.

When there is no user

The display trait assumes someone is watching. Sometimes no one is, and the harness has to know the difference.

Plank selects its front end from how it was started. A terminal on both ends gets the full-screen display. Piped input or output gets the plain line display. A flag gets headless mode, where standard input carries a line protocol from a script or another program and standard output carries the harness's replies in a form the script can parse. The three share the agent, the tools, the renderer and the transcript, and differ in the sink and in what happens when a decision needs a person.

That last point is where headless mode matters for design. Several later features ask the user something: whether to allow a write to a protected directory, which of several options the model should take, whether a plan is approved. In headless mode there is no one to ask, and a harness that blocks on a prompt has hung the script driving it. Plank's

Chapter 4: Streaming without lying

rule is that every such prompt degrades: the ask tool tells the model no user is available, the model-download offer exits with instructions instead of waiting, and a headless run never opens a pane. The README makes the point about the download in one line: with standard input not attached to a terminal there is no prompt to answer, so the harness exits with instructions instead of hanging a script.

The renderer participates in this through the sink. A headless run's sink discards everything except what the protocol needs, so that reasoning and banners do not corrupt the output stream a script is parsing. The renderer does not know; it forwards to a sink as always. Blanka's binary has the plain display and, through the echo engine and piped input, a crude form of the piped case; a true headless protocol is a sink and a reader of standard input, and the shape of both is fixed by what this chapter built.

Where blanka stops short of plank

Plank's stream renderer is a thousand lines, and the inventory of what it does that blanka's does not is a guide to where display work goes once the basics are right.

It detects tool-call markers inside reasoning as well as outside, and handles a stanza that opens inside reasoning according to a setting: by default it discards the stanza with a notice and tells the model on the next pass that tool calls are not allowed inside reasoning, in the reference's exact words; with the setting on, it dispatches the call as ordinary. It tolerates the two observed misspellings of the marker described in chapter 2, in this layer and not in the parser. It shows per-tool visualizations, the read summary, the streaming diff, the collapsing write preview, and runs the edit preflight the moment the old-text parameter closes. It carries bytes rather than characters so a local engine's split multi-byte output is never shown partially. It tracks markdown fence parity across the reasoning boundary so a code block opened before the reasoning and closed after it renders correctly. It takes the first sentence of hidden reasoning as a status line. And it records the display switches in effect in every saved session.

Blanka has the state machine, the carry, the banner, the reasoning flag, the interrupt and the plain sink. Chapter 10 adds the reasoning-side detection when the loop guards need to watch the reasoning stream. The appendix on the full-screen display adds the visualizations. The rest is a reference for what a display accumulates when users watch it every day.

Shown, hidden, and recorded

The model's output has two readers. The transcript takes every byte verbatim, because the transcript is what the model sees next and what a replay reproduces. The user takes a projection: prose shown, reasoning shown or hidden, tool calls announced as a line. A renderer between the engine and the display computes the projection and keeps the record, and it never decides what executes.

Streaming means tags arrive in pieces, so the renderer holds back any tail that could begin a tag and classifies it when the next chunk resolves it, delaying the display by at most a tag's length and the transcript not at all. Reasoning is separated always and shown conditionally, and every renderer must take its flags from the same place or the views drift, as plank's sub-agent pane did.

Chapter 4: Streaming without lying

Stopping is a flag the engine polls between chunks, set by a signal or a key through the same method. An interrupted pass returns its partial text as a result, not an error, so the transcript records exactly what the user saw, followed by a note that tells the model why its reply stops. The agent loop changed in one place: a renderer per pass, borrowing the display, returning the verbatim reply.

CHAPTER 5: SESSIONS THAT SURVIVE

Close the harness and the conversation is gone. Every read the model did, every edit it explained, every decision it reached over forty tool rounds, vanishes with the process, and the next launch starts from an empty transcript. A user who has spent an hour bringing a model up to speed on a codebase and then has to leave for a meeting comes back to nothing. Among the things people using plank's early builds asked for, this was the first, and it is the first thing Part II adds.

The request sounds like it needs a file, and it does need a file, but a file is the least of it. The moment a transcript survives the process, questions arrive that the in-memory version never had to answer. What is a session called, and who chooses the name. What happens when two processes write to the same directory. What happens when a write is interrupted halfway. How does the harness know which of two hundred saved sessions the user means. What gets deleted, and when, and by whom. How does a saved conversation come back onto the screen. And, because the sessions are also the evidence, how does a session that went wrong get handed to someone who can read it. This chapter answers each of those, and the answers set up everything that Part II does with the engine's cache, because a cache is a second file that has to agree with the first.

The record and the sidecar

The transcript from chapter 1 was serializable before anything needed it to be, and now something does. A session on disk is two files. The first is the transcript, every message with its role, in a format that a person can open in an editor and read. The second is a small sidecar of metadata: the session's id, its parent if it was forked, a title, when it was created and last used, how many times it has been resumed, and whether the user has pinned it against deletion.

```
pub struct SessionMeta {  
    pub id: SessionId,  
    pub parent: Option<SessionId>,  
    pub title: String,  
    pub created: u64,  
    pub last_used: u64,  
    pub uses: u32,  
    pub pinned: bool,  
}
```

The reason for two files rather than one is a rule that plank arrived at for its cache and that applies equally to the transcript: the sidecar is advisory. Everything in it is convenient and nothing in it is load-bearing. A listing reads sidecars because they are small and reading two hundred transcripts to show a menu would be slow. A resume reads the transcript and takes the sidecar if it is there, and if it is missing or corrupt, derives what it can, the title from the first prompt, the timestamps from the file's modification time, and carries on. A test deletes the sidecar after a save and asserts that the load succeeds, that the title is recovered, and that the use count, which cannot be recovered, comes back as zero rather than as an error.

Chapter 5: Sessions that survive

Plank's cache documentation states why this matters in a way that goes beyond convenience. A sidecar is a second, unsigned description of the same bytes. The moment it can gate a load, there are two sources of truth about what a file is, and they can disagree. Keeping the sidecar advisory means it can be as rich and as lossy as convenience dictates, because nothing that decides correctness depends on it. For the transcript the stakes are lower than for the cache, where a wrongly trusted blob corrupts the model's view of the conversation, but the discipline is the same and it is cheaper to adopt at the start than to retrofit. Chapter 8 will lean on it hard.

The transcript file is JSON, pretty-printed, because the book's reader will want to open one and look, and because the type already derives serialization. Plank's format is a hand-rolled line protocol with length-prefixed bodies, a trailing record that duplicates listing metadata so the list can be built from a bounded tail read, and optional per-message timestamps that were added later for a statistics feature and parse as zero when absent. Every one of those is a good decision for a harness whose sessions run to megabytes and whose users have hundreds of them. None of them is needed to understand what a session is, and the JSON version can grow into them.

What must be in the file

Before deciding how to write the file, it is worth being exact about what has to be in it, because the answer is stricter than "the messages".

Chapter 1 stated plank's log-everything invariant: anything that reaches a model request must be reconstructible from the session log. A saved session is that log made durable, and the invariant is what makes a resumed session behave like the original rather than like a similar one. Consider what a replay does. It reads the file, hands the messages to the engine, and generates. If the engine sees the same bytes it saw live, the cache from chapter 8 matches and the model's view is the same. If the file omitted something the model saw, a reminder injected under context pressure, a tool result that was rewritten in place, a note that a reply was interrupted, the replay produces a different prefix, the cache misses, and the model is continuing a conversation that differs from the one the user remembers.

So the rule for the file is the rule for the transcript: everything the model saw is a message, and every message is saved. Chapter 4's interruption note is a user-role message for this reason. Chapter 6's pressure reminder will be one too. Chapter 7's compaction, which replaces the head of the transcript with a summary, replaces it in the transcript and therefore in the file, so a resumed compacted session resumes compacted. The alternative design, in which the harness recomputes injected text at request time from settings and state, is tempting because it keeps the file small and the messages "pure", and plank's architecture notes reject it in one sentence: where content is injected at request-assembly time, log the rendered text as a transcript entry at the moment it is injected, rather than recomputing it at replay time. Recomputation drifts. A logged entry does not.

The one thing the file does not contain is the system prompt, and the reason is rate of change. The system prompt is the same for every session on a given build and changes only when the build does, so storing it in every session file would be storing the same

kilobytes hundreds of times and would make every session file wrong the moment the prompt changed. Plank stores it once, fingerprinted, beside the sessions, and records in each session which fingerprint it was written under; chapter 6 builds that arrangement and chapter 8 caches under it. The transcript file holds the conversation. The prompt is context the harness supplies around it, and it is logged and versioned separately.

What a session is called

A session needs a name the moment it has a file, and the obvious choice, a hash or a numeric id, is the wrong one. Users refer to sessions in conversation, type them at a prompt, and compare two of them at a glance. A forty-character hex string serves none of those. Plank's first sessions had exactly that name, and its finding on the subject records the change: a session id is now a memorable slug, an adjective and a surname, minted when the session first saves, and the filename stem is the identity.

```
pub fn memorable_id(rng: &mut Rng) -> String {
    let a = ADJECTIVES[(rng.next_u64() % ADJECTIVES.len() as u64) as usize];
    let s = SURNAMES[(rng.next_u64() % SURNAMES.len() as u64) as usize];
    format!("{a}-{s}")
}
```

Plank draws its surnames half from scientists and half from historical and popular figures, so a session is science-flavoured about half the time, and its adjectives are chosen to be funny; `deadly-einstein` is the documentation's example. Blanka's pools are smaller and drawn from computing, which fits the book, and the shape is the same: two words a user can say aloud.

The pools are small enough that collisions happen. Twenty-five adjectives and twenty-five surnames give six hundred and twenty-five names, and a user with a hundred saved sessions will hit an existing one every few launches. The store handles it by checking whether the file exists and appending a short random suffix if so, retrying until the name is free:

```
pub fn mint_id(&self, rng: &mut Rng) -> SessionId {
    let base = memorable_id(rng);
    let mut id = SessionId(base.clone());
    while self.transcript_path(&id).exists() {
        id = SessionId(format!("{base}-{s}", short_suffix(rng)));
    }
    id
}
```

Plank's version does one more thing here, and the reasoning is worth recording even though blanka does not do it. When the name is taken by a session that is past its expiry age, plank deletes that session and takes the name, judging the age by the transcript's own modification time rather than by the sidecar, because the transcript is the file being replaced and the one file that always exists. A file whose age cannot be read is treated as live, and the comment says why: the cost of guessing wrong is a deleted conversation,

Chapter 5: Sessions that survive

and the cost of being cautious is a suffix on a name. That asymmetry, prefer the cheap mistake, recurs in every decision about deletion below.

The name is the filename stem, and that has a consequence for what a name may be. A user renaming a session is choosing a filename, and a filename that contains a slash or a space or a control character is a problem on some platform or in some shell. So a name is validated against a grammar, ASCII letters and digits and hyphens, at most eighty of them, and a rename that fails validation is refused with a message that names the offending character:

```
pub fn validate_name(name: &str) -> Result<&str, String> {
    let name = name.trim();
    if name.is_empty() {
        return Err("a session name cannot be empty".into());
    }
    if name.len() > NAME_MAX {
        return Err(format!("a session name cannot exceed {NAME_MAX} characters"));
    }
    if let Some(bad) = name.chars().find(|c| !(c.is_ascii_alphanumeric() || *c == '-')) {
        return Err(format!("a session name may contain only letters, digits and hyphens, not {bad:?}"));
    }
    Ok(name)
}
```

Plank's grammar is the same, with one extra exclusion: a name may not begin with the word that names the system-prompt cache file, because that file lives in the same directory and a session called by that prefix would be listed as a cache blob. It is the kind of rule that exists because something once went wrong, and it is a reminder that when several kinds of file share a directory, the naming grammar is where they are kept apart.

Why files, and not a database

A store of a few hundred sessions with metadata, listing, prefix lookup and retention is the shape of problem an embedded database solves well, and a reader may wonder why the answer is a directory of files.

The first reason is inspection. A session that went wrong is read by a person, and a person can open a JSON file in an editor, search it, attach it to a bug report and diff it against another. A row in a database can be exported to do any of those, and the export is a step that has to exist and be remembered. Plank's dumps are files for the same reason, and its sessions are files so that a dump and a session are the same kind of thing.

The second reason is the cache. Chapter 8 puts the engine's cache blobs beside the transcripts, and those blobs are large, hundreds of megabytes for a long session on a big model. A database holding them would be a database that is mostly binary blobs, with all of the operational weight that implies, or a database of pointers to blob files, at which point the directory is doing the work anyway. Plank's store is a directory in which transcripts, sidecars, blobs and their sidecars share a naming grammar, and the sweep walks it as a whole.

The third reason is failure modes. A corrupt file is one lost session. A corrupt database is every session, and the tools for recovering one are a different skill from the tools for recovering a file. Atomic rename gives a file store the one transactional property it needs, which is that a reader never sees a half-written file, and the store needs no other.

What a database would buy is a listing that scales past a few thousand sessions, transactions across several files, and queries by content. Plank has hundreds of sessions and has not needed any of the three; its listing reads a bounded head and tail of each file precisely so that it scales as far as a directory listing does. A harness that expected tens of thousands of sessions, or that needed to search across them, would revisit the decision, and the store's interface, save, load, list, resolve, fork, rename and remove, is small enough that the revisit would be contained.

A format that can grow

The file is JSON because the transcript type already serializes to it and because a reader can open the result. The choice has a consequence for the future that is worth making explicit, because a session format is the one part of a harness that outlives every version of the code.

Sessions saved today are read by the harness of next month, and the harness of next month will want to record things today's does not. Plank's format grew twice in this way. Per-message timestamps were added for a statistics feature, optional in both directions: a record without one parses with a zero timestamp, and a message with a zero timestamp is written without the field, so a session that never carried timestamps stays byte-identical across the change. A record of the project directory was added the same way, because the transcript itself never states where it ran, and a session resumed from a different directory needs to know. Both additions were made without a format version bump and without a migration, because the format tolerated missing records.

JSON with serialization derives gets this behaviour nearly for free: a new field with a default deserializes from a file that lacks it, and a field the reader does not know is ignored. The discipline is to add fields with defaults and never to change the meaning of an existing one. When a change is not additive, the file needs a version, and plank's format begins with a magic line that names its version for that day; blanka's JSON can gain a version field the day it needs one. What the book asks the reader to take from plank's two additions is the habit: when a session file needs to say something new, make the new thing optional, make its absence mean what old files meant, and check that an old file still loads unchanged.

There is a subtler point in plank's timestamp decision. The field was made optional not only for old files but for new ones: a message with no timestamp is written without the field. That is what makes byte-identical the right word. A test that compares a saved file against a fixture, or a cache keyed on the file's bytes, is unaffected by the feature's existence unless the feature is in use. Additions that change every file's bytes, even harmlessly, invalidate more than they mean to.

Writing a file without lying about it

The transcript is written after every turn, and a write can fail halfway. The process can be killed, the disk can fill, the machine can lose power. A file that is half written and then read back as if it were whole is worse than a file that is missing, because a missing file says "nothing here" and a half file says "this is the conversation" and is wrong.

The fix is the oldest one in systems programming: write to a temporary file, then rename it over the target. A rename within a directory is atomic on every file system the book cares about, so a reader sees either the old file or the new one and never a mixture. The store's write path is four lines:

```
fn write_atomic(path: &Path, bytes: &[u8]) -> io::Result<()> {  
    let tmp = path.with_extension(format!("tmp.{}", std::process::id()));  
    std::fs::write(&tmp, bytes)?;  
    std::fs::rename(&tmp, path)  
}
```

The process id in the temporary name is the second half of the fix and the half that is easy to omit. Two harnesses can have the same session directory open, a user with two terminals, or a sub-agent in chapter 12 saving alongside its parent. If both used the same temporary name, their writes could interleave into one temporary file that decodes cleanly and describes nothing, and one of them would rename that file into place. Plank's cache notes describe this exact hazard for the cache blobs and give the same remedy: a process-suffixed temporary and a rename, so an interrupted write cannot leave a half file that reads as valid, and two writers cannot share a temporary path.

A test for the crash case writes a session, then plants a temporary file with junk in it beside the committed one, as a crash between write and rename would leave, and asserts that the load returns the committed transcript and that the listing does not count the temporary as a session. The listing's filter for what counts as a session file, a `.json` suffix that is not a sidecar and does not contain the temporary marker, is what that second assertion pins.

Two processes, one directory

The process id in the temporary name protects against two writers producing one corrupt file. It does not protect against everything two processes can do to each other, and the rest of what a shared directory requires is worth listing, because each item is a rule the later chapters obey.

Two harnesses can be open on different sessions in the same store, and the common case is a user with two terminals. Each saves its own files and neither reads the other's, and the only shared operation is the sweep. A sweep that ran in one process while the other was mid-session could delete the other's session by age, if that session had been idle for a while before the user came back to it. Plank's sweep takes a list of the sessions the running harness is using and never deletes those, and blanka's takes the same list; the agent passes its own session id, and chapter 12 adds its sub-agents' ids. The list is a promise the running process makes about itself, and a sweep run by a different process cannot know about it, which is why plank's sweep also re-reads each file's identity

immediately before deleting it, and skips anything that changed between the scan and the unlink: the verdict was computed about bytes that are no longer there.

Two harnesses can also, in principle, resume the same session. Nothing in the store prevents it, and the result is two processes appending to the same transcript and saving over each other, the last save winning. Plank does not prevent it either, and the reason it has not needed to is that a user does not do this on purpose and a sub-agent, which is the other process that shares a store, always has its own session. A lock file would close the gap, and the cost is a lock that a crashed process leaves behind and the next launch has to reason about. The book leaves the gap open and names it.

The temporary files themselves need a sweep. A crash between write and rename leaves one behind, and while the listing ignores it, it occupies disk and it is confusing to find. Plank's sweep begins by removing orphaned temporaries past an age, and plank's listing filter is where a reader adding that step would start.

Listing, resolving, resuming

A user with saved sessions needs three things: to see them, to name one, and to have it back.

Seeing them is the listing, and the listing reads sidecars only, sorted most recently used first with the id as a tiebreak so the order is stable. A session with no sidecar is loaded to recover a title, which is slow for that one session and correct. The command shows the id, the use count, the title and a pin marker on one line each, and the plain-text shape is deliberate: chapter 4's headless mode has no panes, and plank's rule for every command with an interactive pane is that a static text equivalent exists for the piped path.

Naming one is by prefix. A user who sees `witty-turing` in the list types `witty` and expects it to work, and it does, unless two sessions share the prefix, in which case the store says how many match and asks for more:

```
pub fn resolve(&self, prefix: &str) -> Result<SessionId, String> {
    let all = self.list().map_err(|e| e.to_string())?;
    if let Some(m) = all.iter().find(|m| m.id.0 == prefix) {
        return Ok(m.id.clone());
    }
    let hits: Vec<&SessionMeta> = all.iter().filter(|m| m.id.0.starts_with(prefix)).collect();
    match hits.len() {
        0 => Err(format!("no session matches {prefix}")),
        1 => Ok(hits[0].id.clone()),
        n => Err(format!("{n} sessions match {prefix}; be more specific")),
    }
}
```

An exact match wins over a prefix match, so a session named `tidy` is reachable even when `tidy-boole` also exists. The two-word names make prefix resolution work well in practice: the adjective alone usually suffices, and when it does not the error says so.

Chapter 5: Sessions that survive

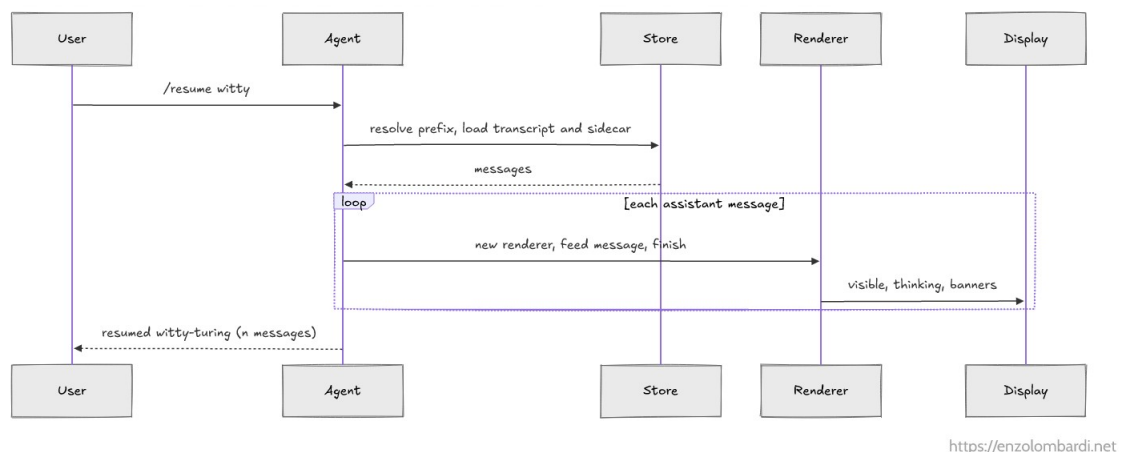
Having it back is the resume, and the resume is where the display from chapter 4 earns a second use. A resumed transcript is replayed through the same renderer that showed it live, so the reasoning comes back dimmed and the tool calls come back as banners rather than as raw stanzas. Plank's user guide states this as a feature: a resumed session replays through the same renderer as a live one, so history comes back as rendered markdown with thinking dimmed, not flat text. The renderer needs no changes for this. Each saved assistant message is fed through a fresh renderer and finished, and the display sees exactly what it saw when the message arrived. This is a small payoff of the projection-not-record rule from chapter 4: because the transcript holds the raw bytes, the projection can be recomputed at any time, and a saved session looks the same as a live one because it is the same bytes through the same code.

The agent's own part in resuming is small. It replaces its transcript with the loaded one and remembers the id, so the next turn saves to the same file:

```
pub fn resume(&mut self, id: &SessionId) -> std::io::Result<> {  
    let Some(store) = &self.store else {  
        return Err(std::io::Error::other("no session store"));  
    };  
    let (t, meta) = store.load(id)?;  
    self.transcript = t;  
    self.session = Some(id.clone());  
    self.meta = Some(meta);  
    Ok()  
}
```

A test runs a turn in one agent, resumes the saved session in a fresh agent built over the same store, asserts the transcripts are equal, runs a second turn, and asserts that the file now holds four messages and a use count of two. That is the whole contract of a session: what one process saved, another process continues.

What replay looks like, for the reader who wants to picture it, is this:



Resuming a session: the saved messages pass through the same renderer that showed them live, one fresh renderer per message.

One renderer per message, not one for the whole replay, for the same reason chapter 4 gave for one renderer per pass: the state machine starts clean for every reply, so a stanza left open by an interrupted message cannot leak into the display of the next. The renderer does not know it is replaying. Plank's does, through a flag, because plank's renderer also runs a preflight check on edits as they stream and a replay must not re-run it against files that have since changed; blanka's renderer has no preflight and needs no flag.

When the file is written

The agent saves after every turn, and the choice of when is less obvious than it looks.

Saving after every pass would be safer against a crash mid-turn, and it would write the file dozens of times during a long turn, most of them with a transcript that ends in a tool result the model has not yet seen. Saving only on exit would write once and lose everything on a crash. After every turn is the compromise: the transcript on disk always ends at a point where the model has answered or been interrupted, which is the point a user would want to resume from, and a crash loses at most the turn in progress.

The id is minted on the first save, not when the agent starts, so a session that never gets past its first prompt leaves no file. Plank does the same, and the reason is what the directory looks like after a month: a harness that minted a name on every launch would fill the store with empty sessions from every time the user opened it to check something and closed it again.

The save itself updates the sidecar's title from the transcript, its last-used time, and its use count, then writes both files atomically. A failure to save is reported to the display through the error channel and does not fail the turn, because the model's answer is on the screen and in memory, and a full disk should not make the harness pretend the answer did not happen.

The order of a save

A save writes two files, and the order in which it writes them is a small decision with a reason.

The transcript goes first and the sidecar second. If the process dies between the two, the transcript on disk is current and the sidecar is one save behind: its use count is low by one, its last-used time is stale, its title may be old. Every one of those is advisory, and the next load derives nothing from them that matters. If the order were reversed and the process died between, the sidecar would describe a save that never landed, claiming a use and a time for a transcript that is still the previous one. Nothing would break, because nothing trusts the sidecar, but a listing would show a session as touched more recently than it was, and a retention sweep would keep it a little longer than it should. The transcript-first order means the worst case of a half save is a sidecar that undersells the session, which is the harmless direction.

Within the save, the title is recomputed from the transcript rather than carried forward, the use count is incremented, and the last-used time is set to now. The use count is the one of these that a reader might expect to be incremented on resume rather than on save, and it is not, for a reason that is about what the number is for: the sweep uses recency

rather than count, and the listing shows count as a rough measure of how much a session has been worked in. A session resumed and immediately abandoned has not been worked in. A session that ran ten turns has been, and its count says ten.

The whole save is skipped when the transcript is empty, which is what keeps a launch that was closed without a prompt from minting a name. And the save's failure is reported and swallowed rather than propagated, because the turn has already happened; a save that fails is a problem for the next resume, not for the answer on the screen, and the user is told so through the display's error channel.

Forking and renaming

Two operations on a saved session are defined in this chapter as operations on the transcript, ahead of the engine state that chapter 9 attaches to them.

A fork is a new session with the same transcript and a pointer back to the original. The original is untouched; the fork continues from where it was, and the user has two conversations that share a history and diverge from a point. Plank's version is richer, a session is a tree of messages and a fork rewinds the live transcript to a chosen prompt while keeping everything after it as a sibling branch, and plank's design documents spend some effort on what a fork means for the engine's cache: a fork leaves the transcript a strict prefix of what it was, so the cached prefix is reused rather than re-read. Blanka's fork is the simple form, a copy with a parent, and it is enough to establish the operation and to test that the original is left where it was:

```
pub fn fork(&self, from: &SessionId, rng: &mut Rng) -> io::Result<SessionId> {
    let (t, mut meta) = self.load(from)?;
    let id = self.mint_id(rng);
    meta.parent = Some(from.clone());
    meta.id = id.clone();
    meta.created = now_secs();
    meta.last_used = meta.created;
    meta.uses = 0;
    meta.pinned = false;
    self.save(&id, &t, &meta)?;
    Ok(id)
}
```

The fork does not inherit the pin. A user who pinned a session pinned that session, and a copy that inherited the pin would accumulate pinned copies the user never asked to keep.

A rename moves both files to a new stem, after validating the name and refusing one that another session holds. The parent pointer survives, so a renamed fork still knows where it came from. The agent-level rename saves first, so the file being moved is current, and updates its own idea of the session's id afterward.

Both operations save before they act. This is a pattern that the chapter's later operations follow and that chapter 9 makes essential: any operation that reads a session's file to do something with it must first make sure the file reflects the live conversation, or it will act

on a stale copy. For a fork, acting on a stale copy would silently drop the last turn from the child. For the cache operations of chapter 9, it would put the cache and the transcript out of step.

A session belongs to a project

A session is a conversation about a codebase, and the codebase is a directory. Plank records that directory in the session file, and the reason is what happens when the recording is missing.

A user resumes a session from a different directory than the one it was started in, because they opened a terminal somewhere else. The transcript is full of relative paths, `src/main.rs` and `Cargo.toml`, that meant something in the original directory and mean something else, or nothing, in the new one. The model, continuing the conversation, reads a file that is not the file it discussed, or fails to find one it edited an hour ago, and either way the conversation has silently changed subject. Recording the directory lets the harness notice, and plank's listing shows the project beside each session so a user can see what a session was about before resuming it.

Blanka's session store does not record the directory in this chapter, and it should; the tool context carries the working directory, and chapter 6, which assembles the session-start context that tells the model where it is, is where the field is added to the sidecar. It is noted here because it is the first instance of a pattern that recurs: a session is not only a transcript but a transcript plus the environment it assumed, and the more of that environment the file records, the more truthfully it can be resumed. Plank's dump carries the same information for the same reason, and plank's session-start context, which chapter 6 covers, is where the directory, the git state and the date are told to the model at the start of every session.

Titles, tags and pins

The sidecar carries three fields for a person's benefit and none of them is essential, which is what makes them a good example of what a sidecar is for.

The title is the first line of the first user prompt, clipped to sixty characters. It is derived, not chosen, because a user starting a session does not want to be asked to name it, and the first thing they typed is almost always what the session is about. It is recomputed on every save from the transcript, so a session whose first prompt was edited, or a fork whose parent's first prompt is its own, has the right title without anyone maintaining it. A test asserts that a two-line prompt yields a one-line title.

A tag is a label the user attaches deliberately, and plank offers one per session through a command. It exists because titles derived from prompts converge: a user working on one project asks similar questions, and ten sessions titled some variant of "fix the parser" are not distinguishable in a list. The tag is where the user writes what the title cannot know. Blanka's sidecar has no tag field; adding one is a field with a default and a command, and the format section above is the argument that it costs nothing to add later.

The pin is the one of the three that affects behaviour, and it affects exactly one thing: a pinned session is never deleted by the sweep, by age or by budget. Plank's user guide

Chapter 5: Sessions that survive

explains pinning in terms of the cache, where the argument is strongest, since the largest snapshots are the most expensive to rebuild and a setup returned to every few weeks is cheaper to pin than to let lapse. For a transcript the argument is about work: a session that represents a day of bringing a model up to speed on a subsystem is worth more than its age says, and the pin is how the user says so. The sweep honours it absolutely, which is why a budget is a target and not a licence.

What gets deleted

A store that only grows fills a disk, and a harness that deletes the wrong session has destroyed work its user cannot recover. Between those two lies a retention policy, and plank's is worth following closely because its first version was wrong in an instructive way.

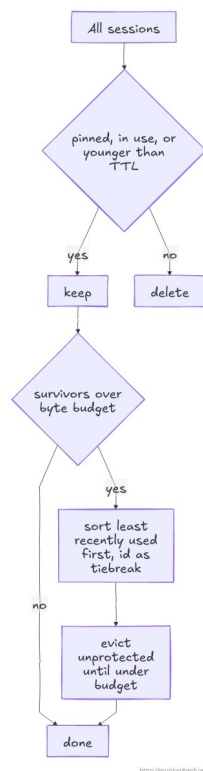
The first attempt, for the cache, kept only the entries that were current and deleted every sibling. It optimized disk perfectly and defeated the cache's purpose, because it made identity and retention the same decision: switch to a different setting and the entry for the setting you left is deleted; switch back and pay the full cost again. The replacement separates the two questions and decides retention on value, estimated from age and use, which is exactly what the sidecar exists to record.

The policy runs in two phases. Phase one is age: a session that is pinned, or in use by the running harness, or used more recently than the time-to-live, survives; anything else is deleted. Phase two runs only if the survivors still exceed a byte budget, and it evicts least recently used first, in a total order with the id as a tiebreak, skipping anything protected, until under budget:

```
pub struct GcPolicy {  
    pub ttl: Duration,  
    pub byte_budget: u64,  
}
```

The flow is this:

Chapter 5: Sessions that survive



Retention in two phases: age decides first, and the byte budget applies only to what age left behind.

Plank's notes record four decisions in this design, each answering a specific failure, and three of them carry over directly. The eviction order is a total order derived from the data, so the outcome is a function of the inputs and not of the order the directory happened to be read in; size-based eviction was originally rejected as order-dependent, and sorting first is what made it acceptable. A budget is a target and not a licence: if everything remaining is pinned or in use, the sweep stops over budget rather than deleting something protected, because bounded disk is the lowest-priority requirement and this is where that ranking shows up in code. And a budget of zero means unbounded, never "evict everything", because read the other way it would wipe the store on every launch for anyone who never configured a limit. The fourth decision, about entries that other entries build on, belongs to the cache's chain of snapshots and arrives in chapter 9.

The tests pin each phase. One seeds four sessions with different ages, pins one, marks one as in use, and asserts that only the old unprotected one is deleted by the age pass. Another sets a budget one byte below the total and asserts that the oldest unpinned session goes first, the pinned one is skipped, and the next oldest goes second, in that order. A third asserts that a zero budget deletes nothing.

What the harness deletes is the transcript and its sidecar together. Chapter 9 adds cache blobs to the set, and the rule that a session's files travel together, so that a blob is never left keyed to a transcript that no longer exists, is stated in plank's code at the one place a session is removed. Blanka's remove function is that place.

The store as the cache's landlord

Everything above is about the transcript, and the store will shortly hold something else. Chapter 8 introduces cache blobs: the engine's computed state for a prefix of the conversation, which lets a resumed session skip re-reading everything the model has already seen. The blobs live beside the transcripts, and three properties of this chapter's store were chosen with them in mind.

The naming grammar keeps the kinds apart. A blob has its own suffix, its own sidecar has another, and the listing filter for transcripts already ignores everything it does not recognize. Plank goes one step further and encodes the model family in the transcript's extension, because a store shared by two models must not let one model's launch evict the other's work. The need arrived when a second DeepSeek generation joined the first in the same directory. A transcript became `<id>.ds4.kv` or `<id>.ds41.kv`, with the tag placed before the final suffix so that every transcript still ends the same way and the id is still everything before the first dot, and the untagged files from before the split were renamed once, at the first launch that knew about families, on the grounds that everything written before then had come from the older model. The listing and the sweep then keep only blobs whose recorded model resolves to the live family, so launching one model can neither evict the other's checkpoints nor restore them. The second half matters more than the first: the two families tokenize differently, and a restore across them would not be a miss but a silently corrupted view of the conversation.

The same routing is how plank retires a model. A family that is no longer served resolves to no family at all, and no family equals no live one, so its era of files is never listed, never swept and never restored. Plank's notes are explicit that the empty answer is deliberate, since folding a retired name into a live family would hand one model's cache to another. They also record the one path around the guarantee, pre-existing and left unfixed: when a blob's sidecar is missing, the store synthesizes metadata from the file's name, size and time, and that synthetic record has an empty model name, which resolves to a live family. A retired body whose sidecar was lost is therefore sweepable. That is the advisory sidecar biting back in a small way, and it is written down so that the next person to touch either side knows the two interact. Blanka's book has one model family and does not need the distinction, and the extension is where it would go.

The sidecar's advisory status is what lets a blob's sidecar be trusted for retention and never for loading. Chapter 8's rule is that a blob's embedded signature is the sole trust input and the sidecar can only affect display and sweep order. The store learns that rule here, on the transcript, where the consequences of getting it wrong are a missing title rather than a corrupted model view.

And the sweep's notion of protected entries grows. This chapter protects pinned and in-use sessions. Chapter 9 adds a third kind of protection, an entry that another entry builds on, because cache snapshots form chains and deleting a parent orphans its children. Plank's fourth retention decision, mentioned above and deferred, is about exactly that: the dependency question is evaluated against the set of entries as it stood before the sweep began, so the outcome does not depend on the order the directory was read, at the cost of a chain that expires from the bottom being collected one level per run. The

Chapter 5: Sessions that survive

store does not need that logic for transcripts, which have no dependencies except a fork's parent pointer, and a fork survives its parent's deletion because it holds a full copy.

The reason to set all this up before the cache exists is the same reason chapter 1 gave for the four-role enum: the shape is cheap to establish while the store is small and expensive to retrofit once blobs of hundreds of megabytes depend on it.

What a resume costs

A resumed session puts the transcript back in memory and the display back on the screen in a fraction of a second. The next turn is another matter, and the reason is chapter 1's climbing number.

The engine remembers nothing between processes. A resumed conversation of forty thousand tokens is, to the engine, forty thousand tokens it has never seen, and the first pass after a resume reads all of them before producing its first token. On a hosted engine that is a longer wait than usual and a larger bill. On a local engine at a few hundred tokens a second of prefill, it is a minute or more of the user watching a progress bar for a conversation they resumed precisely so as not to start over.

Plank's user guide states the difference the cache makes in one sentence: the cache sidecar is restored alongside the transcript, which is the difference between resuming instantly and waiting for the whole conversation to be re-read. That sidecar is chapter 8's subject, and it is the reason this chapter's store, its atomic writes, its advisory metadata and its naming grammar, is built the way it is. A session that survives the process is the first half of the feature. A session that survives the process without paying for its own history again is the second, and the second is only possible because the first was done carefully.

There is one more cost that a resume makes visible and that the harness should report. Plank's exit report gives the session's prefill time and rate per model, and a user who resumes a long session and watches the first turn's prefill number will see, in one figure, what the cache is for. The report began as two run-on lines that wrapped in the middle of a number, and it is now a small table, one block per engine with rows for input, output and tools across columns of tokens, time and rate, and a totals block when more than one engine served the session. The shape changed because the content had outgrown the line, and the lesson is modest but general: a figure the user is meant to compare with another should sit in a column beside it. Blanka's turn report from chapter 2 prints the prompt token count after each turn, and the first turn after a resume is where that count is largest.

Figures the store can answer

Sessions kept on disk with their metadata turn out to answer questions nobody asked when the store was designed. Plank's `/stats` command draws a year of activity as a heatmap of days, with streaks and a handful of headline figures below it, computed entirely from per-session metadata that its insights report already caches. The model is never consulted, and every number is a deterministic function of the files, which is what makes such a report cheap to add once the store exists.

Its history is a small catalogue of how a figure can say something other than what it counts. A session whose recorded creation time lay in the future, from a skewed clock, was already excluded from the grid but still counted in the figures, so it could anchor the most active day or the current streak on a day that had not happened; the fix filters such rows once, where the report is built, so the grid and the figures agree. A figure called "active days" counted the days on which a session was started, so a session begun on Monday and resumed through Thursday lit one cell while the insights report counted four, and the figure was renamed "days started" rather than left to disagree with its neighbour under the same word. A figure called "longest session" measured the wall clock between the first and last message, which read as time spent working and was not: resume a session a month later and it reported thirty-one days. It became "longest span". None of these was a bug in arithmetic. Each was a name promising more than the data behind it could deliver, and in each case plank changed the name rather than pretending the data said more.

The design point for a harness of your own is that the advisory sidecar earns a second life here. It was introduced to serve listings and retention, and it can serve reporting on the same terms: nothing loads through it, so a report built on it can be wrong in a figure without ever being wrong about a conversation.

Sessions with no one watching

Chapter 4 described headless mode, in which a script drives the harness and no person watches. Sessions matter there too, and in one respect more.

A headless run is the shape of the controlled measurements the book has cited since chapter 1. Plank's finding on where a local agent's time goes ran the same request three times under a headless harness with a time limit, and its method section begins with the fact that made the analysis possible: headless runs now save a transcript. Before that change a headless run left nothing behind, and a run that went wrong could only be described from memory. After it, the run's transcript is a session like any other, resumable, dumpable and greppable, and the measurements in the finding, tool calls per pass, reasoning size per turn, failed edits, were computed from the saved files.

Plank's own benchmark harness later found the edge of that guarantee. It wrapped runs in a time limit, and the limit's default signal, SIGTERM, ended plank without saving, so every timed-out run recorded no tool calls, no transcript and no dump, and the report's median looked like a model that never started. Two of the five had in fact finished; a generated file was on disk seconds before the kill. Plank treats SIGINT as an interrupt of the generation and then saves and writes a dump like any other exit, so the harness now sends SIGINT first and a hard kill only after a grace period. The same change exposed a quieter consequence that plank chose to know rather than fix: a plank that exits after an interrupt exits with success, so a run stopped from outside can be recorded as completed when it was interrupted. The dump is where the truth is kept, under an outcome field, and a missing transcript is now recorded as an unknown rather than as zero, because a floor printed as a total is a number nobody questions.

The design consequence is that saving is not a feature of the interactive display. It belongs to the agent, which saves after every turn regardless of what front end is attached or

whether one is. Blanka's save lives in the agent's turn loop for this reason, and a headless binary built on the same agent would save without any change. The only front-end-specific part of sessions is the commands that list and resume them, and those have plain-text answers so that a pipe can drive them.

The dump

The sessions on disk are the harness's memory. They are also its evidence, and the last piece of this chapter is the form that evidence takes when something goes wrong.

Every chapter so far has cited plank findings, and most of those findings were established by reading a file plank calls a repro dump: a self-contained record of what the model saw, written on demand by a command or automatically when a guard stops a looping pass, that can be handed to someone without the live session. Plank's module describes it as capturing the exact input the engine would see plus the runtime knobs that shape generation, and as a read-only snapshot that touches nothing live.

Blanka's dump is a markdown file with a header of facts and the transcript rendered as role-labelled blocks:

```
pub fn build_report(view: &ReproView<'_, now_secs: u64) -> String {
  let mut out = String::new();
  out.push_str("# blanka repro\n\n");
  out.push_str(&format!("- date: {now_secs}\n"));
  out.push_str(&format!("- session: {}{}\n", view.session.unwrap_or("unsaved")));
  out.push_str(&format!("- engine: {}{}\n", if view.engine.is_empty() { "(none)" } else
{ view.engine }));
  out.push_str(&format!("- context size: {}{}\n", view.ctx_size));
  out.push_str(&format!("- show thinking: {}{}\n", view.show_thinking));
  out.push_str(&format!("- messages: {}{}\n", view.transcript.len()));
  out.push_str(&format!("- note: {}{}\n", if view.note.is_empty() { "(none)" } else
{ view.note }));
  out.push_str("## Transcript\n\n");
  out.push_str(&render_transcript(view.transcript));
  out
}
```

Three of those header lines are there because plank learned they were needed. The engine name is there because a report that did not say which model produced a transcript has repeatedly not been enough to tell what was actually loaded; plank's version records the model's name, family, tool dialect, file path and artifact version in a section of their own, with a comment saying that burying the weights' identity under sampling knobs is what made "which model was this" a question worth asking of a report that already answered it. The display setting is there because chapter 4's finding about renderer flags ended with plank recording the switches in effect in every dump, so a dump says what the user was actually looking at. And the note is there so that the user who took the dump can say in one line why.

The transcript is rendered verbatim. Reasoning is there. Raw tool-call stanzas are there. The test asserts on exactly that: a dump of a transcript containing a hidden reasoning

block and a stanza contains both in full. A dump that showed the projection rather than the record would be useless for the one thing dumps are for, which is finding out what the model saw.

The file is named by the time it was taken, with a numeric suffix if a dump already exists for that second, and it never overwrites. Plank's loop guards in chapter 10 write these automatically, with a distinguishing prefix, whenever a pass is stopped, and the chapter's tables of loop measurements are built entirely from reading them. The dumps are also what let a session recording become a unit test, the bridge described in chapters 1 and 2. This chapter puts the file in place so that when a later chapter says "the dump showed", the reader knows what was read.

Dumps when nothing else is possible

Plank has a second kind of dump, and it is worth describing because it is the extreme case of the sessions-as-evidence idea.

When the harness panics, the agent is gone. There is no transcript object to render, no engine to ask for its name, no configuration to report. Plank installs a panic hook that writes a dump anyway, from a copy of the last rendered transcript that the agent stashed before each pass, and the dump's header says that the harness panicked and what the panic message was. The report builder for this case is separate from the ordinary one, and its documentation says why: everything the ordinary report says beyond the transcript comes from the engine or the configuration, and a panic hook has neither. The two dumps share the same transcript fences so the same tooling reads both.

The stash is cheap because it is only taken when a debugging mirror is on, and the design point is the general one: the moment to save evidence is before the failure, not after, because after the failure the thing that could have saved it may not exist. The ordinary dump is taken by a user who noticed something wrong. The panic dump is taken by a process that is about to die. The loop-guard dump of chapter 10 is taken by a guard that has decided the model is stuck. All three write the same shape of file to the same directory, and a maintainer reading the directory does not need to know which kind of failure produced which file to start reading.

Blanka has the ordinary dump and none of the others yet. The panic hook is a dozen lines a reader can add; the loop-guard dump arrives in chapter 10 with the guards.

Testing a store

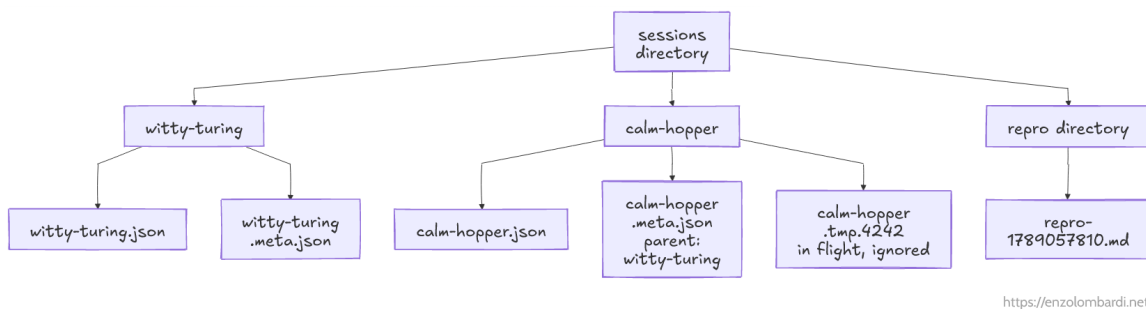
The store's tests follow the pattern of chapter 3's file-system tests, with one addition worth noting.

Each test opens a store in a scratch directory named for the test and the process, so tests do not collide with each other or with the developer's real sessions. The tests cover the round trip, the crash-left temporary, the missing sidecar, fork and rename, and prefix resolution, and each is a specification of one promise the store makes. The retention tests seed sessions with chosen ages and sizes and assert on exactly which ids the sweep removes and in what order, because the order is part of the promise.

The addition is that the agent's tests now touch the store too. A test runs a turn in one agent and resumes in another over the same store; a test forks and checks that the original was left alone. These are the tests that pin the agent's part of the contract, when it saves and what it saves, and they are the ones that will catch a later chapter forgetting to save before an operation that reads the file. Chapter 9's cache operations are exactly such operations, and the fork test here is the template for the tests they will need.

What the store looks like on disk

The layout after a few sessions, a fork and a dump, is this:



A session store: transcript and sidecar per session, a temporary that a crash left and the listing ignores, and dumps in their own directory.

Every file's role is recoverable from its name, which is the point of the naming grammar. A transcript is a stem and a suffix; its sidecar is the same stem with a marker; a temporary has a marker and a process id; a dump has a prefix and a time. Plank's store is denser, with cache blobs and their sidecars and snapshot rungs sharing the directory, and every one of those has a suffix that says what it is. When chapter 8 adds the first cache blob, it follows the same convention, and the listing filter gains one more case.

When a file is bad

A store that lasts for months will eventually contain a file that does not parse. A disk error, a partial copy from another machine, an editor that saved a stray byte, a version of the harness that wrote something the current one cannot read. The store's behaviour in that case is decided by the same rule as everything else in the chapter: fail on the one file, and keep the rest working.

A load of a transcript that does not parse is an error, reported to the user with the parse failure's message, and the live conversation is untouched; the user was trying to resume and could not. A listing that meets a transcript it cannot parse skips it, because a listing is a menu and a menu with an entry that cannot be chosen is worse than one without it. A sidecar that does not parse is ignored and the transcript is loaded without it, as the missing-sidecar test already pins. The sweep, reading sidecars for ages, treats an unreadable one as belonging to a session it cannot judge, and plank's rule from the id-minting code applies: a file whose age cannot be read is treated as live, because the cost of guessing wrong is a deleted conversation.

The file that does not parse is still a file, and this is where the argument for files over a database from earlier in the chapter pays off in the worst case. A user can open it, see

what is there, and often recover the conversation by hand; a JSON transcript with one bad byte is a few seconds in an editor. The store does not try to repair files, because a store that repairs is a store that guesses, and a guess about a conversation's contents is the one kind of guess this chapter's design has refused throughout.

Where blanka stops short of plank

Plank's session module is nearly five thousand lines and blanka's is three hundred, and the difference is again a guide.

Plank's session is a tree, not a line: forking rewinds to a chosen prompt and keeps the abandoned branch as a sibling, a command draws the tree with the active branch marked, and branches persist in the file as extra records after the transcript. Its listing reads a bounded head and tail of each file rather than a sidecar, so a listing never parses a transcript and the metadata cannot drift from the file. Its ids are backward compatible with the forty-hex names of its first weeks, which still load by prefix. Its store also holds the cache blobs, keyed by a fingerprint of the model and the prompt, and the sweep treats a session and its blobs as one unit. Its dumps fold in the sidechains of sub-agents as separate files beside the main one, because a sub-agent's work is truncated out of the main transcript the moment it ends and would otherwise be lost. And its saved sessions carry a record of which model family produced them, in the file extension, so that two models sharing a directory never evict or restore each other's work and a retired model's files become unreachable rather than deleted. Its metadata also feeds a usage report and a tabular exit summary that read the store and never the model.

Blanka has the two-file session, memorable names with validation, atomic writes, a sidecar-only listing with prefix resolution, resume through the renderer, fork and rename as transcript operations, two-phase retention, and the dump. Chapter 9 adds the cache blobs and their place in the sweep. Chapter 12 adds the sub-agent sidechains to the dump. The tree stays in plank; a line is enough to teach what a session is.

The commands, and their plain answers

The chapter's binary gains six commands, and their shape is a small lesson about front ends that the appendix on the full-screen display will need.

A listing command prints one line per session. A resume command takes a prefix, or nothing for the most recent. A fork command continues in a copy and says where. A rename command takes a name and reports the new id or the validation error. A dump command writes a repro file and prints its path. A new command clears the conversation and forgets the id, so the next save mints a fresh one. Every one of them returns a string, and the loop prints it.

The reason each returns a string rather than drawing something is plank's rule for its own commands: a command with an interactive pane in the full-screen display needs a static text equivalent on the plain path, because the plain path is what a pipe, a script and a headless run see. Plank's session browser is a pane with a preview of each session and keyboard navigation, and its plain equivalent is the list above. The pane is a better experience and the list is the one that works everywhere, and a harness that built only the pane would have a resume that a script could not invoke.

Chapter 5: Sessions that survive

The commands are also where the agent's session operations get exercised outside the tests, and the smoke run that accompanies this chapter's code does exactly that against a scratch home directory: a turn, a listing showing one session with one use, a rename, a dump, and a listing showing the renamed session with two uses. It takes a second and it is the fastest way to see that a session's life cycle is what the chapter says it is.

What is left on disk

A session is a transcript file and an advisory sidecar. The transcript is the record and the identity; the sidecar serves listings and retention and can never gate a load. Names are memorable two-word slugs, minted on first save so an abandoned launch leaves nothing, validated as filename stems, and disambiguated on collision by suffix rather than by touching the existing session.

Writes go to a process-suffixed temporary and are renamed into place, so a crash leaves the old file and two writers cannot interleave. Saves happen after every turn. Resume replays through the same renderer as live output, so a saved session looks the way it looked. Fork is a copy with a parent pointer and rename is a move; both save first.

Retention runs age first, then a byte budget applied to the survivors in a total order, skipping the pinned and the in-use, stopping over budget rather than deleting something protected, and treating a zero budget as unbounded. The dump records the transcript verbatim with the facts a reader needs to reproduce it, and it is the form every later chapter's evidence takes.

CHAPTER 6: WHAT THE MODEL IS TOLD

Every pass so far has begun with the conversation and nothing else. The model has been shown a user's words, its own earlier replies, and the framed results of the tools it asked for, and it has done remarkably well with that, because a chat-tuned model has a default personality and a hosted server has a default template. It behaves as a general assistant. It does not know that it is inside a harness with six tools, that its requests must take the shape of one JSON object between two tags, that the result frame it keeps seeing is the harness's own text, that it is sitting in a particular directory of a particular repository on a particular day, or that the project it is looking at has a file of rules its maintainers want followed. Chapter 1 promised that chapter 6 would give it instructions. This is that chapter, and the instructions turn out to be two different things that a harness must keep apart.

The first is text that never changes. Who the model is, what tools it has, how to call them, what rules to follow: the same for every session on a given build of the harness, and different only when the harness itself changes. The second is text that changes every time. The working directory, the state of the repository, the date, the instructions the project carries in a file, the skills the user has installed: different from one launch to the next, and sometimes different between two launches in the same directory a minute apart. Both are things the model is told before the conversation begins. Neither is part of the conversation. And the case for keeping them in separate places, with a line between them that a test enforces, is the thread that runs through the chapter, because chapters 8 and 9 will build a cache on that line and a harness that blurs it pays for the blur on every launch.

Two kinds of system text

Plank's system prompt documentation opens with a table, and the table is the whole design in six rows. On one side is the static tools prompt: tool descriptions, editing rules, the call syntax, the JSON schemas. Its source is one module and a resource file. It is byte-for-byte identical to the C reference, checked by a parity test. It sits in the system section of the transcript. It is cacheable, and it is exactly what the engine's prompt snapshot stores. On the other side is the dynamic session context: git branch and status and recent commits, the discovered instruction files, the current date. Its source is a second module. It is recomputed at every session start. It sits in the first user message of the session, not the system section. And it is deliberately kept out of the cached prefix.

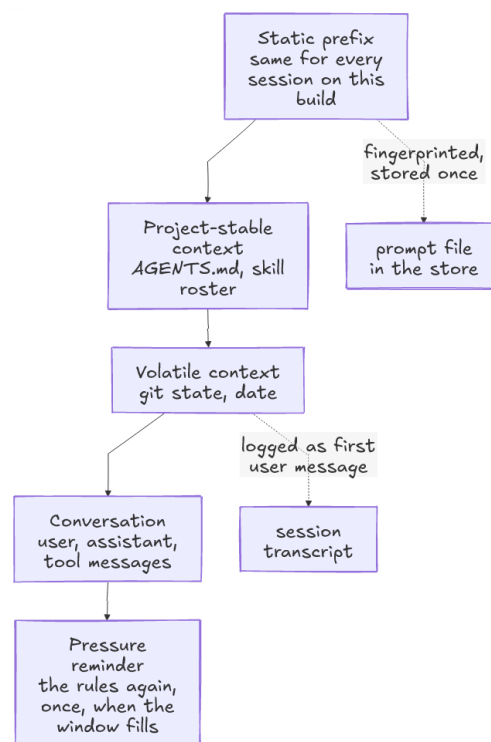
The two columns are separated by a rule that the same document states formally a few sections later, and it is worth having the rule in front of you before any code: an input may enter the composed system prompt only if it is stable across sessions on the same machine and configuration. Anything that changes per session must be injected as the session's first user message instead. The document then says what breaks if the rule is violated, and the answer is instructive because it is not correctness. A date in the system prompt does not make the model misbehave. It makes the prompt's bytes different tomorrow from today, so a cache keyed on those bytes misses every morning, and the several seconds of prefill the cache existed to save are paid on every launch, silently, forever. The rule is a performance boundary that looks like a style choice, and the reason

Chapter 6: What the model is told

to draw it formally now, two chapters before the cache exists, is that a boundary drawn after the fact means finding every volatile byte that crept in while nobody was looking.

Plank drew it on the second day of the port, in the commit that also parallelised the git calls and added a section for server instructions, and the commit's title names the boundary as one of its three subjects. The test that guards it builds the system prompt twice, asserts the two are equal, and then asserts that the prompt contains neither today's date nor any of the opening phrases of the context blocks: the git status header, the branch line, the instruction header. Blanka has the same test in a smaller form, and it is the first thing to write, because it says what the rest of the chapter is allowed to do.

The layers of the prompt, arranged by how often each changes:



<https://enzolombardi.net>

The prompt as stacked blocks by rate of change. The static prefix is the same bytes everywhere; the two context blocks are one user message at the head of the transcript; the reminder appears late and only under pressure.

The middle row is a refinement plank arrived at a week later, when it split the session context itself into a stable half and a volatile half. The instruction files and the skill list change when a file is edited, which is rarely; the git status changes with every commit and the date with every midnight. Putting the stable half first means that a cache of it is a prefix of the whole context, and chapter 9 uses exactly that. For this chapter the split is a rendering order and two methods, and the reason to have it now is the same as the reason for the outer boundary: an order chosen later has to be retrofitted onto files that were written in the old one.

What the static prefix says

The static prefix is the harness talking to the model about the harness, and blanka's is short: about five kilobytes, some thirteen hundred tokens on the reference model, against plank's five thousand tokens. The difference is mostly the parity surface plank carries, which blanka does not have because the reference local model was not trained on any harness's exact text, and partly plank's much larger tool table. What the two share is the structure, and the structure is worth walking through as a piece of writing before it becomes a function, because every sentence in it is something the model will read at the top of every pass for the rest of the session.

It opens with a role. The model is a coding agent in a local workspace; it should use tools for file and system work and not print large file contents as answers, but create or edit files and summarise. That first paragraph is lifted from the reference's Qwen dialect prompt nearly verbatim, and the reason is the reason given in chapter 2 for the strictness of the parser: the wording the model has seen most often is the wording it follows best, and this one is the wording the harness the model family was tuned alongside uses. Then come the tools, one JSON schema per line inside a fence of `<tools>` tags, in the provider registry shape each tool's `describe` already returns. Then the call format, with one example, the result framing the model will see, and the rule against calling tools inside a thinking block. Then the rules.

The rules are where plank's experience is, and where blanka borrows most heavily. Plank's first prompt was the C reference's and nothing more, because parity forbade additions. Over the following weeks a section of plank's own text grew beneath the C text, inside the span the parity fixtures leave free, and each line of it was earned by a measured failure. A session that made forty-six tool calls at two per stanza when it could have made eight batched ones produced the line about batching independent calls. A session that read three hundred and sixty thousand characters before deciding anything produced the line about exploring briefly and then acting. A session whose reasoning looped between two equally good designs produced the line about deciding rather than deliberating, which chapter 10 returns to. A user who watched a long thinking block and could not tell whether the model had stalled produced the line about narrating progress before every tool call. And a change that piped a regex through a throwaway script across several files, one of which the model had not read, produced the line about editing with the edit tool.

The shell rules came from counting. Thirty repro dumps, of the kind chapter 5 built, were searched for tool invocations, and the count said that the model reached for `grep` through the shell a hundred and twenty-five times against forty-five calls to the native search tool, for `cat` and `head` and `sed` thirty-seven times, for `find` and `ls` eighteen times against twelve globs, and that two hundred and twenty-eight of three hundred and seven shell commands began by changing into a directory the shell was already in. The C tool table describes the shell tool as running a shell command and says nothing about what not to run through it, so the routing had to be taught, and the form plank chose names the wrong command beside the right tool: search for content, not `grep`; read for file contents, not `cat`. Blanka's rules file carries the same lines, shortened to the six tools it has. The git rules came from a different observation, that the model runs git in

Chapter 6: What the model is told

about a tenth of its shell calls with nothing telling it which commands lose work, and they are the ones you would expect: commit only when asked, never force-push, stage by name, no interactive flags because there is no interactive input.

The point of recounting where the lines came from is not the lines. It is that every one of them is a fact about the model's behaviour that was observed and then corrected in text, and that the text is therefore a record of the harness's operating experience as much as a set of instructions. A prompt written from first principles before any session has run would contain none of them. Plank's did not. The place to start is the reference's prompt and the sentence that states the role, and the place the rules come from is the dumps.

Why the rules live in a file

There is one detail of how the static text is stored that is an engineering lesson rather than a writing one, and plank learned it on the second day.

The C reference keeps its prompt as string constants, and a port keeps them as Rust string literals. A long literal is naturally written with backslash continuations, so that each line of the prompt is a line of source. Rust's rule for a backslash at the end of a literal's line is that the newline and all leading whitespace of the next line are removed. For prose that is what you want. For the JSON schemas in the tools prompt, which carry indentation the model has seen in training, it silently deletes every indent, and the port's prompt differed from the C's in exactly those bytes while reading identically in the source. The parity test caught it; a harness without one would have shipped it. Plank's fix was to move the affected section into a resource file included with the `include_str` macro, where no continuation rule applies, and to note in the findings file that any string that must survive byte-exact should either avoid continued literals or keep the indentation on the physical line with the newline.

Blanka adopts the fix before hitting the bug. The intro and the call format are raw strings, where a backslash is a backslash. The rules are a text file beside the sources, included as bytes, and a test asserts that the file contains no continuation sequence and that a known indented line is present with its indentation. It is a small thing, and it is the kind of small thing that a chapter about text the model reads exists to point out: the bytes that reach the model are not the bytes you see in the source unless you have made sure they are.

```
pub const SYSPROMPT_VERSION: u32 = 1;
```

```
const INTRO: &str = r#"You are a coding agent running in a local workspace. ...  
<tools>  
"#;
```

```
const CALL_FORMAT: &str = r#"</tools>
```

To call a function, reply with one JSON object per call, each inside its own tags:

```
...  
"#;
```

```
const RULES: &str = include_str!("resources/rules.txt");
```

Chapter 6: What the model is told

```
pub fn static_prefix(tools: &ToolTable) -> String {
    let mut out = String::with_capacity(8 * 1024);
    out.push_str(INTRO);
    for schema in tools.describe_all() {
        let line = serde_json::json!({"type": "function", "function": schema});
        out.push_str(&line.to_string());
        out.push('\n');
    }
    out.push_str(CALL_FORMAT);
    out.push_str(RULES);
    out
}
```

The function takes the tool table and nothing else, which is the boundary rule enforced by the signature: there is no parameter through which a directory or a date could arrive. The tool table is a sorted map, so the schemas come out in name order and two calls with the same tools produce the same bytes. A test asserts exactly that, and a second asserts that every tool's name appears inside the fence and that the rules follow the fence, because the Qwen prompt's shape puts the call instructions after the schema list and a tool advertised below the rules that describe how to call it would be advertised in the wrong place.

The prompt as a message, not a transcript entry

Where the static prefix goes at request time is decided by two facts from earlier chapters. Chapter 1's engine takes either a flat text prefix or a structured message list, and both are views of the transcript. Chapter 5's session file holds every message the model saw and, deliberately, not the system prompt, because the prompt is the same for every session on a build and storing it per session would store the same kilobytes hundreds of times and make every file wrong when the prompt changed.

So the agent holds the prefix as a string of its own, and builds the message list for each pass by placing one system-role message in front of the transcript's messages.

```
pub struct Agent<E: Engine> {
    // ... engine, transcript, tools, ctx, store, session, meta ...
    /// The static prefix: the same bytes for every session on this build.
    /// Sent as the system message of every pass and never stored in the
    /// transcript, only fingerprinted beside it.
    pub system: String,
    /// What this launch found: directory, git state, date, instructions.
    /// Told once, as the first message of a new session.
    pub context: Option<DynamicContext>,
    /// Whether the pressure reminder has been sent in this session.
    pub reminded: bool,
}

impl<E: Engine> Agent<E> {
    pub fn prompt_messages(&self) -> Vec<Message> {
```

Chapter 6: What the model is told

```
    let mut msgs = Vec::with_capacity(self.transcript.len() + 1);
    msgs.push(Message { role: Role::System, content: self.system.clone() });
    msgs.extend(self.transcript.messages().iter().cloned());
    msgs
  }
}
```

The vector is rebuilt on every pass, and that is a few string copies against an inference call; the comment says so, and chapter 8 changes the arrangement anyway when the prefix becomes something the engine remembers rather than something it is handed. What matters here is what the test pins: the first message of every pass is the system prompt, its content is the agent's prefix, and the transcript never contains a system-role message. The hosted engine of chapter 1 already maps the system role onto its wire format, so nothing changes there. The flat renderer labels the system message like any other, so a local engine sees the prefix at the top of its text, which is where chapter 8 will want it.

The reason to make the prompt a field of the agent rather than a field of the transcript is the same rate-of-change argument that kept it out of the file. A transcript is the conversation. The prompt is context the harness supplies around the conversation, and it changes when the harness changes, not when the user speaks. Keeping it as a separate string makes the boundary visible in the type: the transcript can be saved, loaded, forked and compacted without the prompt being involved in any of it, and the prompt can change between two builds without any saved transcript becoming wrong.

Stored once, named by content

Chapter 5 said that plank stores the prompt once, fingerprinted, beside the sessions, and records in each session which fingerprint it was written under. Now blanka does the same, and the fingerprint is the second function of the module.

```
pub fn fingerprint(text: &str) -> String {
    let digest = Sha256::digest(text.as_bytes());
    let mut hex = String::with_capacity(64);
    for b in digest {
        hex.push_str(&format!("{b:02x}"));
    }
    hex
}
```

A SHA-256 of the text, as hex. Plank's is a SHA-1 of the model name and the prompt text with a separator, because plank's fingerprint names an engine snapshot and a snapshot from one model is meaningless to another. Blanka's names the text only, for now; chapter 8 combines it with the model's identity when the snapshot arrives. The property both rely on is that the name is a pure function of the content, so two sessions written under the same prompt point at the same file, and a prompt that differs by one byte gets a file of its own.

Chapter 6: What the model is told

The store gains a method that writes the prompt under its fingerprinted name if no such file exists and does nothing otherwise, using the same atomic write as the transcripts. The save path in the agent computes the fingerprint, calls the method, and records the fingerprint in the sidecar. Two new fields arrive in the sidecar with it, both optional and both defaulting when absent, so that every sidecar written in chapter 5 still loads: the fingerprint, and the working directory, which chapter 5 promised and which belongs here because this is the chapter in which the harness learns to tell the model where it is.

```
pub struct SessionMeta {
    pub id: SessionId,
    pub parent: Option<SessionId>,
    pub title: String,
    pub created: u64,
    pub last_used: u64,
    pub uses: u32,
    pub pinned: bool,
    /// The directory the session ran in. Added in chapter 6; older sidecars
    /// lack it and load as `None`.
    #[serde(default)]
    pub cwd: Option<PathBuf>,
    /// Fingerprint of the system prompt the session was written under.
    #[serde(default)]
    pub sysprompt: Option<String>,
}
```

The test for all of this runs two turns against a store, and then asserts three things: that the file named by the fingerprint holds the agent's prefix, that the sidecar's fingerprint field equals it, and that the store contains exactly one such file after two saves. The third assertion is the one that matters. The prompt is stored once per distinct text, not once per save and not once per session, and a store that has run a thousand sessions under one build has one prompt file.

What the arrangement buys is truthfulness on resume. A session resumed under a different build is resumed under a different prompt, and the sidecar knows it: the fingerprint it recorded is not the fingerprint of the running prefix. Blanka does not act on the mismatch yet, and plank does not act on it either, beyond what chapter 8 makes of it, because a conversation is still a conversation under new rules and the model copes. But the fact is recorded, which is chapter 5's rule that a session is a transcript plus the environment it assumed. The dump of chapter 5 could carry the fingerprint too, and a later chapter has it do so.

A version beside the fingerprint

A fingerprint says that the text changed. It does not say whether the change was meant. Plank's answer to the second question is a version number, and how it uses one is a small design of its own.

Plank ships through a package manager with a two-channel scheme, and its versioning document ties the number to cache maintenance in a table: a patch bump does nothing;

Chapter 6: What the model is told

a minor bump deletes the prompt snapshot, to be rebuilt on the next warm-up; a major bump, a downgrade or an unreadable version marker deletes the snapshot and the image cache too. Session transcripts are user data and are never touched. The rule that follows is stated the other way around from how you might expect: the table drives release numbering. Bump minor whenever the prompt text or the snapshot format changes, and major when older cached state must not be trusted at all. The version is not a label attached to a release after the fact; it is a declaration about what a user's cache directory may still contain.

The document is also honest about the fact that the version is not the mechanism of correctness. Because a beta is always a patch bump, a prompt change shipped in a beta does not trigger the minor-bump drop, and that is safe, because the snapshot carries a fingerprint and is rebuilt whenever the fingerprint does not match. The version-driven deletion is an optimisation that keeps dead bytes off the disk, not the thing that prevents a stale snapshot from being trusted. The fingerprint is that thing. Both exist, and they answer different questions: the fingerprint, whether this text is the one the cache was built from; the version, whether the author knew the text had changed.

Blanka's version is one constant, and the way it is enforced is a fixture test in the parity style of chapter 2. A copy of the static prefix for the default tool table is committed beside the tests. The test regenerates the prefix, compares it to the copy, and fails with a message that says what to do: if the change was intended, regenerate the fixture and bump the version. The fixture can be regenerated with an environment variable set, exactly as plank's parity fixtures can, and the test that follows a regeneration passes. What the arrangement catches is the change nobody meant: a tool whose description was edited for clarity, a rule reworded in passing, a schema that gained a field. Each of those changes the bytes the model reads and, from chapter 8 on, invalidates every cached prefix on every user's machine, and each of them should be a decision rather than a side effect. The failing test is the moment the decision is made.

```
#[test]
fn the_prompt_matches_its_fixture_or_the_version_must_move() {
    let path = concat!(env!("CARGO_MANIFEST_DIR"), "/tests/fixtures/static_prefix.txt");
    let now = static_prefix(&ToolTable::default_set());
    if std::env::var_os("BLANKA_REGEN_FIXTURES").is_some() {
        std::fs::write(path, &now).unwrap();
    }
    let pinned = std::fs::read_to_string(path).expect("fixture present");
    assert_eq!(now, pinned, "static prefix changed: regenerate the fixture and bump
    SYSPROMPT_VERSION");
}
```

The version does not appear in the model-facing text. It appears in the `/context` report and, in chapter 8, in the cache's key beside the fingerprint. Putting it in the prompt would be a volatile byte of the harness's own making, changing the fingerprint whenever the version moved even if nothing else did.

Where the model is

The dynamic context is the other half, and the way to see why it exists is to run the harness without it. Ask the model to fix a failing test, and it will ask which project. Ask it to commit, and it will not know the branch. Ask it to date a changelog entry and it will guess a year from its training data. None of these are failures of the model. They are facts about the world that the model has no way to observe and that the harness can observe in milliseconds, and the harness's job, as chapter 1 put it, is to decide what text to show the model.

Plank's context module collects four things at session start. The git block: whether the directory is inside a work tree, and if so the branch, the main branch, the configured user, the short status clipped to two thousand characters with a notice, and the last five commits in one-line form. The instruction files, covered in the next section. The date, as a local ISO date in a fixed sentence. And later, the roster of sub-agents the model may dispatch, which chapter 12 covers and which is placed here for a reason the section on skills will make plain.

Blanka's version keeps the git block, the instructions, the date and a roster of skills.

```
pub struct DynamicContext {
  pub cwd: PathBuf,
  /// Branch, short status and recent commits, or `None` outside a repository.
  pub git_status: Option<String>,
  /// Local date, `YYYY-MM-DD`.
  pub date: String,
  /// Every discovered `AGENTS.md`, nearest first, each with its path.
  pub agents_md: Option<String>,
  /// Names and descriptions of the skills the model may load by name.
  pub skills: Option<String>,
}

impl DynamicContext {
  pub fn gather(cwd: &Path) -> Self { /* git, date, files; nothing fails */ }
  pub fn render_stable(&self) -> String { /* instructions, then skills */ }
  pub fn render_volatile(&self) -> String { /* git, then the date */ }
  pub fn render(&self) -> String {
    let mut out = self.render_stable();
    out.push_str(&self.render_volatile());
    out
  }
}
```

Two decisions in the gathering deserve a word each. The first is that nothing in it fails. A directory outside any repository has no git block; a git that is not installed has no git block; an instruction file that cannot be read is skipped; a clock the C library cannot format yields the raw epoch seconds inside the same fixed sentence. Plank's findings record that last one as a deliberate mirror of the C, which falls back to raw seconds and keeps the surrounding sentence, and the reason is that the context is told to the model

Chapter 6: What the model is told

once and a launch that aborted because the date could not be formatted would be a harness that refused to work for want of a nicety. The second is the clipping of the status listing. Two thousand characters is the reference's limit, the cut lands on a character boundary because paths are arbitrary UTF-8 and a cut inside a multibyte sequence is a panic, and the notice tells the model that the listing was truncated and how to see the rest. That last clause is chapter 3's lesson applied to a context block: an error or a truncation the model reads is an instruction, so it should say what to do.

The git block's wording is the reference's, which is also the wording a well-known hosted harness uses, and the reason to keep it is not parity but familiarity. The sentence that says the status is a snapshot in time and will not update during the conversation is there because a model that has been told the status once will otherwise trust it forty tool rounds later, after it has itself changed a dozen files.

One message or two

There is a smaller choice inside the placement of the context that is easy to make without noticing, and plank's transcript format makes it visible. The context could be prepended to the user's first prompt, with a blank line between, so that the session's first user message is the context and the request together. Or it could be a user message of its own, followed by the user's prompt as a second user message, with no assistant turn between them. Plank does the second, and so does blanka, and the reasons are worth stating because the first looks tidier.

The first reason is the file. Chapter 5's transcript is a list of messages, and a resume that replays it needs to show the user's words and hide the scaffolding. If the two are one message, the replay has to find the seam inside it, and the seam is a blank line that a user's own prompt might also contain. If they are two messages, the replay recognises the first by its opening line and shows the second whole. The same applies to the title, to the dump, and to anything else that wants the user's first words: two messages keep them separable at no cost, and one message makes every consumer parse.

The second reason is what the model reads. Two consecutive user messages are unusual in a chat transcript, and a template may merge them or may keep them, but either way the model sees the context before the request and reads the request as the request. Prepending the context inside the first message puts several hundred tokens of git status and instruction text between the start of the user's turn and the user's sentence, and a model that weights the end of a message reads the sentence fine while a model that weights the start reads a status listing as the opening of the request. Plank's arrangement puts the request in a message of its own, where it is the whole of what the model is being asked, and the context in a message of its own, where it is background. The transcript format that chapter 1 chose, a list of roles and bodies, expresses that without a special case.

The third reason is the one that decides it, and it is again the cache. Chapter 8 will want the stable half of the context to be a cacheable prefix on its own, which means the volatile half and everything after it must begin at a boundary the engine can find. A message boundary is such a boundary. A blank line inside a message is not, or not reliably, because

the tokenizer may merge it with what follows. Two messages give the cache a place to cut; one message would make it look for one.

Text the user adds to the prompt

Plank's static prefix has a part blanka's does not, and its treatment is a lesson about the boundary from the other side. The reference accepts a system flag: text the user supplies on the command line, appended to the built-in prompt after a blank line. It is stable across sessions in the sense the boundary rule requires, since a flag does not change between two launches with the same command, so it legitimately belongs inside the fingerprinted prefix, and plank's boundary test passes user text through the builder to make sure the fingerprint covers it. But the reference tokenizes the two parts differently, and the port had to as well.

The C's built-in prompt is tokenized as rendered chat, so that the marker characters in its examples become the model's dedicated control tokens, the form the model saw in training. The user's text is tokenized as plain content. The comment in the C says why in two sentences plank quotes in full: arbitrary user text containing a turn marker or a thinking tag or the markup's own marker must remain plain content, not control tokens. A user who put the closing marker of a turn in their system text, by accident or on purpose, would otherwise end the system turn early and begin writing the model's reply. That is a prompt-injection boundary, and plank's findings record the consequence for the port as one of the second day's discoveries: the system prompt is tokenized in two different ways, so composing the parts as one string is fine for display and fingerprinting but not for tokenization. Plank's builder therefore returns the text together with the byte length of its trusted span, and the engine tokenizes up to that length one way and the rest the other.

Blanka's model has no dedicated marker tokens, so its prompt has no span to protect and no flag to append. The chapter mentions the mechanism because chapter 13, which accepts tool schemas from external servers, will need the span for plank's second reason: the schemas are third-party text and sit outside it, beside user text, so that neither can forge a turn boundary. The boundary drawn there is wider than the C's, because the C's prompt is entirely built in, and plank's comment on it ends with an instruction not to widen it further.

The other thing plank's documents say about user text is a design that was planned and, when this was written, not built: per-project files that replace or append to the prompt, in the convention of another harness. The plan is short and most of its pressure comes from the boundary. The files are read from exactly one place, the working directory, with no walk up the tree and no merge with a global copy, because a system prompt is one artifact with one author and merging two produces a prompt nobody wrote. A replacement that dropped the tools block would silently break tool calls, so the plan gives it a placeholder for the block and a loud warning when the placeholder is absent. And a per-project prompt makes the fingerprint per-project, so alternating between two projects thrashes a single cached snapshot; the plan accepts that for a first version, because correctness is unaffected and only users who opt in pay, and notes the fingerprint-keyed file names that would remove it. Blanka does not build the feature either. It is here because it shows what the boundary rule looks like when a new input

Chapter 6: What the model is told

arrives: the first question is whether the input is stable across sessions, the second is what it does to the fingerprint, and the third is who pays.

One file name

The project instructions are the piece of the context that a user writes, and the design question is which files to read. Plank's answer changed once, and the change is more instructive than either answer alone.

The first answer was to read the reference's file name and fall back to a second, widely used name when the first was absent, because many projects carry one and not the other and a harness that ignored it would be ignoring the instructions the project's maintainers wrote. That worked, and it had a cost that took weeks to notice. The model saw a file that was written for another tool, with that tool's name in its heading and that tool's commands in its body, and the harness had no way to tell the model which parts applied. A file higher up the tree, written for a different project's use of the other tool, was read too, because the discovery walks upward, and it applied even less. And the fallback meant that the file the model saw was sometimes one name and sometimes the other, so a user reading a dump could not tell which without checking the disk.

The second answer, in a commit from early September titled for it, reads one name only. A project that carries only the other file is handled at interactive startup: the harness creates a symbolic link from the name it reads to the name that exists, with a relative target so the checkout can move, and says so. A project with neither is offered a command that has the model read the repository and write the file. A headless run does none of this, because it must not write into a checkout or block on a question. And an existing file, even a dangling link, is left alone, because replacing it would destroy the user's intent. Plank later added one sibling to the one name: a personal, uncommitted `AGENTS.local.md`, read after `AGENTS.md` at each level of the walk so that the personal file has the final word where the two disagree. It is a second file, not a second name for the same file, and the rule about one name stands. The commit's message gives the principle in one clause: so the file the model sees is always the one named for it, and a file elsewhere up the tree is left to the tool it was written for.

Blanka reads the one name and links the other.

```
pub fn discover(cwd: &Path) -> Vec<PathBuf> {
  let mut found = Vec::new();
  let mut dir = Some(cwd);
  while let Some(d) = dir {
    let candidate = d.join("AGENTS.md");
    if candidate.is_file() {
      found.push(candidate);
    }
    dir = d.parent();
  }
  found
}

pub enum Startup { Present, Linked(PathBuf), Missing }
```

Chapter 6: What the model is told

```
pub fn prepare(dir: &Path) -> Result<Startup, String> {
    let agents = dir.join("AGENTS.md");
    if agents.symlink_metadata().is_ok() {
        return Ok(Startup::Present);
    }
    if !dir.join("CLAUDE.md").is_file() {
        return Ok(Startup::Missing);
    }
    std::os::unix::fs::symlink("CLAUDE.md", &agents)
        .map_err(|e| format!("cannot link AGENTS.md to CLAUDE.md: {e}"))?;
    Ok(Startup::Linked(agents))
}
```

Discovery walks from the working directory to the filesystem root and collects every file of the one name, nearest first, and the test places one at the root of a scratch tree and one two levels down, with the other tool's file between them, and asserts that both of the right name are found in the right order and that the other is not. The preparation check uses the link-aware metadata call so that a dangling link counts as present, and the test that links asserts that the target is the bare name, that reading through the link yields the file's content, and that a second start finds the link and does nothing. A third test asserts that a directory with neither file is reported as missing and remains empty, which is the promise that the harness writes nothing unless there is something to link to.

Each discovered file is rendered into the context with a header naming its path, so that a model reading two sets of instructions can see which project each came from, and so that a user reading a dump can see which files the model saw. The rendering is the first block of the stable half, because instruction files change when someone edits them and at no other time.

The command that writes a missing file is one prompt. It asks the model to read the repository's build files, readme and main sources and write the file at the root with what the project is, how to build and test it, the layout and the conventions, under eighty lines, using the write tool, and then to summarise. The harness writes nothing itself. That is the right division for the same reason chapter 3's tool descriptions steer rather than enforce: the content of the file is a judgement about the project, and judgement is the model's job. The harness's job is to notice the file is missing and to say what the model should do about it.

Plank's version of the command later grew into six phases and kept the same division. The phases, asking what to set up, surveying the codebase, asking the follow-ups the survey could not answer, writing the project file, writing the personal one and summarising, live in one static prompt that the model executes, using the multiple-choice tool and the sub-agent tool that chapter 12 builds. Keeping the phases in the prompt rather than in the front end is what lets both of plank's front ends share one path, and when there is nobody to ask, the question tool refuses and the prompt tells the model to proceed on its own judgement. Two small rules came with it. Only the offer made at launch clears the session afterwards, because that clear is what rebuilds the context around the newly written file; an init the user typed keeps the conversation it

interrupted. And plank's own benchmark caught the prompt naming the survey tool with a word that another tool also used. Every run, on every model, dispatched the survey to the to-do tool first, got an error back, and only then found the right one. When two tools share a word, a prompt that uses the word informally is read as a tool name, and plank's parity test now checks that the prompt names only real tools and never the wrong one again.

The reminder, and why it is pressure-based

Long into a session, the rules fade. This is not a metaphor. A model attends to its whole context, but what it attends to most is what is recent, and after a hundred tool results the rules at the top of the prompt are a long way from the frontier of the conversation. The symptom is a model that batched its calls for the first hour and stops, or that starts prefixing shell commands with a change of directory again. The reference's answer is to say the rules again, and how often to say them is the design decision.

Plank's findings file records the reference's choice and plank's adoption of it in one line: the reminder is pressure-based, not periodic. It is re-injected only once the token distance since the model last saw it exceeds fifty thousand, a constant the C names and plank keeps. A periodic reminder, every so many turns, would fire in a short conversation of quick turns where nothing had faded, and would fail to fire in a long conversation of few enormous tool results where everything had. Distance in tokens measures the thing that causes the fading, so it is what the reminder is keyed on.

Blanka's reference model has a context window a fraction of the size the reference's constant assumes, so blanka keys its reminder on the share of the window in use rather than on an absolute distance, and fires once when the share crosses three quarters. The floor is the same one chapter 7 will use for compaction, and the two are deliberately aligned: a window that is three quarters full is a window in which the rules are far away and the head is about to be summarised, and saying the rules again before the summary means the summary is written under them.

```
pub const PRESSURE_FLOOR: f32 = 0.75;

pub fn pressure_reminder(used: usize, ctx: usize) -> Option<String> {
    if ctx == 0 {
        return None;
    }
    let share = used as f32 / ctx as f32;
    (share >= PRESSURE_FLOOR).then(crate::sysprompt::reminder)
}
```

What the reminder contains is the rules, between two marker lines, and not the whole prefix. Plank's first reminder re-injected the entire tools prompt, because the reference does, and its later default is a lesson in where a reminder's cost lies. Tokenizing the reminder takes milliseconds. Running its tokens through the model is the seconds, and those rows of engine state are valid only at the position and after the transcript they were computed for, so unlike the prefix at position zero there is nothing to snapshot and reuse. The only lever is the size of the text. Plank's short reminder, on by default, keeps the

Chapter 6: What the model is told

reference's framing markers and carries only what a reminder is for: the tool-call syntax in the loaded model's dialect, the names of the advertised tools, and one line saying that the prompt at the start of the conversation still applies in full. The byte-identical reference reminder remains one setting away, so that drift can be compared against it. The overrides document notes a subtlety that applies to the long form: when a user has replaced the prompt with a file of their own, the reminder must re-inject the prompt in use rather than the built-in one, or the session drifts back toward text the user removed. Blanka has no override mechanism, and the schemas have not faded in the way the rules have, because the model has been using the tools all along and each result frame names one. So blanka's reminder is the rules, which are what the dumps showed being forgotten.

Where the reminder goes was settled in chapter 4 and chapter 5, and this is the place both chapters were pointing at. It is a user-role message, pushed into the transcript, and therefore saved in the file and replayed on resume. Chapter 4 said that a user-role entry is the form the model was trained to read as coming from outside itself, and used it for the interruption note. Chapter 5 said that anything injected at request time is logged as a transcript entry at the moment it is injected, rather than recomputed at replay, because recomputation drifts. The reminder is the second injected text, and it follows both rules. The agent tracks whether it has been sent, so that a session that stays above the floor for thirty more turns is not reminded thirty times; the flag is cleared by a new session and, in chapter 7, by compaction, which brings the share back down and makes the next crossing a new event.

```
// in run_turn, after a tool result is pushed
let used = stats.prompt_tokens + stats.completion_tokens;
if !self.reminded {
  if let Some(text) = crate::context::pressure_reminder(used, self.engine.ctx_size()) {
    self.transcript.push(Role::User, text);
    self.reminded = true;
    sink.status("[system prompt reminder injected]");
  }
}
```

The test for it uses a window of sixty-four tokens and a script of two tool calls and an answer, so that the first tool round already crosses the floor, and asserts that exactly one user-role message beginning with the reminder's opening marker is in the transcript when the turn ends, that the flag is set, and that the status channel carried a line about it. The last assertion is chapter 4's honesty rule in one more place: the user is told that the harness has injected text, because a transcript that gained a message nobody typed and nothing announced is a transcript that lies about itself.

Scaffolding the user did not type

Three kinds of user-role messages now exist that no user typed: the session context, the interruption note, and the reminder. Each is a user-role message because that is the form the model reads correctly, and each is saved because that is what the log-everything rule requires. Two consequences follow, and both need a line of code.

Chapter 6: What the model is told

The first is replay. Chapter 5's resume replays the transcript through the renderer so that a resumed session looks the way it looked, and a replay that showed the session context as if the user had typed a git status listing and a date would look wrong. Plank's fix is a function that recognises the context blocks by their opening lines, and a constant listing those lines, with a test that builds a real context and asserts that every block it can produce is recognised and that two plausible things a user might type, including one that mentions the block's own header in a question, are not.

```
pub const CONTEXT_MARKERS: [&str; 4] = [
  "Project instructions",
  "Available skills",
  "This is the git status at the start of the conversation.",
  "Today's date is ",
];

pub fn is_session_context(text: &str) -> bool {
  let t = text.trim_start();
  CONTEXT_MARKERS.iter().any(|m| t.starts_with(m))
}
```

Recognition by opening line rather than by position is a choice plank's comment explains. The stable half starts with whichever of its parts exists, both halves are stored as ordinary user turns, and the interruption note and the reminder can appear anywhere, so there is nothing else to key on. The alternative, a flag on the message saying it is scaffolding, would be cleaner and would change the transcript format, which chapter 5 arranged to be able to grow; blanka keeps plank's approach because the message the model reads is the message the file holds, and a flag the model cannot see is a second source of truth about a message that already carries the only one that matters.

The second consequence is the title. Chapter 5 made the first user message the session's title, and the first user message is now the session context, so every session in the smoke run of this chapter was titled with the path of an instruction file until the title function learned to skip scaffolding. It is a one-clause change and a one-line test, and it is the kind of consequence that a chapter about injected text has to go looking for: every place that treats the first user message as the user's first words is now wrong, and the store's title was the one blanka had.

Skills, and what belongs in the prompt

The last block of the context is a list of skills, and the reason it is in the context rather than the prompt is the sharpest illustration of the boundary rule in the chapter.

A skill, in plank's design and in blanka's, is a directory holding a markdown file with optional frontmatter, a name and a description, followed by a body of instructions. A user invokes one by typing its name as a command with arguments, and the harness substitutes the arguments into the body and runs the result as a turn, so a user who reviews diffs the same way every time writes the procedure once and types one word thereafter. Plank added the mechanism on its second day, with global skills in the user's

Chapter 6: What the model is told

home directory overlaid by project skills in the checkout, a project skill replacing a global one with the same name, and a shape that chapter 13's plugins reuse.

Two days later plank let the model invoke skills too, through a tool, and that raised the question this section is about. The model needs to know which skills exist in order to choose one. The obvious place to tell it is the tool's description, in the tool table, in the system prompt: a list of names and descriptions in the parameter's description, the way a schema lists an enumeration. Plank did the equivalent for its sub-agent roster and then, in a commit from August titled for the change, moved the roster out of the system prompt and into the session context, and the commit's reasoning is the boundary rule with numbers attached. A definition is a file on disk that a user edits. Editing one changed the tool description, which changed the prompt, which changed the fingerprint, which invalidated the cached prefix, which cost a refill of the whole prompt on the next launch. Moving the list into the stable half of the context meant an edit cost a rebuild of the much smaller project-tier cache instead. The tool's description was fixed to say nothing about which definitions exist, and a test asserts that the roster is in the stable context, not in the volatile one, and not in the prompt at all.

Blanka's skill tool follows the same rule. Its description says that skills are listed in the session context and that a call with no name lists them. The list itself is rendered into the stable half of the context, one name and description per line, and only for skills the frontmatter has not marked as user-only, because a skill the model must not reach for should not be advertised to it. A test asserts that the tool's description names no skill, and a second asserts that a user-only skill is absent from the roster and refused by the tool with an error that lists what is available, so a near miss self-corrects.

```
pub struct Skill {
  pub name: String,
  pub description: String,
  pub body: String,
  /// Whether the model may load it by name. Frontmatter
  /// `model-invocable: false` withholds it; the user can still invoke it.
  pub model_invocable: bool,
}

pub fn load_skills(roots: &[PathBuf]) -> Vec<Skill>;
pub fn render(skill: &Skill, args: &str) -> String;
pub fn roster(skills: &[Skill]) -> Option<String>;
```

The loader rejects a name with whitespace, a slash or a colon, because the name becomes a command and a colon is how chapter 13 addresses a plugin's skill, so a bare skill claiming one would squat on a namespace. It rejects an empty body. It sorts by name so listings are stable. And rendering substitutes the arguments into a placeholder if the body has one, or appends them as a paragraph if it does not, so that arguments are never silently dropped. The tool caps the number of loads in a session, because a skill whose body says to load itself again would otherwise loop, and chapter 10 has more to say about loops of that shape; here the cap is eight and the error says so.

Chapter 6: What the model is told

Plank later compiled six skills into its binary, a code review, a debugging procedure, a memory helper, one that turns a session into a new skill, one that edits settings and one that verifies a change, and the decision worth copying is how they are loaded. They are ordinary skill files, parsed by the same loader as a user's, so a built-in can use nothing a user's skill could not, and they sit beneath the user's and the project's skills in the same override order, so a project skill with the same name replaces the built-in outright rather than colliding with it. Their text was also rewritten once so that a skill restates the facts it needs in its own words rather than copying sentences from the project's documentation, because a skill that quotes a document goes stale with it. Chapter 13 returns to the layering, where plugins complicate it.

Plank's templates are the lighter sibling: a single file with named holes rather than a directory with one placeholder, the same discovery and override rules, and a hard error listing every missing variable rather than a silent empty substitution. Plank stops at skills. The mechanism the two share, and the mechanism the instruction files share with them, is the one this chapter has been describing from the start: text that the harness composes and tells the model, kept in the block that matches how often it changes.

Everything the home directory says

The boundary rule says where each kind of text goes. It does not say how much of it there is, and plank's benchmark harness found that the answer was more than anyone had meant. Run against a scratch project, every model opened a small coding task by loading two skills, one a probe and one a brainstorming procedure the machine's owner had installed, and one model spent passes classifying the task before writing a line. Every run of one task first tried to write to a path in a different project on the same machine, and was refused by the containment of chapter 3. The path came from the cached instructions of an MCP server, which listed the owner's other projects. Skills, plugin hooks, memory, server configuration and settings were all read from the home directory, and every one of them reached the model.

The harness now runs plank under a scratch home that holds the model files and nothing else, and the symptom that led there is the part worth keeping: a refused write to a path the prompt never mentioned means the path came from context, and the dump shows exactly which line. For a harness of your own the conclusion is that the session context is not only what the context module gathers. It is everything any extension contributes, and a measurement that means to test the prompt has to control all of it.

The commands

The chapter's binary gains three commands and one class of them. A context command prints the token accounting by block: the static prefix with its version and the first twelve characters of its fingerprint, the session context, the conversation, and the total against the window as a share, with a line saying whether the reminder has been sent. An init command runs the instruction-writing prompt as a turn. A skills command lists what was loaded, marking the user-only ones. And any other command whose name matches a skill renders the skill with the rest of the line as arguments and runs the result as a turn.

The last of those changes the shape of the command function, which used to return text and now returns either text or a prompt.

Chapter 6: What the model is told

```
pub enum Answer {  
    Text(String),  
    Prompt(String),  
}  
  
pub fn command<E: Engine>(agent: &mut Agent<E>, line: &str, skills: &[Skill]) ->  
    Result<Answer, String>;
```

A command that answers in text is printed and the loop continues. A command that becomes a prompt is run as a turn, exactly as if the user had typed the rendered body, which is what a skill is: a prompt the user did not have to type. The distinction is worth having in the type because the two paths differ in what they cost and what they record. Text is free and leaves nothing in the transcript. A prompt is a pass and lands in the file, and the model reads it as the user's words, which for a skill is the intent.

The context report is the one command in the set that exists for the reader as much as for the user. Run against the reference model, it shows the static prefix at about thirteen hundred tokens, the session context at a few dozen in a small repository and a few hundred in one with a long instruction file, and the conversation growing pass by pass. It is the first time the book puts numbers beside the blocks of the prompt, and chapter 8 begins by asking what those thirteen hundred tokens cost on every pass and whether the engine could be made to remember them.

The smoke run

The run that accompanies the chapter's code is short, and every line of it exercises something the chapter built. A scratch home directory holds one global skill that reviews a diff. A scratch project holds only a file named for the other tool. The harness starts, links the one name to the other and says so. The skills command lists the one skill. Invoking it with an argument produces a turn whose prompt is the skill's body with the argument substituted, which the scripted engine echoes back. The context command shows the prefix, the context and the conversation with their token counts, and the version and fingerprint of the prefix. The init command produces a turn with the instruction-writing prompt. And the session listing shows one session, titled with the first thing the user typed rather than the context, with the store holding its transcript, its sidecar, and one prompt file named by the fingerprint the sidecar records.

The transcript file, opened in an editor, shows what the model saw in order: a user-role message beginning with the project instructions from the linked file and ending with today's date, then the rendered skill, then the echo, then the init prompt, then its echo. No system-role message, because that is the prefix and it is in its own file. If the run were made against the real model instead of the echo engine, the init prompt would produce reads and a write and a summary, and the linked file would be replaced by a real one on the next start, which is the point of the offer.

Where blanka stops short of plank

Plank's prompt module is two thousand lines and blanka's is a hundred and fifty, and the gap is again a map of what the book defers and what it leaves.

Chapter 6: What the model is told

Plank carries three dialects of the tools prompt, one for the reference's markup, one for a newer DeepSeek generation that respells the same markup and has its own syntax reminder, and one for the Qwen family, and a separate prompt for hosted providers that has the same behavioural guidance without the syntax section, because native tool definitions replace it. It carries a trusted span: the leading part of the prompt that is tokenized as rendered chat so that the markup's marker characters become the model's dedicated tokens, with user-supplied text and third-party tool schemas deliberately outside the span so that neither can forge a turn boundary. That is a prompt-injection boundary as well as a tokenization one, and its exact position, wider in plank than in the C because plank appends schemas from external servers, is documented in a comment that ends with the words *do not*. Blanka's model has no such marker tokens and blanka accepts no external schemas until chapter 13, so the span does not exist yet; the chapter on plugins draws it.

Plank's context also carries persistent memory, two flat files of dated, typed notes, one for the user and one for the project, rendered into the stable half beside the instruction files under per-type budgets. Its placement there fixes how it can change: a note the model or the user writes lands on disk at once and joins the context only at the next session start, because splicing it into the live context would force the cache chapter 8 builds to re-read everything after it, and the tool's own result tells the model so. Chapter 7 describes the pass that maintains those notes; blanka builds none of it. Plank has a settings-driven commit trailer instruction that is appended after the trusted span, and a plan, not yet implemented when this was written, for per-project files that replace or append to the prompt, with a placeholder so that a replacement can keep the tools block and a warning when it does not. And plank's reminder re-injects the syntax and the tool roster at a fifty-thousand-token distance, or the whole tools prompt on request, where blanka's re-injects the rules at three quarters of a smaller window.

What blanka has is the boundary, drawn formally and tested; a static prefix that is a pure function of the tool table, stored once by content and pinned by a fixture with a version beside it; a session context in two halves, logged as the first user message and recognised on replay; one instruction file name with a link for the other; a pressure reminder logged as a user message; and skills the user invokes by name and the model loads through a tool whose description names none of them.

Testing text the model reads

The tests in this chapter are of a kind the earlier chapters had fewer of, and the kind deserves a name because chapters 8 and 9 will lean on it. They are tests about bytes the model will read, and what they assert is not that the bytes are correct, which no test can know, but that they are what the author decided.

The determinism test asserts that two calls produce the same prefix. The fence test asserts that every tool is advertised before the rules that say how to call it. The indentation test asserts that the rules file has no continuation sequence and that a known indented line kept its indent. The fixture test asserts that the prefix is the pinned one or the version must move. The boundary tests assert that the stable half of the context contains no date and the volatile half no instructions, and that rendering is their concatenation in that order. The recognition test asserts that every block the context can produce is recognised

Chapter 6: What the model is told

as scaffolding and that two things a user might type are not. The agent tests assert that every pass leads with the prefix, that the transcript never holds it, that the context is told once and only once across two turns, that the date is not in the prefix, that the reminder appears exactly once as a user message when the window fills, and that a store holds one prompt file after two saves.

None of these tests runs a model. All of them would fail loudly on the mistakes plank's history records: the continuation that ate the indentation, the roster that lived in the prompt, the date that would have made the cache single-use. That is the value of the kind. A harness's prompt is a piece of writing that a program assembles, and the program can be tested for whether it assembled what was written.

Standing instructions

What the model is told before the conversation begins is two things, kept apart by how often they change. The static prefix is the same bytes for every session on a build: a role, the tool schemas inside a fence, the call format, and rules each of which records an observed failure. It is a pure function of the tool table, stored once beside the sessions under a fingerprint of its content, pinned by a fixture, and versioned by hand so that a change is a decision.

The session context is what this launch found: the project's instruction file, read under one name and linked from the other, the skills the model may load, the git state and the date. It is told once, as the first user message, stable half before volatile half, saved in the transcript and recognised on replay by its opening lines. The boundary between the two is a rule a test enforces, because chapter 8 keys a cache on the prefix and a volatile byte in it would make that cache miss on every launch.

Under pressure the rules are said again, once, as a user message that the file keeps. Skills are a user's prompts under a name, and the list of them lives in the context rather than the prompt, because editing a file the user owns must never change the bytes the cache is keyed on.

CHAPTER 7: FORGETTING ON PURPOSE

A context window is finite, and a coding session is not. Somewhere between the fortieth and the four hundredth tool result, the transcript the harness has been faithfully appending to since chapter 1 becomes larger than the model can read, and something has to give. The engine does not decide what. A local engine given more tokens than its window holds will fail, or silently drop the beginning, or produce garbage; a hosted one returns an error that says the prompt is too long. Either way, the harness that promised in chapter 5 to keep everything now has to choose what to forget, and it has to choose well, because what it drops is what the model will not know a moment later.

That the choice exists at all is the first thing to be clear about, because a naive harness never confronts it. It sends the whole transcript, and one day the send fails, and the session is over. Plank's port had the layered answer to this on its second day, because the C reference has one and because the reference hosted harness it was compared against has a more elaborate one, and the design of that answer is most of this chapter. It comes in two steps of increasing cost. The cheap step rewrites old tool results in place, with no model call, on the theory that a file the model read thirty rounds ago is the part of the conversation it needs least. The expensive step asks the model to summarise the conversation so far and replaces everything but a recent tail with the summary. Around both sit the questions of when to run them, what must never be dropped, and what to do about a turn that will not end. Plank's history supplies an answer to each, and in one case a two-hour session that answered a question nobody had thought to ask.

What can be dropped

Look at a transcript after an hour of work and most of it is tool output. The user typed a few hundred words. The model wrote a few thousand, in reasoning and replies and requests. The rest, often nine parts in ten, is what the tools returned: files read in full, directory listings, search hits with context, test output, the contents of a file before and after an edit. Of that, most was read once, acted on, and never looked at again. The model asked for a file, saw it, made an edit, and moved to the next; the file's five hundred lines remain in the window for the rest of the session, costing their tokens on every prefill and displacing something the model might have wanted.

That observation is the whole basis of the cheap step, which plank and the reference both call microcompaction. A tool result is a user-role message in the reference's design and a tool-role message in plank's, and either way it is a message the harness wrote, so the harness may rewrite it. The rewrite replaces the body with a short stub that says what happened and what to do about it: an old tool result was cleared to reclaim context, and the tool can be rerun if the output is needed again. The conversation's shape is untouched. The user's words, the model's replies, the requests, and the fact that a result came back are all still there; only the bulk is gone. No model call is made, so the step costs nothing but the rewrite.

```
pub const MICRO_MIN_BYTES: usize = 256;
pub const MICRO_STUB: &str =
```

Chapter 7: Forgetting on purpose

```
"[old tool result cleared to reclaim context; rerun the tool if the output is needed again]";
```

Two things about the stub deserve a sentence each. It is text the model reads, so by chapter 3's rule it is an instruction, and it says the one thing the model can do about the situation: rerun the tool. And it has a fixed wording that plank pins in a fixture, because the reference's model saw exactly that wording in training and a model that has never seen a stub may treat a cleared result as an error rather than as an invitation.

Not every result may be cleared, and the rules about which are the substance of the step. Small results are exempt, because a stub of ninety bytes replacing a body of two hundred reclaims nothing. Results in the current batch are exempt: everything after the last assistant message that precedes a result is what the model just asked for and has not yet read, and clearing it would hand the model a stub for a file it requested a second ago. That one rule keeps two promises, as plank's comment puts it: a result the model has not yet acted on is never cleared, and the results of the batch it just ran are never cleared as a set. And results that belong to the current task, defined later in the chapter, are exempt too. Everything else, oldest first, is a candidate.

```
pub fn clear_set(t: &Transcript) -> Vec<usize> {
    let msgs = t.messages();
    let task_inject = msgs
        .iter()
        .rposition(|m| m.role == Role::User &&
m.content.starts_with(crate::goal::TASK_LIST_OPEN));
    let candidates: Vec<usize> = msgs
        .iter()
        .enumerate()
        .filter(|(i, m)| m.role == Role::Tool && m.content.len() > MICRO_MIN_BYTES)
        .map(|(i, _)| i)
        .collect();
    let batch_start = candidates
        .last()
        .and_then(|&last| msgs[..last].iter().rposition(|m| m.role == Role::Assistant));
    candidates
        .into_iter()
        .filter(|&i| {
            let is_task = task_inject.is_some_and(|ti| i > ti);
            let is_batch = batch_start.is_none_or(|b| i > b);
            !is_task && !is_batch
        })
        .collect()
}
```

The test builds a transcript of five large results and one tiny one across three tool rounds and asserts that the clear set is the three old large ones and nothing else: not the tiny one, not the two in the batch after the last assistant turn. It is the same shape as plank's test, and it pins the same three promises.

Three places to lose bytes

Before the cheap step there is a cheaper one, and it was built in chapter 3 without being named as part of this design. The spill policy that caps a tool result at dispatch time, writing the full output to a file and handing the model a preview and a locator, is the first place a harness controls how much of the window a result occupies. It acts before the result enters the transcript, so it costs no rewrite and invalidates no cache. What it cannot do is know the future: a result that is small enough to keep at dispatch time may still be the one to drop forty rounds later, when the window is full and the result is stale. Microcompaction is the same idea applied at read time instead of write time, to results whose staleness is now known. And the summary is the idea applied to the whole conversation, when even stale results are not enough to drop.

The three sit in a sequence by cost and by loss. The spill loses nothing the model needs at the moment, because the locator gets the rest back with one call. The stub loses a result the model has already acted on, and says how to get it back. The summary loses the model's own judgement of what did not matter, and cannot be undone. A harness that has only the third pays the model call for every full window; a harness that has only the first fills its window with small stale results and never reclaims them. The order is the order in which they should be tried, and the escalation in this chapter is the second and third steps because the first already runs on every dispatch.

The alternative that a harness author reaches for first, and that none of the three is, is a sliding window: drop the oldest messages once the transcript exceeds a size. It is the design that needs no model call and no rules about batches, and it is wrong for a reason that the eight-section summary makes visible by contrast. The oldest messages are the user's request and the model's first reading of the problem, and they are the part of the conversation that everything after depends on. A sliding window forgets the task and remembers the last file read. The summary forgets the last file read and remembers the task, and the tail keeps the last file read anyway. That is why the expensive step is a summary and not a cut.

The two-hour session

Deciding what may be cleared is half the step. Deciding when to clear it turned out to be the other half, and plank got it wrong in a way that is worth telling in full, because the failure is not obvious in advance and the fix is one line that the rest of the chapter depends on.

Plank's first version of the cheap step ran at the end of every turn, opportunistically, whenever it would reclaim more than a few kilobytes. The reasoning was sound as far as it went: clearing is cheap, a fuller window is a slower prefill, so clear early and often. Two refinements followed over August, both about the engine cache that chapters 8 and 9 build, and both are recorded in the findings file as a sequence of three gates each fixing the previous one's blindness. The first gate counted bytes reclaimed and could not see that rewriting an early message invalidates every cached token after it; a measured session paid fourteen thousand tokens of re-prefill to reclaim twelve kilobytes, five times, and spent twenty-six of its twenty-nine minutes prefilling. The second gate weighed bytes reclaimed against tokens re-prefilled and demanded two bytes per token; it refused

Chapter 7: Forgetting on purpose

every pass, and the session fell to fifteen minutes. The third relaxed the ratio as the window filled, on the argument that a byte of context is worth more when the alternative is a full compaction. All three were about cost. None was about whether clearing was the right thing to do at all.

Then, in early September, a session ran for two hours and made no edit. The dump, of the kind chapter 5 built, told the story once it was read. The window was a million tokens and seven percent full. The model was reading a long plan file in chunks and a dozen source files. Every read landed in the clear set at the end of the turn, because the gate's cost term was near zero, the cache having a snapshot just before each result. So at the end of every turn the model's freshly read files were replaced by stubs. At the start of the next turn the model, seeing stubs where its files had been and being told to rerun the tool, reran the tool. At temperature zero the same visible context produces the same reply, so the eight tool calls it made were the same eight tool calls, and the cycle repeated: two hundred and fourteen reads, thirty shell commands, zero edits, two hours. The loop guard of chapter 10 never fired, because its window was ten calls and the cycle was eight, so no signature reached the repeat count.

The findings file records the lesson as a hard precondition placed in front of the whole gate: the cheap step may run only when the window is at least three quarters full. Below that, no pass fires regardless of how cheap the cache arithmetic says it is, because clearing tool output destroys the information the model is working from, and that cost is only worth paying when the window is genuinely short. The constant is the same seventy-five percent that chapter 6 used for the reminder, and it is the same floor here for the same reason: three quarters is where the window has become the problem rather than a resource.

```
pub const PRESSURE_FLOOR: f32 = 0.75;

pub fn microcompact(t: &mut Transcript, used: usize, ctx: usize) -> usize {
    if ctx == 0 || (used as f32 / ctx as f32) < PRESSURE_FLOOR {
        return 0;
    }
    let floor_tokens = (ctx as f32 * PRESSURE_FLOOR) as usize;
    let mut budget = used.saturating_sub(floor_tokens).max(1);
    let stub_tokens = approx_tokens(MICRO_STUB);
    let mut cleared = 0;
    for i in clear_set(t) {
        if budget == 0 {
            break;
        }
        let old = t.rewrite_tool_result(i, MICRO_STUB);
        budget =
budget.saturating_sub(approx_tokens(&old).saturating_sub(stub_tokens));
        cleared += 1;
    }
    cleared
}
```

Blanka's version has the floor and one more idea from the same repro. Plank's first sweep cleared everything clearable in one pass, however much that was, and the two-hour dump showed the cost: a thirteen-chunk read of the plan file lost its first ten chunks before the model reached the end. The pass is now budgeted. It clears the oldest candidates until it has reclaimed enough to bring the window back under the floor, and then stops, so recent work survives for as long as the window allows. The test asserts three things in sequence: under the floor nothing happens; just over it, one old result is enough and the newer ones stay; far over it, everything clearable goes and the current batch still stays. And a second call with nothing left to clear returns zero, which is what makes the pass safe to run every round.

The token arithmetic uses the four-bytes-per-token estimate from chapter 1 rather than the engine's count. Plank's uses the engine's tokenizer, through a closure, because plank's budget feeds a cache decision that has to be exact. Blanka's pass is about relief, and a pass that reclaims a little more or less than intended is not a defect. Precision arrives in chapter 8, where the estimate is replaced by the engine's own count in the one place it matters.

When the cheap step is not enough

A budgeted pass over the clear set can fail to bring the window under the floor, because the clear set is empty or too small: a session whose bulk is the model's own long replies, or the user's, or a single enormous result in the current batch. Then the harness needs the expensive step, and the expensive step is a model call.

The idea is old and the reference hosted harness made it standard. Ask the model to write a summary of the conversation so far, and replace the conversation with the summary. What plank adds, following the reference, is a shape for the summary and a shape for what surrounds it. The request is a fixed prompt, sent as a user message at the end of the live transcript, that asks for eight numbered sections: the primary request and intent, the key technical concepts, the files and code sections with exact paths, the errors and fixes including rejected approaches, every user message condensed in order, the pending tasks, the work in progress at this moment, and the next step if one was requested. It invites the model to reason first inside an analysis block that will be discarded, and to wrap the summary in tags that the parser looks for. It forbids inventing facts, generic narration, and raw file contents, preferring paths and commands that can reload the data. And it ends with the instruction to stop: do not continue the task, do not call tools, do not output thinking or tool-call tags.

The order of those instructions is a design decision plank's comment defends. The no-tools line is the last thing in the prompt so that it is the last thing the model reads before writing, and any caller-supplied instructions go above it, so that nothing can displace it. The reference measured why: on one model generation, a summary request without the trailer produced a stray tool call nearly three percent of the time, against a hundredth of a percent on the previous generation, and a stray tool call in a summary request wastes the pass and, on a local engine, a full re-prefill. The tags are a contract between the prompt and the parser, and plank pins the contract with a test that builds a reply from the tag names the prompt itself uses, so a rename in the prompt moves the test's input with it and only a mismatch between the two can fail.

Chapter 7: Forgetting on purpose

```
pub fn extract_summary(raw: &str) -> String {
    let mut text = raw.to_string();
    while let (Some(start), Some(end)) = (text.find("<analysis>"), text.find("</analysis>"))
    {
        if end < start {
            break;
        }
        text.replace_range(start..end + "</analysis>".len(), "");
    }
    if let Some(start) = text.find("<summary>") {
        let body = &text[start + "<summary>".len()..];
        let body = body.find("</summary>").map_or(body, |end| &body[..end]);
        return body.trim().to_string();
    }
    text.trim().to_string()
}
```

The parser drops every analysis block, unwraps the summary if the tags are there, and falls back to the stripped text if the model ignored the contract, because a model that wrote a good summary without tags has still written a good summary. The test covers the well-formed reply, plain text, an unclosed summary tag, and a reply that is nothing but analysis, which yields an empty string and matters in the next section.

What a summary is for

The eight sections are not an arbitrary outline, and reading them as a specification of what a resumed conversation needs is the way to see why each is there. The primary request and intent is what a sliding window would have lost first. The key concepts are the vocabulary the model has been using, so that the next pass does not rename things. The files and code sections, with exact paths and ranges, are what let the model reload data rather than carry it: the prompt says so, preferring a path and a command that can reload a file to the file's contents. The errors and fixes, including rejected approaches, are the section that stops a model from trying again what already failed, and a summary without it produces a session that re-discovers the same dead end. The user messages, condensed in order, preserve the sequence of intent, which is different from the intent itself: a user who asked for one thing, then narrowed it, then reversed one decision has a history the model must not flatten. Pending tasks and current work are the two tenses of the plan. The next step is asked for only if the user requested one, because a model that invents a next step under the heading of a summary has started working inside a request that told it to stop.

Plank's document lists what the reference adds around the summary and what plank took from it. A wrapper sentence saying that the session is being continued from a summary, a pointer to the transcript file on disk so the model can read what the summary left out, and an instruction not to acknowledge the summary on an automatic compaction, so the user does not receive a paragraph about what the model remembers. Plank kept the framing its model was trained on and marked the do-not-acknowledge line and the transcript pointer as free to add. Blanka's prompt has neither yet, and the pointer is the more interesting of the two: it turns the destructive rebuild into something

less than destructive, because the file the harness overwrote is the file the dump preserved, and a model that can read the dump can recover what the summary dropped. That is a design for a later chapter, once the dump and the compaction know about each other.

The summary is a user-role message when it lands, for the reason chapters 4, 5 and 6 each gave for their own injected text. Plank frames it as a tool result because that is the shape the reference's model was trained on and the parity fixtures pin it; blanka's model has no such training, and a user-role message with a fixed opening line is the form the rest of the book's scaffolding takes.

Summary plus tail

What replaces the conversation is not the summary alone. Plank's rebuild keeps a verbatim tail: the newest messages, walked back from the end until a token budget of an eighth of the window, capped at eight thousand tokens, is spent. The summary goes in front as a user-role message with a fixed opening line, then the tail exactly as it was.

The tail exists because a summary is lossy in a specific way. It preserves what the model judged durable and loses the texture of the last few exchanges: the exact wording of the user's most recent request, the exact error the last test run produced, the file the model was in the middle of editing. Those are what the next pass needs most, and they are also the cheapest to keep, because they are recent and bounded. An eighth of the window is plank's constant, and the reasoning behind it is that the rebuilt transcript lands, by construction, near a quarter of the window: the summary, a tail of at most an eighth, and a re-injection budget of at most another eighth for recently read files, which blanka does not build. A rebuild whose size is bounded by construction cannot leave the window still full, and that property turns out to be worth more than it looks.

```
pub fn compact<E: Engine + ?Sized>(
    engine: &mut E,
    t: &mut Transcript,
    keep_tail: usize,
) -> Result<CompactReport, AgentError> {
    let tokens_before = transcript_tokens(engine, t);
    let mut request: Vec<Message> = t.messages().to_vec();
    request.push(Message { role: Role::User, content: make_prompt() });
    let mut sink = Collect::default();
    let (raw, stats) = engine.generate(Prompt::Structured(&request), &mut sink,
    &InterruptFlag::new());
    let summary = extract_summary(&raw);
    if stats.interrupted || summary.is_empty() {
        return Ok(CompactReport { micro_rewrites: 0, summarized: false, tokens_before,
tokens_after: tokens_before });
    }
    let msgs = t.messages();
    let mut tail_start = msgs.len();
    let mut tail_tokens = 0;
    while tail_start > 0 {
```

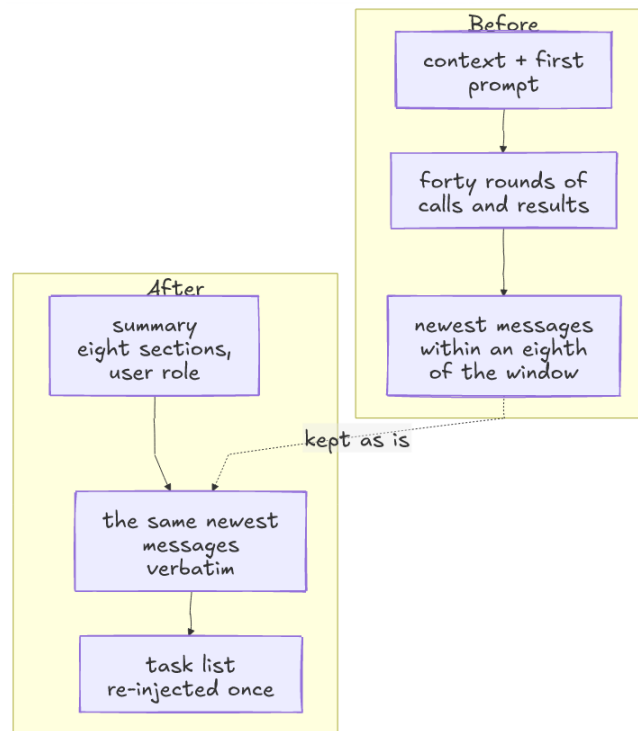

Chapter 7: Forgetting on purpose

```
tail_tokens += engine.count_tokens(&msgs[tail_start - 1].content);
if tail_tokens > keep_tail {
    break;
}
tail_start -= 1;
}
let head = Message { role: Role::User, content: format!("{SUMMARY_OPEN}
{summary}") };
t.replace_head(tail_start, vec![head]);
Ok(CompactReport { micro_rewrites: 0, summarized: true, tokens_before,
tokens_after: transcript_tokens(engine, t) })
}
```

The request is the live transcript with the prompt appended, and the shape is not accidental. Plank's compaction document points out that the reference hosted harness spends an entire subsystem making the summary request share the main conversation's cache, rebuilding the exact system prompt and context so the cache key matches, with a documented trap where one parameter silently invalidates it. A local engine that holds one session across turns gets the same benefit for free, provided the request is the live transcript's bytes followed by a suffix, so that only the suffix needs prefilling. The document's instruction is to keep it that way: any change to how the request is assembled must preserve the live transcript then an appended suffix. Blanka's request is exactly that, and chapter 8 collects the benefit.

The guard on an empty summary is plank's second recorded defect in this area, fixed after the fact. Its first version checked for an interrupt and then rebuilt unconditionally, so a generation that yielded no text, or only an analysis block, destroyed the transcript and replaced it with an empty summary and the tail. The tail survived, so it was not total loss, but it was, as the document says, unforced. The fix returns a status rather than an error, because the main loop propagates errors and tearing down a session over a recoverable failure would be worse than the failure. Blanka's report carries the same status, and the agent tells the user that compaction produced no summary and continues with the window as it is. The test scripts an engine that returns only an analysis block and asserts that the transcript is unchanged afterwards.

The transcript before and after the expensive step:



<https://enzolombardi.net>

Compaction as a rebuild. The head becomes one summary message, the tail is copied unchanged, and the task list is surfaced once after it.

Destroy, or hide

There is a design question inside the rebuild that plank answered differently from the reference, and its compaction document argues both sides.

The reference hosted harness deletes nothing. It appends a boundary marker to the transcript, and every consumer slices the message list from the last marker onward, so the pre-compaction history remains in the file, visible in a verbose view, and available to a resume that wants to relink it. Plank replaces the transcript. The messages before the tail are gone, and the session file is overwritten with the rebuilt list.

The document's table sets the costs against each other. Plank's transcript growth is bounded; the reference's is unbounded per session. Plank's post-compaction size is bounded structurally, by the arithmetic of the previous section; the reference's depends on what the slice yields. And from that second row follows the property that decides it for plank: because the rebuild lands near a quarter of the window by construction, plank cannot enter a compaction loop. The reference needs a circuit breaker, three consecutive automatic compactions that fail to make room and the harness stops trying, precisely because its projection offers no such guarantee. Plank does not need the breaker and, the document says, should not grow one.

What plank pays is auditability and, in its own case, the survival of the transcript branches its fork command creates, which index into the list being replaced and cannot survive it. The document calls that the sharper cost for a tool that ships a session tree, and names adopting the boundary model as the one place where a large change would

Chapter 7: Forgetting on purpose

be a real improvement rather than a stylistic one, and then puts it out of scope, because it touches the session format, resume, the branch tree and the accounting all at once. Blanka has no branches, so it pays only the auditability, and it pays that knowingly: the chapter 5 dump taken before a compaction is the record of what was destroyed, and a user who wants the history has the file the harness wrote before it forgot.

The transcript gains two methods for the rebuild, and their comments say which caller is allowed to use them.

```
impl Transcript {
    /// Replace one tool result's body in place and return the old body. The
    /// cheap compaction pass is the only caller: everything else appends.
    pub fn rewrite_tool_result(&mut self, i: usize, content: &str) -> String {
        debug_assert_eq!(self.messages[i].role, Role::Tool);
        std::mem::replace(&mut self.messages[i].content, content.to_string())
    }

    /// Replace everything before `tail_start` with `head`. The tail is kept
    /// verbatim; the head is destroyed, not hidden.
    pub fn replace_head(&mut self, tail_start: usize, head: Vec<Message>) {
        let tail = self.messages.split_off(tail_start.min(self.messages.len()));
        self.messages = head;
        self.messages.extend(tail);
    }
}
```

Until this chapter the transcript was append-only, and after it the transcript is append-only except at these two seams. The distinction matters to chapters 8 and 9, because an append-only transcript is one whose cached prefix is always valid, and each of these two methods invalidates the cache from the point it touches. Plank's comments on both say so, and its rebuild discards the whole cache ladder at once, for a reason the chapter on the ladder explains: every rung snapshots a prefix of the transcript just replaced, so all of them died with it.

Compaction and the cache

Both steps rewrite the transcript, and chapter 8 will make the cost of a rewrite exact. It is worth stating now what that cost is, because it decides which of the two steps to run and explains why plank's cheap step grew a three-part gate that this chapter leaves out.

A local engine remembers the tokens it has processed, in a cache keyed on the prefix of the prompt. An append-only transcript has a prefix that only grows, so each pass processes the new suffix and nothing else. A rewrite at position *i*, whether a stub replacing a body or a summary replacing the head, changes every token from *i* onward, and the engine must process them again. For the cheap step, that means a rewrite of an early result is not free at all: the bytes reclaimed are cheap and the tokens re-processed are expensive, and plank's first gate could not see the second. The measured session that paid fourteen thousand tokens to reclaim twelve kilobytes is the number to remember.

Chapter 7: Forgetting on purpose

For the expensive step the cost is the same in kind and larger in degree, and it is paid once, after which the transcript is small and append-only again.

Two consequences shape the code in this chapter. The task list is injected once, after the rebuild, and never mid-transcript, because a mid-transcript injection is a rewrite with the same cost. And the summary request is the live transcript with a suffix appended, so that the request itself costs only the suffix. Plank's document treats that second property as a rule rather than an observation, because the reference hosted harness spends a subsystem achieving what a local engine gets by keeping the prefix intact.

The gate that blanka leaves out weighs these costs against the bytes reclaimed, and its history is told in the findings file as three versions, each seeing something the previous one could not, and then a fourth whose accepting branch has never fired outside a unit test. It belongs to chapter 9, because it needs the snapshot ladder to have anything to restore. What this chapter takes from it is the floor in front of it, which is not a cost argument but an information one, and which would have prevented the two-hour session on its own.

When to check

Plank's port had layered compaction on its second day and got the cadence wrong for weeks, and the finding is one line: the C checks every tool round, not only every turn.

The port checked at the start of each user turn, which is where a harness that thinks in turns naturally puts the check. The C reference checks twice, once before the user turn and once at the top of every continuation round, and the difference is the whole of a long turn. A read-edit-test loop of forty rounds appends forty tool results inside one turn, and a harness that checks only at the turn's start sends the forty-first generation against a window it has not measured since the first. The end-of-turn opportunistic pass did not cover it either, because that pass runs only on the path where the turn is already over. Both of plank's loops now check on each round, and the planning document for the turn budget, written later, proposes lifting the whole check to the top of every iteration on the same grounds.

Blanka's loop was built with the check in the right place, because plank's finding came first.

```
for round in 0..crate::compact::MAX_TOOL_ROUNDS {
    // Every round, not only the first: a long read-edit-test loop
    // fills the window from inside one turn.
    let c = self.maybe_compact(sink)?;
    report.micro_rewrites += c.micro_rewrites;
    report.compactions += usize::from(c.summarized);
    let last = round + 1 == crate::compact::MAX_TOOL_ROUNDS;
    if last {
        self.transcript.push(Role::User, crate::compact::FINAL_ROUND_REMINDER);
        sink.status("[tool-round cap reached: asking for a final answer]");
        report.capped = true;
    }
    // ... generate, record, parse, dispatch as before ...
}
```

Chapter 7: Forgetting on purpose

```
    if last {
        break;
    }
    // ...
}
```

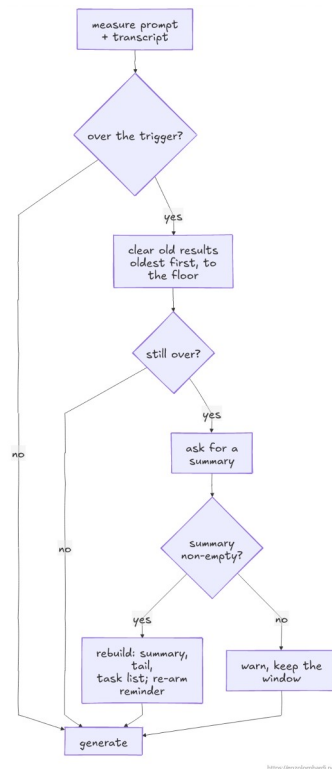
The check itself is one method, and its shape is the escalation the chapter has described.

```
pub fn maybe_compact<S: RenderSink>(&mut self, sink: &mut S) ->
Result<CompactReport, AgentError> {
    let ctx = self.engine.ctx_size();
    let mut report = CompactReport::default();
    let used = self.used_tokens();
    if !should_compact(used, ctx) {
        return Ok(report);
    }
    report.micro_rewrites = microcompact(&mut self.transcript, used, ctx);
    if report.micro_rewrites > 0 {
        sink.status(&format!("{}", old tool results cleared]", report.micro_rewrites));
    }
    if !should_compact(self.used_tokens(), ctx) {
        return Ok(report);
    }
    sink.status("[compacting: summarizing the conversation so far]");
    let full = compact(&mut self.engine, &mut self.transcript, tail_budget(ctx));
    if full.summarized {
        if let Some(block) = self.ctx.tasks.inject_block() {
            self.transcript.push(Role::User, block);
        }
        self.reminded = false;
        sink.status(&format!("{}", compacted: {} -> {} tokens]", full.tokens_before,
full.tokens_after));
    } else {
        sink.error("[compaction produced no summary; continuing with the window as it
is]");
    }
    // ... copy the numbers into the report ...
    Ok(report)
}
```

Measure. If under the trigger, do nothing, and the cost of the check was one token count. If over, run the cheap pass and measure again. If still over, run the expensive one, resurface the task list, and re-arm chapter 6's reminder, because the summary was written under the rules and the rules are now far away again. Plank's order is the same, and its document notes a detail blanka keeps: when the cheap pass was insufficient, the mutation it made is not undone, so the summary is written over the already-shrunk transcript. Every status line goes to the user through the channel chapter 4 built, because a transcript that lost forty messages and said nothing would be a harness lying by omission.

Chapter 7: Forgetting on purpose

The decision made at the top of every round:



The escalation at each tool round. The common case is the first branch; the expensive one runs only when the cheap one could not make room.

Where the trigger sits

Two constants decide the trigger, and both are plank's. Full compaction is due when the window is eighty-five percent used, or when fewer than eight thousand tokens are free, whichever comes first, with the free-token rule capped at a quarter of the window so that a small context still compacts rather than failing.

```
pub fn should_compact(used: usize, ctx: usize) -> bool {  
  if ctx == 0 || used == 0 {  
    return false;  
  }  
  if used * 100 >= ctx * COMPACT_SOFT_PERCENT {  
    return true;  
  }  
  let free_threshold = COMPACT_MIN_FREE_TOKENS.min(ctx / 4);  
  ctx.saturating_sub(used) <= free_threshold  
}
```

Plank's document defends the percentage against the reference's absolute buffer in a paragraph that is easy to skip and should not be. The reference triggers at a fixed number of tokens below its window, thirteen thousand for automatic compaction, because it runs one known model with one known window. Plank runs whatever local model the user

has, across a spread from eight thousand to a million tokens, and an absolute buffer tuned for one window is wrong for the rest. A percentage scales; the free-token floor catches the large windows where a percentage leaves too little absolute room; the quarter cap catches the tiny ones where the floor would trigger constantly. The test pins each of the three cases, including the tiny-window one that would otherwise be the bug found in the field.

There is a consequence for the reference model this book uses, and plank's findings record it for its own small windows. Below thirty-two thousand tokens the free-token rule puts the trigger at exactly three quarters, the same place as the pressure floor, so there is no room in which the cheap pass runs on its own: whenever it is allowed to run, full compaction is already due, and the cheap pass is the first step of the escalation rather than a separate event. On a larger window the two separate, and the cheap pass earns its place by running alone many times before the expensive one is needed. The reader with a thirty-two thousand token model will see the two steps together, and the arithmetic says why.

A turn that will not end

Compaction bounds the window. Nothing so far bounds the turn. A model that keeps calling tools, each result prompting another call, runs the loop of chapter 2 until something external stops it, and the something has been the user's interrupt from chapter 4. That is not a defence. A headless run has no user watching, and an interactive one should not need a person to notice that the model has been reading the same directory for twenty minutes.

Plank's answer was designed before it was built. Its sub-agent loops, which chapter 12 covers, had a round cap of forty from the start, with a reminder pushed on the last permitted round telling the model that no more tools are available and to write its report as plain text now. The main loop had no cap at all, and a planning document proposes copying the sub-agent pattern into it with a budget of two hundred rounds, on the grounds that the reference hosted harness's forked agents use that number and that a turn reaching it is almost certainly stuck. The document is careful about one thing: the reminder text is reused verbatim rather than paraphrased, so that two nearly identical strings cannot drift.

Blanka's cap is sixty-four rounds, smaller than plank's proposed two hundred because the reference model's window is smaller and sixty-four rounds of tool results fill it several times over. The mechanism is the one the loop excerpt above shows. The loop is a bounded iteration rather than an unbounded one. On the last permitted round the harness pushes a user-role message saying that this is the last tool round and to write the final answer as plain text, generates once more, records whatever comes back, and breaks without parsing it for calls. The report says the turn was capped, and the status channel says so to the user.

```
pub const MAX_TOOL_ROUNDS: usize = 64;
pub const FINAL_ROUND_REMINDER: &str =
    "This is the last tool round available in this turn. Do not call any more tools: write your
    final answer as plain text now.";
```


Chapter 7: Forgetting on purpose

The test scripts an engine that emits a tool call on every generation, more of them than the cap, and asserts that the loop terminates with the expected number of passes and rounds, that the reminder is the second-to-last message and the model's final reply the last, and that the status channel carried the word cap. The final reply in the test is another tool call, because the engine is scripted to produce one, and the assertion that nothing was dispatched after it is the point: on the last round the reply is the answer whatever it contains, and a model that ignores the reminder gets its call recorded and not run.

The cap is the last defence, not the first. Chapter 10 builds the loop guards that catch a stuck model long before sixty-four rounds, by noticing that the same call with the same arguments is producing the same result. The cap exists for the case the guards do not recognise, as the two-hour session was not recognised by a guard whose window was too short for the cycle. A harness needs both, and it needs the cap because it is the one that cannot be fooled.

What must never be forgotten

The summary is written by the model, and a model summarising forty rounds of work will, sometimes, leave out the thing that mattered. Plank's findings and the reference's design agree that some state is too important to trust to a summary, and the answer both reached is to keep it outside the transcript and put it back after every rebuild.

Plank's first version of this was a task list, added three days into the port, and its module comment defines it in a sentence that is worth quoting for the distinction it draws: a task list is not a to-do widget; it is the structured part of the model's plan, promoted out of the expendable transcript into session state so it costs a fixed, small number of tokens per turn and is never summarised away. The list lives beside the transcript on the session, so it serialises with it and rides resume and save for free. The model manages it through one tool with an operation argument, because every tool costs a parity fixture entry and one tool with three operations is the smallest table that works. Mutating operations return the fresh list in their result, so the model reads state from results and never needs it injected mid-transcript, which would break the append-only property the cache depends on. And after a compaction rebuild the list is surfaced once, as a user-role message, at a moment when the rebuild has already invalidated the cache and the injection costs nothing extra.

A month later plank added a second piece of durable state above the list, the objective of an autonomous goal loop, and the findings entry for it restates chapter 5's rule in this new setting: the field is the durable fact; the transcript entry is the record of what was shown. The field must never become a second unlogged source of model-visible text, so anything it renders is also pushed into the transcript. Blanka builds the list and leaves the goal loop to plank.

```
pub const TASK_LIST_OPEN: &str = "# Task list\n";

pub struct TaskItem { pub text: String, pub done: bool }

pub struct TaskList { items: Vec<TaskItem> }
```

Chapter 7: Forgetting on purpose

```
impl TaskList {
    pub fn add(&mut self, text: impl Into<String>) -> usize;
    pub fn complete(&mut self, n: usize) -> Result<(), String>;
    pub fn render(&self) -> String;
    pub fn inject_block(&self) -> Option<String>;
}
```

Tasks are numbered from one and never removed, so a number the model used ten turns ago still means the same task after a compaction that forgot the turn it was used in. Completion is by number, and a bad number returns an error naming the list's length, for the model to read and correct. The injected block has a fixed opening line, and that line is what the clear set of the cheap pass looks for: results after the last injection belong to the current task and are not cleared, because a task list that survived compaction while the results of working on it were stubbed would be a plan without its evidence.

The list lives on the tool context, which is what the task tool receives and what the agent owns, and the choice puts the durable state where both the model's tool and the harness's rebuild can reach it without a third structure. Plank keeps it on the session for the same reason with a different shape.

```
impl Tool for TaskTool {
    fn call(&self, args: &Value, ctx: &mut ToolContext) -> Result<String, ToolError> {
        let op = args.get("op").and_then(Value::as_str).unwrap_or("").trim();
        match op {
            "add" => {
                let text = args.get("text").and_then(Value::as_str).map(str::trim).unwrap_or("");
                if text.is_empty() {
                    return Err(ToolError("add needs text".into()));
                }
                let n = ctx.tasks.add(text);
                Ok(format!("added task {n}\n{}", ctx.tasks.render()))
            }
            "done" => {
                let n = args.get("n").and_then(Value::as_u64).ok_or_else(|| ToolError("done
needs n".into()))? as usize;
                ctx.tasks.complete(n).map_err(ToolError)?;
                Ok(format!("completed task {n}\n{}", ctx.tasks.render()))
            }
            "list" => Ok(ctx.tasks.render()),
            other => Err(ToolError(format!("unknown op {other:?}; use add, done or list"))),
        }
    }
}
```

The tool's description tells the model when to use it: record the plan here before long work, because the list survives compaction. That sentence is the whole reason the model would bother, and without it the tool is a curiosity. Chapter 3's finding that descriptions steer behaviour applies with more force here than to any of the six verbs, because there

Chapter 7: Forgetting on purpose

is no shell substitute for a task list and a model that does not know why the tool exists will not reach for it.

The task tool joins the default set, and that changes the static prefix of chapter 6, which changes the pinned fixture, which fails the fixture test until the version constant moves. It moved to two for this chapter. That is the version mechanism doing its job on the first occasion it had.

The agent test that ties it together

One test exercises the whole chapter in a single turn, and how it has to be set up says something about testing against windows.

A first attempt used a window of a few hundred tokens, on the theory that a tiny window makes compaction trigger quickly. It made compaction trigger before the turn began, because chapter 6's static prefix alone is thirteen hundred tokens and the check measures prompt plus transcript. A window smaller than the prompt is a window in which every round compacts an empty conversation, and the test's status channel showed exactly that: a compaction of three tokens into thirty, before any tool had run. The lesson is that a window in a test has to be a window a real prompt fits in, with the remaining room sized so that the test's tool results fill it.

The test that works uses a window of twenty-two hundred tokens. The prompt takes thirteen hundred; the trigger, a quarter free, sits at sixteen hundred and fifty; a scripted tool that returns two kilobytes of body crosses it in one call. The cheap pass cannot help, because that one result is the current batch, so the summary runs, scripted as a one-line summary in tags. The test asserts that exactly one compaction happened and no rewrite, that the summary message precedes the task list injection, that the task added before the turn is in the injected block, that the final answer is the last message, and that the reminder flag was re-armed.

```
#[test]
fn a_window_that_fills_inside_a_turn_is_compacted_and_the_task_list_survives() {
    let ctx = scratch("fill");
    let engine = EchoEngine::scripted(vec![
        BLOB_CALL.to_string(),
        "<summary>1. Was fetching blobs</summary>".to_string(),
        "done".to_string(),
    ])
    .with_ctx(2200);
    let mut a = Agent::new(engine, ToolTable::default_set().with(Box::new(Blob(2000))),
    ctx);
    a.ctx.tasks.add("find the files");
    let mut sink = Channels::default();
    let report = a.run_turn("look around", &mut sink).unwrap();
    assert_eq!(report.compactions, 1, "{:?}", sink.status);
    assert_eq!(report.micro_rewrites, 0);
    let msgs = a.transcript.messages();
    let summary_at = msgs.iter().position(|m|
    m.content.starts_with(SUMMARY_OPEN)).unwrap();
```

Chapter 7: Forgetting on purpose

```
let tasks_at = msgs.iter().position(|m|
m.content.starts_with(TASK_LIST_OPEN)).unwrap();
assert!(summary_at < tasks_at, "summary, tail, then the task list");
assert!(msgs[tasks_at].content.contains("find the files"));
assert_eq!(msgs.last().unwrap().content, "done");
assert!(!a.reminded, "a compaction re-arms the reminder");
}
```

The same arithmetic reshaped chapter 6's reminder test, which had used a sixty-four token window for the same wrong reason. It now uses a forty thousand token window, where the reminder floor is thirty thousand and the compaction trigger is thirty-one thousand eight hundred, and thirty four-kilobyte results cross the first without crossing the second. That the two tests needed the same correction is the finding plank's small-window arithmetic predicted: below thirty-two thousand tokens the reminder and the trigger coincide, and a test that wants to see one without the other has to use a window above it.

A second setup lesson was cheaper but cost more time. The first version of the cap test ran a glob over the temporary directory sixty-four times, and the temporary directory on a developer's machine is large, so the test ran for minutes and looked like a hang. Every test in the chapter now uses an empty scratch directory. A tool that walks the filesystem is not free even when the pattern matches nothing, and a test that calls one in a loop should give it nothing to walk.

What the user sees

Every step in the chapter announces itself through the status channel, and the plain front end gains two commands. A compact command asks for a summary now, whatever the pressure, and prints the token counts before and after and the number of messages left. A tasks command prints the list. The compact command exists because plank's does, and plank's document records why a manual form matters beyond convenience: the reference's accepts instructions, and plank threaded an argument into the prompt so a user can say what the summary should concentrate on. Blanka's takes no argument; the prompt is the fixed one.

A session compacted in this way resumes compacted. The summary is a user-role message in the transcript, the file holds it, and chapter 5's replay shows it as scaffolding rather than as something the user typed, because the summary's fixed opening line is recognised the way the context blocks are. The task list, being outside the transcript, is not in the file yet; it lives on the tool context and dies with the process, which is a gap this chapter leaves and chapter 9 closes when the store learns to keep a session's durable state beside its transcript. Plank serialises the list with the session, and the reason blanka does not yet is that the chapter is about forgetting, and the shape of what survives across processes is a separate decision.

Plank later made the cheap step switchable for the session, and the way it did so says something about which controls deserve to be visible. A setting turns micro-compaction off entirely, a slash command flips it for the running session without touching the file on disk, because it is a switch thrown to watch one turn rather than a preference, and every

check re-reads it, so the change lands on the turn already running. For a while the footer carried the state permanently, a wastebasket beside the loop-guard segment, drawn in both states because a control that disappears when it is off cannot be clicked to turn it back on. A single click reported the state and named the gesture; only a double click toggled. The reason is in the commit message: the step rewrites old tool results in place and costs a cache restore when it does, so a stray press on the status bar must not rewrite the transcript. A later release removed both icons when the footer was cleared of everything a user does not need at a glance, so that it would hold still while a turn runs, and the slash command remains the way to read and flip the switch. The double click is still the lesson to keep for any control that rewrites state behind the user's back.

The run that accompanies the chapter's code is a turn against the echo engine, a manual compaction whose summary is the echo of the request, and a context report showing the rebuilt conversation. It proves the plumbing and nothing about summary quality, which only a real model can show. Against the reference model, a manual compaction after a few turns of real work produces the eight sections, and reading them is the fastest way to learn what the model considered durable and what it did not, which is the information a harness author needs to tune the prompt.

Remembering on purpose

Everything so far forgets within a session. The session itself is forgotten too, in a different sense: a new session starts from the instruction file, the date and the working tree, and whatever the user corrected an hour ago in the previous one is gone. Plank's answer to that is the mirror of this chapter, a small store of facts chosen to outlive every session, and it is worth describing here because it runs into the same two costs compaction does, the model's attention and the cache, and settles them the opposite way.

The store is two flat markdown files, one in the user's plank directory that follows them into every project and one in the project's own directory. Each entry is a dated bullet carrying one of four type tags: who the user is, feedback on how to work, project goals and constraints that the code does not show, and pointers to external references. The taxonomy is closed on purpose, with no catch-all fifth type, because what keeps such a file useful is what it refuses to hold. Plank's document is specific: nothing goes in that the model could reconstruct more accurately by looking, which excludes code patterns, file paths, git history and anything the instruction file already says. A memory that accumulates re-derivable facts rots the way any cache rots when nothing invalidates it.

The placement is the decision that shapes everything else. The rendered memory sits beside the instruction file in the prompt tier that changes rarely, the one chapter 8 will cache to disk and reuse across sessions. That makes memory nearly free to carry, and it makes it impossible to change mid-session without invalidating everything cached after it. So every write lands on disk and nowhere else. The model's remember tool, the user's command and the automatic pass described below all write the file, and the new text joins the model's context at the next session start. The tool's own reply says so, because a model that has saved something a moment ago and sees no effect needs to be told that this is expected.

An entry and its counters

Beside each memory file sits a sidecar holding, per entry, how many times the entry was judged to have borne on the work, when that last happened, and a pin flag. The sidecar is advisory in exactly the sense chapter 5 gave the word: a missing, corrupt or malformed sidecar loads as an empty one, every counter zero and nothing pinned, and memory loads and renders as before. Plank tests that property, because a bookkeeping file easily becomes load-bearing without anyone deciding it should.

An entry's identity is the first twelve hex characters of a hash of its text alone, not its date and not its tag, so re-tagging or re-dating a line keeps its counters. The consequence is the finding worth carrying away. A rewrite of the entry's text produces a new identity, and the sidecar row under the old one no longer matches anything. Plank's update path therefore moves the accumulated row onto the new key in the same step that rewrites the line, and its findings file describes what happens without that step: nothing looks broken, the text is still there and still renders, until someone asks why a fact reworded six times over a month keeps being evicted as if it were new.

Eviction is what the counters are for. Rendering is budgeted per type, a few kilobytes each, so a runaway block of project notes cannot crowd out the block about the user the way one shared budget would allow. Within a type, entries rank pinned first, then by uses, then by most recent use, then by date and position, and are kept until the budget is spent; whatever does not fit is counted in a line saying how many were omitted, never dropped in silence.

Notes taken while nobody is waiting

The remember tool relies on the model thinking to call it, and a model in the middle of a task rarely does. Plank added a passive half, an extraction pass that reads the recent transcript and proposes changes, and its design is a sequence of refusals to spend the user's time.

It runs in two steps. At the end of a turn that produced a final answer with no tool calls, the span of transcript since the last pass is snapshotted into a job: the complete extraction prompt, built from the transcript as it stands, pushed onto a queue. Nothing is generated then, so the prompt comes back the moment the answer is done. The reading happens at the next idle moment, one job at a time, as a sidechain of the kind chapter 9 builds: a snapshot of the session's engine state, a prefill of the pass prompt on top of the live context, one generation with no tool dispatch at all, and a restore. A prompt the user types while the pass runs interrupts it, the job goes back to the front of the queue, and the typed line starts its turn at once. The idle slot is shared with other background work that chapter 11 describes.

A queue that retries needs a rule for when to stop, and plank gives each way out of the queue its own reason. A job leaves when its verdicts are applied. It leaves as unusable when the reply is not a JSON array, because that is a property of the model on this prompt and will not improve on a retry. It leaves with a one-time notice when its prompt no longer fits the remaining context. And it leaves after a fixed number of engine errors, which is what keeps a broken engine from pinning the idle loop on a single span forever. The last rule needs a distinction to be fair: a pass the user interrupted is not an engine

Chapter 7: Forgetting on purpose

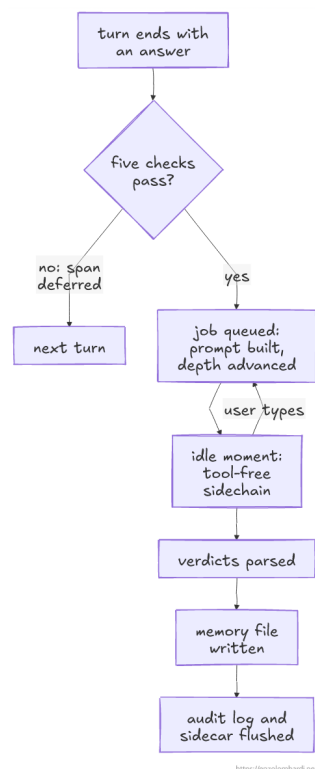
fault, so it goes back to the queue without counting toward the limit, and a user who types briskly never exhausts a job's attempts by typing.

The reply is a JSON array of verdicts, each one of four kinds: add a new entry, update an existing one, delete one as redundant or wrong, or mark one as used, crediting it with having mattered to the work in the excerpt. A verdict naming an identity that no longer exists is discarded, because a person may have edited the file between the pass reading it and the write landing, and the file always wins. The excerpt is capped at thirty-two kilobytes, with tool-result bodies replaced by a one-line placeholder.

The pass reads only the transcript above a recorded depth, the length its last completed run covered, and the invariant stated in plank's documentation is the one to copy: getting that backwards degrades silently into reprocessing everything rather than failing loudly. There is no error state for having read too much; every pass after the mistake re-derives verdicts for text it has already seen, and nothing in the logs points at the cause. So the depth never moves backwards, even when a stale value is offered, and every transcript rewrite re-anchors it: after a compaction, after a rollback, and on clearing or switching sessions, so that a restored session is not shipped wholesale to the model on its first idle turn.

Five checks decide whether a turn queues a job, and their order is the part of the design that is easy to get wrong. The pass must be enabled and the model must not have called remember or forget itself during the turn, because the model's own judgement wins over a second reading of the same turn. The transcript must have grown past the recorded depth. The turn must have lasted longer than a configured minimum, since a four-second exchange rarely holds anything worth a prefill and a generation. A pass must not already be running. And a throttle can thin the pass to every few eligible turns. The duration floor sits above the throttle, so a short turn neither runs the pass nor counts against the throttle, and it does not advance the depth either: its span is deferred rather than discarded, and the next turn that clears the floor reads the short ones too. The order is what makes the word eligible mean something. A turn skipped for suppression, for having nothing new, for brevity or for arriving mid-pass never uses up a throttle slot that a real turn needed.

The life of one span, from the turn that produced it to the file:



One span's path through the extraction pass. An interrupted reading puts the job back at the head of the queue; the log and the counters follow the write, never lead it.

The write lands first

A batch of verdicts touches three things: the memory file, the audit log that records every automatic change with the entry's full text and a fixed label naming which mechanism made it, and the sidecar's counters. Plank's first version wrote the log line and bumped the counters as each verdict was applied, and wrote the file at the end. A failed write, for permissions or a full disk or a directory that vanished, then left three sources of truth disagreeing: the log said an entry was added or deleted, the sidecar held counters for an identity that was never written, and the file had neither. Nothing downstream can reconcile that after the fact, because the log's only purpose is to be the record of what happened.

The fix is general and cheap. Stage the log lines and the sidecar mutations in memory while walking the batch, and flush both only after the file write has succeeded, so that the trail and the cache are only ever behind the write and never ahead of it. A scope whose write failed is left exactly as it was, and the loop moves to the next scope rather than abandoning the batch.

The log earns its place in a second way. The model's forget is a real deletion, through the same atomic, audited path as the user's command, not a hidden retraction flag; an earlier design had one and dropped it, because what the user reads in their own file should be what the model will see. What makes a wrong deletion recoverable is the log line, which carries the full text under a label distinguishing the model's forget from the user's. A command prints the tail of the log. It says what changed, by which hand, and what the

text was, which is enough to restore a wrong deletion, though not why the model made it.

A question answered without generating

Most turns hold nothing worth remembering, and a pass that reads one to conclude so still costs a snapshot, a prefill and a generation. Plank added a gate in front of the queue that asks, before any job is built, whether the span is worth extracting at all, and answers the question without generating a single token.

The mechanism is what plank calls a System-1 decision. The engine prefills the rendered excerpt and a short multiple-choice question ending in an answer prompt, and instead of sampling it reads the log-probabilities of the answer letters at the next position. A softmax over those few values gives a typed answer with a probability attached, and the answer abstains when the top letter is below a confidence floor or when too little of the distribution's mass sits on the letters at all, which means the model was trying to say something other than a letter. The arithmetic is pure and tested against fixed numbers, and the echo engine scripts verdicts for every consumer. The decision runs on a dedicated small session created for the purpose, never on the live one, so it never disturbs the cached prefix this chapter and the next two work to protect.

The gate is built to fail open. An engine that cannot decide, a decision that returns an error, and an abstention all read as worth extracting; only a confident no suppresses the pass. The gate can remove a pass it is sure found nothing, and it must never be the reason a memory is lost.

The first run against a real model found a defect no unit test could have. The question ended in a prompt for the answer followed by a space, and the code scored the bare letter token. On plank's tokenizer a letter with a leading space and the bare letter are different tokens, and after such a prompt the model puts nearly all its mass on the spaced form: measured, the spaced letter at a probability of about 0.4 and the bare one near one in ten million. Every verdict therefore abstained for want of letter mass, and because an abstention runs the pass, the gate was a perfect no-op. The capability reported itself healthy, every test was green, and nothing was ever decided. The fix, in commit 83a6bfe, ends the question with no trailing space and scores the letter with its space; afterwards the mass on the letters was 0.94 and the answer to a question with an explicit fact came back yes at 0.98. The code now carries a comment calling this the single easiest thing to get wrong, and the capability refuses to report itself available unless every scored variant is exactly one token on the loaded model.

A second defect was of the same silent kind. A failed prefill could leave the cached decision session invalid, and keeping it turned one bad call into a permanently broken capability, because nothing upstream tells an engine that cannot decide from one call that went wrong. Commit 7881850 discards the session on failure, and its tests now assert the answers rather than a finite probability, which a read at the wrong position would also produce.

Letter mass has a blind spot that no amount of it can cover. It says the model answered with a letter, not why it chose that letter, and models carry a prior toward option labels as such: permute the options of a multiple-choice question and the answer moves with

the letter. The gate always shows yes as the first option, so a family that leans toward it reads more yeses than the turns deserve while its letter mass looks healthy. Commit 96ecfd6 added the measurement: a command that renders turn-sized spans from the family's saved sessions exactly as the gate sees them and asks the question twice, the second time with yes and no swapped, so that half the difference in the probability of yes is the bias on that span. On DeepSeek V4 Flash, over thirty spans, the bias was 0.089 toward the first letter with a standard error of 0.012, while letter mass ranged from 0.60 to 0.98 and never looked wrong. Asking twice on every turn would double the gate's cost at turn exit, since a decision session on that family cannot be rewound, so the correction is measured offline and folded into the threshold as a per-family offset, copied into the settings by hand. The same run held the larger finding. With the bias removed, the mean probability of yes was about 0.15, so the gate declares nearly every turn not worth remembering, and without ground truth for which spans held something durable, nothing yet says whether that is accurate or a memory quietly starved.

The gate ships off, and the reason is its position. The decision runs synchronously at turn exit, the one place whose design property is that nothing is generated, and it prefills up to thirty-two kilobytes of excerpt on a session whose cached prefix rarely matches a rolling tail. That is far cheaper than the generation it can avoid, and it still lands between the answer and the prompt.

Where blanka stops short of plank

Plank's compaction module is a thousand lines and blanka's is two hundred, and the difference is mostly the cache.

Plank's cheap pass has a three-part gate in front of the pressure floor, weighing bytes reclaimed against tokens the rewrite forces back through prefill, with a floor that relaxes as the window fills, and it reports the earliest index it would rewrite so the agent can restore a cache snapshot from just below it. The findings file is candid that the accepting branch of that gate has never fired in a live session, that one of its refinements cost a minute and a half per session and delivered no realised benefit, and that it is kept as the precondition for a later step. Chapter 9 builds the snapshot ladder that gives the gate something to restore, and revisits the arithmetic then. Plank's full compaction re-injects up to five recently read files within an eighth of the window after the tail, because the reference does and because a model that has just lost its context usually wants the file it was editing back; blanka does not, and a reader who wants it has plank's budgeted builder as a model. Plank's summary prompt accepts user instructions and pins its text in a fixture. Plank fires hooks before and after a compaction, and its document records the defect of firing them on one of two front ends and not the other, which chapter 13 takes as the argument for one orchestrator behind a trait. And plank's goal loop keeps an objective above the task list and adjudicates it every iteration, which is a feature the book leaves.

Plank's persistent memory is the largest thing in this chapter that blanka does not build: the two files and their closed taxonomy, the advisory sidecar with its carried counters, per-type budgets, the idle extraction pass with its depth invariant and five ordered checks, the write-first audit log, and the System-1 gate with its dedicated decision session. Blanka's durable state stops at the task list, which lives and dies with one session.

Chapter 7: Forgetting on purpose

A reader who adds memory should start from the placement argument, because the rest follows from the rule that a write lands on disk and joins the context only at the next start.

What blanka has is the two steps and the escalation between them, the pressure floor and the budgeted oldest-first pass, the summary contract and the empty-summary guard, a verbatim tail and a destructive rebuild with the argument for it, the check at every tool round, the round cap with its reminder, and a task list the model keeps and the rebuild restores.

Chosen amnesia

A window fills, and the harness decides what the model will not know. The cheap decision rewrites old tool results in place, oldest first, until the window is back under three quarters, and never runs below that floor, because a session that cleared what the model had just read re-read it for two hours. Small results, the current batch and the current task's results are never candidates.

The expensive decision asks the model for a summary in eight sections with a no-tools instruction last, keeps the newest eighth of the window verbatim behind it, and rebuilds rather than hides, so that the result is bounded by construction and no compaction loop is possible. An empty summary changes nothing. The check runs at the top of every tool round, not only every turn, because a turn fills a window from inside.

Sixty-four rounds is the cap, with a reminder on the last one and whatever comes back recorded as the answer. The task list is the state too important for a summary: kept beside the transcript, managed through one tool whose results carry the list, and surfaced once after every rebuild, when the cache is already gone and the injection is free.

What outlives the session is chosen as deliberately as what is dropped within it. Plank's memory writes to disk and joins the context only at the next start, so a note never breaks the cached prefix; its pass reads only what lies above a recorded depth, at an idle moment, and its log follows the write rather than leading it. The gate in front of the pass answers from log-probabilities without generating, fails open on every doubt, and taught, on its first real run, that a green test suite can sit on top of a capability that never decides anything.

CHAPTER 8: WHERE THE TIME GOES

Type a request into a coding agent running on a laptop and watch what happens. For a while, nothing. Then the first word of the reply appears, and the rest follows at a pace the eye can keep up with, a few words a second, until the reply is done. The second part is what the agent looks like when it is working. The first part is what it looks like when it is stuck, and it is not stuck. It is reading.

A language model turn has two phases with different costs. Prefill reads the prompt and builds, for every token in it, the internal state that later tokens will attend over. Decode then produces the reply one token at a time, each step attending over everything already in that state. Decode is what you watch happen. Prefill is what you wait through before anything appears, and on a local machine it is the larger of the two by a margin that decides whether an agent is usable.

Plank measured the margin, on a twenty-seven billion parameter model quantised to four bits on a Metal backend, with the server's prompt cache off so that every turn paid full price. A bare prompt of sixty-one tokens took 1.4 seconds to the first token. The same request with sixty-six tool schemas in front of it, 13,600 tokens, took 80.8 seconds. With a hundred and forty schemas, 24,700 tokens, it took 156.4 seconds. Decode speed across the three was flat within noise, between seven and eight tokens a second. The cost was time to first token, and only that. The finding puts the arithmetic in one sentence: what users reported as the agent being much slower than the bare model was a prefill rate of about a hundred and sixty tokens a second multiplied by a prompt two orders of magnitude larger, which reads as a catastrophic speed only when the measurement divides tokens by wall clock.

A second measurement, made more carefully, separated the two effects. With the cache on and each prefix warmed once, decode was sampled four times per prompt with the same question in every body. Fifty-nine tokens of prompt gave 6.55 tokens a second; 2,863 tokens gave 5.37, eighteen percent slower; 10,679 tokens gave 5.14, twenty-two percent slower. So a long prompt does slow every token, and the intuition that it does is correct. Cold time to first token over the same three prompts was 0.5, 16.1 and 54.8 seconds. Warm, all three were between 0.2 and 0.3 seconds. Both effects are real, they compound, and time to first token dominates a cold turn by an order of magnitude.

Getting that number took three attempts, and plank's findings file keeps the two failures because they are the lesson. Short natural generations of fifty tokens are pure noise: one body measured twenty tokens a second and eight tokens a second minutes apart. The obvious fix, forcing equal-length generations by ignoring the end-of-sequence token, inverted the result on a server with speculative decoding, because past its natural end the model emits degenerate text, the draft model stops predicting it, acceptance collapses, and the shortest prompt looks slowest because it reaches that point first. Warm-cache repeat sampling is the method that worked, and even then the absolute values tracked machine load, the same prompts giving twenty-two to twenty-six tokens a second on an idle box and five to seven under a load average of seven. Compare figures measured back to back, and quote ratios.

Chapter 6 built a static prefix of about thirteen hundred tokens and chapter 7 a transcript that grows by kilobytes per tool round. Every pass so far has sent all of it, and against a hosted engine that is what a pass costs and there is nothing the harness can do about it beyond what the provider's own caching does. Against a local engine there is a great deal the harness can do, because the state prefill builds for a prefix does not depend on anything after it. If the engine has read the system prompt, the context and the first three exchanges, and the next prompt begins with exactly those tokens, the state for them is still valid, and only the remainder needs reading. The rest of the chapter is about making that true on purpose, keeping it true across launches, and never letting it be true by accident.

Quadratic and linear

Consider a conversation of three turns under the naive arrangement, where each pass hands the engine the whole prompt and lets it read from the beginning. The first turn reads the system prompt, the context and the first question. The second reads all of that again, plus the first answer and the second question. The third reads everything a third time. Turn n re-reads what turns one through n minus one already read, so the total prefill work over a conversation grows with the square of its length.

With reuse, the first turn reads the same as before, the second reads only the first answer and the second question, and the third reads only the second answer and the third question. The total is linear. At a thousand-token system prompt the difference is an annoyance. At plank's prompt sizes it is the difference between a usable agent and one that stalls for most of a minute before every reply, and at the ten thousand tokens a real session's transcript reaches after an hour it is the difference between a turn that starts in a second and one that starts in a minute, on every turn.

The engine already does the reuse within one process if the harness lets it. A local engine holds one live state, and when the next prompt extends the prompt that built it, the engine compares, finds the common prefix and processes the suffix. Chapter 7's compaction prose called this out as the reason a summary request is the live transcript plus a suffix, and the reason the task list is injected once after a rebuild rather than mid-transcript. This chapter makes the mechanism visible in the engine trait and then extends it in the direction the engine cannot go alone: across the end of the process.

Four requirements

Plank's cache document states four requirements in priority order, because they conflict and the order is how the conflicts are settled.

The first is reuse across turns. Within one run, turn n plus one must not re-read what turn n read. The second is reuse across launches. Quitting the agent and starting it again must not cost a full rebuild of the system prompt. This is a separate requirement because it forces the state onto disk, and the disk introduces every hard problem in the design. The first requirement alone could be met by keeping one engine alive in memory.

The third dominates both, and understanding why is the key to everything that follows. Never reuse wrongly. A wrongly reused state does not raise an error. It produces a model whose attention state was built from a different prompt than the one on screen, and the

model answers confidently, using tool definitions it no longer has, or a system prompt the user edited an hour ago, or a conversation that belongs to a different session. There is no exception, no log line, and no plausible way for a user to diagnose what happened. Against that, a cache miss costs seconds. Every fingerprint, version byte and signature in the chapter exists to make the wrong-reuse failure impossible rather than rare.

The fourth is bounded disk. A snapshot of the attention state for a large model is large: hundreds of megabytes for a system prompt, around a gigabyte for a long conversation. Plank's document records that this reached tens of gigabytes on the author's machine before a retention policy existed. Blanka's store gains its sweep in chapter 9; this chapter keeps the number of blobs small by construction, one per static prefix and one per project.

Four constraints

The third requirement would be easy if identity were simple, and four properties of the machinery make it hard. Plank learned each of them by getting it wrong, and each shapes a line of blanka's code.

The unit of identity is tokens, not text. The natural way to check whether a cached state matches a prompt is to compare the prompt text, and it does not work. Byte-pair encoding is many to one: many token sequences decode to the same string, and the tokenizer picks one canonical segmentation, the one its merge order produces. The sampler is under no such constraint. It can emit "in" followed by "to" where the encoder would have produced the single token "into", split a number at a different boundary, or produce a rare standalone token the merge rules would have absorbed. So detokenising a reply and retokenising it is not the identity function on token ids, even though it is on text, and one differing id shifts every position after it. A cache validated on text but indexed by position is a cache that can be confidently wrong. The consequence for the on-disk format is that a blob carries what it was built from and text alone cannot reconstruct it. Blanka's blob carries the engine's opaque bytes and a token count; plank's carries the token sequence itself, because plank's engine can be handed tokens and blanka's mock cannot.

The engine can only extend, never rewrite. A backend keeps more than a token count describes: sliding-window attention rows, compressed rows, indexer state. Its sync can extend the live state forward and cannot roll it back behind the live end, because a position count does not say how to undo those structures. The counter-intuitive consequence is that a prompt which is a strict prefix of the live state cannot be reused at all. It matches on every token and the reuse is still impossible. That is exactly what a reset command produces, and plank's document records that resetting a session used to rebuild the whole system prompt in silence. It also records a user-visible corollary that is worth carrying: if the code reports the raw matching prefix length in that situation, the progress bar arrives full and then sits there through a multi-thousand-token prefill with no feedback, which to anyone watching is a hang. So plank reports the reusable length as zero unless the whole live state is kept:

```
// plank src/engine.rs at 7047808
pub fn reusable_prefix(pos: i32, common: i32) -> i32 {
```


Chapter 8: Where the time goes

```
if pos > 0 && common == pos { pos } else { 0 }  
}
```

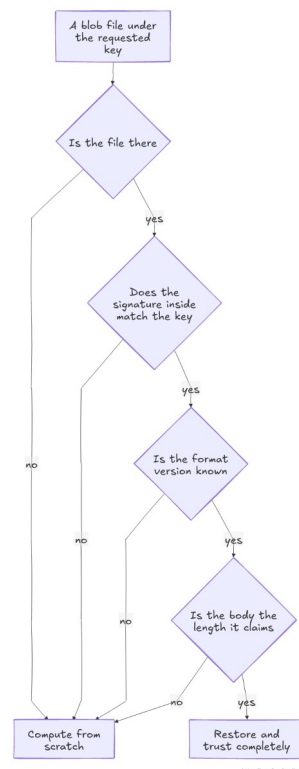
The same constraint bites in a less obvious place, and chapter 7 promised the number. Microcompaction rewrites old tool results in place. That is not appending a message or removing one; it is a rewrite behind the live end, at whatever depth the rewritten result sits, and everything after it must be processed again, including the long stretch of transcript that did not change. Plank measured an eighteen-turn session that paid 72,769 tokens of re-prefill this way across five rebuilds. The mitigation is chapter 9's; this chapter builds the engine that can count the cost.

A cache boundary has to fall on a message boundary. It would be convenient to snapshot the state in the middle of a long block of context text, at whatever offset suits. It cannot be done. Byte-level encoding merges across a seam, so the tokens of a stable text are not necessarily a prefix of the tokens of the stable text with a volatile tail appended; two pieces that concatenate cleanly can tokenise to sequences that diverge at the join. On top of that, the chat template wraps each message and closes the wrapper at the message's end, so a mid-message split lands inside a structure that is not closed. This is why plank injects its session-start context as two user messages, a stable one and a volatile one, rather than one block, and why chapter 6's single context message becomes two in this chapter. The text the model sees is the same. The tokenisation now has a boundary where the cache needs one.

A snapshot is the whole session, not a range. The capture primitive serialises the entire live state, and there is no call for the first n positions. That sounds like a limitation and is a scheduling rule: the order of operations when building a layered prefix is fixed. Sync to the end of tier i , capture and store tier i , and only then sync tier i plus one. Building the whole prefix and attributing parts of it to tiers afterwards is not possible. Worse, the mistake is undetectable by fingerprint. Storing tier i after tier i plus one has been processed writes the deeper state under a key that is correctly computed for the shallower tier. The key is right and the bytes are wrong, and nothing downstream can tell.

The shape of the decision

Put the third requirement together with the four constraints and the rule the whole implementation serves fits in one sentence: reuse only a genuinely matching token prefix, and rebuild anything else rather than trust it. The decision a loader makes has this shape:



Every rejection leads to the same place, and a hit is correct only because every check passed

Two commitments come out of that shape. Every rejection path leads to the same place. Absent, stale, truncated and written by an older version are not distinguished by the caller, because there is no useful difference between them: all four mean compute instead. Collapsing them removes a category of bug where one rejection reason is handled and another is overlooked. And a rejection must be cheap and routine, never exceptional. If a miss were expensive or awkward to handle, there would be pressure to avoid one, and that pressure is what produces a wrongly trusted cache.

Separate by rate of change

The most important design decision is that the prompt prefix is not one cache entry but several, split by how often each part changes. This is not a size optimisation. It follows from the second and third requirements together. The static prefix changes when the build does, perhaps weekly. Project context changes when an instruction file is edited, perhaps daily. Git status changes between turns. If those live in one entry, the entry's lifetime is the lifetime of its most volatile member, and the expensive stable part is thrown away every time the cheap volatile part moves.

Chapter 6 drew the boundary between static and volatile for the model's sake, so that the static prefix would be the same bytes for every session. This chapter draws two more for the engine's sake. Blanka has three tiers. The static tier is the system prompt. The project tier is the stable half of the session context: the instruction file and the skill roster. The session tier is the volatile half: the git state and the date. Plank has a fourth, the conversation itself, cached per session so that a resumed session does not re-read its own

Chapter 8: Where the time goes

history; that is chapter 9's subject, because it needs the ladder of snapshots that chapter also builds.

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum Tier {
    Static,
    Project,
    Session,
}

pub struct TieredPrompt {
    /// Part of every key: a snapshot from one model is meaningless to another.
    pub model: String,
    pub tiers: Vec<(Tier, String)>,
}
```

Three decisions inside that structure need their own explanation, and each is a line in the tier module.

Each fingerprint chains its parent's. The project tier's key is computed from the static tier's key, not merely alongside it, and this is what makes lookup sound. Because a child's key embeds its parent's, a valid deep tier is proof that every tier above it is valid too. The walk can therefore restore the deepest hit without independently revalidating the chain above it, and a stale blob cannot be mistaken for a fresh one, because staleness anywhere in the chain changes every key below it.

```
pub fn fingerprint_through(&self, tier: Tier) -> String {
    let mut fp = {
        let mut h = Sha256::new();
        h.update(self.model.as_bytes());
        h.update(b"\0");
        h.update(SYS_PROMPT_VERSION.to_string().as_bytes());
        h.update(b"\0");
        h.update(self.tiers[0].1.as_bytes());
        hex(&h.finalize())
    };
    for (t, text) in self.tiers.iter().skip(1) {
        if *t > tier {
            break;
        }
        let mut h = Sha256::new();
        h.update(fp.as_bytes());
        h.update(b"\0");
        h.update(text.as_bytes());
        fp = hex(&h.finalize());
    }
    fp
}
```

Chapter 8: Where the time goes

The session tier is never cached. Caching it would be strictly wasteful: its content changes every launch, so a blob written for it could never be read by anything. Recognising which parts of a system are not worth caching is as much a part of cache design as deciding what is, and the tier module says so in its key function, which returns no key for the session tier and no key for a project tier with nothing in it.

```
pub fn key(&self, tier: Tier) -> Option<String> {
    match tier {
        Tier::Static => Some(format!("sysprompt-{}", self.fingerprint_through(tier))),
        Tier::Project => self
            .text(Tier::Project)
            .map(|_| format!("project-{}", self.fingerprint_through(tier))),
        Tier::Session => None,
    }
}
```

The model and the version are key material. Chapter 6's fingerprint named the text and nothing else, and said that this chapter would combine it with the model's identity when the snapshot arrived. It arrives here. A state built by one model is meaningless to another, so the model's identity goes into the static tier's hash, and the engine trait gains a method to supply it. The prompt version goes in beside it, so that a build that changes the prompt text and bumps the version produces a new key even in the pathological case where the text change hashes the same, and so that the version has a home outside the model-facing text, as chapter 6 promised.

Plank's key material includes two inputs that do not change a byte of the prompt, and they are the better illustration of why text is not identity. The reasoning level matters because one setting prepends a reasoning preamble ahead of the system prompt: identical system text, different token prefix. The trusted length is subtler. It marks where plank's tokenizer stops treating the prompt as control text; inside that span the DSML delimiter is the model's own vocabulary token, outside it the same characters become spelled-out pieces. Identical bytes, different token stream, the same positions pointing at different things. A blob keyed on text alone would be restored under the wrong setting and processed against a state that does not describe it, found weeks later by someone noticing that the model behaved oddly at one reasoning level. Blanka's mock has no such settings and its key has no such inputs, and the place for them, when a real engine arrives, is the static tier's hash beside the model.

The model's identity turned out to need care in both directions. Chapter 5 described how plank's second model family shares the cache directory with the first, with each blob's recorded model deciding which launch may list, sweep or restore it, because a restore across families that tokenize differently would be a silent corruption rather than a miss.

The opposite case is a weight delta. Plank can load a small patch file that describes a derived model as a byte difference against a base checkpoint, applied to a copy-on-write clone of the base so that only the diverged blocks cost disk; on one edited checkpoint that was about 1.2 gigabytes against an 87 gigabyte base. The engine reports the same compile-time shape name for the base and the derived weights, so the two share every cache, and plank's findings record that this is accepted on purpose. The edit touched

thirty-three tensors at rank one, which sits within the noise the model already tolerates from quantisation, and prefilling the system prompt again per variant would cost more than it buys. The note ends with an instruction worth copying for any deliberate exception: do not fix it by suffixing the model name without also deciding that the cache split is worth it. Identity in a key is a claim about which states are interchangeable, and sometimes the right claim is a coarser one than the files suggest.

The tests pin the properties rather than the hashes. The static fingerprint is the same for two prompts that differ in project and session text. The project fingerprint changes when the static text does, which is the chaining. The key changes with the model. Trailing whitespace does not make a new tier, for a reason that has its own section below.

The engine grows a memory

The engine trait from chapter 1 has three methods: generate, count tokens, and the window size. It gains seven, and every one of them has a default.

```
pub struct KvBlob {
    pub signature: String,
    pub tokens: usize,
    pub bytes: Vec<u8>,
}

pub trait Engine {
    // generate, count_tokens, ctx_size as before, then:
    fn model_id(&self) -> String { String::new() }
    fn warm_reset(&mut self) {}
    fn warm_append(&mut self, _text: &str) -> Result<(), EngineError> { Ok(()) }
    fn warm_sync(&mut self) -> Result<usize, EngineError> { Ok(0) }
    fn warm_len(&self) -> usize { 0 }
    fn get_kv(&self) -> Option<KvBlob> { None }
    fn set_kv(&mut self, _blob: &KvBlob) -> Result<(), EngineError> {
        Err(EngineError::Protocol("this engine keeps no state to restore".into()))
    }
}
```

The defaults describe an engine with no memory. Warming does nothing, a sync processes nothing, there is no state to capture, and a restore refuses. That last default matters more than the others. It would be easy to have the default restore succeed silently, on the theory that an engine without state has nothing to get wrong. A caller that believed a restore happened would then skip the prefill it still owed, and against a real engine that would be the wrong-reuse failure in a new costume. Refusing by default means an engine has to opt in to being trusted. The echo engine and the hosted engine keep every default, and the hosted engine gains one line, returning its model name, because the model name is part of a key even for an engine that will never produce a blob.

A hosted engine keeps the defaults for a reason beyond the obvious one that it has no state to hand over. Providers do cache prompts, and the economics are different: the

Chapter 8: Where the time goes

cache is keyed on an exact prefix of the request as the provider sees it, it is priced per token, and it expires on a schedule the harness cannot see. The static and project tiers built here still help, because a request that begins with the same bytes every time is what a provider's cache needs too. What the provider cannot use is a blob, and what the harness cannot do is warm it in advance. Chapter 7 noted that the reference hosted harness spends a whole subsystem making its summary request share the main conversation's cache key; that effort is the hosted equivalent of this chapter, and the trait leaves room for it without requiring it.

The split between appending and syncing is plank's, and it is there for a reason the warm walk below depends on. Appending extends a buffer that describes what the engine's state should cover. Syncing computes state up to the end of that buffer and reports how many tokens it had to process, which is zero when the state already covered the buffer. Keeping the two apart lets the walk describe a prefix that was restored from disk without processing it, and it is the only way the walk can tell the engine about tiers it did not have to pay for.

The blob is three fields. The signature is what the store wrote it under and is the only thing a loader trusts. The token count is for reports and for chapter 9's arithmetic. The bytes are opaque to everything except the engine that made them. Plank's equivalent adds the token transcript for the reason given above; blanka's mock encodes its state as text and a real engine would encode its own tensors, and the store does not care which.

A boxed engine forwards all seven, so that the binary can still choose its engine at runtime and hand it to an agent that does not know which one it holds. That forwarding block is where a reader who adds a method to the trait later will forget to add it, and the compiler will not say so, because the default will silently apply to the box. It is worth reading the block once to know it is there.

One writer, one reader

Every blob in plank, whether it is a system-prompt checkpoint, a project tier or a session's conversation, is the same type in the same format, written by one function and read by one. That was not the original state. The document records three near-identical fingerprint-plus-bytes implementations and two payload shapes before the consolidation, and gives the reason it mattered here more than it would elsewhere: a cache whose correctness depends on a signature check is only as trustworthy as its least careful reader. Three readers means three places for someone to add a well-intentioned fallback, and one fallback is enough to reintroduce the wrong-reuse failure. With one reader, the trust decision exists in one place and can be audited by reading a dozen lines.

Blanka's dozen lines are these.

```
pub fn decode(bytes: &[u8], expect: &str) -> Option<KvBlob> {
    let nl = bytes.iter().position(|&b| b == b'\n')?;
    if &bytes[..nl] != expect.as_bytes() {
        return None;
    }
    let body = bytes.get(nl + 1..)?;
    if *body.first()? != FORMAT_VERSION {
```

Chapter 8: Where the time goes

```
    return None;
}
let tokens = u32::from_le_bytes(body.get(1..5)?.try_into().ok()?) as usize;
let len = u64::from_le_bytes(body.get(5..13)?.try_into().ok()?) as usize;
let bytes = body.get(13..)?;
if bytes.len() != len {
    return None;
}
Some(KvBlob { signature: expect.to_string(), tokens, bytes: bytes.to_vec() })
}
```

The file is a signature line, a version byte, a token count, a length and the bytes. The read is fallible by value: it returns an option, and every rejection is the same option. A missing file is handled by the caller that opens it, with the same answer. The version byte exists so that a change to the format invalidates every blob on disk, which is safe because every blob is a cache and the cost of losing all of them is one warm-up. The length is blanka's addition, and the reason is the file cut short by a crash. Without a length, a truncated body decodes to a shorter state that an engine would accept and a model would answer from. Plank's format detects truncation because its token transcript has its own framing; blanka's blob has no internal structure the reader could check, so the length carries the check.

The store's write is atomic for the same reason chapter 5's transcript write was: a temporary file beside the target, then a rename. Plank's document adds a detail that chapter 5 did not need. Two instances of the agent can have the same cache directory open, and a half-written body that still parses would be a wrongly trusted cache. The rename makes the transition atomic, so an interrupted write cannot leave a half-blob that reads as valid, and the process id in the temporary name keeps two writers from sharing a temporary file, since interleaved writes to one path could splice two snapshots into a body that decodes cleanly and describes nothing.

The blobs live in the session directory, beside the transcripts, as chapter 5 said they would, with an extension of their own. Plank's history explains why the extension is not an afterthought. Its transcript files and its system-prompt checkpoints originally shared one extension, which forced the garbage collector to filter candidate filenames by prefix so as not to delete a user's saved conversation while cleaning up checkpoints. A deletion routine whose safety rests on a filename prefix is one careless glob away from destroying user data. Bodies got their own extension, the transcript extension came to mean transcript and nothing else, and the safety property became structural rather than vigilant. Blanka's transcripts are JSON and its blobs are not, so the split is there from the start, and chapter 9's sweep will filter on the blob extension so that a transcript is not merely unlikely to be deleted but unreachable by the code that deletes things.

The tests are the trust decision's negative space. A blob comes back under its own key and under no other. A file renamed to another key on disk is still a miss, because the name on disk is not the trust input; the signature inside is. A body cut short is a miss, a body with a foreign version byte is a miss, an empty file is a miss, and none of them is a panic. A save leaves no temporary behind, because a stray temporary would one day be read by a globbing sweep.

There is no sidecar yet. Plank keeps a small JSON file beside each blob recording what it is, which blob it extends, how often it has been used and whether it is pinned, and the document is emphatic that this metadata is advisory: the signature in the body is the only trust input, and a missing, corrupt or disagreeing sidecar degrades the display and never gates a load. Chapter 5 adopted that rule for the transcript's sidecar, where the consequences of breaking it are a missing title. The blob's sidecar arrives in chapter 9 with the sweep that needs it, and the rule arrives with it unchanged.

An engine made of counters

The real local engine arrives in chapter 9, wrapped around llama.cpp, and it will implement the seven methods against actual tensors. This chapter needs something else first: a way to watch the cache work, in a test, in milliseconds. What the cache is about is not a model. It is a memory, and the cost of not having one. So the mock engine holds no model. It holds a list of the messages it has computed state for, and a count of the tokens it had to process to get there.

```
pub struct MockKvEngine {
    /// The messages the engine holds state for, in order.
    live: Vec<String>,
    /// The messages a warm walk has described so far.
    buffer: Vec<String>,
    script: VecDeque<String>,
    ctx: usize,
    pub prefill_calls: usize,
    pub tokens_prefilled: usize,
    pub rebuilds: usize,
}
```

Its two rules are a real engine's rules, and one function enforces both.

```
fn sync_to(&mut self, want: &[String]) -> usize {
    let common = self.live.iter().zip(want).take_while(|(a, b)| a == b).count();
    let from = if common == self.live.len() {
        common
    } else {
        // Extend, never rewrite: a divergence means a rebuild from zero.
        self.rebuilds += 1;
        0
    };
    let cost: usize = want[from..].iter().map(|s| approx_tokens(s)).sum();
    if cost > 0 {
        self.prefill_calls += 1;
        self.tokens_prefilled += cost;
    }
    self.live = want.to_vec();
    cost
}
```

Chapter 8: Where the time goes

State is a prefix: a prompt that extends the live state costs only the extension. And state cannot be rolled back: a prompt that diverges from the live state anywhere costs the whole prompt again, because the engine has no way to undo what it holds past the divergence. That second rule is the reusable-prefix rule from plank's engine, expressed as a counter, and it is deliberately harsh. A real backend might salvage the common prefix in some cases; modelling the case it cannot salvage is what makes the tests honest about the worst cost.

The pass follows the same function. A structured prompt is a list of messages, each message is one unit of state, and the pass syncs to the list, replies from its script or with a fixed word, and then appends the reply to its live state, because a real engine's state after decoding covers the reply too and the next pass, whose prompt ends with that reply as an assistant message, extends it. The stats report the tokens processed as the prompt tokens, which is what a real engine's stats report, and what the agent's turn report shows.

A blob is the live state encoded as text with a separator, and a restore decodes it back. The tests for the mock are short and each pins one rule. A second pass that extends the first pays for the new user message and nothing else. A pass whose system message changed pays for everything, and the rebuild counter moves. A pass that sends a strict prefix of the live state, which is what a reset command sends, is a rebuild too, and that test is the reset problem from plank's document written down before the code that solves it exists. A blob captured from one mock and restored into another leaves the second holding the first's state with its processed-token count at zero, because a restore computes nothing.

The warm walk

At startup the agent has a tiered prompt and a directory of blobs, and it has to get the engine into the best state it can. The walk that does it is forty lines, and plank's version of it is the part of the cache document with the most lessons per line.

```
pub fn warm<E: Engine + ?Sized>(
  engine: &mut E,
  prompt: &TieredPrompt,
  store: Option<&KvStore>,
) -> Result<WarmReport, EngineError> {
  let mut report = WarmReport::default();
  let tiers: Vec<(Tier, &str)> = prompt.messages().collect();
  engine.warm_reset();

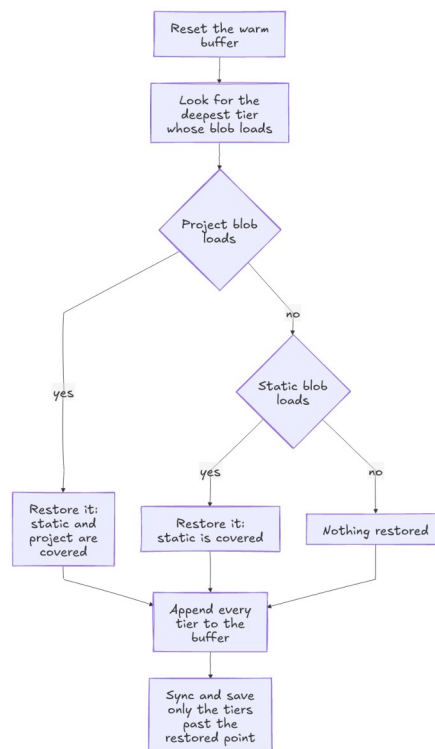
  // Restore: deepest first. A hit deep in the chain proves the chain
  // above it, because each key folds in its parent's fingerprint.
  let mut resume = 0;
  if let Some(store) = store {
    for (i, (tier, _)) in tiers.iter().enumerate().rev() {
      let Some(key) = prompt.key(*tier) else { continue };
      let Some(blob) = store.load(&key) else { continue };
      if engine.set_kv(&blob).is_ok() {
        report.restored = Some(*tier);
      }
    }
  }
```

Chapter 8: Where the time goes

```
        resume = i + 1;
        break;
    }
    // Keyed right, refused by this engine: a miss, not an error.
}
}

// Extend: append every tier, sync and capture from the restored point on.
for (i, (tier, text)) in tiers.iter().enumerate() {
    engine.warm_append(text)?;
    if i < resume {
        continue;
    }
    report.prefilled += engine.warm_sync()?;
    if let (Some(key), Some(store)) = (prompt.key(*tier), store) {
        if let Some(blob) = engine.get_kv() {
            if store.save(&key, &blob).is_ok() {
                report.saved.push(*tier);
            }
        }
    }
}
Ok(report)
}
```

The shape, as a walk over the three tiers:



<https://lenzoland.net>

Chapter 8: Where the time goes

The warm walk: restore the deepest hit, describe the whole prefix, pay for the rest

The restore is deepest first, and the chaining is what makes that sound. A project blob that loads under a key computed from the static fingerprint is proof that the static prefix matches too, so the walk does not check the static blob separately. A blob that is keyed correctly but refused by the engine is a miss, not an error; the walk keeps looking upward rather than trusting bytes the engine would not take. Plank's version says the same and adds a notice to the user that the cache is incompatible with this build and is being rebuilt, because a refused blob usually means the engine changed under the cache.

Three properties of the extend loop are load-bearing, and plank's document says each was learned by getting it wrong.

Every tier is appended, including restored ones. The engine's buffer has to describe the whole prefix its state covers. Skip the append for a restored tier and the buffer has a hole; the next sync sees a common prefix shorter than the buffer, treats the difference as a divergence, and throws the restored state away. A deep cache hit then costs more than a cold start, which plank's document calls a memorably confusing thing to debug, and which the commit that fixed it names in one line: keep restored tiers in the warm token buffer. In blanka the append is unconditional and the skip covers only the sync, and the test for the second launch checks that the engine's live state describes three messages after restoring two.

Each tier is captured at its own boundary, for the whole-session reason from the constraints. The capture happens inside the loop, after this tier's sync and before the next tier's append, and it cannot move. Capturing after the next sync would store the next tier's state under this tier's key, and the key would be correct for what it claims and wrong for what it holds. The cold-walk test asserts three sync calls for three tiers, which is the observable shape of a capture at every boundary, and a refactoring that batched the syncs for speed would fail it.

A tier the restore skipped is never written. This follows from the other two and is the subtlest consequence in the design. The walk restores the deepest valid tier and skips everything above it; a skipped tier is never synced; a tier never synced is never captured. So once the project tier is valid, the static blob stops being written, and if it was never written before that, it never will be. For the main engine this is invisible, because the project tier is a superset of the static tier and restoring it is strictly better. It matters for any consumer that needs the static tier alone, which is exactly the situation of a sub-agent running on a different engine than its parent, and chapter 12 will have to warm the static tier explicitly rather than assume the main walk left it on disk. The second-launch test pins the property from the other side: a restored tier is not written again.

The tests walk the mock through the situations a user produces. A cold walk with a store prefills everything and saves the two cacheable tiers. A second launch with the same prompt restores through the project tier and pays for the session tier only, which in the test is ten tokens against eleven hundred. An edited instruction file restores through the static tier, pays for the project and session tiers, and saves a new project blob, leaving the old one in place for chapter 9's sweep. A changed static prefix restores nothing and pays for everything. A walk without a store still warms the engine, so that the first turn costs

its own suffix even when nothing can be kept. And a walk over the echo engine does nothing and writes nothing, because an engine without memory has nothing to capture, and the store stays empty rather than filling with blobs of nothing.

What the agent does differently

The agent gains a method that splits its prompt into tiers and a method that walks them, and the binary calls the second before the first prompt is read.

```
pub fn tiered_prompt(&self) -> crate::kv::TieredPrompt {
    let (project, session) = match &self.context {
        Some(c) => (c.render_stable(), c.render_volatile()),
        None => (String::new(), String::new()),
    };
    crate::kv::TieredPrompt::new(self.engine.model_id(), &self.system, &project,
    &session)
}

pub fn warm(&mut self) -> Result<crate::kv::WarmReport, EngineError> {
    let prompt = self.tiered_prompt();
    let report = crate::kv::warm(&mut self.engine, &prompt, self.kv.as_ref())?;
    self.warm_report = Some(report.clone());
    Ok(report)
}
```

The other change is in the first turn of a fresh session, and it is the one chapter 6 deferred. The session context was one user message with the stable half before the volatile half. It is now two messages, the stable one first, and the stable one is omitted when there is nothing in it. Chapter 6's replay, title and dump logic recognised the context by its opening lines, and both halves open with a marker, so nothing downstream changes; the test that counted the context as one user message now counts two.

The property the whole arrangement rests on is that the strings the walk appended and the strings the first pass sends are the same strings. If they differ by a byte, the engine holds state for a prefix it is never asked about, the first pass diverges at that tier, and the walk was wasted without anyone noticing, because the turn still works. Plank hit exactly this, and the commit that fixed it is worth quoting for the shape of the bug: canonicalize tier text so warm prefill matches the turn. Its context injection ended with a newline; its turn path trimmed trailing whitespace from every message before tokenising; the warm path did not. The tiers matched on every launch, the blobs restored, and every first turn re-processed the project context and everything after it. The fix was to trim at planning time, so the tier text is exactly the text the turn will send. Blanka trims both tiers at construction and trims the two context messages before pushing them, and a test does what plank's finding suggests a test should have done from the start: warm the mock, run a turn, and assert that the messages the pass sent begin with the messages the walk left in the engine.

```
#[test]
fn the_tiers_are_exactly_the_messages_the_turn_sends() {
```

Chapter 8: Where the time goes

```
let mut a = agent("same");
a.warm().unwrap();
let warmed = a.engine.live().to_vec();
a.run_turn("hi", &mut Channels::default()).unwrap();
let sent: Vec<String> = a.prompt_messages().iter().map(|m|
m.content.clone()).collect();
assert_eq!(&sent[..warmed.len()], &warmed[..]);
}
```

The companion test is the measurement the chapter opened with, reduced to a counter. After a warm walk, the first turn's prompt tokens equal the tokens of the word the user typed, and the rebuild counter is zero. Before the walk the engine had processed over a thousand tokens for the static prefix alone. The difference between the two numbers is what a user waited through on every launch before this chapter, and what they wait through once per build after it.

Chapter 6's comment on the prompt-message builder said that this chapter would make the prefix an engine concern, and the way it does so is worth being precise about, because the builder itself barely changes. Every pass still sends the system prompt in front of the transcript, as strings, rebuilt each pass. What changed is on the other side of the trait. The engine matches those strings against the state it already holds and processes only the remainder, so the copies are cheap and the state behind them is not recomputed. The builder's comment now says that, and the mock's counters are the proof.

The context report gains a line. After the token counts by block it shows what the last warm walk did: which tier was restored, how many tokens were processed, how many blobs were written. It is the fastest way to answer the question a user asks when a launch felt slow, and the answer is usually that the instruction file changed.

What a resumed session costs

Chapter 5's resume replays a transcript from disk, and the transcript's first two messages are the context that launch found: the instruction file as it was that day, the git status as it was that day, that day's date. The walk at the start of the new launch warmed today's tiers, and today's session tier is a different string. So the first pass of a resumed session diverges from the engine's state at the third message, and under the mock's rules that is a rebuild from zero, static prefix included. The mock is being harsh where a real backend would salvage the common prefix of two messages, but the tail is lost either way: everything from the old volatile context to the end of the resumed conversation is processed again, and for a session resumed after an hour of work that is the whole hour.

This is the honest cost of resume at the end of this chapter, and it is worth stating because chapter 5 promised that the cache would make resume instant and this chapter does not deliver that yet. What it delivers is the static and project tiers, which are the part of the cost that every launch shares. The part that a resumed session adds, its own history, is a fourth tier keyed on the project fingerprint and the transcript, saved when the session is saved and restored when it is resumed, and it is chapter 9's first job. The reason it cannot be done here is the second constraint again. The old volatile context sits between the project tier and the conversation, so a blob of the conversation is a blob of a prefix that

begins with the old context, and restoring it means the engine holds the old context while the walk has described the new one. Plank resolves this by making the conversation blob extend the project tier directly and treating the session's own context messages as part of the conversation, which is what they are once they are in the transcript. Blanka will do the same, and the walk built here is the base it extends.

Until then the arrangement has one property that matters for the reader following along with a real engine: a resumed session is never wrong, only slow. The engine diverges, rebuilds, and answers from a state that describes exactly the transcript on screen. Slow is the failure the third requirement permits.

Interrupts and the cut

Chapter 4 recorded an interrupted reply verbatim, as far as it got, and said that the reason would become clearer when the cache arrived. It is this. The engine's state after an interrupted pass covers the prompt and every token it produced up to the cut. The transcript's assistant message holds exactly those tokens, because chapter 4's renderer returned what the engine produced and nothing else. So the next pass, whose prompt ends with that message and the interruption notice, extends the engine's state, and the interrupted reply costs nothing to keep.

A harness that recorded something other than what the engine produced would pay for it here. Trimming the partial reply to the last sentence, or appending a marker to it, or dropping it and recording only the notice, would put a message in the transcript that the engine's state does not describe, and the next pass would diverge at that message. Against the mock the rebuild counter would move. Against a real engine the user would wait through the whole conversation again as the price of having pressed a key. Chapter 4's rule, record what the engine produced, was stated for the sake of the log-everything invariant; the cache gives it a second, measurable reason, and the mock is where a reader can measure it.

The same reasoning explains a choice in the mock that might otherwise look like a shortcut. After a pass the mock appends the reply to its live state, as a real engine's state covers the decoded tokens. Without that line the mock would charge the next pass for the reply as if it were new prompt text, and every test that counts the cost of a second turn would be wrong by the length of the first answer.

A lookup on the wrong side of a cut

The extend-only constraint has a quieter consequence, and plank found it in its own reconciliation code. When a pass ends with the model's reasoning left open, which a guard stop or an interrupt can cause, the harness closes it before continuing, so the next prompt carries the recorded reply plus exactly the closing tag. Re-tokenising that span from text produces ids unrelated to the ones the model sampled, for the reason the first constraint gave, and the engine rebuilds from that point on. A recorded session lost a prefix of fifty-six thousand tokens to it, over a difference of eight bytes. Plank had a splice path for precisely this case, keeping the sampled ids and inserting the close in front of the recorded end-of-sequence token.

The path was unreachable. The reconciliation truncated its token spans to the common prefix first and then looked for the span to splice, and the span it looked for was the first one the truncation had dropped. Every lookup found nothing, for every input, with no error and no log line, and every guard-stopped pass paid a full re-prefill to append eight bytes. The fix moved the decision before the truncation and widened what the truncation keeps by one span when a close is due. The findings file generalises it in a sentence that applies well beyond this call site: a lookup positioned at a truncation boundary is silently dead on the wrong side of it. The decision now lives in a module that needs no native engine, where the test suite compiles it and a test asserts both orders, the wrong one answering nothing and the right one splicing.

A related property of the native engine decides what an interrupted prefill leaves behind. The cancel check is polled per token during prefill as well as during generation, and a prefill that runs in chunks rewrites the engine's checkpoint after every chunk, so an interrupt in the middle of a long prefill leaves a valid shorter prefix rather than nothing. Not every path chunks, and one reports its only cacheable point at the end, so how much survives an interrupt depends on which path the engine took. Plank's rule is that anything reporting the cost of the next prefill reads the checkpoint that actually survived rather than assuming one.

Giving the memory back

Every mechanism in this chapter exists to keep engine state around. Plank has one that throws it away on purpose, and it tests the design from the other side.

The live session of a large local model is several gigabytes of attention state, and on Apple hardware part of it is wired: the kernel cannot page it out. The model's weights are a memory-mapped file the kernel can drop and read back; the session is not. So when another application drives the machine into memory pressure, plank's session is the largest thing it holds that the system cannot reclaim by itself, and the system will page out everything else and still be short. Plank listens for the kernel's pressure notifications, stops at a token boundary when the level turns critical, frees the session outright, and comes back when the pressure has cleared.

The deliberate omission is the snapshot. Nothing on the yield path captures the state. Serialising the session would allocate a second copy of the thing that is too large, at the moment memory is scarce, which is the failure the yield exists to avoid. The yield frees; it does not save. What makes that affordable is that the resume already has a floor on disk. If a snapshot from the ladder the next chapter builds survives at a depth at or below the current transcript, the deepest one is the restore point; otherwise the tier blobs of this chapter are, and the static tier alone spares the largest single span. The cost of a yield is bounded by the distance from the deepest surviving blob to the live cursor, not by the length of the conversation, and the resume is an ordinary warm walk, so it needs no new trust machinery: the signature checks already decide what may be restored.

The decisions are asymmetric, and the constants say so. Yielding happens on the first critical reading, with no dwell, because the point is to return memory to a thrashing machine as fast as possible. Resuming waits for thirty seconds of continuous normal readings, because a resume costs a prefill and a premature one pays it twice. And a

Chapter 8: Where the time goes

minimum of two minutes between yields stops a resume whose own prefill re-triggers the pressure from turning into a loop; the module's comment says plainly that being killed by the out-of-memory killer is preferable to an hour of re-prefilling in a cycle.

Three orderings in the implementation are load-bearing. The restore plan is decided when the session is freed, not when the pressure clears, and pinned into the sweep's keep set, because in the interval the blob the resume will need is, by every measure the sweep uses, a cold file nobody holds open. The session is freed before the cancel reason is cleared, because in the other order a turn racing the resume can enter generation, which clears the flag on entry, and start a pass against a session about to disappear. And the yielded state ends when the session is rebuilt, not when the pressure clears, so the line announcing the re-prefill appears before the wait it describes rather than after thirty seconds of silence. Two refusals complete it. A session holding image state is never yielded, because the rebuild would re-prefill the image positions with no embeddings behind them and produce a session that looks whole and is silently ungrounded. And micro-compaction is suppressed while yielded, because restoring a snapshot below its rewrite would lazily re-acquire the very session the yield had freed.

The yield taught two lessons about state machines that plank's findings record in general form. The first: the hysteresis commits its yielded flag before the caller acts on the verdict, so every path on which the caller then finds it cannot free, an image session, a turn in the wrong phase, an engine that declines, must roll the flag back by hand. Miss one and the machine stays latched with nothing freed, and later pressure is swallowed as already handled, which reads as plank ignoring memory pressure and shows up in no test that drives only the happy path. The second: the minimum interval was enforced inside the state machine's observe method, and the mid-pass yield never called it. That path reported its yield to the machine afterwards and never asked permission first, so the guard looked fully wired while covering half the paths that could yield, and because the rebuild at the next turn cannot be cancelled by pressure, sustained critical pressure meant one full rebuild and free per turn, the loop the constant existed to forbid. The fix is a query both paths consult before acting. A guard that lives inside a state machine protects only the callers that go through it, and any out-of-band path that tells the machine what happened has to ask it on the way in as well.

Precision where it matters

Chapter 7's cheap compaction pass used the four-bytes-per-token estimate to budget how many results to clear, and said that precision would arrive in this chapter where it matters. The pass now takes the engine's count as a closure and the agent passes its engine's tokenizer.

```
pub fn microcompact(
  t: &mut Transcript,
  used: usize,
  ctx: usize,
  count: &dyn Fn(&str) -> usize,
) -> usize
```

Chapter 8: Where the time goes

The reason it matters here and did not in chapter 7 is the second constraint. A rewrite costs the engine its state from the rewritten point onward, so every result the pass clears beyond what the floor needs is a result that was lost for nothing and paid for twice: once in the tokens the model can no longer see and once in the re-prefill of everything after it. An estimate that runs high clears too few and the pass runs again next round, which is cheap. An estimate that runs low clears too many, and against a real engine the cost of the extra rewrites is seconds per round. With the engine's count the pass stops exactly when it has done enough. For the mock and the hosted engine the closure is still the estimate, so nothing in chapter 7's tests changed except the extra argument.

The cost of the rewrite itself, which chapter 7 called out and deferred, is now a number the mock reports. A microcompaction that rewrites a result at message twenty of a sixty-message transcript makes the next pass diverge at message twenty, and the mock's rebuild counter moves and its token counter grows by the whole prompt. Against plank's engine the same rewrite cost the 72,769 tokens of the measured session. Chapter 9's ladder is the mitigation: snapshots taken at depths along the transcript, so that a rewrite at depth twenty restores the snapshot below it and processes forty messages instead of sixty from zero. The mock built here is the engine those tests run against.

The smoke run

The binary gains an engine option for the mock, and the run that accompanies the chapter is three launches from a scratch project with an instruction file, against a fresh home directory.

The first launch prints a cache line before the prompt: nothing restored, 1,471 tokens processed, two tiers saved. The context report after one turn agrees: the static prefix is 1,380 tokens, the session context 91, and the turn cost the two tokens of the word typed. The second launch, with nothing changed, prints a different line: restored through project context, 62 tokens processed, no tiers saved. The sixty-two are the session tier, the git status and the date, which is the volatile text that can never be cached and is the whole cost of a launch once the blobs exist. The third launch follows an edit to the instruction file, and the line reads: restored through static prefix, 96 tokens processed, one tier saved. The static blob was trusted, the project tier was recomputed and a new project blob written, and the old project blob is still in the directory, where chapter 9's sweep will find it.

The session directory afterwards holds the transcript and its sidecar, the system prompt text file from chapter 6, one static blob and two project blobs. Two fingerprints appear beside each other in the listing and they differ, which is worth explaining rather than leaving for the reader to wonder at. The text file is named by chapter 6's fingerprint of the prompt text alone, because it exists to let a dump or a resume show the exact text that was sent. The blob is named by the static tier's chained fingerprint, which folds in the model identity and the prompt version, because it exists to be trusted by an engine and the same text is a different state on a different model. Two purposes, two keys, one text.

The run proves the plumbing and the arithmetic. Against the mock the sixty-two tokens cost nothing that a stopwatch could see. Against the local model Appendix D wraps, the

same three launches are the difference between a start that waits through thirteen hundred tokens of prefill and one that waits through sixty, and the reader who has watched the first will not need the second explained.

Where blanka stops short of plank

Plank's cache subsystem is several thousand lines across a planner, a store, a metadata type, a tree view, a garbage collector and a snapshot ladder, and its document runs to fifteen hundred lines. Blanka's chapter is four hundred lines, and the gaps are of three kinds.

Some are engine-specific and blanka's engine does not exist yet. Plank splits its first tier in two at the trusted length, because the two halves change at different rates: the built-in prompt above the cut moves when plank is rebuilt, and the tail below it, tool schemas and the sub-agent roster, moves whenever a tool set does. Undivided, the cheap change paid for the expensive one, and a few hundred tokens of schema appended at the end re-processed a fourteen-thousand-token prompt, about a minute of wall clock. The cut is legal at that one offset and nowhere else, because plank's engine already sends the two halves through separate tokenizer calls, so the boundary is a hard token boundary and a blob may sit on it. The document is explicit that this must not be generalised to any other offset in the prompt, and blanka's engine, when it arrives, will have to earn the same exception on its own evidence before taking it.

Some are user-facing and wait for chapter 9. Plank explains a static-tier miss before the progress bar starts moving, by diffing the prompt text behind the previous blob against the current one and printing the first change, so that a benign cause, a new date in the wrong place or a ticking tool count, is obvious at a glance. It keeps a sidecar per blob, browses the cache as a lineage tree from a command, pins blobs, and sweeps by age and then by a ceiling with a keep list that protects the live session. Blanka has the blobs and the keys, and the sweep, the sidecar and the command arrive when the ladder does, because a ladder is what makes the directory grow.

The largest gap is the ladder itself and the session payload. Plank caches the conversation as a fourth tier, keyed on the project fingerprint and the transcript, so that a resumed session does not re-read its own history, and it keeps up to three snapshots at depths along the transcript so that a rewrite behind the live end restores the nearest rung below the edit instead of rebuilding from zero. That is the mitigation for the 72,769 tokens, it is what turns a reset command from a full rebuild into a restore at the tier boundary, and it is the whole of chapter 9.

Blanka's mock has no pressure to respond to and no wired memory to return, so the yield stays in plank, along with the family routing in the cache directory and the weight deltas that deliberately share a key. The splice before the truncate has no blanka analogue either, because the mock's state is text and a close tag costs it nothing, but the lesson from it applies to any engine a reader wraps: decide what to keep before cutting, never after.

What blanka has is the requirement stated in order and the four constraints that make it hard, three tiers split by rate of change with chained fingerprints, a model and version in every key, a trait with defaults that describe an engine without memory and a restore that refuses by default, one writer and one reader with a read that fails by value, an

atomic write with the process id in the temporary name, a warm walk with its three load-bearing properties each pinned by a test, a first turn whose messages are exactly the tiers the walk left in the engine, a cheap compaction pass that counts with the engine's tokenizer, and a mock engine whose counters make the whole arrangement measurable in a test that runs in milliseconds.

The bill for remembering

A turn is two phases, and on a local machine the one you wait through is the one that reads the prompt. Plank measured it at a hundred and sixty tokens a second against a prompt of thousands, which is why a bare model feels fast and the same model inside an agent feels slow, and why the fix is not a faster model but a harness that stops asking the engine to read what it has already read.

The engine remembers a prefix and can extend it, never rewrite it, and never shrink it. So the harness keeps the prompt's prefix stable, splits it by rate of change into a static tier, a project tier and a session tier, and stores the state for the two that can recur, under keys that chain each tier's fingerprint into its child's and fold in the model and the prompt version. A walk at startup restores the deepest key that loads, describes the whole prefix to the engine, restored tiers included, and pays only for what remains, capturing each cacheable tier at its own boundary and never writing a tier it skipped.

Trust lives in one function. A blob is restored only when the file exists, the signature inside matches the key asked for, the version is known and the body is the length it claims, and every other outcome is the same answer, compute instead. Nothing outside that function decides whether cached bytes may be used, and nothing ever will, because a wrongly trusted cache does not fail; it answers.

The same discipline covers the moments when state is thrown away. Plank's yield under memory pressure frees the session without saving it, because the floor for the resume is already on disk, and every ordering around it, the plan pinned when the session is freed, the free before the flag, the rebuild as the end of the yield, exists so that giving memory back never costs more than the distance to the deepest blob that survived.

CHAPTER 9: RUNGS, ROLLBACK AND FORKS

Chapter 8 ended with a number it could count and not yet avoid. Plank measured an eighteen-turn session that paid 72,769 tokens of re-prefill across five full rebuilds, and every one of those tokens was still valid: the transcript before the rewrite had not changed, the engine's state for it was correct, and the engine threw it away anyway, because a rewrite had landed behind its live end and it can only extend forward. The cheap compaction pass from chapter 7 was the cause each time. It rewrote an old tool result to a stub, the next pass diverged at that message, and the mock engine's rebuild counter, had it been watching, would have moved by one for each rewrite. Chapter 8's tests made it watch, and it did.

The tiers from chapter 8 do nothing for this case. A tier boundary sits at a fixed depth, at the end of the static prefix or the project context, and a rewrite can land anywhere past it in the conversation. What this case needs is a snapshot that can be taken anywhere a rewrite might later occur, so that the restore can pick whichever one predates the actual edit. The shape of that answer follows from two facts about the cheap pass itself, and the chapter opens with them because everything else is a consequence.

The first fact is that the edit point only moves forward. The pass walks the transcript oldest to newest, and a cleared body becomes a stub too small to be a candidate again, so the message it rewrites next is always deeper than the one it rewrote last. The pass rewrites in place, never inserting or removing, so indices do not shift under it. The second fact is that a snapshot only helps when it predates the edit. One at the live end is useless, because the live end is exactly where the next edit is not. Together these mean a single rolling snapshot cannot work, since it always postdates the next edit, but a small ladder of snapshots at increasing depths does: whichever rewrite comes next, some rung on the ladder already sits behind it.

Plank calls the snapshots rungs, holds at most three per session, and restores the deepest one below an edit before the rewrite happens, so that the engine's live end becomes the rung's depth and the next pass extends forward from there instead of from zero. Blanka builds the same ladder over the mock engine from chapter 8, and then uses the same primitives for three things a user asks for by name: a checkpoint to return to, a branch from an earlier point, and a session that resumes with its history already read.

The one rule

A rung is a blob in chapter 8's store, and the store trusts only the signature inside a blob, so the question that decides whether the ladder works at all is what key a rung is written under. The answer is the fingerprint of the transcript as it stood at capture time, which is to say truncated to the rung's own depth. At restore time the transcript has grown. Fingerprinting it whole would produce a key nothing was ever written under, and the load would miss on every call, silently, while the feature looked implemented. Plank's findings file marks this in capitals, and its invariants list states it as a property to check before changing anything: a rung is looked up under the fingerprint of the transcript truncated to its own recorded depth, never the current transcript. Get this backwards

Chapter 9: Rungs, rollback and forks

and every rung misses forever. Nothing short-circuits, nothing errors, the code path never hits.

```
pub fn transcript_fingerprint(prompt: &TieredPrompt, messages: &[Message]) -> String
{
    let mut h = Sha256::new();
    h.update(prompt.fingerprint_through(Tier::Project).as_bytes());
    for m in messages {
        h.update(b"\0");
        h.update(m.role.label().as_bytes());
        h.update(b"\0");
        h.update(m.content.as_bytes());
    }
    h.finalize().iter().map(|b| format!("{b:02x}")).collect()
}

pub fn rung_key(prompt: &TieredPrompt, messages: &[Message]) -> String {
    format!("rung-{}", transcript_fingerprint(prompt, messages))
}
```

The fingerprint chains the project tier's, so a rung is tied to the prompt it was taken under: the same conversation under an edited instruction file is a different key, as it should be, because the engine's state for it would be different bytes. The session tier is not in the chain, and the reason is worth a sentence. Once a session has begun, its context messages are in the transcript, and the transcript is what is hashed. A rung taken today over a session begun yesterday covers yesterday's context as the messages it is, which is exactly what the engine's state covers too.

A second property of the fingerprint is easy to miss and plank's finding names it. A match does not say that the blob's depth matches the current transcript. The fingerprint is computed over the truncated transcript, so a match tells you that the truncated prefix is unchanged since capture, and says nothing about what comes after the rung's depth, which is precisely the part about to be rewritten. That is the intended behaviour, not a gap. The rung is a state for a prefix, the prefix is what is checked, and the rewrite is the caller's business.

Plank's finding adds the condition that makes the truncated fingerprint reproducible: its transcript renderer reads only the transcript and the system prompt and has no length-dependent formatting, pinned by a test with a name that says what it forbids, that the renderer never injects the task list mid-transcript. Blanka's transcript is a list of messages hashed one by one, so the condition holds by construction, and chapter 7's decision to inject the task list only after a rebuild is what keeps it holding.

The test for the rule is the one the plan asked for, written against the wrong key as well as the right one.

```
// The one rule: look up under the prefix as it stood at capture.
assert!(store.load(&rung_key(&prompt(), &t.messages()[..rung.depth])).is_some());
```


Chapter 9: Rungs, rollback and forks

```
assert!(store.load(&rung_key(&prompt(), t.messages())).is_none(), "the whole transcript is not the key");
```

The rung itself holds no bytes. It remembers a depth in messages, a token count, a slot index from a counter that never reuses a value, and the key its blob was written under so the blob can be deleted when the rung goes. Lookups do not use the stored key. They recompute it from the transcript, which is the rule made structural: a caller cannot look a rung up wrongly, because the only way to look one up is the function that truncates.

Where a rung is taken

Plank's first design captured rungs at turn ends, and it could never have worked. The findings file records the measurement: an eighteen-turn session captured eleven turn-end rungs and used none of them. The reason is where the edit lands. The cheap pass clears oldest first, so the edit sits near the start of the transcript, while within a single turn the transcript jumps from one message to six, and no turn boundary exists at a usable depth. A rung has to be taken at the exact index a later rewrite will be known by, and that index is the position of the large tool result itself.

So the anchor is taken immediately before a large tool result is appended. At that instant the transcript's length is exactly the index the result will occupy, which is exactly the index the clear set will later report as the edit point, so the rung's depth and the edit are equal and the deepest-rung-below-the-edit test holds with equality. It is also the one moment in the loop when the engine's state describes exactly the transcript: the pass has returned, its reply has been appended, and the mock's live list is the prompt plus that reply, which is the system message plus the transcript.

```
report.tool_rounds += 1;
// A snapshot at the depth this result will occupy, taken while the
// engine's state still describes exactly the transcript: this is
// the depth a later rewrite of the result will be known by.
if feedback.len() > crate::compact::MICRO_MIN_BYTES {
    self.maybe_anchor();
}
self.transcript.push(Role::Tool, feedback);
```

The threshold is chapter 7's minimum for a clearable result, so a rung is only ever taken where the cheap pass could later rewrite. A small result is never a candidate for clearing and so never earns a rung.

Whether the ladder wants a rung at that depth is two rules in order. If nothing on the ladder covers an edit at this depth, yes: the first anchor for an index is free of competition and always taken. If something does, only when the new rung would cover materially more tokens than the one that does. Plank sets that spacing at 8,192 tokens, and its comment gives the arithmetic: at a measured hundred tokens a second of prefill, 8,192 tokens is about eighty seconds saved if the pass fires, three orders of magnitude above the capture cost of a few tenths of a second and a few hundred megabytes written, while halving the capture rate the old blind turn-end spacing produced. Blanka uses the same

Chapter 9: Rungs, rollback and forks

number and lets the tests lower it, because a test that wants three rungs from three results cannot wait for thirty thousand tokens of transcript.

```
pub fn wants_rung(&self, depth: usize, tokens: usize) -> bool {
    match self.select(depth) {
        None => true,
        Some(best) => tokens.saturating_sub(best.tokens) >= self.spacing,
    }
}
```

Three rungs is the cap, and plank's comment is explicit that this is a disk decision and not a latency one. Each rung on a real engine is two to four hundred megabytes, and a cache directory that already runs to tens of gigabytes cannot afford many. When a fourth arrives, one interior rung is evicted: whichever one's removal least widens the largest remaining gap, never the shallowest, because it alone can cover an edit near the start of the transcript, and never the newest, because it was wanted a moment ago. The evicted rung's blob is deleted in the same call, so the ladder and the directory agree.

The ladder tests walk the mock through the shape. Three rounds with a large result each produce rungs at depths two, four and six; a restore for an edit at message five picks the rung at four, and a fresh engine restored from it holds a state five messages long, the system prompt plus four. A fourth round evicts the rung at four and leaves two and eight, and the store holds three blobs, not four. A spacing of ten thousand tokens refuses a second rung a hundred tokens deeper than the first.

Decide, then move the engine

Where the restore happens is a rule plank learned from a review finding it marks as critical, and blanka's cheap pass now runs in the order the finding demands.

A restore replaces the engine's live state with the rung's shorter prefix. If the harness restores first and then decides whether to compact, the refusal path is a disaster: the transcript is never rewritten, but the engine has already been rewound, so the very next pass re-reads exactly the tail the harness declined to spend, and since the reclaimable total only grows, the same refusal and the same waste repeat every turn after. Plank's document calls this strictly worse than not having the feature. The fix is that nothing which moves the engine runs before the decision that justifies it, and the same selection result drives both the decision and the restore.

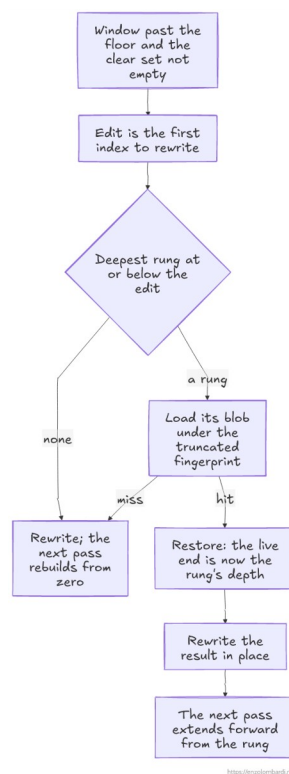
In blanka the decision is chapter 7's: the pass fires when the window is at or past the floor and the clear set is not empty. Both are known before anything touches the engine. Only then does the agent look for a rung.

```
// The rewrite is decided; now, and not before, the engine may be
// moved. Restoring a rung the pass then declined to use would leave
// the engine short of state for nothing.
if let (Some(kv), Some(&edit)) = (&self.kv,
crate::compact::clear_set(&self.transcript).first()) {
    let prompt = self.tiered_prompt();
    if let Ok(Some(depth)) = self.ladder.restore_before(&mut self.engine, &prompt,
```

Chapter 9: Rungs, rollback and forks

```
&self.transcript, edit, kv) {  
    report.rung_restored = Some(depth);  
    sink.status(&format!("[engine rewound to message {depth} before clearing]"));  
}  
}  
let count = |s: &str| self.engine.count_tokens(s);  
report.micro_rewrites = crate::compact::microcompact(&mut self.transcript, used, ctx,  
&count);
```

The edit is the first index the pass will rewrite, and the restore is strictly before the rewrite. Restoring after the mutation would fingerprint a prefix that no longer exists. The sequence for one rewrite, against a ladder of three rungs:



A rewrite behind the live end, with and without a rung below the edit

A miss anywhere on that path, no rung shallow enough, a stale fingerprint, a blob that would not load, leads to the same place as having no ladder at all: the rewrite proceeds and the next pass re-reads, exactly as chapter 7 left it. The restore is a performance mechanism only, and plank's document says so in as many words. It can make a pass cheaper and it can never make one wrong.

The agent tests make the saving visible with the mock's counters, and their setup is chapter 7's small-window arithmetic again. An eight-thousand token window puts the floor and the trigger both at six thousand, and the prompt is about fourteen hundred. Six four-kilobyte results cross the floor inside one turn, the cheap pass clears the oldest, and it does so again on each of the next two rounds as the window refills. With a cache store and a ladder, the first large result earned a rung at its own depth, every later edit is

Chapter 9: Rungs, rollback and forks

at or past that depth, the engine is rewound to it before each rewrite, and the rebuild counter stays at zero. Without a store the same turn rebuilds once per rewrite, and the test asserts equality between the two counts, because that is the relationship: every rewrite behind the live end is a rebuild from zero, unless something sits behind it.

```
#[test]
fn a_rewrite_behind_the_live_end_rewinds_to_a_rung_instead_of_rebuilding() {
  let mut a = agent("rung", true);
  let mut sink = Channels::default();
  let report = a.run_turn("fetch", &mut sink).unwrap();
  assert!(report.micro_rewrites >= 1, "{:?}", sink.status);
  assert_eq!(report.compactions, 0);
  assert_eq!(a.engine.rebuilds, 0, "the rung covered the edit");
  assert!(sink.status.iter().any(|s| s.contains("rewound")), "{:?}", sink.status);
  assert_eq!(a.ladder.rungs().len(), 1, "later results were too shallow past the first to
earn a rung");
}
```

The last assertion is the spacing rule at work. Each later result is about a thousand tokens deeper than the one before, far short of the 8,192 the ladder demands for a second rung, so one rung serves the whole turn. Against a real engine that single rung is the difference between re-reading fourteen hundred tokens of prefix plus a growing conversation three times in one turn and re-reading a few thousand tokens of tail.

Following the transcript

A rung describes a prefix, and a rung describing a prefix that no longer exists is worse than no rung. The ladder keeps reporting the depth as covered, so no new anchor is taken there, while a restore hands back a state for messages that are gone. So the ladder has to follow the transcript through every operation that is not a plain append, and plank's document enumerates them.

A full compaction destroys every prefix the rungs described, so the ladder is discarded and its blobs deleted, in the same place the task list is re-injected. A reset to a new session, a resume of a saved one and a rollback to a checkpoint each replace the transcript, and each discards the ladder first, while the transcript the rungs describe still exists. That ordering is another of plank's findings: the in-memory ladder must be discarded whenever the session it describes is discarded, not only the blobs on disk. Left pointing at the old session's rungs, the ladder compares the new session's smaller token counts against a stale deepest rung, finds no gain, and never captures again until the new session outgrows the old one, while its restores hand back prefixes that were never on the live transcript at all.

A fork is different, and it is the operation that gives the ladder its truncation. Chapter 5's fork continued in a copy of the whole transcript, and for that shape every rung still describes an intact prefix and stays. This chapter gives the fork an optional depth: keep the first *n* messages and branch from there. Rungs at or below the cut still describe a prefix that exists in the branch and stay; rungs deeper than the cut were captured over

Chapter 9: Rungs, rollback and forks

messages the branch does not have, and go. The method that does it hands the dropped rungs back so their blobs can be deleted, which is the same contract as eviction.

```
pub fn truncate_to(&mut self, depth: usize) -> Vec<Rung> {
    let (keep, dropped): (Vec<Rung>, Vec<Rung>) = self.rungs.drain(..).partition(|r|
r.depth <= depth);
    self.rungs = keep;
    dropped
}
```

The fork at a depth also has an engine problem that the ladder solves for free. The branch's transcript is a strict prefix of what the engine holds, and chapter 8 established that a strict prefix cannot be reused: the engine cannot shrink. Without help, the branch's first turn rebuilds from zero. With the ladder, the fork asks for the deepest rung at or below the cut and restores it, so the engine's live end is at or before the branch point and the first turn extends. The rung was taken for the cheap pass; the fork uses it because a fork is, to the engine, the same event as a rewrite at the cut.

Renaming a session, which in plank drops the in-memory ladder because rungs are named after the session id, costs blanka nothing. Blanka's rung keys are fingerprints of content, and a rename changes no content. This is one of the places where a design decision made for one reason, trusting only content, pays for itself somewhere unrelated.

One operation that looks like a setting is a transcript event to the ladder. Plank lets the user change the model's reasoning level mid-session, and its session fingerprint folds in the level's name, so every rung captured under the old level can never be found again once it changes. Left in place, such rungs are worse than absent: the anchor rule keeps measuring against a rung that no lookup will return, the ladder reads as covered, no new anchor is ever captured, and the blobs leak on disk, all silently. Plank discards the ladder on every accepted change of level, unconditionally rather than only when the rendered preamble differs, because two levels share an empty preamble and not a fingerprint, and gating on the preamble would leave exactly that pair broken. A key that includes an input is a promise that the ladder follows every change to that input, whatever the menu calls it.

Sub-agents, which run as a fork of the live transcript and fold their messages back out when they finish, are chapter 12's subject, and the rule they will need is already stated in plank's invariants: a sidechain never writes the live session's cache, because a rung or a payload captured over messages about to be truncated describes a prefix that will not exist. Blanka's ladder truncation is the mechanism that chapter will call when a sidechain ends.

The rule has a counterpart that plank found later, when a sidechain's own pass had to rebuild. A reasoning guard that stops a pass inside a sub-agent rewrites the tail of a message the engine had already run past, a divergence behind the live end, and on the main path plank rescues that shape by restoring the deepest rung below the divergence before the engine syncs. The sub-agent path did not call the rescue, and the rescue refused inside a sidechain anyway, so each such stop re-read the whole parent context and the sidechain from token zero, once per trip. The fix made the rescue three-tiered

and ran it before every quiet sub-agent pass: first the innermost fork snapshot, which sits exactly at the fork point, so only the sidechain's own messages are processed again; then the deepest ladder rung below the divergence, which is valid inside a sidechain because rungs are fingerprinted over the intact parent prefix; then a rebuild. Reading from the ladder in a sidechain is safe where writing to it is not, and the distinction is the whole rule. Both tiers are switched off while a sub-agent runs on a separate clean-room engine, whose state is not the session's and whose prompt is small enough to rebuild, and plank keeps a depth counter for exactly that purpose.

Two moments, and who owns them

Plank's ladder module is pure logic: depths, token counts and a spacing rule, with no engine and no store in it. Everything that touches an engine or a disk happens in the agent, and plank's document is precise about what the agent owns: two moments. It captures a rung once per large result, after any compaction that turn already ran, so that the prefix is stable when it is signed. And it restores a rung strictly before the cheap pass mutates the transcript, using the already-known edit point to pick the deepest usable one.

Blanka's ladder is slightly less pure, because its capture and restore methods take the engine and the store as arguments rather than leaving the calls to the agent. The choice was made for the reader: a method that captures, writes, records and evicts in one place is easier to follow than four calls the agent has to make in the right order. What stays with the agent is the knowledge the ladder cannot have, which is when the engine's state equals the transcript. The ladder can be told a depth and a token count; only the loop knows that the pass has returned, the reply is appended and nothing else has moved. A rung taken at any other moment is signed for a prefix the engine's state does not describe, and no fingerprint can tell.

The same division explains why the restore is called from the compaction path and nowhere else. The ladder has a general selector, the deepest rung at or below a depth, and the fork uses it too. But the decision to move the engine belongs to whoever knows a rewrite is coming, and only the cheap pass and the fork know that. A helper that restored a rung whenever one seemed useful would restore on the refusal path plank's review caught, and the engine would be worse off than with no ladder at all.

The reset command, revisited

Chapter 8 described the reset command as the clearest case of the extend-only constraint. A new session's prompt is the system message, today's context and the user's first line, and the engine holds the whole previous conversation, of which that prompt is a strict prefix. It matches on every token and cannot be reused, and plank's document records that resetting a session used to rebuild the entire system prompt in silence.

The fix plank adopted, and blanka adopts here, is to treat a reset as what it is to the engine, a rewrite at the end of the project tier, and to restore a blob there. A reset discards the ladder and the checkpoints, replaces the transcript, and then runs chapter 8's warm walk again. The walk finds the project tier's blob under the key it was written under at launch, restores it, and syncs the session tier on top. The engine's live end is now the end

Chapter 9: Rungs, rollback and forks

of the context, the first turn of the new session extends it, and the test asserts that the turn costs under a hundred tokens and the rebuild counter does not move.

Without a store the walk still runs and still computes, and against the mock that is a rebuild, because the mock's sync finds the buffer diverging from a longer live state and starts from zero. That is the honest cost of a reset on an engine with no blob to fall back on, and the reason chapter 8 wrote the project tier to disk on the first launch rather than only when the process ended.

The reset also shows, in one method, the housekeeping order the ladder section demanded. The ladder is discarded while the old transcript still exists, so that the keys of the blobs to delete can be read from the rungs. The checkpoints are cleared, because a checkpoint holds a transcript that is no longer this session's. Only then is the transcript replaced and the engine moved. A reset that replaced the transcript first would have a ladder pointing at prefixes it can no longer name, and plank's finding about the in-memory ladder is the description of what that looks like a turn later.

A place to come back to

The ladder is a mechanism the user never sees. Its rungs are taken and restored by the harness for its own reasons, and the only trace is a status line. The same two engine methods, capture and restore, give the user something they do ask for by name: a point in the conversation to return to.

Plank added checkpoints to close an issue that asked for named in-session rollback points, and the design document that came with them settles four questions that a reader building the feature would otherwise settle by accident.

What a checkpoint holds is the whole transcript at capture time, not an offset into it. An offset would be smaller, and it would be wrong the moment a compaction rewrote the transcript underneath it, because the messages at that offset would no longer be the ones the user meant. Holding the whole transcript lets a rollback cross a compaction boundary: the pre-compaction conversation comes back exactly, however the live one was rewritten in between. A checkpoint also holds the task list, because a rollback that kept tasks the rolled-back conversation had not yet added would be lying about the state, and it holds the engine's state when the engine can give one, so that the turn after a rollback extends instead of rebuilding.

```
pub struct Checkpoint {  
    pub name: String,  
    pub created: u64,  
    pub messages: Vec<Message>,  
    pub tasks: TaskList,  
    pub kv: Option<KvBlob>,  
}
```

Why the command to return is a separate verb, rollback, rather than a sub-command of checkpoint, is a question of what reads well at a prompt and of the convention the rest of the command set uses. A destructive operation and a non-destructive one get different verbs, so that a user cannot type the dangerous one by mistyping the safe one. Why a

Chapter 9: Rungs, rollback and forks

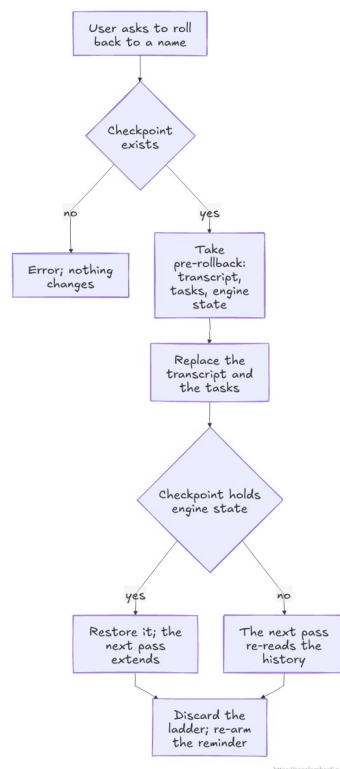
bare checkpoint command lists rather than auto-naming is a conflict the original issue did not notice it had: it sketched both behaviours for the same input. The implementation picks the unambiguous one. Bare lists; creating requires a name; and the one implicit checkpoint the system takes for itself has a fixed name.

That implicit checkpoint is the fourth decision, and the most useful. Before a rollback replaces the transcript, the harness saves the transcript it is about to replace under the name pre-rollback. A rollback is therefore undoable by rolling back to that name, and a user who returns to a checkpoint and regrets it has lost nothing. The state being left is worth as much as the state being restored, and the harness does not ask the user to have thought of that in advance.

```
pub fn rollback<E: Engine + ?Sized>(
    &mut self,
    name: &str,
    engine: &mut E,
    t: &mut Transcript,
    tasks: &mut TaskList,
) -> Result<bool, String> {
    let cp = self.get(name).cloned().ok_or_else(|| format!("no checkpoint named {name}"))?;
    // The state being left is worth as much as the one being restored.
    self.take(PRE_ROLLBACK, engine, t, tasks);
    *t = Transcript::from_messages(cp.messages);
    *tasks = cp.tasks;
    Ok(match &cp.kv {
        Some(blob) => engine.set_kv(blob).is_ok(),
        None => false,
    })
}
```

The return value says whether the engine took the state, and the command reports it, because the difference is what the user will feel on the next turn. On the mock, and on a real local engine, the checkpoint's blob goes back into the engine and the next pass extends from the checkpoint's end. On an engine without state, the echo engine and any hosted one, the checkpoint recorded only the text, the rollback restores the text, and the next pass re-reads. Plank's document notes one caveat for its engine: the restore clears the engine's memory of its last sampled reply, so the final assistant turn is re-read from text on the next pass, and the saving is near-zero rather than zero. The bulk of the prefix stays.

The agent wraps the store's rollback with the housekeeping the ladder section demanded. The ladder is discarded, because every rung described the transcript that was replaced, and the pressure reminder is re-armed, because the transcript it was sent into is gone. The rollback as a sequence:



A rollback saves what it replaces, then restores text, tasks and state together

Checkpoints are per session and in memory. They are cleared when the session is replaced, by a reset, a resume or a fork at a depth, and never written to disk. Plank's document defers persistence to its per-session cache format and says it is not required; blanka does the same, and the reason is that a checkpoint is a place in a conversation the user is having now. A place worth keeping across launches is a session, and chapter 5 already keeps those.

The tests are the plan's: a rollback, then a rollback to pre-rollback, restores the original transcript and its tasks; the pass after a rollback with state extends and the rebuild counter does not move; an engine without state rolls back the text and reports that it did not take the state; a reused name replaces rather than duplicates, and the listing shows the newest prompt and marks checkpoints that carry state.

The session's own history

Chapter 8 left resume slow on purpose and said the fix was this chapter's first job. A resumed session's transcript begins with the context of the launch that started it, the walk at the start of the new launch warmed today's tiers, and the first pass diverged at the old context and re-read everything. The fix is plank's fourth tier, the conversation itself, and the ladder's fingerprint is already the key it needs.

On every save, if the engine can give a state, the agent writes it to the store under the fingerprint of the project tier and the whole transcript, with a role prefix of its own, and records the key in the session's sidecar. The previous save's blob described a shorter transcript that nothing will ask for again, and it is deleted when the new one arrives, so a session holds one payload at a time. On resume, the agent recomputes the key from

Chapter 9: Rungs, rollback and forks

the loaded transcript and the running prompt, asks the store for it, and hands the blob to the engine.

```
let Some(kv) = &self.kv else { return Ok(false) };
let key = crate::kv::ladder::payload_key(&self.tiered_prompt(),
self.transcript.messages());
let Some(blob) = kv.load(&key) else { return Ok(false) };
Ok(self.engine.set_kv(&blob).is_ok())
```

The sidecar's record of the key is advisory, in exactly chapter 5's sense. It exists so that the sweep knows which blobs are a saved session's history and so that a listing can show them, and it is never the key resume asks for. Resume recomputes. A sidecar that names a blob for a prompt that has since changed, because the instruction file was edited between the save and the resume, asks for a key that no longer exists, and that is a miss, not a wrong restore. The test for it changes the project context between two launches, resumes, and checks that the transcript came back, that the engine took no state, and that the next turn still answers. Slow and right, which is the failure chapter 8's third requirement permits.

Plank's document records what happens when the payload restore and the warm walk are run in the wrong order, and the lesson transfers. Restoring a payload and then running the walk is strictly destructive: the walk's last act per tier is a restore of a blob whose transcript is empty by construction, which rewinds the live state from the end of the conversation back to the tier boundary. Plank skips the walk entirely after a payload restore, because the payload is a superset of every tier and there is nothing left to warm. Blanka's binary warms once at startup, before any command, and a resume replaces the engine's state afterwards, so the order is right by construction; a reader who moves the warm into the resume path should read that paragraph first.

The task list rides in the sidecar too. Chapter 7 kept it on the tool context and said it died with the process, which was a gap this chapter was to close when the store learned to keep a session's durable state beside its transcript. The list is small and structured, which makes it sidecar material rather than blob material, and it is state too important for a summary, which makes it exactly the kind of thing a resumed session must get back. Save copies it into the sidecar; resume copies it out. The payload test adds a task before the first turn and finds it after the resume.

The test that ties the section together saves a two-turn session with a task, opens a fresh agent over a fresh mock, warms it, resumes, and asserts four things: the engine took the state, its live list is the system message plus the saved transcript, the next turn cost the tokens of the word typed and nothing else, and the keep set names exactly one session payload. A third launch in the smoke run confirms it with the binary: the resume line says the engine state was restored, and the turn that follows costs two prompt tokens.

What the sweep may and may not touch

Chapter 5's sweep retired old sessions by age and then by a byte budget. Chapter 8 said blobs would join the store and the sweep would learn about them here. The blobs now

Chapter 9: Rungs, rollback and forks

have four roles, and the sweep treats them differently because they are worth different things.

A static or project tier blob is worth a rebuild of the prompt, and one on a different key from the running prompt is a previous version of the prompt or of the instruction file that may come back. It lives for the policy's tier lifetime, thirty days idle by default. A session payload is history: it is what makes a resume instant, and it is worth keeping as long as the session is. A rung is neither. It is a live-session accelerator, worthless the moment its process is gone, and an orphaned one, left behind by a crash or a kill or a machine losing power mid-session, has no reason to survive as long as a saved session. Plank gives rungs a backstop lifetime of one day, derived so that a stricter session lifetime can only tighten it, and blanka does the same.

```
pub fn sweep_blobs(kv: &KvStore, policy: &GcPolicy, keep: &[String], now: u64) ->
std::io::Result<Vec<String>> {
    let mut removed = Vec::new();
    for entry in kv.entries()? {
        if keep.contains(&entry.key) {
            continue;
        }
        let ttl = if entry.role() == "rung" { RUNG_BACKSTOP_SECS } else
{ policy.ttl.as_secs() };
        if now.saturating_sub(entry.modified) >= ttl {
            kv.remove(&entry.key);
            removed.push(entry.key);
        }
    }
    Ok(removed)
}
```

The keep list is what protects a running session from its own sweep. It names the tiers of the prompt in use, the live rungs, and the payload of the current session, and the binary adds the payload of every saved session, so that a sweep cannot collect a session's history from under it while the session's transcript is still there. Plank's sweep computes the same set and adds a detail blanka's test suite borrows in spirit: warm and sweep must run in the order a launch runs them, walk first and then sweep with the walk's tiers on the keep list, because a sweep before the walk deleted what the walk was about to restore, and neither was wrong alone.

Plank's finding about rungs and parentage is worth carrying even though blanka has no parent pointers yet. Plank's sweep spares any blob with a surviving child, so that an expired system prompt with a live session below it stays. Parenting a rung to its session would make the single most disposable blob in the cache the thing protecting the session payload, and the tier above that, from ever being collected. Rungs are recorded with no parent and rely on their short lifetime and on being first to go under a byte budget. When blanka's sweep gains lineage, in the shape plank's has, the rung stays parentless.

The listing command shows what the sweep sees: each blob's role, size and age, its key, and whether the running session is using it. The binary marks the live static tier, the live

Chapter 9: Rungs, rollback and forks

project tier and the current payload, and shows the stale project blob from the edited instruction file as unmarked, which is what the sweep will collect in thirty days. The command takes a sub-command to sweep now and one to remove a single blob by key prefix, refusing a prefix that matches nothing or more than one.

Plank's listing is a tree, because plank records lineage: system prompts at the roots, project contexts hanging off them, session payloads below those, with hit counts, ages and expiry, and keys to pin and delete. The document says why that is more than a convenience. Before the metadata existed, the tier chain was implied by a chain of hashes and was therefore unobservable, so the only way to answer why the system prompt rebuilt this morning was to reason about it from first principles. Making the structure visible converted a class of reason-it-out problems into look-at-it problems. Blanka's flat listing is the first step, and the sidecar per blob that would make it a tree is left for a reader who wants it, with plank's shape to copy.

One finding from that pane is general enough to state here. Plank's pane addressed blobs by their position in a directory scan, and a scan is a snapshot of a directory that another process can change. A blob unlinked by a second instance between the pane being drawn and a key being pressed shifted every later index down one, and the delete hit the neighbouring body. The fix carried an identity alongside the position and refused to act when the two disagreed, which turned a silent wrong-file deletion into a visible instruction to reopen the pane. Blanka's remove-by-prefix resolves the prefix against the directory at the moment of the command and refuses ambiguity, which is the same principle at a smaller scale.

What a rung costs

The ladder's constants are plank's, and each carries a measurement. A capture on plank's engine takes about four tenths of a second and writes two to four hundred megabytes; a restore takes about a tenth. A rebuild of a long conversation takes minutes. So the latency argument for rungs is not close, and if disk were free the ladder would hold a rung at every large result. Disk is not free, and the cache directory on plank's author's machine reached tens of gigabytes before a retention policy existed, so three rungs per session is where the two costs meet. The spacing constant is the other half of the same trade: a capture is a certain cost against a probabilistic gain, since the pass may never fire at that depth, and 8,192 tokens is where the gain if it fires, about eighty seconds at a hundred tokens a second, is three orders of magnitude above the cost.

Two findings about the disk side are worth carrying even though blanka's keys sidestep them. Plank names its rung blobs by session id and a slot number, and the slot number comes from a counter that never reuses a value, so a long session's three live rungs all have indices past the ladder's width. Cleanup code that iterated from zero to the ladder's capacity found none of them and left gigabytes on disk; the fix matched rung files by name pattern at any index. Blanka's rungs are keyed by content and deleted by the key the rung remembers, so there is no range to get wrong, but the shape of the bug generalises: a cleanup that enumerates what it expects to find, rather than what is there, misses whatever it did not expect.

The second is the crash. Every path that discards a ladder deletes its blobs, and a process that dies mid-session takes none of those paths. The rungs it leaves are worthless, since no process will ever compute their keys again for a transcript that no longer grows, and they are large. The one-day backstop in the sweep exists for exactly this case and no other. A reader who finds rung blobs in the store a day after a crash is seeing the backstop work; one who finds them a week after is seeing a sweep that did not run, because the binary sweeps only at launch.

There is a third cost the counters do not show, and chapter 7's two-hour session is the proof that it is real. A rung makes a rewrite cheap in tokens, and the arithmetic that weighs a pass sees only tokens. What it cannot see is that a result the model read a moment ago is a result the model is working from, and clearing it costs the model the information rather than the harness the prefill. Plank's floor at seventy-five percent exists because a fresh rung made the token cost zero and the arithmetic said yes after every turn. The ladder makes the cheap pass cheaper, and that is precisely why the floor in front of it must stay.

Tests that would pass with the feature deleted

Plank's cache document keeps a part on what went wrong, and two of its entries are about tests rather than code. They belong here because the ladder is the kind of feature that can be fully wired and completely inert, and the tests are the only thing standing between the two.

The first entry is about fixtures. Plank's cache pane displayed two fingerprints for each blob, one derived from the filename stem and one from the sidecar's contents, and for session payloads they legitimately differ. Every test fixture happened to construct entries where the two coincided, so a code path that confused them passed twelve reviews before a whole-branch pass caught it. A fixture that makes two distinct things equal cannot detect code that confuses them. Blanka's ladder test for the one rule is written with that entry in mind: it asserts that the key over the truncated transcript hits and that the key over the whole transcript misses, in the same test, on the same store. A test that asserted only the hit would pass against an implementation that keyed on the whole transcript, as long as the test happened to look the rung up before the transcript grew.

The second entry is about a test that stopped testing. A retention test whose fixtures were freshly written stopped guarding anything the moment retention became age-based, because freshness alone kept its entries alive, and it passed with the entire feature deleted. Any test that asserts something survives a policy has to make sure the policy's other protections are not silently doing the work. Blanka's blob sweep test writes four blobs and then sweeps with a clock two days ahead, so that the rung's one-day backstop is the only thing deciding, and then forty days ahead, so that the tier lifetime is; and it names two of the four on the keep list, so that survival by keep and survival by youth are asserted separately. The test would fail against a sweep that ignored roles, against one that ignored the keep list, and against one that swept nothing.

The same discipline explains why the ladder's agent tests come in a pair. A test that showed the rebuild counter at zero with a ladder would pass against a mock that never counted rebuilds. The companion test runs the same turn without a store and asserts one

rebuild per rewrite, which is what proves the counter counts and what makes the zero in the first test mean something. Plank's document lists, among the things its tests deliberately pin, that a second launch restores instead of prefilling using two separate engines, because a launch is a fresh process and a test that reused one engine would be showing the engine's own memory rather than the store's. Chapter 8's second-launch test and this chapter's resume test both use two engines for that reason.

A silent hit and a silent miss

A restore that worked and a restore that did nothing look identical from outside: the turn runs either way, and one is slower. Plank's cache document has a part on diagnosing a miss, and its central device is an enumeration of outcomes rather than a boolean. Its tier restore distinguishes a tier that is not cacheable, a key with nothing on disk under it, a file that is present and keyed right but would not load, and bytes that loaded but the engine refused, because each has a different cause and a different fix, and a bare false sends an investigation the wrong way.

Blanka's equivalents are the status lines and the context report. The warm walk reports which tier was restored and how many tokens it processed. The cheap pass reports the depth the engine was rewound to when a rung applied, and says nothing when none did, which is the case worth teaching a reader to notice: a compaction status line without a rewind line before it means the next pass rebuilt. The context report lists the ladder's rungs by depth and token count, and the resume command says whether the engine took the state. A reader who wants plank's precision has the shape in front of them, an enum returned from the restore with one variant per cause, and the reason to want it, which is that the miss that costs a minute per turn is the one nobody notices for a week.

The most common cause of a miss in plank is the fingerprint, and the document's example is one blanka will meet in chapter 12: the sub-agent roster is part of the system prompt, so a project with its own agent definitions keys the static tier differently from the same model in any other directory, and a user who expects the static blob to be shared across projects finds it rebuilt in that one.

The gate blanka leaves out

Chapter 7 deferred plank's three-part gate in front of the cheap pass to this chapter, and this is the place to say what it is and why blanka does not build it.

Plank's cheap pass, once the pressure floor is past, weighs what it would reclaim against what the rewrite would cost: bytes cleared per token re-read, where the tokens re-read are the transcript's length minus whatever the selected rung covers. That ratio is compared to a floor, and the floor is not fixed. It is strict at the pressure threshold and relaxes linearly to a small epsilon at the point full compaction fires, anchored to the same threshold as the expensive step so that the cheap decision can never sit blocking in front of the expensive one. The reason it relaxes is that the value of reclaimed context is not constant. A measured run offered 12,344 bytes for 10,674 re-read tokens, a ratio of 1.16, in a window two percent full, where the right answer is no: three thousand tokens of room are not worth ninety-eight seconds of prefill when nothing is short. The same trade at seventy percent of the window is worth taking, because there the alternative is a full compaction, a model round trip plus a total rebuild. The floor bottoms out at an epsilon

rather than at zero because zero means accept any pass at all, and a pass barely clearing the minimum could spend a restore and a rewrite moments before a full compaction discarded the ladder and rebuilt anyway.

That is a careful design, and plank's document is candid about its status in one sentence that belongs in this book: the refusals are measured, the acceptance is designed. The accepting branch has never fired in a live session. It needs the window around forty percent full before the floor relaxes far enough, and no session plank has run has filled a million-token window past three percent. What has been measured is the refusing path, and the two-hour session from chapter 7, where the pass fired after every tool-heavy turn in a window seven percent full because a fresh rung made the re-read cost zero and the arithmetic could not see the information cost of clearing what the model was working from.

Blanka keeps the precondition that finding produced, the seventy-five percent floor, and stops there. Below the floor nothing fires, so the two-hour session cannot recur. Above it the pass clears oldest first to a budget and the ladder makes the rewrite cheap when a rung is behind it. What blanka does not do is weigh a pass that the floor permits, and the reason is the sentence above. A gate whose accepting branch has never been observed is a gate whose tuning is a hypothesis, and a build-along should not ask a reader to implement a hypothesis. The ladder, by contrast, is measured on both sides in this chapter's tests, and the number it removes, the 72,769 tokens, was measured before it existed.

Branching as a way of working

A fork at a depth is a checkpoint the user can walk away from. A checkpoint returns the live session to a point; a fork leaves the live session where it is and starts a second one from the point, with its own file, its own name and its own future. The two answer different questions. A user who tried an approach and wants to try another from the same place rolls back, because the failed approach is not worth keeping. A user who wants both approaches, or who wants to ask a side question without polluting the conversation the model is in the middle of, forks, and the original session's transcript is untouched on disk while the branch grows.

To the engine the two are the same event. Both cut the transcript at a depth behind the live end, and both are served by the rung at or below the cut. The difference is what happens to the state being left. A rollback saves it under the pre-rollback name, because the user may want it back. A fork saves it as the parent session, because chapter 5 already did that on every turn, and the branch's sidecar records the parent so that the relationship is visible in the session list.

Plank's forks go further, and the direction is worth knowing because chapter 11 and chapter 12 walk it. A sub-agent in plank is a transcript fork: the framed task is appended to the live transcript, the ordinary turn loop runs, and the sidechain is truncated back out leaving only the report. It is cheap precisely because it never leaves the parent's session, and the per-turn prefix reuse serves the child as it serves the parent. What that shape cannot do is host two children at once, because two forks are two divergent continuations of one prefix and the engine has one live end. Plank's session-clone design

Chapter 9: Rungs, rollback and forks

names the missing primitive: take a live session, produce a second live session that starts from the first one's exact state, and let them diverge independently. The document lists what already existed towards that, a session split from the model weights, a cooperative scheduler, a snapshot-and-restore pair, and the one assumption not yet proven, that a snapshot of a still-live session restored into a fresh sibling yields a usable clone with no cold re-read and no damage to the original.

Blanka's fork at a depth is that primitive's smallest form, and the capture-and-restore pair from chapter 8 is the whole of its engine surface. A clone of a live session is a blob captured from one engine and restored into another, which is exactly what chapter 8's blob round-trip test does with two mocks. The vehicle plank chose to prove it, an aside answered concurrently with the main conversation, is chapter 11's, and the parallel sub-agents that rest on it are chapter 12's.

What the user sees

The plain front end gains three commands and a sub-command, and two existing ones say more.

A checkpoint command with a name takes a checkpoint and says whether it replaced one; without a name it lists them, with message count, age, the newest prompt at capture and a marker for those that carry engine state. A rollback command with a name returns to it, reports the message count and whether the engine took the state, and names the checkpoint that undoes it. A cache command lists the blobs with role, size, age and a live marker, and takes a sub-command to sweep now or to remove one blob by key prefix. The fork command takes an optional number of messages to keep and reports the branch's name and length. The resume command adds a phrase when the engine state came back, and the context report gains a line listing the ladder's rungs by depth and token count.

Two status lines appear inside turns. When the cheap pass is about to clear and a rung applies, the engine is rewound and the line says to which message. At launch, after the warm walk's line, a sweep that removed anything says how many blobs went.

Every command answers in plain text so that it works on a pipe, which is how the smoke run drives it and how chapter 5's replay was tested. Plank's equivalents open panes in its full-screen front end, with the cache as a foldable tree and keys to pin and delete, and its plain front end prints the same rows statically; the rule that both front ends read the same rows is the one blanka's front end will inherit when it gains a second face in appendix material.

The smoke run

The run that accompanies the chapter is against the mock, from the scratch project chapter 8 left, over three launches.

The first launch asks a question, takes a checkpoint named start, asks a second question, and lists checkpoints: one entry, four messages, the first question as its summary, and a marker that it carries engine state. A rollback to start reports four messages and the engine state restored, and says that a rollback to pre-rollback undoes it. A third question

Chapter 9: Rungs, rollback and forks

then costs its own tokens. The context report shows no rungs, because no tool ran and nothing large was appended, and the fork at a depth of three continues in a new session with three messages.

The second launch asks a question and quits, so that a session exists with a payload. The third resumes it: the resume line says four messages and the engine state restored, and the turn that follows costs two prompt tokens. The cache listing after it shows two project blobs, one live and one stale from the edited instruction file, one live static blob, and one live session payload.

The rungs themselves do not appear in the smoke run, because the mock's replies contain no tool calls and no result is ever large enough to earn one. They appear in the agent tests, where a scripted turn of six large results earns exactly one rung and the rewind line appears before each clearing. Against the model chapter 8 described, the same turn is the difference between a minute of re-reading three times over and a few seconds once.

No new methods

Everything in this chapter was built on chapter 8's engine trait without adding to it. The ladder captures with the same method the warm walk used to write a tier, restores with the same method the walk used to read one, and measures depth with the token count the walk reported. Checkpoints use the same pair. The session payload uses the same pair. The fork and the reset use the restore. Seven methods, of which three did all the work here, were the whole surface between a harness that forgets and one that remembers across turns, launches, rollbacks and branches.

That is not an accident of blanka's design; it is where plank arrived after going the other way first. Plank's engine trait grew a method per feature as the cache subsystem grew, snapshot and restore for checkpoints alongside separate warm and tier operations, and a commit in its history reduces the trait to four methods, capture, restore, a warm reset and a warm sync, and moves every feature onto them. The commit message says what was learned: the features differ in when they capture and when they restore, and in what they key the blob on, and not in what they ask the engine to do. A trait that exposed one method per feature would have made each feature's engine work look different when it was the same, and would have given every new engine seven things to implement where four sufficed.

The consequence for a reader adding a real engine, in the appendix that wraps llama.cpp, is that the engine's obligations are already fixed and small. It must be able to serialise its live state to bytes and read them back, extend a buffer of text and process it, and say how much it holds. The ladder, the checkpoints, the payload, the fork and the reset ask nothing else, and every test in this chapter and the last is a test of what the harness does with those answers, not of how the engine produces them. Swapping the mock for the real engine changes what the counters cost and nothing about what they count.

There is one thing the chapter did add outside the engine, and it is the shape of every addition that follows: a key function. Chapter 8 keyed blobs by tier; this chapter keys them by transcript prefix, with a role prefix to tell a rung from a session's history. The store did not change. It trusts a signature, and it does not know or care what the signature is a fingerprint of. A reader who wants a fifth kind of blob, and chapter 12 will want one

for sub-agents on their own engine, writes a key function and a role, and the store, the sweep and the listing take it from there.

Where blanka stops short of plank

Plank's ladder module is the size of blanka's, and the difference is around it.

Plank's rungs have a second selector, by token count rather than by message depth, for a case blanka does not handle: a prompt that diverges behind the live end for a reason other than the cheap pass, where the divergence is known by the token at which it happens rather than by the message it hits. A rung strictly below that token is a prefix of the prompt as well as of the state, and plank's rebuild fallback uses it. Blanka's mock treats every divergence as a rebuild from zero, which is honest about the worst case and blind to that recovery. Plank runs the same rescue inside sub-agents, from the fork snapshot first and the ladder second, and discards the ladder when the reasoning level changes; blanka has neither a level to change nor a sidechain yet.

Plank's sweep is two phases over a metadata graph: age with a surviving-child rule read against the node set as it stood before the sweep, then a byte budget applied in a globally sorted order with rungs first. Its verdicts resolve to paths and are re-checked against the disk before each unlink, because two blobs can carry the same fingerprint and a sibling process can move a sidecar during the window. Blanka's blob sweep is one phase by role and age, with no budget, no lineage and no sidecar, and the transcript sweep from chapter 5 is unchanged. The byte budget matters on a machine where a rung is four hundred megabytes and blanka's mock writes six kilobytes; it arrives with the real engine.

Plank's fork is a first-class branch that keeps the parent's live state intact through a snapshot stack, so that a sub-agent's sidechain can diverge the engine and fold back out without the parent paying to re-read its own context. Blanka's fork at a depth restores a rung where one exists and otherwise rebuilds. The stack is chapter 12's.

And plank's checkpoints, forks and payloads all run against an engine that holds a real model, where a capture is a few tenths of a second and a restore a tenth, against minutes of prefill. Blanka's run against counters. The appendix on a local engine wraps llama.cpp behind the seven methods chapter 8 added, and every test in this chapter runs unchanged against it, because none of them asked the engine anything but whether it extended or rebuilt.

What blanka has is the ladder with its one rule and the test that would catch the rule reversed, an anchor taken at the depth a rewrite will be known by and spaced by tokens, eviction that keeps the shallowest and the newest, a restore that runs after the decision and before the rewrite, a ladder that follows the transcript through compaction, reset, resume, rollback and forks, checkpoints that hold the transcript, the tasks and the state and save what they replace, a session payload keyed on the same fingerprint that makes resume extend instead of re-read, a task list that survives the process, a sweep that knows a rung from history, and a listing that marks what the running session is using.

Steps you can climb back down

The engine holds a state for a prefix and can only extend it, and the cheap compaction pass rewrites behind the live end, so every clearing cost a rebuild from zero until something sat behind the edit. A ladder of at most three snapshots at increasing depths is that something, and it works because the edit point only moves forward and a snapshot only helps when it predates the edit.

A rung is keyed on the fingerprint of the transcript truncated to its own depth, never the whole transcript, and the lookup recomputes that key so it cannot be asked wrongly. It is taken at the index a large result is about to occupy, which is the index a later rewrite will be known by, and it is restored after the pass has decided to rewrite and before it does, so that a refusal never leaves the engine short of state for nothing.

The same two engine methods give the user a checkpoint that holds the transcript, the tasks and the state, a rollback that saves what it replaces, a fork from an earlier point that restores the rung below the cut, and a resume that finds the conversation's own state under the same fingerprint and extends it. The sweep knows that a rung is worth a day and a session's history is worth the session, and it never deletes what the running launch has on its keep list.

CHAPTER 10: WHEN THE MODEL LOOPS

A session plank ran against a request that a hosted agent finished in minutes took two hours. The transcript afterwards showed forty-six tool calls and no edit to the project, and the reader who went looking for the hundred wasted calls found instead that the time was one assistant message. It held 149,000 characters of reasoning inside a single think block, in which three paragraphs, opening with a line about maybe adding a flag to a parser, repeated 263 times until the engine's cap of 50,000 generated tokens stopped them. At the twenty-five tokens per second the local model managed, that is about fifty minutes of a screen showing nothing, followed by an empty reply, because the cap fell inside the reasoning and nothing was ever said after it.

A second session, a week earlier, had shown the other face of the same failure. Eight tool calls, reading the same eight files, were issued in the same order after every turn for two hours: 214 reads and no edit. That one had a cause elsewhere, in the compaction cadence chapter 7 discussed, where an opportunistic rewrite kept turning the files the model had read into stubs so that every turn began from the same state and, at temperature zero, produced the same eight calls. Plank had a guard for repeated tool calls at the time. It watched a window of ten calls for one call appearing three times, and a cycle of eight distinct calls never puts one of them in the window three times.

Those two sessions, and a dozen shorter ones that followed, are the material of this chapter. Each guard plank built was cut by a dump that showed the guard before it was not enough, and the order of the cuts is the argument for the design. Blanka builds the guards in the order the argument runs, and keeps, as plank's findings document does, the cases where a plausible detector was measured against the record and left unshipped.

The pass is a function of the prompt

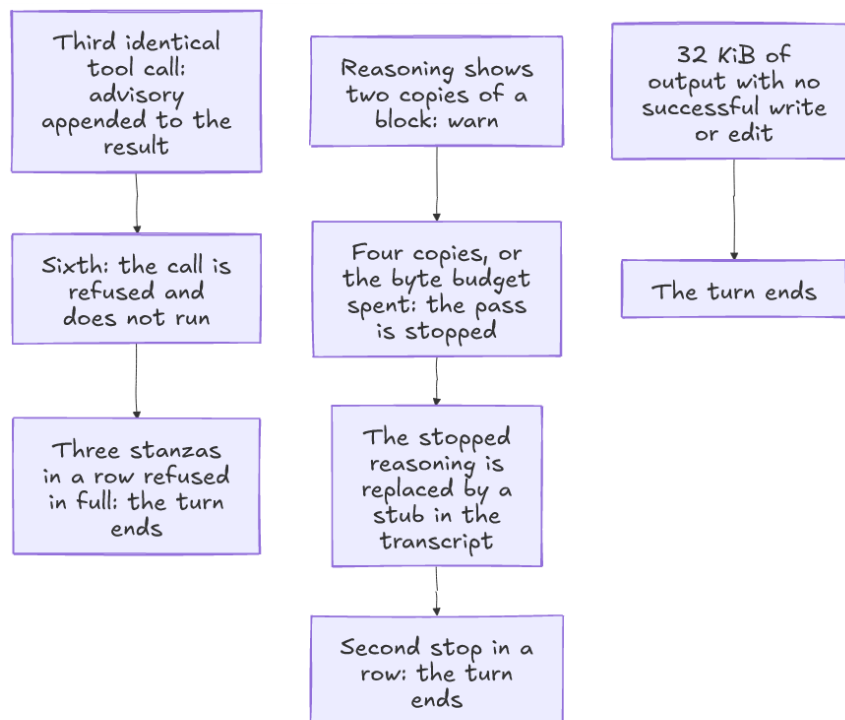
One fact organises everything that follows. At temperature zero the model's output is determined by its input. The same prompt produces the same pass, token for token, for as long as anyone cares to resubmit it. Plank runs its local model greedily by default, and most of the loops it recorded were greedy; sampling at a low temperature changes the letter of the argument and not its force, because a distribution that has collapsed onto a cycle gives the same continuation nearly every time.

The consequence for a guard is severe. Suppose the harness notices that the model has called the same tool with the same arguments six times and refuses the seventh, returning a tool error that says so. The model now sees a prompt that differs from the last one by a single sentence, and by a single digit in the count, and it answers that prompt the way it answered the last one: with the same stanza. Plank's dump of exactly that showed the model re-emitting an identical three-read stanza every pass for six minutes after the refusal, until the user pressed Ctrl-C. Suppose instead the harness stops a pass whose reasoning is cycling and appends an error asking the model not to resume that reasoning. The next pass opens on a context whose last eight kilobytes are the cycle, and a sentence appended after them does not outrank them. Plank's later dumps showed the pass after such a stop reproducing the stopped reasoning character for character, including the point at which it had been cut.

Chapter 10: When the model loops

So a guard that only refuses, or only appends, is a guard that reports a loop and then watches it. Every detector in this chapter ends in a rung that does one of two things: it ends the turn, handing the prompt back to the only party who can change it materially, or it changes the transcript itself, so that the next pass is not the same pass. Both are the harness's decisions, made in the turn loop, and the detectors below only supply the evidence.

The detectors work at three scales, because repetition shows up at three. A tool call can repeat, and the agent loop sees each one as it is about to dispatch it. A pass can repeat inside itself, one paragraph of reasoning after another, and only the renderer sees those bytes as they stream. A turn can go nowhere without either of those, each pass reasonable on its own and the whole changing nothing on disk, and only a counter that lives across the turn can see that. The rungs they escalate through, from an advisory to the end of the turn, look like this:



<https://enzolombardi.net>

Three detectors, one for each scale of repetition, each ending in a rung that ends the turn

The call guard

The identity of a tool call is its name and its arguments. Blanka hashes the arguments as serialised JSON, and because the parser of chapter 2 stores them in a map with sorted keys, two calls that differ only in the order the model wrote the keys hash the same. The guard keeps a window of the last thirty-two dispatched calls and a count per signature, and it answers each observation with one of three things: nothing, an advisory line to append to the call's result, or a refusal.

Chapter 10: When the model loops

```
pub const REPEAT_THRESHOLD: u32 = 3;
pub const BLOCK_THRESHOLD: u32 = 5;
pub const STANZA_TRIP: u32 = 3;
pub const MAX_WINDOW: usize = 32;
```

```
pub enum Nudge {
    None,
    Advise(String),
    Block(String),
}
```

```
pub struct LoopGuard {
    window: VecDeque<Sig>,
    repeats: HashMap<Sig, u32>,
    refused: HashMap<Sig, u32>,
    blocked_stanzas: u32,
    disabled: bool,
}
```

The third identical call gets the advisory, and so do the fourth and fifth: three chances to read the line and do something else. From the sixth the call is refused and never dispatched. Plank added the refusal after a dump in which a stanza was re-issued fifty-two times through every advisory, each result carrying the line asking the model not to call again and the model calling again. The advisory is the polite rung; the refusal is the one that changes what the model can do, because a stuck model that reads the advisory, writes "let me stop" in its reasoning and emits the identical call anyway is physically prevented from running it.

The window is thirty-two calls, not ten, because of the eight-call cycle. A cycle of period eight puts one signature in a ten-call window at most twice. Thirty-two holds two full revolutions of it, and the guard checks for those directly: after every observation it asks whether the last p calls equal the p before them, for every p from two up to half the window, and reports the shortest period that fits. Only a full repeat counts, so a cycle is reported once per revolution, on the call that closes it, and a partial overlap such as a b c a b d is not a cycle.

```
pub fn observe(&mut self, call: &ToolCall) -> Nudge {
    let sig = sig_of(call);
    if let Some(&dispatched) = self.repeats.get(&sig) {
        if dispatched >= BLOCK_THRESHOLD {
            let refused = self.refused.entry(sig).or_insert(0);
            *refused += 1;
            let count = dispatched + *refused;
            return self.answer(Nudge::Block(format!(
                "you have called this tool with these identical arguments {count} times. \
                This call is refused. Do something different: act on what you already know, \
                or tell the user what is missing"
            )));
        }
    }
}
```

Chapter 10: When the model loops

```
}
if self.window.len() >= MAX_WINDOW {
  if let Some(oldest) = self.window.pop_front() {
    if let Some(c) = self.repeats.get_mut(&oldest) {
      *c = c.saturating_sub(1);
      if *c == 0 {
        self.repeats.remove(&oldest);
      }
    }
  }
}
self.window.push_back(sig.clone());
let count = {
  let c = self.repeats.entry(sig).or_insert(0);
  *c += 1;
  *c
};
if count >= REPEAT_THRESHOLD {
  return self.answer(Nudge::Advise(format!(
    "you have called this tool with these identical arguments {count} times. \
    Do not call it again: act on what you already know, or tell the user what is missing"
  )));
}
match self.repeated_period() {
  Some(period) => self.answer(Nudge::Advise(format!(
    "your last {period} tool calls repeat the {period} calls before them, with the \
    same results. You are in a loop. Do not run them again: decide now and act"
  ))),
  None => Nudge::None,
}
}
```

The refusal branch returns before the window is touched, and that detail is a finding rather than a tidiness. Plank's first refusal rung pushed refused calls into the window like any other. Once the window was full, every new identical call aged out one identical old call, the per-signature count went down by one and up by one, and the number the model was shown stopped changing. The dump reads "11 times" on every refusal for six minutes. That made the prompt not nearly identical across passes but exactly identical, and a deterministic model answered it identically. The fix keeps refused calls out of the window, since they did not run and must not push real history out, and gives them a counter of their own that never ages, so the count the model reads keeps rising and every refusal is at least a different sentence. That is still too small a change to break the loop on its own, which is why the same commit added the rung above it.

```
pub fn note_stanza(&mut self, all_blocked: bool) {
  self.blocked_stanzas = if all_blocked { self.blocked_stanzas + 1 } else { 0 };
}

pub fn tripped(&self) -> bool {
```

Chapter 10: When the model loops

```
!self.disabled && self.blocked_stanzas >= STANZA_TRIP
}
```

The agent calls `note_stanza` once per stanza, after observing its calls, with whether every call in it was refused. A stanza with even one call that ran resets the run: a model that is making some progress, re-reading one file it should not while editing another, is never cut off. Three stanzas in a row refused in full is deterministic-loop territory, and there the turn ends. In `blanka` that lives in the agent's dispatch, which replaces the table's plain `dispatch_all` for the guarded path:

```
fn dispatch_guarded(&mut self, calls: &[ToolCall]) -> (String, bool) {
  if calls.is_empty() {
    return (frame_error("empty tool call block"), false);
  }
  let mut out = String::new();
  let mut blocked = 0;
  for (i, call) in calls.iter().enumerate() {
    let nudge = self.guards.calls.observe(call);
    let output = match nudge {
      Nudge::Block(text) => {
        blocked += 1;
        frame_error(&text)
      }
      Nudge::Advise(text) => {
        let (mut output, ok) = self.tools.dispatch_one(call, &mut self.ctx);
        self.guards.progress.progress(&call.name, ok);
        output.push_str(&format!("{}", "[advisory: {}]\n", text));
        output
      }
      Nudge::None => {
        let (output, ok) = self.tools.dispatch_one(call, &mut self.ctx);
        self.guards.progress.progress(&call.name, ok);
        output
      }
    };
    out.push_str(&frame_result(i + 1, &call.name, &output));
  }
  let all_blocked = blocked == calls.len();
  self.guards.calls.note_stanza(all_blocked);
  (out, all_blocked)
}
```

A refused call gets the refusal as its result, inside the same `Tool result N (name) :` frame the model expects, so the reply's shape is unchanged and only its content says no. An advised call runs, and the advisory follows its output. The `ok` flag that `dispatch_one` returns alongside the output is for the third detector, and its meaning is narrower than it looks; the section on progress explains why.

Chapter 10: When the model loops

One legitimate pattern looks exactly like this loop. Polling an asynchronous job, which chapter 11 adds, means calling a status tool with identical arguments until the job is done, and a guard that refuses the sixth poll breaks the feature. Plank exempts the polling path explicitly at the call site rather than teaching the guard about it, and blanka will do the same in chapter 11.

Reading the reasoning as it streams

The first loop, the one with 263 copies of three paragraphs, never issued a tool call the guard above could count. It was one pass, and the only party that saw it as it happened was the renderer of chapter 4, which was classifying those bytes as thinking and, by default, dimming them onto the screen. Plank already had a detector for exactly this shape. It had been written for the insights report, whose recommendation sections would sometimes repeat one clause until their budget ran out, and it watched a one-kilobyte tail of the output for a block repeated four times. It was wired into the report sections and nowhere else. Wiring it into every generation pass was the fix, and doing so uncovered three rules that are easy to break.

The first is to watch reasoning only. Visible output and tool arguments repeat legitimately: a `write` of a table whose rows are identical is, to a byte detector, four cycles of a twelve-byte period, and so is a list of files with a common prefix. The reasoning is where a stuck model goes round, and it is the only channel where repetition has no honest reading. In blanka that means the renderer feeds the guard from the one place that knows which bytes are reasoning, its own thinking state, whether or not the user has asked to see them.

The second is that the window must hold four copies of a paragraph loop. The insights default of one kilobyte was sized for repeated clauses. The observed period was about six hundred bytes, and a one-kilobyte tail can never hold four copies of it; the turn guard uses eight kibibytes, and the interval between checks scales with the window so that the scan, which is quadratic in the tail, stays a bounded fraction of the bytes streamed.

The third is that the stop must travel through a channel the turn loop already understands. Plank fed it through the same path an engine error takes, so the existing "interrupted, but it was a tool error and not the user" handling carried it. Blanka has the interrupt flag of chapter 4, which every engine already polls between chunks, and the renderer pulls it. The engine returns what it had with the interrupted bit set, exactly as if the user had pressed Ctrl-C, and the renderer keeps a note of whose finger it was.

```
pub const REPEAT_WINDOW: usize = 8 * 1024;
pub const REPEAT_WARN_CYCLES: usize = 2;
pub const REPEAT_STOP_CYCLES: usize = 4;
pub const REPEAT_THINK_BUDGET: usize = 16 * 1024;
pub const MIN_PERIOD: usize = 12;
```

```
pub enum RepeatState {
    Quiet,
    Warn { period: usize },
    Stop { period: usize },
}
```

Chapter 10: When the model loops

```
BudgetExceeded,  
}
```

Twelve bytes is the shortest block treated as a cycle; below that, ordinary prose repeats itself for ordinary reasons. Two copies are enough to say something is repeating, and plank shows that as a marker in its status footer; four copies are needed to stop, because two copies of a paragraph is a model restating a point and four is a model that will not stop restating it. The warn level is sticky for the pass: the tail drifts in and out of alignment between checks, and a marker that flickered would read as a bug.

```
fn feed_tail(&mut self, bytes: &[u8]) -> RepeatState {  
    self.tail.extend_from_slice(bytes);  
    self.total += bytes.len();  
    if self.tail.len() > self.window {  
        let cut = self.tail.len() - self.window;  
        self.tail.drain(..cut);  
    }  
    self.since_check += bytes.len();  
    if self.since_check < self.check_every {  
        return RepeatState::Quiet;  
    }  
    self.since_check = 0;  
    if let Some(period) = self.cycle_period(REPEAT_STOP_CYCLES) {  
        self.repeating = true;  
        return RepeatState::Stop { period };  
    }  
    if let Some((period, cycles)) = self.extend_latched() {  
        if cycles >= REPEAT_STOP_CYCLES {  
            self.repeating = true;  
            return RepeatState::Stop { period };  
        }  
    }  
    if let Some(period) = self.cycle_period(REPEAT_WARN_CYCLES) {  
        self.repeating = true;  
        if self.latched.is_none() {  
            self.latched = Some(Latched {  
                block: self.tail[self.tail.len() - period..].to_vec(),  
                end: self.total,  
                cycles: REPEAT_WARN_CYCLES,  
            });  
        }  
        if self.over_budget() {  
            return RepeatState::BudgetExceeded;  
        }  
        return RepeatState::Warn { period };  
    }  
    if self.over_budget() {  
        return RepeatState::BudgetExceeded;  
    }  
}
```

Chapter 10: When the model loops

```
RepeatState::Quiet
}

fn cycle_period(&self, cycles: usize) -> Option<usize> {
    let t = &self.tail;
    let len = t.len();
    (MIN_PERIOD..=len / cycles).find(|&p| {
        let block = &t[len - p..];
        (1..cycles).all(|back| &t[len - p * (back + 1)..len - p * back] == block)
    })
}
```

The cycle check is the obvious one: does the tail end in `cycles` copies of its last `p` bytes, for the smallest `p` that works. Its bound is the line that matters. A period can only be found if that many copies are resident, so the largest period the check can see is the window divided by the number of copies it needs. The `latched` field and the `extend_latched` call in the middle exist because of what that bound did to `plank`, and they get a section of their own.

On the renderer's side the guard is an option the agent installs per pass, and the stop is a value the agent reads back before it takes the reply:

```
pub enum GuardStop {
    Cycle { period: usize },
    Budget { fed: usize },
}

fn thinking(&mut self, text: &str) {
    if self.show_thinking {
        self.sink.thinking(text);
    }
    if self.guard_stop.is_some() {
        return;
    }
    if let Some((guard, interrupt)) = &mut self.guard {
        let stop = match guard.feed(text) {
            RepeatState::Quiet | RepeatState::Warn { .. } => None,
            RepeatState::Stop { period } => Some(GuardStop::Cycle { period }),
            RepeatState::BudgetExceeded => Some(GuardStop::Budget { fed: guard.fed() }),
        };
        if let Some(stop) = stop {
            self.guard_stop = Some(stop);
            interrupt.request();
        }
    }
}
```

Every place the renderer used to send thinking to its sink now goes through this method, including the tail it flushes when the stream ends. A test feeds a renderer a think block

of eight copies of a forty-byte line in seven-byte chunks, the way tokens arrive, and asserts that the stop names a period of forty and that the flag is set; a second test feeds sixty identical visible table rows and asserts that nothing happens, because visible text never reaches the guard.

The stop rung saw a quarter of the window

The cleanest loop on plank's record is the one the guard could not stop. A user reported the footer's looping marker lit for the whole of a pass, and the hand-saved dump showed 46,692 bytes of reasoning with no tool stanza at all: 3,466 bytes of pre-loop thought, then a block of exactly 2,434 bytes, byte-exact, repeated seventeen times. Eighteen occurrences, every delta exactly 2,434, no drift. The block was seven paragraphs, from "maybe the script's append is wrong" through "let us not debug, the script is flawed" and "given the repeated failures we should stop and report" and a fenced excerpt of the script, back to the start, and it locked from the first copy without the stutter phase most loops go through.

The warn rung saw it and the stop rung never could, and the reason is the bound on `cycle_period`. Both rungs searched the same eight-kibibyte window, but the warn rung needed two copies resident and the stop rung four, so their period ceilings were different: 4,096 bytes for the warning, 2,048 for the stop. Any period between a quarter and a half of the window lands in that gap, and the gap is not a latency problem that more cycles eventually close. The stop condition is unreachable there. The footer latched, the model ran to its generation cap, and the only thing that ended the turn was the user.

This also inverted a reassurance plank had written down a few days earlier. Its table of measured dumps showed every real loop stopped at the designed four cycles, some 150 to 800 tokens after the loop began, and concluded that the guard's latency was no longer the problem. Every dump in that table had a period under 2,048 bytes. The table was measuring the loops the guard could see.

The obvious fix is to widen the window, and it is the wrong one. At thirty-two kibibytes a stop still costs four full periods, about ten kilobytes of reasoning for this loop, and the ceiling has moved rather than gone, so the next loop with a longer period reproduces the finding. Plank instead made the stop rung independent of the window. When the warn rung matches a block, the block is kept, with the stream offset at the end of its last verified copy and a count of two. On every later check the guard asks whether enough bytes have arrived for another copy, and if so whether the bytes at that offset are the block. Each match advances the offset and the count; the stop rung fires at four as before. This needs two periods of window rather than four, and caps nothing.

```
fn extend_latched(&mut self) -> Option<(usize, usize)> {
    let mut l = self.latched.take()?;
    let period = l.block.len();
    let win_start = self.total - self.tail.len();
    while self.total >= l.end + period {
        if l.end < win_start {
            return None;
        }
    }
```


Chapter 10: When the model loops

```
let at = l.end - win_start;
if self.tail[at..at + period] != l.block[..] {
  return None;
}
l.end += period;
l.cycles += 1;
}
let out = (period, l.cycles);
self.latched = Some(1);
Some(out)
}
```

Two branches drop the latch, and both are load-bearing. A copy that does not match means the cycle broke, and the guard forgets it; the same check is then free to latch onto whatever is cycling now, so a loop that breaks and is replaced by a different loop is still caught. A copy whose start has scrolled out of the window before it could be checked is treated the same way, because a copy that cannot be verified must not be counted. Counting forward without verifying each copy would turn ordinary prose after a broken cycle into a stop, and blanka pins that with a test that feeds two copies of a nine-kilobyte block, a sentence of something else, one more copy and then twelve kilobytes of distinct prose, and asserts that nothing stops. Replayed against the dump, plank's latched rung fired at 12,925 of the 46,692 bytes, leaving about 8,500 tokens ungenerated.

A cycle longer than the whole window

The next dump closed the gap the latch had left. A parent session ran fifty minutes with no edit, and its own four passes were 435, 1,690, 2,768 and 4,328 bytes of reasoning, none of them cyclic, ending in a forty-eight-byte reply after a tool error that said a sub-agent had been interrupted. By every signal the guard publishes, the model never looped. The loop was in the sub-agent's sidecar: thirteen ordinary passes, then a fourteenth of 189,744 bytes that was one block of 9,042 bytes repeated twenty times, up to the generation cap. The block was not a paragraph but a plan: step back, consider using a sub-agent, notice that this is already a sub-agent, partition thirty-two files into three lettered batches, propose a script, argue that the script breaks the builder structs, and step back.

Both rungs were blind, and it is one inequality. The latch made the stop rung independent of the window, but the warn rung still needs two copies resident to latch onto anything, so a period above half the window is invisible to it, and nothing counts forward from a cycle nothing has latched. The previous finding closed the range between a quarter and a half of the window; this one is a period above a half, and the latch does not touch it.

Two lessons in plank's write-up generalise past the dump. A sidecar loop is invisible in the parent, because the parent's transcript holds the sub-agent's task and its final result and nothing between, so a sidechain that burned 47,000 tokens cycling shows up as one line saying a tool failed; reading the top-level dump and concluding "not a loop" is the failure mode, and chapter 12 will carry this forward. And cycle length scales with the size of the decision, not the length of the prose. Every earlier dump cycled a paragraph,

Chapter 10: When the model loops

from 123 bytes to 2.4 kilobytes. This one cycled a plan, and the request that produces repeated plans, "use sub-agents for the independent parts" over a thirty-two-file refactor, is exactly the kind of request the harness exists for. A detector whose ceiling is any fixed multiple of a fixed window will keep being outrun.

Plank's answer was a rung that recognises nothing. A cycle detector needs two copies before it can name a cycle, so its latency floor is twice the period, eighteen kilobytes here, and unreachable anyway when that exceeds the window. A budget on reasoning bytes per pass has no floor and no ceiling, because it does not look at the bytes; it counts them. It is the only rung whose latency is bounded and the only one that catches a drifting loop, where no two copies are byte-equal. Its price is that it cannot tell a loop from hard thinking, so what it says when it fires has to be weaker than what the cycle rung says.

```
pub fn with_think_budget(mut self, bytes: usize) -> Self {
    self.budget = Some(bytes);
    self
}

pub fn think_budget(ctx_tokens: usize) -> usize {
    (ctx_tokens / 10).max(REPEAT_THINK_BUDGET)
}
```

Plank sized the budget against its dumps, thirty-two of them at the time, holding 939 generation passes, and published the numbers as the argument:

think budget at 7 KiB	89 s	7	27 of 923 (2.93%)
think budget at 16 KiB	204 s	5	1 of 923 (0.11%)
no-progress at 32 KiB	about 7 min	the non-cyclic turn	1 of 575 (0.17%)

The lower budget catches two more loops and trips twenty-six more healthy passes, inverting the ratio from five to one in favour of the guard to nearly four to one against it, and a rung that fires on three percent of good reasoning is a rung the user turns off. The loops worth catching here are enormous, 190, 46 and 45 kilobytes, so latency is the cheap axis and false positives are the expensive one. Plank's document adds that the corpus is thin in its healthy tail and drawn from two projects, so the tenth of a percent is an order of magnitude and not a rate.

Sixteen kibibytes did not survive contact with a design task. Three sessions on one feature request, about forty minutes in all, produced seven budget stops and no cycle. The first pass of each stalled on genuine design forks, the kind of question the user could have answered in seconds, and the recovery passes decided in one sentence as the error asked and then drafted the whole implementation as fenced code inside the think block until the budget cut them mid-function. Sixteen kibibytes is smaller than a multi-file feature written out as code, so a task of that shape could never finish through reasoning. Plank moved the budget to a tenth of the context window, floored at the old figure, which

Chapter 10: When the model loops

is what `think_budget` does above: about 102 kilobytes on a million-token window, six times the reasoning any of those passes had managed. The rung went back to being a backstop against drift rather than a ration, and the cycle rungs and the progress budget still stop the shorter loops first.

Two choices in the wording follow from what the budget knows. Its stop is reported to the user as an over-budget pass and never as a loop, because it is the weaker claim, and sending whoever reads the dump looking for a cycle that is not there is how the next finding gets misdiagnosed. And the text fed back to the model tells it to close the thinking and deliver one small result rather than accusing it of repeating itself, which would be a false statement it then has to reconcile.

```
pub const REPEAT_LOOP_ERROR: &str = "generation stopped: the reasoning was
repeating the same text \
over and over. Do not resume that reasoning. Decide now and act: emit the tool calls for
the change \
you already planned, or answer the user.";

pub const THINK_BUDGET_ERROR: &str = "generation stopped: the reasoning exhausted
its budget; this \
does not establish a loop. Do not restart it or restate the options. Close the thinking and
deliver \
one small result now: the next edit, a supported finding, or a concise question if you
cannot proceed.";
```

What the model sees after a stop

Stopping a pass is half of the job. Plank's first dumps after the reasoning guard went live were pairs: the guard stopped a pass, the error went back as a tool result, and the next pass opened with "need to stop repeating, we have enough, need to implement" and then produced twenty kilobytes of reasoning that ended in the same shape of loop. The session file showed a third pass starting with "we need to stop repeating, let us implement now" and ending after 697 bytes, because the user quit. Nothing was edited.

The main turn had learned nothing across passes. A stop was counted once, reported once, and forgotten, so the turn would loop, stop, loop, stop until the user gave up. Plank's sub-agent path had already met this and answered it with a cap on stops in a row, and the main turn got the same: a second stop in a row ends the turn, leaving the guard's error as the last message so that the next prompt, whatever the user types, sees why the previous one ended. The user, and not a retry, is what changes the prompt.

Then a later pair of dumps, fifty-seven seconds apart in one session, sharpened the lesson. The cap worked: trip one stopped a pass and fed the error back, trip two stopped the next pass and ended the turn, with no third pass and no Ctrl-C. But the pass after the error had reproduced the stopped reasoning character for character, including the point where it truncated. The instruction not to resume that reasoning was in the prompt, and it lost to the eight kilobytes of repetition sitting in front of it. It is not that a one-line error is too weak a nudge. It is that the model is being asked to continue a context whose tail

Chapter 10: When the model loops

is the degenerate cycle, and "let me look at the docs for the SSH section, let me also look at the docs for the SSH section" has exactly one likely continuation.

The fix is to change the transcript, not the wording. After the dump is written, so that the evidence keeps the reasoning verbatim, the guard rewrites the final think block of the message it stopped down to a stub. Anything the pass emitted after leaving the block is visible output the user has already been shown and is kept, so a partial answer is not withdrawn from the model's own context.

```
pub const STOPPED_REASONING_STUB: &str = "[Previous reasoning was cut short for
looping \
and has been removed. Do not reconstruct it.>";

pub fn stub_last_reasoning(text: &mut String) {
    const OPEN: &str = "<think>";
    const CLOSE: &str = "</think>";
    let close = text.rfind(CLOSE);
    let open = match close {
        Some(c) => text[..c].rfind(OPEN),
        None => text.rfind(OPEN),
    };
    let (head, tail) = match (open, close) {
        (Some(o), Some(c)) if o < c => (&text[..o], &text[c + CLOSE.len()..]),
        (None, Some(c)) => (&text[..0], &text[c + CLOSE.len()..]),
        (Some(o), None) => (&text[..o], &text[text.len()..]),
        _ => return,
    };
    *text = format!("{head}{OPEN}{STOPPED_REASONING_STUB}{CLOSE}{tail}");
}
```

Three shapes are handled. A block that was closed, in which case its content goes and the text either side stays. A block that was opened and never closed, which is the usual shape for a stopped pass, since the guard cut the model inside its reasoning; the stub closes it. And a block with no opening tag, because the engine opened the reasoning implicitly and only the close is in the text. A reply with no reasoning at all is left alone, since there is nothing to excise and no gain in guessing.

The cost of this rewrite against the engine's state of chapter 8 turned out to be nothing, and it is worth saying why, because the obvious reading is that this is chapter 7's mid-transcript rewrite and has to pay chapter 9's rung restore. It does not. The rewritten message is the transcript's last, so every earlier byte still matches the engine's live state, the common prefix is the whole transcript up to that message, and the recovery pass prefills the stub and the error and nothing else. No rung is invalidated either: every rung sits at a shallower depth and still describes an intact prefix. A tail rewrite is cheap by construction; the expensive rewrites are the ones with transcript behind them.

What the rewrite trades is the model's ability to see what looped. The error still says that the reasoning was stopped and why, so the model is not left guessing at the reason, only

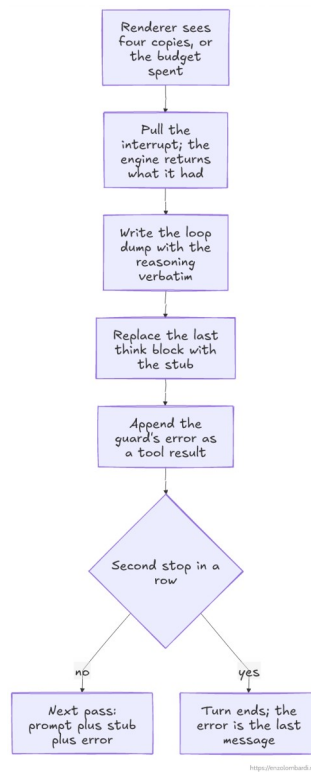
Chapter 10: When the model loops

at the text, which is the text the harness is trying to keep it from rebuilding. Together the pieces make the branch of the turn loop that a guard stop takes:

```
if let Some(stop) = guard_stop {
    self.interrupt.clear();
    report.guard_stops += 1;
    self.guards.repeat_trips += 1;
    let (note, error) = match stop {
        GuardStop::Cycle { period } => {
            (format!("reasoning cycled with a period of {period} bytes"),
REPEAT_LOOP_ERROR)
        }
        GuardStop::Budget { fed } => {
            (format!("reasoning spent its budget at {fed} bytes; no cycle proven"),
THINK_BUDGET_ERROR)
        }
    };
    sink.error(&format!("[guard: {note}]"));
    self.loop_dump(&note, sink);
    if let Some(last) = self.transcript.last_mut() {
        stub_last_reasoning(&mut last.content);
    }
    self.transcript.push(Role::Tool, frame_error(error));
    if self.guards.repeat_trips >= MAIN_REPEAT_TRIP_CAP {
        sink.error(&format!("[guard: {}]", REPEAT_TRIPS_NOTICE));
        report.tripped = Some(REPEAT_TRIPS_NOTICE.to_string());
        break;
    }
    continue;
}
self.guards.repeat_trips = 0;
```

The order inside the branch is the order the findings arrived in. The interrupt is cleared first, because the flag the renderer pulled is the same flag Ctrl-C sets and the next pass must start clean. The dump is written before the stub, so the file holds the cycle and the transcript does not. The error goes in after the stub, as the last message. And the trip count resets on the line after the branch, which only runs when a pass completed: two stops with a completed pass between them are two separate incidents, not a trip. The path from a stopped pass to the next prompt looks like this:

Chapter 10: When the model loops



After a stop: evidence first, then the transcript changes, then the cap decides who speaks next

The test that pins the branch scripts two cycling passes and a third reply that must never be reached. It asserts two stops, two passes, a tripped turn that is not reported as a user interrupt, five messages of which both assistant messages are exactly the stub inside a think block, the guard's error as the last message, two dump files whose names begin with the loop prefix and whose contents still hold the cycle, and an error line on the sink naming the forty-byte period. A companion test puts a completed tool pass between two cycling passes and asserts that the turn is not tripped.

Progress is a changed file

The third detector exists for a turn no per-pass check can see. In the fifty-minute parent session of the sub-agent loop, the parent's own passes were short, distinct and reasonable; the turn as a whole edited nothing. Plank's answer was a budget on bytes generated in one turn without a call to a tool that changes the world, initially the set plan mode blocks: write, edit, bash and the job stop.

That set was wrong, and the dump that showed it ran five hours and seven minutes to 635,000 transcript tokens, with 153 shell calls, 305 edits and 201 reads. Late in the turn the model read the same three lines, containing a doubled call on a bounds method, over and over, and reasoned verbatim about why its own mechanical replacement was correct. The call guard eventually refused the identical reads and ended the turn, hours after the work had stalled. The progress budget existed and never fired, because it reset on the parsed call name: every edit counted as progress, and many of those edits had returned an anchor error; every shell call counted, and the shell calls were builds,

Chapter 10: When the model loops

searches and other read-only commands. Invocation and exit status do not prove that a task advanced.

Blanka resets the budget only on evidence that a file changed, and the evidence is the tool's own success value:

```
pub const NO_PROGRESS_BYTE_BUDGET: usize = 32 * 1024;

pub struct ProgressBudget {
    bytes: usize,
}

impl ProgressBudget {
    pub fn output(&mut self, n: usize) -> bool {
        self.bytes += n;
        self.bytes > NO_PROGRESS_BYTE_BUDGET
    }

    pub fn progress(&mut self, tool: &str, ok: bool) {
        if ok && matches!(tool, "write" | "edit") {
            self.bytes = 0;
        }
    }
}
```

The `ok` that `dispatch_one` returns is whether the tool's `call` returned `Ok`, and for `write` and `edit` of chapter 3 that is only true after the file was written. A failed edit, whose anchor was not found, is not progress. A shell command is deliberately not progress either, whatever it returned: detecting an arbitrary shell mutation reliably would need a witness on the workspace itself, and an exit status is not one. The tests pin a failed edit, a successful read-only shell call and a read against a full budget and assert that none of them resets it, and a successful write against the same budget and assert that it does.

The agent counts every pass's reply into the budget and checks it after the stanza's result is in the transcript, alongside the call guard's trip, so that the dump and the next prompt both show what the last round did:

```
if all_blocked && self.guards.calls.tripped() {
    sink.error(&format!("[guard: {}]", LOOP_TRIPPED_NOTICE));
    self.loop_dump("three stanzas in a row refused in full", sink);
    report.tripped = Some(LOOP_TRIPPED_NOTICE.to_string());
    break;
}

if self.guards.enabled && self.guards.progress.exceeded() {
    sink.error(&format!("[guard: {}]", NO_PROGRESS_NOTICE));
    self.loop_dump("output budget spent with no write or edit", sink);
    report.tripped = Some(NO_PROGRESS_NOTICE.to_string());
    break;
}
```


Chapter 10: When the model loops

The notice for this rung names the absence rather than a loop: the model generated thirty-two kibibytes without writing or editing a file, nothing was changed, narrow the request or name the file to start with. The test for it scripts twelve passes of four kilobytes of prose each ending in a distinct, harmless tool call, so that the call guard has nothing to say, and asserts that the ninth pass ends the turn with that notice.

A witness on the working tree

The words of that notice turned out to matter as much as the threshold. Plank's first version said the turn had generated its budget "without writing a file, editing one, or running a command", and a code-review session ended with that sentence right after the model had run the test suite and the linter, both successfully. The stop was correct, since the session had changed nothing, but the third clause was false, and a notice that miscounts the evidence reads as a malfunction to a user who has watched the command run. Plank reworded it to name what counts: a guard's message is a claim about the session, and the user checks it against what they saw.

The harder problem was the one blanka's test pins on purpose. A turn doing real work through the shell, `sed`, a formatter, a code generator, scored zero progress and could trip the budget for doing what it was asked. An exit status is still not a witness, so plank built one on the workspace itself. Before and after every opaque tool, the shell family, the code runner and the tools of external servers, it fingerprints the git working tree: for each dirty entry, the path, its status bits and the file's size and modification time. A difference between the two fingerprints marks the round as having touched the tree and resets the budget. Write and edit are left out because they already report themselves precisely, and a command that succeeds and changes nothing still resets nothing, which is the property the original rule protected. Outside a repository the fingerprint is absent on both sides and the budget falls back to write and edit alone.

The witness shipped blind to most of what it was meant to see. Git reports an untracked directory as one entry unless asked to recurse, so for a new `src/` the stat that entered the hash was the directory's, and a directory's size and time do not move when a file inside it is rewritten in place. A repository with nothing committed has every directory collapsed that way, which is the state a new project stays in for its first hour. The module's documentation had called the setting a deliberate limit, on the argument that a new file shows up as a new entry: sound for adding a file, wrong for changing one, which is most of what an agent does. A limit argued for in prose is not a limit that has been measured. Recursing into untracked directories costs nothing measurable, and a regression test on an empty repository pins it.

Even with a working witness, plank moved this rung behind its own setting, off by default and subordinate to the master switch. The budget measures volume, and a long read-only investigation, reading a codebase to answer a question about it, is a legitimate turn it stops anyway. The cycle rungs have evidence that a pass is going round; the progress budget has only the absence of a change, and plank decided that absence alone should not end a turn unless the user asked for it.

Quoted, not requested

A smaller finding of plank's belongs here because it produced a loop with no repetition in it. The reference implementation forbade tool-call markup inside the think block, and the first port of that rule rejected a reply as soon as the opening marker appeared inside reasoning. The model then quoted the marker while reasoning about a call, emitted a correct call after the reasoning, was told it had done something it had not, and rewrote correct markup in a loop until the user stopped it. The fix evaluated the prohibition at the stop token, over the finished reply, where it could see that the quoted marker was followed by a real call outside the block.

Blanka's parser of chapter 2 had the same exposure in a different form. It scans the whole reply for opening tags, and a stanza the model mentioned inside its reasoning without a close would parse as an incomplete call, with the frame error for that fed back and the model, having done nothing wrong, asked to do it differently. The agent now parses the reply with its reasoning removed:

```
pub fn strip_thinking(text: &str) -> String {
    const OPEN_T: &str = "<think>";
    const CLOSE_T: &str = "</think>";
    let mut out = String::with_capacity(text.len());
    let mut rest = text;
    loop {
        let open = rest.find(OPEN_T);
        let close = rest.find(CLOSE_T);
        match (open, close) {
            (Some(o), Some(c)) if c < o => rest = &rest[c + CLOSE_T.len()..],
            (None, Some(c)) => rest = &rest[c + CLOSE_T.len()..],
            (Some(o), Some(c)) => {
                out.push_str(&rest[..o]);
                rest = &rest[c + CLOSE_T.len()..];
            }
            (Some(o), None) => {
                out.push_str(&rest[..o]);
                return out;
            }
            (None, None) => {
                out.push_str(rest);
                return out;
            }
        }
    }
}
```

The same three shapes as the stub: closed blocks, a close with no open because the engine opened the reasoning implicitly, and an open that never closes because the pass was cut inside it, in which case everything after it is reasoning. The parser itself is unchanged and stays strict; what changed is what it is shown. The test hands it a reply that quotes the opening tag inside a closed think block and then answers, and asserts that the

Chapter 10: When the model loops

stripped text parses as no call while the raw text would have parsed as an incomplete one.

One dump per rung

Every rung in plank's timeline shipped with its own evidence channel, because the stall before it had been undiagnosable from what was on disk. The first stopped passes left no trace unless the user typed the dump command; the fix was an automatic dump whenever a guard fires, named so that the name itself is evidence. A file whose name begins with the loop prefix was written by a guard. A plain dump beside a stall, saved by hand, says that no guard fired, which is what the sub-agent case showed.

```
pub fn dump_loop(view: &ReproView<'_, dir: &Path, now_secs: u64) ->
std::io::Result<PathBuf> {
    dump_kind(view, dir, "repro-loop", now_secs)
}
```

The agent writes it through a method that takes the note the guard composed, the period in bytes or the count at which the budget was spent, and announces the path on the sink, so a user who sees a red guard line also sees where the evidence went. It writes to a directory the agent holds as a field rather than one it derives at the moment of writing. That distinction is another of plank's findings: its first automatic dumps resolved their directory from the home environment inside the dump function, and a unit test that drove a scripted loop through the agent wrote a fake loop dump, four hundred copies of a forty-five-byte line, into the real dump folder every time the suite ran. Two of those sat among the real dumps until someone noticed the transcript said "do the task". The folder could not be trusted without checking each header, and the four-hundred figure was meaningless for latency because the scripted engine delivered the whole reply at once. Resolving the directory once at construction, and letting tests point it at a scratch folder, closed both.

Blanka's main sets the field to a directory beside the sessions when a store opened, and leaves it unset otherwise, in which case a guard fires without writing. The tests use a temporary directory per test and read the files back to check that the cycle is in them.

The switch that keeps counting

Diagnosing the guards themselves needs a way to turn them off, because every rung fires before the behaviour under study can be observed. Plank's switch is a setting and a command, and it is read at every check rather than captured when a turn or a guard begins, so that typing it mid-turn lands on the generation already streaming. Blanka keeps the switch on the bundle of detectors the agent owns:

```
pub struct Guards {
    pub enabled: bool,
    pub calls: LoopGuard,
    pub progress: ProgressBudget,
    pub repeat_trips: usize,
}
```

Chapter 10: When the model loops

```
impl Guards {
  pub fn begin_turn(&mut self) {
    self.calls = LoopGuard::new();
    self.calls.set_enabled(self.enabled);
    self.progress = ProgressBudget::new();
    self.repeat_trips = 0;
  }

  pub fn repeat_guard(&self, ctx_tokens: usize) -> RepeatGuard {
    let mut g = RepeatGuard::new().with_think_budget(think_budget(ctx_tokens));
    g.set_enabled(self.enabled);
    g
  }
}
```

The call guard and the progress budget start over each turn; the reasoning guard is built per pass, since it watches one pass; the switch keeps its position across all of them. And a silenced guard keeps feeding. Every `feed` and every `observe` does its bookkeeping whether or not the switch is on, and only the answer is suppressed. The reason is the same determinism argument from the other side: a switch that stopped the bookkeeping would hand a looping model a fresh allowance every time it was flicked, and switching the guards back on would be a way to make a loop look healthy. The tests for both detectors drive them past the trip point while disabled, assert silence, enable them, and assert that the very next observation answers from the history they kept. The command is `/loopguard`, with `on` and `off`, and its `off` message says as much.

Plank later found a second reason to lower the guards. Its `/init` command, which chapter 12 describes, runs a canned prompt whose phases re-read the same tree by design, the shape the call guard refuses. The fix is a scope, not a flag: for that one turn every rung is silenced by an object whose `drop` restores the previous state, so a failed turn cannot leave the protection off unnoticed, and the scope ranks below an explicit `/loopguard`, so a user's word outranks a canned prompt. A script sees the guards too: a headless one-shot run whose turn a guard stopped exits with status 3, distinct from 1 for a failure and 124 for a `timeout` wrapper.

Measured, with the biases stated

Plank's numbers were computed against its own dump corpus, and its findings document is careful about what that corpus can and cannot say. The loop half of it is guard-selected. A dump is written when a guard trips or when a user sat through a stall and saved one by hand, so loops that were shorter, subtler or silently absorbed produced no dump, and "loops caught: five" is recall over loops that were already catchable. The healthy half is ordinary sessions and not guard-selected, so the false-positive rates stand on firmer ground than the catch counts. The table of stopped loops that once showed every real dump stopping at four cycles was, as the latching section said, a table of the loops the guard could see.

Two detectors were measured and left unshipped, and both negative results are kept so that nobody re-derives them. The first is a fuzzy detector for the stutter that precedes

most exact cycles: before the byte-exact loop forms, the model emits a run of lines sharing a prefix and differing in the tail, twenty-five to twenty-nine of them in the two dumps that named the pattern, and the exact guard fires only once the variation dies out. A detector defined as a normalised line of at least forty bytes seen three times in the window would have saved between half a kilobyte and 2.6 kilobytes per real stop, and it fired on 52 of 664 ordinary passes in the other dumps, on repeated parameter lines and repeated read-plan lines. Any drift detector, the document concludes, has to be think-only, line-normalised, and require several distinct lines each recurring, and it still needs a false-positive pass over the non-loop dumps before it ships.

The second is a detector on intent. In one late session the stall was visible eight messages before the exact cycle, at the level of what the reasoning said it would do: "let me check the docs for the SSH section" recurred across four passes, three of them after the model had already read that section, and "let me verify the build commands work" was announced in three consecutive passes without a single shell call. An intent restated and never executed looks like a cheaper trip-wire than character repetition. Scored over thirty-nine dumps, a sentence recurring in every pass of a three-pass window fired on four of thirteen loop dumps and three of twenty-six ordinary ones; restricting it to intent phrasings made it worse rather than better. A rung that fires as often on good turns as on bad ones is not a rung, and it also bought almost no latency, since the first hit in that session's own dump came three passes before the exact guard stopped it anyway. The half that would discriminate, whether the announced action was ever executed, needs the announcement matched against the calls, which is semantic matching and not a guard rung.

A third candidate was raised by readers of plank's account and remains unmeasured. Greedy decoding over local weights means the sampled token's probability is available at every step, and a collapsing entropy might see the 9,042-byte cycle before any byte detector could. It might. The engine's FFI returns only the token id, so exposing the probability is a change; the corpus is text, so scoring the rung means replaying every dump through the engine and recording a trace per token; the oldest dumps were sampled at a temperature above zero, so replays must be pinned to each dump's own settings; and healthy code and tool-call generation is low-entropy, so the same passes that fooled the fuzzy detector would look loop-like to a confidence trace. Plank's position is that the rung is not mentioned in its account until the numbers exist, and blanka takes the same one.

The prompt's side

Guards are the harness's half. The other half is what the model is told, and plank's working-style rules, which chapter 6 put into the static prompt, grew alongside the guards. "Decide, do not deliberate" says that when two designs are both plausible the model should pick the smaller one, say so in one line and proceed, because the user can redirect a decision they can see and cannot use one that was never made. "Keep reasoning short and forward-moving" says that each thought must add a fact or a decision, and that noticing oneself restating an earlier thought is the cue to stop thinking and emit the calls already planned. "Narrate progress outside your thinking" asks for a line after the reasoning and before each stanza, because reasoning is hidden by default

Chapter 10: When the model loops

and a long silent think block looks like a stall. Plank also made its low-thinking mode the default, which shortened passes at the cost of a stutter that appears with low thinking too.

The last rule arrived with the last finding in the record. A code-review request ran twelve healthy minutes of reading, and then the synthesis pass wrote the entire review, thirty-one numbered findings in thirty kilobytes, inside its think block and never closed it. Under the old budget it would have been stopped near the seven-minute mark; under the raised one nothing stopped it, and with thinking hidden the user saw a token counter for twelve minutes and pressed Escape. The user typed "resume", and the model, with its own thirty kilobytes of analysis in context, did not read them back: it restarted the synthesis inside the block, produced seventeen kilobytes and fell into a five-line cycle the exact rung caught. Eighteen minutes, no visible output, no review. A second run the same evening failed the same way from a different trigger.

Three conclusions follow, and plank drew them. The budget's size is not the lever: the morning's dumps were a fixed budget cutting off design that was not looping, the evening's was a raised budget letting a deliverable be drafted in reasoning until the user gave up, and moving the number trades one for the other. The narration rule does not reach the synthesis, because it is written around tool rounds and the final pass has no stanza to hang a line on. And a resume restarts rather than resumes, because prior reasoning in context is used as evidence to re-derive from and not as a draft to finish. The rule that followed names the place and the unit, the way the narration rule does: write findings as you find them; when the answer is a list, emit each item to the user as soon as you have it and move to the next; do not accumulate the list in your thinking, because a stop loses all of it. Blanka adds that rule and the one on short reasoning to its rules file, which bumps the prompt version and, as chapter 8 warned, rebuilds the static tier once.

Plank shipped a detector alongside the rule, a draft rung that counts numbered deliverable headings and bytes inside code fences in the reasoning and stops the pass once either passes a threshold, on the argument that a rule only reaches a model that reads it. Blanka leaves that rung out. It is the youngest in the record, its thresholds were set against two evenings of dumps, and its message, close the thinking and emit the items you already have, is the rule restated as an error. The chapter's last section lists it with the other things left out.

The recovery had nowhere to write

The draft rung's own record is the reason for that caution. Within a day the same review request stopped twice on the rung, and the second stop exhausted the trip cap and ended the turn with nothing delivered. Both stops landed a few bytes past the byte gate, so the rung was acting as a reasoning cap rather than a shape detector. And the second pass had obeyed. It opened by saying it would stop the exhaustive analysis and deliver its findings, and then listed fourteen of them, inside the think block it was already in, because every pass on plank's model begins inside a block the chat template opens. The model followed the instruction in the only place it had.

Chapter 10: When the model loops

Two changes followed the same day. The pass after any reasoning stop is generated with thinking off, so its prefix starts past the closing tag and it can write only the answer and the tool calls. Each stop error gained a sentence telling the model that its next reply has no reasoning step. The second change was to the accounting: counting a draft stop against the cap of two had been the design error, since a draft stop carries no evidence of a loop. It now counts on a tally of its own, three, which on a local engine is nearly unreachable once the recovery pass cannot draft. The cost, stated plainly in plank's design note, is that a model looping in visible output after a stop is watched by nothing, since the cycle rungs ignore visible text.

The presentation changed with the accounting. Every other stop removes something, the stubbed reasoning or the rest of the turn; a draft stop keeps the reasoning and is followed at once by the answer, yet it was reported as a red guard line, an accusation about an event the user can do nothing about. Plank now words it as a pause and mirrors it to the debug console. A stop that removes work should be loud and say what it removed; a stop that only redirects where the next bytes go should be quiet and should not accuse.

A cycle that rennumbers itself

A hand-saved dump from the same week showed a loop every rung missed for sixteen minutes: a numbered list of twenty-six items cycling at over seven kilobytes per copy. The copies were not byte-equal, because the ordinals kept climbing, and the period was larger than the byte window anyway. The draft rung saw no headings, because plain numbered items are the shape it excludes.

Plank closed it by hashing rather than widening. Each numbered line is hashed with its leading ordinal stripped, into a history of 256 items bounded by count rather than bytes, so the detectable period no longer depends on the window, and four copies are reported to the reasoning guard as an ordinary cycle. What keeps it off honest enumeration is a shape requirement that is the part to preserve if the rung is ever touched: a repeated block must hold at least three distinct substantial bodies and total 256 bytes, and any prose or code fence between items clears the history, so a list written once with commentary never accumulates a period. A loop repeats structure, and byte equality is only the cheapest proxy for it.

A smaller fix made the cheapest loops describable. The dump's table of passes reports a cycle through the latched block, and a period short enough to show four copies in one check stopped the pass without latching, so a twenty-byte bullet repeated eleven times appeared as a guard stop with no cycle beside it. The direct hit now latches the block it matched.

The smoke run

The echo engine of chapter 1 answers a prompt it has no script for by echoing the last user message, which makes it a way to hand the model's side of a loop to the guards from the keyboard. A prompt that is an opening think tag followed by four hundred copies of a forty-byte line comes back as a pass whose reasoning cycles. The first pass is stopped with a red line naming a period of forty bytes and a status line naming the dump; the second pass, the same echo of the same prompt, is stopped the same way, and the trip cap ends the turn. The summary line reads two passes, no tool rounds, stopped

Chapter 10: When the model loops

by a guard, and the session's dump folder holds two files whose names begin with the loop prefix.

A prompt that is one glob stanza on a line comes back as that stanza on every pass, because the echo is of the last user message and a tool result is not one. The banner shows the glob running five times, the third to fifth results carrying the advisory, then three refused stanzas, and the turn ends with the notice about three refusals, eight passes in. With the guards switched off and a different pattern, the same shape runs to the tool-round cap of chapter 7 as it did before this chapter, forty passes of the same glob. The switch's off message says that every detector keeps counting.

One more thing the run showed is in the section on limits: a stanza split across two lines of input arrives at the echo as an opening tag with no close, and the incomplete-call error was fed back forty times to the cap, because nothing in this chapter watches that path.

Where blanka stops short of plank

The draft rung, as above, and with it the thinking-off recovery pass and the separate tally; a reader who adds the rung should add the recovery with it, or the rung will end turns whose model obeyed it. The numbered-cycle rung, the direct-hit latch, the working-tree witness and the scoped suspension are not built either, and blanka's progress budget counts only write and edit and stays on by default. Plank's sub-agent trip cap, where a second stop pushes the final-round reminder and a third fails the sidechain rather than retrying, waits for chapter 12 with the sub-agents themselves; so does the rule that a stopped sidechain must keep its partial text, which plank's first implementation discarded, so that the sidecar dump could show what the model had been doing. The status marker at the warn level needs a footer to live in, and the plain REPL clears its status line the moment output streams, so it arrives with the TUI in the appendix; the renderer already exposes `repeating` for it.

Plank's dumps carry a table of passes, one row each with its end time, tokens, reasoning bytes, a latched cycle as period and copies, the draft counts and the stop reason, because reading a dump meant reconstructing those figures by hand. Blanka's dump has the transcript and the guard's note; a passes table is a chapter 14 addition once the dump format settles. A per-dispatch wall-clock deadline on a single tool call, off by default in plank, is a different failure from a looping model and is not built.

And the smoke run's own finding: an incomplete or malformed stanza fed back as an error, pass after pass, is a loop no rung here sees, because the call guard counts dispatched calls and a stanza that did not parse dispatches none. Plank's record has no dump of it, which is one reason it has no rung; the count of consecutive parse failures is the obvious place for one, and it would end the turn the way the others do.

Breaking the cycle honestly

The chapter opened with two hours that were one message and two hours that were eight calls, and the reason both ran so long was the fact that shaped every guard afterwards: at temperature zero a pass is a function of its prompt, so a guard that refuses an action or appends an error leaves the model exactly where it was. Each detector ends in a rung that either ends the turn or changes the transcript.

Chapter 10: When the model loops

The call guard hashes name and arguments, advises on the third identical call, refuses from the sixth, sees a repeated run of calls in a thirty-two-call window, keeps refused calls out of that window so the count keeps rising, and ends the turn after three stanzas refused in full. The reasoning guard is fed only thinking bytes by the renderer, warns at two copies and stops at four, latches the block it matched so that a period longer than a quarter of the window can still be stopped, and carries a byte budget sized from the context window for the loops no cycle detector can see, reporting that stop as a spent budget and not as a loop. A stopped pass is dumped verbatim, then stubbed in the transcript, then answered with an error, and a second stop in a row ends the turn. The progress budget counts a turn's output and resets only on a write or edit that succeeded. The parser reads the reply without its reasoning, so a quoted tag is not a request. Every guard writes a dump named for what fired, into a directory the agent was handed. One switch silences every rung and stops none of the counting.

The measurements behind the thresholds came with their biases: a guard-selected corpus, catch counts that are recall over the catchable, a fuzzy detector that fired on fifty-two healthy passes, an intent detector that fired as often on good turns as bad, and an entropy rung nobody has scored. Chapter 11 turns to the other reason a user sits watching a silent screen: not a model going round, but a harness that cannot hear the keyboard while the model works.

CHAPTER 11: TYPING WHILE IT THINKS

The front end blanka has had since chapter 1 reads a line, runs a turn, prints the reply, and reads the next line. Everything between the first and the third step happens with the keyboard dead. When the model decides that the right next move is a full test run, the user watches a banner say so and then watches nothing for as long as the tests take, and if a thought arrives during that time, that the tests were the wrong ones, or that the file the model is about to touch was renamed yesterday, there is nowhere to put it. The turn has to end first. A harness in that shape is a batch job with a conversational skin, and the model's work, which is the slow part, sets the pace of the conversation instead of the other way round.

The same shape pins the model as well as the user. Plank's shell tool returns a running job when a command outlives its refresh time, with a line telling the model how to look at it again. From there the model has two options and both are bad. It can block in the status tool for another refresh interval, burning wall clock while the engine sits idle, or it can move on and forget the job, in which case the job runs until something sweeps it out. Nothing ever tells the model that job three finished with exit zero unless the model asks. A hosted agent runs a long command in the background, ends its turn, and is woken by the harness when the process exits, with the user free to type in between. That is a harness feature, not a tool feature, and it needs the harness to be listening while nothing is generating.

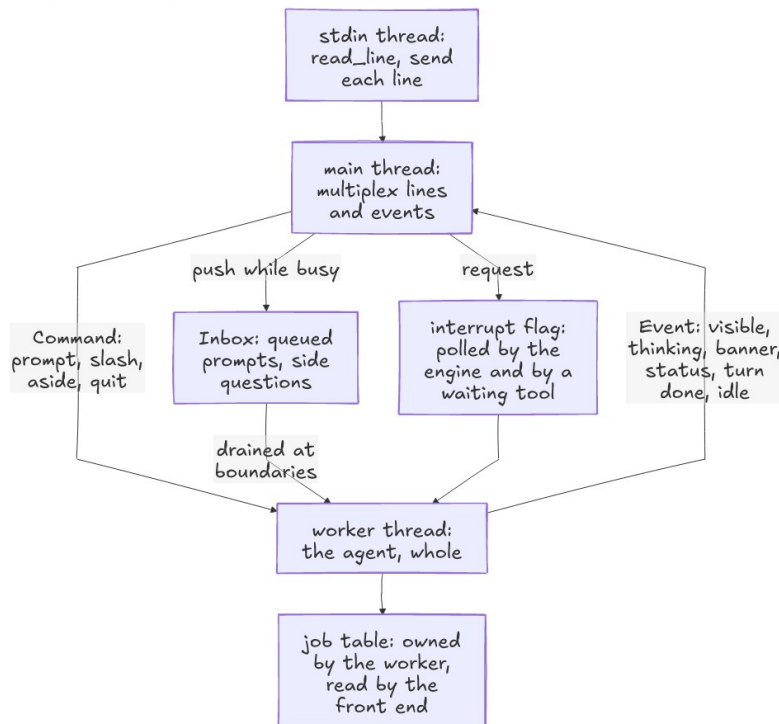
The chapter puts generation on a thread of its own and rebuilds the front end around what that makes possible: a keyboard that works while the model works, lines typed then that join the conversation at the right moment and no other, a background job that wakes the model once and only at a boundary, and a question asked on the side that is answered from the shared context without becoming part of it.

Who owns what

The design decision everything else follows from is what moves and what is shared. The agent moves. The whole of it, engine, transcript, tools and job table, goes onto the worker thread and stays there, and the front end never touches any of it directly again. What is shared is three things, each chosen because it is small and because its access pattern is a queue or a flag: the interrupt flag of chapter 4, an inbox of lines typed while a turn was running, and the job table, which the front end reads to list jobs and the worker owns for everything else. The two sides talk through channels. Commands go in, events come out, and there is no async runtime anywhere, because the standard library's channels and threads are the whole requirement.

Plank made the same split when it ported the C reference's model worker thread, and its reasons transfer. The engine callback is the wrong place to redraw a screen or read a key, because it runs at the engine's cadence and not the terminal's. The transcript has one owner, and every rule in chapters 5 through 9 about what may be appended and when depends on that owner being the only one. And a front end that owns nothing but its screen can be replaced by a different front end, which the appendix on the full-screen display will do, without the agent noticing. The threads and what runs between them:

Chapter 11: Typing while it thinks



<https://enzolombardi.net>

Three threads; the agent owns the transcript, and the front end reaches it only through commands, the inbox and the flag

```
pub enum Command {  
    Prompt(String),  
    Slash(String),  
    Aside(String),  
    Quit,  
}  
  
pub enum Event {  
    Visible(String),  
    Thinking(String),  
    Banner(String),  
    Error(String),  
    Status(String),  
    Text(String),  
    TurnDone(TurnReport),  
    Idle,  
}
```

The event list is the render sink of chapter 4 with three additions: the text a slash command produced, the report a turn ended with, and a message saying the worker has nothing left to do. That last one is not decoration. The front end decides whether the worker is busy from that event and from nothing else, and the smoke run at the end of the chapter shows what happened when it briefly decided otherwise.

Events, not shared state

Chapter 4 separated what the renderer classifies from where the classified text goes, and put the second half behind a trait with five methods. The payoff arrives here. A sink that sends each call over a channel is a dozen lines, and the renderer, the agent and every tool run unchanged on the worker with that sink in place of the terminal one:

```
pub struct ChannelSink(pub Sender<Event>);

impl RenderSink for ChannelSink {
    fn visible(&mut self, text: &str) {
        let _ = self.O.send(Event::Visible(text.to_string()));
    }
    fn thinking(&mut self, text: &str) {
        let _ = self.O.send(Event::Thinking(text.to_string()));
    }
    fn tool_banner(&mut self, text: &str) {
        let _ = self.O.send(Event::Banner(text.to_string()));
    }
    fn error(&mut self, text: &str) {
        let _ = self.O.send(Event::Error(text.to_string()));
    }
    fn status(&mut self, text: &str) {
        let _ = self.O.send(Event::Status(text.to_string()));
    }
}
```

The ignored results are a policy. A receiver that has hung up, because the front end crashed or was closed mid-turn, makes every send fail, and the worker's answer to that is to keep going: the turn completes, the transcript is saved, and the text that nobody was there to see is lost from the screen and not from the session. Plank's channel sink documents the same choice in the same words, and it is the one place where the thread split changes what the user can lose: before it, a front end that died took the turn with it.

On the worker's side the thread is a loop over the command channel with a timeout, and the timeout is where the idle work lives:

```
loop {
    match rx_cmd.recv_timeout(IDLE_TICK) {
        Ok(Command::Quit) | Err(RecvTimeoutError::Disconnected) => break,
        Ok(Command::Slash(line)) => { /* run the command; send Text, Prompt or Aside;
send Idle */ }
        Ok(Command::Aside(q)) => { /* answer it; send Idle */ }
        Ok(Command::Prompt(p)) => {
            run(&mut agent, p, &mut sink, &tx_ev);
            while let Some(next) = agent.inbox.pop_queued() {
                let _ = tx_ev.send(Event::Status(format!("[queued prompt: {next}]")));
                run(&mut agent, next, &mut sink, &tx_ev);
            }
        }
    }
}
```

```
    let _ = tx_ev.send(Event::Idle);
  }
  Err(RecvTimeoutError::Timeout) => {
    match agent.wake_for_jobs(&mut sink) {
      Ok(Some(report)) => { /* send TurnDone, then Idle */ }
      Ok(None) => {}
      Err(e) => { /* send Error, then Idle */ }
    }
  }
}
```

A quarter of a second is the tick. It is the interval at which a job that finished while nothing was generating gets noticed, and it is the same figure plank uses for its plain front end's idle loop, chosen because a human does not perceive a quarter-second delay before a notification and a process does not mind being polled that often.

Why threads and channels

The choice not to use an async runtime deserves a paragraph, because a harness that talks to a network engine, reads a keyboard and watches child processes is the textbook case for one. The argument against it is what the worker actually does. A generation pass is one call into the engine that returns when the pass is over, and inside that call a local engine is running a compute kernel while a hosted one is reading a socket; neither yields anything a scheduler could use, and there is exactly one of them at a time because there is one engine. A tool dispatch is a file read or a child process, again one at a time, because the model's calls are answered in order. What is left is a keyboard and a clock, and a thread reading stdin plus a timeout on a channel receive cover both. An async runtime would put an executor, a set of traits and a colouring of every function between the reader and the same three threads, and would buy no concurrency the design wants, since the design's whole point is that the transcript has one owner and the engine has one caller.

The channels also carry an ordering guarantee the design leans on. Events from the worker arrive in the order they were sent, so a `TurnDone` is never printed before the last chunk of the reply it summarises, and the `Idle` that follows it is never seen before the turn's summary. The front end can therefore keep one boolean for whether the worker is busy and update it from the event stream alone, which is what the smoke run's race was about: the boolean was set from an assumption instead of from the stream, for the length of one thread start-up, and a line fell through the gap.

Plank's worker split has one more property `blanka` inherits without building anything: because the worker owns the agent and the agent owns the transcript, a slash command that reads state, the context report of chapter 6 or the cache listing of chapter 9, runs on the worker and sees the transcript between passes, consistent and complete. The cost is that such a command waits for the turn. Plank's full-screen front end answers a few of them from snapshots the worker publishes, the job list among them, so that a user can see what is running without waiting for the model to stop; `blanka`'s job list reads the shared table for the same reason, and everything else waits.

The stdin thread

The main thread's problem is that a blocked `read_line` cannot be woken. While it waits for the user, nothing can print an event that arrived from the worker, and nothing can notice that a job finished. Plank's plain front end met this when it added job notifications and moved stdin onto a helper thread that sends each line over a channel, with the loop waiting on that channel with a timeout and checking jobs when it fires. End of input and read errors travel over the same channel as lines. Ctrl-C was unaffected, because the interrupt flag had never been tied to the read call; it is a signal handler storing to an atomic, and the engine polls the atomic.

Blanka does the same, and its main loop has one rule that took a smoke run to learn: events first, all of them, before any line is looked at.

```
loop {
  loop {
    match worker.events().try_recv() {
      Ok(Event::Visible(t)) => sink.visible(&t),
      /* ... the other render events ... */
      Ok(Event::TurnDone(report)) => { /* print the summary line */ }
      Ok(Event::Idle) => {
        if let Some(q) = worker.inbox.pop_aside() {
          worker.send(Command::Aside(q));
        } else if let Some(p) = worker.inbox.pop_queued() {
          worker.send(Command::Prompt(p));
        } else {
          busy = false;
          prompted = false;
        }
      }
    }
    Err(_) => break,
  }
}
if eof && !busy {
  break;
}
match rx_in.recv_timeout(Duration::from_millis(20)) {
  Ok(Input::Line(line)) if busy => {
    if let Some(q) = line.strip_prefix("/btw ") {
      if let Some(dropped) = worker.inbox.push_aside(q.trim().to_string()) {
        sink.status(&format!("[btw queue full; dropped: {dropped}]"));
      }
      sink.status("[btw queued]");
    } else {
      worker.inbox.push_queued(line);
      sink.status("[queued for the next boundary]");
    }
  }
  Ok(Input::Line(line)) => { /* send Slash or Prompt; busy = true */ }
```


Chapter 11: Typing while it thinks

```
Ok(Input::Eof) => eof = true,  
Err(RecvTimeoutError::Timeout) => {}  
Err(RecvTimeoutError::Disconnected) => eof = true,  
}  
}
```

A line typed while the worker is busy goes to one of two queues and reaches the agent only through them. A line typed while it is idle becomes a command. The `Idle` handler's first two branches exist because of a race: a line can arrive in the instant between the worker finishing and the front end learning that it finished, and be queued for a boundary that will never come. When the idle event does arrive, the queue is checked and anything in it is sent as the next command, so a line is never stranded because of when it was typed. The `/quit` line sets the end-of-input flag rather than breaking out, so that a turn in progress finishes and a queued prompt runs before the process leaves.

Two join sites

A prompt typed during a turn has to enter the conversation somewhere, and the transcript's rules from chapter 8 say where it cannot: not in the middle, because everything behind the engine's live end is what the cache describes, and not inside a pass, because the model is mid-sentence. The C reference plank ports drains queued lines between tool rounds, as user-role messages appended after the tool result and before the next pass, and that is the one sanctioned place for a message to join mid-turn. The model sees the tool result, then the user's new line, then generates, so the line lands where a user speaking up would have landed if the turn had paused to let them.

Blanka's agent does that in a method called at exactly one point in the loop, after the tool result is pushed and before the next pass:

```
fn drain_boundary<S: RenderSink>(&mut self, sink: &mut S) -> Result<usize,  
AgentError> {  
    while let Some(q) = self.inbox.pop_aside() {  
        if let Err(e) = self.aside(&q, sink) {  
            sink.error(&format!("[btw failed: {e}]"));  
        }  
        if self.interrupt.clear() {  
            let n = self.inbox.clear_asides();  
            if n > 0 {  
                sink.status(&format!("[btw queue cleared: {n}]"));  
            }  
            break;  
        }  
    }  
    let mut joined = 0;  
    while let Some(line) = self.inbox.pop_queued() {  
        sink.status(&format!("[queued prompt joined: {line}]"));  
        self.transcript.push(Role::User, line);  
        joined += 1;  
    }  
}
```

Chapter 11: Typing while it thinks

```
let finished = match self.ctx.jobs.lock() {
  Ok(mut jobs) => jobs.take_finished(),
  Err(_) => Vec::new(),
};
if !finished.is_empty() {
  sink.status(&format!("{}", background job(s) finished)", finished.len()));
  self.transcript.push(Role::User, crate::jobs::notification(&finished));
  joined += 1;
}
Ok(joined)
}
```

Three kinds of thing are waiting at a boundary, and the order is deliberate. Side questions are answered first, because the user asked them and an answer that waits behind a joined prompt and a whole pass is an answer too late. Queued prompts join next. Finished jobs are announced last, after anything the user said, because a notification is an event and a prompt is a person.

The second site is the one that is easy to forget. A turn ends when a pass produces no tool call, and a line typed during that last pass has no boundary left to join. Plank's finding names the consequence of wiring only the first site: its full-screen front end shows queued prompts indented under the status line until they join, and a line that joined through the leftover path at the end of the turn stayed indented forever while its text was already in the transcript. The screen and the session disagreed, with nothing logged. In blanka the leftover path is the worker loop's `while let` over the queue after a turn returns, and each leftover line runs as a turn of its own, in order, announced by a status line so that the screen says what the transcript did. The test scripts two tool rounds and an answer, queues a line before the turn starts, and asserts that it joined mid-turn, in one turn, with the report counting one joined message; then queues another line before a turn that has no tool round and asserts that two turns ran.

What a queued line does not do is interrupt the pass that is running. Other harnesses offer that under the name of steering: a line typed mid-pass stops the pass, joins, and the model starts again with the new instruction in view. Plank built the mechanism, for the side question rather than for prompts, and wrote down its cost: the interrupted pass is regenerated from the last committed boundary, so its partial output on screen is replaced and the redo is fresh sampling. The transcript stays safe because a pass is only committed when it completes. Blanka does not preempt. A line joins at the next boundary, and a user who wants the current pass stopped has Ctrl-C, which chapter 4 made truthful and which the section on interrupts below revisits.

A job the model stopped watching

The shell tool of chapter 3 ran a command and waited for it, with a deadline after which the process was killed. That is one tool over one blocking call, and it made the model's turn as long as the longest command in it. The rewrite is three tools over one job table:

```
pub const DEFAULT_TIMEOUT_SEC: i64 = 120;
pub const DEFAULT_REFRESH_SEC: i64 = 30;
```

Chapter 11: Typing while it thinks

```
impl Tool for Bash {
  fn call(&self, args: &Value, ctx: &mut ToolContext) -> Result<String, ToolError> {
    let command = arg_str(args, "command", "bash");
    let timeout = Duration::from_secs(arg_int(args, "timeout", DEFAULT_TIMEOUT_SEC,
1, 86_400) as u64);
    let refresh = Duration::from_secs(arg_int(args, "refresh", DEFAULT_REFRESH_SEC,
0, 86_400) as u64).min(timeout);
    let mut jobs = ctx.jobs.lock().map_err(|_| ToolError("bash: job table
poisoned".into()))?;
    let id = jobs
      .start(command, &ctx.cwd, &ctx.spill_dir, timeout)
      .map_err(|e| ToolError(format!("bash: spawn failed: {e}")))?;
    Ok(jobs.wait_and_observe(id, refresh, &ctx.interrupt).unwrap_or_default())
  }
}
```

Two durations with two meanings. The timeout is when the command is killed, as before. The refresh is how long the tool waits before handing the command back as a running job, and it defaults to thirty seconds so that a build the model expected to be quick is still reported whole, while a test suite that runs eight minutes comes back as a job after thirty. `bash_status` looks at a job again, waiting up to its own refresh, and `bash_stop` ends one. The model's intent to run something in the background is expressed by a short refresh and moving on, which is what the reference prompt teaches and which plank kept rather than adding an argument that would have changed the pinned tool schema.

A job's output streams into a file under the spill directory, written by two reader threads so that a chatty command cannot fill its pipe and hang, and what the model sees of it depends on when it looks:

```
pub fn observation(&mut self) -> String {
  self.poll();
  let elapsed = self.started.elapsed().as_secs_f64();
  let mut out = String::new();
  if self.running {
    out.push_str(&format!(
      "bash job={ } pid={ } status=running elapsed_sec={elapsed:.1} timeout_sec={:.0}\n",
      self.id, self.pid, self.timeout.as_secs_f64()
    ));
  } else {
    out.push_str(&format!(
      "bash job={ } pid={ } status=done elapsed_sec={elapsed:.1} timed_out={ }
\nexit_status={ }\n",
      self.id, self.pid, u8::from(self.timed_out), self.exit_status
    ));
  }
  /* the output: whole if short and done on first look; else the head on the
  first look and the tail of twenty lines on every later one, with the
```

Chapter 11: Typing while it thinks

```
file's path so the model can read more */
if self.running {
  out.push_str(&format!(
    "\nUse bash_status job={} to look again, or bash_stop job={} to stop it. \
    You may also end your turn: blanka will tell you when the job finishes.\n",
    self.id, self.id
  ));
}
self.observed_once = true;
out
}
```

The first look shows the head, because a header or an early error is what matters first; later looks show the tail, because progress is what matters after. The last sentence is the one that changes the model's behaviour, and plank found that without it the feature is silent: a model that has not been told it will be woken keeps polling, and a status tool called ten times in a row looks exactly like the loop chapter 10's guard exists to stop. The guard exempts `bash_status` for that reason, at the dispatch site rather than in the guard, and the rules file gains a sentence saying the same thing the observation says, so the model hears it before it has started anything.

The notification itself is the observation the model would have got by polling, inside a wrapper that tells it what kind of message this is:

```
pub fn system_reminder(body: &str) -> String {
  format!("<system-reminder>\n{} \n</system-reminder>", body.trim_end())
}

pub fn notification(finished: &[(JobId, String)]) -> String {
  let mut body = String::from(
    "[BACKGROUND JOB NOTIFICATION - NOT USER INPUT]\nA bash job you started
earlier has finished. \
    This is an automated event, not a message from the user. Do not treat it as an
answer to any pending question.\n",
  );
  for (_, obs) in finished {
    body.push('\n');
    body.push_str(obs);
  }
  system_reminder(&body)
}
```

It is a user-role message, like a queued line or the session context of chapter 6, so it persists, compacts and replays like any other. The framing exists because the model's last message may have been a question to the user, and a message that arrives next in the user's slot will be read as the answer unless it says, in capitals, that it is not.

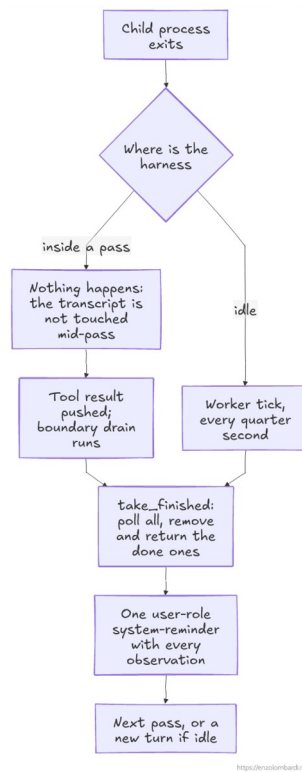
One notification, at a boundary

The rule plank added to its architecture list for this feature is a single sentence: a notification is appended only at a turn boundary and only once per job, and the job table is the sole source of truth for whether a job has been announced. The first half is the transcript rule again. A job can finish at any instant, including in the middle of a pass, and nothing may be appended then; the notification waits for the next boundary, which is either the tool-round boundary the previous section drains or the idle loop. The second half is where the design got simpler than its plan. The plan had a flag per job, set when the model observed the job done and set again when the harness announced it. What shipped has no flag, because the table already removes a job the moment the model observes it as done, so anything still in the table and not running finished unseen:

```
pub fn take_finished(&mut self) -> Vec<(JobId, String)> {
    let mut out = Vec::new();
    let mut i = 0;
    while i < self.jobs.len() {
        self.jobs[i].poll();
        if self.jobs[i].running {
            i += 1;
            continue;
        }
        let mut job = self.jobs.remove(i);
        out.push((job.id, job.observation()));
    }
    out
}
```

Removal is the announcement. A job comes out of this method at most once because it is no longer there to come out again, and a job the model saw finish through any of the three tools never comes out at all, because the observation that showed it done removed it. The tests pin both halves: a short command observed done is not announced afterwards, and a long one reported running is announced once when it finishes and not again on the next call. From a finished process to a model that knows about it:

Chapter 11: Typing while it thinks



A finished job is noticed only at a boundary, announced once, and the table forgets it as it is announced

The idle side is a method the worker calls on every tick that found no command:

```
pub fn wake_for_jobs<S: RenderSink>(&mut self, sink: &mut S) ->
Result<Option<TurnReport>, AgentError> {
    let finished = match self.ctx.jobs.lock() {
        Ok(mut jobs) => jobs.take_finished(),
        Err(_) => Vec::new(),
    };
    if finished.is_empty() {
        return Ok(None);
    }
    sink.status(&format!("[{} background job(s) finished; waking the model]",
finished.len()));
    self.transcript.push(Role::User, crate::jobs::notification(&finished));
    self.continue_turn(sink).map(Some)
}
```

`continue_turn` is the turn loop of chapters 2 through 10 with the prompt push split off, so that a turn can start from a notification the harness wrote as readily as from a line the user typed. Running the turn on the notification, rather than only appending it and waiting for the user, is what makes the feature a wake rather than a note: the model reports on the build it started, and the user, who may have gone for coffee, comes back to a report. The test scripts a pass that starts a job with a zero refresh and answers, then a reply for the wake; it drains the worker to idle, waits for the tick to fire, asserts the

Chapter 11: Typing while it thinks

wake status line and the second reply, and then asserts that nothing further arrives in the next half second, because the job was announced once.

Plank's plan listed guards against the wake, and what shipped is instructive. Draft protection, holding a notification while the user has unsent text in the editor, shipped in the full-screen front end and does not apply to a line front end, which has no draft to protect. A budget on wakes in a row without user input was dropped, because a notification is only ever produced by a job the model itself started, so a model that starts a new job every time it is woken is bounded by the tool-call guard of chapter 10 on the turn that starts it. Interrupts during a woken turn behave exactly as during a typed one. And a sidechain, which chapter 12 introduces, never drains notifications, because they belong to the main transcript.

What a boundary is

The word has carried weight in three chapters now and it is worth stating what it means, because everything this chapter adds hangs on it. A boundary is a point in the turn loop where two things are true at once: the transcript is at a legal append point, meaning the last message is complete and the next thing to happen is a generation pass that will read it, and the engine's state describes exactly the transcript, no more and no less. Chapter 8's tiers are captured at one, chapter 9's rungs are taken at one and its restore happens before a rewrite that begins at one, chapter 10's turn-ending rungs fire at one so that the dump and the next prompt both show the last result, and chapter 7's compaction check runs at the top of every round, which is the same point seen from the other side.

This chapter adds three more things that happen there and nothing that happens anywhere else. A queued prompt joins at a boundary because a user message appended anywhere else would either interrupt a pass or sit behind the engine's live end. A job notification joins at a boundary for the same reason, and the idle loop is a boundary too, the one after the last pass of a turn, which is why a wake can start a turn from it. A side question is answered at a boundary because that is where the transcript is stable enough to be copied, and its snapshot and restore are what keep the second half of the definition true across it: the engine's state after the aside equals the transcript, as it did before.

The order of work at a boundary in blanka's loop is therefore fixed and worth reading as a list once, since it will not be repeated: the tool result is pushed; asides are answered, on a state that is restored after each; queued prompts join; finished jobs are announced; the guards' turn-ending rungs are checked; the pressure reminder of chapter 6 is considered; and the next round begins with the compaction check. A reader adding a feature in a later chapter that needs to touch the transcript mid-turn has one question to answer, which is where in that list it goes, and one rule, which is that it goes in that list and not between two tokens.

Interrupts under concurrency

Chapter 4 made the interrupt a flag the engine polls between chunks and Ctrl-C sets from a signal handler, and it made an interrupted pass truthful: the text produced so far goes into the transcript with a note, and the turn ends. Putting the agent on a thread does not change the flag's shape, and that is the point of having made it a flag. The signal handler runs on whatever thread the operating system chooses, stores to an atomic, and

Chapter 11: Typing while it thinks

the worker's engine, polling that atomic between tokens, stops. No thread has to be signalled, joined or woken. Plank's architecture note records the same fact from the other side: in its full-screen front end the worker's interrupt closure reads a flag the interface thread sets on a key press, and the plain front end's engine reads the signal atomic directly, and the loop cares about neither distinction.

Two things do change. The first is that a tool can now be the thing waiting when the user presses the key. A `bash` call in its refresh wait is not generating, so an engine that polls between tokens is not polling, and chapter 4's contract, that an interrupt stops the current work, would be broken for the length of the refresh. The job table's wait therefore polls the flag too:

```
pub fn wait_and_observe(&mut self, id: JobId, refresh: Duration, interrupt:
&InterruptFlag) -> Option<String> {
    let idx = self.index(id)?;
    let deadline = Instant::now() + refresh;
    loop {
        self.jobs[idx].poll();
        if !self.jobs[idx].running {
            break;
        }
        if interrupt.is_set() {
            self.jobs[idx].terminate();
            break;
        }
        if Instant::now() >= deadline {
            break;
        }
        thread::sleep(Duration::from_millis(20));
    }
    Some(self.observe_at(idx))
}
```

A pending interrupt kills the job rather than waiting the refresh out, and the reason is one plank wrote down when it made the same choice: the shell runs as a separate process, so the terminal's Ctrl-C does not reach it, and a user pressing the key during a long command expects the command to stop, not the harness to stop waiting for it. The tool context carries the turn's flag for this purpose, set at the start of every turn so a tool always waits on the flag of the turn it is running in. Plank goes one step further and puts each job in its own process group so that the kill reaches the command's children; `blanka` kills the child it spawned, which is enough for a shell running one command and not for a shell that started a server in the background, and the section on limits says so.

The second change is that an interrupt now has to know what it is interrupting. During a main pass it ends the turn, as before. During a side question, which the next section builds, it should end the side question and not the turn, because the user asked the question and is now saying they have heard enough, not that the work should stop. Plank's design puts it in one sentence: cancelling an aside flushes the remaining aside queue, with a one-line notice, and the main turn is never aborted by cancelling an aside.

Chapter 11: Typing while it thinks

The boundary drain above does exactly that, clearing the flag after each aside and, if it was set, emptying the queue and reporting how many were dropped before returning to the turn. Queued prompts, by contrast, survive an interrupt. The user typed them for the conversation, and an interrupted turn followed by a queued line is a user who stopped one thing and started the next.

The side question

A user watching a long turn often has a question rather than an instruction. What did that flag mean, which file is it in now, how many tests were there. Answering it from the conversation is easy for the model, and everything about the answer is defined by what it must not do. Plank's design lists four properties and each is a restriction: the exchange is ephemeral, never entering the transcript and so never saved, compacted or seen by a later turn; it is read-only, with no tools, so the model answers only from what it knows; it is one shot, a single pass with no follow-up; and it is non-interfering, leaving the main turn's transcript, cache discipline and tool loop untouched. Two hosted agents have the feature under the same name, and one open-source harness whose implementation plank studied has it too. All three deliver the answer on a channel separate from the conversation and none of them persist it.

The framing the model reads with the question does most of the work of enforcing the restrictions, because the model cannot be prevented from trying to call a tool, only told that none exists and shown that nothing ran:

```
pub fn frame(question: &str) -> String {
  format!(
    "<system-reminder>This is a side question from the user. Answer it directly in a
    single response.\n\
    \n\
    IMPORTANT CONTEXT:\n\
    - You are a separate, lightweight agent spawned to answer this one question\n\
    - The main conversation is NOT interrupted; this exchange will not become part of it\n\
    - You share the conversation context but are a completely separate instance\n\
    - Do NOT reference being interrupted or what you were \"previously doing\"; that
    framing is incorrect\n\
    \n\
    CRITICAL CONSTRAINTS:\n\
    - You have NO tools available: you cannot read files, run commands, search, or take any
    actions\n\
    - This is a one-off response; there will be no follow-up turns\n\
    - You can ONLY provide information based on what you already know from the
    conversation context\n\
    - NEVER promise to take an action; if you do not know the answer, say so\n\
    \n\
    Answer the question with the information you have.</system-reminder>\n\
    \n\
    {question}"
  )
}
```

Chapter 11: Typing while it thinks

The line about not referencing an interruption is there because the model's natural reading of a question arriving mid-task is that it was interrupted, and an answer that opens with "before I was interrupted" is wrong on the facts and confusing to the user. Plank's parity discipline made this text interesting for a different reason: the C reference has no side question at all, so there was no fixture to match, and the framing was validated the only way left, by watching the local model answer side questions correctly from shared context with tools disabled and without derailing the task it was in the middle of.

The answer runs as one pass over the conversation plus the framed question, and the pass writes nothing back:

```
pub fn ask_aside<E: Engine + ?Sized, S: RenderSink>(
    engine: &mut E,
    messages: &[Message],
    question: &str,
    interrupt: &InterruptFlag,
    show_thinking: bool,
    sink: &mut S,
) -> Result<String, EngineError> {
    let mut msgs = messages.to_vec();
    msgs.push(Message { role: Role::User, content: frame(question) });
    let saved = engine.get_kv();
    let mut renderer = StreamRenderer::new(&mut
    *sink).show_thinking(show_thinking);
    let result = engine.generate(Prompt::Structured(&msgs), &mut renderer, interrupt);
    let (reply, _) = renderer.finish();
    if let Some(blob) = saved {
        let _ = engine.set_kv(&blob);
    }
    result?;
    if !matches!(parse_qwen(&strip_thinking(&reply)), ParseOutcome::None) {
        sink.error(TOOLS_DENIED);
    }
    Ok(reply)
}
```

The messages are a copy, so the transcript is untouched by construction and not by discipline; a test asserts it anyway, because the property is the feature. A reply that contains a stanza is not dispatched and not fed back; the user is shown a notice that the answer asked for a tool and nothing ran, and that is the end of it, since there is no follow-up pass for a tool result to go into.

What the cache work bought

The two lines around the generation, taking the engine's state before and putting it back after, are where chapters 8 and 9 pay for themselves, and the reason they are needed is a mistake plank made first.

Chapter 11: Typing while it thinks

Plank's first side question ran on the live engine with nothing around it. The design said the next real turn would cost nothing extra, because the side question was appended after the full transcript, the prefix stayed byte-stable, and the per-turn prefix sync would roll the question and answer back for free. That reasoning was wrong, and chapter 8's constraint says why: the engine reuses its state only for a prompt that extends it. After the aside the live end is transcript plus question plus answer, and the next main pass's prompt is transcript plus the next message, which diverges behind the live end. That is the reset branch, and plank measured it at 13,981 tokens rebuilt where 238 would have done. A second consequence followed a week later. Chapter 9's ladder takes a rung before a large tool result, fingerprinted over the transcript at that depth. In plank's turn loop the aside drain sat between the assistant push and the rung capture, so a rung taken after an aside described an engine state that was strictly beyond the transcript it was signed against. Such a rung claims coverage it does not have, and once restored hands the next prompt a state it cannot extend, so the engine rebuilds from zero, the exact pathology the ladder exists to prevent. Plank fixed it with a one-shot mark, set by the aside drain and consumed by the next anchor point, which skips one rung. An anchor is an optimisation, so skipping one is cheap, and a snapshot per side question was judged not to be.

Blanka takes the snapshot, and the arithmetic is different for it because the snapshot primitive already exists. Chapter 8 gave the engine `get_kv` and `set_kv` for the tiers and chapter 9 used them for rungs and checkpoints; an aside is one more caller. Before the question is prefilled, the engine's state, which is the transcript up to the live end, is captured to memory. The aside then extends that state with the question and its answer, and afterwards the captured state is put back. The next main pass extends a state that is exactly the transcript, no reset, no rebuild, and no rung has been taken on a state the transcript does not describe, because the state never differed from the transcript at any point a rung could see. A test drives the mock engine of chapter 8 through a turn, an aside and a further turn, and asserts that the live state after the aside equals the state before it and that the rebuild counter did not move.

This is the mechanism plank later built for a harder case and documented as its snapshot-and-restore design. The harder case is a side question asked in the middle of a pass rather than at a boundary, where the live end holds a partial reply not yet in the transcript. Plank's engine investigation established that halting the token loop leaves a valid state at the cursor, that the FFI already exposed a snapshot and a restore of a single session's state, and that the aside could therefore freeze the main task, answer on the same session, and restore, with the main task resuming at zero re-prefill. Two mechanisms were compared: a snapshot around the aside on the one live session, and an ephemeral second session sharing the weights. The snapshot won for being lighter, one transient buffer against a second full context, and for restoring the cursor exactly; the second session was kept as the path to a concurrent aside, which on one GPU with one command queue would be time-sliced and not parallel, and so buys little for the side question alone.

One more finding belongs to this design, because it is the kind of thing that only shows up when a turn generates twice into one message. Plank's first mid-pass aside was fast and its second one stalled for a minute with the prefill counter climbing through the whole conversation. The resumed pass had produced a second assistant span in the

engine's token transcript, the partial and then the continuation, while the front end had spliced the two halves into one assistant message. At the next freeze the reconciliation compared one merged message against a span holding only the partial, found them different, and rebuilt from the divergence, and the prompt was now a prefix of the live state rather than an extension of it. The fix merged a continuation into the last assistant span, with the whitespace trimming moved from record time to comparison time so that a partial ending in a newline compared equal to what the front end had concatenated. Blanka is not exposed to this, because it answers asides at boundaries, where no pass is partial, and because it restores rather than resumes; the finding is recorded here because a later engine with a mid-pass aside would meet it on the first day.

For a hosted engine, none of this applies and none of it costs anything. Its `get_kv` returns nothing, its `set_kv` is never called, and the aside is one more request with a prompt cache the provider manages. The restriction that matters for it is the same as for the local engine: nothing is written back.

Where an aside is answered

Three places, and one policy. At idle, a `/btw` line is a command, and the worker answers it at once. During a turn, the line goes into the aside queue and is answered at the next boundary, before queued prompts join, because the person is waiting. After the last pass of a turn, asides that arrived during it are answered before the turn returns, since a question asked is a question owed an answer whether or not a tool round happened to follow it. The queue is first-in first-out with a cap of twenty, and a push past the cap drops the oldest and says so, following the bounded-buffer policy plank adopted from the open-source harness it studied, on the argument that silent growth and silent loss are both worse than one visible line.

```
pub fn push_aside(&self, question: String) -> Option<String> {
    let mut q = self.asides.lock().ok()?;
    q.push_back(question);
    if q.len() > crate::btw::ASIDE_QUEUE_CAP {
        q.pop_front()
    } else {
        None
    }
}
```

What the side question sees is the transcript as it stands at the boundary: every completed pass of the current turn, its tool results included, and not the pass that was running when the question was typed. Plank's design calls this the correct fidelity for a boundary-scheduled answer, and adds a deliberate deviation from a naive drain: at a mid-turn boundary the question is answered against the live transcript as it is, partial task state and all, because the framing has told the model it is a separate instance sharing context, so partial state is expected and harmless.

The agent's own method wraps the call with two status lines, an opening one carrying the question and a closing one saying the exchange is not part of the conversation, so that on a line front end, which has no second panel to draw the answer in, the answer is

at least bracketed. Plank's full-screen front end splits the output area while an answer streams, the conversation keeping the left three fifths and the answer the right two, behind a labelled border, and the panel is ephemeral by construction because its output log is never merged into the main scrollbar. The appendix on the full-screen display will draw that panel; the events it needs already exist, since every render call the aside makes goes through the same channel sink with the status lines marking where the side answer begins and ends.

Work for an idle moment

A worker that owns the agent and a front end that owns the keyboard make a third kind of work possible, besides the turn and the aside: work the user did not ask for, done while nobody is waiting. Plank found two uses for it, and the order in which they arrived is instructive, because the first was built at the wrong moment and had to be moved.

The first is note-taking. Plank keeps a persistent memory of durable facts about the user and the project, two small markdown files loaded into the session context the way chapter 6 loads the instruction file, and a background pass maintains it by reading what the last turn said and proposing additions, rewrites and deletions. The pass is a generation like any other: a snapshot of the engine's state, a prefill of an extraction prompt on top of the live context, one reply with no tools, and a restore, the same sidechain shape a side question has. The first version ran it at the end of the turn, synchronously. In the full-screen front end that meant on the interface thread after the worker had been joined, so nothing repainted and no key was read for the whole generation, and on a local engine that generation is tens of seconds after the answer the user was waiting for had already appeared. A harness whose prompt returns late because it is writing notes about the conversation has put its own housekeeping on the critical path.

The fix split the pass in two, and the split is the pattern worth taking. At the end of a turn, only the capture happens: the span of transcript since the last pass is rendered into the complete extraction prompt and pushed onto a queue on the agent, and nothing is generated. The span is retired as it is captured, so a follow-up turn, a compaction or a cleared session between the capture and the reading cannot lose it, and an interrupted turn queues nothing, leaving its span for the next turn that completes. The reading happens at the next idle moment, one job per idle slot, oldest first, on the worker, behind the same busy loop as a turn, so the screen keeps drawing and the keyboard keeps working.

The rule that makes it tolerable is the one the aside established from the other direction. A prompt typed during the reading interrupts it, the job goes back to the front of the queue uncounted, and the typed line starts its turn at once: the pass must never block the prompt. That makes the pass interruptible by design, which would be expensive if every retry started from zero, and plank's answer uses chapter 8's primitives once more. The pass first prefills its prompt alone, with a generation limit of zero tokens, snapshots that state into the job, and only then samples. A retry restores the snapshot and re-issues the stored prompt byte for byte, so it pays only for the reply. The order is forced by the engine's extend-only rule: a snapshot taken at the interrupt would hold a partial reply, and the retry's prompt would be a prefix of it rather than an extension, which is a rebuild.

Chapter 11: Typing while it thinks

An interrupt that lands during the prefill, the long phase on a local model, keeps a snapshot too, because the engine stops at a token boundary and leaves a valid shorter prefix behind.

The same small correction had to be made for quitting. The busy loop had no arm for Ctrl-D, so during a notes pass the key was dropped silently while the screen looked like an idle prompt. Plank now lets Ctrl-D on an empty line interrupt the pass and quit, as it would at idle, while Ctrl-D during a real turn stays inert, because nobody should lose a generation in flight to a stray key. The plain front end, which cannot cut a generation short on a keystroke, reads one job per quarter-second idle tick, the same tick blanka's worker uses for job wakes, so a line typed meanwhile waits out the job that is running. The headless paths have no idle moment to come back to and drain the queue before the process exits.

Once the pass ran behind the prompt, the question became how much of it to show, and the first cut answered it badly. It changed the window title, printed a start line into the scrollbar, put a verb in the footer, drew a progress line under the output and posted a notice whenever the model returned nothing usable. Housekeeping the user had not asked for was filling the conversation the user had. Plank's settled answer is one mark: a writing hand in the footer per queued span, the running one included, in place of the state word, so two hands mean one more is waiting. A pass that wrote something ends with a single dim line giving its elapsed time and a summary of the change; a pass that found nothing says nothing, and the model's reply is never rendered. The pass's prefill and generation figures are still measured, but they appear only behind a command-line flag that is off by default, because they serve someone tuning the pass and not someone using the agent.

The episode is worth reading for what did not change. Across the releases that moved the pass from the turn end to the idle slot, its cost in tokens stayed exactly where it was: a snapshot, a prefill, a generation and a restore, none of it counted in any turn's statistics. The pass had shipped switched off because of that cost, and it ships switched on now, with the same cost. What moved is when the bill comes due. The argument about the default had treated on or off as the decision, when the decision that mattered was which side of the turn boundary the work sits on. A user who types the moment an answer lands never sees a pass run; a user who reads the answer for a minute gets the notes taken while reading. The idle loop, the worker interrupt and the engine snapshot are all things a harness owns and an external hook cannot reach, which is why the move was possible at all.

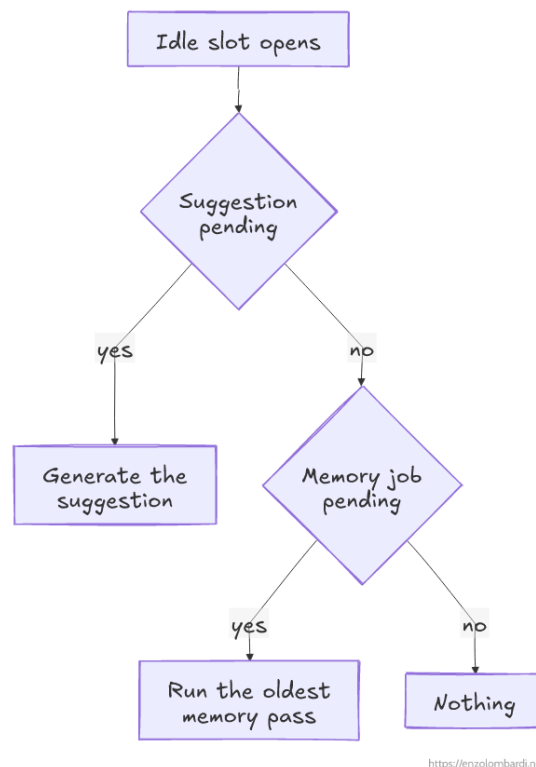
A guess at the next line

The second use arrived later and changed the first. After each turn, plank's full-screen front end shows a guess at the next thing the user will type, as dim ghost text in the empty input line: Tab or the right arrow places it, Enter sends it, any other key dismisses it. The guess is a short generation off the warm session, run as a sidechain like the notes pass, with the same capture-then-read split: the turn end records that a suggestion is wanted, and the idle slot produces it.

Chapter 11: Typing while it thinks

Three properties make it affordable enough to ship on by default. Before generating, plank asks the engine whether the prompt would extend the live state or rebuild it, and skips the suggestion outright if it would rebuild from zero, since no guess is worth a full re-prefill. The pass's sink is forced to a null sink for its duration: moving the generation onto the worker, so the screen would not freeze, had also made it visible, and the guess briefly streamed into the sub-agent pane with a throbber and a throughput readout at an idle prompt. And a suggestion carries the transcript depth it was generated against, and is dropped the moment the transcript has moved, because a guess about a conversation that has since changed is noise. Every path that replaces the session clears it as well, because adopting a different session whose transcript happens to have the same length would otherwise offer a guess about the previous conversation.

The two jobs now compete for one idle slot, and the scheduler between them is a pure function with a strict order:



A pending suggestion always wins the idle slot; a memory job takes the next one

The asymmetry follows from what each loses by waiting. A suggestion that lands after the user has started typing is worthless, while a deferred notes pass loses nothing but time, so the suggestion always goes first. The first version did not trust that order to be safe. It feared that a fast back-and-forth would lock memory out indefinitely, and added a starvation limit, five minutes by default, that handed the slot to a memory job once it had waited that long even with a suggestion pending. The limit cost more than it protected: after a long stretch of work the ghost text could arrive only after a whole memory pass, instead of right after the answer. And the starvation it guarded against cannot happen, because a suggestion is queued at most once per turn and its flag clears when it runs, so the oldest memory job always gets the very next idle moment. Plank

Chapter 11: Typing while it thinks

removed the setting, and the timestamp on each job that existed only to feed it. Because the function is pure, the tests pin every row of that chart without an engine.

The rest of the suggestion code is a sanitizer, and each of its rules is a finding. The model was asked for a line the user would type, and it sometimes answered as itself, so a reply opening in the assistant's voice, a first-person report or a cheerful "sure", is rejected rather than shown. A leading slash is rejected, because Enter submits a placed suggestion immediately and a guessed "/" clear the session" would run a command on one keystroke; a leading exclamation mark is rejected for the same reason, since plank runs a line starting with it as a shell command without confirmation. A reply longer than one editor line is rejected rather than truncated, since half a sentence is a worse hint than none. And a reply whose reasoning never closed is rejected, because on a model that generates inside an implicit think block the absence of the closing tag means the budget ran out mid-thought: one real run produced the start of the model's reasoning about what the user might want, and only the length cap kept it off the input line.

That last rule replaced a fix that was worse than the problem. The obvious way to keep a suggestion out of the reasoning is to generate it with thinking off, and plank did, on the strength of a comment saying the mode change was free. It was free on one model family. On the newer family with a numeric reasoning effort, the effort appears as a line in the assistant prefix and thinking off carries no such line, so switching modes changed the prefix, invalidated the engine's token transcript and forced a full re-prefill of the whole conversation, twice per turn once the mode was switched back. That is the exact cost the probe before the generation existed to avoid. The revert left the mode alone, raised the suggestion's token budget from 40 to 160 so reasoning and a short answer both fit, which costs nothing when unused because generation stops at the end token, and let the sanitizer reject the replies that still ran out. A mode is part of the prompt, and on a local engine anything that changes the prompt behind the live end is a rebuild, however small the change looks from the settings.

Blanka builds neither job. Its worker already has the idle tick where they would run, and chapter 8's snapshot primitives are the mechanism both depend on; what blanka lacks is a memory to maintain and a full-screen input line to ghost into, and the section on limits says so.

Tests that cross a thread

Testing a worker is testing a race unless the test removes the race, and the three worker tests remove it the same way: the inbox is filled before the turn starts, so the timing of the typed line does not depend on the thread scheduler, and every assertion waits for the worker's idle event rather than for a fixed time. The helper that drains events until idle is the whole synchronisation story, and the scripted echo engine makes the rest deterministic: a script of two tool calls and an answer produces exactly two boundaries, and a queued line placed before the turn is guaranteed to join at the first.

The job test is the one that has to wait on the clock, because the property under test is that a job finishing while nothing is generating wakes the model. It scripts a command that sleeps a third of a second with a zero refresh, drains the turn that started it to idle, and then drains again, which blocks until the tick after the child exits produces a wake.

Chapter 11: Typing while it thinks

Its last assertion is a negative one with a timeout: nothing arrives for a further six tenths of a second, which is two ticks, so the job was announced once. A negative assertion on a channel is the closest a test can come to "and then nothing happened", and the timeout has to be longer than the tick it is guarding against and shorter than the reader's patience.

The aside test uses the slash channel as a probe. After the turn with the queued question completes, it sends a command the test's own slash handler answers with the transcript's length, and asserts four: user, the assistant's call, the tool result, the assistant's answer. The side question and its answer were generated on the worker, streamed through the same sink, and left no trace the count could see, which is the property stated at the top of the aside module and the reason the module exists.

The smoke run

The echo engine answers a prompt it has no script for by echoing the last user message, so a stanza typed as a prompt comes back as a call. A `bash` stanza with a one-second sleep and a zero refresh, typed on the first line, ran the command and reported it as job one, still running. Two lines typed during that turn, a `/btw` question and a plain sentence, were acknowledged at once as queued, and then, at the boundary after the tool result, the side question was answered first, bracketed by its two status lines, and the sentence joined as a user message and became the next pass. The summary line reported two passes, one tool round and one joined message. A `/jobs` typed at the next prompt listed job one, running, at a tenth of a second.

Then nothing was typed for two and a half seconds. The job finished during that time, the worker's tick noticed, and a status line announced one finished job and that the model was being woken. The turn that followed had the notification as its prompt, a system reminder wrapped around the job's final observation, exit status zero, six bytes of output, the word "built" in a tail of one line. A `/jobs` after that reported no background jobs, because the announced job had left the table. A `/btw` typed at idle was answered immediately.

An earlier run of the same script found the race described with the main loop: the first line was queued for a boundary because the worker's first idle event had not yet been read when the line arrived, and a queue with no turn to join stays a queue. The fix, dispatching a queued line when the idle event does arrive, is in the loop above, and the run after it shows every line landing.

Where blanka stops short of plank

Plank preempts a running pass for a side question and re-runs the pass afterwards, on the argument that a question asked during a long reply should not wait for the reply to end; blanka answers at boundaries only. Plank's mid-pass suspend, which freezes the token loop, answers on the same session and restores with zero re-prefill, needs an engine whose generation is a loop the harness owns, and blanka's engines hand back a finished pass; the appendix on the llama.cpp engine is where that loop lives, and the snapshot-and-restore around the aside is already the shape that suspend needs.

The split panel, the footer's running-job count and the draft protection that holds a notification while the user is mid-sentence all belong to the full-screen front end and

Chapter 11: Typing while it thinks

wait for its appendix. Plank's headless mode emits a marker on standard error when jobs finish so a driver can tell a woken turn from a reply to its own prompt; blanka has no headless mode yet. Plank kills a job's whole process group; blanka kills the child it spawned. A desktop notification when a job finishes while the terminal is unfocused, and a hook event for the same moment, are chapter 13 material once hooks exist.

Plank's two idle jobs, the memory pass and the prompt suggestion, are not built, and neither is the scheduler between them. Both are sidechains on the warm session, taken and restored with the primitives blanka already has, so the work they need is a memory store for the first and an input line that can show ghost text for the second, which is again the full-screen front end's.

One thing plank has not done either, and its plan records as the condition for turning the notification on by default, is the measurement: count the status-tool calls per session on a corpus of real runs before and after the prompt sentence, and flip the default only if the count falls. Blanka ships the sentence on, because it has no corpus to measure against yet, and notes the debt.

Two things at once, one transcript

The turn moved onto a worker thread, whole, and the front end kept its keyboard. Three things are shared between them, an interrupt flag, an inbox and a job table, and everything else crosses as a command in or an event out, over standard channels with no async runtime. The render sink of chapter 4 became a channel sink in a dozen lines, and a receiver that hangs up loses text from the screen and not from the session.

A line typed during a turn joins at one of two sites and no other: between a tool result and the next pass, as a user message, or after the turn ends, as a turn of its own, and both sites announce the join so the screen and the transcript agree. A line queued for a boundary that never comes is dispatched when the worker turns idle. The shell became three tools over a job table whose removal is its announcement: a job the model saw finish is never announced, a job it stopped watching is announced once, at a boundary, as a system reminder the model is told is not the user, and the model is told in its prompt to end its turn rather than poll. The interrupt flag reaches a waiting tool as well as a generating engine, kills the job rather than the wait, and during a side question cancels the asides and not the turn. The side question runs one pass over the shared context plus a framing that names its restrictions, writes nothing back, and takes the engine's state before and restores it after, so the main task resumes with no rebuild and no rung ever describes a state the transcript does not. Plank learned the need for that snapshot from a 13,981-token rebuild and a rung that claimed coverage it did not have. The idle slot the worker's tick opens carries plank's background work too, a notes pass captured at the turn end and read later, interruptible by the next keystroke, and a guess at the next prompt that yields the slot to memory only when memory has waited too long; both obey the same rule as the aside, that nothing a user did not ask for may stand between the user and the prompt.

Chapter 12 takes the fork that chapter 9 built and the boundary discipline built here and runs a second agent on it: a sub-agent with its own transcript, its own tool loop and its own guards, whose report is the only thing that comes back.

CHAPTER 12: DELEGATION AND ISOLATION

A turn that answers "where is the retry logic" by reading eleven files leaves eleven files in the transcript. The answer is one sentence; the evidence for it is forty kilobytes that the model will carry, and pay for, on every pass until compaction clears it. Chapter 7 made that clearing cheap and chapter 8 made the carrying cheap, but neither made the eleven files worth keeping, and a session of ordinary coding work is mostly this: questions whose answers are small and whose research is large. Plank's rules say it in one line, added after watching sessions fill with exactly this material: your own context should hold the conclusions, not the file dumps.

The instrument for that is a second model run with its own context. Hand it the question, let it read the eleven files in a transcript of its own, take back the sentence. A hosted agent calls this a sub-agent and the reader has probably used one without thinking about what it costs, because on a hosted engine it costs a second conversation and nothing else. On a local engine it costs the whole prompt prefix again unless the harness is careful, and the ways of being careful are the substance of this chapter: a sub-agent that is a fork of the live transcript rather than a fresh session, folded back out when it is done so that only its report remains; a loop for it that is bounded and guarded with a shorter fuse than the main turn; definitions of named sub-agents in files, with the engine they run on and the name of a key variable, never a key; a checkout of the repository the sub-agent can be given so that its edits land nowhere the user is working; and two tools that turn the model's guesses into questions, one that asks the user to choose and one that holds the model read-only until a plan is approved.

Plank built all of this, twice in places, and the second builds are where the lessons are. A sub-agent that published no status looked hung for thirteen minutes. A report that carried the sub-agent's reasoning was judged unreliable by the parent, which redid the work. A worktree removal that trimmed git's output lost the first filename it was meant to protect. Each of those is a paragraph below, and each changed a line of blanka before the line was written.

What comes back is the point

A sub-agent is defined by what it returns, not by what it does. It may read, search, run commands and edit; it has the whole tool table except the one tool that would let it delegate again. What it hands back is a report, and the report is the only thing the parent conversation ever sees. Everything the sub-agent read is discarded with the sub-agent. That discarding is the feature. A delegation that left its research in the parent transcript would be a very expensive way of calling the read tool.

The tool contract of chapter 3 gets in the way here, and the way it gets in the way is instructive. A tool receives the tool context and nothing else: not the transcript, not the engine. That was deliberate, because a tool that could see the conversation could also edit it, and a tool that could see the engine could drive it, and the whole point of the table was that the loop stays the only place where those two meet. A delegation needs both. It must append to the transcript, generate against the engine, and fold the transcript back. So the `agent` tool is described in the table like any other, which keeps the system prompt

Chapter 12: Delegation and isolation

honest about what the model may call, and its `call` method is never reached from the main loop. The loop intercepts the name before dispatch:

```
/// One call. The `agent` tool is the one name the table describes but
/// the loop runs: a delegation needs the engine and the transcript, and
/// the tool contract withholds both on purpose.
fn run_one<S: RenderSink>(&mut self, call: &ToolCall, sink: &mut S) -> (String, bool) {
    if call.name == "agent" {
        return self.delegate(&call.args, sink);
    }
    self.tools.dispatch_one(call, &mut self.ctx)
}
```

The one path that does reach the tool's own `call` is a call made from inside a sub-agent, and that is exactly the nesting the depth cap forbids, so the tool's body is the refusal:

```
/// How deep delegation goes: a turn may delegate, its sub-agent may not.
pub const SUBAGENT_DEPTH_CAP: usize = 1;

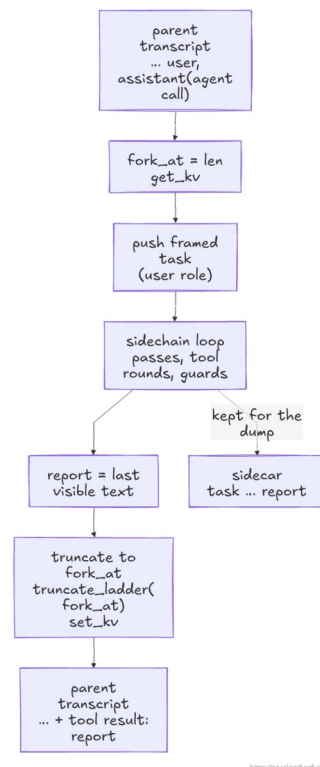
pub const NESTING_REFUSED: &str = "sub-agent nesting limit reached; complete this
work directly";

impl Tool for AgentTool {
    fn call(&self, _args: &Value, _ctx: &mut ToolContext) -> Result<String, ToolError> {
        Err(ToolError(NESTING_REFUSED.into()))
    }
}
```

Plank reached the same shape by the same reasoning, with the tool handled at the agent level rather than in its dispatch table and a depth cap of one (plank, 32560f9). A sub-agent that could delegate would be a sub-agent whose cost was unbounded, and a depth of one is what the rule "a turn may delegate, its sub-agent may not" spells as a number. The refusal text tells the model what to do instead, because a refusal that only says no is a refusal the model retries.

The whole shape, from the parent's call to the report in its transcript:

Chapter 12: Delegation and isolation



A delegation on the parent's engine. The sidechain grows the live transcript, and everything it grew is cut back out; the report crosses as an ordinary tool result and the sidechain's messages survive only in a sidecar.

A fork, not a session

The cheapest sub-agent on a local engine is not a new session. A new session starts with the system prompt and the project context, which chapter 8 taught the engine to restore from a blob, and then has to read the task, which is small. That sounds cheap until the task says "fix the failing test in the module we were discussing", at which point the sub-agent needs the discussion, and the discussion is the parent transcript, and a fresh session that receives it as text pays for every token of it. Plank chose the other design: the sub-agent is a fork of the live transcript. The framed task is appended as a user message, the sidechain runs on the same engine holding the same state, and its first pass extends what the engine already holds by the length of the task. The sub-agent reads the conversation for free because it is the conversation, continued.

The cost of that choice comes at the end. When the sidechain finishes, its messages are truncated out and the report is appended in their place. To the engine that is a divergence: the live end sits past the sidechain's last message, and the parent's next prompt branches away from it at the fork point. Chapter 8 established what a divergence costs, which is the whole prompt from zero, and chapter 9 built the tool that avoids it. Before the fork, the engine's state is captured; after the fold, it is put back. The parent's next pass then extends from the fork point by the length of the report:

```
EngineSpec::Parent => {  
  let fork_at = self.transcript.len();
```


Chapter 12: Delegation and isolation

```
// The state before the fork, put back after it: the next
// parent pass then extends what the engine holds instead of
// rebuilding, and nothing anchored on a sidechain pass can
// outlive the sidechain.
let saved = self.engine.get_kv();
self.transcript.push(Role::User, framed);
let end = run_sidechain(&mut self.engine, &self.system, &mut self.transcript,
&self.tools, &mut self.ctx, &self.interrupt, show, guards_on, sink);
let messages = self.transcript.messages()[fork_at..].to_vec();
self.transcript.truncate(fork_at);
self.truncate_ladder(fork_at);
if let Some(kv) = saved {
    let _ = self.engine.set_kv(&kv);
}
(end, messages, fork_at)
}
```

This is the same pair of engine methods chapter 11 used around a side question, and for the same reason, and the second line of the comment is chapter 9's invariant arriving where that chapter said it would. A rung captured during the sidechain would describe a prefix that includes sidechain messages, and after the fold that prefix does not exist. Plank's rule is that a sidechain never writes the live session's cache, and blanka keeps it by cutting the ladder back to the fork point, which deletes any rung deeper than it. The ladder's truncation method was written for the fork at a depth; the sidechain is the second caller, and it is a fork at a depth that happens to be the transcript's own length.

The test that pins this uses the mock engine from chapter 8, whose one job is to count rebuilds:

```
#[test]
fn the_parent_extends_after_a_delegation_instead_of_rebuilding() {
    let dir = scratch("extend");
    let script = vec![delegation("look around"), "nothing much".to_string(), "then we are
done".to_string()];
    let mut a = Agent::new(MockKvEngine::scripted(script), ToolTable::default_set(),
ToolContext::new(&dir));
    a.run_turn("go", &mut Channels::default()).unwrap();
    // Three passes drove the engine: the delegating pass, the sidechain's
    // pass on the fork, and the parent's continuation. The fork diverged
    // from the parent, and the parent still never rebuilt, because its
    // state was put back before the report was appended.
    assert_eq!(a.engine.rebuilds, 0, "the parent pass after the fork extended what the
engine held");
    assert_eq!(a.engine.live().len(), 5, "system, user, delegating reply, report, answer");
}
```

Remove the capture and the restore and the count goes to one: the parent's continuation diverges from a live end that includes the sidechain, and the mock, like a real engine, rebuilds from the first message. Plank measured the real version of that on its own engine

and recorded the number in its findings for the aside; a sidechain that forks and folds without the snapshot pair pays the whole parent prefix on the pass after every delegation, which on a conversation of a hundred thousand tokens is minutes.

The sidechain is a function

The loop the sub-agent runs is not a method on the agent. It is a function of an engine, a system prompt, a transcript, a tool table, a context, an interrupt and a sink, and it appends to the transcript it is given and returns how it ended:

```
pub fn run_sidechain<S: RenderSink>(
    engine: &mut dyn Engine,
    system: &str,
    transcript: &mut Transcript,
    tools: &ToolTable,
    ctx: &mut ToolContext,
    interrupt: &InterruptFlag,
    show_thinking: bool,
    guards_enabled: bool,
    sink: &mut S,
) -> SidechainEnd;

pub struct SidechainEnd {
    /// The last assistant text with its reasoning removed, when there was
    /// one. `None` is "no report", which the parent reads as a failure.
    pub report: Option<String>,
    /// Set when a guard, the interrupt or the engine ended it early. The
    /// transcript still holds what the model said, for the sidecar dump.
    pub error: Option<String>,
    pub passes: usize,
    pub tool_rounds: usize,
}
```

The shape is chosen for a reason that arrives two sections from here. A fork on the parent's engine passes the parent's engine and the parent's transcript. A sub-agent on a different engine passes that engine and a transcript holding nothing but the task. Both are the same loop, and a loop written as a method on the agent, reaching for `self.engine` and `self.transcript`, would have to be written twice or bent around a swap of the agent's own fields. Plank's version is a method, and plank swaps: its alternate-engine path replaces the agent's engine with the alternate one, stashes the transcript prefix, runs the loop, and puts both back, with a comment saying that a leaked swap "would leave the whole session pointed at the wrong engine, which is the worst failure this design can produce" (plank, 6090a5a). That is a correct implementation of a design that made the swap necessary. The function avoids the swap by never owning what it borrows.

The engine parameter is a trait object rather than a generic, and that is the one place the chapter bends chapter 1's shape. The agent is generic over its engine so that the tests run against a scripted stub and the binary against whatever the user chose, with no dynamic dispatch in the loop. The sidechain has to run on the parent's engine in one branch and

a provider engine in the other, and those are different types. A generic function would force the caller to name one type per call site, which is fine for the parent branch and impossible for a provider engine constructed at run time from a definition file. The trait object costs one indirect call per pass, against a pass that costs seconds.

Bounded, and quiet in the right way

A sub-agent gets forty rounds where the main turn gets sixty-four. The number is plank's, and the reasoning is that a delegated task is meant to be bounded, and a sub-agent that needs more rounds than that has been given the wrong task; the parent should have split it or done it itself. What matters more than the number is what happens at the last round. The main loop of chapter 7 tells the model that its next reply is the final one and takes whatever comes back as the answer. The sidechain does the same, with the same reminder text, because the alternative is a sub-agent that calls a tool on every one of its forty passes and returns nothing, having discarded everything it found. Plank's comment on the shared reminder says why it is shared: "so the two cannot drift" (plank, 6090a5a). Blanka reuses chapter 7's constant for the same reason.

The guards of chapter 10 run in the sidechain with their own state. A fresh `Guards` is built for each delegation, because the parent's call window and progress budget describe the parent's turn, and a sub-agent that inherited them would be charged for repeats it never made. The call guard runs without the one exemption the main loop grants: polling a background job with `bash_status` is identical calls by design in the parent, and exempt there, but a sub-agent that polls a job has been given a task that waits, and the parent is the one that should wait, at a boundary, with the notification machinery of chapter 11 behind it. A sidechain has no boundaries. Nothing from the inbox joins it, no job notification is drained into it, no side question is answered inside it. It is a straight line from task to report, and everything that would make it a conversation is left to the conversation it returns to.

The reasoning guard is where plank's second build of the sub-agent loop earned its keep. A fan-out sub-agent in one recorded session looped in its reasoning for ten minutes until the repeat guard stopped it, then generated for thirteen more minutes with nothing on screen moving but a clock, and the user pressed Escape. Plank's write-up separates three gaps, each closed at its own layer. The first was display: only the main pass published the status snapshots the roster row was drawn from, so a sub-agent's row froze for the length of its pass, and a frozen row is indistinguishable from a hung process. The second was the recovery: the guard's error was fed back and nothing else changed, so at temperature zero the next pass was the same prompt plus one line and produced the same loop, and the sub-agent loop let that repeat towards its forty rounds. The third was evidence: the interrupted pass returned a bare error, so the sidechain dump ended on the guard's tool result and could not show what the model had been doing for thirteen minutes.

Chapter 10 already gave blanka the main turn's answer to the second gap, which is to stub the looping reasoning and end the turn on the second stop. A sub-agent has a different second stop available to it, and it is better than ending anything. The final-round reminder is a message that changes the prompt materially, and it turns the next pass into the report:

Chapter 12: Delegation and isolation

```
/// Consecutive reasoning stops after which the sub-agent is asked for its
/// report instead of another pass. Two, like the main turn, but the second
/// stop changes the prompt materially (the final-round reminder) rather than
/// ending anything; the third loop fails the sidechain.
pub const SUBAGENT_REPEAT_TRIP_CAP: usize = 2;
```

Inside the loop the cap reads as a small state machine. A stop stubs the last reasoning block, feeds the guard's error back as a tool result, and counts. At two, the reminder is pushed and a flag says the next pass is the report whatever it contains. If that pass loops too, the sidechain fails with the trips notice and the parent reads a tool error saying so, which at temperature zero is the honest outcome: a model that loops on the task and loops again on being asked to summarise what it has is not going to produce a report on the third attempt, and the parent is better placed to decide what to do than forty more rounds are. Plank's fix landed as the same two-then-fail rule (plank, 5d5508a), and blanka's test drives it with the cycling pass from chapter 10's tests:

```
#[test]
fn two_reasoning_stops_ask_for_the_report_and_a_third_fails_it() {
    let cycling = crate::agent::guard_tests::cycling_pass();
    let (end, t) = run(vec![cycling.clone(), cycling.clone(), "the report anyway".into()],
true);
    assert_eq!(end.report.as_deref(), Some("the report anyway"));
    assert_eq!(end.passes, 3);
    let reminders = t.messages().iter().filter(|m| m.role == Role::User && m.content ==
crate::compact::FINAL_ROUND_REMINDER).count();
    assert_eq!(reminders, 1, "the second stop changed the prompt: {t:?}");
    let (end, _) = run(vec![cycling.clone(), cycling.clone(), cycling], true);
    assert_eq!(end.report, None);
    assert_eq!(end.error.as_deref(), Some(guard::REPEAT_TRIPS_NOTICE));
}
```

The third gap is closed by a rule the sidechain applies on every exit path: what the model said is pushed into the sidechain before anything decides what it meant. An interrupted pass, a guard stop, a failed pass all leave their partial text in the sidechain's messages, and those messages are what the sidecar dump is made of. The main loop of chapter 4 had always kept its partial text; plank's sidechain path was written separately and missed it, which is the kind of divergence a shared function prevents by construction.

The first gap, the frozen display, is why the sidechain takes a sink at all. Plank's original sub-agent loop generated into a discarding sink, on the reasonable grounds that a sub-agent's output is not the answer and should not be printed as one. The recorded session showed what that costs: a user watching nothing move cannot tell looping from working, and a user who cannot tell presses Escape. Blanka's sidechain streams through the parent's sink, framed by two status lines, one when the sub-agent starts with the first line of its task and one when it ends with its outcome and its pass count. Guard stops inside it are red lines naming the sub-agent. On the plain front end the sub-agent's text scrolls past like the parent's, which is what plank's plain REPL does too; a panel of its own is the TUI appendix's business.

The report is the answer, not the reasoning

The report is the last thing the sub-agent said with its reasoning removed, and the removal is the fix for a failure plank recorded in a commit message worth quoting: the raw last assistant message "still carries the `<think>` block, because the KV prefix depends on the transcript holding thinking unaltered. So the parent received a report narrating half-abandoned alternatives, judged it unreliable, and redid the work by hand, defeating the delegation" (plank, d6134be). The sidechain's transcript must hold the reasoning, because the engine's state was computed over it and chapter 10 established that a transcript the engine does not match is a rebuild. The parent's copy must not, because the parent reads the report as a colleague's answer and a colleague who thinks aloud in a written answer is a colleague whose answer gets checked.

```
/// The report is the visible text of the reply: reasoning is not carried
/// back, and an empty remainder is no report.
fn report_of(reply: &str) -> Option<String> {
  let text = strip_thinking(reply).trim().to_string();
  (!text.is_empty()).then_some(text)
}
```

The same commit changed the task's framing, and the change is the second half of the fix, because no parser can find reasoning the model narrated as ordinary prose in the report body. The envelope asks for the plain answer stated once and gives the reason: the reader cannot see the reasoning and will treat a report that argues with itself as unreliable and redo the work. Blanka's framing carries that sentence, along with the two facts the sub-agent needs to know about its situation, that only the report is carried back and that everything else is discarded:

```
pub fn task_message(instructions: Option<&str>, task: &str) -> String {
  let mut out = String::from(
    "<system-reminder>\nYou are now acting as a sub-agent, handling a task delegated
from the main conversation. Complete the task using your tools, then end with a final
report of your results. Only that report is carried back into the main conversation;
everything else is discarded.\nWrite the report as the plain answer, stated once: what
you found, and nothing else. Do not narrate how you got there, do not weigh alternatives
you have already ruled out, and do not hedge or revisit your conclusion. The reader
cannot see your reasoning and will treat a report that argues with itself as unreliable
and redo the work.\n</system-reminder>\n\n",
  );
  if let Some(text) = instructions.map(str::trim).filter(|t| !t.is_empty()) {
    out.push_str("Instructions:\n");
    out.push_str(text);
    out.push_str("\n\n");
  }
  out.push_str("Task: ");
  out.push_str(task.trim());
  out
}
```

Chapter 12: Delegation and isolation

The report arrives in the parent as a tool result with its own framing, and the framing has two jobs. It tells the model to continue from the report, because a pass runs on it as soon as it lands and the report is the delegated work coming back, not background reading. And it tells the model not to repeat the work, because the tool calls behind the report are gone from the transcript and re-running them is the tempting failure. Notes go in front of the report, so that the model reads why its chosen agent did not apply, or where an isolated sub-agent left its files, before it reads the report those conditions produced.

Sidecars

Once the fork is folded out, the parent transcript has no trace of the sub-agent beyond its report. That is the design working, and it has a cost the dumps of chapter 5 and chapter 10 have to pay for. A repro dump of the parent shows a delegation as one assistant message with a call and one tool result with a report, and if the sub-agent looped for ten minutes in between, the dump does not show it. Plank's write-up of the hung sub-agent leans on the sidechain dump for exactly that, and blanka keeps the same record:

```
/// A finished sidechain, kept beside the parent for the dumps. The parent
/// transcript has no trace of it once the fork is folded out, so this is
/// the only record of what a sub-agent did.
pub struct Sidecar {
  pub label: String,
  pub task: String,
  /// `report`, `no report`, or `failed: <why>`.
  pub outcome: String,
  /// Where in the parent transcript the fork was, so the sidecar can be
  /// lined up against the main dump. Zero for a clean-room run.
  pub fork_at: usize,
  pub messages: Vec<Message>,
}

/// How many finished sidechains the agent remembers for `/repro`.
pub const SIDECARS_KEPT: usize = 8;
```

The agent keeps the last eight in memory and writes them beside every dump, the user's and the guards', as numbered files with the main dump's name as their stem. The fork depth is what makes a sidecar readable against the main file: message twelve of the parent is the delegating call, and the sidecar that says it forked at twelve is the sidechain that call ran. The count is plank's, and it is a memory bound rather than a policy; a session that delegates fifty times keeps the last eight sidechains, which is enough to explain the most recent failure and not enough to hold a session's worth of research in memory twice.

Definitions in markdown

A general-purpose sub-agent is the model with a task. A named one is the model with a task and a persona, and the persona lives in a file, because the person who knows how

Chapter 12: Delegation and isolation

a reviewer should review is the user, and the user edits files. The format is the one chapter 6 gave skills: a markdown file whose frontmatter names and describes the agent and whose body is the instructions, loaded from the user's directory and then the project's directory on top, so that a project can override a name:

```
pub struct AgentDef {
  /// The `name` the model passes to the `agent` tool. Defaults to the file
  /// stem.
  pub name: String,
  /// One line, shown in the roster.
  pub description: String,
  /// The body of the file: the instructions the sidechain opens with.
  pub prompt: String,
  pub engine: EngineSpec,
  /// Run each delegation in a throwaway git worktree, so a fan-out of
  /// sub-agents editing the same files cannot overwrite each other.
  pub isolate: bool,
}

pub enum EngineSpec {
  /// The engine the parent conversation is running on. The sidechain is a
  /// fork of the parent transcript and shares its cached state going in.
  Parent,
  /// A hosted, OpenAI-compatible endpoint. The sidechain is clean-room:
  /// only the framed task crosses the wire, never the parent transcript.
  /// `api_key_var` is the name of the environment variable holding the
  /// key, never the key itself, so a definition file stays committable.
  Provider { base_url: String, model: String, api_key_var: String },
}
```

The key variable's name is the design decision in that enum. A definition that named a provider would be useless without a credential, and a definition that held the credential could never be committed to the project's repository, which is where a project's agent definitions belong. Plank's solution is that only the variable's name is configurable, with a default per provider, so that one definition file works on every machine that has the right variable set and leaks nothing on the machines that do not. Blanka's default is the same variable its own provider engine reads.

Where the model learns the names is the second decision, and plank got it wrong once in a way that is easy to repeat. The `agent` tool's schema advertised a `name` parameter, and nothing put the roster of names anywhere the model could read it. The parameter was, in plank's words, "an enum with no values", and a wrong guess was hard-failed before the fork started, so the model learned to always omit the name, and named definitions became something only the user could dispatch (plank, 82a17f4). The roster has to be in the model's context. It must not be in the system prompt, because chapter 8 keys the shared cache on the prompt's bytes and a project with its own agents would then key differently from the same model anywhere else, and editing a definition would invalidate every session's prefix. Chapter 6 already solved this for skills: the list rides in

Chapter 12: Delegation and isolation

the stable tier of the session context, where a change costs a small project-tier rebuild. The agent roster joins it, with its own opening line among the context markers so that replay and dumps know it is scaffolding:

```
pub fn roster(defs: &[AgentDef]) -> Option<String> {
    let visible: Vec<&AgentDef> = defs.iter().filter(|d| key_present(d)).collect();
    if visible.is_empty() {
        return None;
    }
    let mut out = format!("{ROSTER_MARKER} (select one with the agent tool's name
argument):\n");
    for d in visible {
        let engine = match &d.engine {
            EngineSpec::Parent => String::new(),
            EngineSpec::Provider { model, .. } => format!("{}", model),
        };
        out.push_str(&format!("- {}: {}{}\n", d.name, if d.description.is_empty()
{ "(no description)" } else { &d.description }));
    }
    Some(out)
}
```

The filter on the key is the third decision. A provider definition whose variable is not set cannot run, and offering it to the model would cost a round for the model to learn that. So it vanishes from the roster and stays in the user's listing, labelled with its engine and the exact variable to set, because from the user's side a mistyped variable and a forgotten one look the same and the listing answers both. The delegation path applies the same gate a second time, when the model names an agent: a name that is unknown, or known but unrunnable, runs the general-purpose sub-agent and says so in a note in front of the report. Plank's reasoning is that a name the model could not have known about must not burn a round, and the fallback with a note costs nothing the model was not already paying.

Another engine for the child

A definition may name a hosted engine, and the sidechain then runs there. Two things change and one does not. The transcript the sidechain runs on holds only the framed task, because the parent's conversation must not cross the wire to a provider the user did not choose for it; plank calls this a clean room and blanka keeps the term. The engine is built from the definition each time, because a hosted engine keeps no state, so nothing is lost by constructing one per delegation and nothing has to be cached or swapped. The loop does not change at all, which is what the function-not-method decision bought:

```
EngineSpec::Provider { base_url, model, api_key_var } => {
    let mut alt = crate::openai::OpenAiEngine::new(base_url, model)
        .with_api_key(std::env::var(&api_key_var).ok())
        .with_tools(self.tools.describe_all());
    let mut t = Transcript::new();
    t.push(Role::User, framed);
}
```

Chapter 12: Delegation and isolation

```
let end = run_sidechain(&mut alt, &self.system, &mut t, &self.tools, &mut self.ctx,
&self.interrupt, show, guards_on, sink);
(end, t.messages().to_vec(), 0)
}
```

The system prompt is the parent's, tools and rules included, so a sub-agent on a hosted model reads the same instructions the local model does. The tool table is the parent's too, and the tool context is the parent's with its depth raised, so the sub-agent's edits land in the same working directory under the same write roots and the same plan-mode gate.

Plank's version of the other direction, a local sub-agent under a hosted parent, taught a lesson blanka inherits by not repeating the situation. The local engine had to be warmed with the system prompt before its first sidechain, and the first implementation prefilled it, which on plank's engine takes long enough that the front end froze for the length of a cold system prompt, because the warm could not be interrupted and on one path ran on the thread that draws. The fix was to restore the system tier from a blob where one existed and otherwise leave the engine cold, so that the sidechain's own first pass paid the prefill with the progress bar and the interrupt a pass has and a warm does not (plank, 9c08931). The comment on that fix ends with a line about why the outcome is always reported, hit or miss: "a silent hit is indistinguishable from a silent miss, which is the position this landed in twice". Blanka's hosted sub-agents have no warm to do, and the local engine of the appendix, when it arrives, will be the parent, with the warm walk of chapter 8 already done once at startup. A local sub-agent under a hosted parent is a configuration the appendix can add if the reader wants it, and the rule it will follow is plank's: restore, never prefill, and say which happened.

A checkout of its own

Two sub-agents editing the same file is a race, and the parent editing the file while a sub-agent edits it is the same race with fewer participants. A speculative change, the kind the user wants to see before deciding whether to keep it, has the same shape without any sub-agent at all: work that should not land in the tree the user has open. Git has the primitive for both, a second working directory of the same repository on its own branch, and plank wrapped it as isolation in three forms: a pair of tools for one task mid-session, a flag that starts a whole session inside one, and a frontmatter field that gives every run of a named sub-agent its own throwaway checkout (plank, 7040cfd). Blanka builds the first and the third.

The mechanics are one module and a pair of tools over it. A worktree has a name, which is both a directory under the repository's own hidden folder and a branch with a fixed prefix, and the name is validated before any side effect, because a name is a path and a path with a parent segment in it walks out of the folder the worktrees are meant to stay in:

```
/// Where a repository's worktrees live, relative to its root.
pub const WORKTREES_DIR: &str = ".blanka/worktrees";

pub struct Worktree {
    pub name: String,
```

Chapter 12: Delegation and isolation

```
pub path: PathBuf,  
pub branch: String,  
/// The main checkout, where git commands about the worktree run: never  
/// from inside the directory being deleted.  
pub root: PathBuf,  
/// The directory the session was in before it entered.  
pub from: PathBuf,  
}  
  
pub fn enter(from: &Path, name: &str) -> io::Result<Worktree>;
```

Entering resolves the main checkout first, through git's common directory rather than by walking up for a `.git` entry, so that a worktree created from inside another worktree still lands in the repository's own folder. It creates the branch if it does not exist and reuses it if it does, so a name is stable across sessions. And on first use it adds the worktrees folder to the repository's exclude file, the one under the git directory that is never committed, so that the user's own status stays clean without a gitignore entry they would have to explain to their colleagues.

What entering does to the session is one field. Every tool since chapter 3 resolves relative paths against the context's working directory and checks writes against its roots, so moving the session into a worktree is setting the directory and adding a root:

```
/// Move into a worktree: paths resolve there and writes are allowed  
/// there. The directory entered from stays a write root, so a tool that  
/// was given an absolute path into the main checkout still works.  
pub fn enter_worktree(&mut self, w: crate::worktree::Worktree) {  
    self.cwd = w.path.clone();  
    self.write_roots.push(w.path.clone());  
    self.worktree = Some(w);  
}
```

That last sentence of the comment is a compromise worth being honest about. The isolation a worktree provides is that relative paths land in it. A model that was handed an absolute path into the main checkout earlier in the conversation, and uses it, edits the main checkout, and no write root would stop that without also stopping every legitimate absolute path. Plank found this with sub-agents and fixed it in the prompt rather than the tool: an isolated sub-agent's instructions open with a notice saying where it is, that any absolute path it was given refers to the main checkout, and that it must translate before reading or editing. Blanka carries the same notice, and the test for isolation checks that the sub-agent's first message contains it. It is a prompt fix for a tool problem, which is the weakest kind, and the alternative, a tool that rewrites absolute paths under the main checkout to the matching path under the worktree, is a reasonable thing for the reader to add.

Fail closed

Leaving a worktree is where the design lives. Keeping it is nothing: the session moves back, the checkout and its branch stay for the user to review or merge. Removing it

Chapter 12: Delegation and isolation

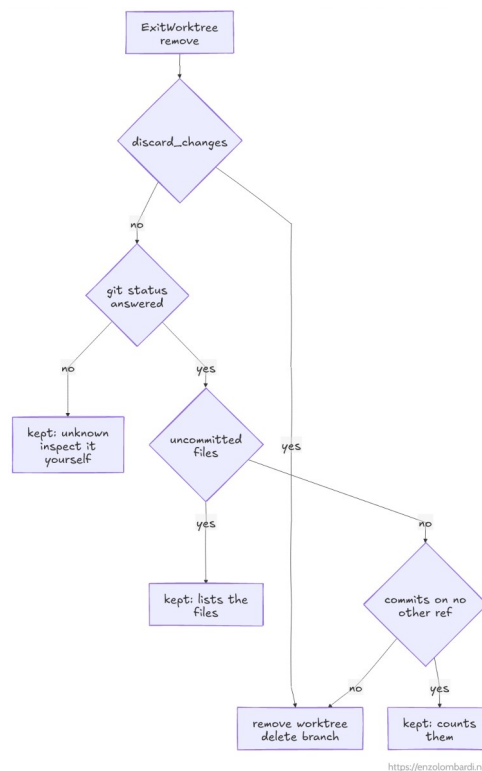
deletes a directory and a branch, and a directory that holds an hour of a sub-agent's edits is deleted the same way as an empty one. So removal asks git two questions first, and refuses when it dislikes either answer or gets no answer at all:

```
pub enum RemoveVerdict {
    Removed,
    /// Files modified or added and not committed.
    KeptUncommitted(Vec<String>),
    /// Commits reachable from the worktree's branch and from no other ref.
    KeptUnmerged(usize),
    /// Git could not say. Treated exactly like work that would be lost.
    KeptUnknown,
}

/// Remove the worktree and its branch. Refuses, fail-closed, unless
/// `discard` is set: uncommitted files, unmerged commits, or a git that
/// could not be asked each keep it in place.
pub fn remove(w: &Worktree, discard: bool) -> io::Result<RemoveVerdict> {
    if !discard {
        match changes(w) {
            None => return Ok(RemoveVerdict::KeptUnknown),
            Some(c) if !c.files.is_empty() => return
Ok(RemoveVerdict::KeptUncommitted(c.files)),
            Some(c) if c.commits > 0 => return
Ok(RemoveVerdict::KeptUnmerged(c.commits)),
            Some(_) => {}
        }
    }
    git(&w.root, &["worktree", "remove", "--force", &w.path.to_string_lossy()]);
    // After the worktree is gone git has released its hold on the ref.
    let _ = git(&w.root, &["branch", "-D", &w.branch]);
    Ok(RemoveVerdict::Removed)
}
```

The decision as the tool applies it:

Chapter 12: Delegation and isolation



Removal fails closed. Three of the five outcomes keep the worktree, and the refusal text names what would have been lost and the two ways out: keep it, or discard on purpose.

The unknown branch is the one that separates this from a convenience wrapper. Plank's document for the change puts it in one sentence: not knowing is not the same as knowing there is nothing there. A git that fails, a worktree whose link to its repository is broken, a status that cannot be parsed, each answers `None` from the function that counts changes, and every caller must read `None` as a refusal. Blanka's test for it breaks the worktree's pointer file so that git cannot find the repository, and asserts the verdict is unknown rather than clean.

Three bugs plank's integration tests caught, none visible to unit tests, shaped the counting function. The repository's common directory is reported as a relative path, so a structural check that compared paths textually failed until they were resolved. Trimming git's status output ate the leading space of an unstaged modification's line, which is column-significant, and corrupted the first filename the refusal was meant to name; blanka slices the line from its fourth byte and never trims it. And counting commits against the remotes alone reports a repository with no remote as entirely unpushed, so nothing could ever be swept; blanka asks a different question, commits reachable from this branch and from no other ref, which is what "would be lost" means when the branch is deleted. The verdict is named for that question, unmerged rather than unpushed, and the plan's earlier name was changed when the question was.

The refusal is a tool error with the list in it, and the list is what makes it actionable rather than a wall:

Chapter 12: Delegation and isolation

```
RemoveVerdict::KeptUncommitted(files) => {  
  let listed: Vec<&str> = files.iter().take(10).map(String::as_str).collect();  
  format!(  
    "worktree '{}' has unsaved work, so it was not removed: {} uncommitted file(s): {}{}.  
    Commit or merge the work first, or pass discard_changes true only if it is genuinely  
    disposable.",  
    w.name,  
    files.len(),  
    listed.join(", "),  
    if files.len() > 10 { ", ..." } else { "" }  
  )  
}
```

A refusal leaves the session exactly where it was, inside the worktree, and the tool test pins that too: after a refused remove the context still points at the worktree, and only the discarded remove brings the directory back. The tool version of leaving runs the gate while still inside, which is the order plank's tool uses and the reason it borrows the session rather than taking it.

An isolated sub-agent applies the same verdict without the discard flag and without a second chance. When the sidechain ends, the parent leaves the worktree and asks for removal. A clean one goes. One holding work stays, and the parent's report gains a note naming the path and saying that the changes are not in its working copy. Plank's commit calls this a deliberate departure from its reference, which deleted the worktree, and gives the reason in one line: the alternative silently discards the work the agent was asked to do. Blanka's test writes a file from inside an isolated sub-agent and checks three things afterwards: the main checkout has no such file, the worktree still exists and does, and the parent is back in the directory it started from. A second delegation that changes nothing leaves no second worktree behind.

Plan mode as a gate

A change the user has not approved should not land, and a model that has been told so in the prompt will still edit when it believes the change is obvious. Plan mode makes the belief irrelevant. While it is active, the tools that change the workspace are refused, the model researches with the ones that do not, and it leaves by presenting a plan the user approves or sends back.

Plank implemented the gate as a flag on the tool context checked at dispatch, and blanka does the same, for a reason that chapter 8 supplies. The obvious alternative is a smaller tool table, with the mutating tools removed while the mode is on. But the tool table renders into the system prompt, and the system prompt's bytes are the cache key; a table that changed per mode would be a prompt that changed per mode, and every entry into plan mode would cost the prefix. The gate leaves the prompt alone and refuses at the last moment, with a result that names the way out:

```
/// The tools that change the workspace. `bash` is here because it can run  
/// anything; `EnterWorktree` because a checkout is a change to the disk.  
pub const PLAN_BLOCKED_TOOLS: &[&str] = &["write", "edit", "bash", "EnterWorktree"];
```

Chapter 12: Delegation and isolation

```
pub fn refusal(name: &str) -> String {  
    format!("{name} is not available in plan mode. Research with read-only tools, then  
    present your plan with ExitPlanMode; edits are allowed once the user approves it.")  
}
```

The check sits in the table's single dispatch method, before the tool is looked up, so no tool has to know about the mode and a tool added later is gated by name alone. The test runs a write under the gate and checks two things, that the result says plan mode and that the file does not exist, then runs a glob and checks that it still works.

Leaving is a question to the user, and the question is the plan itself. The exit tool takes the plan as text, shows it, and offers two answers, approve and refine. Approval lifts the gate and tells the model it may edit. Anything else keeps the gate and tells the model what the user said, so that a refinement is a response to words rather than a guess at why. The mechanism it asks through is the next section's, and it has one degenerate case that matters: when there is nobody to ask, the gate lifts and the result says so. A gate that stayed down forever with no user to lift it would turn every unattended session that entered plan mode into a loop of refusals, and plank's first version auto-approved headless for the same reason. Blanka's result text names the situation rather than pretending the user approved.

Asking instead of guessing

The model's trained move when a turn is ambiguous is to write prose and stop, or to pick and proceed. Both are wrong when the choice is the user's and the paths diverge: a migration that can be forwards-compatible or clean, a rename that touches the public API or does not. An ask tool gives the model a third move, two to six labelled options and a wait, and the tool is easy to write. What is not easy, after chapter 11, is the wait. The model runs on the worker thread and the keyboard is read by the stdin thread and multiplexed on the main thread, so a tool that wants a line from the user cannot read one. It has to send the question up and block until the answer comes down, and while it blocks it has to keep watching the interrupt, because a user who has changed their mind about the whole turn will press the key that says so and expect the question to go away.

The tool itself knows nothing about threads or screens. It validates the call and hands the question to whatever asker the front end installed:

```
pub struct AskRequest {  
    pub question: String,  
    pub options: Vec<AskOption>,  
    pub multi: bool,  
}  
  
pub enum AskOutcome {  
    Answered(Vec<String>),  
    /// An empty answer: the user declined to choose.  
    Declined,  
    Interrupted,
```


Chapter 12: Delegation and isolation

```
}

/// Whatever can put a question in front of a person and wait.
pub trait Asker: Send {
    fn ask(&mut self, req: AskRequest) -> AskOutcome;
}
```

The asker the plain front end installs is a channel in each direction. The question goes out as an event on the worker's existing event channel, where the main loop prints it as a numbered list; the answer comes back on a second channel whose sending end the front end holds; and the wait polls the interrupt every hundred milliseconds so that Ctrl-C lands:

```
impl Asker for ChannelAsker {
    fn ask(&mut self, req: AskRequest) -> AskOutcome {
        if self.events.send(Event::Ask(req)).is_err() {
            return AskOutcome::Declined;
        }
        loop {
            match self.answers.recv_timeout(Duration::from_millis(100)) {
                Ok(outcome) => return outcome,
                Err(RecvTimeoutError::Disconnected) => return AskOutcome::Declined,
                Err(RecvTimeoutError::Timeout) => {
                    if self.interrupt.is_set() {
                        return AskOutcome::Interrupted;
                    }
                }
            }
        }
    }
}
```

On the front end's side, the question changes what the next line means. Chapter 11's rule was that a line typed while the worker is busy is queued for a boundary, or is a side question. A line typed while a question is up is the answer, and goes to the worker's answer channel instead of the inbox. The line is resolved against the options by number or by label, an empty line declines, and a line that matches nothing is an answer too, because the user has said something the options did not offer and the model should read it rather than be told the input was invalid. The result the model reads names the labels chosen, never an index.

Who installs the asker is decided once, from one fact: whether standard input is a terminal. A pipe has no person behind it, and a tool that asked a pipe a question would wait on a line that is never typed for it. With no asker the tool returns a sentence telling the model to proceed on its judgment, and plank's version says the same. A sub-agent never reaches the asker either, whatever the front end, because its task is meant to be complete and a question from three layers down has no line to the user; the tool checks the context's depth first and returns a different sentence, telling the sub-agent to decide and note the assumption in its report.

Chapter 12: Delegation and isolation

The interrupt case is what the sidechain and the ask tool have in common, and it forced one change in the main loop. Chapter 11 made the interrupt reach a tool waiting on a job. A tool waiting on a question, and a sidechain running its own passes, are two more places an interrupt can land inside a tool round, and in each the flag was set when the round returned and the loop went on to generate the next pass, which then saw the flag and produced an empty message. The loop now checks the flag once the tool result is in the transcript and ends the turn there, with the interrupted marker the user expects, before any pass runs on a flag that was never meant for it.

Plank's ask panel gained one row blanka does not have, an escape hatch labelled as a request to talk it over rather than choose, added after users found themselves picking the least wrong option because the panel offered no way to say none of these (plank, 71f5513). In blanka a free-text line is that hatch; a numbered row for it is the TUI appendix's.

A setup the model runs

The two tools of this chapter, one that delegates and one that asks, compose into something neither does alone, and plank's `/init` command is the worked example. Chapter 6 described its six phases from the prompt's side: interview, survey, follow-up questions, the shared instruction file, the personal one, a summary, all in one static text the model executes. Seen from this chapter's side, the phases are the machinery built above. The interviews are `ask` calls, answered by whichever asker the front end installed, a panel in one and a numbered list in the other, and on a pipe the asker declines and the tool's sentence tells the model to proceed on its judgment. The survey is an `agent` call, so the parent's context receives a report rather than the contents of every file the survey read, which matters for a command whose last phases need room to write. And the writes are the ordinary `write` tool inside the ordinary containment.

That is why the phases could live in a prompt rather than in a front end. A front end driving them would show a form, run a survey, show another form, and would have to be written twice, once per front end, the cost the appendix on the full-screen display counts for every slash command. A prompt that calls tools needs each front end to have installed its asker and nothing more, and plank's first phased version shipped without the usual mirrored change for exactly that reason.

The prompt's reliance on the tools is also where its two failures came from. The first is chapter 6's: the prompt called the survey "a `task` sub-agent", and every benchmark run, on every model, called the to-do tool first. A prompt that orchestrates tools is held to the tool table's vocabulary, word for word. The second is chapter 10's: phases that interview, survey and write all revisit the same tree, which is the shape the call guard refuses, and a turn the user asked for was stopped mid-phase for following its instructions, until plank gave that one command a scoped suspension of the guards. Both are the price of putting behaviour in a prompt. The harness's rules were written for turns the model improvises, and a turn the harness scripted meets them as a stranger.

Blanka's chapter 6 command writes the instruction file in one turn. Everything the phased version needs, the asker, the delegation and the fork that keeps the survey out of

the parent's context, exists by the end of this chapter, so a reader who wants the six phases writes a prompt, not a front end.

Tests across the fork

The delegation tests pin the fork's arithmetic. A parent that delegates once, receives a report and answers has four messages: the prompt, the reply with the call, the report as a tool result, the answer. The report contains the sub-agent's visible text and none of its reasoning, the transcript rendered flat contains no trace of the framed task, and the one sidecar records a fork depth of two and two messages. The passes counted in the turn report are the parent's two, because the sidechain's passes are its own, and a dump written afterwards produces a numbered sidecar file beside the main one.

A sub-agent that only loops fails, and the parent is told in a tool error, with its own trip count untouched; the parent transcript holds no reasoning at all and the sidecar holds the stubbed cycle. A name the roster does not have runs the general sub-agent and the note leads the result. The isolated sub-agent test builds a repository, delegates a write, and checks the three facts from the worktree section. The nested test runs an `agent` call from inside a sidechain and finds the refusal in the sidechain's tool result.

The worktree tests use a real git repository in a scratch directory, because a mock of git is a test of the mock. A clean worktree is removed and the main checkout's status stays empty; a worktree with an untracked file is kept with the file named; a commit on the branch is kept as one unmerged commit; a discard removes it; a broken pointer file yields unknown. The tool test enters, writes, checks the file landed in the worktree and not the checkout, is refused a remove, and is granted one with discard.

The ask test on the worker spawns it interactive, sends a prompt whose scripted reply is a question, receives the question as an event, resolves an answer against it, sends the answer back and reads the model's next reply from the visible text. Plan mode's tests run the gate through the table, then approve and refuse through a scripted asker, then lift unattended.

One test changed for a reason that will recur. The worker tests glob for a pattern that matches nothing in a directory that was, until this chapter, the whole temporary directory, and the walk was fast enough to pass. Under a parallel test run that now also creates git repositories there, the walk took long enough to look like a hang and trip the ten-second wait. The tests got a scratch directory each. The lesson is chapter 11's in a new place: a test that waits on a thread must give the thread nothing slow to do.

The smoke run

The run uses the echo engine against a project directory holding a small git repository and three agent definitions in the home directory: a reviewer with instructions and no engine, a builder with worktree isolation, and a remote definition naming a hosted model and a key variable that is not set. The listing command shows all three, the remote one labelled with its model and the sentence that it is hidden from the model because the variable is unset.

Chapter 12: Delegation and isolation

A delegation stanza naming the reviewer produces the start line with the task, then the sub-agent's reply, which on the echo engine is the framed task itself: the reminder, the reviewer's instructions under their heading, and the task. The end line reports one pass and no tool rounds, and the turn's summary counts two passes and one tool round for the parent. A delegation naming an agent that does not exist runs the general sub-agent and its result opens with the note. The loop dump written later by a guard has the sidecar beside it.

The worktree stanza enters, the status command reports the name, path and branch, a write lands in the worktree, and a remove is refused with the filename; the echo engine re-issues the same refused call until the call guard of chapter 10 ends the turn, which is that guard doing its job on a tool this chapter added. A remove with discard succeeds and the status command reports no worktree; the repository's own status is clean throughout, because the worktrees folder is excluded.

The ask stanza needs a terminal, so the run drives the binary through a pseudo-terminal. The question appears with its two numbered options and the prompt line, a typed digit answers it, an empty line declines, a free sentence is passed through as the answer, and Ctrl-C during the fourth repetition ends the turn with the interrupted marker and a summary of four passes, four tool rounds. Plan mode on a pipe refuses a write with the plan-mode text and leaves no file, and the exit lifts the gate with the unattended sentence.

The run also caught a front-end bug from chapter 11. Slash commands typed while the worker was busy were queued with the other lines and joined the conversation at the next boundary as user text, so the echo engine repeated them back. A command is for the harness, not the model; they now wait in the front end and run when the worker is idle.

Where blanka stops short of plank

Plank fans out: a tool that takes a list of subtasks and runs each as a named sub-agent, joining the reports in order. On its local engine the subtasks run one after another on one command queue, and the tool's description promises what that delivers, "structure, not speed". The concurrent version, for provider-backed definitions, rests on the session clone chapter 9 described and plank has not finished, and blanka does not attempt either form. Advertising the tool also changed plank's system prompt and with it the fingerprint of every cached prefix, which plank's findings record as churn to be batched with other prompt changes; blanka paid the same churn once this chapter, when six tools and two rules joined the prompt and its version moved to five.

Plank gates the model's initiative with settings: whether the model may pick a definition on its own and how many may run at once, alongside a per-definition opt-out. Blanka offers every runnable definition and has no parallelism to cap. Plank sweeps abandoned throwaway worktrees at startup, only ones whose names have the exact shape it generates, and only after git confirms they hold nothing; blanka names its throwaway worktrees so that such a sweep could recognise them and leaves the sweep to chapter 14. Plank replaces the git backend with hooks for a non-git version control system, starts whole sessions inside a worktree from the command line, and copies gitignored files a project lists into each new checkout. Plank's sub-agent output has a roster below the

Chapter 12: Delegation and isolation

status bar with one row per run and a pane the user can expand; blanka's scrolls past. The roster needed two small fixes blanka's scrolling output does not: a sub-agent without a name was labelled "sub-agent", so a log holding several could not tell them apart, and each now draws the next word of the NATO alphabet from one counter on the agent, which a nested call's save and restore of its label cannot rewind; and stepping into the roster with the keyboard had resurrected every row that had already expired, so the cursor now walks only the rows that are drawn. Plank's ask panel has a configurable option cap, a header chip, and the escape-hatch row.

And plank's alternate-engine sidechain covers the fourth quadrant, a local sub-agent under a hosted parent, with the restore-only warm described above. Blanka's local engine is the appendix's, and the appendix inherits the rule.

Hands off, eyes on

A sub-agent is what the parent takes back from it. Everything this chapter built serves that sentence: a fork so that the child reads the conversation for free, a fold so that the conversation keeps only the answer, a snapshot around both so that the engine never notices, a ladder cut so that no rung describes a prefix that is gone, a loop with a shorter fuse that asks for the report instead of running out, an envelope that asks for the answer stated once, and a sidecar so that what the child did is not lost with it.

Around the sub-agent sit the three things a harness owes a user who has handed work off. A definition in a file, with the engine it runs on and the name of a variable, never a key, and a roster that lives where editing it costs a small tier and not every session's prefix. A checkout the work can happen in, whose removal fails closed, because not knowing is not the same as knowing there is nothing there. And two tools for the moments when the model should not decide alone: one that holds it read-only until a plan is approved, and one that asks a question and waits, with a sentence ready for the case where nobody is there to answer. Put together in one static prompt, the two make a phased setup the model runs on either front end, provided the prompt names its tools exactly.

CHAPTER 13: HOOKS, SKILLS, MCP AND PLUGINS

Everything blanka has done so far it has done with its own text and its own processes. The system prompt is a file in the crate, the tools are types in the crate, the only text from outside that reaches the model is what the user types and what the tools read from disk, and the only processes that run are the ones the shell tool was asked to run. A harness people use for long does not stay that way. Someone wants a script to run before every edit, to refuse the ones that touch a generated file. Someone wants a code-graph server's query tools in the table beside read and search. Someone has a directory of prompts that turn the model into a careful reviewer and wants to share it as a unit. Each of these is an extension point, and each is one of two things: text someone else wrote reaching the model, or a process someone else wrote running on the user's machine at the harness's request.

That framing decides most of the design. Foreign text needs a place in the prompt that does not disturb what chapter 8 cached, a name that cannot collide with the harness's own, and a boundary the model is told about. A foreign process needs an environment it did not inherit by accident, a timeout, a process group so the timeout means something, and a protocol for saying what it found. The four mechanisms of the chapter, hooks, skills, MCP servers and plugins, are four arrangements of those two things, and plugins are where the arrangement itself becomes the problem: when the user, the project and three plugins all define a reviewer, which one runs is a question the harness has to answer the same way every time.

Plank built all four, then found that the interesting part was the fifth: installing a plugin written for another harness. Its hooks file has the events one level too deep, its commands find their own files through a variable the other harness's runner sets, and two of its events name things plank does not do. Adapting that contract without teaching the hook runner about any of it is the chapter's worked example, and blanka does it at a different moment than plank did, for a reason the plugin loader's shape makes visible.

A hook is a process

A hook is a shell command the user attaches to a moment in the loop. Four moments, in blanka: before a tool runs, after it ran, when a turn is about to end, and before a compaction pass. The command reads a JSON payload on its standard input describing the moment, and it answers with its exit code. Zero is nothing to say. Two is the number that means the hook has something for the model: before a tool, that refuses the tool and the hook's standard error is the error the model reads; after a tool, the standard error is appended to what the model reads; at the end of a turn, the standard error is fed back and the model goes on working. Any other non-zero exit is a warning for the user and nothing for the model, because a hook that crashed has not made a decision, and treating a crash as a refusal would let a broken script block every edit.

The event names are another harness's names, and that is deliberate:

```
impl HookEvent {  
    /// The names the configuration file uses. They are the names another  
    /// harness uses too, on purpose: a hooks file written for it loads here.
```

Chapter 13: Hooks, skills, MCP and plugins

```
pub fn name(self) -> &'static str {
  match self {
    HookEvent::PreTool => "PreToolUse",
    HookEvent::PostTool => "PostToolUse",
    HookEvent::TurnEnd => "Stop",
    HookEvent::PreCompact => "PreCompact",
  }
}
```

Plank shipped the first three under the same names with the same exit-code contract (plank, c00d096), and later widened the set to a dozen events, of which the worked example at the end of the chapter will meet two that blanka does not have. The choice to spell the events as the other harness spells them is what lets a hooks file travel between harnesses at all, and it costs nothing: the names are strings in a JSON file.

The design in the hook runner is in what the process does not get. Plank's runner execs the shell with the environment it inherited and injects nothing, which is what a plugin's author later found out the hard way. Blanka goes one step further and clears the environment, putting back only the path and the home, because a hook is told everything it needs through the payload and nothing the user's shell happened to export should reach a script the user did not write:

```
/// Run one command with the payload on stdin. The environment is cleared
/// and only the path and the home are put back: a hook is told what it needs
/// through the payload, and nothing the user's shell happened to export
/// reaches it by accident.
pub fn run_one(hook: &Hook, payload: &Value, cwd: &Path) -> HookResult {
  let mut cmd = Command::new("/bin/sh");
  cmd.arg("-c").arg(&hook.command).current_dir(cwd).env_clear();
  for keep in ["PATH", "HOME"] {
    if let Some(v) = std::env::var_os(keep) {
      cmd.env(keep, v);
    }
  }
  cmd.env("BLANKA_HOOK_EVENT", hook.event.name());
  cmd.stdin(Stdio::piped()).stdout(Stdio::piped()).stderr(Stdio::piped());
  // Own process group: the timeout must take down `sleep 600; echo ok`
  // whole, not only the shell that forked the sleep.
  cmd.process_group(0);
}
```

The process group is the second half of the timeout. A hook that runs `sleep 600; echo ok` is a shell that forked a sleep, and killing the shell leaves the sleep running with the pipe open, which is the difference between a timeout that fires and a timeout that hangs. Plank's runner puts each hook in its own group and kills the group, and its test for it is named for exactly that compound command; blanka's test does the same with a one-second timeout and asserts the whole call returned in under five. The third half, if a design can have three, is that both pipes are drained on threads of their own before the wait loop starts. A hook that writes more than a pipe buffer to standard output and then

exits would otherwise block on the write while the harness blocked on the wait, and both would sit there until the timeout, which would then blame the hook for being slow.

What the results mean is decided once, in a function the loop reads rather than in the loop:

```
/// What a set of hook results means for the harness.
pub struct HookOutcome {
    /// The first exit-2 stderr: for the model.
    pub block: Option<String>,
    /// Every other failure: for the user.
    pub warnings: Vec<String>,
    /// Exit-0 stdout of every hook, joined: context for the model, on the
    /// events that take it.
    pub context: Option<String>,
}
```

The first block wins, and the remaining hooks still run, because a hook that logs every tool call should see the call whether or not another hook refused it. The context field is for the one event where standard output means something: before a compaction, a hook that prints text is injecting it into the conversation so the summary pass can see it. Plank uses the same channel for its session-start and prompt-submit events, which blanka does not have, and the shape of the outcome is ready for them.

Why stdin, and why an exit code

The protocol between the harness and a hook is deliberately the oldest one there is: bytes in on standard input, a number out as the exit status, and standard error for the words. Three other designs were available and each was rejected for a reason worth having.

The payload could have gone on the command line as arguments. It does not, because a tool call's arguments can be a whole file's contents, and an argument list has a length limit that a large edit would hit; because quoting a JSON document into a shell command line is a source of bugs that never ends; and because a process's arguments are visible to every other process on the machine through the process listing, and a payload that includes the content of an edit to a credentials file should not be. Standard input has no length limit, needs no quoting, and is private to the process reading it.

The hook could have answered with JSON on standard output instead of an exit code. Plank added an answer envelope of that kind later, additively: a hook may print a JSON object with a field that halts the turn or a message for the user, and the exit-code contract stays authoritative when it does not (plank, 8d869aa). Blanka keeps the exit code alone, and the reason for starting there is that a hook written in three lines of shell can produce an exit code and cannot easily produce well-formed JSON, and a protocol that a three-line hook cannot speak is a protocol with fewer hooks in it. The reason the code is two rather than one is the crash case from the first section: a script that fails for its own reasons exits one, and if one meant "refuse" then every broken hook would be a hook that refused everything. Two is a number a script has to choose on purpose.

Standard output means something on only one of the four events, and the asymmetry is deliberate too. Before a compaction, a hook's output is context for the summary pass, and there is nowhere else for that text to go. Before or after a tool, a hook's output has no natural destination: it cannot be the tool result, because the tool has one, and it cannot be a user message, because the model is in the middle of a tool round. So on those events standard output is ignored, and a hook author who wants to say something to the model says it on standard error with exit code two, which is the one channel the contract defines for that. Making every channel mean something on every event would have made the contract wider and every hook author's job of remembering it harder.

Where the loop listens

Two of the four moments are inside the one function the loop uses to run a call, which chapter 12 introduced for the delegation intercept:

```
fn run_one<S: RenderSink>(&mut self, call: &ToolCall, sink: &mut S) -> (String, bool) {
    use crate::hooks::{tool_payload, HookEvent};
    // Before: a hook that exits 2 refuses the call, and its stderr is
    // the error the model reads. The tool never runs.
    let payload = tool_payload(HookEvent::PreTool, &call.name, &call.args, None,
    &self.ctx.cwd);
    if let Some(why) = self.fire_hooks(HookEvent::PreTool, &call.name, &payload).block {
        return (frame_error(&format!("blocked by hook: {why}")), false);
    }
    let (mut output, ok) = if call.name == "agent" { self.delegate(&call.args, sink) } else
    { self.tools.dispatch_one(call, &mut self.ctx) };
    // After: a hook that exits 2 has something to add to what the model
    // reads, and it is appended, not substituted.
    let payload = tool_payload(HookEvent::PostTool, &call.name, &call.args,
    Some(&output), &self.ctx.cwd);
    if let Some(note) = self.fire_hooks(HookEvent::PostTool, &call.name, &payload).block {
        if !output.ends_with('\n') {
            output.push('\n');
        }
        output.push_str(&format!("[hook: {note}]\n"));
    }
    (output, ok)
}
```

A refused call reports failure to the progress budget of chapter 10, which is right: a hook that refuses every write is a turn making no progress, and the budget should notice. The payload carries the tool's name and its arguments as JSON, so a matcher too coarse to express "writes to a dotfile" can be replaced by a hook that reads the payload and decides, which is what the test does with a one-line shell pipeline. Plank's matchers went further and grew an argument glob, so that a matcher can say `bash(git *)` and mean commands beginning with git; blanka's matcher is a list of tool names, and the payload is where finer decisions live.

The turn-end hook has a rule the other two do not need. When the model produces a reply with no tool calls in it, the turn is about to end, and a hook that exits 2 sends the model back to work with its standard error as a user message. It gets to do that once per turn:

```
ParseOutcome::None => {
  // The turn is about to end. A turn-end hook that exits 2
  // sends the model back to work with its stderr, once per
  // turn: the hook is a reviewer, not a supervisor.
  if !self.stop_hooks_fired && self.hooks.iter().any(|h| h.event ==
crate::hooks::HookEvent::TurnEnd) {
    self.stop_hooks_fired = true;
    let payload = crate::hooks::lifecycle_payload(crate::hooks::HookEvent::TurnEnd,
&[("last_message", &crate::toolcall::strip_thinking(&reply))], &self.ctx.cwd);
    let out = self.fire_hooks(crate::hooks::HookEvent::TurnEnd, "", &payload);
    if let Some(why) = out.block {
      sink.status("[turn-end hook sent the model back to work]");
      self.transcript.push(Role::User, format!("[A turn-end hook objected: {why}]"));
      continue;
    }
  }
  break;
}
```

Without the once, a hook that always objects is a loop, and chapter 10 spent its length on the fact that a loop at temperature zero does not fix itself. With it, the hook is a reviewer who gets one round of comments, and the model's second answer stands whatever the reviewer thinks of it. Plank's Stop hook has the same once-per-turn rule for the same reason. The test scripts a hook that always objects and checks that the turn took two passes and the transcript holds the objection between them.

The compaction hook runs before the summary pass and its standard output is pushed as the last message, so the model writing the summary has it in front of it. A project that keeps its state in a file the model should remember across a compaction can print that file from the hook, and the summary will carry it. That is the same idea as chapter 7's task list surviving compaction, opened to text the harness does not know about.

Rules, hooks and servers, in layers

The three mechanisms that reach the model are best understood by watching one problem use all of them. A code-graph server exists to keep the model from reading thirty files to learn what a query could tell it in five hundred tokens. Registering the server gives the model the tools. It does not make the model use them, because a model with a read tool and a habit will read. The setup for one such server, described by its author, has four layers, and the layers are instructive because each catches what the one above misses.

The first layer is rules: a paragraph in the project instructions telling the model to prefer graph queries over file reads and never to launch an exploring sub-agent when the graph

can answer. That is the primary mechanism, and it works most of the time, which is not the same as always. The second layer is a hook before the tool that launches sub-agents: when the model ignores the rule and delegates an exploration anyway, the hook refuses the call and its standard error says what to do instead. The model reads a tool error that names the graph tool, and calls it. The third layer is permissions, so the server's tools run without a confirmation each time, because a tool that interrupts the user on every call is a tool the model learns to avoid. The fourth is the server registration itself, without which none of the others has anything to point at.

Blanka has three of the four, and the reader can wire the example in a hooks file and a rules paragraph. The hook is one line: a pre-tool hook matching the delegation tool that exits two with a sentence, gated on whether the graph server is connected, which the hook can learn from the payload's working directory and a file the server leaves behind. What the example teaches is the relationship between the layers. A rule is cheap and reaches the model's judgment, and it fails silently when the judgment goes the other way. A hook is blunt and reaches the model's hands, and it cannot fail silently, because a refused call is in the transcript. A server is neither: it is the thing both are steering the model towards. A harness that had only rules would depend on the model; one that had only hooks would refuse without teaching; the two together turn the refusal into a lesson the model reads at the moment it needs it. That is also why the pre-tool hook's standard error becomes the tool error rather than a warning for the user: the model is the one that has to change course, and the user is not the one who made the call.

Skills, and what one is for

Skills arrived in chapter 6 as prompt files the user invokes by name and the model may load by name, and nothing about them changes here except where they can come from. They are worth a paragraph anyway, because the plugin section is about to move them around and the reason they exist bears on the reason plugins do.

A skill is a prompt that earned reuse. The reader who has asked a model to audit a module for magic numbers, or to find the flaky test in a suite, has written the same three paragraphs more than once and refined them each time: what to look for, what counts, what shape the answer should take. The observation that specificity is what turns a general assistant into a specialist is the whole case for keeping those paragraphs in a file, and the file's frontmatter, a name and a line of description, is what lets the model choose to load it without the user asking. A skill that says "when reviewing, check the invariants each struct maintains and whether the tests pin them" is more use in a plugin a team shares than in one person's home directory, and that is the first thing a plugin bundles.

Plank's skills started as user-invoked commands, and its rule for plugin skills came from a collision that looked harmless. A plugin's skill was registered under its bare name and under a namespaced alias, so `/greet` and `/demo:greet` both worked. Then a plugin shipped a skill whose bare name matched one the user already had, and the user's own command started running someone else's prompt. The fix was to register plugin skills under the namespaced form only (plank, `ac6a9ec`): a skill is a command the user types, and a plugin taking a command away from the user by arriving later is not a collision worth resolving with a warning, it is one that must not happen. Blanka adopts the rule as the first of the plugin section's namespacing decisions.

The other side of a tool call

The tools of chapter 3 live in the binary, and the model's picture of the world is exactly as wide as that set. An MCP server widens it. The Model Context Protocol is a small JSON-RPC vocabulary a process speaks on its standard input and output: a client says hello, asks for the list of tools, and calls them; the server answers with schemas and results. A code-graph server that has indexed a repository answers "who calls this function" from a database in a few hundred tokens where the read tool would answer it in thirty files. A server wrapped around an async runtime's telemetry answers "which tasks are starving" from a live stream. From the harness's side, each of those is a tool with a name, a description and a schema, and the harness's job is to make it indistinguishable from the built-in ones except in the name.

Seen from the server's side, the protocol teaches something about tool design that the chapter's client inherits. A server that exposes its raw data, every task record, every graph node, gives the model something it cannot use, because a context window is not a database and a result of ten thousand rows is a result the model reads the top of. The servers that work answer specific questions with bounded results: not "give me all tasks" but "which tasks are consuming the most time", not "dump the graph" but "the symbols connected to this one, and the files they live in". One such server keeps a running count of the tokens its answers replaced, computed from the sizes of the files a query touched, and after a working session the count is in the hundreds of thousands. That number is the reason the rules of chapter 12 tell the model to prefer conclusions over file dumps, and an MCP server is the most direct way of giving the model a source of conclusions.

The client is one struct and it is synchronous on purpose:

```
pub struct McpClient {
    pub server: McpServer,
    child: Child,
    stdin: ChildStdin,
    /// Lines from the child's stdout, read on a thread so a deadline can be
    /// kept without polling a pipe.
    lines: Receiver<String>,
    stderr_tail: Arc<Mutex<String>>,
    next_id: i64,
    pub timeout: Duration,
    alive: bool,
}
```

One tool call blocks the worker for one round trip, exactly like every other tool, and plank's port of the client says so in its first paragraph. What the client must not do is block forever, so a request has a deadline and the deadline covers the whole exchange. A server may send notifications before its reply, and a careless one may write log lines to standard output that are not JSON at all; the read loop skips both and keeps waiting, but it waits against the same clock it started with, so a server that streams notifications forever still has to answer in time. The reader thread is what makes the deadline cheap: the child's standard output is read line by line on a thread and sent over a channel, and

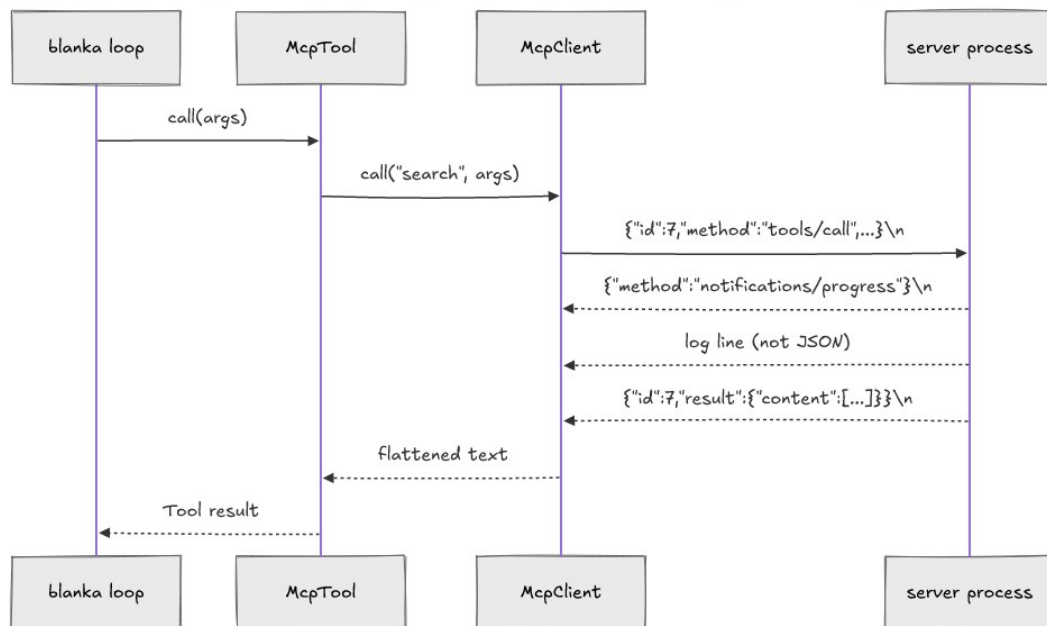
Chapter 13: Hooks, skills, MCP and plugins

the request waits on the channel with a timeout, which the standard library provides, instead of polling a pipe with a system call, which plank does with the C library.

```
pub fn request(&mut self, method: &str, params: Value) -> Result<Value, McpError> {
    if !self.alive {
        return Err(McpError::Transport("server is not running".into()));
    }
    self.next_id += 1;
    let id = self.next_id;
    self.write_line(&json!({"jsonrpc": "2.0", "id": id, "method": method, "params":
params}));
    let deadline = Instant::now() + self.timeout;
    loop {
        let remaining = deadline.saturating_duration_since(Instant::now());
        let line = match self.lines.recv_timeout(remaining) {
            Ok(l) => l,
            Err(RecvTimeoutError::Timeout) | Err(RecvTimeoutError::Disconnected) => {
                self.alive = false;
                return Err(McpError::Transport(self.failure_reason()));
            }
        };
        let Ok(v) = serde_json::from_str::<Value>(&line) else { continue };
        if v.get("id").and_then(Value::as_i64) != Some(id) {
            continue;
        }
        if let Some(err) = v.get("error") {
            let msg = err.get("message").and_then(Value::as_str).unwrap_or("MCP error");
            return Err(McpError::Protocol(msg.to_string()));
        }
        return Ok(v.get("result").cloned().unwrap_or(Value::Null));
    }
}
```

A server that misses its deadline is marked dead, and every later call fails at once rather than waiting another thirty seconds each. That is the only reasonable reading of a missed deadline: the server is not slow, it is gone, and a model that keeps calling its tools should be told so in the first line of each result.

The whole exchange, for one call:



<https://enzolombardi.net>

One MCP tool call over stdio. The client skips the notification and the log line and returns when the reply with its id arrives, or when the deadline it set before writing runs out.

What a dead server says

The one piece of the client that plank rewrote after shipping is the stderr tail, and the finding is short enough to quote. Plank spawned servers with their standard error discarded, so that only JSON-RPC reached the client, and reported a timeout and a closed pipe through the same message. When a code-graph server was started outside an indexed project, it printed one line explaining that no index was found and exited before answering the handshake, and the client reported that the server had not answered within thirty seconds, which had not elapsed. The one actionable line had been thrown away (plank, ab704a1).

The fix has three parts and blanka has all three. Standard error is piped into a bounded rolling tail, a few kilobytes, drained on a thread; keeping it costs nothing and standard output stays clean. When a read fails, the client asks whether the child has exited, and gives the exit a short grace period, because the end of the standard output pipe races the exit status and an immediate check says "still running" for a process that has already died. And the message quotes the last line of the tail rather than the first, because a startup error is usually followed by a usage dump, and the usage dump is not the error:

```

fn failure_reason(&mut self) -> String {
    // EOF on stdout races the exit status; give it a moment.
    let grace = Instant::now() + Duration::from_millis(250);
    let exited = loop {
        match self.child.try_wait() {
            Ok(Some(s)) => break Some(s),
            Ok(None) if Instant::now() < grace =>

```



```
std::thread::sleep(Duration::from_millis(10)),
    _ => break None,
  }
};
let mut msg = match exited {
  Some(s) => match s.code() {
    Some(c) => format!("server exited with status {c}"),
    None => "server was killed by a signal".to_string(),
  },
  None => format!("no response within {}s", self.timeout.as_secs()),
};
let tail = self.stderr_tail.lock().map(|t| t.trim().to_string()).unwrap_or_default();
if let Some(last) = tail.lines().next_back() {
  msg.push_str(": ");
  msg.push_str(last);
}
msg
}
```

The integration test for it runs the fake server with a flag that makes it print a complaint and a usage line to standard error and exit before the handshake, and asserts the message names the exit status and quotes the usage line, which is the last one. A server that exits and a server that ignores you are different problems, and the operator can act on the difference only if the message states it.

Tools that join the table before the prompt exists

Connecting a server produces tools, and the tools go into the same table as read and write, each under the name the model will type:

```
/// The name the model sees: `mcp__<server>__<tool>`. Two underscores
/// because a server name may contain one.
pub fn tool_name(server: &str, tool: &str) -> String {
  format!("mcp__{server}__{tool}")
}
```

The prefix keeps a server's search tool from colliding with the built-in one and with another server's, and the double separator is what makes the name splittable when the server's own name has an underscore in it; the configuration loader refuses a server name containing the separator for the same reason. Each tool holds the client behind a mutex, because the table hands out shared references and a call needs to write to the child's pipe. A result the server marked as an error becomes a tool error, so the model reads it as one; a result with no text becomes a line saying so, because an empty result reads as a tool that did nothing when the tool may have done exactly what was asked.

Where this meets chapter 8 is timing. The tool table renders into the system prompt, the system prompt is fingerprinted, and the fingerprint keys the cache. So the MCP tools have to be in the table before the agent is built, which is why the binary's startup connects servers and registers their tools before constructing the agent, and it means the static

prefix is static per configuration rather than per build. A project with a code-graph server has a different prompt from one without, and a different cache key, and that is right: it is a different set of tools. What must not happen is the prompt differing between two launches of the same configuration, and there are two ways a server can make it differ. It can return its tools in a different order each time, which the handshake defeats by sorting them by name before they render. And it can put a number in a description. Plank's finding names the trigger: a server advertised a tool whose description included the size of its index, a count that ticked between launches, so the fingerprint missed on every start and a snapshot of a hundred and thirty megabytes was rebuilt cold each time. The fix is on the server, keep descriptions static and put counts in results, and the diagnosis is a diff of two launches' prompts, where the culprit is almost always a same-length change in the middle.

The other boundary is trust. A tool description is text the model reads as part of its instructions, and it was written by whoever wrote the server. Plank's port draws the line explicitly: its reference tokenizes the built-in prompt as trusted and everything appended from outside, user-supplied prompt text and MCP schemas alike, as plain content that can never become a control token, so that a server cannot forge a turn boundary by putting the right characters in a description. Blanka's engine trait takes messages, not tokens, and the local engine of the appendix inherits the rule where the tokenizing happens. What blanka can do at this layer is what it does for every tool: the description is data in a JSON object, rendered by the same code that renders the built-in schemas, and nothing in it is interpreted.

Plank also measured what a large tool set costs, on a hosted-protocol engine where every tool schema is sent with every request, and the numbers are worth a table because they settle an argument the reader will have with themselves:

none	61	1.4 s
66	13,600	80.8 s
140	24,700	156.4 s

Decode speed was flat across the three within noise. The cost of a wide tool table is time to first token, which is prefill, which a prompt cache pays once per session and again whenever anything perturbs the prefix. Plank's answer is a directory: a server's primary tools get a full schema in the prompt and the rest get one line each, with a describe tool for the model to ask for a full schema on demand. That comes with a trap the finding records for the hosted path: on an OpenAI-compatible endpoint the model can only call a function that was declared, and a server that builds a grammar from the declared set makes an undeclared name ungeneratable, so hiding a schema silently deletes the tool unless an escape hatch is declared that takes any name and its arguments. Blanka declares every connected tool and has no directory; the table is the argument for building one, and the trap is the thing to build it with.

The smallest server that works

The round-trip test needs a partner, and a scripted one is more useful than a real one because it can misbehave on cue. The crate ships a second binary, forty lines, that reads

lines, answers the handshake, advertises two tools in the wrong order, writes a log line that is not JSON to standard output before anything else, sends a progress notification before every reply, and with one flag prints a complaint to standard error and exits before answering anything. The integration test spawns it by the path the build system exports, and every assertion in the test is a promise about the client: the tools come back sorted, the log line and the notification were skipped, an error result is a tool error, an unknown method is a protocol error, a slow reply hits the deadline and marks the server dead, a dying server is explained by its last line, and two servers connected together contribute two tools and one warning.

The fake is also the sketch of a server of your own. Everything a server has to do to be callable from a harness is in it: parse a line, dispatch on the method, answer with the same id, and flush. A server that answers one useful question about the reader's own project, bounded, is an afternoon on top of that.

Plugins as bundles

A plugin is a directory. In it, any of: a skills directory, an agents directory, a hooks file, a server configuration, and a settings file, plus a manifest with a name and a description. The manifest may be missing, in which case the directory's name is the plugin's name and the presence of any component is what makes it a plugin. Plank added the manifest and the single-directory load in one step (plank, 821d8e6) and the scan roots and the merge rules over the following weeks; blanka builds the finished shape.

```
pub struct Plugin {
  pub name: String,
  pub description: String,
  pub root: PathBuf,
  pub skills: Vec<Skill>,
  pub agents: Vec<AgentDef>,
  pub hooks: Vec<Hook>,
  pub mcp: Vec<McpServer>,
  /// The plugin's settings, before the refused sections are dropped.
  pub settings: Value,
  /// Complaints raised while loading this plugin.
  pub warnings: Vec<String>,
}
```

Each component goes through the loader chapter 6, chapter 12 or this chapter already wrote for the user's own files. A plugin's skills are loaded by the skill loader from the plugin's skills directory, its agents by the agent loader, its hooks by the hook parser. There is no second format and no second parser, which is what makes the namespacing rules the only new thing a plugin introduces.

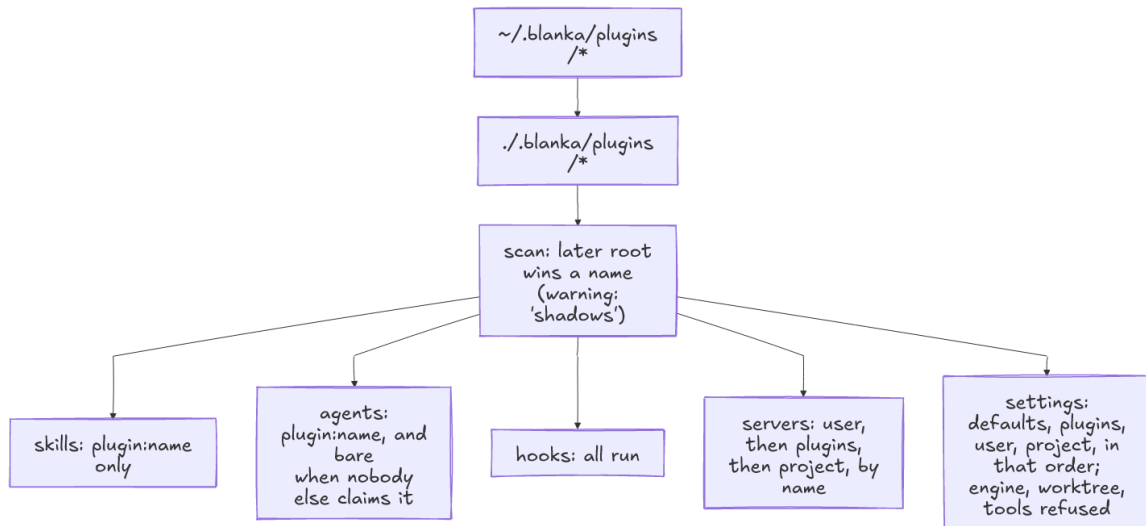
Precedence is the design problem

With one plugin the merge is trivial. With the user's directory, the project's directory, and two plugins each contributing a reviewer, the harness has to decide which reviewer the model gets, and it has to decide the same way every launch, and the decision has to be one the user can predict without reading the source. That is the design problem of

Chapter 13: Hooks, skills, MCP and plugins

plugins, and it has two halves: which plugin wins when two have the same name, and which entry wins when two plugins, or a plugin and the user, contribute the same skill or agent.

The first half is settled by the scan order:



<https://enzolombardi.net>

The plugin scan and what each component does with a collision. Two roots in a fixed order, then five rules, one per component, each stated in a sentence.

Roots are visited in order, the user's then the project's, subdirectories sorted by name within each, and a later plugin of the same name replaces an earlier one with a warning that names both directories. Plank has four roots, with fetched plugins below the user's own so that a plugin the user wrote outranks one that arrived from someone else's repository, and command-line directories on top of everything; blanka's two are the same idea with fewer origins. The warning is not optional: a plugin that silently replaced another is a plugin the user cannot find when its behaviour surprises them.

The second half is one rule per component, and the rules differ because the components differ in who types the name. Skills are namespaced only, for the reason the skills section gave: a skill is a command the user types, and the user's `/review` must never become someone else's. Agents get the namespaced name always and the bare name when no local definition and no other plugin claims it, because the model reads the roster and a bare name that resolves is worth having; a contested bare name is a warning and the namespaced forms. Hooks all run, because they have no names to collide on. Servers overlay by name in the order user, plugins, project, so a project can point a plugin's server at a different binary without editing the plugin. And settings layer with a refusal, which has a section of its own below.

Plank's skills gained a layer beneath all of these when it began to ship six built-in skills compiled into the binary. The built-ins are ordinary skill files parsed by the same loader as a user's, and the stack runs built-ins, then the user's directory, then the project's, each layer replacing the last by name, so a project that writes its own code-review skill replaces plank's rather than colliding with it. One detail of the wiring carries the section's

argument. The loader that reads a directory from disk stays disk-only, and the built-ins are seeded by the layered loader above it, because the plugin scan loads every plugin's skills through the disk loader, and seeding the built-ins there would have handed each plugin a copy of plank's own skills under the plugin's namespace (plank, 4a41fa5). Where a contribution enters the stack decides whose name it carries, and a layer added at the wrong level misattributes everything beneath it.

```
/// Agents from plugins join under `plugin:name`, and under the bare name too
/// when no local definition and no other plugin claims it. The model reads
/// the roster, so a bare name that resolves is worth having; a contested one
/// is a warning and the namespaced names.
pub fn merge_agents(mut local: Vec<AgentDef>, plugins: &[Plugin]) -> (Vec<AgentDef>,
Vec<String>)
```

Plank's version of this function is one generic merge with a flag saying whether plugin entries may take the bare name, false for skills and true for agents, and a comment explaining an invariant the flag depends on: a plugin name and an entry name may not contain a colon, so the only colon in the merged list is the one the merge inserts, which is why a check for "is this bare name already taken by a local entry" stays correct even though aliased copies of earlier plugin entries are being pushed into the same list as it runs. Blanka's two functions are that one function with the flag written out, and the same validation of names keeps the same invariant.

The test builds two roots with a plugin of the same name in each, a skill in both, an agent in one, and a directory with a readme that is not a plugin, and asserts the project's plugin won with a warning, that the user's own review skill kept its bare name beside the plugin's namespaced one, that an uncontested plugin agent has both names, and that a local agent with the same name keeps the bare one and pushes the plugin's to its alias with a warning.

Adapting a plugin written for another harness

Plugins for another harness exist in numbers, and the shape is close enough that plank chose to load them rather than define its own and ask authors to convert. Three differences remained, and each is a small lesson in what a contract is.

The first is spelling. The manifest lives in a directory named for the other harness, the hooks file lives one directory down and the events inside it are wrapped in an outer object named hooks. The manifest and file locations are two spellings each, tried in order. The wrapper is unwrapped in the hook parser, which accepts both shapes and warns about neither: a file that puts the events at the top level and a file that wraps them load to the same hooks, and the test asserts they do. Plank did the unwrapping at install time instead (plank, 65568c6), and its commit explains what happens if nobody does it: the runner sees a single top-level key named hooks, refuses the plugin as naming an event it does not implement, or under a force flag installs a file whose hooks can never fire because no event is found at the top level.

The second is the variable. The other harness's hook and server commands find files inside their own plugin through a variable its runner sets before exec, spelled as a shell

expansion in the command string. Blanka's runner clears the environment and plank's injects nothing, so the variable expands to nothing and a command like `{CLAUDE_PLUGIN_ROOT}/check.sh` becomes `/check.sh` and misfires without a word. Plank resolves the variable at install time, rewriting the two files whose contents become command lines and saying so in the install output (plank, 0b813d0). Its comment gives the reason it could not do it at load: plank's loader flattens every source's hooks into one list with no per-hook provenance, so injecting the right root at exec time would mean threading an owning directory through the hook types and the merge order, a change to load-bearing code for a problem the boundary can solve. The cost it accepts is that the installed tree differs from upstream and stops working if moved.

Blanka's loader has the provenance, because it loads each plugin's components inside a function that knows the plugin's root, and that is the moment to substitute:

```
/// The variable another harness's plugins use in their hook and server
/// commands to find their own directory. Its runner sets it; blanka's clean
/// environment does not, so the value is substituted at load time.
pub const FOREIGN_ROOT_VAR: &str = "${CLAUDE_PLUGIN_ROOT}";

/// Replace the foreign root variable in every string of a JSON tree with
/// this plugin's directory. Done on the parsed values, never on the text: a
/// root containing a quote spliced into the text would leave a file that
/// does not parse, and the hooks would vanish without a word.
pub fn substitute_root(v: &mut Value, root: &str) -> bool {
    match v {
        Value::String(s) if s.contains(FOREIGN_ROOT_VAR) => {
            *s = s.replace(FOREIGN_ROOT_VAR, root);
            true
        }
        Value::Array(items) => items.iter_mut().fold(false, |acc, i| substitute_root(i, root) | acc),
        Value::Object(map) => map.values_mut().fold(false, |acc, i| substitute_root(i, root) | acc),
        _ => false,
    }
}
```

The file on disk is never touched, so the plugin can be moved and still works, and a warning says what the variable resolved to so the user can see it happened. The substitution runs on parsed values, a detail plank's comment insists on for the reason in blanka's: a root containing a quote or a backslash spliced into JSON text is a file that no longer parses, and a hooks file that fails to parse is a plugin whose hooks vanish. Only the hooks file and the server configuration are touched, because those are the two whose contents become command lines; a skill or an agent is text the model reads, and rewriting a path into it would change what the model reads rather than what a subprocess runs.

The third is events. The other harness fires two, one for notifications and one when a sub-agent stops, that blanka has no equivalent of, and a config may also carry a typo or an event from a newer release. A hook under such a name would be installed and never

fire, which is a silent failure, so the parser reports every unknown event as a warning naming it and loads the rest. Plank refuses the install outright unless forced, and installs those hooks permanently inert under the force flag; blanka, loading rather than installing, cannot refuse and warns instead. The test writes a plugin in the other harness's layout with a wrapped hooks file using the variable, an unknown event, a server configuration using the variable in its arguments, and a settings file that tries to set the engine, and asserts one hook loaded with the root resolved, the server's command and argument both resolved, the unknown event warned about, and the engine setting still present on the plugin, waiting to be refused by the next section.

The fourth is the answer, and plank's record shows it surfacing only after installation looked complete. A widely used skills plugin written for the other harness bootstraps itself with a session-start hook that prints a JSON envelope whose one useful field carries context for the model. Plank's envelope reader knew the fields that halt a turn and message the user, and it injected standard output as context only when that output was not an envelope, so the bootstrap was parsed, recognised as an envelope and discarded whole (plank, 71d7882). The same hook named the sources it wanted in its matcher, startup, clear and compact, and plank's lifecycle events passed an empty target to the matcher, so a matcher naming anything matched nothing and the hook never ran (plank, 4ae5616). Neither failure produced a warning, because in each case the runner did exactly what its own contract said. The fixes were to read the context field under both harnesses' spellings and inject it on the events that accept context, and to pass each lifecycle event's own discriminator, the source of a session start or the trigger of a compaction, as the thing its matcher is tested against, while an empty matcher still matches everything. The evidence that both were needed was a headless run whose saved transcript finally carried the plugin's bootstrap message. The general form is worth keeping beside the event warning above: a foreign contract can fail by being understood too literally as well as by not being understood at all, and the only test that catches the first kind is one that runs the foreign plugin end to end and looks for its effect.

What installing a plugin means

A plugin's hooks run on the user's machine with the user's permissions, and a plugin's server is a binary the plugin points at. Loading a plugin is installing software, whatever the directory listing looks like, and the design has to treat it as such in two places.

The first is where plugins come from. Blanka loads what is on disk under two directories the user controls, and has no command that fetches one; putting a plugin there is an act the user performs with tools that show them what they are copying. Plank has the fetching command, and the length of its safety list is the measure of what the command takes on: only a manifest it recognises, refused if a symlink inside the tree points outside it, refused if a marketplace entry's source resolves outside the tree it was fetched with, staged outside the scan roots so a half-fetched plugin is never loaded, archives only over an encrypted connection except to the local machine, and a size cap on what it will unpack. Each of those is a way a directory from the network could do something the user did not ask for, and each was closed one at a time. A harness that fetches has to close

all of them; a harness that loads only what the user placed can leave them to the user's judgment and the tools they used to place it.

The second is what a loaded plugin may reach. Its hooks get the clean environment every hook gets, so a plugin's script does not see the user's exported tokens. Its settings cannot choose the engine, the tools or the worktree behaviour. Its skills are commands only under its own name. Its agent definitions can name a hosted engine and the name of a key variable, never a key, and a definition whose variable is unset is hidden from the model, so a plugin cannot cause a call to a provider the user did not configure. What a plugin can do is run a hook the user did not read and start a server the user did not inspect, and no rule in this chapter prevents that, because those are the two things a plugin is for. The listing command that shows every plugin's contributions and warnings exists so that the user can read what they installed, and the warnings about a refused setting or an unknown event are the harness saying, in effect, that this plugin expected something it did not get.

Settings, and what a plugin may not set

Settings are the preferences that outlive a run: whether reasoning is shown, whether the guards are on, how long a server may take to answer. Two files, the user's and the project's, the later winning key by key, and plugin settings underneath both:

```
/// Sections a plugin's settings may not touch.
pub const PLUGIN_REFUSED_SECTIONS: &[&str] = &["engine", "worktree", "tools"];
```

Underneath is not enough, and plank's comment on the same list says why in a sentence: a plugin layer sits below the user, but below is not enough for a key the user never set, because the plugin's value would then win by default (plank, 6bd220e). Most keys can be allowed to win by default; a plugin that turns off the display of reasoning is a plugin the user can overrule with one line. Which model runs, what a worktree symlinks in, and which tools exist are different: a plugin that could set them could set them to something the user never chose and never sees, and so those sections are dropped from a plugin's settings before the overlay looks at them, with a warning for each key that names the section and tells the user where to set it themselves if they want it.

```
pub fn merge(user_layers: &[Value], plugins: &[Plugin]) -> (Value, Vec<String>) {
    let mut out = defaults();
    let mut warnings = Vec::new();
    for p in plugins {
        let Value::Object(map) = &p.settings else { continue };
        let mut allowed = serde_json::Map::new();
        for (k, v) in map {
            if PLUGIN_REFUSED_SECTIONS.contains(&k.as_str()) {
                // one warning per refused key
                continue;
            }
            allowed.insert(k.clone(), v.clone());
        }
        overlay(&mut out, &Value::Object(allowed));
    }
}
```

```
}  
for layer in user_layers {  
    overlay(&mut out, layer);  
}  
(out, warnings)  
}
```

Plank records one more case that blanka inherits as a note rather than code. A safety setting whose default is "unset" rather than false is a setting a plugin wins by default without touching a refused section, and plank chose to honour it and make it visible with a warning, on the grounds that a project settings file already has the same power over the same key. The lesson is general: a layered configuration has to be read with the defaults in mind, because a layer below the user is a layer above the defaults, and any key the user has not written is a key the lowest layer that mentions it decides.

Secrets are not settings, and both harnesses say so in the module's first paragraph. The project file lives in the working tree and is committed by accident, so an API key has no settings key; it stays in the environment, named by a definition or a server configuration and never held by one.

The order everything is composed in

The startup of the binary is now a composition, and its order is fixed because each layer sits on the ones before:

- Scan the plugins, both roots. Everything else needs them.
- Load the user's two settings files and merge them over the plugins' settings with the refused sections dropped. Read the display and guard preferences and the server timeout from the result.
- Load the user's and project's skills, then the plugins' skills under their namespaced names, and build the tool table with the skill tool.
- Load the user's and project's server configurations, overlay the plugins' and the project's by name, connect every server with the configured timeout, and register the tools that came back. This happens before the agent exists, because the table renders into the prompt.
- Load the user's and project's hooks, then every plugin's, into one list.
- Load the agent definitions, merge the plugins' by the agent rule, and build the roster.
- Build the agent, hand it the hooks and the definitions, apply the guard setting.

Plank's listing commands also became switches. Its `/hooks` and `/skills` commands take `on` and `off`, which flip a process-wide gate for the rest of the session and write nothing to disk, so a restart returns to the configured behaviour (plank, 586c2df, 3fe503f). The gates are placed where the design of this chapter says they must be. The hook gate sits in the one function every event's runner funnels into, so all of plank's hook kinds are covered and a closed gate returns the same outcome as no hooks configured, which every caller already handles; the worktree hooks follow it, so git takes back the

worktree operations while hooks are off. The skill gate covers both routes a skill has, the command the user types and the tool the model calls, because gating only the command would leave skills running through the tool while the display said they were off. A launch flag goes further and loads no skills at all, built-in, user, project or plugin, which is what distinguishes it from an empty skills directory (plank, 03403d3). The switches exist for diagnosis: when a session behaves strangely, the fastest question is whether an extension is responsible, and a switch that answers it without editing a file the user will forget to restore is the cheapest instrument there is.

Every warning from every step is printed as it arises and kept for the listing commands, because a plugin that loaded with a complaint is a plugin the user will ask about later, and the `/plugins` command answers with the plugin's contributions and its warnings under it. The `/hooks` command lists every hook with its event, matcher and timeout; the `/mcp` command lists every configured server with the tools it contributed or the fact that it did not connect.

Tests across a process boundary

The hook tests run real shell commands. The payload test reads its own standard input with a pipeline and exits 2 if the tool name is missing, which it is not, and a second hook with a matcher for two tools warns on one and a third refuses another; the run for the first tool produces two results, no block and one warning, and the run for the second tool blocks with the third hook's text. The environment test asserts a variable the build system sets in the test process is absent in the hook while the event variable and the path are present, and then runs a thirty-second sleep under a one-second timeout and checks the call returned in under five. The parser test loads the same events with and without the foreign wrapper and asserts the hooks are equal.

The agent-level hook tests script a write that a pre-tool hook refuses by reading the payload, and check the tool error the model read and the file that does not exist, then a glob the matcher excluded; a post-tool hook that appends and a failing hook that only warns, with the warning on the sink and not in the result; and a turn-end hook that always objects, which produces two passes, one objection in the transcript, and a second answer that stood.

The MCP tests are the four integration tests against the fake server described above. The plugin tests are the two described in the precedence and adaptation sections, and the settings tests assert the user's value over the plugin's, the refused sections absent from the result with three warnings, an unknown section carried through, a plugin value applying when the user is silent, and a project value overlaying the user's key by key with the untouched keys surviving.

The smoke run

The run sets up a home with one plugin in the other harness's layout: a manifest, a wrapped hooks file with a post-tool hook on the write tool whose command uses the root variable, an unknown event, a skill, an agent, and a settings file setting the engine and the display. The home also has a hooks file refusing writes to a dotfile and a settings file setting the display back on, and the project has a server configuration naming the fake server twice, once normally and once with the flag that makes it die.

At startup two lines print: the plugin's engine setting refused, with the sentence about setting it yourself, and the dead server explained by its exit status and its usage line. The plugin listing shows one plugin contributing a skill, an agent, a hook and settings, with two warnings under it, the unknown event and the resolved root. The hook listing shows the user's pre-tool hook and the plugin's post-tool hook with its command resolved to an absolute path under the plugin. The server listing shows the live server with its two tools and the dead one as not connected. The skill listing shows the plugin's skill under its namespaced name only, and the agent listing shows the plugin's reviewer under both names.

A call to the server's echo tool returns its text over the pipe. A write to the dotfile is refused with the hook's sentence and the file is not created. A write to an ordinary file succeeds and its result carries the plugin's post-tool note appended. Each of those turns then repeats until the call guard of chapter 10 ends it, because the echo engine re-issues whatever it last heard, and the guard's refusals in the loop dumps sit beside the hook's refusal as two different sentences, which is as it should be.

Where blanka stops short of plank

Plank's hooks fire on a dozen events, including session start and end, prompt submit, post-compaction with the summary, and two that replace git as the worktree backend, and its hooks may be prompts rather than commands, may run asynchronously, and may answer with a JSON envelope that halts the turn or warns the user in addition to the exit code. Its matchers glob on arguments, and lifecycle events match on their own source or trigger. It reads a context field from a hook's envelope in two harnesses' spellings. Its hooks and skills can be switched off for a session, and a launch flag loads no skills at all. Blanka fires four events, all commands, all synchronous, exit codes only, and its matchers are tool names.

Plank's MCP client speaks a second transport over HTTP with session binding, lists and reads a server's resources, splits a server's tools into primary ones with full schemas and a directory with a describe tool and an escape-hatch call tool, keeps a failed global server's last advertisement as an offline shadow so the prompt bytes and their cache key do not move when a server fails to start, and keys project-local servers into the project tier of the cache so a project's servers do not fork the shared prompt tier. Blanka speaks stdio, declares every tool, and lets the prompt differ by configuration.

Plank fetches plugins from repositories, marketplaces and archives with a command, stages them outside the scan roots, checks their hook events and refuses or forces, and rewrites the root variable into the installed tree; it also loads command templates and a WebAssembly component kind, and takes plugin directories on the command line. Blanka loads what is on disk. Plank refuses a fourth settings section to plugins, the one holding the options the user sets for plugins, and audits each refused key into a warning at load. Plank's settings have a command that reads and writes them live, a provenance report saying which file set each key, and a documented list of which keys apply live and which are restart-bound because the system prompt they feed is cached. Blanka's settings are read once at startup.

Other people's text

Four mechanisms, two things. A hook is someone else's process: it gets a payload, a clean environment, a process group, a deadline and one exit code that means something, and it gets one objection per turn. An MCP server is someone else's process speaking a protocol: the client keeps a deadline over the whole exchange, keeps the server's last words, sorts what it advertises, names its tools so they cannot collide, and puts them in the table before the prompt is fingerprinted. A skill is someone else's text the user or the model invokes by a name that stays theirs. A plugin is a directory of all three plus settings, and what it adds is the question of precedence, answered by a fixed scan order and one rule per component, with three settings sections refused whatever the layer order says.

The worked example is the shape of every extension point. A plugin written for another harness arrives with its events one level too deep, a variable its own runner would have set, and two events the runner does not know. Loading it means unwrapping without teaching the runner about wrappers, substituting where the provenance is known, and warning about what will never fire. None of that touched the hook runner, the parser of the events it does know, or the loop, and that is the property to keep as extension points multiply: the foreign contract is adapted at the boundary, and everything inside it stays ignorant of the quirks of one of its inputs.

CHAPTER 14: SHIPPING A HARNESS

A harness that runs on its author's machine has had every problem of this book except one. The model file is already on disk because the author put it there. The binary was built from the working tree a minute ago. The cache directory was created by an earlier build and has been compatible with every build since, because the author moved the prompt and the blob format together, on purpose, and knew when they had done it. Nobody else has ever run it, so nobody else's machine has ever held a half-downloaded model, an older binary's cache, or a terminal that closed in the middle of a download. Handing the harness to someone else is the moment all of that stops being true at once.

Plank crossed that line early, and in the least forgiving shape a coding agent can take. Its model is a single file of about eighty-seven gigabytes, with a vision encoder and a speculative-decoding drafter beside it, and the harness cannot do anything useful until all three are present and match. It installs through Homebrew on machines its author has never seen. Its release cadence is several builds a week, some of them betas that change the system prompt, which chapter 8 established is the one change that invalidates every cached prefix. And some of its users drive it from scripts, not from a terminal, which means its exit status, its standard output and its behaviour under a signal are an interface as much as its prompt is.

The chapter follows those problems in the order a user meets them. First the files the harness needs beside itself, and how a harness learns there are newer ones, fetches them without trusting a half-finished download, and installs them without pulling a file out from under a running process. Then the version number, which plank turned from a label into an instruction. Then the engine moving to another machine, and one loaded model serving many sessions. Last, the parts of shipping that no unit test reaches: smoke tests against real providers, and the callers that are programs rather than people. Plank grows a manifest, a resumable downloader, an upgrade step, a server and a shared-engine host along the way.

A filename is not a contract

Plank's first answer to "is there a newer model" was to ask Hugging Face. It listed the repository's file tree, grouped the files into quantization families by stripping a four-digit date tag off each filename, and picked the newest commit within the family the user had installed. It worked, in the sense that it found newer files. Its own later documentation lists what was wrong with it. It made a filename convention into a wire contract: the day someone uploaded a file whose name did not follow the pattern, the grouping would silently put it in the wrong family or none. It only ever tracked the main model; the vision encoder and the drafter were pinned to constants compiled into the binary and were never upgraded at all. And the only integrity check was comparing the `Content-Length` the server reported with the size of the file on disk, which says a download finished and nothing about whether what finished is the right file.

The replacement is a small JSON document, `ds4.manifest`, committed at the root of plank's repository and fetched from it. It names each artifact by kind, with a URL, a size in bytes and a SHA-256, and it carries one integer, `version`. The module that reads it

Chapter 14: Shipping a harness

opens with the sentence that sums up the whole change: `version` is the entire comparison, and ordering by commit dates and reasoning about year boundaries stop being anyone's problem. A newer set of files is a larger number. Nothing about the files' names, dates or hosting is interpreted any more, which means the people who publish the files are free to rename them, move them, or split one into two, and the harness notices only because the number went up.

That shape is worth copying even for a harness with a much smaller model, and `blanka`'s is the same idea with one difference in layout. `Plank`'s manifest is a map from kind to entry; `blanka`'s is a list, which keeps the order the artifacts should be fetched in and makes a duplicated kind something the parser has to reject rather than something the format cannot express.

```
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct Manifest {
    pub version: u64,
    pub artifacts: Vec<Artifact>,
}

/// One file of the set. `kind` is what the harness calls it ("main",
/// "drafter"); it is also the file's name in staging and in the install
/// directory, so two kinds never collide.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct Artifact {
    pub kind: String,
    pub url: String,
    pub sha256: String,
    pub bytes: u64,
}
```

The kind doubles as a file name, and that single decision is why the parser is stricter than a JSON schema would be. A manifest is fetched from the network, and a kind of `../../../../zshrc` would turn the downloader into a way of writing anywhere the user can. So `Manifest::parse` refuses a kind that is not a plain word, a kind listed twice, the reserved name of the manifest itself, and a digest that is not sixty-four hexadecimal digits, and it refuses the whole document rather than the bad entry. Installing half of a set is worse than installing none, because the half that arrives will be loaded beside files from another release.

```
pub fn parse(text: &str) -> Result<Self, ManifestError> {
    let m: Manifest = serde_json::from_str(text).map_err(|e|
ManifestError::Invalid(e.to_string()))?;
    let mut seen = std::collections::BTreeSet::new();
    for a in &m.artifacts {
        let safe = !a.kind.is_empty() && a.kind.chars().all(|c| c.is_ascii_alphanumeric() || c
== '_' || c == '-');
        if !safe || a.kind == "manifest" {
            return Err(ManifestError::Invalid(format!("bad artifact kind {:?}", a.kind)));
        }
    }
}
```



```

        if !seen.insert(a.kind.as_str()) {
            return Err(ManifestError::Invalid(format!("artifact kind {:?} listed twice",
a.kind)));
        }
        if a.sha256.len() != 64 || !a.sha256.chars().all(|c| c.is_ascii_hexdigit()) {
            return Err(ManifestError::Invalid(format!("{}", sha256 is not a hex digest",
a.kind)));
        }
    }
    Ok(m)
}

```

The comparison that decides whether there is anything to do is one line, and it has one case worth a thought: no installed manifest means everything is new.

```

pub fn is_newer_than(&self, installed: Option<&Manifest>) -> bool {
    installed.is_none_or(|i| self.version > i.version)
}

```

Asking once a day

A manifest is small, but fetching it is a network request at startup, and a harness that makes one on every launch has a launch time that depends on the user's connection. Plank's answer is two limits. The fetch has a three-second timeout, so a slow or absent network costs at most three seconds and never blocks the prompt indefinitely. And it happens at most once every twenty-four hours: the time of the last successful check is recorded, and a launch inside the window reads the cached copy without touching the network.

Blanka's version separates the policy from the transport so that the policy can be tested without either a network or a clock. `fetch_manifest` always asks, over HTTP or from a `file://` path, which is what the tests and an air-gapped install use. `check` is the wrapper startup calls: it takes the cache directory, the current time in seconds and the `fetch` as a closure.

```

pub fn check<F>(cache: &Path, now: u64, fetch: F) -> Result<Manifest, ManifestError>
where
    F: FnOnce() -> Result<Manifest, ManifestError>,
{
    let (doc, stamp) = cache_paths(cache);
    let cached = std::fs::read_to_string(&doc).ok().and_then(|t| Manifest::parse(&t).ok());
    let checked: Option<u64> = std::fs::read_to_string(&stamp).ok().and_then(|s|
s.trim().parse().ok());
    if let (Some(m), Some(at)) = (&cached, checked) {
        if now.saturating_sub(at) < CHECK_INTERVAL_SECS {
            return Ok(m.clone());
        }
    }
    match fetch() {

```

Chapter 14: Shipping a harness

```
Ok(m) => {
    std::fs::create_dir_all(cache).map_err(|e|
ManifestError::Transport(e.to_string()))?;
    crate::downloader::write_atomic(&doc, m.to_json().as_bytes()).map_err(|e|
ManifestError::Transport(e.to_string()))?;
    crate::downloader::write_atomic(&stamp, now.to_string().as_bytes()).map_err(|
e| ManifestError::Transport(e.to_string()))?;
    Ok(m)
}
Err(e) => cached.ok_or(e),
}
}
```

Two details carry the design. A failed fetch falls back to the cached copy, because a harness that is offline should still know what it last knew. And a failed fetch does not advance the stamp, so the next launch tries again rather than waiting a day to discover that the network came back. The test moves the clock instead of waiting for it:

```
#[test]
fn a_cached_manifest_younger_than_a_day_is_not_refetched() {
    let d = dir("cache");
    let calls = Cell::new(0);
    let fetch = |v| {
        calls.set(calls.get() + 1);
        Ok(sample(v))
    };
    let t0 = 1_800_000_000;
    assert_eq!(check(&d, t0, || fetch(1)).unwrap().version, 1);
    // An hour later, and again just short of a day: the cache answers.
    assert_eq!(check(&d, t0 + 3_600, || fetch(2)).unwrap().version, 1);
    assert_eq!(check(&d, t0 + CHECK_INTERVAL_SECS - 1, || fetch(2)).unwrap().version,
1);
    assert_eq!(calls.get(), 1);
    // A day on, the network is asked and the newer set is seen.
    assert_eq!(check(&d, t0 + CHECK_INTERVAL_SECS, || fetch(2)).unwrap().version,
2);
    assert_eq!(calls.get(), 2);
}
```

The day the manifest shipped, plank also had to answer a question that only arises on the day a mechanism replaces another one, which is what to do for the users who already have the files. They had downloaded eighty-seven gigabytes under the old scheme and had no installed manifest, and by the rule above, no installed manifest means everything is new. Plank adopts on first sight: when no manifest has been recorded and every artifact on disk already has the size the manifest names, the manifest is recorded as installed and nothing is offered. The rule is narrow on purpose, a size match on every kind, and it exists for exactly one population, the installs that predate the manifest. It is the kind of migration a harness only needs once, and a harness that forgets it greets its most loyal users with an offer to re-download what they have.

A download that outlives the harness

An artifact of eighty-seven gigabytes takes hours on an ordinary connection. A download that runs inside the harness has to survive the user quitting it, which they will do, and a download that blocks the prompt makes the harness unusable for the afternoon. Plank's downloader is therefore a separate process: the same binary, started as `plank --model-downloader`, detached from the terminal, with a lock file taken with `flock` so that at most one runs per machine however many plank instances noticed the new manifest at once.

Being detached shapes everything about how the two processes talk. The harness that started the download may exit at any moment, and a new one may start and want to show progress, so there is no in-memory channel between them. Progress is a small JSON state file the helper rewrites as it goes, carrying its process id so that a reader can tell a helper stalled on a slow socket from one that died. Cancellation is a flag file the helper polls between chunks, not a signal, because a signal needs a live sender that knows the receiver's process id, and the harness that wants to cancel may be about to exit or may never have known which process was downloading. A file works from any process, at any time, including one that writes it and quits.

Blanka keeps the three primitives and leaves the progress file out. The cancel flag is two functions:

```
/// Whether someone asked the download in `dir` to stop. A file rather than a
/// signal, so a harness that is about to exit can still ask.
pub fn cancel_requested(dir: &Path) -> bool {
    dir.join(CANCEL_FILE).exists()
}

/// Ask a running download in `dir` to stop at its next chunk.
pub fn request_cancel(dir: &Path) -> io::Result<()> {
    std::fs::create_dir_all(dir)?;
    std::fs::write(dir.join(CANCEL_FILE), b"")
}
```

The detached start is `blanka download <manifest-url>` launched in a process group of its own, with the three standard streams closed. The process group is what matters. A terminal that closes sends its hangup to the foreground process group, and a child that shares the harness's group dies with it; a child in its own group does not. The standard library has had `process_group` on Unix since Rust 1.64, so no `unsafe` is needed:

```
pub fn spawn_detached(exe: &Path, manifest_url: &str) -> io::Result<u32> {
    use std::os::unix::process::CommandExt;
    use std::process::{Command, Stdio};
    let child = Command::new(exe)
        .arg("download")
        .arg(manifest_url)
        .stdin(Stdio::null())
```

```
.stdout(Stdio::null())
.stderr(Stdio::null())
.process_group(0)
.spawn()?;
Ok(child.id())
}
```

Blanka does not take a lock, and the gap section says what that costs: two harnesses launched in the same minute would start two downloads into the same staging directory. The fix is the one plank has, a lock file taken before the first byte, and it is left out only because blanka's artifacts are small enough that a user who manages to start two will lose a few minutes rather than an afternoon.

Hashing while it streams

The downloader verifies every artifact against its SHA-256, and it has to compute the hash as the bytes arrive. Hashing afterwards would mean reading the whole file a second time, which for eighty-seven gigabytes is minutes of disk time spent after the user has already waited hours. Plank's findings record that `sha2` is a direct dependency of the whole project for this reason alone; everything else in plank that needs a digest shells out to `shasum`, which cannot hash a file that is still being written a megabyte at a time.

Streaming the hash creates a problem at the moment a download resumes. The hash state of the interrupted attempt lived in the memory of a process that no longer exists, and `sha2` has no serializable state to have saved it with. A resumed download therefore reads the existing `.part` file back through a fresh hasher before it fetches a single new byte, and plank's findings add an instruction worth keeping verbatim in spirit: do not add a hasher-state sidecar. A sidecar would be a second source of truth about the same bytes, one that can disagree with them after a crash, and the digest over what is actually on disk is the only one worth comparing.

Blanka's `stage` does exactly that, and the order of its checks is the design:

```
fn stage_in_chunks(a: &Artifact, staging: &Path, chunk: u64) -> io::Result<PathBuf> {
    std::fs::create_dir_all(staging)?;
    let done = staging.join(&a.kind);
    let part = staging.join(format!("{ }.part", a.kind));
    if done.exists() && std::fs::metadata(&done)?.len() == a.bytes {
        return Ok(done);
    }
    let mut hasher = Sha256::new();
    let mut have = Ou64;
    if let Ok(mut f) = File::open(&part) {
        let len = f.metadata()?.len();
        if len <= a.bytes {
            have = io::copy(&mut f, &mut HashWriter(&mut hasher))?;
        }
    }
    if have == 0 {
        // Nothing usable on disk, or a `.part` longer than the artifact:
    }
}
```

Chapter 14: Shipping a harness

```
// start clean rather than trust it.
File::create(&part)?;
}
let mut out = OpenOptions::new().append(true).open(&part)?;
while have < a.bytes {
    if cancel_requested(staging) {
        return Err(io::Error::new(io::ErrorKind::Interrupted, "download cancelled"));
    }
    let end = (have + chunk).min(a.bytes) - 1;
    let bytes = fetch_range(&a.url, have, end)?;
    if bytes.is_empty() {
        return Err(io::Error::new(io::ErrorKind::UnexpectedEof, format!("{}", server sent
nothing at {have}", a.kind)));
    }
    hasher.update(&bytes);
    out.write_all(&bytes)?;
    have += bytes.len() as u64;
}
out.sync_all()?;
let digest = hex(&hasher.finalize());
if !digest.eq_ignore_ascii_case(&a.sha256) {
    // A wrong file is worse than no file: throw it away so a retry
    // starts from zero instead of resuming onto bad bytes.
    let _ = std::fs::remove_file(&part);
    return Err(io::Error::new(io::ErrorKind::InvalidData, format!("{}", sha256 {digest},
expected {} ", a.kind, a.sha256)));
}
std::fs::rename(&part, &done)?;
Ok(done)
}
```

A verified file is renamed from `<kind>.part` to `<kind>`, and only a file that has passed the digest ever carries the bare name. A file that fails the digest is deleted rather than kept for a resume, because resuming onto bad bytes produces the same bad digest forever. The cancel check sits at the top of each chunk, so a cancel costs at most one chunk's wait and leaves the `.part` on disk for a later resume.

The test for the resume is the one the plan for this chapter named, and it is stated as the property that matters: a download interrupted partway and resumed produces the same bytes, and therefore the same digest, as one that ran without interruption.

```
#[test]
fn a_resumed_part_produces_the_same_digest_as_a_fresh_download() {
    let d = dir("resume");
    let a = source(&d, "main", 10_000);
    // A fresh download, in small chunks so the loop runs many times.
    let fresh = stage_in_chunks(&a, &d.join("fresh"), 1_000).unwrap();
    // An interrupted one: the first 3,500 bytes are already on disk.
    let staging = d.join("resumed");
```

Chapter 14: Shipping a harness

```
std::fs::create_dir_all(&staging).unwrap();
let all = std::fs::read(fresh.clone()).unwrap();
std::fs::write(staging.join("main.part"), &all[..3_500]).unwrap();
let resumed = stage_in_chunks(&a, &staging, 1_000).unwrap();
assert_eq!(std::fs::read(resumed).unwrap(), all);
assert!(staging.join("main.part").exists());
}
```

The interrupted `.part` ends at 3,500 bytes, which is not a chunk boundary, on purpose: a resume that only worked from chunk boundaries would pass a test that interrupted at one.

Ranges that name their end

The resume logic above has a dependency that is easy to leave implicit: the server has to agree to send bytes from the middle of the file. HTTP says how to ask, with a `Range` header, and the common way to ask for "everything from here on" is an open range, `bytes=3500-`. Plank's downloader asked that way, and for its first artifact set it worked.

When plank added the DeepSeek V4.1 set, whose main artifact is 341 gibibytes, the same request came back `400 Bad Request`. The findings record the probe that established why, against the live artifact: a bounded range such as `bytes=100-199` answered `206 Partial Content`, the open range `bytes=100-` answered `400`, and a quarter-gigabyte bounded range answered `206` with exactly the length asked for. The V4.1 files were served by a different storage backend from the V4 files, and that backend refused open ranges while the older one accepted them. Two reviews of the fix contradicted each other for exactly that reason: one had probed a V4 object from the old manifest and concluded that open ranges were fine. The finding's instruction is general enough to belong to any harness that downloads anything: always probe the artifact you actually ship.

The consequence was worse than a failed resume. Before the fix, an interrupted download could not resume at all, so the whole `.part` rehash machinery was dead weight, and one real interruption cost eighty-eight gibibytes of transfer. The fix names the end offset on every request, which is correct against both backends and so is done unconditionally, and fetches in chunks of 256 mebibytes. That changes what end of stream means: the end of a response is the end of a chunk, not the end of the file, and the loop continues until the byte count says the file is complete.

Blanka's `fetch_range` asks for bounded ranges only, and treats a `200` as acceptable only for the first chunk of a file, because a server that ignores ranges sends the whole file from byte zero, which is only usable if byte zero is what was wanted:

```
let mut resp = ureq::get(url)
    .header("range", &format!("{}", bytes={start}-{end}"))
    .call()
    .map_err(|e| io::Error::other(e.to_string()))?;
let status = resp.status().as_u16();
// A server that ignores ranges answers 200 with the whole file; that is
// only usable for the first chunk of a file that fits in one.
if status != 206 && !(status == 200 && start == 0) {
```

Chapter 14: Shipping a harness

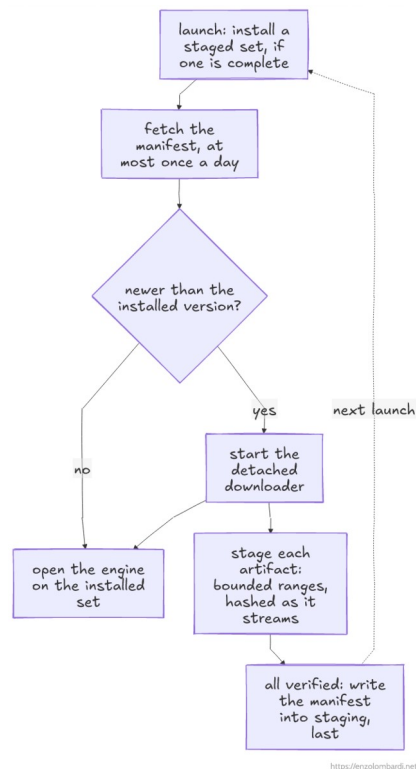
```
return Err(io::Error::other(format!("{url}: status {status} for a range request")));
}
let mut buf = Vec::with_capacity(want as usize);
resp.body_mut().as_reader().take(want).read_to_end(&mut buf)?;
Ok(buf)
```

The `take(want)` is the other half of the rule. A server that answered a bounded request with more than was asked for would otherwise have its extra bytes appended to the file, and the digest would catch it only after hours of download.

Installed at the next launch

A verified artifact is not installed when it finishes. It waits in the staging directory until the next time the harness starts, and that delay is not a matter of caution. A running plank has its eighty-seven-gigabyte model memory-mapped. Replacing the file underneath the mapping would be, depending on how it was done, either impossible or a way to have the running process read pages of the new model through a mapping of the old one. The only moment at which no process is using the files is before any process has opened them, which is the start of the next launch.

The flow a user sees is therefore in two halves, and the chart below shows where each step happens:



Staging runs beside the harness; installing waits for the next launch, before the engine opens the files

Blanka's startup does the same in the same order: install whatever is staged, then check the manifest, then open the engine. The install step has to come first because the engine

will open the files, and a check that ran before the install could decide that a set already sitting complete in staging needed downloading again. So `models_at_startup` also compares the newest manifest with the one in staging, and starts no download for a set that is already staged and waiting.

The manifest moves last

Installing a staged set is a sequence of renames, one per artifact and one for the manifest, and the order of those renames is the one rule in the whole subsystem that must never break. The installed manifest is the claim that the set is installed. If it moves first and the machine loses power after it, the install directory holds a new manifest over a mixture of old and new files, and the harness believes it has a matched set. Plank's findings describe what that looks like from the user's side: a main model, a vision encoder and a drafter that can be a year apart in build, and a load error from the engine that is cryptic about why. Moved last, an interruption leaves the old manifest in place, or none, and the next launch finishes the job.

Finishing the job needs one more property: an artifact that was already moved has to be recognizable as moved. Blanka's swap checks each kind in turn. A kind still in staging is renamed into place; a kind missing from staging but present in the install directory at the right size was moved by an earlier, interrupted swap and is left alone; a kind in neither place is an error, and the manifest stays where it is.

```
fn swap_with(staging: &Path, install: &Path, after_move: &mut dyn FnMut(&str) ->
io::Result<()>) -> io::Result<bool> {
    let staged_doc = staging.join(MANIFEST_FILE);
    let Ok(text) = std::fs::read_to_string(&staged_doc) else {
        return Ok(false);
    };
    let m = Manifest::parse(&text).map_err(|e| io::Error::new(io::ErrorKind::InvalidData,
e.to_string()))?;
    std::fs::create_dir_all(install)?;
    for a in &m.artifacts {
        let from = staging.join(&a.kind);
        let to = install.join(&a.kind);
        if from.exists() {
            std::fs::rename(&from, &to)?;
            after_move(&a.kind)?;
        } else if !std::fs::metadata(&to).is_ok_and(|md| md.len() == a.bytes) {
            return Err(io::Error::new(io::ErrorKind::NotFound, format!("{}", a.kind): neither staged
nor installed", a.kind)));
        }
    }
    std::fs::rename(&staged_doc, install.join(MANIFEST_FILE))?;
    Ok(true)
}
```

Chapter 14: Shipping a harness

The `after_move` hook exists for one caller, the test. The public `swap_staged` passes a hook that does nothing; the test passes one that fails after the first rename, which is the closest a test can come to pulling the power cord at the worst moment:

```
#[test]
fn a_swap_interrupted_after_the_first_move_leaves_no_manifest_installed() {
    let d = dir("swap");
    let (staging, install, m) = staged_set(&d);
    let mut moves = 0;
    let crash = swap_with(&staging, &install, &mut |_| {
        moves += 1;
        if moves == 1 { Err(io::Error::other("power cut")) } else { Ok(()) }
    });
    assert!(crash.is_err());
    assert!(install.join("main").exists());
    assert!(!install.join(MANIFEST_FILE).exists(), "the manifest moved before the set
was complete");
    // The next launch finishes the job, and only then claims the set.
    assert!(swap_staged(&staging, &install).unwrap());
    assert_eq!(crate::manifest::installed(&install), Some(m));
    assert!(install.join("drafter").exists());
}
```

Reverse the manifest rename and the artifact loop and the second assertion fails, which is the whole reason to write the test that way. The same order holds in staging: `stage_all` writes the manifest into the staging directory only after every artifact has verified, so the presence of a staged manifest means a complete set is waiting, and a swap that finds none has nothing to do.

More than one set

Plank now serves three model families, the DeepSeek V4 and V4.1 sets and a Qwen set, all running on DwarfStar, the native inference engine plank links, and each has its own manifest, its own staging directory and its own install location. The separation looks like tidiness and is in fact the same invariant applied one level up. The manifest moving last makes a swap safe for one set. A shared staging area would let a half-staged download of one family be read as proof about another, because the proof is the presence of a manifest in a directory. Plank's module comment says so in as many words, and a test asserts that no two sets share a path, over a list of every set that is itself checked against the enum, so that adding a family without adding it to the test is a compile-time mismatch rather than a narrowed test.

The families also differ in what they contain, and the manifest absorbs the difference without a special case. The V4 set has three kinds; V4.1 has two, because its upstream documentation states that the drafter is not implemented for it. A manifest may name kinds a build does not know, which are carried through and ignored, so a future release can add a fourth artifact without breaking every client that predates it. Blanka's parser keeps unknown kinds too, by the simpler route of not having a list of known ones: it stages whatever the manifest names.

A derived model without a second download

One more artifact shape arrived late and is worth describing because it bends the rule about what a model is. Plank can load a `.ggd` file, a weight delta: the byte-wise difference between a derived checkpoint and its base, for instance an edit that removes a refusal direction from a few dozen tensors. Given as the model, the delta is applied to an APFS clone of the base file, made with `clonefile`, patched in place under plank's own directory, and the patched clone is what the engine opens. The clone costs disk only for the blocks that changed, about 1.2 gigabytes for one such derived checkpoint against its eighty-seven-gigabyte base, and the original files are never written.

The findings record why the delta could not be applied in memory, which is the obvious design: DwarfStar maps the model read-only and wraps slices of the mapping directly as Metal buffers, with no hook to substitute bytes and a comment explaining that the mapping flags were chosen around an operating system bug. They also record a compression measurement that generalizes: the raw bytes of an edited eight-bit tensor deflated to 92 percent of their size, while the byte-wise difference from the base deflated to 37 percent, because most weights moved by zero or one. Diff, then compress. And they record a decision that looks like a bug and is not. The engine reports the same model name for the base and the derived weights, so the two share every cached prefix. Chapter 8 insisted that a model's identity belongs in every cache key; plank accepts the sharing on purpose, on the argument that an edit this small is within the noise the model already tolerates from quantization and that re-reading the system prompt per variant would cost more than it buys. The note ends by telling the next reader not to fix it by changing the name without also deciding that the split cache is worth it.

Blanka does not load deltas. The pattern it would need is the one plank used: the delta is resolved into an ordinary model file before anything reads a model header, so every later layer sees a model and never a delta.

The patch number is the channel

Plank ships through Homebrew only, as two formulas. Every release whose patch number is zero, `v5.2.0`, is stable, and updates the stable formula; every release with a patch above zero, `v5.2.1`, `v5.2.2`, is a beta, and updates the beta formula. There is no channel flag anywhere in the release automation. When a release is published, the workflow reads the tag's patch component and routes the build to the matching formula, and the binary knows the rule too, showing `BETA` in its version banner when its patch is above zero. The number is the channel.

Promotion follows from the rule. Promoting the betas of a series closes it by tagging the next minor twice from the same code: `.0` as the new stable, built fresh from its own tag, and `.1` as the first beta of the new series, so both channels restart aligned and the beta users move onto the new series with nothing to reinstall. A stable hotfix cannot be a patch bump, because a patch above zero is the beta channel by definition; it is a fresh `.0` on the next minor, cut from the stable tag, with the fix cherry-picked forward into the next beta. Majors are never bumped by automation. They are a manual decision, taken when older on-disk state must not be trusted at all.

Blanka's repository carries the same scheme in a `VERSIONING.md` beside its crate, and the scheme is worth adopting even for a harness with one user, because of what the other two components are then free to mean.

A version number that does things

A binary has no downstream compiler to reassure. Nothing links against a harness, so the contract semantic versioning offers, that a major bump means a breaking API, has no reader. Plank, and before it another tool by the same author, repurposes the empty space: the version number is a message from the release to the binary that runs next, and the transition between the version that last ran and the one running now says what maintenance the new binary must do on its first launch.

Plank's upgrade step reads a marker file in its home directory and classifies the transition. A patch needs nothing, and the marker advances. A minor drops the system prompt's cached state so that it is rebuilt at the next warm-up. A major, a downgrade or a missing marker drops that and the image cache as well. Session transcripts are user data and are never removed, whatever the number does. And plank's versioning document is explicit about the division of labour underneath: the cached prompt state carries a fingerprint of the prompt text and is rebuilt whenever the fingerprint does not match, so a stale checkpoint is never trusted, and the version-driven drop is an optimization that keeps dead bytes off the disk rather than the mechanism that guarantees correctness. That is why a beta that changes the system prompt is safe even though it ships as a patch and triggers no cleanup: the fingerprint catches it, and the next promotion's minor bump clears the dead blob once for everyone.

Blanka's `upgrade.rs` is the same classification with blanka's own cleanup. A minor bump is where the model-facing contract moves, because a change to the static prompt bumps `SYSPROMPT_VERSION` from chapter 6 and a change to the blob layout bumps `FORMAT_VERSION` from chapter 8, and either ships on a minor; so a minor drops the static-prefix blobs. A major drops every blob. The classifier is a comparison of three numbers, and its test reads as the table in the versioning document:

```
pub fn classify(previous: Option<&str>, current: &str) -> Transition {
    let (Some(p), Some(c)) = (previous.and_then(parse), parse(current)) else {
        return Transition::Major;
    };
    if c < p || c.0 != p.0 {
        Transition::Major
    } else if c.1 != p.1 {
        Transition::Minor
    } else if c.2 != p.2 {
        Transition::Patch
    } else {
        Transition::Same
    }
}
```

`#[test]`

```
fn each_component_classifies_its_own_cleanup() {
    assert_eq!(classify(Some("0.4.2"), "0.4.3"), Transition::Patch);
    assert_eq!(classify(Some("0.4.9"), "0.5.0"), Transition::Minor);
    assert_eq!(classify(Some("0.9.3"), "1.0.0"), Transition::Major);
    assert_eq!(classify(Some("0.5.1"), "0.5.0"), Transition::Major, "a downgrade trusts
nothing");
    assert_eq!(classify(None, "0.5.0"), Transition::Major);
    assert_eq!(classify(Some("0.5.0"), "0.5.0"), Transition::Same);
}
```

A downgrade is classified as a major because the older binary cannot know what the newer one wrote. The marker advances after the cleanup even when a removal failed, for a reason the other tool learned first: a file that cannot be deleted now will not become deletable later in the same upgrade, and retrying at every launch would turn a one-time cleanup into a permanent one. The cost of the scheme is real and worth naming. A glance at two version numbers no longer says whether features or fixes landed between them; a release with a dozen features that touches neither the prompt nor the blob format is a patch. What the scheme buys is an upgrade the user never has to think about.

The engine on another machine

The engine trait has hidden where tokens come from since chapter 1, and the last step it makes possible is putting the engine on a different machine from the harness. The case is ordinary: the model needs a large machine and the user works on a small one, or a team shares one machine that holds the weights. Plank designed two flavours of remote engine and built the first as `plank serve`. Its design document names the insight that made it cheap: the client is a dumb transport and the server is the local engine behind a socket. The prompt bytes, the tool-call markup and the cache discipline all stay on the machine that holds the model, unchanged, so there is no protocol drift and no second rendering of the prompt to keep in parity. A dropped connection is safe by construction, because the next pass resends the whole rendered prompt and the server's engine reuses whatever common prefix it still holds, exactly as an in-process turn would.

Blanka's server follows the same shape, written on `std::net` with a thread per connection, because the engine contract is synchronous and a runtime would buy nothing. Three endpoints cover the trait: `GET /info` for the model and the context size, `POST /tokenize` so the client's token counts come from the server's tokenizer rather than an estimate, and `POST /generate`, which answers with a stream of server-sent events. The stream is the chunks the engine produced, each as an event, and then exactly one terminal event, a `Done` carrying the accounting or an `Error` carrying the message.

```
/// Streams each chunk as an event. A write that fails means the client went
/// away, which is how an interrupt reaches the server: the flag is raised and
/// the engine stops at its next poll.
struct EventSink<'a> {
    stream: &'a TcpStream,
    interrupt: InterruptFlag,
}
```

Chapter 14: Shipping a harness

```
impl EventSink<'_> {
    fn send(&mut self, e: &WireEvent) {
        let line = format!("data: {}\\n\\n", serde_json::to_string(e).unwrap_or_default());
        let mut s = self.stream;
        if s.write_all(line.as_bytes()).and_then(|()| s.flush()).is_err() {
            self.interrupt.request();
        }
    }
}
```

Interruption crosses the wire without a message of its own. When the user interrupts on the client, the client stops reading and drops the connection; the server's next write fails, and the failed write raises the interrupt flag the engine polls between tokens. Plank's server has an explicit cancel endpoint as well, keyed by a client id, because its shared mode runs several clients' passes and needs to cancel one of them by name; for one pass at a time, the closed socket is the whole protocol.

The terminal event is where chapter 1's lesson returns. A hosted provider's stream that failed after its 200 OK was read as an empty answer, and the turn ended as though the model had chosen to say nothing. Blanka's client does not repeat the mistake with its own server: a stream that ends without a Done is a transport error, and the text received before the cut is not presented as an answer.

```
pub fn read_events<R: BufRead>(reader: R, sink: &mut dyn Sink, interrupt:
&InterruptFlag) -> Result<(String, WireStats), EngineError> {
    let mut text = String::new();
    for line in reader.lines() {
        if interrupt.is_set() {
            let stats = WireStats { prompt_tokens: 0, completion_tokens:
approx_tokens(&text), interrupted: true };
            return Ok((text, stats));
        }
        let line = line.map_err(|e| EngineError::Transport(e.to_string()))?;
        let Some(payload) = line.strip_prefix("data:") else {
            continue;
        };
        let event: WireEvent = serde_json::from_str(payload.trim()).map_err(|e|
EngineError::Protocol(format!("{e}: {payload}")))?;
        match event {
            WireEvent::Text(t) => {
                sink.text(&t);
                text.push_str(&t);
            }
            WireEvent::Done(stats) => return Ok((text, stats)),
            WireEvent::Error(e) => return Err(EngineError::Transport(format!("server:
{e}"))),
        }
    }
    Err(EngineError::Transport("the stream ended before the server said it was
```

Chapter 14: Shipping a harness

```
done".into()))  
}
```

The client itself, `RemoteEngine`, connects by asking for `/info` and refuses a server that speaks a different protocol version rather than guessing. Its `count_tokens` asks the server and degrades to the four-bytes-per-token estimate when the request fails, because accounting can be rough and a turn should not die of it. It keeps the trait's defaults for warming and snapshots, which describe an engine with no memory, and that is honest: the memory is on the server, and the client has none to warm. The test runs the server on a free loopback port with the echo stub and checks the one property that matters, that a remote pass returns what the engine would have returned locally, and then that the prompt crossed the wire intact, by letting the stub's fallback echo it back:

```
#[test]  
fn a_remote_pass_returns_what_the_engine_would_have_returned_locally() {  
    let reply = "The file has three functions; the second one is unused.";   
    let url = start(EchoEngine::scripted(vec![reply]), None);  
    let mut remote = RemoteEngine::connect(url, None).unwrap();  
    assert_eq!(remote.ctx_size(), EchoEngine::new().ctx_size());  
    let mut sink = Collect::default();  
    let msgs = vec![Message { role: Role::User, content: "look at lib.rs".into() }];  
    let (text, stats) = remote.generate(Prompt::Structured(&msgs), &mut sink,  
    &InterruptFlag::new()).unwrap();  
    assert_eq!(text, reply);  
    assert_eq!(sink.0, reply);  
    assert!(stats.interrupted);  
    // Once the script is spent the echo stub answers from the prompt,  
    // which proves the prompt crossed the wire intact.  
    let (echo, _) = remote.generate(Prompt::Structured(&msgs), &mut Collect::default(),  
    &InterruptFlag::new()).unwrap();  
    assert_eq!(echo, "echo: look at lib.rs");  
}
```

Who may connect

A server that drives a model is a server that drives whatever the client does with the model's output, and a coding agent's client runs shell commands. The exposure is therefore not only the model's compute. Plank's server compares its bearer token in constant time before it reads any request body, answers 413 to a body larger than sixteen mebibytes before buffering a byte of it, gives every connection a thirty-second read timeout so a peer that sends half a request and stalls holds a thread and nothing more, and refuses to start on a non-loopback address without a token unless the operator passes `--insecure` explicitly. An address it cannot parse is treated as exposed, so a typo errs toward refusing.

Blanka keeps each of those except the escape hatch. The token comes from `BLANKA_SERVE_TOKEN`, the same variable the client reads, and without one only a loopback address is served:


```
pub fn serve<E: Engine + Send + 'static>(engine: E, bind: &str) -> io::Result<()> {
    let token = std::env::var(TOKEN_VAR).ok().filter(|t| !t.is_empty());
    if token.is_none() && !is_loopback(bind) {
        return Err(io::Error::new(io::ErrorKind::PermissionDenied, format!("refusing to
serve {bind} without {TOKEN_VAR}: bind 127.0.0.1 or set a token")));
    }
    let listener = TcpListener::bind(bind)?;
    serve_on(engine, listener, token)
}
```

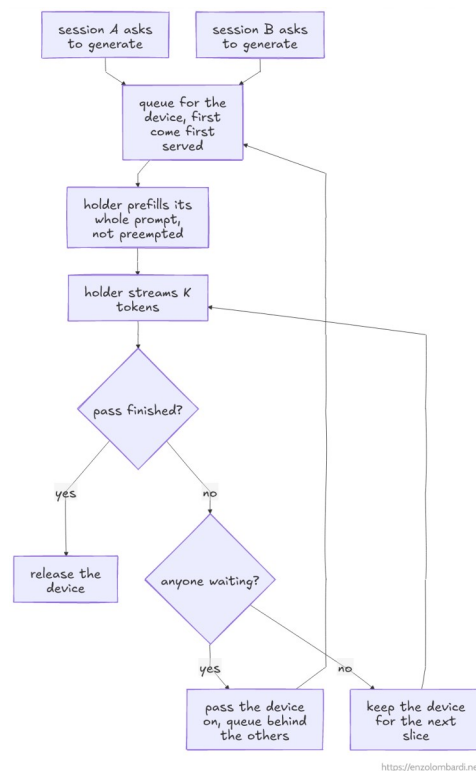
The comparison runs to the end of the token whatever it finds, so the time a refusal takes says nothing about how much of the token was right. None of this is transport security. A token sent over plain HTTP can be read by anyone on the path, and the right deployment for a server off the local machine is behind a tunnel or a proxy that terminates TLS. The server's job is to refuse the obvious mistakes, not to replace one.

One model, many sessions

A server with one engine behind a mutex serves one pass at a time, which matches one user and fails the first time two people share a machine. Plank's design for the general case starts from an observation about its engine that its own documentation calls the load-bearing change. The engine struct conflated two things: the model, meaning the weights, the tokenizer and the Metal command queue, immutable after loading and about eighty-two gigabytes resident; and the session, meaning the cached state of one conversation and its cursor. DwarfStar's C interface never forced them together; creating several sessions against one loaded engine was always supported, and plank's single session was a choice. Splitting the two into separate Rust types was worth doing on its own, because per-session state, instant session switches and sub-agent sidechains all wanted the same split, and once it was done the shared engine was a small step on top.

The shared model lives behind an `ARC`, whose strong count is the reference count: the last session to detach, after the host, frees the model and its Metal context, and there is no manual count to get wrong. The harder constraint is the device. One command queue runs one computation at a time, and two threads calling into the engine concurrently would be unsound as written and would buy no throughput if made sound. So concurrency is time-sliced, not parallel. Plank runs every engine call on one dedicated thread and interleaves sessions cooperatively at token granularity: each runnable session gets a slice of a few tokens, streams them to its client, and cedes to the next. Prefill is not preemptible in the first version: a session being admitted reads its whole prompt before rotation resumes, which the design document flags as the place a very large cold prefill can starve the others. The following chart shows one rotation:

Chapter 14: Shipping a harness



Round-robin at a K-token slice: fairness over latency, with total throughput fixed by the one queue

Two more pieces make it practical. A new session does not re-read the shared system prompt: the host warms it once, captures the state as an immutable snapshot, and restores the snapshot into each new session, which costs a buffer copy rather than a prefill. And admission is controlled. Each session's cached state grows with its context, so resident memory grows linearly in the number of sessions on top of the one-time weights, and the host refuses a session past a configured count or an aggregate byte budget with an error rather than letting the machine run out of memory later and worse. In plank all of this sits behind a `--shared-engine` flag that is off by default; the split itself is always present, and the single-owner path is a session that happens to be the only one on a model loaded for it.

Blanka's engines hand back a finished pass rather than exposing a token loop, which chapter 11 noted when it declined the mid-pass suspend, so blanka's host cannot interleave inside an engine the way plank's scheduler does. It can interleave at the one place a blanka engine shows its progress, which is the sink. Each session owns its own engine, cloned from the warmed one the host holds, which is blanka's analogue of restoring the warm snapshot into a fresh session. What the sessions share is the device, represented by a baton: a fair, first-come lock that at most one session holds while its engine computes.

```
/// Pass the baton on if anyone is waiting, and queue up behind them.
/// Returns whether it was passed.
fn yield_if_waited(&self, ticket: u64) -> bool {
    if self.waiting() == 0 {
```

Chapter 14: Shipping a harness

```
        return false;
    }
    self.release();
    self.acquire(ticket);
    true
}
```

A session's `generate` takes the baton, hands its engine a sink that counts streamed chunks, and every `slice_tokens` chunks offers the baton to whoever is waiting. The engine does not know any of this happened: its sink call took longer than usual, because the session was queued behind another one while it waited.

```
impl Sink for SlicingSink<'_> {
    fn text(&mut self, chunk: &str) {
        self.inner.text(chunk);
        self.count += 1;
        if self.count % self.every == 0 && self.baton.yield_if_waited(self.ticket) {
            self.yielded += 1;
        }
    }
}
```

This is cooperative scheduling in the literal sense. It works because every engine blanka has streams in pieces and calls the sink between them; an engine that computed a whole reply and then emitted it in one chunk would hold the device for the whole pass, and the host would be fair only between passes. Admission is a compare-and-swap on the live count, and the refusal is the test the plan asked for:

```
#[test]
fn the_host_refuses_the_session_past_its_limit() {
    let host = EngineHost::new(EchoEngine::new(), 2, 8);
    let a = host.session().expect("first");
    let _b = host.session().expect("second");
    assert!(host.session().is_none(), "max_sessions + 1 was admitted");
    assert_eq!(host.refcount(), 3);
    // Detaching frees a place.
    drop(a);
    assert_eq!(host.live_sessions(), 1);
    assert!(host.session().is_some());
}
```

A second test holds the baton while two sessions queue behind it, releases it, and asserts that each session passed the device on at least once and that both replies arrived whole: slicing changes when text is computed, never what it is. A session is itself an `Engine`, so the server of the previous section can front a host as easily as a single engine, and an `Agent` runs on a session unchanged.

Tests that need a real provider

Everything so far is covered by tests that run in milliseconds against the echo stub and a temporary directory. A shipped harness also depends on things no unit test reaches, and plank keeps a document of manual smoke tests for them. Its introduction states the boundary well: the unit suite covers the logic against the stub with no network, and what it cannot cover is a real provider's wire format, real latency, real rate limits, and whether the model actually chooses to delegate. The document is a matrix. A local main agent with a local sub-agent, a local main agent with a sub-agent on a hosted provider, a hosted main agent with a sub-agent on another provider, and the fourth quadrant, a hosted main agent with a sub-agent on the local model, which costs the full local residency and is opt-in by one explicit spelling so that nobody pays eighty gigabytes of memory by leaving a field empty. Each cell has steps, the expected output, and the behaviours of particular providers worth provoking on purpose, such as two keys on two accounts and a failure in the middle of a stream.

The practice is less glamorous than a test suite and more honest about what it tests. A smoke test is run by a person, against credentials, before a release that touched the code it covers, and its value is in being written down: the next release is tested the same way, and a regression shows up as a step whose output no longer matches.

Callers that are not people

A harness that is useful gets driven by scripts: continuous integration asking it to fix a lint, a benchmark running it a hundred times, an editor plugin sending it one prompt. Plank's headless modes grew out of those callers. With a prompt given by `-p`, `--ui console` runs one turn and prints the answer; `--ui chart` silences standard output for the turn and paints only a live throughput chart on the terminal; `--ui quiet` prints a few lines of notes instead. Without `-p`, the console mode speaks a line protocol on standard input for a driver that holds a conversation. The interface a script sees is the exit status and the output streams, and plank's exit statuses were sharpened by exactly that audience: a one-shot run whose turn a loop guard stopped exits 3, distinct from 1 for plank failing and from the 124 that `timeout` returns on expiry, so a caller can tell the three apart.

The sharpest lessons came from a benchmark harness that runs plank against itself, a matrix of models, prompts and iterations, and they are lessons about any program that drives a harness. The benchmark's own bookkeeping files first sat in the model's working directory, and a setup phase dutifully surveyed them and wrote an instruction file about "a transient benchmark scratch directory" instead of about the code; the model's tree and the harness's metadata belong in sibling directories. The benchmark measured the author's machine rather than the prompt, because plank reads skills, hooks, memory, server configurations and settings from its home directory, and every one of them reached the model, down to a refused write into another project whose path had come from one server's instructions; the fix runs plank under a scratch home holding the model files and nothing else. A run killed by `timeout` left nothing behind, because the default signal ends plank without saving, while an interrupt makes it save its transcript and write its repro dump like any other exit, so the harness now sends the interrupt first and a kill after a grace period. And a leftover plank from an earlier run held the model

Chapter 14: Shipping a harness

lock and failed every run behind it, forty-five of them in one session, with the reason visible only inside a per-phase log; the benchmark now refuses to start while another instance holds the lock.

Two further findings are small and general. `timeout` does not exist on a stock macOS, and a wrapper that assumed it recorded every run as a failure while enforcing no timeout at all. And a program that exits zero after an interrupt looks like a success to anything that only reads the status; plank's repro dump records the outcome, and the benchmark reads that rather than the exit code. Blanka has no headless mode, and the gap section lists it; the lessons are here because they are the shape of the interface a script will expect, and they are cheaper to design in than to discover.

The smoke run

The chapter's code can be exercised without a model. Start a server on the echo stub with `blanka serve`, which binds `127.0.0.1:7878` by default, and in another terminal start `blanka --remote http://127.0.0.1:7878`: every prompt crosses the wire, and the echo stub's answers come back through the same renderer as a local engine's. For the artifacts, write a manifest whose URLs are `file://` paths to a couple of files, set `models.manifestUrl` in `~/.blanka/settings.json` to it, and launch twice. The first launch starts `blanka download` in the background and reports it; the files appear in `~/.blanka/staging` with their `.part` suffix while they stream and without it once verified, and the manifest arrives in staging last. The second launch reports that it installed the staged set, and `~/.blanka/models` holds the files and the manifest. Bumping the manifest's version starts the cycle again; interrupting the download by creating `~/.blanka/staging/cancel` leaves the `.part` files for the next attempt.

Where the harness ends

Plank began as a port of the agent that ships with DwarfStar, and the chapter on tool calls drew the line that has held since: the wire format and the prompt text are what the model was trained on, and everything else is engineering. Fourteen chapters later the line is in the same place, and it is worth saying what sits on each side of it, because that is where the harness ends.

DwarfStar owns the weights format, the tokenizer, the kernels, the model's chat template and the byte sequences the model reads as structure. Plank does not reinterpret any of them, and its parity test checks, against committed fixtures and against the reference's source when it is present, that the tool framing, the tool-call syntax and the system prompt are byte for byte the reference's. When the reference changes a spelling, as V4.1 did with its tool-call tags, plank follows. The port owns everything else, and everything else turned out to be most of the work: the transcript and its persistence, the tool table and its containment, the display, the tiers and the snapshot ladder that make a local model affordable over a long session, compaction, loop guards, concurrency, delegation, extensions, and the whole of this chapter. None of those are in the reference, because the reference is a demonstration that the model can drive tools, and a demonstration does not need to survive a user.

Where blanka stops short of plank

Plank's downloader publishes its progress in a state file with the helper's process id, so any plank can show a progress bar and tell a stalled download from a dead one; blanka's downloader reports nothing while it runs. Plank takes a lock so that one downloader runs per machine; blanka does not, and two launches in the same minute would stage into the same directory twice. Plank adopts an existing install on first sight when every artifact already matches; blanka has no installs that predate its manifest to adopt. Plank's cancel command can keep or discard the partial files and offers the download again the next day; blanka's cancel keeps them and leaves the retry to the next launch that sees a newer manifest. Plank keeps separate manifests, staging areas and install locations per model family; blanka has one set. And plank resolves weight deltas into patched clones before any model header is read; blanka does not load deltas.

Plank's server has an explicit cancel endpoint keyed by a client id, sweeps idle shared sessions after thirty minutes, and an `--insecure` flag for an operator who wants a token-less server off loopback; blanka cancels by closing the socket, has no idle sweep, and has no escape hatch. Plank's shared host runs every engine call on one thread and slices inside the token loop, restores a warm snapshot into each new session, and admits by an aggregate byte budget as well as a count; blanka slices at the sink, clones a warmed engine, and admits by count only. Plank's upgrade step also drops an image cache on a major. Plank's releases are built and published by workflows from a tag; blanka's `VERSIONING.md` describes the scheme and leaves the automation to the reader's own continuous integration. And plank has the headless modes, the exit statuses a script can read, and a benchmark harness that runs the whole thing a hundred times; blanka's front ends are the line REPL and, from the appendix, a full-screen one.

Handed over

Shipping a harness is the work of making it survive machines and moments its author never sees. Model files are described by a manifest whose one integer is the whole comparison, fetched with a short timeout at most once a day, validated before a byte is trusted, and adopted without a download for the installs that predate it. They are fetched by a helper that outlives the harness, cancelled by a file rather than a signal, hashed as they stream, resumed by re-reading what is on disk, and requested in ranges that name their end because one storage backend refused the open kind and cost a user eighty-eight gibibytes before anyone probed the artifact actually shipped. They are installed at the next launch, because the running process has them mapped, and the manifest moves last, because its presence is the proof that the set landed. The version number is a message to the next binary, with the patch as the channel and the minor as the place the model-facing contract moves, and a fingerprint behind it catching whatever the number misclassifies. The engine can move to another machine behind a dumb transport, and one model can serve many sessions by sharing the weights, cloning the warm state and taking turns on the device. The rest is written down rather than tested: smoke runs against real providers, and the exit statuses and signals a script will lean on. What the reference owns stays byte for byte the reference's; everything else is the harness, and by now it is most of the program.

CONCLUSION

The harness at the end of the book is small enough to read in an afternoon and large enough to do real work. Blanka holds a conversation with any engine behind one trait, parses the model's tool calls strictly and answers the broken ones in words the model can act on, runs six tools behind a single write-containment check, and streams its output through a renderer that knows the difference between what the model said and what it thought. It saves sessions under memorable names with atomic writes and a sweep that respects what is in use. It separates its prompt by rate of change, fingerprints each tier, and keeps a ladder of snapshots so that a rewrite behind the live end restores instead of rebuilding. It compacts in two steps, notices when the model is going around in circles, keeps the prompt live while a pass runs, delegates to sub-agents on forks of the session, lets hooks, skills, MCP servers and plugins into the loop in a fixed order of precedence, and ships its artifacts with a manifest that moves last. Every one of those claims has a test that would fail if the claim stopped being true, and most of those tests run in milliseconds against an engine made of counters.

None of it is plank, and the difference is worth stating one last time, because it is the difference between a teaching codebase and a harness someone uses every day.

What blanka is, next to plank

Blanka is the core of each idea, cut down to the part that can be explained in a chapter and pinned by a test. Plank is the same ideas after a few months of contact with one model, one engine and one user. It is more than a hundred and fifty thousand lines of Rust, and nearly all of the difference is response to a specific session going wrong in a specific way: a parser tolerance for a misspelled marker the model produced twice, a guard rung cut to the shape of a review written inside the think block, a sandbox root added because a build tool writes a cache outside the project, a rule about which side of a truncation a lookup has to sit on. Each chapter ended with an inventory of those additions, and the inventories were not apologies. They were the record of what a harness accumulates when it is used, and a reader's own harness will accumulate a different set, in response to a different model's habits.

What the two share is the shape. The loop that generates, dispatches and feeds back is the same loop. The trait boundary around the engine is the same boundary, and the fact that blanka's tests run unchanged against a llama.cpp engine in the appendix is the evidence that it was drawn in the right place. The one rule of the ladder, that a rung is looked up under the fingerprint of the transcript truncated to its own depth, is the same rule, and it fails the same silent way when it is reversed. A reader who has built blanka can open plank's source and recognise every subsystem, which is the purpose of having built it.

What the reference still owns

Plank began as a port of a C reference agent, and the port drew one boundary more sharply than any other. The text the model reads is not the port's to change. The syntax of a tool call, the framing of a tool result and the words of the system prompt are what the model was trained on, and plank carries them byte for byte, enforced by a fixture test

Conclusion

that diffs its output against the reference. When the reference moved, when a new model family respelled the tool syntax or the system prompt gained a line, plank followed, and the cost was measured in regenerated fixtures and a rebuilt cache, never in an argument about whether the new text was better.

The engine is the reference's too. Plank calls into it through a thin foreign interface and treats its behaviour as a fact to be measured rather than a design to be improved: which paths chunk their prefill and keep a partial checkpoint on cancel, which families can rewind a session and which invalidate it, how a model name is reported, what a think prefix contains at each effort level. Several of the book's findings are statements about the engine that were only true once someone dumped the tokens or read the C, and the habit worth keeping is that one: a claim about prompt bytes is settled by looking at the bytes.

Everything else belongs to the port. Sessions, the cache discipline, the loop guards, concurrency, delegation, memory, extensibility and shipping have no counterpart in the reference, or only a small one, and they are where plank spent most of its effort. That split is not specific to plank. Whatever engine and model a reader's harness talks to, the model-facing text is a contract owned by whoever trained the model, and the rest is engineering owned by whoever writes the harness.

What has not settled

The book is published while parts of its subject are still moving, and the parts that move fastest are the youngest. Decisions read directly from the logprobs of a handful of answer letters, without generating a token, arrived while the later chapters were being drafted. Plank's first use of them sits in front of its memory extraction pass, asking one bit about each turn before paying for a full reading, and it ships switched off: the model's bias toward one answer letter has been measured and corrected, but the threshold has not been checked against real outcomes, and the gate currently rejects nearly every span it is shown. Prompt suggestions, generated off a session that is already warm, went through a day of fixes before they stopped leaking reasoning, commands and stale guesses into the input line. Both are described as they stand, and both will change. Other uses of the same primitive, a judge in front of network access and a compaction pass that asks of each old tool result only whether it is still needed, are designs rather than code.

The measurements are in the same position. The loop guards were tuned against a corpus of recorded failures, and the corpus contains the failures the guards could see. The draft rung's thresholds were set against two evenings of sessions. The cost of a memory pass, a yield under memory pressure or a rebuilt static tier depends on the machine, the model and the length of the conversation. Every number in the book is given with the session or the dump it came from, so that a reader can tell a measured figure from a general claim, and every one of them is a candidate for revision when a better measurement arrives.

Where to go from here

A reader who has built blanka has three obvious directions, and each of them starts from something the book deliberately left open.

Conclusion

The first is the tool table. Blanka's six tools are the minimum a coding agent needs, and a real project will want others: a language server, a test runner that reports structure rather than text, a database client, a domain tool nobody else has. Chapter 3's pattern applies to each of them unchanged. Resolve every write through the one containment check, bound every result, write the description for the model rather than for the reader of the code, and treat the first session in which the model misuses the tool as the specification of its next revision.

The second is the engine. The appendix on `llama` shows the trait holding up against a real local model, and the same trait will take a different runtime, a different model family, or a hosted provider with its own cache semantics. The work there is measurement: run the three reruns of chapter 8 against the new engine, find out where its time goes, and decide which of the cache layers earns its complexity on that hardware. On a provider with a generous prompt cache, most of Part II may be unnecessary. On a small local model, all of it will be.

The third is the model's habits. Every model loops in its own way, drafts in its own place, misspells its own markup and ignores its own subset of the prompt. The guards, the parser tolerances and the prompt rules in this book were cut to one model's behaviour, and a harness built for another will need its own. The instrument for finding them is the one the book used throughout: a repro dump written whenever something goes wrong, a table of passes that says what each generation did, and the discipline of reading the dump before changing the code.

The program around the model

A language model produces text, and everything a coding agent appears to do is done by the program around it. Blanka is that program in its smallest honest form, and plank is the same program after months of use. The model-facing text belongs to whoever trained the model; the sessions, the cache, the guards, the concurrency, the delegation and the extensions belong to the harness, and on a local engine they decide whether the agent is usable at all. Some of the newest parts of the subject, decisions without decoding above all, have not found their final shape, and the online edition of this book will follow them as they do. What a reader takes away is a harness that works, a test for every claim it makes, and a way of finding out what the next model will need.

APPENDIX A: A FULL-SCREEN TERMINAL FRONT END

The line REPL that blanka has used since chapter 4 is honest about what it is. Output scrolls up the terminal and is gone from view, the prompt is whatever line the cursor happens to be on, and the only status is a bracketed note printed between two replies. It is also the front end every test and every pipe can drive, which is why the book kept it. A harness people use for hours wants more from its screen: the conversation where it can be scrolled back, a prompt that stays in one place while the model writes above it, and a line of status that says what the harness is doing now rather than what it did a paragraph ago.

Chapter 4 designed for this moment without building it. The stream renderer never writes to a terminal; it classifies text into visible output, reasoning, tool banners, errors and status notes, and hands each to a `RenderSink`. Chapter 11 moved the agent onto a worker thread whose only link to the front end is a channel of events. A full-screen front end is therefore a second sink and a second event loop, and nothing below them changes: the same worker runs the same turns, the same slash-command handler answers the same commands, and the same tests keep passing. This appendix builds blanka's version with Ratatui, behind a `tui` feature so that the default build stays small, and uses plank's much larger one to show where a full-screen display spends its effort once the basics are right.

The second sink

The sink is the smallest part. It receives the five channels the renderer produces and appends each to a log, with one rule: streamed chunks of visible text or reasoning join the entry they continue, because the model's reply arrives in pieces and reads as one block.

```
impl RenderSink for TuiSink {
    fn visible(&mut self, text: &str) {
        self.log.push(Kind::Visible, text);
    }
    fn thinking(&mut self, text: &str) {
        if self.show_thinking {
            self.log.push(Kind::Thinking, text);
        }
    }
    fn tool_banner(&mut self, text: &str) {
        self.log.push(Kind::Banner, text.trim());
    }
    fn error(&mut self, text: &str) {
        self.log.push(Kind::Error, text.trim());
    }
    fn status(&mut self, text: &str) {
        self.log.push(Kind::Status, text.trim());
    }
}
```

Appendix A: A full-screen terminal front end

The sink does not run on the worker. The worker keeps the `ChannelSink` of chapter 11, which turns each call into an event, and the full-screen loop replays those events into its `TuiSink`. That split is what keeps the display responsive: the worker never waits for a frame to be drawn, and a frame never waits for a token. Hidden reasoning is dropped at the sink rather than at the worker, so the switch that shows it can change without the worker knowing, and the transcript, which the worker owns, holds the reasoning either way.

The log is drawn, not printed

A line REPL prints each piece of text once and forgets it. A full-screen front end redraws the whole screen many times a second, and what it draws is a window onto a log that only grows. The obvious implementation of the window is to wrap the whole log at the terminal's width and take the last screenful, and plank's findings record exactly where that goes. In long sessions mouse selection became sticky. Every frame was proportional to the whole transcript: it cloned every line of the log, cloned it again to count wrapped rows, and re-wrapped each line above the viewport one at a time. A mouse drag emits an event per cell of movement, and each one redraws, so the cost showed up as lag on the one interaction that redraws fastest. At forty thousand lines a release build spent seconds per frame re-wrapping text that was not on screen. The fix caches each line's wrapped height by width, extends the cache lazily, cuts it back wherever the log is truncated, and serves the frame from it, cloning only the lines at or below the viewport.

Blanka's log is far smaller, and it takes the cheaper half of the same idea: wrap only what can reach the screen. The window walks the entries backwards from the end, wraps each, and stops as soon as it has enough rows for the viewport plus the scroll offset:

```
pub fn window(&self, width: usize, rows: usize, scroll: usize) -> Vec<Line<'static>> {
    let want = rows + scroll;
    let mut tail: Vec<Line<'static>> = Vec::new();
    for (kind, text) in self.entries.iter().rev() {
        let mut block = Vec::new();
        let mut fence = false;
        for raw in text.split('\n') {
            let styled = style_line(*kind, raw, &mut fence);
            block.extend(wrap(styled, width.max(1)));
        }
        block.extend(tail);
        tail = block;
        if tail.len() >= want {
            break;
        }
    }
    let end = tail.len().saturating_sub(scroll);
    let start = end.saturating_sub(rows);
    tail[start..end].to_vec()
}
```

Appendix A: A full-screen terminal front end

The frame at the bottom of a conversation of any length costs the same few entries. Scrolling up costs proportionally to how far up the user went, which is the cost plank's cache removes and blanka accepts. The wrap is done by hand rather than by Ratatui's paragraph wrapping, for the reason that makes scrolling exact: the window has to know how many screen rows each entry occupies before it decides which entries to draw, and a widget that wraps as it renders cannot say.

The test pins the tail and the scroll together:

```
#[test]
fn the_window_is_the_tail_and_scrolls_up_from_it() {
    let mut sink = TuiSink::default();
    for i in 0..100 {
        sink.status(&format!("line {i}"));
    }
    let text = |s| sink.log.window(20, 3, s).iter().map(|l| l.to_string()).collect::<Vec<_>>();
    assert_eq!(text(0), vec!["line 97", "line 98", "line 99"]);
    assert_eq!(text(5), vec!["line 92", "line 93", "line 94"]);
}
```

Reading markdown as it streams

The model writes markdown, and a screen that shows the asterisks and backticks raw is readable the way source code is readable. Blanka's styling is a line-at-a-time reading that needs no parser: headings bold, fenced code in a colour and indented, inline code highlighted, everything else plain. The only state carried from one line to the next is whether a fence is open, and it resets per entry, so a reply that opens a fence and never closes it colours its own tail and nothing after it.

```
if raw.trim_start().starts_with("`") {
    *fence = !*fence;
    return Line::from(Span::styled(raw.to_string(), Style::default().fg(Color::DarkGray)));
}
if *fence {
    return Line::from(Span::styled(format!("{}", raw),
    Style::default().fg(Color::LightBlue)));
}
```

Plank renders real markdown and highlights code with tree-sitter, and that choice brings a cost blanka's line reading does not have. Parsing and highlighting a growing code block on every token would recompile the highlight on every token, so plank re-renders a live markdown segment at most once every hundred milliseconds and defers the tokens in between. Its commit log records the bug that throttle produced. The deferred tokens were committed only by the next piece of visible text or by the segment closing, and a pass that stops writing prose to open a tool-call stanza does neither: the last words of a sentence stayed off screen for as long as the tool ran, and then the sentence completed itself when the tool's result closed the segment. The fix commits a due deferred render on the draw clock, once per frame, for the main log and every side panel.

Appendix A: A full-screen terminal front end

The general form is worth keeping: a throttle that defers work must have a clock of its own that eventually flushes it, or the deferred part waits on an event that may not come.

The same highlighter paints plank's diff cards, the before-and-after rendering of an edit. The language is chosen from the file's extension only, and an extension that is not on the list falls back to flat red and green rows, on the grounds that a wrong grammar looks worse than none. Each side of a hunk is reassembled into one buffer and highlighted whole, so a block comment spanning several rows is coloured consistently, and the highlight ranges are clipped back onto each row afterwards.

Two rows of status

Under the log sit two rows of status. Blanka's first row says where and on what, the directory and the engine; the second says what is happening now, how many prompts are queued for the next boundary, and what the last turn cost. Both are computed by a pure function that takes a small struct and a width, so they are tested without a terminal and cut to the width before they are drawn:

```
pub fn status_rows(s: &StatusInfo, width: usize) -> [String; 2] {
  let first = format!("{}", s.dir, s.engine);
  let mut second = s.state.to_string();
  if s.queued > 0 {
    second.push_str(&format!("{}", s.queued));
  }
  if let Some(r) = &s.last {
    second.push_str(&format!("{}", r.last_turn {} pass(es), {} tool round(s), {}↑ {}↓",
r.passes, r.tool_rounds, r.stats.prompt_tokens, r.stats.completion_tokens));
  }
  [first, second].map(|row| row.chars().take(width).collect())
}
```

Plank's footer is where most of its display work ended up, because it is where every subsystem of this book reports itself. Read left to right it carries the directory and git branch; a context gauge showing tokens used against the window, which opens the context report when clicked; a count of running background jobs from chapter 11, which opens the jobs panel; the task counter from chapter 7; a power suffix when the machine is throttled; and a marker when a remote controller is attached. Transient marks join it as features arrive: a writing hand while a memory pass takes notes at idle, a floppy disk while the model streams weights from the disk, a brain that toggles whether reasoning is shown, and a camera at the end of the directory prefix that writes a repro dump of the moment it was clicked, added because a hand-saved dump was the only record of a loop no guard fired on, and typing a command in the middle of a stall is the last thing a user does.

What the footer no longer carries is as instructive as what it does. It used to hold every number a turn produced: a spinner and a verb, then elapsed time, prefill and generation token counts and their rates, and the per-step figures of speculative decoding, beside the micro-compaction and loop-guard switches and an idle word. All of it changed while a turn ran, so the one row meant to be read at a glance was the row that never held still.

Appendix A: A full-screen terminal front end

Plank moved the transient figures to the right end of the rule under the prompt, the way the session name floats on the rule above it; the line pinned under the output kept only the throbber, the verb and the clock; the two switches left for their slash commands, and the idle word went because the prompt caret already says so. The move exposed a second width bug of the same family as the one below. When a turn ended, its figures were cleared from the rule, and a label glyph the terminal draws wider than the renderer measured left a blank notch where they had been, because the frame diff compared cells by the renderer's measurement and believed one of them unchanged. The fix rewrites that row whole when the worker ends, and only that row, since repainting the full screen flickers.

A footer that dense has to fit, and fitting it exposed a measurement bug that any status bar with symbols in it will meet. Plank measured the footer's width by counting characters after stripping colour codes, which gave every emoji one column instead of the two a terminal draws, so on a narrow terminal it picked a footer that fitted by its count and overflowed on screen. Measuring in display columns fixed most of it, and two symbols were still wrong: the thermometer and the wastebasket are emoji whose default presentation is text, so the width table reports one column for the bare code point while terminals draw both in colour at two. Appending the variation selector that requests emoji presentation brought the measurement in line with what terminals do, and a test now pins the width of every footer mark by referring to the real constants. Blanka's status rows count characters and use one symbol, the middle dot, which is one column everywhere; the day a reader adds an emoji to them, the counting has to change first.

The cursor is drawn

The prompt is a one-line editor at the bottom of the screen. Its cursor position is counted in characters rather than bytes, so an accented letter moves it by one, and it is drawn as a reversed cell rather than requested from the terminal:

```
pub fn line(&self) -> Line<'static> {
    let at = self.byte_at(self.cursor);
    let (before, rest) = self.text.split_at(at);
    let mut chars = rest.chars();
    let under = chars.next().map_or(" ".to_string(), String::from);
    Line::from(vec![
        Span::styled("> ", Style::default().fg(Color::Green)),
        Span::raw(before.to_string()),
        Span::styled(under, Style::default().add_modifier(Modifier::REVERSED)),
        Span::raw(chars.as_str().to_string()),
    ])
}
```

Plank arrived at drawing its cursor for a reason that is instructive about terminals in general. It had coloured the terminal's own cursor with an escape sequence, and one widely used terminal ignores that sequence, because it draws its cursor as a widget of its own interface rather than as a cell of the grid. There is no error, only silence, which the finding calls the worst failure mode for an indicator, because it works on the author's machine. Plank now paints the cursor itself, after the text so that the painter can read the

Appendix A: A full-screen terminal front end

cell's colour and width, and it records two consequences that are easy to break. Hidden is not absent: terminals anchor the input-method candidate window and screen-reader focus to the position of the cursor they track, so the hidden cursor is still moved to the caret after each draw, and skipping that breaks input in languages that compose characters with nothing visibly wrong. And the caret is process-global state, so every test that draws a prompt takes a lock, acquired at the top of the test and never inside a shared helper, because the lock is not reentrant and a helper that takes it deadlocks the suite instead of failing it.

A prompt that is always live

The event loop has two sources, the worker's events and the keyboard, and it interleaves them on a short poll. It drains every pending worker event into the sink, draws a frame, and waits up to thirty milliseconds for a key. What a key does depends on whether the worker is busy, and the routes are the ones chapter 11 gave the line REPL, because a line typed during a turn means the same thing whichever front end it was typed into: a side question goes to the aside queue, a slash command waits for idle, and anything else joins the conversation at the next boundary.

```
if busy {
  // The same three routes as the line REPL: an aside, a
  // command for idle, or a prompt for the next boundary.
  if let Some(q) = line.strip_prefix("/btw ") {
    let _ = worker.inbox.push_aside(q.trim().to_string());
  } else if line.starts_with('/') {
    pending_slash.push_back(line);
  } else {
    worker.inbox.push_queued(line);
  }
  continue;
}
```

Raw mode changes one thing about interruption. The terminal no longer turns Ctrl-C into a signal, so the interrupt flag of chapter 4 is never set by the handler it installed; the key arrives as an ordinary key event instead, and the loop raises the flag itself while a turn runs, and clears the prompt line when none does. Escape interrupts as well, and Ctrl-D on an empty prompt asks to quit, which waits for the worker to become idle so that a turn in flight is not cut off by a keystroke meant for the prompt. Plank draws the same line more finely than blanka: Ctrl-D quits during a background memory pass, because nothing the user asked for is lost, and stays inert during a turn.

Menus that stay out of the way

Typing a slash opens a small menu of the commands the input is a prefix of, and Tab completes to the first. It opens only while the input is a lone word starting with a slash, and it closes the moment a space is typed:

```
pub fn slash_matches(input: &str) -> Vec<&'static str> {
  if !input.starts_with('/') || input.contains(' ') {
```

Appendix A: A full-screen terminal front end

```
    return Vec::new();
  }
  COMMANDS.iter().copied().filter(|c| c.starts_with(input) && *c != input).collect()
}
```

Plank's menus taught two rules that blanka's is small enough to satisfy by construction and a larger one would not. The first came from history recall. Both of plank's key loops re-targeted the completion menus after every key, Up and Down included, so recalling a line that started with a slash opened the command menu on the recalled text; the menu sees keys before the history handler and took Up and Down for its own selection, and walking back through history stalled on the first slash command it reached. A recall now raises a flag that closes the menus instead of re-targeting them, because a recalled line is not something the user is composing. The second came from a file-mention popup with no matching rows. It drew nothing, so from the user's side it was not open at all, but it still sat in front of every key, and the first Enter after a fruitless mention was swallowed to close a popup that was never on screen. Every key but Escape now falls through a popup with no rows. The rule both findings share is that a popup's claim on the keyboard must be no larger than what it shows.

Ghost text

Chapter 11 described how plank guesses the next thing the user will type, with a short generation off the warm session at an idle moment. The guess is shown in the prompt itself, as dim text after the cursor. Tab or the right arrow places it into the editor, Enter sends it, and any other key dismisses it. That last binding is why the suggestion's sanitizer is as strict as it is: Enter over a placed suggestion submits immediately, so a suggestion beginning with a slash would run a command on one keystroke, and one beginning with an exclamation mark, which plank runs as a shell command without confirmation, would run that. Both are refused before the suggestion is shown. A suggestion is also bound to the transcript length it was generated against and is dropped once the conversation moves, and every path that replaces the session clears it, because a guess about one conversation offered in another is worse than no guess. The line REPL has no line to ghost into, so the feature is full-screen only, and it is the clearest case in plank of a capability that exists because a second front end does.

Panels and reports

A full-screen display can show a report without pushing the conversation off the screen. Plank's reports open as dismissable panels over the log: the context report, the jobs panel that stays live while a turn runs, the output of a double-bang shell command in a scrollable panel of its own, and `/stats`, an activity heatmap of the user's sessions. The heatmap reads nothing but the session metadata that chapter 5 kept as an advisory sidecar, which is a second life for a file that was designed to be expendable, and its shading had to be fixed in a way worth recording: ranking each day's total pulled both ends of the scale towards the middle as soon as two days tied, so three equally busy days took a middling green; ranking the distinct totals instead puts the busiest day at the bright end however the days tie. The session's closing summary, printed after the screen is restored, went the same way: two run-on lines that wrapped in the middle of numbers

Appendix A: A full-screen terminal front end

became a small table, one block per engine with input, output and tool rows across tokens, time and rate.

One report taught a rule about the log itself. Plank collapses the streaming preview of a file being written into a single line, a counter of lines so far, rewritten in place on each tick: the last line of the log is the counter, popped and re-pushed. The contract broke the moment anything else appended to the log while a write streamed, a context report typed mid-turn for instance: the next tick popped the report instead of the counter, and a stale count sat above a fresh one. Every push that is not part of the stream now retires the counter first, and the next tick re-pushes it below whatever landed. Blanka's log has no in-place lines, which is the simplest way to satisfy the rule.

Plank's sub-agent roster added one more, about addressing anything on screen. Its rows are a filtered view of the runs, because a finished run's row expires a minute after it ends while the run is kept for its output, so the drawn row number stops being the run's index the moment one row has expired, and a click on a row opened the wrong agent. The frame now records its own map from drawn row to run and the click reads that map, because a row can expire between the frame the user clicked and the moment the click is handled, and only the drawn frame knows what the user aimed at.

Leaving the terminal as it was found

A full-screen program changes the terminal's mode: raw input, an alternate screen, and in plank's case mouse reporting and bracketed paste as well. Whatever ends the program has to put all of it back, and blanka does that with a guard whose drop restores the terminal, so a return, an error and a panic all take the same path:

```
/// Restores the terminal however the loop ends, a panic included.  
struct Restore;
```

```
impl Drop for Restore {  
    fn drop(&mut self) {  
        ratatui::restore();  
    }  
}
```

Plank's findings show what happens when there is more than one path. A plain Ctrl-D left the shell prompt reading a string of numbers and semicolons: mouse-motion reports the terminal had queued while plank's power-off animation played, with motion reporting still on and no event loop reading them. Mouse capture now goes off before the animation, and a single teardown ends by flushing the terminal's input on a real terminal, so anything that slipped in is dropped rather than typed into the shell. The same teardown replaced the bare library restore in the forced-quit path and in a panic hook, which had never turned off mouse reporting, paste handling or focus events at all, so a forced quit or a panic left the terminal in mouse mode for good. One teardown, run on every exit, is the rule; blanka's guard is that rule at the size blanka needs.

Two paths per command

Plank's project notes carry a warning that has cost it more than any single bug in this appendix: slash commands are handled in two parallel paths, the plain REPL and the full-screen front end, and a change to one usually needs the mirror change in the other. The duplication was not designed; it accumulated, because a full-screen front end wants to open a pane where the plain one prints text, and the easiest place to decide that is inside the command. Plank's rule for keeping the two honest is that every command with a pane has a static text equivalent on the plain path, so the interactive cache browser has `pin`, `unpin`, `rm` and `gc` subcommands that print their result.

Blanka avoids the duplication by keeping every command in one handler. The full-screen front end passes each slash line to the worker, which calls the same `repl : : command` the line REPL uses, and the answer comes back as text that the log shows. What that gives up is exactly what plank's second path buys: a command in blanka cannot open a pane, only print. A reader who adds panes should keep the handler as the one place a command's effect is decided and let the front end choose only how to show its answer, which is the discipline plank's warning is trying to recover.

Testing a screen

Ratatui draws into a buffer, and its test backend is a buffer with no terminal behind it, so a frame can be drawn and read back cell by cell. Blanka's frame test draws the whole layout at a fixed size and reads the rows as strings:

```
#[test]
fn a_frame_shows_the_log_the_two_status_rows_and_the_prompt() {
    let mut sink = TuiSink::default();
    sink.visible("The tests pass.");
    let status = StatusInfo { dir: "~/proj".into(), engine: "echo".into(), state: "idle", queued: 2,
last: None };
    let editor = Editor { text: "next".into(), cursor: 4 };
    let rows = screen(&View { sink: &sink, status: &status, editor: &editor, scroll: 0,
asking: None }, 40, 6);
    assert!(rows.iter().any(|r| r == "The tests pass.));
    assert_eq!(rows[3], "~/proj · echo");
    assert_eq!(rows[4], "idle · 2 queued");
    assert_eq!(rows[5], "> next");
}
```

The other tests stay below the frame: that streamed chunks join one entry and hidden reasoning is dropped, that long lines wrap at the width and keep their style, that a fence styles its lines until it closes, that the editor counts characters and draws its cursor on the right one, and that the menu offers prefix matches only for a lone word. Only `run_tui`, the loop that reads real keys from a real terminal, is left to the smoke run: build with `cargo run --features tui -- --tui`, type while the echo stub answers, press Page Up during a long reply, and quit with Ctrl-D.

Where blanka stops short of plank

Plank's log caches wrapped heights by width so that scrolling far back is as cheap as the bottom of the conversation; blanka wraps from the end on every frame. Plank renders full markdown and highlights code with tree-sitter on a throttle with a draw-clock flush; blanka styles a line at a time. Plank has mouse scrollbar, selection and clickable footer segments; blanka has Page Up and Page Down. Plank draws diff cards for edits, a collapsing write preview, a sub-agent roster with a pane per run, a side panel for asides, and dismissable report panels; blanka's log has entries of six kinds and a popup for questions and the command menu. Plank's footer carries every segment described above and measures itself in display columns; blanka's two rows count characters. Plank's editor has history recall, multi-line input, file mentions and ghost-text suggestions; blanka's has one line and a cursor. And plank's teardown also handles mouse reporting, paste and focus events, flushes pending input, and runs from a chained panic hook; blanka's guard restores what blanka changed.

A second pair of eyes

A full-screen front end is a second sink over the same stream and a second loop over the same worker, and the book's earlier chapters are what make it that small. The sink appends classified text to a log; the frame wraps only the entries that can reach the screen, because a frame that re-wraps the whole conversation makes the longest sessions the slowest to draw; the markdown is read a line at a time, and a renderer that throttles has to flush on a clock of its own. The status rows are a pure function of state and width, measured the way the terminal measures, and the cursor is drawn because a terminal can ignore a request to colour its own. The prompt stays live, with the same three routes for a line typed during a turn as the line REPL has, and Ctrl-C becomes a key the loop turns into an interrupt. Menus claim no more of the keyboard than they show, rows on screen are addressed by what was drawn, and one teardown puts the terminal back on every exit. The one discipline plank's experience recommends above all the others is to keep each command's effect in one handler, so that a second front end changes how an answer looks and never what the command does.

APPENDIX B: DRIVING THE HARNESS FROM ELSEWHERE

A long turn on a local model can run for twenty minutes. The person who asked for it is rarely still looking at the terminal when it ends: they have switched to a browser, walked to another room, or opened the laptop they carry rather than the machine that holds the weights. The harness finishes, prints its answer into a window nobody is watching, and waits. Two separate problems sit in that moment. The first is telling someone that something happened, which is a matter of notification and of what the window itself says. The second is letting them act on it from where they are, which is a matter of a second front end reaching into a running session.

Plank solved both, and the solutions share one idea that is worth stating before either is described: neither adds anything to what a turn is. A notification is a side effect of a state the harness already knows it is in. A remote client is another consumer of the event stream that chapter 11 introduced between the worker and the screen, and another producer of the prompts, interrupts and side questions that the same chapter gave a home. Everything in this appendix is transport and policy over machinery the book has already built.

The window says what the turn is doing

The cheapest signal is the terminal's own title. Every terminal emulator shows it in the tab bar and the window list, and a user scanning a row of tabs reads it before anything else. Plank keeps it in step with a handful of states: loading while the front end is not yet accepting input, including the system-prompt prefill that is the slowest step of a launch; ready while idle at the prompt; the prompt text, truncated to twenty characters and preceded by an animated glyph, while a turn runs; waiting while the `ask` tool of chapter 12 holds the turn open for an answer; compacting while chapter 7's summary runs; and a separate state while plank reads back the user's own history for an insights report.

The states that are not a turn deserve their own names for a reason plank's module comment gives in one line: they interrupt whatever the user asked for, they take long enough to be worth naming, and they are the phases in which a window in the background should say that the harness is not stalled but waiting, either on itself or on the person. A title that read "busy" through a compaction would send the user back to check for nothing, and one that read "busy" while the model waited for an answer would leave the turn blocked on someone who believes it is still working.

The mechanism is a single escape sequence, operating system command zero, written to standard error in one write. Standard error reaches the same terminal as standard output but bypasses the full-screen front end's frame buffer, so a title change emitted from the worker thread can never tear a half-drawn frame. The write is skipped when standard error is not a terminal, which covers piped runs, tests and the headless modes, where a title would be noise in a log. The waiting state is always set through a scope that restores the previous title however the question ends, including a declined or interrupted one, because a title stuck on "waiting" after the answer arrived is worse than no title at all.

A banner for the person who left

A title helps someone who glances at the machine. A desktop notification reaches someone who is using it for something else. Plank fires one at the lifecycle points where the user's attention is wanted: when a turn completes, when the `ask` tool is waiting for input, and when a background job from chapter 11 finishes while the user is away from the prompt and wakes the model to report on it.

Delivery is best effort and never on the turn's path. The notification is posted from a detached thread, so a slow or failing notification service costs nothing, and a failure is swallowed rather than reported, because a missing banner is an inconvenience and a turn that stalled on one would be a defect. A completed turn notifies only if it ran for at least a configurable number of seconds, ten by default, because a banner for a four-second exchange tells the user what they are already looking at. The waiting-for-input notification ignores that threshold and is sticky, since a question that times out of the notification centre leaves the turn blocked with no trace.

The more interesting setting is when to notify at all. Plank has three modes: always, never, and only while the terminal window is not focused, which is the case where a banner is information rather than redundancy. Focus is known from the terminal's focus events, which only the full-screen front end receives. The plain line front end never learns whether it is focused, and plank's choice there is written into the module: the focus flag starts false and stays false, so the unfocused mode behaves like always. The comment beside it gives the reason as a preference between two failures, a redundant banner or a silently missing one, and chooses the first. The same reasoning appears in more than one place in this book: when the harness cannot know which of two states it is in, it picks the one whose mistake the user can see.

One small detail shows how much a platform decides for you. A command-line program on macOS has no application bundle, so the system would stamp every banner with a generic icon. Plank cannot set an arbitrary icon for an unbundled process, so it borrows the host terminal's, deriving the bundle identifier from the `TERM_PROGRAM` variable the terminal sets and falling back to the system terminal for anything unrecognized, and attaches its own logo as the banner's content image. It is the kind of workaround a harness accumulates once real people use it, and it is recorded in the module header rather than left for the next person to rediscover.

Another front end, not another engine

Notifications tell the absent user that something happened. Acting on it from elsewhere is a larger design, and plank's record of it opens with a boundary that decides everything after it: a remote client attaches to the running session and does not fork it. There is still one engine, one live key-value session and one transcript. The remote client becomes another front end over the worker's event channel, the same serialization boundary the full-screen display already uses, and never a second session with its own cache.

That boundary is what makes the feature affordable. A second session would mean a second copy of the conversation in the engine's memory, a second prefill of everything, and two streams contending for one GPU, which is the cost chapter 11's side questions were designed to avoid. Attaching instead of forking means the remote path inherits

Appendix B: Driving the harness from elsewhere

interrupt, queued prompts, side questions, tool banners, status snapshots and the whole cache discipline of Part II without reimplementing any of them.

The design also refused a backend. The hosted agent whose remote control plank's design studied bridges to a service of its own, with account authentication, environment registration and work polling, and plank has no service and did not grow one. What shipped is a local WebSocket server bound to the loopback interface, and reaching it from another machine is the user's own SSH tunnel. Chapter 14's remote engine, which serves one loaded model to several sessions, is a different feature solving a different problem; remote control serves one session to several viewers.

One bus, many subscribers

Chapter 11 connected blanka's worker to its front end with one channel of events and a sink that forwards render calls into it, and it gave that sink one property on purpose: a receiver that has hung up drops the text, the worker keeps going, and the transcript, which the worker owns, stays authoritative. Plank's worker had the same shape, and the single structural change remote control needed was to turn the one sender into a fan-out. The bus holds a list of senders, one per subscriber, and every event is cloned to each of them:

```
// plank, src/worker.rs (bus from e49e47d, reset clause from f6d1f4b)
pub fn broadcast(&self, event: UiEvent) {
    let mut inner = Self::lock(&self.inner);
    // A reset invalidates everything before it: drop the retained tail so a
    // client attaching afterwards is not replayed the transcript the
    // operator just cleared. Ids keep climbing, so an already-connected
    // client's `resume_from` stays meaningful across the reset.
    if matches!(event, UiEvent::SessionReset) {
        inner.scrollback.clear();
    }
    let id = inner.next_id;
    inner.next_id += 1;
    let seq = SeqEvent { id, event };
    inner.scrollback.push_back(seq.clone());
    while inner.scrollback.len() > SCROLLBACK_CAP {
        inner.scrollback.pop_front();
    }
    inner.subscribers.retain(|tx| tx.send(seq.clone()).is_ok());
}
```

Three things in those lines carry the design. The last one prunes a subscriber the moment a send to it fails, which is the chapter 11 contract extended to many receivers: a vanished browser costs one failed send and is gone, and no remote client can ever block the worker. The sequence number on every event is what makes reconnection cheap, because a client that knows the last id it saw can ask for everything newer. And the bounded ring of recent events, four thousand of them, is what a client attaching in the middle of a turn is shown first, so that it starts from a coherent picture rather than half a line.

Appendix B: Driving the harness from elsewhere

The reset clause arrived after the rest, and its story is the kind this book collects. Clearing the session replaced the transcript and cleared the local display, both of which are local operations, and nothing told the bus. A browser kept showing a conversation that no longer existed, and a client that attached afterwards was replayed the very transcript the user had just cleared. The fix is a reset event that attached clients act on and that empties the ring, with ids that keep climbing so that a client's resume position stays meaningful across it.

The protocol is the event enum

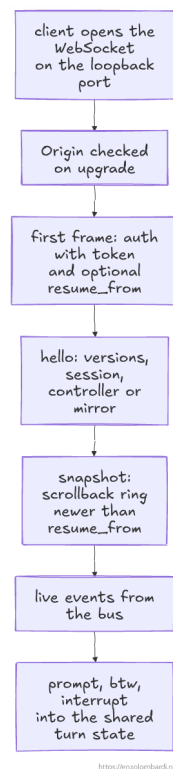
The wire format is one JSON object per WebSocket text frame, discriminated by a type field inside a versioned envelope. The server's frames are an image of the worker's events: visible text, reasoning, tool banners, errors, dim and plain log lines, status snapshots, the boundaries of a side answer and the task counter. The client's frames are an image of what the shared turn state accepts: a prompt, a side question, an interrupt, a slash command, and a request to take or release control. The design document says the point in one sentence, that the remote path adds a transport and not new turn semantics, and a CLI client and the browser page speak the same schema.

Two frames were not in the design, and both exist because a remote front end is not the local one. The reset frame is the one described above. The notify frame carries the payload of the local desktop notification at the end of a turn, because the local banner reaches whoever is at the machine plank runs on, which is by definition the person a remote client is not. A third late addition rides inside the status frame rather than as a frame of its own: the working directory, branch and reasoning setting from the footer travel on every status update, so a client that attaches mid-session learns them from the first frame it sees. Status frames came only from engine callbacks during a turn, and without an idle snapshot published when the turn ends, the last state a remote client ever saw was "generating".

The browser client sends everything as a prompt, including slash commands, because the agent's own dispatcher already routes a leading slash when it drains the queue, and a second parser in the page would only be a worse copy of it. That is a small instance of a rule this book keeps returning to: one path through the harness, reached from several places, is cheaper to keep correct than several paths that are meant to agree.

A client's first minutes with a running session follow a fixed order:

Appendix B: Driving the harness from elsewhere



Attaching to a running plank: the connection is refused before the handshake for a foreign origin and after the first frame for a wrong token, and everything after the snapshot is the same event stream the local display reads

Who holds the input line

Several clients may attach at once, and plank allows all of them to see and at most one of them to act. Control is a single token held either by the local user or by one remote session, and every other client is a mirror. A mirror's prompt, interrupt or command is answered with a denial; its side question is accepted, because a side question is ephemeral and read-only by construction, which gives an observer a safe way to ask about what it is watching.

The reason for a single controller is the one that has shaped every concurrency decision in the book. There is one engine session and one transcript, and two people submitting prompts concurrently would interleave turns unpredictably and break the prefix discipline that makes a local model affordable. Truly independent sessions would need independent engine contexts, which is the duplicate cache and GPU contention cost that remote control exists to avoid.

The consent model changed between design and shipping, and the change simplified it. The design proposed a handshake in which a remote client asks for control and the local user grants it. What shipped first treated the command that starts the server as the consent: typing it means the operator wants to drive the session from elsewhere, so an attaching client's request is granted immediately. The handshake arrived later as an explicit alternative, a variant of the command that starts the same bridge without pre-authorization. A client's request is then refused, a line appears locally naming the session

Appendix B: Driving the harness from elsewhere

that wants control, and the operator grants the oldest waiting request or a named one. Granting clears the whole pending list, since there is one controller and saying yes to one request says no to the rest, and a waiter that disconnects is pruned so that a grant never lands on a dead socket.

Role changes after the handshake reach the client through a frame of their own, sent by the connection thread when it notices the change on its next poll. The poll is a deliberate choice. The grant runs on the local UI thread, which has no socket, and the bus carries transcript events for every client, not facts about one session, so the connection thread asking is simpler than anything else telling it.

Some commands are refused outright rather than queued, and the refusal list names two families. The first is commands that would take over the local terminal, such as the interactive cache browser or the interactive session picker, whose argument forms are non-interactive and stay allowed. The second is commands that would saw off the branch the client sits on: stopping the server, quitting plank, and the grant command itself, which from the requesting client would be a self-grant.

The token and the loopback

The server binds the loopback address on an ephemeral port, so starting it never collides with another plank or a stale listener, and it mints a fresh token on every activation. The printed output is a one-click link carrying the token and a ready-to-paste SSH forwarding line. A client's first frame must authenticate; anything else closes the connection with a policy-violation code, and a wrong token closes it with a custom unauthorized code. The comparison is constant time, failures are rate limited, and there is no default token and no unauthenticated mode. Turning the server off drops the token with it, so a link from a previous activation is refused.

Binding to loopback is defence in depth rather than the authentication boundary, and the design is precise about why. Any local user can reach a loopback port, which the token defends against. A web page in the user's own browser can try to open a WebSocket to a loopback port, which is the cross-site risk of local servers, and the defence there is the upgrade's origin header, checked before the handshake completes:

```
// plank, src/remote/control.rs (66a597f)
pub fn origin_allowed(origin: Option<&str>, allowed: &[String]) -> bool {
    match origin {
        None => true,
        // Browsers send the literal "null" for opaque origins (e.g. a page
        // opened from a file:// URL); treat it as a non-loopback browser origin
        // that must be explicitly allow-listed.
        Some(o) => is_loopback_origin(o) || allowed.iter().any(|a|
a.eq_ignore_ascii_case(o)),
    }
}
```

A missing origin is allowed because native clients send none, and a loopback origin is allowed because that is the page plank serves itself. Every other browser origin is refused. The allow-list parameter survives from a flag that could add entries to it; the

Appendix B: Driving the harness from elsewhere

command that starts the server today passes an empty list, which makes the default deny unconditional.

What the design explicitly does not defend is stated as plainly as what it does. A leaked token or a compromised SSH endpoint gives full control of that plank instance, and plank can run shell commands under its sandbox, so the token is equivalent to shell access on the box. The document's advice is to treat it like an SSH key. The one-click link puts the token in the URL, where it lands in browser history, and the trade is recorded as accepted for a loopback listener whose lifetime is a single toggle, not as a claim that the link is a secret.

Transport encryption is left to SSH. The socket carries plaintext JSON, but only over the loopback interface, which is on no wire, and off-box confidentiality and authentication are what the tunnel provides. Adding TLS and certificate management to a loopback server was judged complexity without benefit. The same reasoning kept an asynchronous runtime out: plank is synchronous throughout, with blocking HTTP, threads and channels, and the server is one accept thread plus one blocking thread per connection over a synchronous WebSocket library, rather than the largest dependency and paradigm change in the codebase for one endpoint.

Flags that made a second server

The design proposed a launch flag to start the server, and what shipped first was that flag and four siblings for the token, the control policy, the allowed origins and the outbound queue size. All five were later deleted, and the reason is a bug class rather than a bug.

The flags created a server the runtime command could not see. In a session started with the flag, stopping the server from inside the session reported that it was already off while a listener was serving, and starting it bound a second listener and orphaned the first server's bus, silently cutting off clients that were already attached. Each symptom could have been patched by teaching the command about the flag-started server. Plank deleted the flags instead, so that the command typed inside a running session became the only way in, and the class of two-servers bugs went with them.

Headless server mode went too. The design's primary target had been a server with no local display at all, and once only a command typed in the full-screen front end could start the server, the headless and piped remote-drive paths were unreachable. They were written, found unreachable, and removed; the command declines in the line front end and says why, since only the full-screen loop drains the remote queue. A feature the design called primary ended up not existing, and the design record says so at the top rather than editing the history away. Plank's convention for design documents is that where the document and the code disagree, the code wins, and the divergences are called out inline because they are the interesting part.

Slow clients and short outages

A remote client on a flaky connection must never stall a turn, and the bus's pruning is only the first half of that guarantee. Each connection also caps its unsent output at one mebibyte and writes with a short timeout, so a stalled client's data accumulates in a

Appendix B: Driving the harness from elsewhere

buffer rather than blocking its connection thread, and once the buffer passes the cap the client is evicted. Its thread exits, the bus prunes the dead subscriber on the next broadcast, and the client reconnects and resynchronizes from the ring.

Status snapshots can arrive once per token, and they are pure state rather than a log, so the server coalesces them to at most one every hundred milliseconds per connection, sending the latest and dropping the intermediates. Text frames are never coalesced, because they are the transcript. Native WebSocket pings detect dead peers, so that control can be reclaimed and threads freed.

A controller that drops keeps control for fifteen seconds, so a brief network blip resumes seamlessly, and after that window control returns and a mirror may claim it. Local control never expires. The grace window has one known rough edge, which plank's design record lists as still open rather than fixed: reloading the browser page creates a new session with no resume position, and for the length of the window its request for control is denied with the reason that another client holds control. The denial is correct. The reason string is not, and distinguishing a control held for a reconnect from one held by someone else would be the honest fix.

Two clients, one schema

Plank ships two clients. The command-line client is a subcommand of the same binary, so it shares the protocol types, and it makes a running session scriptable over the tunnel, which was the original request: submit a prompt from another process and read the output. It streams visible text, reasoning and banners to its terminal with the plain front end's styling, and it reads lines from standard input as prompts, commands or side questions, with Ctrl-C sent as an interrupt frame. One implementation detail is forced by the WebSocket library: a socket cannot be read and written from two threads at once, so the client owns the socket on its main thread, polls it with a short read timeout, and reads standard input on a helper thread that delivers lines over a channel. It is the worker-and-channel shape of chapter 11 once more, on the other side of the wire.

The browser client is a single static page served by the same server at its root, with no framework and no build step: a scrollback pane, a status footer bound to status frames, an input box and buttons for a side question and an interrupt. It is enough to drive plank from a phone through the tunnel, which is the use the design had in mind when it rejected the hosted backend's synchronisation as out of scope.

What blanka already has for it

Blanka has most of what remote control is built from. Chapter 11's worker already sends a clonable event enum over a channel, its sink already drops text into a receiver that has hung up without stopping the worker, and its inbox already accepts prompts, slash commands and side questions from whoever holds the sending end. The additions are the ones this appendix has walked through: a bus in place of the single sender, with sequence numbers and a bounded ring; a reset event; a connection thread per client that authenticates, replays and pumps; a single-controller policy; and a server started by a command rather than a flag. None of them touches the engine, the transcript or the cache, which is the property that makes the feature worth building.

Appendix B: Driving the harness from elsewhere

Notifications and the window title are smaller still. A title is one escape sequence at each state change and a scope for the states that must be restored, and a notification is a detached thread at the end of a turn, a threshold, and a mode that defaults to the visible mistake when focus cannot be known.

Reaching in without forking

A harness that runs for minutes at a time needs ways to reach the person who asked, and ways for that person to reach back. Plank's window title names the phase of the turn, including the phases that are not the turn, in one write that cannot tear a frame. Its notifications are best effort, thresholded, and prefer a redundant banner to a missing one. Its remote control attaches to the running session instead of forking it, over a bus that fans the worker's events out to every subscriber and drops any that fall behind, with a protocol that is an image of the event enum, one controller and many mirrors, a per-activation token, an origin check, and SSH for everything that crosses a wire. The launch flags that created a second, invisible server were deleted rather than patched, and the design record keeps the divergences from its own plan in plain view. Every piece of it is transport and policy over the event channel of chapter 11, which is why none of it had to change what a turn is.

APPENDIX C: WASM PLUGINS

By the end of chapter 13 plank had four ways for someone else's code to reach the session, and each had been solved on its own terms. A hook is a shell command, forked and executed once per event, with no state between calls and no access to the display. An MCP server is a whole subprocess with a JSON-RPC handshake and a cold start of tens of milliseconds per server. A slash command is an entry in a static table compiled into the binary, and so is every cell of the status bar. Each is a different authoring experience with a different answer to the question a user eventually asks about any extension: what can this code reach.

Plank's WebAssembly plugins are its attempt at one answer to that question for the cases the other mechanisms serve badly. A plugin is a module that runs inside plank's process, in a sandbox, with nothing granted unless its manifest asks and the user approves. It can contribute slash commands, tools the model can call, cells on the status bar, observers of the session's lifecycle, and a component that draws the whole screen. The design document is candid about the gap between what it describes and what shipped: four surfaces of five, nine events of roughly twenty, five capabilities of ten. This appendix follows what shipped and the reasoning for what was cut, because the cuts say as much about extending a harness as the parts that were built.

Why a sandbox and not another process

The argument for the feature is not that the existing mechanisms are bad. It is that several extension points had accumulated with several trust stories, and a single sandboxed interface could serve most of them with one artifact format, one manifest, one permission model and one place to reason about reach. The document is equally clear about what WebAssembly is not for, and the non-goals are the part of the design worth reading first.

It is not a replacement for MCP. An MCP server talks to the network, holds long-lived credentials, and is often a third-party binary that a user wants out of the harness's process; a sandbox that forbids the network would make those servers impossible, and a sandbox that allowed it would stop being much of a sandbox. It is not a replacement for hooks either: a one-line jq in a hooks file should stay a one-line jq, and forcing it through a compiler and a manifest would be a regression for the common case. And it is not general native extensibility. There is no dynamic loading of native libraries in plank and the design says there never will be; if a piece of logic cannot be expressed inside the sandbox, it does not belong in a plugin.

What remains is logic that belongs close to the harness's own loop: a status cell that reads a file and summarises it, a slash command that formats something, a tool whose work is pure computation, an observer that keeps a tally across a session. For those, a process per call is the wrong cost, and a shell command is the wrong trust model.

A component kind, not a second system

The first decision was where WebAssembly lives, and plank settled it before writing any of the loader. Chapter 13 described a plugin as a directory bundling contributions, skills, agent definitions, templates, hooks, an MCP configuration and a settings file, found

Appendix C: WASM plugins

under a fixed order of scan roots with a later root winning a collision and every entry addressable as `<plugin>:<name>`. A WebAssembly module joined that list as one more component kind. It did not become a second system with its own directories, its own precedence rule and its own noun.

The reasoning is that the loader already carried almost everything a plugin system needs: ordered locations, two manifest spellings, namespacing with bare names when uncontested, collision warnings, and settings merged strictly below the user's. A second resolution order beside it would have meant two precedence rules for users to learn and two implementations to keep in agreement, to buy nothing the first did not already do.

The cost of reusing it was one asymmetry, and the design adopts it deliberately. The project-local scan root means that cloning a repository can bring plugins with it, and chapter 13's loader activates a project's skills and MCP servers without asking. That is tolerable for a skill, which is text. It is not tolerable for a module that might hold a shell capability. Project-local WebAssembly components are therefore denied by default even though project-local skills are not, and the design's justification is a sentence worth keeping: the trust question is about what the code can reach, not about where the directory sits.

Extism, and what it costs

Plank runs modules through Extism, which puts a small host interface and a set of plugin development kits on top of wasmtime. The choice was made against two alternatives. Raw wasmtime with the component model's interface types would let the type system enforce, for example, that a background component cannot even name the drawing functions; it would also mean building and maintaining a multi-language toolchain story nobody on the project had time for. Wasmi is small and written in Rust, but its interpreter is ten to fifty times slower, survivable for simple loops and unpleasant for anything else, and the hand-rolled interface work would be the same.

What Extism buys is reach: ready-made development kits for Rust, Go, JavaScript, Python, C#, Zig and C++, so a plugin written in a language other than plank's is a real outcome. What it costs is its calling convention. Every call is a byte buffer in and a byte buffer out, so structure lives in the payload encoding rather than in any type system, and nothing checks at load time that a module's exports speak the shape the host expects. Plank substitutes a handshake: every module must export a function returning the interface version it implements, and the host cross-checks that number against the manifest. A mismatch is a load error naming the plank version that speaks the module's interface, never a warning, because Extism will happily let a module export a function with the right name and the wrong payload.

The other cost is size. The runtime added eighteen mebibytes to the binary, measured at 134 against 152 today. Plank kept the feature behind a Cargo feature, off, through its feasibility stage, and turned it on by default once the system stopped being experimental, on the argument that a plugin nobody can run without rebuilding plank is not a plugin system. The lean build still exists with default features disabled, and continuous integration builds it specifically, because it has become the configuration that can rot unnoticed. The whole host sits behind a trait with a no-op implementation when the

feature is off, which is the same shape as the `Engine` trait and the echo stub, for the same reason: the harness must stay buildable and testable without its heaviest dependency.

What a component may own

A component declares the surfaces it claims, and a component claiming a surface whose exports it lacks is refused at load rather than failing the first time a user picks it. Surfaces add permissions, never privilege: claiming the drawing surface grants no file access, and claiming a tool grants no drawing.

The command surface contributes slash commands to the menu and to both front ends' dispatch. Their specifications are read once at load and never per keystroke, and a command's output can print lines, place text in the input box, open the component's own full-screen view, or return a prompt to submit to the model. A component may open only its own view; opening someone else's would be a capability, and this is a convenience.

The tool surface is where the design touches the model, and it inherits chapter 13's rules about third-party text. A component's tools join the model's registry beside the built-ins and the MCP servers' tools. The call shape is deliberately MCP's, a list of specifications and a call that takes arguments and returns output and an error flag, so that an MCP server whose logic is pure computation can be recompiled as a module without redesign and so that one dispatch table with one collision policy serves both. The component receives the arguments already parsed into a JSON object and never sees or builds the tool-call syntax, which means the byte-parity contract of chapter 2 stays entirely inside plank's parser and a plugin cannot break it. The schemas are appended to the system prompt after the trusted span, exactly like MCP's. A name is used bare when nothing contests it and is qualified with the component's name when something does, with a warning either way, and the built-ins always win, so no component can quietly replace the shell.

The tool surface is also where the cache of Part II reaches plugins. Adding or removing a tool component changes the tool list, which changes the system prompt, which moves the fingerprint of the static tier. Plank resolves tool components before the prompt is fingerprinted, so the two configurations keep separate checkpoints instead of invalidating each other, and it never changes the tool list of a running session. A component approved in the middle of a session is dispatchable at once but reaches the prompt only at the next launch.

The segment surface contributes a cell to the status bar, with one deviation from the design that is a lesson in cadence. The design said a cell renders once per repaint. The full-screen front end repaints on every keystroke and every animation tick, so that would have run a guest dozens of times a second on the UI thread while the user typed, to refresh data that changes every few seconds. A cell renders at most once a second and is read from a cache in between, with the throttle inside the registry so that callers can ask at whatever boundary suits them. A cell that returns nothing is quiet rather than broken, because the status bar must never be a place where a component is disabled for having nothing to say, while a trap still counts against it. Cells carry a priority, and when the bar does not fit, contributed cells are dropped lowest priority first; built-in cells are never

dropped on a plugin's behalf, because a user who cannot see the context gauge because a component had something to say is worse off than one who cannot see the component.

The observer surface owns nothing and exists to receive events. The frame surface owns the whole terminal for a component that draws at animation rates, and it is the one surface where JSON was rejected: a screen of 120 by 40 cells is 4,800 glyphs, and encoding that thirty times a second is a measurable fraction of a core, so a frame returns a packed binary glyph buffer. Its decoding is total, so a malformed buffer costs the frame and never the session, and the glyph count is validated against the declared area before anything is allocated, so two edited bytes cannot ask the host for a huge allocation. A frame receives no ambient clock and no ambient randomness: time and a seed arrive in the payload, so the same seed replays exactly in a test.

The fifth surface, a panel owning a region of the screen, was cut. It had no consumer, and it was the only reason the design had an open question about arbitrating layout between competing plugins. Cutting it deleted the question rather than answering it, and adding a surface later is an additive change that needs no interface bump.

Events, and who may answer

Every component, whatever its surfaces, may subscribe to events, and an unsubscribed event costs nothing. The events that shipped are fired from the same call sites as the shell hooks of chapter 13, so a component sees what a hook would have seen: session start and end, prompt submission, turn start and end, before and after a tool call, and before and after a compaction.

Events come in three classes by what a reply means. A notify event's reply is ignored. A veto event's reply may block the action, with a reason that goes to the model for a tool event and to the user otherwise. A transform event's reply may replace the payload, and replacements chain through subscribers in load order, so that a redactor and a summarizer compose instead of competing. The first block wins and stops the chain, on the reasoning that once an action is refused, asking the remaining components about it is asking them to weigh in on something that will not happen.

The class is enforced by the host and never trusted from the guest:

```
// plank, src/wasmevents.rs (Old4ecb)
/// Drops anything the event's class does not permit, so a component cannot
/// veto a notify event or rewrite something that is not transformable.
#[must_use]
pub fn constrained_to(mut self, class: Class) -> Self {
    match class {
        Class::Notify => {
            self.block = None;
            self.replace = None;
        }
        Class::Veto => self.replace = None,
        Class::Transform => {}
    }
}
```

```
    self  
}
```

A component implements a single event export rather than one per event, which is a small deviation from the design with a clear argument behind it. Extism cannot type-check an export's signature, so several exports would buy no safety over one, and a new event would otherwise oblige every interested component to grow an export, which the rule for evolving payloads says an addition must never do.

One event was cut on principle. A per-token-batch event would have put a plugin call inside chapter 4's stream renderer, on the hot path of every pass and on the one path under a byte-parity contract with the DwarfStar reference, and no known consumer needed that granularity: a usage tracker wants totals, and a redactor belongs after the tool call, where a veto is honest about its timing. If a live visualisation of the token stream is ever wanted, the design says it will arrive as a sampled event with an explicit interval, never as a subscriber on every batch.

The events also produced the most instructive defect of the feature. A live session with a real component installed found that three events, session start, prompt submission and turn end, were defined, parseable, subscribable and documented, and that nothing ever fired them. Two sat behind guards that returned early when no shell hook was configured, guards written before any other kind of subscriber existed. A session with components and no hooks is the ordinary case, and it was exactly the case that fired nothing. The third had no firing site at all. Fifteen integration tests had passed throughout, because they drove the registry and the dispatcher directly: they proved dispatch worked and said nothing about whether anything called it. The fix moved the dispatch outside the hook guards, gave the missing event a site on both front ends, since a component must not observe a different number of turns depending on which front end the user runs, and added a test asserting that every event kind has a firing site. It is chapter 9's lesson about tests that would pass with the feature deleted, found in a different subsystem.

Capabilities are imports

Everything a component can do to the world outside its own memory is an imported host function, granted per component in the manifest, with nothing granted by default. Five are wired: logging to plank's debug log, which never reaches the transcript; printing lines to the scrollbar; a per-component key-value store under plank's home directory, which is the only persistence most components need and requires no file grant; desktop notifications; and a small set of sound cues.

Every host function is provided to every component and checks its own grant when called, rather than being linked only for components that asked. The difference is in the error a developer sees: a missing grant reads as a sentence saying the component was not granted that capability, instead of an opaque import failure at load. The state store has quotas, a mebibyte per value, 255-byte keys, and 256 keys and sixteen mebibytes in total per component, and an over-quota write is refused with the component's identifier before anything is written.

The three capabilities that would undo the sandbox, file access, network access and command execution, are designed and deliberately not wired. The design treats command execution in particular not as a capability so much as a declaration that the component is not really sandboxed, and specifies that it would require explicit confirmation at install and a visible warning in the plugin listing. Leaving it unbuilt keeps that conversation from having to happen until a real component needs it.

Budgets instead of fuel

The harness's loop must never be blocked by a plugin, and the design's answer was fuel: metering the guest's instructions so a runaway loop runs out. Fuel needs wasmtime configuration that Extism does not expose, so what shipped is wall-clock time. A one-second manifest timeout is the outer bound, and each surface has a target measured on the host side: fifty milliseconds for a frame step, twenty for a status cell or a key, a second for a tool call, two hundred for anything else. A call fails at four times its target rather than at it, because a wall-clock measurement on a busy machine measures the machine, and a frame steps twenty times a second; enforcing the target exactly would strike out a working component on a loaded laptop. The document names the gap this leaves, that a guest can still burn its budget inside a single host call, which is the half of the problem fuel would have covered.

A component that traps or overruns three times in a row is disabled for the rest of the session, with one line saying so. The rule behind the strike-out is stated in the design in one line: a plugin that breaks should degrade its feature, never the session. Memory is bounded the same way as time, with linear memory capped at 64 mebibytes, generous for a component and small next to the model the same process holds resident, and a guest that grows past it traps and takes a strike rather than failing the host. A single instance is never called re-entrantly; plank serializes calls per instance, runs the drawing surfaces on the UI thread where they are already frame-synchronous, and runs tools and observers on a worker.

Trust is a hash

Sandboxing is the primary defence and signing is about provenance, and plank keeps the two apart. Installing a component shows its identifier, its surfaces and, prominently, its capabilities, then records the module's SHA-256 in a trust file. A changed hash on a later load asks again. It is the discipline chapter 6 applied to the static prompt, stored once and named by its content: the hash is the identity.

Approval is a typed act. A component waiting for approval is listed by the plugins command with what it wants and the exact command that approves it, rather than being raised as a modal question in front of the first turn, where a user answers whatever gets the prompt back fastest. Disabling a component keeps its approval, so re-enabling something the user turned off does not ask again about capabilities they already read, and a disabled component lands in the listing under its own verdict, because a user reading that plank has never seen something they disabled themselves would go looking for the wrong problem. Both take effect at the next start, and a live reload is refused with its reason, because unloading a component mid-session would take its tools out of the

Appendix C: WASM plugins

fingerprinted system prompt, and a reload that silently skipped tool components would behave differently depending on what a plugin contributes.

Signatures are optional and advisory. A detached minisign signature beside the module, checked against a publisher key the user has accepted, lets updates from a known publisher install without asking again. A missing signature is not an error; a bad one is. A first install is never made quiet by a signature, since what signing buys is quiet updates, and the capabilities are always shown once. An update that adds a capability asks again even when its signature is valid, because grants are never widened silently.

Two fixes to this path are worth recording, both found by review rather than by tests. The first was a time-of-check to time-of-use gap: the trust decision hashed one read of the module file, the signature check read it a second time, and the host loaded a third read, so anyone able to swap the file between reads could have an approved hash admit different bytes. The loader now reads the module once into a buffer and hashes, verifies and loads that same buffer. The second was data loss in the trust file itself, introduced with signing: the writer placed a comma after each reserved key on the assumption that a component entry always followed, so a file holding only publisher keys, or only disabled identifiers, was written as invalid JSON and then discarded by the reader on the next load. A trust store that forgets on a malformed write fails open, which is the direction a trust store must never fail.

Two version numbers

A component carries two versions, and the design keeps them apart on purpose. The interface major version is the one the handshake checks, bumped only when an export's signature or a payload's shape changes incompatibly. The component's own semantic version is plank's business only for update checks. The rule for evolving payloads is that fields are only ever added, never removed or retyped: components ignore fields they do not know, and the host tolerates missing optional ones. The design notes that this is the discipline the tool-call wire format lives under, and for the same reason, the cost of breaking a consumer you cannot recompile. It is the same idea as the prompt version of chapter 6, applied to a boundary where the consumer is someone else's binary rather than a model's training.

A component kind for blanka

Blanka's plugins, from chapter 13, are directories of skills, agents, hooks and MCP configuration, loaded in a fixed order with the user's settings above everything a plugin brings. A WebAssembly component would join them the way it joined plank's, as one more kind in the same directory with the same precedence, behind a host trait whose default does nothing, so that the book's tests would keep running without the runtime. The tool surface would feed the same registry chapter 13 built for MCP, with the same rule that third-party schemas sit after the trusted part of the prompt and never replace a built-in. The events would fire from the four hook sites blanka already has, and the one test worth writing first is the one plank wrote last: that every event kind has a site that fires it.

Code that has to ask first

Plank's WebAssembly plugins are a sandboxed component kind inside the plugin format of chapter 13, not a parallel system, run through Extism with a version handshake standing in for the type checking Extism lacks. A component owns only the surfaces it declares, its tools reach the model after the trusted span and never displace a built-in, and a tool set is fixed for the life of a session because it is part of the cached prompt. Events are classed as notify, veto or transform and the class is enforced by the host. Capabilities are imports checked at call time, with the three that would undo the sandbox designed and left unbuilt. Budgets are wall-clock with room for a busy machine, three strikes disable a component and never the session, and trust is a content hash, with signatures buying quiet updates and never a quiet first install. The cuts carry as much of the design as the parts that shipped: a panel nobody needed, a per-token event on the one path under a parity contract, and fuel the runtime could not provide.

APPENDIX D: A LOCAL ENGINE WITH RLLAMA

Chapter 8 ended on a promise it could not keep in its own pages. The mock engine counted tokens instead of spending seconds, and the smoke run showed three launches whose counters read 1,471, 62 and 96, with the remark that against a real model the same three launches are the difference between a start that waits through thirteen hundred tokens of prefill and one that waits through sixty. Chapter 9 made a second promise: the engine's obligations were fixed and small, and swapping the mock for a real engine would change what the counters cost and nothing about what they count. This appendix keeps both promises with `llama.cpp`, the C++ inference library that runs GGUF models on Metal, CUDA or a plain CPU, and with a small crate named `rllama` that wraps it.

The target is the book's reference model, Qwen3 8B quantised to Q4_K_M, about five gigabytes of weights that sit comfortably in sixteen gigabytes of memory. Nothing in the code depends on that choice. Any GGUF that `llama.cpp` loads and that was trained on the ChatML template will run, and the tests at the end of the appendix assert on what the cache did rather than on what the model said, so they pass on any model at all.

Two crates, and a cycle Cargo refuses

The plan for this appendix put the engine in the second crate: `rllama` would depend on `blanka` and implement `blanka::Engine` for a `LlamaEngine`, and `blanka`'s binary would offer it behind a feature. Cargo refuses that arrangement before compiling a line, because `blanka`'s optional dependency on `rllama` and `rllama`'s dependency on `blanka` form a cycle, and Cargo rejects package cycles even when one edge is optional:

```
error: cyclic package dependency: package `blanka v0.1.0` depends on itself. Cycle:
package `blanka v0.1.0`
  ... which satisfies path dependency `blanka` of package `rllama v0.1.0`
  ... which satisfies path dependency `rllama` of package `blanka v0.1.0`
```

The cycle is a design signal as much as a build error. The two halves of a local engine want different knowledge. One half knows `llama.cpp`: how to load weights, tokenize, decode a batch, sample a token, and serialise a sequence's state. The other half knows the harness: what a message is, which roles exist, how a transcript becomes tokens, and what the trait promises. Plank draws the same line, with a foreign-function layer that mirrors the C headers of DwarfStar, the inference engine behind its DeepSeek models, and a separate engine module that implements plank's trait on top of it. So `rllama` became a crate that knows tokens and nothing about `blanka`, and the adapter that implements the trait lives in `blanka` itself, in `src/llama.rs`, compiled only when the `rllama` feature is on. The dependency now runs one way, and the wrapper can be tested on its own.

```
# code/rllama/Cargo.toml
[dependencies]
# No default features: OpenMP and the Android C++ runtime are not wanted on
# macOS, and Metal is enabled by the sys crate on Apple targets anyway.
llama-cpp-2 = { version = "0.1.157", default-features = false }
```

Appendix D: A local engine with rllama

The bindings crate, `llama-cpp-2`, vendors `llama.cpp` and builds it with CMake on the first compile, which takes a few minutes and needs a C++ toolchain and CMake on the path. After that the build is cached like any other dependency. On an Apple machine the `sys` crate enables the Metal backend by itself, and a Linux reader with an NVIDIA card turns on the `cuda` feature instead.

A session that knows what it holds

`llama.cpp` keeps a key-value cache per context, and it will tell you how many cells are occupied and at which positions. It will not tell you which tokens those cells were computed for. That is a reasonable division of labour for a library, and it is exactly the information a harness needs, because every decision chapter 8 made about reuse starts from comparing the prompt about to be sent with the prompt the cache already describes. So the session keeps that list itself:

```
pub struct Session {  
    model: Model,  
    ctx: LlamaContext<'static>,  
    live: Vec<Token>,  
}
```

The field that matters is `live`, the exact token list the cache holds, in order. Every operation on the session keeps the two in step: a decode appends to both, a truncation shortens both, a restore replaces both, and a failure that leaves the cache in doubt empties both. Plank's engine keeps the same record under the name of a token transcript, and chapter 8 noted it when the blob gained its fields. The reason is the same in both places. A comparison made against text would be a comparison against something the engine never saw, since the same text can tokenize in more than one way, and the unit of identity for a cache is tokens.

Two details of the struct are worth a sentence each, because both are places where the borrow checker asks a question the design has to answer. A `llama.cpp` context borrows the model it was created from, so a struct that owns both would be self-referential. `rllama` answers by giving the model a 'static lifetime: `Model::load` leaks the loaded weights on purpose, on the grounds that a harness loads one model at startup and uses it until it exits, which is what plank does with its eighty-seven gigabytes as well. And a context is not `Send`, because it holds a raw pointer. The session asserts that it is, with an argument rather than a hope: a `llama.cpp` context is not tied to the thread that created it, it must not be used from two threads at once, and the session is not `Sync` and exposes the context only through its own methods, so moving it to another thread hands over sole access. Chapter 11's worker, which takes the engine onto its own thread for the length of a turn, is the one pattern blanka needs, and the `unsafe impl` carries that reasoning in its comment.

Keep, drop, recompute

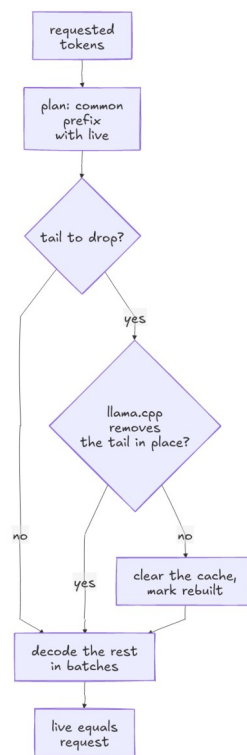
A `sync` brings the cache to a requested token sequence at the least cost the library allows, and it decides what to do before it touches anything. The decision is a pure function, which is what lets it be tested with no model loaded:

Appendix D: A local engine with rllama

```
pub fn plan_sync(live: &[Token], want: &[Token], need_logits: bool) -> SyncPlan {  
    let common = live.iter().zip(want).take_while(|(a, b)| a == b).count();  
    let keep = if need_logits && common == want.len() { common.saturating_sub(1) } else  
    { common };  
    SyncPlan { keep, compute: want.len() - keep }  
}
```

The common prefix survives and everything after it is recomputed. The one subtlety is the second line. The cache holds keys and values for past positions, not the probability distribution over the next token, which llama.cpp produces only for positions that were decoded with logits requested. When a generation is about to start and the requested prompt is already entirely cached, the last token has to be decoded again so that there is a distribution to sample from. A warm walk, which only needs the state and never samples, asks for no logits and pays nothing.

The flow of one sync, from the plan to a cache that matches the request, looks like this:



One sync: keep the common prefix, drop what follows it, decode only what is new

The middle diamond is where llama.cpp and DwarfStar part ways, and the difference is instructive for anyone reading chapters 8 and 9 with a real engine in mind. On an ordinary transformer, llama.cpp can remove the cells from a position onward and leave everything before it valid, so a prompt that diverges behind the live end costs only the tokens after the divergence. DwarfStar cannot. Plank records the reason in the comment above the function that decides how much of a prompt its engine will actually reuse (plank, src/engine.rs, commit a2fd398, doc comment as of 17f8e16):

Appendix D: A local engine with rllama

```
/// `ds4_session_common_prefix` answers "how many leading tokens match?", which
/// is not the same as "how many will be kept". `ds4_session_sync` reuses the
/// live KV only when the prompt extends the live end
/// (`prompt_len >= pos` && starts_with(prompt, checkpoint)`); anything else takes
/// the reset branch and re-prefills from zero. The C is explicit about why:
/// "Extending exactly at the live end is safe; rewriting behind it is not an
/// in-place operation"
// (the comment goes on to list the backend state a token count cannot roll back)
#[must_use]
pub fn reusable_prefix(pos: i32, common: i32) -> i32 {
    if pos > 0 && common == pos { pos } else { 0 }
}
```

The comment's next line names the reason: the backend still holds raw sliding-window rows, compressed rows, indexer rows and compressor frontiers for the old suffix, state that a truncation cannot unwind, so for plank a divergence behind the live end is a rebuild from zero, and that fact is what the whole snapshot ladder of chapter 9 exists to soften. The mock engine copied the harsher rule on purpose. On Qwen3 8B through llama.cpp the rule is gentler, and a micro-compaction that rewrites an old tool result in place costs the tokens from that result to the end of the transcript rather than the whole transcript. The ladder still earns its place across launches, where there is no live cache to truncate, and it still earns it on the models where llama.cpp refuses the truncation too: recurrent architectures and some sliding-window caches answer the removal with `false`, and the session then clears the cache, sets `rebuilt`, and starts over. The harness's correctness never depended on the gentle case. It was built and tested against the engine that cannot truncate, and an engine that can is only cheaper.

The tokens still to decode go through llama.cpp in batches, and `live` grows after each batch rather than at the end:

```
let todo = &want[self.live.len()..];
let n_batch = self.model.opts.n_batch.max(1) as usize;
for (i, chunk) in todo.chunks(n_batch).enumerate() {
    if interrupted() {
        report.interrupted = true;
        break;
    }
    let last_chunk = (i + 1) * n_batch >= todo.len();
    self.decode(chunk, need_logits && last_chunk)?;
    report.computed += chunk.len();
}
```

That ordering makes an interrupt during a long prefill leave a valid shorter prefix behind instead of nothing, so that the next pass extends what was done. Plank found the same property in DwarfStar's chunked prefill, which rewrites its live checkpoint on every chunk and keeps the partial one on cancel, and found also that not every path in that engine chunks, so the depth an interrupt leaves is path-dependent. In `rllama` it is always the last completed batch, because the loop is the harness's own.

Messages as tokens

The adapter in blanka decides what a message looks like as tokens, and on a local engine that decision has consequences a hosted API hides. Qwen was trained on ChatML, in which a message is a role header, the content, and an end marker:

```
<|im_start|>user
Run the tests.<|im_end|>
```

The markers `<|im_start|>` and `<|im_end|>` are dedicated tokens in the vocabulary, and `rllama` tokenizes with special-token parsing on so that the text form of a marker becomes its id rather than a dozen punctuation tokens. That is the same tokenization point chapter 6 made about plank's trusted span, and here it has the same edge: a user who types the literal marker into a prompt would produce the real token, which is a turn-boundary forgery. Blanka's reference model has no tool that reads text from an untrusted source into a system position, so the appendix notes the edge and leaves the defence to the chapter that drew it.

Every message is rendered as two segments, the header and then the body with its end marker, and each segment is tokenized on its own:

```
pub fn segments(role: Role, content: &str) -> [String; 2] {
  let (who, body) = match role {
    Role::System => ("system", content.to_string()),
    Role::User => ("user", content.to_string()),
    Role::Assistant => ("assistant", content.to_string()),
    // Qwen was trained to read tool output as a user turn wrapped in
    // this tag; blanka's own frame goes inside it unchanged.
    Role::Tool => ("user", format!("<tool_response>\n{content}\n</tool_response>")),
  };
  [format!("<|im_start|>{who}\n"), format!("{body}<|im_end|>\n")]
}
```

Tokenizing the segments separately and concatenating the results makes every segment boundary a hard token boundary. The tokens for the first n messages then never depend on what follows them, and a prompt that adds a message can only extend the previous prompt's tokens, never change the last few. Chapter 8 described plank's cut at the trusted length as legal at one offset and nowhere else, because only there did plank's engine already tokenize the two halves in separate calls. The adapter here makes the same arrangement the rule for every message, which is what lets a pass match the cache message by message. The split between header and body has its own reason: generation starts right after an assistant header, so the next pass's rendering of that reply begins with exactly the tokens the engine decoded before it sampled.

The reply itself is where exactness runs out. The engine sampled a sequence of tokens, and the transcript recorded their text. On the next pass the adapter renders that text again and tokenizes it, and a byte-level tokenizer does not promise that the text of a sampled sequence tokenizes back to the same sequence. Most of the time it does. When it does not, the divergence sits inside the last reply, `llama.cpp` removes the tail from that

point, and the pass pays for the end of one reply rather than the conversation. Plank met the sharper form of the same problem when it closed an unfinished think block by appending eight bytes, and the splice it built to keep the sampled ids is described in chapter 8. On an engine that truncates, blanka accepts the small cost and the test below asserts only that it stays small.

Two further choices follow from the same concern. Qwen's official chat template strips the reasoning of earlier assistant turns from the prompt, which saves tokens and rewrites every past reply on every pass. Blanka keeps replies verbatim, because a rendering that changes text behind the live end is a cache miss by construction. And the warm walk appends message texts with no roles, because the trait's `warm_append` takes text alone, so the adapter assigns roles by position: the first message of a walk is the system prompt and every later one is a user message, which is exactly how chapter 6 logs the session context and how a fresh session's first pass begins.

Bytes that are not yet characters

A token is a run of bytes, not a run of characters. A byte-level vocabulary ends a token in the middle of a multi-byte UTF-8 sequence whenever that is the cheaper encoding, and decoding each token on its own would show half an emoji followed by a replacement character. Chapter 4 met the same problem at the tag level, where a marker could arrive split across two chunks, and answered it with a carry. `rllama` has a carry of its own at the byte level:

```
pub fn push(&mut self, bytes: &[u8]) -> String {
    self.pending.extend_from_slice(bytes);
    let mut out = String::new();
    loop {
        match std::str::from_utf8(&self.pending) {
            Ok(s) => {
                out.push_str(s);
                self.pending.clear();
                return out;
            }
            Err(e) => {
                let good = e.valid_up_to();
                out.push_str(std::str::from_utf8(&self.pending[..good]).unwrap_or_default());
                match e.error_len() {
                    // Incomplete sequence at the end: wait for more bytes.
                    None => {
                        self.pending.drain(..good);
                        return out;
                    }
                    // Invalid bytes in the middle: replace and continue.
                    Some(bad) => {
                        out.push(char::REPLACEMENT_CHARACTER);
                        self.pending.drain(..good + bad);
                    }
                }
            }
        }
    }
}
```


Appendix D: A local engine with rllama

```
    }  
  }  
}  
}
```

The standard library's UTF-8 error already distinguishes the two cases that matter. An error with no length is an incomplete sequence at the end, which the next token may complete, so the bytes wait. An error with a length is a byte that can never be valid, which is replaced so that one bad token costs one replacement character instead of stalling the stream. The stream renderer from chapter 4 then receives only whole characters, and its own carry for split tags works unchanged on top.

Generating, and what joins the cache

Generation samples after the live state, streams text, and appends each sampled token to both the cache and `live`. It stops at an end-of-generation token, a full window or an interrupt, and the end-of-generation token itself does not join the cache, because the next prompt renders its own end marker after the reply. Leaving the sampled end marker in the cache as well would put two in a row, and the next pass would diverge at the first of them.

The sampler is greedy by default. Chapter 10 argued that at temperature zero a model that produced a bad pass will produce it again, and the loop guards were designed and measured under that assumption, so the local engine starts where the guards expect it to. A temperature above zero switches to a temperature-scaled distribution with a fixed seed, which keeps two runs of the tests comparable.

The adapter's `generate` is short, because the work lives in the session:

```
fn generate(  
    &mut self,  
    prompt: Prompt<'_,>,  
    sink: &mut dyn Sink,  
    interrupt: &InterruptFlag,  
) -> Result<(String, GenStats), EngineError> {  
    let want = self.prompt_tokens(prompt)?;  
    let synced = self.sync(&want, true, interrupt)?;  
    if synced.interrupted {  
        // The prefix decoded so far stays in the cache; the next pass  
        // extends it rather than starting over.  
        return Err(EngineError::Interrupted);  
    }  
    let room = self.session.n_ctx().saturating_sub(want.len() + 1);  
    let generated = self  
        .session  
        .generate(self.max_tokens.min(room), self.temperature, &mut |t| sink.text(t), &| |  
        interrupt.is_set())  
        .map_err(engine_err)?;  
    let stats = GenStats {  
        prompt_tokens: synced.computed,
```

Appendix D: A local engine with rllama

```
    completion_tokens: generated.tokens,
    prefill: synced.elapsed,
    decode: generated.elapsed,
    interrupted: generated.interrupted,
};
Ok((generated.text, stats))
}
```

The statistics are the first place the trait's accounting stops being an estimate. `prompt_tokens` is what the sync decoded, not the length of the prompt, so a status line built on it reports the cost of the pass rather than the size of the conversation. `prefill` and `decode` are measured separately, which is the separation chapter 8 needed and could only argue for: the time to first token is the sync, and the rate the eye can follow is the generation.

Blobs on disk

The trait's `get_kv` and `set_kv` turn into two `llama.cpp` calls that serialise and deserialise one sequence's state. The bytes `llama.cpp` produces describe the cells and nothing about the tokens behind them, so the blob `rllama` writes carries both, behind a four-byte magic and a version number:

```
pub struct StateBlob {
    /// The tokens the state covers, in order.
    pub tokens: Vec<i32>,
    /// llama.cpp's sequence state, opaque to everything but llama.cpp.
    pub state: Vec<u8>,
}
```

The token list is what makes a restored state usable: after a restore, `live` is the blob's tokens, and the next sync compares against them exactly as it would against a state computed in this process. The magic and version are what make a stray file harmless. A blob from a different layout, or a file that is not a blob at all, is refused by the decoder before its bytes reach `llama.cpp`'s deserialiser, and the decoder is tested against both cases.

The two `llama.cpp` calls are `unsafe` in the bindings, and each carries its argument in the source. The capture sizes its buffer to exactly what `llama.cpp` reports for the sequence, and nothing runs on the context between the two calls, so the copy cannot overrun. The restore is the more delicate one, because it hands bytes from a file to a C parser. The argument there is chapter 8's rule doing a second job: the store loads a blob only under a key whose fingerprint matched, the blob's embedded signature is the only thing the loader trusts, and `llama.cpp` validates the layout against the context and reports failure rather than guessing. When it does fail, the session clears the cache and `live`, so that a half-restored state is never trusted by the next sync; the error propagates, the warm walk treats it as a miss, and the tier is computed.

The size of a blob is arithmetic from the model's configuration rather than a measurement. Qwen3 8B has thirty-six layers and eight key-value heads of dimension

Appendix D: A local engine with rllama

128, so with the cache in sixteen-bit floats each token costs two tensors of 36 by 8 by 128 values at two bytes, about 144 kibibytes. The cold launch below prefills 2,447 tokens, so a blob of the whole prefix is about 350 megabytes, which is the same order as the four-hundred-megabyte rungs plank's findings quote. Chapter 9's byte budget, deferred there because the mock's blobs were six kilobytes, becomes a real concern the moment this engine writes its first ladder, and a reader who keeps three rungs of a long session will want the sweep's ceiling set before the directory fills a disk.

Wiring it in

The binary takes the engine behind the feature, with `--model` reinterpreted as a path:

```
// Appendix D: a local model through llama.cpp; --model is a GGUF path.
#[cfg(feature = "rllama")]
"rllama" => match blanka::llama::LlamaEngine::load(std::path::Path::new(&model),
rllama::ModelOptions::default()) {
    Ok(e) => Box::new(e),
    Err(e) => {
        eprintln!("{e}");
        return ExitCode::FAILURE;
    }
},
```

With the weights downloaded from Hugging Face or pulled through a model manager that stores GGUF files, `cargo run --release -p blanka --features rllama -- --engine rllama --model Qwen3-8B-Q4_K_M.gguf` starts a session whose every chapter-8 and chapter-9 mechanism now spends real time: the warm walk at launch, the blobs saved at each tier, the rungs pushed during a long turn, the checkpoint that rolls back a mistake. None of that code changed. The engine answers the seven methods chapter 8 added, and the harness never asks it anything else.

Tests that need weights

The tests split along the same line as the crates. Everything that needs no model is pure and runs on every `cargo test`: the sync plan in its five shapes, the blob round trip and its two refusals, the UTF-8 carry with a split emoji, an invalid byte and a stream that ends mid-character, and on the blanka side the ChatML rendering, the tool-response wrapping, the assistant header and the warm roles. Everything that needs weights is gated on an environment variable, `BLANKA_TEST_GGUF`, and returns early without it, so the suite stays offline by default and `cargo test --workspace --features rllama` passes on a machine with no model.

The gated tests assert on the cache, not on the text. One syncs a prefix and then its extension, and checks that the second sync kept the first and decoded only the difference. One diverges behind the end and checks that the common prefix survived. One captures a state, encodes and decodes it, restores it into a fresh session, and checks that the next sync computed nothing. On the blanka side, one warms the static and project tiers and checks that the first pass prefilled exactly the user message and the reply header, which is chapter 8's first-turn property measured in real tokens; one runs two passes and checks

that the second recomputed less than half the conversation, the test that tolerates the re-tokenized tail described above; and one round-trips a blob through the trait and checks that a warm walk over the restored state syncs nothing. Because none of them depends on what the model says, they pass on any GGUF llama.cpp can load. They were run against a small model from a different family, which is enough to prove the mechanism and says nothing about Qwen3's speed.

Three reruns

Plank's findings file has an entry called "Three reruns of one request", in which the same task was run three times while levers were added one by one, and the table is what made the case for each lever. The measurement this appendix owes is narrower and belongs to chapter 8: the same launch three times, first with nothing on disk, then after an edit to the project's instruction file, then with nothing changed. The example `reruns` in the `blanka` crate performs exactly that sequence in a scratch project, with one loaded model and a fresh session per launch so that each launch starts with an empty cache, and prints a table:

```
cargo run --release -p blanka --features rllama --example reruns -- Qwen3-8B-Q4_K_M.gguf
```

The token column does not need the weights. The example accepts a vocabulary-only GGUF with `--vocab-only`, and llama.cpp's source tree ships one for the Qwen2 tokenizer, which Qwen3 reuses with a handful of added special tokens. Run against it, the three launches of the scratch project prefill these counts, in real tokens of the ChatML rendering:

cold	nothing	2,447	to be measured
warm static	static prefix	97	to be measured
warm session	project context	21	to be measured

[Three launches of one scratch project: token counts measured with the Qwen2 vocabulary; times to be measured on the reference model]

The counts differ from chapter 8's smoke run for three reasons, all of them expected. The mock counted a token as four bytes, and a prompt full of tool schemas has more tokens than that rule predicts. The static prefix has grown since chapter 8, because chapters 10 to 13 added tools and rules to it. And the scratch project is not a git repository, so the session tier carries only the date, which is why the last launch reads 21 tokens where the smoke run read 62. The shape is the one chapter 8 predicted: a cold launch reads the whole prompt, an instruction edit costs the project and session tiers, and an unchanged launch costs the volatile tail and nothing else.

The time column is left empty on purpose. The machine this appendix was written on had no Qwen3 8B weights, and chapter 8's own rule is to compare figures measured back to back and to quote ratios, which a number borrowed from another model would break. What can be said without measuring is the ratio of work: the cold launch prefills about a hundred and sixteen times as many tokens as the warm one, and the warm launch

adds the time to read a blob of about 350 megabytes from disk, which on a laptop's solid-state drive is a fraction of a second. When you run the example on your own machine, the three rows are the numbers to put in this table, and the ratio between the first and the last is the saving the whole of chapter 8 was about.

Where rllama stops short of plank's engine

Plank's engine does a great deal that `rllama` does not, and each gap is a pointer to another part of the book. It runs a drafter for speculative decoding and keeps the tail of a cut speculative block as shadow tokens, which is appendix E's subject. It refuses to capture or yield a session that holds image embeddings, because a rebuild would re-fill the image positions with nothing behind them, and `rllama` loads no vision model. It frees its session under memory pressure and restores from the blobs on disk, which chapter 8 described and which a `llama.cpp` engine could adopt with the same ordering rules. It reads the logprobs of answer letters after one forward pass for the System-1 decisions of chapter 7, and `llama.cpp` exposes the logits that would need; a `decide` method on this engine is a few dozen lines over `get_logits_ith` and a softmax, and the leading-space lesson from chapter 7 applies to Qwen's tokenizer as much as to DeepSeek's. And it uses one sequence per context, as `rllama` does, where `llama.cpp` offers several, which is where a reader would start for the concurrent sub-agents plank's fan-out cannot yet run on a local engine.

Counters that became seconds

`rllama` is a `llama.cpp` session that remembers the tokens its cache holds, plans every sync as keep, drop and recompute, decodes in batches so an interrupt leaves a shorter valid prefix, streams whole characters, and serialises its state together with its tokens behind a header that refuses foreign bytes. Blanka's adapter renders messages in ChatML with every boundary tokenized on its own, keeps replies verbatim so that nothing behind the live end is rewritten, and answers the seven methods chapter 8 added without the harness changing a line. The one surprise relative to the mock is favourable: `llama.cpp` can truncate a plain transformer's cache in place, so a divergence costs its tail rather than a rebuild, where DwarfStar cannot and plank needs its ladder. The tests assert on the cache and pass on any model, and the three reruns are waiting for the reference model to fill in the column that turns chapter 8's counters into seconds.

APPENDIX E: SPECULATIVE DECODING FROM THE HARNESS'S SIDE

Plank's footer once reported that speculative decoding was running at 2.1x, on a session that was forty percent slower than the same session without it. Nothing was broken. The figure was the number of tokens the engine committed per speculative step, which is a true statement about the mechanism and a false statement about the wall clock, and it had been labelled as if it were a speedup. The episode is a fair summary of how speculative decoding looks from a harness. The technique belongs to the engine, and a harness author could reasonably decide it is none of their business, but it reaches the harness in at least four places: the shape in which tokens arrive, the state the engine is left in when a pass stops early, the sampling settings a turn runs under, and the question of whether the feature pays at all. Each of those places produced a finding in plank, and each finding would reappear in any harness that runs a local model with a drafter.

This appendix explains the technique only as far as those four places need it, then walks through them. There is no blanka code here. Blanka's engines hand back a finished pass, and the mock engine of chapter 8 has no drafter to speak of. The appendix on `llama` wraps an engine where speculation is available, and the last section says what blanka would have to change to meet it.

Guesses checked in bulk

Generation is serial. To produce a token the model needs the one before it, and each step reads the whole weight set out of memory to compute a single token. On the hardware a local model runs on, the arithmetic units spend most of a decode step waiting for memory, so a single-user decode loop uses a small fraction of what the machine could compute. The escape is that the same pass which reads the weights once can score many positions almost as cheaply as one. If something proposes the next few tokens, the model can check all of them in one forward pass, keep the ones that match what it would have produced anyway, and discard the rest.

That is the whole idea, and one property of it matters more than any other to a harness: the output is identical to what the model would have produced alone. Speculation changes when the tokens arrive, not which tokens arrive. A harness that treats a speculative engine as a faster engine is correct about the text and wrong about almost everything else, as the rest of this appendix shows.

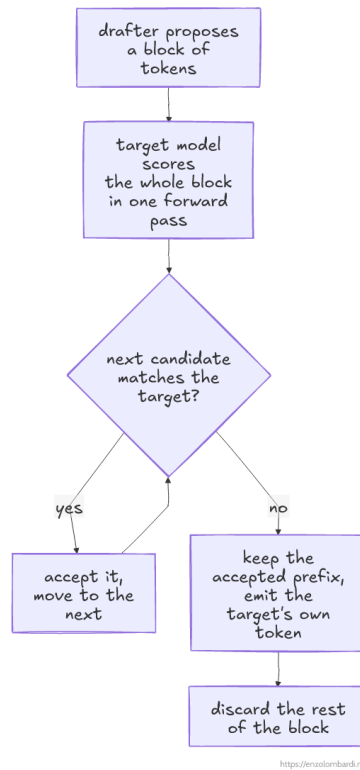
The accounting has one subtlety worth carrying around. A draft proposes a block of candidates, and verification walks the block from the front and stops at the first mismatch. A candidate at position five counts only if positions one to four all survived, so the expected number of accepted tokens is not the average acceptance rate. It is the sum of running products:

$$\mathbb{E}[\text{accepted}] = p_1 + p_1 * p_2 + p_1 * p_2 * p_3 + \dots$$

With a per-position acceptance of 0.9 across a block of seven, a verification pass advances the stream by about 5.2 tokens. Let the later positions slip to 0.78 and the yield drops

Appendix E: Speculative decoding from the harness's side

noticeably, because the deep positions are hostage to the shallow ones. The walk that produces this rule looks like this:



Verification walks the block from the front and stops at the first mismatch, so the output matches plain decoding

The other cost is the draft itself. Every millisecond spent guessing is a millisecond the target model is not verifying, so a drafter that predicts well and slowly can lose to one that predicts crudely and instantly. The families of drafter in circulation are different moves on that trade. Multi-token prediction bolts extra heads onto the target model at training time, so the draft costs almost nothing but each head predicts from a state that knows nothing of what the heads before it chose, and accuracy collapses a few positions out. Block drafters are small separate models that emit a whole block per pass, which makes the draft cost roughly constant in the block length, at the price of a tail that decays because no position knows what the others decided. DSpark, the drafter paired with DeepSeek V4 Flash, keeps the parallel block and adds a very small sequential head that nudges each position by what the previous one turned out to be, plus a confidence head whose per-position survival estimates let a scheduler choose how many candidates are worth verifying at the current load. None of these choices is the harness's, and plank takes whatever drafter the engine supports for the loaded model. What the harness inherits is the behaviour of the loop that consumes the drafts.

A run arrives, not a token

A plain decode loop hands the harness one token per step. A speculative loop hands it a run: the sampled token plus however many drafted tokens verification accepted, committed to the engine's state in one call. In DwarfStar, the native engine plank links

Appendix E: Speculative decoding from the harness's side

for its DeepSeek models, `ds4_session_eval_speculative_argmax` returns that run, and by the time the call returns every token in it is already in the KV cache. Nothing downstream can reject part of the run, because the engine has already paid for all of it.

For the stream renderer of chapter 4 this is almost invisible. The renderer was designed to accept text in chunks of any size, with tags split across chunks and a carry for the fragment that has not finished arriving, so a run of five tokens is a slightly larger chunk. The consequence that is not invisible is the stop. Chapter 4 gave the renderer the job of noticing that a tool stanza has closed, and the agent loop the job of stopping the pass at that moment, because anything the model writes after a complete stanza is at best waste. With a run, the stanza can close at token two of five.

Plank's first speculative loop checked for a stop once per run, and it checked only for end-of-sequence and the token budget. The model kept sampling after a complete stanza, and in nine of 1,446 recorded assistant stanzas it spent those tokens writing a second stanza. The C reference agent does not have this problem: its worker sets a stop flag the moment its parser reaches the end of a stanza and handles the remainder of the run. Plank's fix polls the interrupt once per token inside the accepted run rather than once per run, after each token has been handed to the renderer, so a stanza that has just closed is visible to the check. The trap in the fix is worth knowing. The stop rides the same interrupt channel a user's key press does, because that is the only stop channel the engine's `generate` has, so it comes back to the caller marked as an interruption, and every site that asks whether the user stopped the pass has to distinguish the two, or a turn is abandoned with a complete, parsed tool call that never runs.

The tail that was committed anyway

Polling per token answers what the renderer shows. It does not answer what to do with the rest of the run, which is already in the engine's state and which the harness has decided not to use.

The C reference agent rewinds. When a stanza closes or end-of-sequence lands in the middle of a run, it rolls the session back to the last token it kept, so the engine's state matches the transcript again. Plank copied that, and on DeepSeek it was a disaster that looked like nothing at all. DwarfStar cannot roll a DeepSeek session's compressor state back by truncating it, so a rewind marks the session's checkpoint invalid instead. On the next pass, the common prefix between the new prompt and the live state was zero, and the engine re-read the whole conversation. A debug log from an eighteen-thousand-token session showed it on every tool round: a rewind that kept six tokens, followed by a pass that reused nothing and prefilled 17,885 tokens. A later session paid 117,000 tokens the same way. The ladder rescue of chapter 9 did not fire either, because its condition for recognising a rebuild required a non-zero common prefix, and an invalid checkpoint produced exactly zero, so the rescue returned silently.

The fix was to stop pretending the tail had not happened. The tokens the engine committed but the pass never rendered are kept as *shadow tokens*, and they are recorded as part of the assistant's span in the token transcript, after the rendered reply and before the end-of-sequence marker. The token buffer the harness keeps is then an exact mirror

Appendix E: Speculative decoding from the harness's side

of the engine's state, and the next prompt extends it instead of diverging from it. Plank's version of that record is short (`plank, src/ds4engine.rs, commit bc0289d`):

```
fn assistant_span_tokens(prefix: &[i32], reply: &[i32], shadow: &[i32], eos: i32) ->
Vec<i32> {
    let mut span = Vec::with_capacity(prefix.len() + reply.len() + shadow.len() + 1);
    span.extend_from_slice(prefix);
    span.extend_from_slice(reply);
    span.extend_from_slice(shadow);
    if span.last() != Some(&eos) {
        span.push(eos);
    }
    span
}
```

The cost is that the model sees, in its own history, a handful of tokens it generated and nobody displayed: at most one draft block between the close of the stanza and the end of the message, and a single token with the default one-token draft. The transcript text the user can read does not contain them, which is a small exception to chapter 4's rule that the transcript records exactly what the model produced. It is the right exception. The alternative is a transcript that matches the display and an engine state that matches neither, and on a local engine that mismatch is paid for in full prefills. The same fix made the ladder rescue log an invalid checkpoint instead of returning quietly, which is the general lesson from chapter 8 applied once more: a path that silently falls back to a rebuild from zero looks, in every test that checks only the text, like a path that works.

A serial loop never overshoots, so none of this applies without speculation. That is the reason it was found late: every behaviour was correct with the drafter off, and the drafter was on by default.

A gate that opens only at zero

Speculative verification in DwarfStar is `argmax`: a candidate survives when it is the token the target would have chosen greedily. DwarfStar therefore speculates only at temperature zero, and both the C reference agent and plank fetch the draft block size only under that condition. At the default temperature of 0.7 the speculative entry point is never called. The model loads, the drafter loads alongside it, the run looks normal, and speculation buys nothing. Plank's findings file records the consequence for anything that reports on speculation: a status segment, a benchmark or a bug report has to run at temperature zero, or it is measuring plain decode with an extra model in memory.

Plank's answer was to pin the temperature to zero whenever speculation was on, and speculation was on by default. That settled the temperature during argument parsing, before the model family was known, and speculation can still fall away after that point. The DSpark drafter exists for DeepSeek V4 Flash alone; V4.1 ships no drafter and decodes target-only. And DwarfStar refuses to open a model at all when a drafter is paired with a checkpoint it does not match, which at first meant that V4.1 and one newer V4 checkpoint failed to load until the user found the flag that switched speculation off. Commit `4dff7d5` stopped auto-pairing a drafter with V4.1 and made any other failed

Appendix E: Speculative decoding from the harness's side

open retry once without the companion plank had chosen, while never dropping one the user named explicitly.

That fixed the load and exposed the side effect. A run on a model with no working drafter now sampled greedily, because a feature that never started had already set the temperature. Commit `f1e774c` records whether the user typed a temperature at all and settles the value only once the engine can say whether it is capable of speculating; an explicit temperature, including an explicit zero, always wins. The footer and the command that toggles speculation already read the engine's answer rather than the configuration flag, so both stayed truthful through the fallback.

The general form of the lesson is one that recurs across the book: a setting that exists to serve a feature must be applied when the feature is known to be running, not when it is requested. Configuration is a statement of intent, and an engine that can decline the intent is the authority on what actually happened.

Does it pay

Whether speculation pays is an empirical question about one engine build on one machine, and plank's record is a catalogue of ways to get the answer wrong.

The first is the one this appendix opened with. Tokens per speculative step is a property of the drafter and the verification walk. It says nothing about time, because a step that commits three tokens and takes three times as long as a plain step has gained nothing. Plank renders the figure as tokens per step now, and its findings file says it must never be labelled with an $\times$. A companion trap sits in the same counters: the drafted count adds the block size once per step, because the engine's entry point does not report how many candidates were actually proposed, so the block fill it implies is a lower bound, ten percent on a run where the engine's own counters said sixty-seven.

The second is to trust the acceptance rate. On one configuration the engine's own statistics reported an acceptance rate of 74 percent, which looks healthy, next to a net time saving that was negative by two minutes, because verification and replay cost more than acceptance saved. The number to read is the net, and the acceptance rate looked fine in both the losing regime and the winning one.

The third is to quote a verdict without the DwarfStar commit beside it. On an M5 Max, DSpark was a clear loss, about 0.71 of plain decode, until an upstream DwarfStar commit pipelined the Metal verifier, after which the same binary, prompt and model read 1.19. Nothing in plank changed between the two measurements. Building DwarfStar's own command-line tool from the same source and running the same prompt reproduced the loss exactly, which ruled the port out in one step and was far cheaper than reasoning about the foreign function interface.

The fourth is to benchmark carelessly, and it produced two wrong answers before a right one. Short natural generations are noise: one body measured twenty and eight tokens per second minutes apart. Forcing equal lengths by ignoring end-of-sequence inverts the result on a speculative engine, because past the natural end the model emits degenerate text the drafter cannot predict, acceptance collapses, and the shortest prompt reaches that region first and looks slowest. A machine that throttles after a heavy run produces a

fake improvement for whatever runs second. What worked was warm-cache repeat sampling, interleaved pairs with cooldowns between them, and ratios measured back to back rather than absolute figures compared across days.

Underneath all four sits the fact that decides the answer, and it is a curve rather than a number. A verification pass over N positions costs some multiple of a single decode step, and speculation pays only when that multiple is well below the number of tokens a pass is expected to accept. Plank measured the curve for llama.cpp on the same machine with a dense Qwen model and its block drafter: a forward over two positions cost 1.11 times one position, over four 1.44 times, over eight 2.2 times, and the drafter was worth about 21 percent. DwarfStar's DSpark verify, in one measurement, cost 2.03 times a plain decode at two positions, worse than llama.cpp at eight. At 1.44 for four rows a two-thirds acceptance rate is very profitable; at 2.03 for two rows it cannot be. The confound is recorded next to the figures: the Qwen model is dense and DeepSeek V4 Flash is a large mixture of experts, where verifying N tokens can touch up to N times as many distinct experts, so the dense model's win shows what a good curve looks like without proving the other one can reach it. Plank's conclusion was that nothing on its side of the drafter was wrong, and that the gap lives in DwarfStar's kernels.

The measurement a user can take is simpler, and plank puts it where it cannot be missed. The exit message reports, for each model, how long the session spent prefilling and how long generating, with the average rate for each and the time spent in tools. The totals are kept per model rather than per run, because a quantized model with a drafter and the same model without one are different machines as far as throughput goes, and they are scoped to the session and never persisted, because figures from last week were measured on a different engine build, a different context length and a cooler machine. Two sessions of the same work, one with speculation and one without, answer the question for that machine and that build, which is the only form in which the question has an answer.

Three promises a drafting engine must keep

Blanka's `Engine::generate` streams text into a sink and returns the finished pass, and its interrupt flag is checked between chunks. A speculative engine behind that trait needs three things the book's engines do not have, and all three are visible in plank's record above.

The first is a stop check inside the run. If the engine delivers a run as one chunk, the renderer's stanza detection sees the close only after the whole run has been written to the sink, and the tokens after the close have already been shown. An engine that speculates has to hand the run to the sink token by token, or at least check the interrupt between tokens of the run, and the agent loop has to tell a stanza stop from a user stop, which blanka's loop already does for the non-speculative case.

The second is an honest warm length. Chapter 8's warm walk depends on `warm_len` describing exactly the prefix the engine holds state for, because the next pass's reuse is computed against it. An engine that commits tokens the pass did not render has to count them, whether it records them as shadow tokens the way plank does or rewinds where the engine can rewind cheaply. The mock engine's counters would catch a mistake here

Appendix E: Speculative decoding from the harness's side

immediately: a test that stops a pass mid-run and asserts that the next pass extends rather than rebuilds is the test plank did not have.

The third is accounting. `GenStats` records prompt and completion tokens and two durations, which is enough to compute the only number that matters, time per generated token, and not enough to explain it. An engine that speculates would add steps, committed tokens and the net time saved, and a front end would show the last of those rather than tokens per step.

The appendix on `rlama` wraps an engine that exposes drafters. None of the three changes alters the trait's shape; each is a promise the implementation has to keep.

Guessed ahead, paid in order

Speculative decoding lets an engine check several guessed tokens for the cost of one step and keep the prefix that matches, so the output is unchanged and only the timing moves. For a harness, that shows up in four places. Tokens arrive in runs, so the check for a finished tool stanza has to happen inside the run, not after it. The engine commits a whole run even when the harness uses part of it, so the harness must either record the unused tail or rewind cheaply, or the next pass re-reads the conversation. Speculation runs only at temperature zero, so a setting applied for its sake has to wait until the engine confirms speculation is running. And whether it pays is a question about one engine build on one machine, answered by net time rather than acceptance rate, measured in interleaved pairs, and reported per model in a place the user will see.

APPENDIX F: READING A REPRO DUMP

Every finding about looping in chapter 10, and most of the cache findings in chapters 8 and 9, began as a single markdown file that a harness wrote about itself. A repro dump is the record of what the model saw at the moment something went wrong: the exact text the engine was given, the settings that shaped the generation, and, in plank's later versions, a table of every pass the turn ran. It is the one artifact that lets a maintainer reason about a session without having been there, and reading one well is a skill with a small number of rules.

This appendix describes the format plank settled on, section by section, and then reads one dump that plank's loop findings discuss, using only the figures those findings already record. Every example block below is schematic: the field names are plank's, the values are placeholders. A real dump contains everything the model was given, and the section on the transcript explains why that makes a dump something to strip before it leaves the machine. Blanka's dump from chapter 5 has the same skeleton with fewer parts, and the last section compares the two.

What the name says

The first fact about a dump is in its file name. Plank writes four kinds into one directory, each named by its kind and the Unix time it was taken. A `repro-<secs>.md` was asked for, by the `/repro` command or by a camera glyph at the end of the footer's directory prefix. A `repro-loop-<secs>.md` was written by the harness itself when a loop guard stopped a pass. A `repro-debug-<secs>.md` is written on the way out of any session run with the debug flag, on every front end. A `repro-panic` is the last resort. Two dumps in the same second get a numeric suffix rather than overwriting one another.

The difference between the first two kinds is information on its own. A loop dump is evidence that a rung fired. A plain dump taken during a stall is evidence that none did, because the user noticed the loop before the harness and saved it by hand. Several findings in chapter 10 start from exactly that observation: the prefix was the tell that the guards were blind to a shape, and the file was the specimen that showed which shape.

The name can also mislead. For a while one of plank's tests drove a looping pass through the real agent code, and nothing redirected where its dump was saved, so every test run added a fake stall to the real directory: hundreds of copies of one short line, delivered by a scripted engine in a single chunk, meaningless as a measure of guard latency. The fix resolved the directory once, when the agent is constructed, and pointed the test agent at a temporary one. The habit it left is to read the session name in the header before trusting that a file in the folder records a real session.

The header

A dump opens with a title carrying the build that wrote it and a short list of facts about the session. Schematically:

```
# plank repro <version>
```


Appendix F: Reading a repro dump

- date: <day>
- note: model looped; repro saved automatically
- session: <session id>
- context size: <tokens>
- transcript tokens: <tokens>
- last ctx used: <tokens>

The note says why the file exists; a loop dump fills it in, and a hand-saved one carries whatever the user typed. The two token counts are worth comparing. The first is the transcript measured by plank's own tokenizer, the second is the context size the last pass actually used. Chapter 1 told the story of a hosted endpoint whose failed passes were exposed by these two numbers agreeing too closely: a server-reported figure would not track the local tokenizer within three tokens, and plank's own fallback count does, so near-equality meant the server had never reported anything.

A `## Model` section follows, and it answers the question "which model was this" four times over, on purpose. The configured path is often a symlink the user has since repointed. The name the engine reports is the shape it matched, which selects the tool dialect. The family decides the transcript extension and the drafter slot. The artifact set version names the weights, which is the one thing a maintainer cannot recover from the other three. Plank's module comment records that a report carrying only the path had repeatedly failed to say which build of which model was loaded.

Then a `## Generation` section lists the knobs that shaped the passes: the reasoning level, whether thinking and tool calls were shown, the token cap, temperature, the sampling parameters, the seed, whether the loop guards were armed and the think budget in effect. The last two lines were added after a dump had to be read without them. Chapter 10's review that was written in the wrong place could not at first be diagnosed from its dump, because the file did not say whether the thirty-kilobyte pass had been ended by the user or by a rung, nor what the budget was set to, and the answer changed the conclusion.

The table of passes

The section that changed how dumps are read is `## Passes`, one row per generation pass, oldest first. Its columns, schematically:

#	ended	Δ	agent	tokens	tok/s	reasoning	cycle	headings	fenced	stop	
---	-----	---	-----	-----	-----	-----	-----	-----	-----	-----	
1	<time>		main	...	...	...	-	0	0	tool calls: 1	
n	<time>	+...	main	...	...	...	...	...	...	guard: cycle	

Each column answers a question that used to be reconstructed by hand. `ended` and `Δ` give the wall clock and the gap since the previous pass, which is where a slow tool or a long prefill shows up. `agent` separates the main loop from sub-agents. `tokens` and `tok/s` are the pass's output and its rate. `reasoning` is the number of bytes the guard saw inside the think block, which is the column to watch for a pass that spent itself deliberating. `cycle` is a latched loop reported as a period and a number of copies. `headings` and `fenced` are the draft rung's two counts, numbered deliverable headings

Appendix F: Reading a repro dump

and bytes inside code fences. And `stop` names how the pass ended in a small closed vocabulary: `tool calls: N`, `answer`, `interrupted by user`, `guard: cycle`, `guard: draft`, `guard: budget`, or `tool error`.

Plank's loop findings ask that future entries quote this table rather than re-derive it, and most of the later findings in chapter 10 are, in effect, readings of one row against its neighbours. A draft stop followed by another draft stop with no tool round between them is the recovery that had nowhere to write. A pass whose reasoning is fifty kilobytes with every guard column empty and a stop of `interrupted by user` is a loop no rung could see, which is how the self-renumbering cycle was found.

The transcript, as the engine saw it

After the table comes the rendered transcript, fenced by `----- BEGIN TRANSCRIPT` and `----- END TRANSCRIPT -----`. It is the exact engine input of the last pass: the `[system]` block, then every `[user]` and `[assistant]` message, with reasoning, raw tool-call stanzas and framed tool results left verbatim. Chapter 4's rule that the transcript records what the model produced is what makes this section trustworthy, and chapter 1's rule that everything is logged is what makes it complete.

Complete is the operative word, and it is the reason a dump should not be shared as written. The `[system]` block is not only the built-in prompt. It carries the tool schemas, the project's `AGENTS.md`, the bodies of any skills that were loaded, the instructions every configured MCP server advertises, and whatever else the home directory contributed, which chapter 6 showed can include the paths of the user's other projects. The tool results carry the contents of every file the model read. A dump is therefore as sensitive as the working directory and the home directory together. Plank does not redact it, on purpose, because a redacted dump no longer reproduces the session; the author of a bug report is the one who strips paths, project names and proprietary text before the file leaves the machine.

Inside the fences, plank inserts markers that are not part of the engine input:

```
[timestamp] <time> (msg 0, +0s)
...
[timestamp] <time> (msg 20, +<elapsed>)
...
[timestamp] <time> (end, +<elapsed>)
```

A marker appears every twenty messages and once at the end, with the wall clock and the time since the first. They are added only to the report, never to the text used for token counting, so the cache probe that compares prompts is unaffected by them. Their use is spacing: a tight loop shows up as many markers with small deltas, and a slow session as few markers with large ones.

Finding the loop

The stopped pass is the last `[assistant]` block before the end marker, and its tail is what the guard saw. Plank's findings give the measurement that tells how late the guard fired: find the smallest period p such that the tail ends in k copies of its last p bytes; then

Appendix F: Reading a repro dump

k times p is the repeated span, and everything before it is reasoning from before the loop began. Measured over the early dumps, every real stop landed at the designed four copies, a stop cost about four periods, from 0.6 to 3 kilobytes, and the latency of the guard stopped being the interesting number.

The dump plank's findings call "a list enumeration collapses to one item" is a clean example of reading past that number. Its thirtieth pass began listing the tables inside a generated parser file, and at temperature zero greedy decoding collapsed the list into the same twenty-byte bullet eleven times over. The byte rung saw four copies at once inside its window and stopped the pass after 2,568 bytes of reasoning. Detection was fine. Two other things were not, and both are visible only in the table.

The first was the row itself, which read `cycle: - beside stop: guard: cycle`. The snapshot that fills the column reported only a latched block, and a period short enough to show four copies in one check returned a match without ever latching, so the cheapest loops to catch were exactly the ones the table could not describe. Plank now latches the block a direct hit matched, with the copy count as a floor, and the regression test feeds that dump's bullet and asserts the period appears. A column that disagrees with the stop reason in the same row is a bug in the dump, not in the reading.

The second was the rest of the table. The loop was the least expensive part of the turn. Thirty passes and eighteen minutes and forty-seven seconds went by with nothing written to the user. Passes 22 to 30 each re-derived the same four facts at four to eight kilobytes of reasoning per pass, the semantic circle of chapter 10's hour-long turn, which no byte rung is the instrument for. And throughput fell from about 37 to about 21 tokens per second between passes 8 and 9, when three large tool results landed, which is the context penalty chapter 8 measured rather than a loop symptom, and which doubled the cost of every re-derivation after it. The guard's stop explains why the turn ended. The table explains why it failed, and the two are different questions.

Where the sub-agents went

A sub-agent's sidechain is folded out of the main transcript the moment it ends, which chapter 12 showed is what keeps the parent's context clean. It also means the main dump has no trace of what a sub-agent did, and a loop inside one is invisible there. Plank keeps the last few finished sidechains in memory and writes each one beside the main file as `<main>.sub-<n>.md`:

```
# plank repro <version>, sub-agent sidecar <n>
```

- main repro: <main file>
- label: <agent label>
- outcome: <ended | no report | failed: ...>
- forked at parent message: <index>
- messages: <count>
- system prompt: identical to the main repro (not repeated)

A `##` Task section and a fenced sidechain transcript follow, with the same timestamp markers. The `forked at` index ties the sidecar to the point in the parent where the fork

Appendix F: Reading a repro dump

happened, and the `outcome` line is where a sidechain's failure is recorded, because a sub-agent that stopped on a guard trip keeps its partial text, and the rule that it must keep it was written after a first version discarded exactly the text the sidecar existed to show. The label is also how the quiet background passes of chapter 11 appear in a dump: a memory reading or a prompt suggestion is a sidechain like any other, and its sidecar is often the only record that it ran.

A dump taken while the turn was running

For a long time a dump could only be taken between turns, because the worker thread owned the agent while a turn ran, and typing `/repro` mid-turn queued the text as a prompt for the model. A loop no rung fired on could therefore only be recorded after it had run its course. Plank now publishes the pass's report at the start of each main pass and streams the pass into a buffer the front end can read, so `/repro` and the camera glyph both work mid-turn. Such a dump carries an extra section at the end, `## In-progress pass`, with the user's note and the text the running pass had produced so far. The transcript above it is that pass's engine input, the table of passes stops at the pass before it, and the two together are the state of the model at the moment of the dump.

Blanka's dump, beside plank's

Blanka's `repro.rs` from chapter 5 writes the same skeleton at a smaller scale. Its header records the date, the session, the engine, the context size, whether thinking was shown, the message count and the note. A `## Transcript` section follows with every message as a `[role]` block, verbatim, reasoning and stanzas included. A loop dump is named `repro-loop-<secs>.md` and a requested one `repro-<secs>.md`, neither ever overwrites, and chapter 12 added the sidecars, written beside the main file as `.sub-<n>.md` with a label, the task, the outcome, the fork point and the sidechain's messages.

What blanka's dump lacks is the part that made plank's readable at scale. There is no model section beyond the engine name, because blanka has one engine per build. There is no generation section with the guard state and the budget, no table of passes, no timestamp markers and no in-progress section. Chapter 10 listed the table as the addition to make once the format settles, and this appendix is the argument for it: every column in plank's table exists because a finding had to be derived by hand from a dump that did not have it. A reader extending blanka would add the pass notes where the loop pushes each pass's message, and the table would follow.

Evidence the harness keeps about itself

A repro dump is the harness's record of what the model was given and what came back, written so that someone who was not there can reason about it. Its name says why it exists and whether a guard fired. Its header and model section identify the session and the weights. Its generation section records the settings that shaped the passes. Its table of passes turns what used to be hours of reconstruction into a reading of rows. The fenced transcript is the exact engine input, which is what makes a dump both reproducible and as sensitive as the directories it came from. Sidecars keep the sub-agent work the main transcript folds away, and a dump taken mid-turn catches the loop no

Appendix F: Reading a repro dump

rung could see. The reading rule underneath all of it: check that the columns agree with each other before drawing a conclusion from any one of them.

THANK YOU

This book rests on other people's work more than most.

Plank exists because of DwarfStar, the small native inference engine and reference agent written by Salvatore Sanfilippo and its contributors. The reference's decision to keep its engine narrow, its prompt text exact and its tool table small made it an unusually good thing to port, and every chapter that says "the reference does" is a debt to that code.

Thanks go to the people who build and maintain the open models the book talks to, the DeepSeek and Qwen teams among them, and to the maintainers of llama.cpp, Ollama and the Rust crates blanka stands on: ratatui, serde, the HTTP and hashing crates, and the rest of an ecosystem that makes a project like this possible for one person in a few months.

Thanks also go to the readers of the Medium posts where many of these findings first appeared. Their comments, corrections and counter-examples, on agent loops in particular, sharpened more than one chapter.

And thanks to you, for reading a book that is still being written. The online edition will be revised as its subject settles, and the best way to shape the next revision is to say what worked and what did not. If the book helped you build something, a review, positive or not, helps other readers decide whether it is for them and helps the next edition improve.

May your prefixes be reused and your loops be caught early.

